and yet the cost of customization cannot be effectively amortized since few extreme computers are made. However, when purchasing thousands of servers at a time, there *are* great volume discounts. WSCs are so massive internally that there is economy of scale even if there are not many WSCs. As we will see in Sections 6.5 and 6.10, these economies of scale led to commercial cloud computing because the lower per-unit costs of a WSC meant that companies could rent servers at a profit below what it costs outsiders to do so themselves. The flip side of 100,000 servers is failures. Figure 6.1 shows outages and anomalies for 2400 servers. Even if a server had a mean time to failure (MTTF) of an amazing 25 years (200,000 h), the WSC architect would need to design for five server failures a day. Figure 6.1 lists the annualized disk failure rate as 2%–10%. Given two disks per server and an annual failure rate of 4%, with 100,000 servers the WSC architect should expect to see one disk fail per *hour*. However, software failures vastly outnumber hardware failures, as Figure 6.1 shows, so the system design must be resilient to server crashes caused by software bugs, which would happen even more frequently than disk failures. With the thousands of servers in these very large facilities, WSC operators become very good at changing disks, so the cost of disk failure is much lower for a WSC than a small data center. The same applies to DRAMs. Plausibly, WSCs could use even less reliable components if cheaper ones were available.

Approx. number events in 1st year	Cause	Consequence
1 or 2	Power utility failures	Lose power to whole WSC; doesn't bring down WSC if UPS and generators work (generators work about 99% of time).
4	Cluster upgrades	Planned outage to upgrade infrastructure, many times for evolving networking needs such as recabling, to switch firmware upgrades, and so on. There are about nine planned cluster outages for every unplanned outage.
1000s	Hard-drive failures	2%–10% annual disk failure rate (Pinheiro et al., 2007)
	Slow disks	Still operate, but run 10× to 20× more slowly
	Bad memories	One uncorrectable DRAM error per year (Schroeder et al., 2009)
	Misconfigured machines	Configuration led to ~30% of service disruptions (Barroso and HÖlzle, 2009)
	Flaky machines	1% of servers reboot more than once a week (Barroso and HÖlzle, 2009)
5000	Individual server crashes	Machine reboot; typically takes about 5 min (caused by problems in software or hardware).

Figure 6.1 List of outages and anomalies with the approximate frequencies of occurrences in the first year of a new cluster of 2400 servers. We label what Google calls a cluster an *array*; see Figure 6.5. Based on Barroso, L.A., 2010. Warehouse Scale Computing [keynote address]. In: Proceedings of ACM SIGMOD, June 8–10, 2010, Indianapolis, IN.

Example Calculate the availability of a service running on the 2400 servers in Figure 6.1. Unlike a service in a real WSC, in this example the service cannot tolerate hardware or software failures. Assume that the time to reboot software is 5 min and the time to repair hardware is 1 h.

Answer We can estimate service availability by calculating the time of outages because of failures of each component. We'll conservatively take the lowest number in each category in Figure 6.1 and split the 1000 outages evenly between four components. We ignore slow disks—the fifth component of the 1000 outages—because they hurt performance but not availability, and power utility failures, because the uninterruptible power supply (UPS) system hides 99% of them.

$$\text{Hours Outage}_{\text{service}} = (4 + 250 + 250 + 250) \times 1\,\text{h} + (250 + 5000) \times 5\,\text{min}$$
$$= 754 + 438 = 1192\,\text{h}$$

Since there are 365×24 or 8760 h in a year, availability is

$$\text{Availability}_{\text{system}} = \left(\frac{8760 - 1192}{8760}\right) = \frac{7568}{8760} = 86\%$$

Without software redundancy to mask the many outages, a service on those 2400 servers would be down on average one day a week—zero "nines"—which is far below the 99.99% of availability is the goal of WSCs.

As Section 6.10 explains, the forerunners of WSCs are *computer clusters*. Clusters are collections of independent computers that are connected together using local area networks (LANs) and switches. For workloads that did not require intensive communication, clusters offered much more cost-effective computing than shared-memory multiprocessors. (Shared-memory multiprocessors were the forerunners of the multicore computers discussed in Chapter 5.) Clusters became popular in the late 1990s for scientific computing and then later for Internet services. One view of WSCs is that they are just the logical evolution from clusters of hundreds of servers to tens of thousands of servers.

A natural question is whether WSCs are similar to modern clusters for high-performance computing. Although some have similar scale and cost—there are HPC designs with a million processors that cost hundreds of millions of dollars—they historically have had more powerful processors and much lower-latency networks between the nodes than are found in WSCs because the HPC applications are more interdependent and communicate more frequently (see Section 6.3). The programming environment also emphasizes thread-level parallelism or data-level parallelism (see Chapters 4 and 5), typically emphasizing latency to complete a single task in contrast to bandwidth to complete many independent tasks via request-level parallelism. The HPC clusters also tend to have long-running jobs that keep the servers fully utilized, even for weeks at a time,

whereas the utilization of servers in WSCs ranges between 10% and 50% (see Figure 6.3 on page 441) and varies every day. Unlike supercomputer environments, thousands of developers work on the WSC code base and deploy significant software releases every week (Barroso et al., 2017).

How do WSCs compare to conventional data centers? The operators of a traditional data center generally collect machines and third-party software from many parts of an organization and run them centrally for others. Their main focus tends to be consolidation of the many services onto fewer machines, which are isolated from each other to protect sensitive information. Thus, virtual machines are increasingly important in data centers. Virtual machines are important for WSCs as well, but they play a different role. They are used to offer isolation between different customers and to slice hardware resources into different-sized shares to rent at several price points (see Section 6.5). Unlike WSCs, conventional data centers tend to have a great deal of hardware and software heterogeneity to serve their varied customers inside an organization. WSC programmers customize third-party software or build their own, and WSCs have much more homogeneous hardware; the WSC goal is to make the hardware/software in the warehouse act like a single computer that typically runs a variety of applications. Often the biggest cost in a conventional data center is the people to maintain it, whereas, as we will see in Section 6.4, in a well-designed WSC, the server hardware is the greatest cost, and people costs shift from the topmost to the bottommost. Conventional data centers also don't have the scale of a WSC, so they don't get the economic benefits of the scale previously mentioned.

Thus, although a WSC might be considered as an extreme data center in that computers are housed separately in a space with special electrical and cooling infrastructure, traditional data centers share little with the challenges and opportunities of a WSC, either architecturally or operationally.

We start the introduction to WSCs with their workload and a programming model.

6.2 Programming Models and Workloads for Warehouse-Scale Computers

If a problem has no solution, it may not be a problem, but a fact—not to be solved, but to be coped with over time.

Shimon Peres

In addition to the public-facing Internet services such as search, video sharing, and social networking that make them famous, WSCs also run batch applications, such as converting videos into new formats or creating search indexes from web crawls.

A popular framework for batch processing in a WSC is MapReduce (Dean and Ghemawat, 2008) and its open-source twin Hadoop. Figure 6.2 shows the increasing popularity of MapReduce at Google over time. Inspired by the Lisp functions

Month	Number of MapReduce Jobs	Average completion time (s)	Average no. servers per job	Avg. no. cores per server	CPU core years	Input data (PB)	Intermediate data (PB)	Output data (PB)
Sep-16	95,775,891	331	130	2.4	311,691	11,553	4095	6982
Sep-15	115,375,750	231	120	2.7	272,322	8307	3980	5801
Sep-14	55,913,646	412	142	1.9	200,778	5989	2530	3951
Sep-13	28,328,775	469	137	1.4	81,992	2579	1193	1684
Sep-12	15,662,118	480	142	1.8	60,987	2171	818	874
Sep-11	7,961,481	499	147	2.2	40,993	1162	276	333
Sep-10	5,207,069	714	164	1.6	30,262	573	139	37
Sep-09	4,114,919	515	156	3.2	33,582	548	118	99
Sep-07	2,217,000	395	394	1.0	11,081	394	34	14
Mar-06	171,000	874	268	1.6	2002	51	7	3
Aug-04	29,000	634	157	1.9	217	3.2	0.7	0.2

Figure 6.2 Monthly MapReduce usage at Google from 2004 to 2016. Over 12 years the number of MapReduce jobs increased by a factor of 3300. Figure 6.17 on page 461 estimates that running the September 2016 workload on Amazon's cloud computing service EC2 would cost $114 million. Updated from Dean, J., 2009. Designs, lessons and advice from building large distributed systems [keynote address]. In: Proceedings of 3rd ACM SIGOPS International Workshop on Large-Scale Distributed Systems and Middleware, Co-located with the 22nd ACM Symposium on Operating Systems Principles, October 11–14, 2009, Big Sky, Mont.

of the same name, Map first applies a programmer-supplied function to each logical input record. Map runs on hundreds of computers to produce an intermediate result of key-value pairs. Reduce collects the output of those distributed tasks and collapses them using another programmer-defined function. Assuming the Reduce function is commutative and associative, it can run in log N time. With appropriate software support, both functions are fast yet easy to understand and use. Within 30 min, a novice programmer can run a MapReduce task on thousands of computers.

Figure 6.2 shows the average job uses hundreds of servers. Other than a few highly tuned applications from high-performance computing, such MapReduce jobs are the most parallel applications today, whether measured in total CPU time or number of servers utilized.

Here is a MapReduce program that calculates the number of occurrences of every English word in a large collection of documents. Following is a simplified version of that program, which shows just the inner loop and that assumes only one occurrence of all English words found in a document (Dean and Ghemawat, 2008):

```
map(String key, String value):
          // key: document name
          // value: document contents
          for each word w in value:
          EmitIntermediate(w, "1"); // Produce list of
          all words

reduce(String key, Iterator values):
          // key: a word
          // values: a list of counts
          int result = 0;
          for each v in values:

          result += ParseInt(v); // get integer from key-
          value pair
          Emit(AsString(result));
```

The function `EmitIntermediate` used in the Map function emits each word in the document and the value one. Then the Reduce function sums all the values per word for each document using ParseInt() to get the number of occurrences per word in all documents. The MapReduce runtime environment schedules map tasks and reduce tasks to the nodes of a WSC. (The complete version of the program is found in Dean and Ghemawat (2008).)

MapReduce can be thought of as a generalization of the single instruction stream, multiple data streams (SIMD) operation (Chapter 4)—except that a function to be applied is passed to the data—that is followed by a function that is used in a reduction of the output from the Map task. Because reductions are commonplace even in SIMD programs, SIMD hardware often offers special operations for the reductions. For example, Intel's AVX SIMD instructions include "horizontal" instructions that add pairs of operands that are adjacent in registers.

To accommodate variability in performance from hundreds of computers, the MapReduce scheduler assigns new tasks based on how quickly nodes complete prior tasks. Obviously, a single slow task can hold up completion of a large MapReduce job. Dean and Barroso (2013) label such a situation *tail latency*. In a WSC, the solution to slow tasks is to provide software mechanisms to cope with such variability that is inherent at this scale. This approach is in sharp contrast to the solution for a server in a conventional data center, where traditionally slow tasks mean hardware is broken and needs to be replaced or that server software needs tuning and rewriting. Performance heterogeneity is the norm for 50,000–100,000 servers in a WSC. For example, toward the end of a MapReduce program, the system will start backup executions on other nodes of the tasks that haven't completed yet and take the result from whichever finishes first. In return for increasing resource usage a few percentage points, Dean and Ghemawat (2008) found that some large tasks completed 30% faster.

Dependability was built into MapReduce from the start. For example, each node in a MapReduce job is required to report back to the master node periodically

with a list of completed tasks and with updated status. If a node does not report back by the deadline, the master node deems the node dead and reassigns the node's work to other nodes. Given the amount of equipment in a WSC, it's not surprising that failures are commonplace, as the prior example attests. To deliver on 99.99% availability, systems software must cope with this reality in a WSC. To reduce operational costs, all WSCs use automated monitoring software allowing one operator to be responsible for more than 1000 servers.

Programming frameworks such as MapReduce for batch processing and externally facing SaaS such as Search rely upon internal software services for their success. For example, MapReduce relies on the Google File System (GFS) (Ghemawat et al., 2003) or on Colossus (Fikes, 2010) to supply files to any computer, so that MapReduce tasks can be scheduled anywhere.

In addition to GFS and Colossus, examples of these scalable storage systems include Amazon's key value storage system Dynamo (DeCandia et al., 2007) and the Google record storage system BigTable (Chang et al., 2006). Note that such systems often build upon each other. For example, BigTable stores its logs and data on GFS or Colossus, much as a relational database may use the file system provided by the kernel operating system.

These internal services usually make different decisions than similar software running on single servers. For example, rather than assuming storage is reliable, such as by using RAID storage servers, these systems often make complete replicas of the data. Replicas can help with read performance as well as with availability; with proper placement, replicas can overcome many other system failures, like those in Figure 6.1. Systems like Colossus use error-correcting codes rather than full replicas to reduce storage costs, but the constant is cross-server redundancy rather than within-a-server or within-a-storage array redundancy. Thus, failure of the entire server or storage device doesn't negatively affect availability of the data.

Another example of the different approach is that WSC storage software often uses relaxed consistency rather than following all the ACID (atomicity, consistency, isolation, and durability) requirements of conventional database systems. The insight is that it's important for multiple replicas of data to agree *some time*, but, for most applications, they do not need to be in agreement at all times. For example, eventual consistency is fine for video sharing. Eventual consistency makes storage systems much easier to scale, which is an absolute requirement for WSCs.

The workload demands of these public interactive services all vary considerably; even a prominent global service such as Google Search varies by a factor of two depending on the time of day. When factoring in weekends, holidays, and popular times of year for some applications—such as photograph-sharing services after New Year's Day or online shopping before Christmas—a much greater variation in server utilization becomes apparent. Figure 6.3 shows average utilization of 5000 Google servers over a 6-month period. Note that less than 0.5% of servers averaged 100% utilization, and most servers operated between 10% and 50% utilization. Stated alternatively, just 10% of all servers were utilized more than 50%. Thus, it's much more important for servers in a WSC to perform

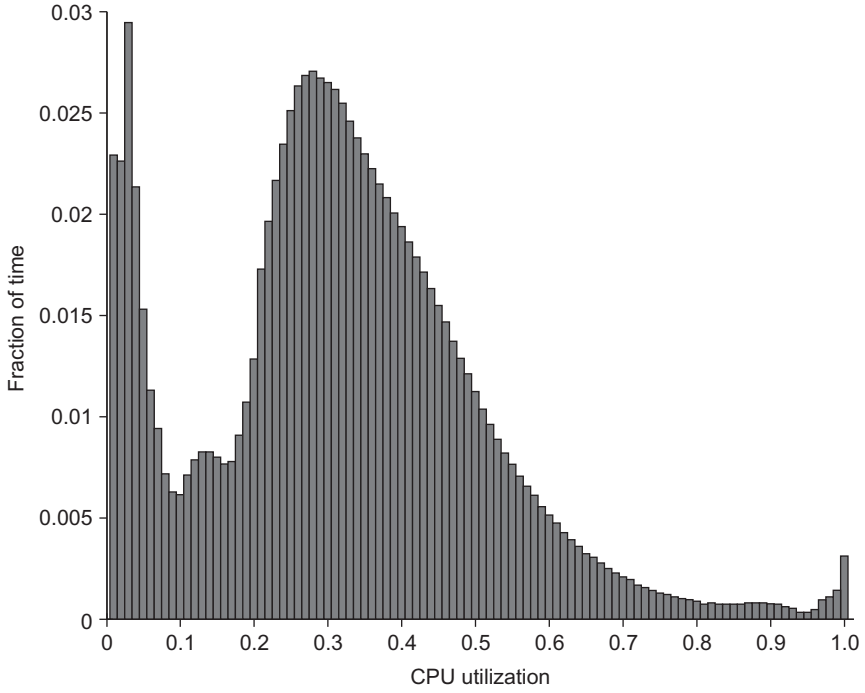


Figure 6.3 Average CPU utilization of more than 5000 servers during a 6-month period at Google. Servers are rarely completely idle or fully utilized, instead operating most of the time at between 10% and 50% of their maximum utilization. The third column from the right in Figure 6.4 calculates percentages plus or minus 5% to come up with the weightings; thus 1.2% for the 90% row means that 1.2% of servers were between 85% and 95% utilized. From Figure 1 in Barroso, L.A., Hölzle, U., 2007. The case for energy-proportional computing. IEEE Comput. 40 (12), 33–37.

well while doing little than to perform efficiently only at their peak, as they rarely operate at their peak.

In summary, WSC hardware and software must cope with variability in load based on user demand and in performance and dependability because of the vagaries of hardware at this scale.

Example As a result of measurements like those in Figure 6.3, the SPECpower benchmark measures power and performance from 0% load to 100% in 10% increments (see Chapter 1). The overall single metric that summarizes this benchmark is the sum of all the performance measures (server-side Java operations per second) divided by the sum of all power measurements in watts. Thus, each level is assumed to be equally likely. How would the numbers summary metric change if the levels were weighted by the utilization frequencies in Figure 6.3?

Load	Performance	Watts	SPEC weightings	Weighted performance	Weighted watts	Figure 6.3 weightings	Weighted performance	Weighted watts
100%	2,889,020	662	9.09%	262,638	60	0.80%	22,206	5
90%	2,611,130	617	9.09%	237,375	56	1.20%	31,756	8
80%	2,319,900	576	9.09%	210,900	52	1.50%	35,889	9
70%	2,031,260	533	9.09%	184,660	48	2.10%	42,491	11
60%	1,740,980	490	9.09%	158,271	45	5.10%	88,082	25
50%	1,448,810	451	9.09%	131,710	41	11.50%	166,335	52
40%	1,159,760	416	9.09%	105,433	38	19.10%	221,165	79
30%	869,077	382	9.09%	79,007	35	24.60%	213,929	94
20%	581,126	351	9.09%	52,830	32	15.30%	88,769	54
10%	290,762	308	9.09%	26,433	28	8.00%	23,198	25
0%	0	181	9.09%	0	16	10.90%	0	20
Total	15,941,825	4967		1,449,257	452		933,820	380
				ssj_ops/W	3210		ssj_ops/W	2454

Figure 6.4 SPECpower result using the weightings from Figure 6.3 instead of even weightings.

Answer Figure 6.4 shows the original weightings and the new weighting that match Figure 6.3. These weightings reduce the performance summary by 30% from 3210 ssj_ops/watt to 2454.

Given the scale, software must handle failures, which means there is little reason to buy "gold-plated" hardware that reduces the frequency of failures. The primary impact would be to increase cost. Barroso and Hölzle (2009) found a factor of 20 difference in price-performance between a high-end Hewlett Packard shared-memory multiprocessor and a commodity Hewlett Packard server when running the TPC-C database benchmark. Not surprisingly, Google and all other companies with WSCs use low-end commodity servers. In fact, the Open Compute Project (http://opencompute.org) is an organization where such companies collaborate on open designs of servers and racks for data centers.

Such WSC services also tend to develop their own software rather than buy third-party commercial software, in part to cope with the huge scale and in part to save money. For example, even on the best price-performance platform for TPC-C in 2017, adding the cost of the SAP SQL Anywhere database and the Windows operating system increases the cost of the Dell PowerEdge T620 server by 40%. In contrast, Google runs BigTable and the Linux operating system on its servers, for which it pays no licensing fees.

Given this review of the applications and systems software of a WSC, we are ready to look at the computer architecture of a WSC.

6.3 Computer Architecture of Warehouse-Scale Computers

Networks are the connective tissue that binds 50,000–100,000 servers together. Analogous to the memory hierarchy of Chapter 2, WSCs use a hierarchy of networks. Figure 6.5 shows one example. Ideally, the combined network would provide nearly the performance of a custom high-end switch for 100,000 servers at about the cost per port of a commodity switch designed for 50 servers. As we will see in Section 6.6, networks for WSCs are an area of active innovation.

The structure that holds the servers is a rack. Although the width of racks varies per WSC—some are the classic 19-in. wide rack; others are two or three times wider—the height tends to be no higher than 6–7 ft since people must service them. Such a rack has roughly 40–80 servers. Because it is often convenient to connect the network cables at the top of the rack, this switch is commonly called a *Top of Rack* (*ToR*) switch. (Some WSCs have racks with multiple ToR switches.) Typically, the bandwidth within the rack is much higher than between racks,

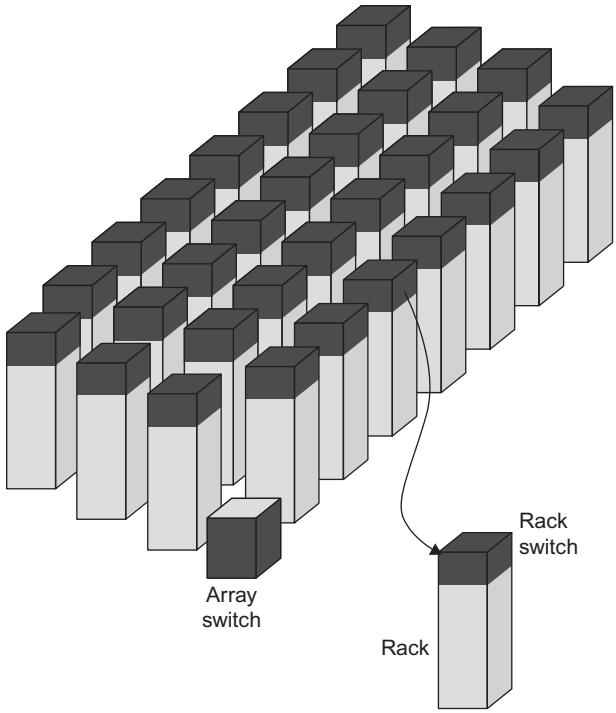


Figure 6.5 Hierarchy of switches in a WSC. Based on Figure 1.1 in Barroso, L.A., Clidaras, J., Hölzle, U., 2013. The datacenter as a computer: an introduction to the design of warehouse-scale machines. Synth. Lect. Comput. Architect. 8 (3), 1–154.

so it matters less where the software places the sender and the receiver if they are within the same rack. This flexibility is ideal from a software perspective.

These switches often offer 4–16 uplinks, which leave the rack to go to the next higher switch in the network hierarchy. Thus, the bandwidth leaving the rack is 6–24 times smaller than the bandwidth within the rack. This ratio is called *oversubscription*. However, large oversubscription means programmers must be aware of the performance consequences when placing senders and receivers in different racks. This increased software-scheduling burden is another argument for network switches designed specifically for the data center.

The switch that connects an array of racks is considerably more expensive than the ToR switch. This cost is due in part because of the higher connectivity and in part because the bandwidth through the switch must be much greater to reduce the oversubscription problem. Barroso et al. (2013) reported that a switch having 10 times the *bisection bandwidth*—basically, the worst-case internal bandwidth—of a rack switch costs about 100 times as much. One reason is that the cost of switch bandwidth for n ports can grow as n^2. Sections 6.6 and 6.7 describe the networking above the ToR switch in great detail.

Storage

A natural design is to fill a rack with servers, minus whatever space needed for the switches. This design leaves open the question of where the storage is placed. From a hardware construction perspective, the simplest solution would be to include disks inside the rack and rely on Ethernet connectivity for access to information on the disks of remote servers. An expensive alternative would be to use network-attached storage (NAS), perhaps over a storage network like InfiniBand. In the past, WSCs generally relied on local disks and provided storage software that handled connectivity and dependability. For example, GFS used local disks and maintained replicas to overcome dependability problems. This redundancy covered not only local disk failures but also power failures to racks and to whole clusters. The flexibility of GFS's eventual consistency lowers the cost of keeping replicas consistent, which also reduces the network bandwidth requirements of the storage system.

Today the storage options are considerably more varied. Although some racks are balanced in terms of servers and disks, as in the past, there may also be racks deployed without local disks and some racks loaded with disks. System software today often uses RAID-like error correction codes to lower the storage cost of dependability.

Be aware that there is confusion about the term *cluster* when talking about the architecture of a WSC. Using the definition in Section 6.1, a WSC is just an extremely large cluster. In contrast, Barroso et al. (2013) used the term cluster to mean the next-sized grouping of computers, containing many racks. In this chapter, to avoid confusion, we will use the term *array* to mean a large collection of racks organized in rows, preserving the original definition of the word cluster to represent anything from a collection of networked computers within a rack to an entire warehouse full of networked computers.

WSC Memory Hierarchy

Figure 6.6 shows the latency, bandwidth, and capacity of memory hierarchy inside a WSC, and Figure 6.7 shows the same data visually. These figures are based on the following assumptions (Barroso et al., 2013):

	Local	Rack	Array
DRAM latency (μs)	0.1	300	500
Flash latency (μs)	100	400	600
Disk latency (μs)	10,000	11,000	12,000
DRAM bandwidth (MB/s)	20,000	100	10
Flash bandwidth (MB/s)	1000	100	10
Disk bandwidth (MB/s)	200	100	10
DRAM capacity (GB)	16	1024	31,200
Flash capacity (GB)	128	20,000	600,000
Disk capacity (GB)	2000	160,000	4,800,000

Figure 6.6 Latency, bandwidth, and capacity of the memory hierarchy of a WSC (Barroso et al., 2013). Figure 6.7 plots this same information.

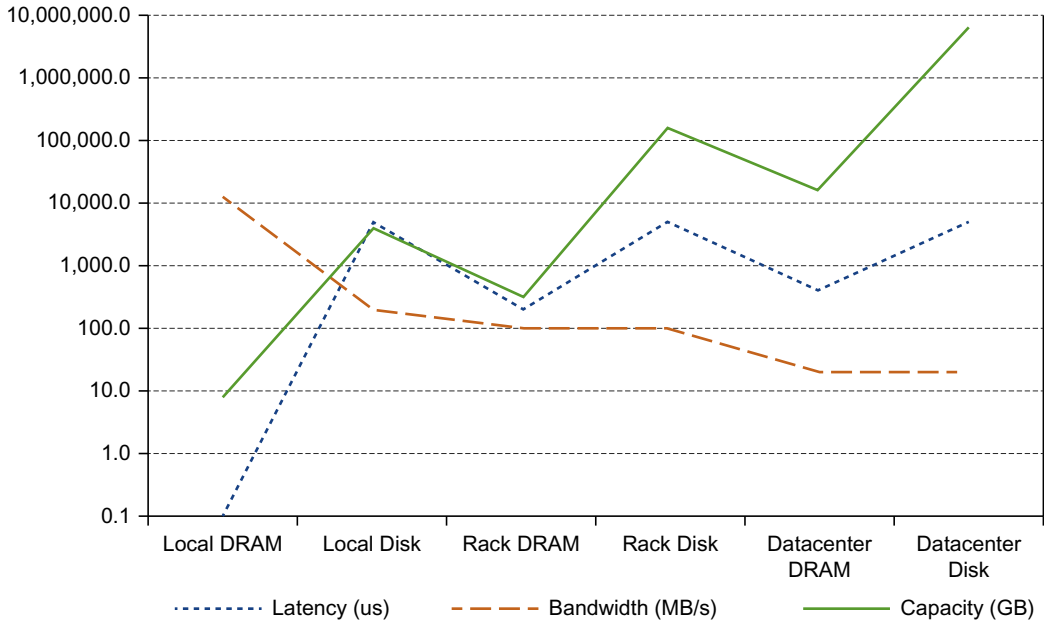


Figure 6.7 Graph of latency, bandwidth, and capacity of the memory hierarchy of a WSC for data in Figure 6.6 (Barroso et al., 2013).

- Each server contains 16 GiB of memory with a 100-ns access time and transfers at 20 GB/s, 128 GiB of Flash with 100-μs latency and transfers at 1 GB/s, and 2 TB of disk that offer a 10-ms access time and transfer at 200 MB/s. There are two sockets per board, and they share one 1 Gbit/s Ethernet port.
- In this example, every pair of racks includes one rack switch and holds 80 servers. Networking software plus switch overhead increases the latency to DRAM to 100 μs and the disk access latency to 11 ms. Thus, the total storage capacity of a rack is roughly 1 TB of DRAM, 20 TB of Flash, and 160 TB of disk storage. The 1 Gbit/s Ethernet limits the remote bandwidth to DRAM, Flash, or disk within the rack to 100 MB/s.
- The array is 30 racks, so storage capacity of an array goes up by a factor of 30: 30 TB of DRAM, 600 TB of Flash, and 4.8 PB of disk. The array switch hardware and software increases latency to DRAM within an array to 500 μs, to 600 μs for Flash, and disk latency to 12 ms. The bandwidth of the array switch limits the remote bandwidth to either array DRAM, array Flash, or array disk to 10 MB/s.

Figures 6.6 and 6.7 show that network overhead dramatically increases latency between local DRAM and Flash, rack DRAM and Flash, or array DRAM and Flash, but all still have more than 10 times better latency than accessing the local disk. The network collapses the difference in bandwidth between rack DRAM, Flash, and disk and between array DRAM, Flash, and disk.

The WSC needs 40 arrays to reach 100,000 servers, so there is one more level in the networking hierarchy. Figure 6.8 shows the conventional Layer 3 routers to connect the arrays together and to the Internet.

Most applications fit into a single array within a WSC. Those that need more than one array use *sharding* or *partitioning*, meaning that the dataset is split into independent pieces and then distributed to different arrays. As an analogy, it's like picking up registration packets for a conference with one person handling names A to M and another doing N to Z. Operations on the whole dataset are sent to the servers hosting the pieces, and the results are coalesced by the client computer.

Example What is the average memory latency assuming that 90% of accesses are local to the server, 9% are outside the server but within the rack, and 1% are outside the rack but within the array?

Answer The average memory access time is

$$(90\% \times 0.1) + (9\% \times 100) + (1\% \times 300) = 0.09 + 27 + 5 = 32.09\ \mu s$$

or a factor of more than 300 slowdown versus 100% local accesses. Clearly, locality of access within a server is vital for WSC performance.

Example How long does it take to transfer 1000 MB between disks within the server, between servers in the rack, and between servers in different racks in the array? How much faster is it to transfer 1000 MB between DRAM in the three cases?

Answer A 1000 MB transfer between disks takes

$$\text{Within server} = 1000/200 = 5 \text{ s}$$

$$\text{Within rack} = 1000/100 = 10 \text{ s}$$

$$\text{Within array} = 1000/10 = 100 \text{ s}$$

A memory-to-memory block transfer takes

$$\text{Within server} = 1000/20000 = 0.05 \text{ s}$$

$$\text{Within rack} = 1000/100 = 10 \text{ s}$$

$$\text{Within array} = 1000/10 = 100 \text{ s}$$

Thus, for block transfers outside a single server, it doesn't even matter whether the data are in memory or on disk because the rack switch and array switch are the bottlenecks. These performance limits affect the design of WSC software and inspire the need for higher-performance switches (see Section 6.6).

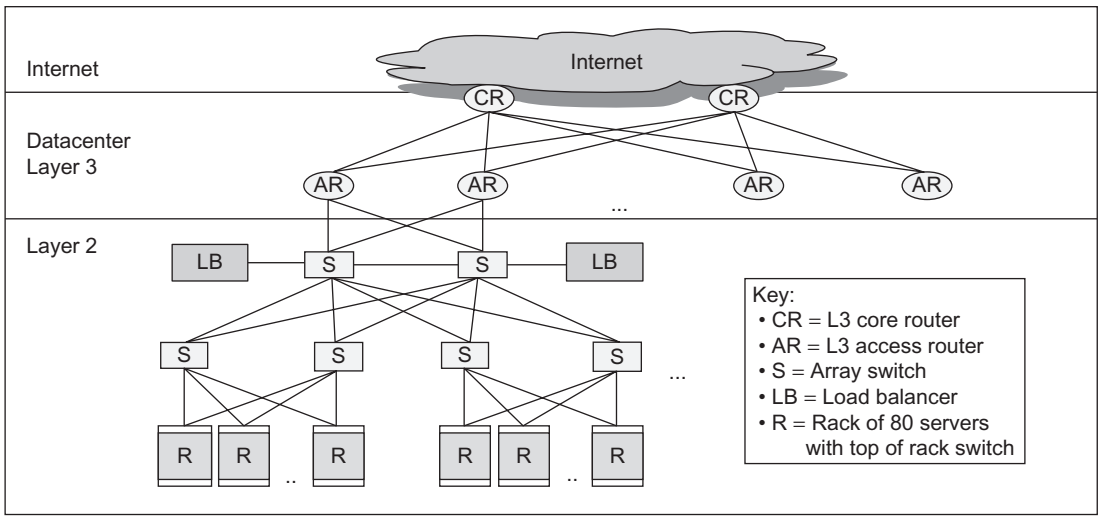


Figure 6.8 A Layer 3 network used to link arrays together and to the Internet (Greenberg et al., 2009). A load balancer monitors how busy a set of servers is and directs traffic to the less loaded ones to try to keep the servers approximately equally utilized. Another option is to use a separate *border router* to connect the Internet to the data center Layer 3 switches. As we will see in Section 6.6, many modern WSCs have abandoned the conventional layered networking stack of traditional switches.

Although these examples are educational, note that computers and networking equipment can be much larger and faster than these examples from 2013 (see Section 6.7). Servers are being deployed in 2017 with 256–1024 GiB of DRAM, and recent switches have reduced delays to only 300 ns per hop.

Given the architecture of the IT equipment, we are now ready to see how to house, power, and cool it and to discuss the cost to build and operate the whole WSC, as compared to just the IT equipment within it.

6.4 The Efficiency and Cost of Warehouse-Scale Computers

Infrastructure costs for power distribution and cooling are the majority of the construction costs of a WSC, so we concentrate on them. (Section 6.7 describes the power and cooling infrastructure of a WSC in detail.)

A *computer room air-conditioning* (*CRAC*) unit cools the air in the server room using chilled water, similar to how a refrigerator removes heat by releasing it outside the refrigerator. As a liquid absorbs heat, it evaporates. Conversely, when a liquid releases heat, it condenses. Air conditioners pump the liquid into coils under low pressure to evaporate and absorb heat, which is then sent to an external condenser where it is released. Thus, in a CRAC unit, fans push warm air past a set of coils filled with cold water, and a pump moves the warmed water to the chillers to be cooled down. Figure 6.9 shows the large collection of fans and water pumps that move air and water throughout the system.

In addition to chillers, some data centers leverage colder outside air or water temperature to cool the water before it is sent to the chillers. However, depending on the location, the chillers may still be needed during the warmer times of the year.

Surprisingly, it's not obvious how to figure out how many servers a WSC can support after subtracting the overhead for power distribution and cooling. The *nameplate power rating* from the server manufacturer is always conservative: it's the maximum power a server can draw. The first step then is to measure a single server under a variety of workloads to be deployed in the WSC. (Networking is typically about 5% of power consumption, so it can be ignored at first.)

To determine the number of servers for a WSC, the available power for IT equipment could be divided just by the measured server power; however, this would again be too conservative according to Fan et al. (2007). They found that there is a significant gap between what thousands of servers could theoretically do, in the worst case, and what they will do in practice, since no real workloads will keep thousands of servers all simultaneously at their peaks. They found that they could safely oversubscribe the number of servers by as much as 40% based on the power of a single server. They recommended that WSC architects should do so to increase the average utilization of power within a WSC; however, they also suggested using extensive monitoring software along with a safety mechanism that de-schedules lower priority tasks in case the workload shifts.

Here is the power usage inside the IT equipment for a Google WSC deployed in 2012 (Barroso et al., 2013):

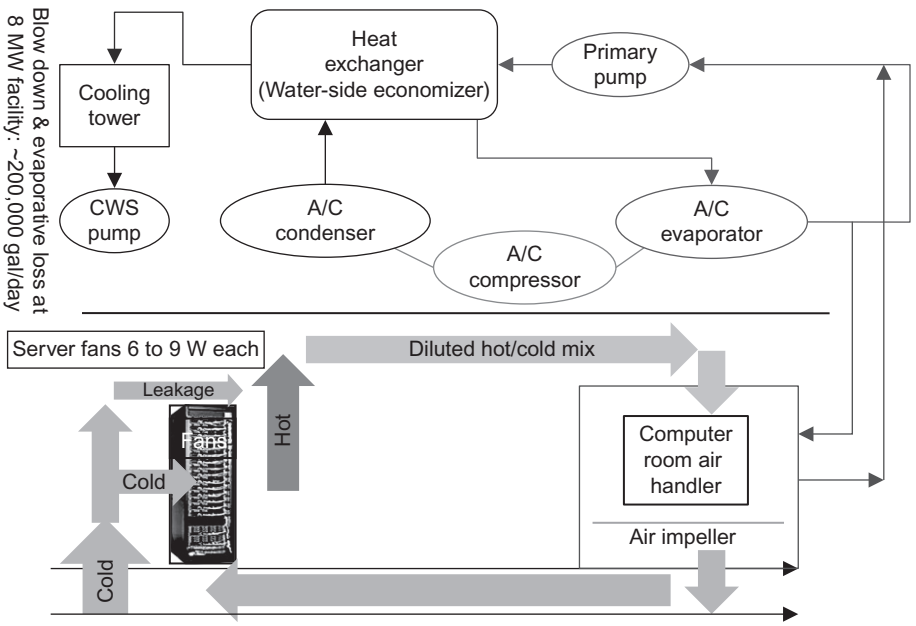


Figure 6.9 Mechanical design for cooling systems. CWS stands for circulating water system. From Hamilton, J., 2010. Cloud computing economies of scale. In: Paper Presented at the AWS Workshop on Genomics and Cloud Computing, June 8, 2010, Seattle, WA. http://mvdirona.com/jrh/TalksAndPapers/JamesHamilton_GenomicsCloud20100608.pdf.

- 42% of power for processors
- 12% for DRAM
- 14% for disks
- 5% for networking
- 15% for cooling overhead
- 8% for power overhead
- 4% miscellaneous

Measuring Efficiency of a WSC

A widely used, simple metric to evaluate the efficiency of a data center or a WSC is called *power utilization effectiveness* (or *PUE*):

$$\text{PUE} = (\text{Total facility power})/(\text{IT equipment power})$$

Thus, PUE must be greater than or equal to 1, and the bigger the PUE, the less efficient the WSC.

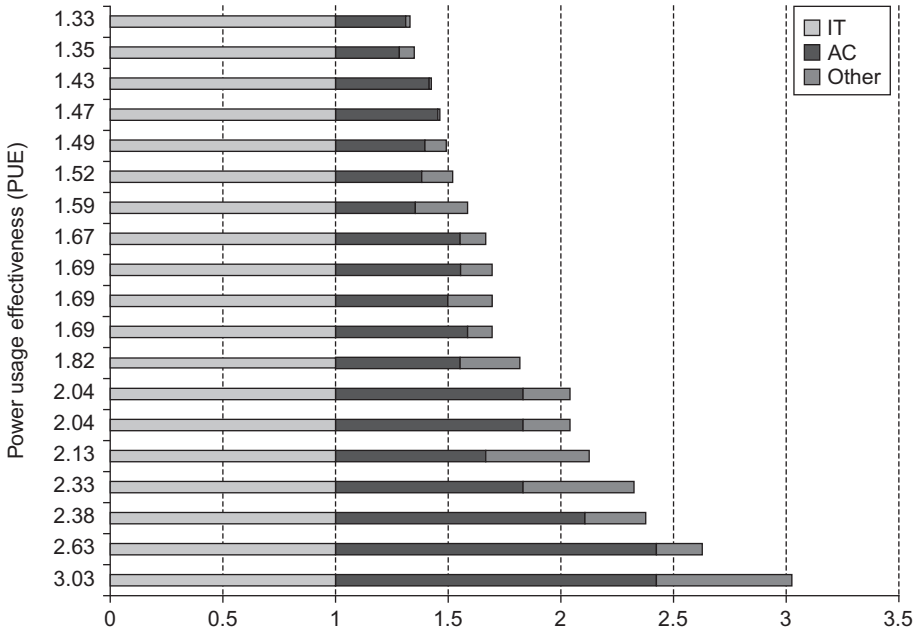


Figure 6.10 Power utilization efficiency of 19 data centers in 2006 (Greenberg et al., 2009). The power for air conditioning (AC) and other uses (such as power distribution) is normalized to the power for the IT equipment in calculating the PUE. Thus, power for IT equipment must be 1.0, and AC varies from about 0.30 to 1.40 times the power of the IT equipment. Power for "other" varies from about 0.05 to 0.60 of the IT equipment.

Greenberg et al. (2009) reported on the PUE of 19 data centers and the portion of the overhead that went into the cooling infrastructure. Figure 6.10 shows what they found, sorted by PUE from most to least efficient. The median PUE is 1.69, with the cooling infrastructure using more than half as much power as the servers—on average, 0.55 of the 1.69 is for cooling. Note that these are average PUEs, which can vary daily depending on workload and even external air temperature, as we will see (Figure 6.11).

With attention paid to PUE in the past decade, data centers are much more efficient today. However, as Section 6.8 explains, there is no universally accepted definition of what is included in PUE: If the batteries to preserve operation during a power failure are in a separate building, are they included or not? Do you measure from the output of the power substation, or where power first enters the WSC? Figure 6.10 shows the improvement in the average PUE of all Google data centers over time, which Google measures inclusively.

Since performance per dollar is the ultimate metric, we still need to measure performance. As Figure 6.7 shows, bandwidth drops and latency increases depending on the distance to the data. In a WSC, the DRAM bandwidth within a server is 200 times greater than within a rack, which in turn is 10 times greater than within an array. Thus, there is another kind of locality to consider in the placement of data and programs within a WSC.

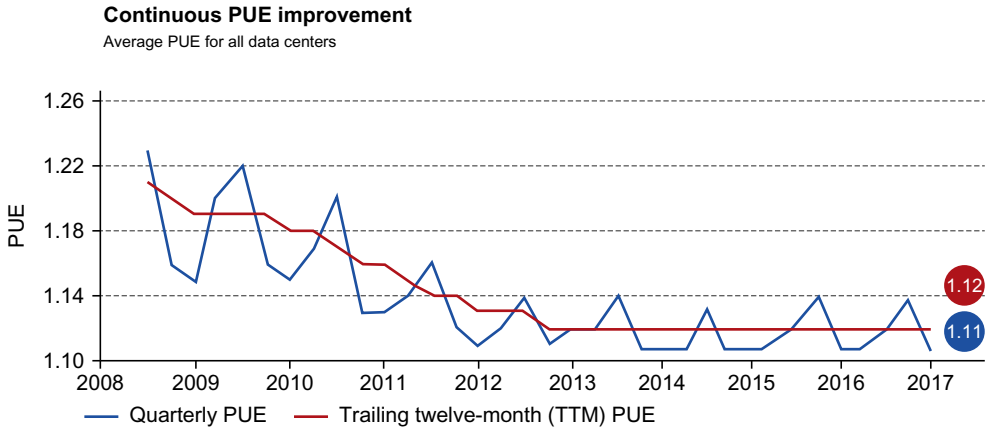


Figure 6.11 Average power utilization efficiency (PUE) of the 15 Google WSCs between 2008 and 2017. The spiking line is the quarterly average PUE, and the straighter line is the trailing 12-month average PUE. For Q4 2016, the averages were 1.11 and 1.12, respectively.

Although designers of a WSC often focus on bandwidth, programmers developing applications on a WSC are also concerned with latency because latency is visible to users. Users' satisfaction and productivity are tied to response time of a service. Several studies from the timesharing days report that user productivity is inversely proportional to time for an interaction, which was typically broken down into human entry time, system response time, and time for the person to think about the response before hitting the next entry (Doherty and Thadhani, 1982). The results of experiments showed that cutting system response time by 30% shaved the time of an interaction by 70% (Brady, 1986). This implausible result is explained by human nature: people need less time to think when given a faster response, as they are less likely to get distracted and remain "on a roll."

Figure 6.12 shows the results of a more recent experiment for the Bing search engine, where delays of 50–2000 ms were inserted at the search server (Schurman and Brutlag, 2009). As expected from previous studies, time to next click roughly doubled the delay; that is, a 200 ms delay at the server led to a 500 ms increase in time to next click. Revenue dropped linearly with increasing delay, as did user satisfaction. A separate study on the Google search engine found that these effects lingered long after the 4-week experiment ended. Five weeks later, there were 0.1% fewer searchers per day for users who experienced 200 ms delays, and there were 0.2% fewer searches by users who experienced 400 ms delays. Given the amount of money made in search, even such small changes are disconcerting. In fact, the results were so negative that they ended the experiment prematurely.

Because of this extreme concern with satisfaction of all users of an Internet service, performance goals are typically specified so that a high percentage of requests are below a latency threshold, rather than just offer a target for the average latency. Such threshold goals are called *service level objectives* (*SLOs*). An SLO might be that 99% of requests must be below 100 ms. Thus, the designers of

Server delay (ms)	Increased time to next click (ms)	Queries/ user	Any clicks/ user	User satisfaction	Revenue/ user
50	–	–	–	–	–
200	500	–	−0.3%	−0.4%	–
500	1200	–	−1.0%	−0.9%	−1.2%
1000	1900	−0.7%	−1.9%	−1.6%	−2.8%
2000	3100	−1.8%	−4.4%	−3.8%	−4.3%

Figure 6.12 Negative impact of delays at the Bing search server on user behavior (Schurman and Brutlag, 2009).

Amazon's Dynamo key-value storage system decided that for services to offer good latency on top of Dynamo, their storage system had to deliver on its latency goal 99.9% of the time (DeCandia et al., 2007). For example, one improvement of Dynamo helped the 99.9th percentile much more than the average case, which reflects their priorities.

Dean and Barroso (2013) proposed the term *tail tolerant* to describe systems designed to meet such goals:

> Just as fault-tolerant computing aims to create a reliable whole out of less-reliable parts, large online services need to create a predictably responsive whole out of less-predictable parts.

The causes of unpredictability include contention for shared resources (processors networks, etc.), queuing, variable microprocessor performance because of optimizations like Turbo mode or energy-saving techniques like DVFS, software garbage collection, and many more. Google concluded that instead of trying to prevent such variability in a WSC, it made more sense to develop tail-tolerant techniques to mask or work around temporary latency spikes. For example, fine-grained load balancing can quickly move small amounts for work between servers to reduce queuing delays.

Cost of a WSC

As mentioned in the introduction, unlike most architects, designers of WSCs worry about the cost to operate as well as the cost to build the WSC. Accounting labels the former costs as *operational expenditures* (*OPEX*) and the latter costs as *capital expenditures* (*CAPEX*).

To put the cost of energy into perspective, Hamilton (2010) did a case study to estimate the costs of a WSC. He determined that the CAPEX of an 8-MW facility was $88 million and that the roughly 46,000 servers and corresponding networking equipment added another $79 million to the CAPEX for the WSC. Figure 6.13 shows the rest of the assumptions for the case study.

Size of facility (critical load watts)	8,000,000
Average power usage (%)	80%
Power usage effectiveness	1.45
Cost of power ($/kWh)	$0.07
% Power and cooling infrastructure (% of total facility cost)	82%
CAPEX for facility (not including IT equipment)	**$88,000,000**
Number of servers	45,978
Cost/server	$1450
CAPEX for servers	**$66,700,000**
Number of rack switches	1150
Cost/rack switch	$4800
Number of array switches	22
Cost/array switch	$300,000
Number of layer 3 switches	2
Cost/layer 3 switch	$500,000
Number of border routers	2
Cost/border router	$144,800
CAPEX for networking gear	**$12,810,000**
Total CAPEX for WSC	**$167,510,000**
Server amortization time	3 years
Networking amortization time	4 years
Facilities amortization time	10 years
Annual cost of money	5%

Figure 6.13 Case study for a WSC, rounded to nearest $5000. Internet bandwidth costs vary by application, so they are not included here. The remaining 18% of the CAPEX for the facility includes buying the property and the cost of construction of the building. We added people costs for security and facilities management in Figure 6.14, which were not part of the case study. Note that Hamilton's estimates were done before he joined Amazon, and they are not based on the WSC of a particular company. Based on Hamilton, J., 2010. Cloud computing economies of scale. In: Paper Presented at the AWS Workshop on Genomics and Cloud Computing, June 8, 2010, Seattle, WA. http://mvdirona.com/jrh/TalksAndPapers/JamesHamilton_GenomicsCloud20100608.pdf.

Hamilton's study works out to $11/watt for the building, power, and cooling. Barroso et al. (2013) reported consistent results for several cases, with the cost at $9 to $13/watt. Thus, a 16-MW facility costs $144 million to $208 million, *not* including the computing, storage, and networking equipment.

We can convert CAPEX into OPEX by a cost of capital conversion, assuming 5% borrowing cost, which is a standard convention in US accounting rules. That is, we can just amortize CAPEX as a fixed amount each month for the effective life of

the equipment. Figure 6.14 breaks down the monthly OPEX for Hamilton's case study. Note that the amortization rates differ significantly for his case study, from 10 years for the facility to 4 years for the networking equipment and 3 years for the servers. Thus, the WSC facility lasts a decade, but the servers are replaced every 3 years and the networking equipment every 4 years. By amortizing the CAPEX, Hamilton came up with a monthly OPEX, including accounting for the cost of borrowing money (5% annually) to pay for the WSC. At $3.8 million, the monthly OPEX is about 2% of the CAPEX (or 24% annually).

This figure allows us to calculate a handy guideline to keep in mind when making decisions about which components to use when being concerned about energy. The fully burdened cost of a watt per year in a WSC, including the cost of amortizing the power and cooling infrastructure, is

$$\frac{\text{Monthly cost of infrastructure + monthly cost of power}}{\text{Facility size in watts}} \times 12 = \frac{\$765\text{K} + \$475\text{K}}{8\text{M}} \times 12 = \$1.86$$

The cost is roughly $2 per watt-year. Thus, reducing costs by saving energy should not result in spending more than $2 per watt-year (see Section 6.8).

Note that in Figure 6.14, more than a third of OPEX is related to power, with that category trending up while server costs are trending down over time. The networking equipment is significant at 8% of total OPEX and 19% of the server CAPEX, and networking equipment is not trending down as quickly as servers are, perhaps because of the continuing demand for higher network bandwidth (see Figure 6.22 on page 467). This difference is especially true for the switches in the networking hierarchy above the rack, which represent most of the networking costs (see Section 6.6). People costs for security and facilities management are just 2% of OPEX. Dividing the OPEX in Figure 6.14 by the number of servers and hours per month, the cost is about $0.11 per server per hour.

Expense (% total)	Category	Monthly cost	Percent monthly cost
Amortized CAPEX (85%)	Servers	$2,000,000	53%
	Networking equipment	$290,000	8%
	Power and cooling infrastructure	$765,000	20%
	Other infrastructure	$170,000	4%
OPEX (15%)	Monthly power use	$475,000	13%
	Monthly people salaries and benefits	$85,000	2%
	Total OPEX	$3,800,000	100%

Figure 6.14 Monthly OPEX for Figure 6.13, rounded to the nearest $5000. Note that the 3-year amortization of servers means purchasing new servers every 3 years, whereas the facility is amortized for 10 years. Thus, the amortized capital costs for servers are about three times more than for the facility. People costs include three security guard positions continuously for 24 h a day, 365 days a year, at $20 per hour per person, and one facilities person for 24 h a day, 365 days a year, at $30 per hour. Benefits are 30% of salaries. This calculation does not include the cost of network bandwidth to the Internet because it varies by application nor vendor maintenance fees because they vary by equipment and by negotiations.

Barroso et al. (2013) evaluated CAPEX and OPEX in terms of cost per watt per month. Thus, if a 12-MW WSC is depreciated over 12 years, the depreciation cost is $0.08 per watt per month. They assumed the company got the capital for the WSC by taking out a loan at 8% annually—corporate loans are typically between 7% and 12%—and the interest payments added another $0.05, giving a total of $0.13 per watt per month. They factored in the cost of servers similarly. A 500 watt server that cost $4000 was $8 per watt, and the 4-year depreciation was $0.17 per watt per month. An 8% interest on a loan for the servers added $0.02. They estimated networking at $0.03 per watt per month. They reported that the typical OPEX cost for multiple MW WSCs varied from $0.02 to $0.08 per watt per month. The grand total was $0.37 to $0.43 per watt per month. For an 8-MW WSC, the monthly cost minus the cost of electricity is about $3.0 million to $3.5 million. If we subtract the monthly power use from Hamilton's calculation, his estimate of the monthly rate will be $3.3 million. Given the different approaches to predicting costs, these estimates are remarkably consistent.

Example The cost of electricity varies by region in the United States from $0.03 to $0.15 per kilowatt-hour. What is the impact on hourly server costs of these two extreme rates?

Answer We multiply the critical load of 8 MW by the average PUE from Figure 6.13 (second row) to calculate the average power usage:

$$8 \times 1.45 \times 80\% = 9.28 \text{ Megawatts}$$

The monthly cost for power then goes from $475,000 in Figure 6.14 to $205,000 at $0.03 per kilowatt-hour and to $1,015,000 at $0.15 per kilowatt-hour. These changes in electricity cost alter the hourly server costs from $0.11 to $0.10 and $0.13, respectively.

Example What would happen to monthly costs if the amortization times were all made to be the same—say, 5 years? How would that change the hourly cost per server?

Answer The spreadsheet is available online at *http://mvdirona.com/jrh/TalksAndPapers/PerspectivesDataCenterCostAndPower.xls*. Changing the amortization time to 5 years changes the first four rows of Figure 6.14 to

Servers	$1,260,000	37%
Networking equipment	$242,000	7%
Power and cooling infrastructure	$1,115,000	33%
Other infrastructure	$245,000	7%

and the total monthly OPEX is $3,422,000. If we replaced everything every 5 years, the cost would be $0.103 per server hour, with more of the amortized costs now being for the facility rather than the servers, as in Figure 6.14.

The rate of about $0.10 per server per hour can be much less than the cost for many companies that own and operate their own (smaller) conventional data centers. The cost advantage of WSCs led large Internet companies to offer computing as a utility where, like electricity, you pay only for what you use. Today, utility computing is better known as *cloud computing*.

6.5 Cloud Computing: The Return of Utility Computing

If computers of the kind I have advocated become the computers of the future, then computing may someday be organized as a public utility just as the telephone system is a public utility…. The computer utility could become the basis of a new and important industry.

John McCarthy,
MIT centennial celebration (1961)

Driven by the demand of an increasing number of users, Internet companies such as Amazon, Google, and Microsoft built increasingly larger warehouse-scale computers from commodity components, making McCarthy's prediction eventually come true, but not as he thought because of the popularity of timesharing. This demand led to innovations in systems software to support operating at this scale, including BigTable, Colossus, Dynamo, GFS, and MapReduce. It also demanded improvement in operational techniques to deliver a service available at least 99.99% of the time despite component failures and security attacks. Examples of these techniques include failover, firewalls, virtual machines, and protection against distributed denial-of-service attacks. With the software and expertise providing the ability to scale and increasing customer demand that justified the investment, WSCs with 50,000–100,000 servers have become commonplace in 2017.

With increasing scale came increasing economies of scale. Based on a study in 2006 that compared a WSC with a data center with only 1000 servers, Hamilton (2010) reported the following advantages:

- *5.7 times reduction in storage costs*—It cost the WSC $4.6 per GB per year for disk storage versus $26 per GB for the data center.
- *7.1 times reduction in administrative costs*—The ratio of servers per administrator was over 1000 for the WSC versus just 140 for the data center.
- *7.3 times reduction in networking costs*—Internet bandwidth cost the WSC $13 per Mbit/s/month versus $95 for the data center. Not surprisingly, one can negotiate a much better price per Mbit/s by ordering 1000 Mbit/s than by ordering 10 Mbit/s.

Another economy of scale comes during purchasing. The high level of purchasing leads to volume discount prices on virtually everything in the WSC.

Economies of scale also apply to operational costs. From the prior section, we saw that many data centers operate with a PUE of 2.0. Large firms can justify hiring mechanical and power engineers to develop WSCs with lower PUEs, in the range of 1.1–1.2 (see Section 6.7).

Internet services need to be distributed to multiple WSCs both for dependability and to reduce latency, especially for international markets. All large firms use multiple WSCs for that reason. It's much more expensive for individual firms to create multiple, small data centers around the world than a single data center in their corporate headquarters.

Finally, for the reasons presented in Section 6.1, servers in data centers tend to be utilized only 10%–20% of the time. By making WSCs available to the public, uncorrelated peaks between different customers can raise average utilization above 50%.

Thus, economies of scale for a WSC offer factors of 5–7 for several components of a WSC plus a few factors of 1.5–2 for the entire WSC.

Since the last edition of this book, the concerns about security have flipped for the cloud. In 2011 there was skepticism about placing critical data in the cloud because that could make it easier for hackers to break into than if the data were kept on premises ("on prem") locked down in the local data center. In 2017 data break-ins into such data centers are so routine that they barely make the news.

For example, this insecurity has even led to rapid growth of *ransomware*—where criminals break in, encrypt all the data of an organization, and won't release the key until paid a ransom—costing firms $1 billion in 2015. In contrast, WSCs are continuously under attack, their operators respond more quickly to halt them and thus build better defenses. As a result, ransomware is unheard of inside WSCs. WSCs are clearly more secure than the vast majority of local data centers today, so many CIOs now believe that critical data is safer in the cloud than "on prem."

Although there are several cloud computing providers, we feature Amazon Web Services (AWS) since it is one of the oldest and currently the largest commercial cloud provider.

Amazon Web Services

Utility computing goes back to commercial timesharing systems and even batch processing systems of the 1960s and 1970s, where companies only paid for a terminal and a phone line and then were billed based on how much computing they used. Many efforts since the end of timesharing have tried to offer such pay-as-you-go services, but they were often met with failure.

When Amazon started offering utility computing via the Amazon Simple Storage Service (Amazon S3) and then Amazon Elastic Computer Cloud (Amazon EC2) in 2006, it made some novel technical and business decisions:

- *Virtual machines*. Building the WSC using x86-commodity computers running the Linux operating system and the Xen virtual machine solved several

problems. First, it allowed Amazon to protect users from each other. Second, it simplified software distribution within a WSC, in that customers needed to install only an image and then AWS automatically distributed it to all the instances being used. Third, the ability to kill a virtual machine reliably made it easy for Amazon and customers to control resource usage. Fourth, virtual machines could limit the rate at which they used the physical processors, disks, and the network as well as the amount of main memory, which gave AWS multiple price points: the lowest price option by packing many virtual cores on a single server, the highest price option of exclusive access to all the machine resources, as well as several intermediary points. Fifth, virtual machines hid the identity of hardware, allowing AWS to continue to sell time on older machines that might otherwise be unattractive to customers if they knew the age of the machines. Finally, virtual machines allowed AWS to introduce new and faster hardware either by packing even more virtual cores per server or simply by offering instances that had higher performance per virtual core; virtualization meant that offered performance need not be an integer multiple of the performance of the hardware.

- *Very low cost*. When AWS announced a rate of $0.10 per hour per instance in 2006, it was a startlingly low amount. An instance is one virtual machine, and at $0.10 per hour, AWS allocated two instances per core on a multicore server. Thus, one EC2 computer unit is equivalent to a 1.0–1.2 GHz AMD Opteron or Intel Xeon of that era.
- *(Initial) reliance on open source software*. The availability of good-quality software that had no licensing problems or costs associated with running on hundreds or thousands of servers made utility computing much more economical for both Amazon and its customers. AWS later started offering instances including commercial third-party software at higher prices.
- *No (initial) guarantee of service*. Amazon originally promised only best effort. The low cost was so attractive that many could live without a service guarantee. Today AWS provides availability SLOs of up to 99.95% on services such as Amazon EC2 and Amazon S3. Additionally, Amazon S3 was designed for durability by saving multiple replicas of each object across multiple locations. (According to AWS, the chances of permanently losing an object are one in 100 billion.) AWS also provides a Service Health Dashboard that shows the current operational status of each of the AWS services in real time so that AWS uptime and performance are fully transparent.
- *No contract required*. In part because the costs are so low, all that is necessary to start using EC2 is a credit card.

Figures 6.15 and 6.16 show the hourly price of the many types of EC2 instances in 2017. Expanding from the 10 instance types in 2006, there are now more than 50. The fastest instance is 100 times quicker than the slowest, and

	Instance	Per hour	Ratio to m4.large	Virtual cores	Compute units	Memory (GiB)	Storage (GB)
General-purpose	t2.nano	$0.006	0.05	1	Variable	0.5	EBS only
	t2.micro	$0.012	0.11	1	Variable	1.0	EBS only
	t2.small	$0.023	0.21	1	Variable	2.0	EBS only
	t2.medium	$0.047	0.4	2	Variable	4.0	EBS only
	t2.large	$0.094	0.9	2	Variable	8.0	EBS only
	t2.xlarge	$0.188	1.7	4	Variable	16.0	EBS only
	t2.2xlarge	$0.376	3.5	8	Variable	32.0	EBS only
	m4.large	$0.108	1.0	2	6.5	8.0	EBS only
	m4.xlarge	$0.215	2.0	4	13	16.0	EBS only
	m4.2xlarge	$0.431	4.0	8	26	32.0	EBS only
	m4.4xlarge	$0.862	8.0	16	54	64.0	EBS only
	m4.10xlarge	$2.155	20.0	40	125	160.0	EBS only
	m4.16xlarge	$3.447	31.9	64	188	256.0	EBS only
	m3.medium	$0.067	0.6	1	3	3.8	1 × 4 SSD
	m3.large	$0.133	1.2	2	6.5	7.5	1 × 32 SSD
	m3.xlarge	$0.266	2.5	4	13	15.0	2 × 40 SSD
	m3.2xlarge	$0.532	4.9	8	26	30.0	2 × 80 SSD
Compute-optimized	c4.large	$0.100	0.9	2	8	3.8	EBS only
	c4.xlarge	$0.199	1.8	4	16	7.5	EBS only
	c4.2xlarge	$0.398	3.7	8	31	15.0	EBS only
	c4.4xlarge	$0.796	7.4	16	62	30.0	EBS only
	c4.8xlarge	$1.591	14.7	36	132	60.0	EBS only
	c3.large	$0.105	1.0	2	7	3.8	2 × 16 SSD
	c3.xlarge	$0.210	1.9	4	14	7.5	2 × 40 SSD
	c3.2xlarge	$0.420	3.9	8	28	15.0	2 × 80 SSD
	c3.4xlarge	$0.840	7.8	16	55	30.0	2 × 160 SSD
	c3.8xlarge	$1.680	15.6	32	108	60.0	2 × 320 SSD

Figure 6.15 Price and characteristics of on-demand general-purpose and compute-optimized EC2 instances in the Virginia region of the United States in February 2017. When AWS started, one EC2 computer unit was equivalent to a 1.0–1.2 GHz AMD Opteron or Intel Xeon of 2006. Variable instances are the newest and cheapest category. They offer the full performance of a high-frequency Intel CPU core if your workload utilizes less than 5% of the core on average over 24 h, such as for serving web pages. AWS also offers Spot Instances at a much lower cost (about 25%). With Spot Instances, customers set the price they are willing to pay and the number of instances they are willing to run, and then AWS runs the bids when the spot price drops below their level. AWS also offers Reserved Instances for cases where customers know they will use most of the instance for a year. They pay a yearly fee per instance and then an hourly rate that is about 30% of column 1 to use the service. If a Reserved Instance is used 100% for a whole year, the average cost per hour including amortization of the annual fee will be about 65% of the rate in the first column. EBS is Elastic Block Storage, which is a raw block-level storage system found elsewhere on the network, rather than in a local disk or local solid stage disk (SSD) within the same server as the VM.

	Instance	Per hour	Ratio to m4.large	Virtual cores	Compute units	Memory (GiB)	Storage (GB)
GPU	p2.xlarge	$0.900	8.3	4	12	61.0	EBS only
	p2.8xlarge	$7.200	66.7	32	94	488.0	EBS only
	p2.16xlarge	$14.400	133.3	64	188	732.0	EBS only
	g2.2xlarge	$0.650	6.0	8	26	15.0	60 SSD
	g2.8xlarge	$2.600	24.1	32	104	60.0	2 × 120 SSD
FPGA	f1.2xlarge	$1.650	15.3	8 (1 FPGA)	26	122.0	1 × 470 SSD
	f1.16xlarge	$13.200	122.2	64 (8 FPGA)	188	976.0	4 × 940 SSD
Memory-optimized	x1.16xlarge	$6.669	61.8	64	175	976.0	1 × 1920 SSD
	x1.32xlarge	$13.338	123.5	128	349	1,952.0	2 × 1920 SSD
	r3.large	$0.166	1.5	2	6.5	15.0	1 × 32 SSD
	r3.xlarge	$0.333	3.1	4	13	30.5	1 × 80 SSD
	r3.2xlarge	$0.665	6.2	8	26	61.0	1 × 160 SSD
	r3.4xlarge	$1.330	12.3	16	52	122.0	1 × 320 SSD
	r3.8xlarge	$2.660	24.6	32	104	244.0	2 × 320 SSD
	r4.large	$0.133	1.2	2	7	15.3	EBS only
	r4.xlarge	$0.266	2.5	4	14	30.5	EBS only
	r4.2xlarge	$0.532	4.9	8	27	61.0	EBS only
	r4.4xlarge	$1.064	9.9	16	53	122.0	EBS only
	r4.8xlarge	$2.128	19.7	32	99	244.0	EBS only
	r4.16xlarge	$4.256	39.4	64	195	488.0	EBS only
Storage-optimized	i2.xlarge	$0.853	7.9	4	14	30.5	1 × 800 SSD
	i2.2xlarge	$1.705	15.8	8	27	61.0	2 × 800 SSD
	i2.4xlarge	$3.410	31.6	16	53	122.0	4 × 800 SSD
	i2.8xlarge	$6.820	63.1	32	104	244.0	8 × 800 SSD
	d2.xlarge	$0.690	6.4	4	14	30.5	3 × 2000 HDD
	d2.2xlarge	$1.380	12.8	8	28	61.0	6 × 2000 HDD
	d2.4xlarge	$2.760	25.6	16	56	122.0	12 × 2000 HDD
	d2.8xlarge	$5.520	51.1	36	116	244.0	24 × 2000 HDD

Figure 6.16 Price and characteristics of on-demand GPUs, FPGAs, memory-optimized, and storage-optimized EC2 instances in the Virginia region of the United States in February 2017.

the largest offers 2000 times more memory than the smallest. Rent for the cheapest instance for a whole year is just $50.

In addition to computation, EC2 charges for long-term storage and for Internet traffic. (There is no cost for network traffic inside AWS regions.) Elastic Block Storage (EBS) costs $0.10 per GB per month when using SSDs and $0.045 per GB monthly for hard disk drives. Internet traffic costs $0.01 per GB going to EC2 and $0.09 per GB coming from EC2.

Example Calculate the cost of running the average MapReduce job in Figure 6.2 on page 438 on EC2 for several months over the years. Assume there are plenty of jobs, so there is no significant extra cost to round up so as to get an integer number of hours. Next calculate the cost per month to run all the MapReduce jobs.

Answer The first question is, what is the right size instance to match the typical server at Google? Let's assume the closest match in Figure 6.15 is a c4.large with 2 virtual cores and 3.6 GiB of memory, which costs $0.100 per hour. Figure 6.17 calculates the average and total cost per year of running the Google MapReduce workload on EC2. The average September 2016 MapReduce job would cost a little over $1 on EC2, and the total workload for that month would cost $114 million on AWS.

	Aug-04	Sep-09	Sep-12	Sep-16
Average completion time (h)	0.15	0.14	0.13	0.11
Average number of servers per job	157	156	142	130
Cost per hour of EC2 c4.large instance	$0.100	$0.100	$0.100	$0.100
Average EC2 cost per MapReduce job	$2.76	$2.23	$1.89	$1.20
Monthly number of MapReduce jobs	29,000	4,114,919	15,662,118	95,775,891
Total cost of MapReduce jobs on EC2/EBS	$80,183	$9,183,128	$29,653,610	$114,478,794

Figure 6.17 Estimated cost to run the Google MapReduce workload for select months between 2004 and 2016 (Figure 6.2) using 2017 prices for AWS EC2. Because we are using 2017 prices, these are underestimates of actual AWS costs.

Example Given the cost of MapReduce jobs, imagine that your boss wants you to investigate ways to lower costs. How much might you save using AWS Spot Instances?

Answer The MapReduce jobs could be disrupted by being kicked off a spot instance, but MapReduce is designed to tolerate and restart failed jobs. The AWS Spot price for c4.large was $0.0242 versus $0.100, which meant a savings of $87 million for September 2016, but there were no guarantees on the response times!

In addition to the low-cost and a pay-for-use model of utility computing, another strong attractor for cloud computing users is that the cloud-computing providers take on the risks of over-provisioning or under-provisioning. Because either mistake could be fatal, risk avoidance is a godsend for startup companies. If too much of the precious investment is spent on servers before the product is ready for heavy use, a company could run out of money. If the service suddenly became popular but there weren't enough servers to match the demand, a company could

make a very bad impression with the potential new customers it desperately needs in order to grow.

The poster child for this scenario is FarmVille from Zynga, a social networking game on Facebook. Before FarmVille was announced, the largest social game was about five million daily players. FarmVille had one million players 4 days after launching and 10 million players after 60 days. After 270 days, it had 28 million daily players and 75 million monthly players. Because FarmVille is deployed on AWS, it is able to grow seamlessly with the number of users. Moreover, it is able to shed load based on customer demand and time of day.

FarmVille was so successful that Zynga decided to open its own data centers in 2012. In 2015, Zynga returned to AWS, deciding it was better to let AWS run its data centers (Hamilton, 2015). When FarmVille dropped from the most popular Facebook application to 110th in 2016, Zynga was able to downsize gracefully with AWS, much as it grew with AWS in the beginning.

In 2014, AWS offered a new service that hearkened back to the timesharing days of the 1960s that John McCarthy was referring to in the opening quote of this section. Instead of managing virtual machines in the cloud, *Lambda* lets users supply a function in source code (such as Python) and lets AWS automatically manage the resources required by that code to scale with input size and to make it highly available. Google Cloud Compute Functions and Microsoft Azure Functions are equivalent capabilities from competing cloud providers. As Section 6.10 explains, Google App Engine originally offered a quite similar service in 2008.

This trend is referred to as *Serverless Computing*, in that users don't have to manage servers (but these functions *are* in fact run on servers). The tasks provided include operating system maintenance, capacity provisioning and automatic scaling, code and security patch deployment, and code monitoring and logging. It runs code in response to events, such as an http request or database update. One way to think of Serverless Computing is as a set of processes running in parallel across the entire WSC that share data through a disaggregated storage service such as AWS S3.

There is no cost for Serverless Computing when a program is idle. The AWS accounting is six orders of magnitude finer than EC2, recording usage per 100 ms instead of per hour. Cost varies depending on the amount of memory needed, but if your program used 1 GiB of memory, the cost is $0.000001667 per 100 ms or about $6 per hour.

Serverless Computing can be thought of as the next evolutionary step toward realizing the cloud computing ideals of the data center as a computer, as pay-as-you-go pricing, and as a means for automatic dynamic scaling.

Cloud computing has made the benefits of WSC available to everyone. Cloud computing offers cost associativity with the illusion of infinite scalability at no extra cost to the user: 1000 servers for 1 h cost no more than 1 server for 1000 h. It is up to the cloud computing provider to ensure that there are enough servers, storage, and Internet bandwidth available to meet the demand. The previously mentioned optimized supply chain, which drops time-to-delivery to a week for new computers, is a considerable aid in providing that illusion without

bankrupting the provider. This transfer of risks, cost associativity, pay-as-you-go pricing, and greater security is a powerful argument for companies of varying sizes to use cloud computing.

How Big Is the AWS Cloud?

AWS started in 2006 and grew so large that Amazon.com, rather than use a separate computing infrastructure, became one of AWS's customers in 2010. Figure 6.18 shows that AWS had facilities in 16 locations around the world in 2017, with two more on the way. As a point of interest, Figures 6.19 and 6.20 show similar maps for Google and Microsoft.

Each AWS location consists of two to three nearby facilities (one or two kilometers apart) called *availability zones*. They are so named because it should be safe to have your software running on two of them to ensure dependability as it is unlikely that both would fail simultaneously because of power outages or a natural disaster (Hamilton, 2014). These 16 locations contain 42 availability zones, and each of those zones has one or more WSCs. In 2014 each WSC had at least 50,000 servers, and some had more than 80,000.

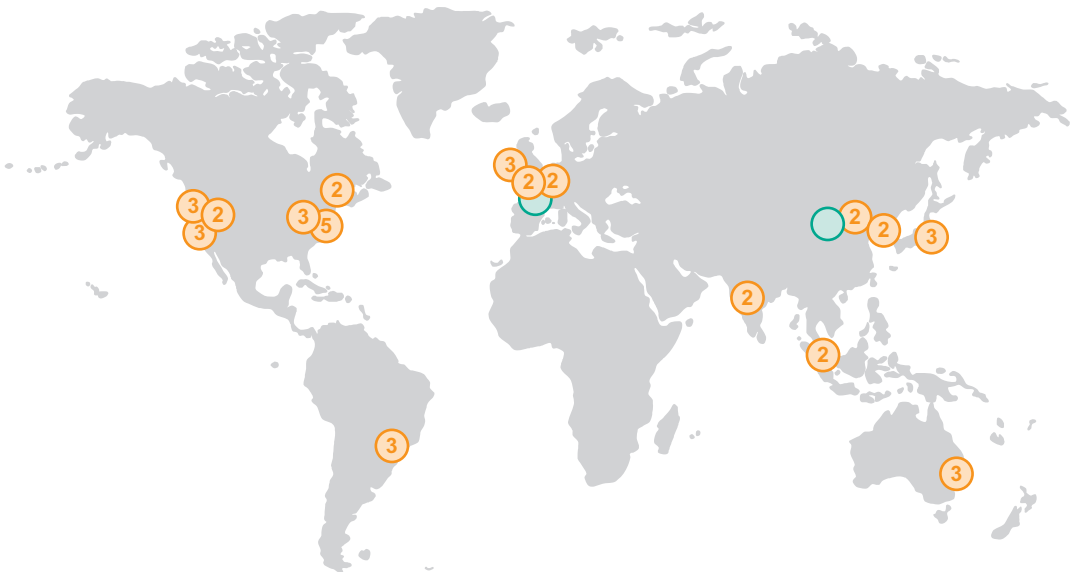


Figure 6.18 In 2017 AWS had 16 sites ("regions"), with two more opening soon. Most sites have two to three *availability zones*, which are located nearby but are unlikely to be affected by the same natural disaster or power outage, if one were to occur. (The number of availability zones are listed inside each circle on the map.) These 16 sites or regions collectively have 42 availability zones. Each availability zone has one or more WSCs. https://aws.amazon.com/about-aws/global-infrastructure/.

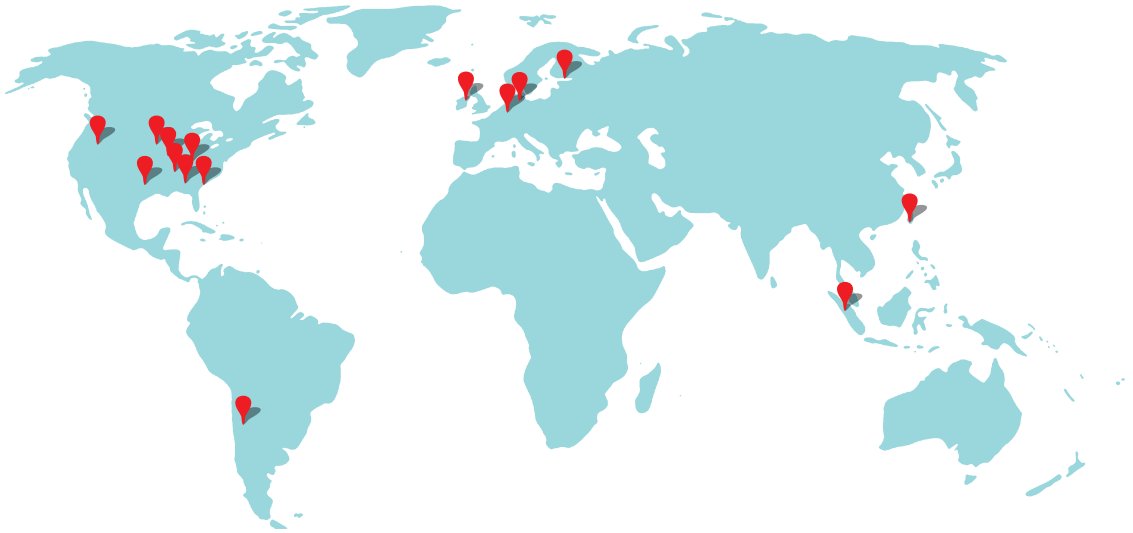


Figure 6.19 In 2017 Google had 15 sites. In the Americas: Berkeley County, South Carolina; Council Bluffs, Iowa; Douglas County, Georgia; Jackson County, Alabama; Lenoir, North Carolina; Mayes County, Oklahoma; Montgomery County, Tennessee; Quilicura, Chile; and The Dalles, Oregon. In Asia: Changhua County, Taiwan; Singapore. In Europe: Dublin, Ireland; Eemshaven, Netherlands; Hamina, Finland; St. Ghislain, Belgium. https://www.google.com/about/datacenters/inside/locations/.

Hamilton (2017) says its best to have at least three WSCs per region. The reason is simply that when one WSC fails, the other in the region needs to take on the load of the failed WSC. If there were only one other WSC, each would have to reserve half of its capacity for failover. With three, they could be used at two-thirds of capacity and still handle a quick failover. The more data centers you have, the less reserved excess capacity; AWS has regions with more than 10 WSCs.

We have found two published estimates of the total number of servers in AWS in 2014. One estimate was 2 million servers, when AWS had just 11 regions and 28 availability zones (Clark, 2014). Another estimate was between 2.8 and 5.6 million servers (Morgan 2014). If we extrapolate from 2014 to 2017 based on the increased number of availability zones, the estimates will grow to 3.0 million servers on the low end and 8.4 million on the high end. The total number of WSCs (data centers) is 84–126. Figure 6.21 shows the growth over time, using extrapolations from these two projections to offer high and low estimates of the number of severs and WSCs over time.

AWS is understandably mum on the actual number. They said that AWS had more than 1 million customers in 2014 and that "every *day* AWS adds enough physical server capacity equivalent to that needed to support Amazon.com in 2004" when it was a $7 billion annual revenue company (Hamilton, 2014).

One way to check the validity of these estimates is to look at investments. Amazon spent $24 billion in capital investments in property and equipment

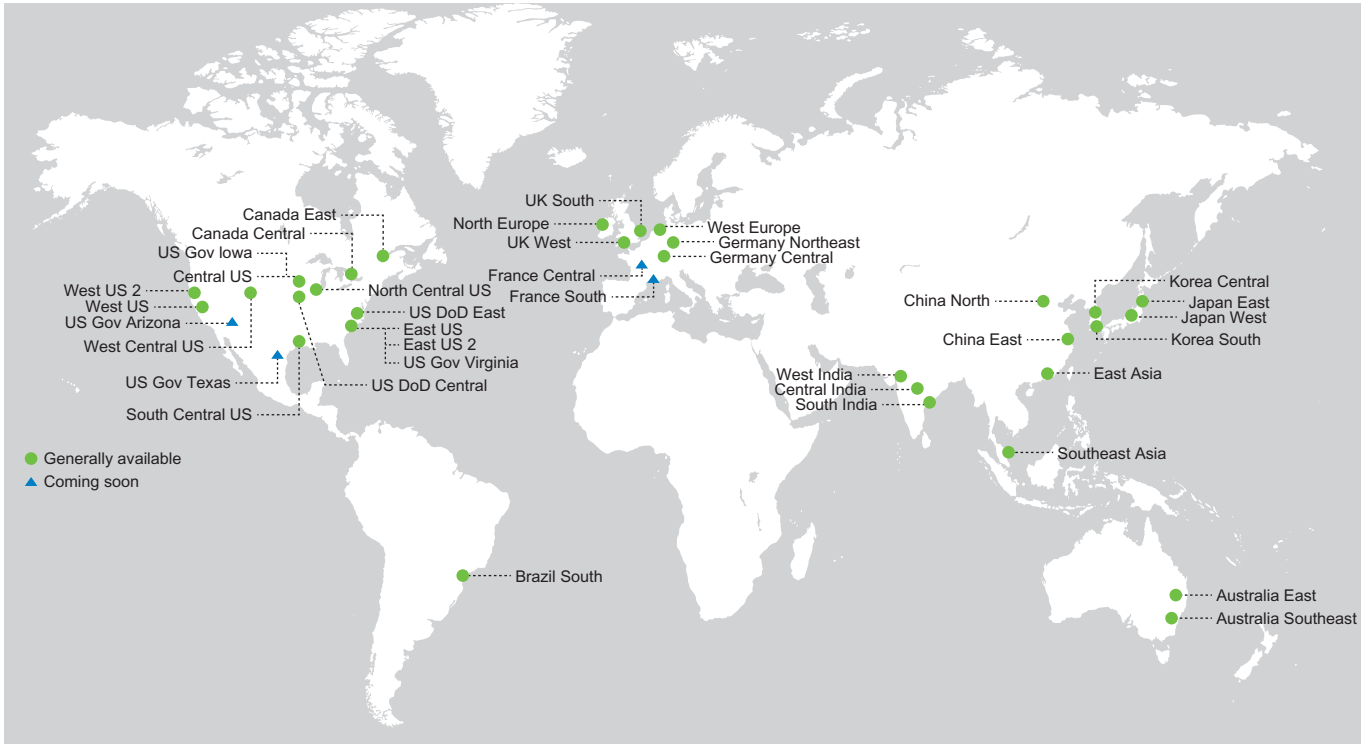


Figure 6.20 In 2017 Microsoft had 34 sites, with four more opening soon. https://azure.microsoft.com/en-us/regions/.

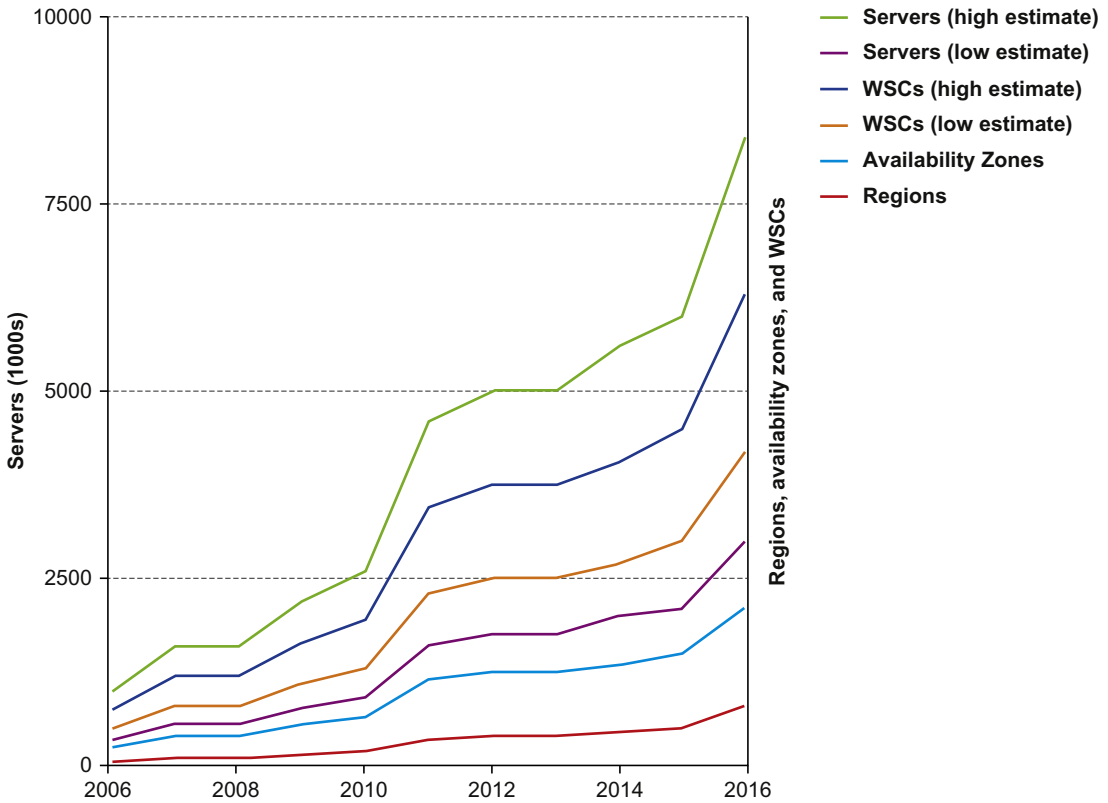


Figure 6.21 Growth of AWS regions and availability zones (right vertical axis) over time. Most regions have two or three availability zones. Each availability zone can have one or more WSCs, with the largest having more than 10 WSCs. Each WSC has at least 50,000 servers, with the biggest having more than 80,000 servers (Hamilton, 2014). Based on two published estimates for the number of AWS servers in 2014 (Clark, 2014; Morgan 2014), we project the number of servers per year (left vertical axis) and WSCs (right vertical access) as a function of the actual number of availability zones.

between 2013 and 2015, and one estimate is that two-thirds of that investment was for AWS (Gonzalez and Day 2016; Morgan 2016). Assume that it takes a year to construct a new WSC. The estimate in Figure 6.21 for 2014 to 2016 is from 34 to 51 WSCs. The cost per AWS WSC will then be $310 million to $470 million. Hamilton states that "even a medium sized datacenter (WSC) will likely exceed $200M." (Hamilton, 2017). He goes on to say that today cloud providers currently have "O(10^2)" WSCs; Figure 6.21 estimate is 84–126 AWS WSCs. Despite the fuzziness of these estimates, they appear to be surprisingly consistent. He goes on to predict that to meet the future demands, the largest cloud providers will eventually rise to "O(10^5)" WSCs, or *1000* times more WSCs than today!

No matter how many servers and WSCs are in the cloud, two cross-cutting issues that shape the cost-performance of WSCs and thus cloud computing are the WSC network and the efficiency of the server hardware and software.

6.6 Cross-Cutting Issues

Net gear is the SUV of the datacenter.

James Hamilton (2009)

Preventing the WSC Network From Being a Bottleneck

Figure 6.22 shows the network demands doubling every 12–15 months for Google, resulting in a 50× growth in traffic from the servers in Google's fleet of WSCs in just 7 years. Clearly, without great care, the WSC network could easily become a performance or cost bottleneck.

In the previous edition, we pointed out that a data center switch could cost almost $1 million, or more than 50 times as much as a Top of Rack switch. Not only was such a switch expensive, the resulting oversubscription affected the design of software and the placement of services and data within the WSC. The WSC network bottlenecks constrained data placement, which in turn complicated WSC software. Because this software is one of the most valuable assets of a WSC company, the cost of this added complexity was significant.

The ideal WSC network would be a black box whose topology and bandwidth are uninteresting because there are no restrictions: any workload could

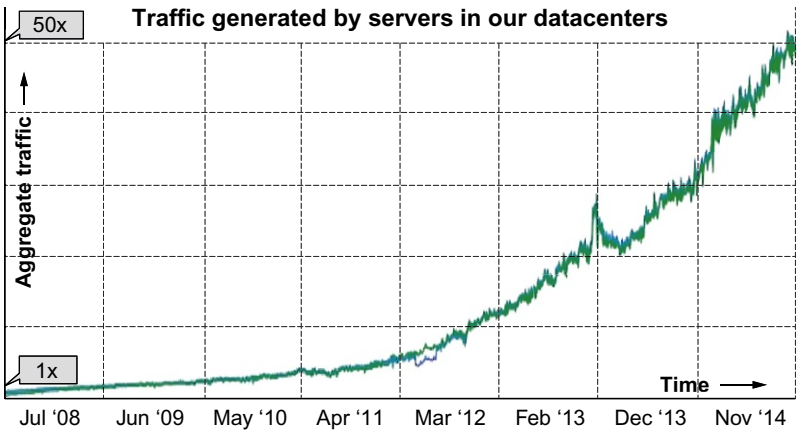


Figure 6.22 Network traffic from all the servers in Google's WSCs over 7 years (Singh et al., 2015).

be placed anywhere and optimized for server utilization rather than network traffic locality. Vahdat et al. (2010) proposed borrowing networking technology from supercomputing to overcome the price and performance problems. The latter proposed a networking infrastructure that could scale to 100,000 ports and 1 Pbit/s of bisection bandwidth. A major benefit of these novel data center switches is to simplify the software challenges because of oversubscription.

Since that time, many companies with WSC have designed their own switches to overcome these challenges (Hamilton, 2014). Singh et al. (2015) reported on several generations of custom networks used inside Google WSCs, which Figure 6.23 lists.

To keep costs down, they built their switches from standard commodity switch chips. They found that the features of traditional data center switches that were used in part to justify their high costs—such as decentralized network routing and protocols to manage support of arbitrary deployment scenarios—were unnecessary in a WSC because the network topology could be planned in advance of deployment

Data center generation switch	First deployed	Merchant silicon	Top of rack (ToR) switch config	Edge aggregation block	Spine block	Fabric speed	Host speed	Bisection BW
Four-Post CRs	2004	Vendor	48 × 1 Gbps	–	–	10 Gbps	1 Gbps	2 Tbps
Firehose 1.0	2005	8 × 10 Gbps 4 × 10 Gbps (ToR)	2 × 10 Gbps up 24 × 1 Gbps down	2 × 32 × 10 Gbps	32 × 10 Gbps	10 Gbps	1 Gbps	10 Tbps
Firehose 1.1	2006	8 × 10 Gbps	4 × 10 Gbps up 48 × 1 Gbps down	64 × 10 Gbps	32 × 10 Gbps	10 Gbps	1 Gbps	10 Tbps
Watchtower	2008	16 × 10 Gbps	4 × 10 Gbps up 48 × 1 Gbps down	4 × 128 × 10 Gbps	128 × 10 Gbps	10 Gbps	n × 1 Gbps	82 Tbps
Saturn	2009	24 × 10 Gbps	24 × 10 Gbps	4 × 288 × 10 Gbps	288 × 10 Gbps	10 Gbps	n × 10 Gbps	207 Tbps
Jupiter	2012	16 × 40 Gbps	16 × 40 Gbps	8 × 128 × 40 Gbps	128 × 40 Gbps	10/40 Gbps	n × 10 Gbps/ n × 40 Gbps	1300 Tbps

Figure 6.23 Six generations of network switches deployed at Google WSCs (Singh et al., 2015). The Four-Post CRs used commercial 512 port, 1 Gbit/s Ethernet switches, and 48-port, 1 Gbit/s Ethernet Top of Rack (ToR) switches, which allowed 20,000 servers in the array. The goal of Firehose 1.0 was to deliver 1 Gbps of nonblocking bisection bandwidth to each of 10,000 servers, but it ran into problems with the low connectivity of the ToR switch that caused problems when links failed. Firehose 1.1 was the first custom-designed switch with better connectivity in the ToR switch. Watchtower and Saturn followed in the same footsteps, but used new, faster merchant switch chips. Jupiter uses 40 Gbps links and switches to deliver more than 1 Pbit/s of bisection bandwidth. Section 6.7 describes the Jupiter switch and the Edge Aggregation and Spine Blocks of Clos networks in more detail.

and the network had only a single operator. Google instead used centralized control that relied on a common configuration that was copied to all data center switches. The modular hardware design and robust software control allowed these switches to be used both for inside the WSC and for wide area networks between WSCs. Google scaled the bandwidth of its WSCs networks by 100X in a decade, and offered more than 1 Pbit/s of bisection bandwidth in 2015.

Using Energy Efficiently Inside the Server

Although PUE measures the efficiency of a WSC, it has nothing to say about what goes on inside the IT equipment. Thus, another source of electrical inefficiency is the power supply *inside* the server, which converts input of high voltage to the voltages that chips and disks use. In 2007 many power supplies were 60%–80% efficient, which meant there were greater losses inside the server than there were going through the many steps and voltage changes from the high-voltage lines at the utility tower to supply the low-voltage lines at the server. One reason was that the power supply was often oversized in watts for what was on the motherboard. Moreover, such power supplies were typically at their worst efficiency at 25% load or less, even though, as Figure 6.3 on page 441 shows, many WSC servers operate in that range. Computer motherboards also have voltage regulator modules (VRMs), and they can have relatively low efficiency as well.

Barroso and Hölzle (2007) said the goal for the whole server should be *energy proportionality*; that is, servers should consume energy in proportion to the amount of work performed. A decade later, we've gotten close but have not hit that ideal goal. For example, the best-rated SPECpower servers in Chapter 1 still use about 20% of the full power when idle and almost 50% of full power at just 20% load. That represents huge progress since 2007 when an idle computer used 60% of full power and 70% at a 20% load, but there is still room to improve.

Systems software is designed to use all of an available resource if it potentially improves performance, without concern for the energy implications. For example, operating systems use all of memory for program data or for file caches, although much of the data will likely never be used. Software architects need to consider energy as well as performance in future designs (Carter and Rajamani, 2010).

Given the background from these six sections, we are now ready to appreciate the work of the Google WSC architects.

6.7 Putting It All Together: A Google Warehouse-Scale Computer

Because many companies with WSCs are competing vigorously in the marketplace, most have been reluctant to share their latest innovations with the public (and each other). Fortunately, Google has continued its tradition of providing details on recent WSCs for new editions of this book, once again making this

edition likely the most up-to-date public description of a Google WSC, which is representative of the current state-of-the-art.

Power Distribution in a Google WSC

We start with power distribution. Although there are many variations deployed, in North America electric power typically goes through multiple voltage changes on the way to the server, starting with the high-voltage lines at the utility tower of over 110,000 V.

For large-scale sites with multiple WSCs, power is delivered to on-site substations (Figure 6.24). The substations are sized for hundreds of megawatts of power. The voltage is reduced to between 10,000 and 35,000 V for distribution to WSCs on the site.

Near the buildings of the WSC, the voltage is further reduced to around 400 V (Figure 6.25) for distribution to the rows of servers on the data center floor. (480 V is common in North America, but 400 V in the rest of the world; Google uses 415 V.) To prevent the whole WSC from going offline if power is lost, WSCs have their version of an uninterruptible power supply (UPS), just as most servers do in conventional data centers. Diesel generators are connected to the power distribution system at this level to provide power in the event of an issue with the utility power. Although most outages are less than a few minutes, WSCs store thousands of gallons of diesel on site for an extended event. The operators even make provisions with local fuel companies for continuous delivery of diesel should a site need to operate from generators for days or weeks.

Inside the WSC, power is delivered to the racks via copper bus ducts that run above each row of racks, as Figure 6.26 shows. The last step splits the three-phase power into three separate single-phase powers of 240–277 V delivered by power



Figure 6.24 An on-site substation.



Figure 6.25 This image shows transformers, switch gear, and generators in close proximity to a WSC.



Figure 6.26 Row of servers with the copper bus ducts above that distribute 400 V to the servers. Although hard to see, they are above the shelf on the right side of the photo. It also shows a cold aisle that operators use to service the equipment.

cables to the rack. Near the top of the rack, power converters turn the 240 V AC current into 48 V DC to bring the voltage down to what boards can use.

In summary, power is distributed in a hierarchy in a WSC, with each level of the hierarchy corresponding to a distinct failure and maintenance unit: the whole WSC, arrays, rows, and racks. Software is aware of the hierarchy, and it spreads work and storage topographically to increase dependability.

WSCs around the world have different distribution voltages and frequencies, but the overall design is similar. The primary places for improvement in power

efficiency are in the voltage transformers at each step, but these are highly optimized components, so there is little opportunity left.

Cooling in a Google WSC

Now that we can deliver power from the utility poles to the floor of the WSC, we need to remove the heat generated from using it. There are considerably more opportunities for improvement in the cooling infrastructure.

One of the easiest ways to improve energy efficiency is simply to run the IT equipment at higher temperatures so that the air does not need to be cooled as much. Google runs its equipment at 80+°F (27+°C), which is considerably higher than traditional data centers that are so cold that you need to wear a jacket.

Airflow is carefully planned for the IT equipment, even using Computational Fluid Dynamics simulation to design the facility. Efficient designs preserve the temperature of the cool air by reducing the chances of it mixing with hot air.

For example, most WSCs today have alternating aisles of hot air and cold air by orienting servers in opposite directions in alternating rows of racks so that hot exhaust blows in alternating directions. They are referred to as *hot aisles* and *cold aisle*s. Figure 6.26 shows a cold aisle that people use to service the servers, and Figure 6.27 shows the hot aisle. The hot air from the hot aisle rises through ducts into the ceiling.



Figure 6.27 Hot aisle in a Google data center, which is clearly not designed to accommodate people.



Figure 6.28 The cool air blows into the room containing the aisles of servers. The hot air goes through large vents into the ceilings where it is cooled before returning to these fans.

In conventional data centers, each server relies on internal fans to ensure a sufficient flow of cool air over the hot chips to maintain their temperature. These mechanical fans are one of the weakest components in servers; for example, the MTBF of fans is 150,000 h versus 1,200,000 h for disks. In a Google WSC, the server fans work synergistically with dozens of giant fans in the room to ensure airflow for the whole room (Figure 6.28). This division of labor means the small server fans use as little power as possible while delivering maximum performance at the worst-case power and ambient conditions. The large fans are controlled using air pressure as the control variable. The fan speeds are adjusted to maintain a minimum pressure difference between the hot and cold aisles.

To cool this hot air, they add large-scale fan-coils at either end of the rows of racks. Hot air from the racks is delivered to the fan-coils above via a horizontal plenum inside the hot aisle. (Two rows share the pair of cooling coils, as they are placed above the cold aisle between the two rows.) The cooled air is sent via a plenum in the ceiling to the wall with the big fans in Figure 6.28, which return the cooled air to the room containing the racks.

We'll describe how to remove the heat from the water in the cooling coils shortly, but let's reflect on the architecture so far. It separates the racks from the cooling capacity provided by the fan-coils, which allows for sharing of cooling across two rows of racks in the WSC. Thus, it efficiently provides more cooling to high-power racks and less to low-power racks. With thousands of racks in a WSC, they are unlikely to be identical, so power variability between racks is common, which this design accommodates.

Cool water is supplied to the individual fan-coils via a network of pipes from a cooling plant. Heat is transferred into the water via forced convection in the cooling coils, and warm water returns to a cooling plant.

To improve the efficiency of WSCs, architects try to use the local environment to remove the heat whenever possible. Evaporative *cooling towers* are common in WSCs to leverage the colder outside air to cool the water instead of it being chilled mechanically. The temperature that matters is called the *wet-bulb temperature*, which is the lowest temperature that can be achieved by evaporating water with air. It is the temperature a parcel of air would have if it were cooled to saturation (100% relative humidity) by the evaporation of water into it, with the latent heat being supplied by the parcel. Wet-bulb temperature is measured by blowing air at the bulb end of a thermometer that has water on it.

Warm water is sprayed inside in the cooling tower and collected in pools at the bottom, transferring heat to the outside air via evaporation and thereby cooling the water. This technique is called *water-side economization*. Figure 6.29 shows the steam rising above cooling towers. An alternative is to use cold water instead of crisp air. Google's WSC in Finland uses a water-to-water heat exchanger that takes the frigid water from the Gulf of Finland to chill the warm water from inside the WSC.

The cooling tower system uses water caused by evaporation in the cooling towers. For example, an 8-MW facility might need 70,000–200,000 gallons of water per day, thus the desire for the WSC to be located near ample sources of water.

Although the cooling plant is designed so that heat can be removed without artificial cooling most of the time, mechanical chillers aid in rejecting the heat in some regions when the weather is warm.



Figure 6.29 Steam rising from the cooling towers that transfer heat to the air from the water used to cool equipment.

Racks of a Google WSC

We saw how Google gets power to rack and how it cools the hot air that exhausts from the rack. Now we're ready to explore the rack itself. Figure 6.30 shows a typical rack found inside a Google WSC. To put this rack into context, a WSC consists of multiple arrays (which Google calls clusters). Although arrays vary in size, some have one to two dozen rows with each row holding two to three dozen racks.

The 20 slots shown the middle of the rack in Figure 6.30 hold the servers. Depending on their width, up to four servers can be placed in a single tray. The power converters near the top of the rack turn the 240 V AC current into 48 V DC, which is run on copper bus bars down the back of the rack to power the servers.

The diesel generators that provide backup power for the whole WSC take tens of seconds before they can offer power. Instead of populating a large room with

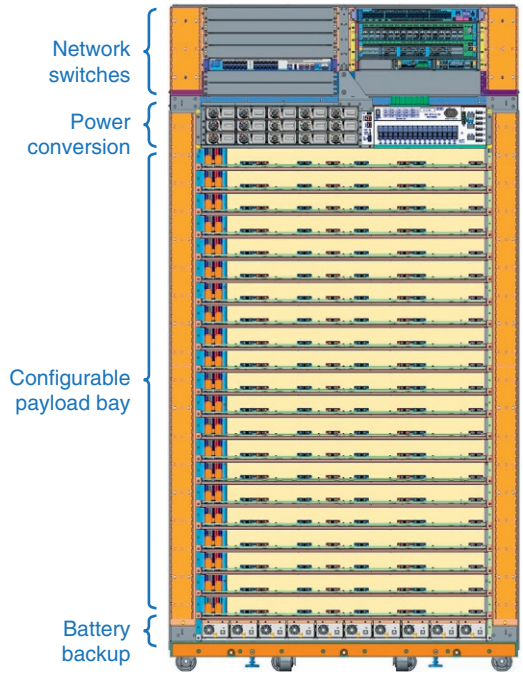


Figure 6.30 A Google rack for its WSC. Its dimensions are about 7 ft high, 4 ft wide, and 2 ft deep (2 m × 1.2 m × 0.5 m). The Top of Rack switches are indeed at the top of this rack. Next comes the power converter that converts from 240 V AC to 48 V DC for the servers in the rack using a bus bar at the back of the rack. Next is the 20 slots (depending on the height of the server) that can be configured for the various types of servers that can be placed in the rack. Up to four servers can be placed per tray. At the bottom of the rack are high-efficiency distributed modular DC uninterruptible power supply (UPS) batteries.

enough batteries to power the whole WSC for several minutes—which was a common practice in the early WSCs—Google puts small batteries at the bottom of each rack. Because UPS is distributed to each rack, the cost is incurred only as racks are deployed, instead of paying upfront for the UPS capacity of a full WSC. These batteries are also better than the traditional batteries because they are on the DC side after the voltage conversions, and they use an efficient charging scheme. In addition, replacing the 94%-efficient lead batteries with the 99.99%-efficient local UPS helps to lower the PUE. It's a very efficient UPS system.

It is comforting that the top of the rack in Figure 6.30 does indeed contain the Top of Rack switch, which we describe next.

Networking in a Google WSC

The Google WSC network uses a topology called *Clos*, which is named after the telecommunications expert who invented it (Clos, 1953). Figure 6.31 shows the structure of the Google Clos network. It is a multistage network that uses low port-count ("low radix") switches, offers fault tolerance, and increases both the network scale and its bisection bandwidth. Google increases the scale simply by adding stages to the multistage network. The fault tolerance is provided by its inherent redundancy, which means a failure of any link has only a small impact on the overall network capacity.

As Section 6.6 describes, Google builds customer switches from standard commodity switch chips and uses centralized control for network routing and management. Every switch is given a consistent copy of the current topology of the network, which simplifies the more complex routing of a Clos network.

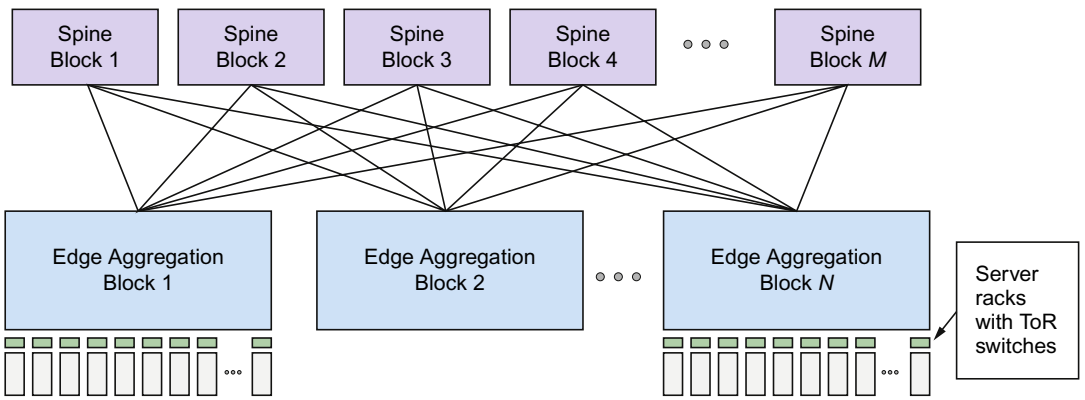


Figure 6.31 A Clos network has three logical stages containing crossbar switches: ingress, middle, and egress. Each input to the ingress stage can go through any of the middle stages to be routed to any output of the egress stage. In this figure, the middle stages are the *M* Spine Blocks, and the ingress and egress stages are in the *N* Edge Activation Blocks. Figure 6.22 shows the changes in the Spine Blocks and the Edge Aggregation Blocks over many generations of Clos networks in Google WSCs.

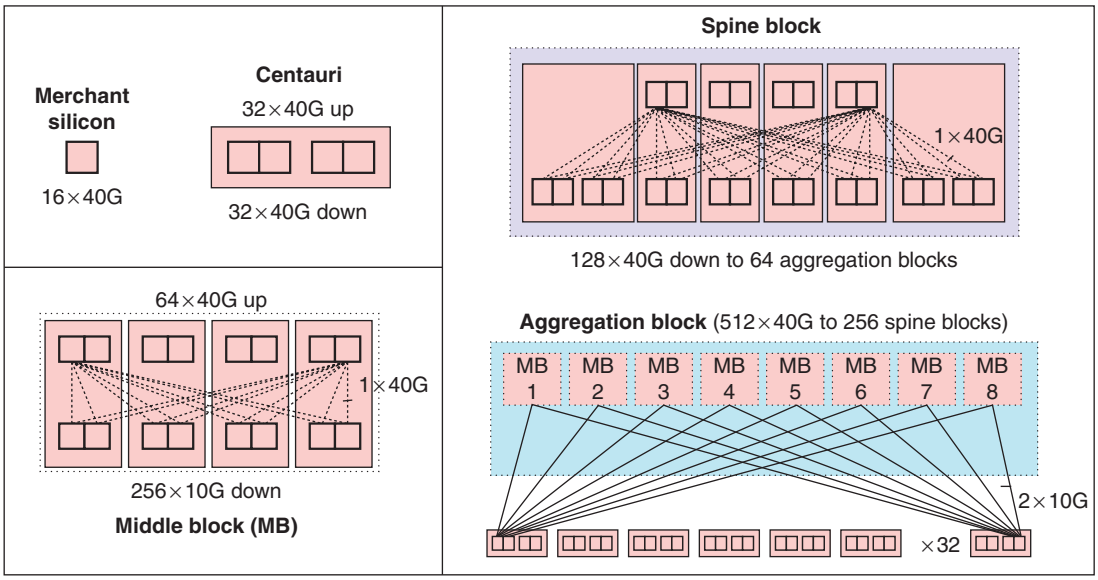


Figure 6.32 Building blocks of the Jupiter Clos network.

The latest Google switch is Jupiter, which is the switch's sixth generation. Figure 6.32 shows the building blocks of the switch, and Figure 6.33 shows the wiring of the middle blocks housed in racks. All the cables use bundles of optical fibers.

The commodity switch chip for Jupiter is a 16×16 crossbar using 40 Gbps links. The Top of Rack switch has four of these chips, which are configured with 48 40-Gbps links to the servers and 16 40-Gbps links to the network fabric, yielding an oversubscription of just 3:1, which is better than earlier generations. Moreover, this generation was the first time that servers were offered with 40-Gbps links.

The middle blocks in Figures 6.32 and 6.33 consist of 16 of the switch chips. They use two stages, with 256 10-Gbps links for the Top of Rack connectivity and 64 40-Gbps links to connect to the rest of the network fabric through the spine. Each of the chips in the Top of Rack switch connects to eight middle blocks using dual redundant 10-Gbps links.

Each aggregation block is connected to the spine block with 512 40-Gbps links. A spine block uses 24 switch chips to offer 128 40-Gbps ports to the aggregation blocks. At the largest scale, they use 64 aggregation blocks to provide dual redundant links. At this maximum size, the bisection bandwidth is an impressive 1.3 Pbit (10^{15}) per second.

Note that the whole Internet might have a bisection bandwidth of just 0.2 Pbit/s; one reason is that Jupiter was built for a high bisection bandwidth, but the Internet was not.



Figure 6.33 Middle blocks of the Jupiter switches housed in racks. Four are packed in a rack. A rack can hold two spine blocks.

Servers in a Google WSC

Now that we have seen how to power, cool, and communicate, we are finally ready to see the computers that do the actual work of the WSC.

The example server in Figure 6.34 has two sockets, each containing an 18-core Intel Haswell processor running at 2.3 GHz (see Section 5.8). The photo shows 16 DIMMs, and these servers are typically deployed with 256 GB total of DDR3-1600 DRAM. The Haswell memory hierarchy has two 32 KiB L1 caches, a 256 KiB L2 cache, and 2.5 MiB of L3 cache per core, resulting in a 45 MiB L3 cache. The local memory bandwidth is 44 GB/s with a latency of 70 ns, and the intersocket bandwidth is 31 GB/s with a latency of 140 ns to remote memory. Kanev et al. (2015) highlighted the differences between the SPEC benchmark suite and a WSC workload. An L3 cache is barely needed for SPEC, but it is useful for a real WSC workload.

The baseline design has a single network interface card (NIC) for a 10 Gbit/s Ethernet link, although 40 Gbit/s NICs are available. (Other cloud providers



Figure 6.34 An example server from a Google WSC. The Haswell CPUs (2 sockets × 18 cores × 2 threads = 72 "virtual cores" per machine) have 2.5 MiB last level cache per core or 45 MiB using DDR3-1600. They use the Wellsburg Platform Controller Hub and have a TFP of 150 W.

moved to 25 Gbit/s or multiples thereof.) While the photo in Figure 6.34 shows two SATA disk drives, each of which can contain up to 8 TB, the server also can be configured with SSD flash drives with 1 TB of storage. The peak power of the baseline is about 150 watts. Four of these servers can fit in a slot of the rack in Figure 6.30.

This baseline node is supplemented to offer a storage (or "diskfull") node. The second unit contains 12 SATA disks and is connected to the server over PCIe. Peak power for a storage node is about 300 watts.

Conclusion

In the previous edition, the Google WSC we described had a PUE of 1.23 in 2011. As of 2017, the average PUE of the whole Google fleet of 16 sites dropped to 1.12, with the Belgium WSC leading the way with a 1.09 PUE. The energy-saving techniques include

- Operating servers at higher temperatures means that air has to be chilled only to 80+°F (27°C) instead of the traditional 64–71°F (18–22°C).
- A higher target for cold air temperature helps put the facility more often within the range that can be sustained by cooling towers, which are more energy-efficient than traditional chillers.

- Deploying WSCs in temperate climates to allow use of evaporative cooling exclusively for large portions of the year.
- Adding large fans for entire rooms to work in concert with the small fans of the servers to reduce energy while satisfying worst-case scenarios.
- Averaging the cooling per server to whole racks of servers by deploying the cooling coils per row to accommodate warmer and cooler racks.
- Deploying extensive monitoring hardware and software to measure actual PUE versus designed PUE improves operational efficiency.
- Operating more servers than the worst-case scenario for the power distribution system would suggest. It is safe since it's statistically improbable that thousands of servers would all be highly busy simultaneously as long as there is a monitoring system to off-load work in the unlikely case that they did (Fan et al., 2007; Ranganathan et al., 2006). PUE improves because the facility is operating closer to its fully designed capacity, where it is at its most efficient because the servers and cooling systems are not energy-proportional. Such increased utilization reduces demand for new servers and new WSCs.

It will be interesting to see what innovations remain to further improve the WSC efficiency so that we are good guardians of our environment. It is hard to imagine now how engineers might halve the power and cooling overhead of a WSC prior to the next edition of this book, as they did between the previous edition and this one.

6.8 Fallacies and Pitfalls

Despite WSC being just 15 years old, WSC architects like those at Google have already uncovered many pitfalls and fallacies about WSCs, often the hard way. As we said in the introduction, WSC architects are today's Seymour Crays.

Fallacy *Cloud computing providers are losing money.*

When AWS was announced, a popular question about cloud computing was whether it was profitable at the low prices at the time. Amazon Web Services has grown so large that it must be recorded separately in Amazon's quarterly reports. To the surprise of some, AWS has proved to be the most profitable portion of the company. AWS had $12.2 billion in revenue for 2016, with an operating margin of 25%, whereas Amazon's retail operations had an operating margin of less than 3%. AWS is consistently responsible for three-fourths of Amazon's profits.

Pitfall *Focusing on average performance instead of 99th percentile performance.*

As Dean and Barroso (2013) observed, developers of WSC services worry about the tail more than they care about the mean. If some customers get terrible performance, that experience can drive them away to a competitor, and they'll never return.

Pitfall *Using too wimpy a processor when trying to improve WSC cost-performance.*

Amdahl's Law still applies to WSC. There will be some serial work for each request and that can increase request latency if this work runs on a slow server (Hölzle, 2010; Lim et al., 2008). If the serial work increases latency, then the cost of using a wimpy processor must include the software development costs to optimize the code to return it to the lower latency. The larger number of threads of many slow servers can also be more difficult to schedule and load balance, and thus the variability in thread performance can lead to longer latencies. When required to wait for the longest task, a 1-in-1000 chance of bad scheduling is probably not an issue with 10 tasks, but problematic with 1000 tasks.

Many smaller servers can also lead to lower utilization because it's clearly easier to schedule fewer things. Finally, even some parallel algorithms get less efficient when the problem is partitioned too finely. The Google rule of thumb is to use the low-end range of server class computers (Barroso and Hölzle, 2009).

As a concrete example, Reddi et al. (2010) compared embedded microprocessors (Atom) and server microprocessors (Nehalem Xeon) running the Bing search engine. They found that the latency of a query was about three times longer on Atom than on Xeon. Moreover, the Xeon was more robust. As load increases on Xeon, quality of service degrades gradually and modestly. The Atom design quickly violates its quality-of-service target as it tries to absorb additional load. Although the Atom design is more energy-efficient, the response time affects revenue, and the revenue loss is likely much greater than the cost savings of less energy. Energy-efficient designs that cannot match the response-time goals are unlikely to be deployed; we'll see another version of this pitfall lesson in the next chapter (Section 7.9).

This behavior translates directly into search quality. Given the importance of latency to the user, as Figure 6.12 suggests, the Bing search engine uses multiple strategies to refine search results if the query latency has not yet exceeded a cutoff latency. The lower latency of the larger Xeon nodes means they can spend more time refining search results. Thus, even when the Atom had almost no load, it gave worse answers in 1% of the queries than Xeon. At normal loads, 2% of the answers were worse.

Kanev et al. (2015) has more recent, yet consistent, results.

Pitfall *Inconsistent measure of PUE by different companies.*

Google's PUE measurements start from the power before it reaches the substation. Some measure at the entrance to the WSC, which skips voltage step downs that represent a 6% loss. There will also be different results depending on the season of the year if the WSC relies on the atmosphere to help cool the system. Finally, some report the design goal of the WSC instead of measuring the resulting system. The most conservative and best PUE measurement is a running average of the past 12 months of the measured PUE, starting from the feed of the utility.

Fallacy *Capital costs of the WSC facility are higher than for the servers that it houses.*

Although a quick look at Figure 6.13 on page 453 might lead one to that conclusion, that quick glimpse ignores the length of amortization for each part of the full WSC. However, the facility lasts 10–15 years, whereas the servers need to be repurchased every 3 or 4 years. Using the amortization times in Figure 6.13 of 10 years and 3 years, respectively, the capital expenditures over a decade are \$72 million for the facility and $3.3 \times \$67$ million, or \$221 million, for servers. Thus, the capital costs for servers in a WSC over a decade are a factor of three higher than for the WSC facility.

Pitfall *Trying to save power with inactive low power modes versus active low power modes.*

Figure 6.3 on page 441 shows that the average utilization of servers is between 10% and 50%. Given the concern about operational costs of a WSC from Section 6.4, one would think low power modes would be a huge help.

As Chapter 1 mentions, DRAMs or disks cannot be accessed in these *inactive low power modes*, so they must be returned to fully active mode to read or write, no matter how low the rate. The pitfall is that the time and energy required to return to fully active mode make inactive low power modes less attractive. Figure 6.3 shows that almost all servers average at least 10% utilization, so long periods of low activity might be expected, but not long periods of inactivity (Lo et al., 2014).

In contrast, processors still run in lower power modes at a small multiple of the regular rate, so *active low power modes* are much easier to use. Note that the time to move to fully active mode for processors is also measured in microseconds, so active low power modes also address the latency concerns about low power modes.

Fallacy *Given improvements in DRAM dependability and the fault tolerance of WSC systems software, there is no need to spend extra for ECC memory in a WSC.*

Because ECC adds 8 bits to every 64 bits of DRAM, potentially a ninth of the DRAM costs could be saved by eliminating error-correcting code (ECC), especially since measurements of DRAM have claimed failure rates of 1000–5000 FIT (failures per billion hours of operation) per megabit (Tezzaron Semiconductor, 2004).

Schroeder et al. (2009) studied measurements of the DRAMs with ECC protection at the majority of Google's WSCs, which was surely many hundreds of thousands of servers, over a 2.5-year period. They found 15–25 times higher FIT rates than had been published, or 25,000–70,000 failures per megabit. Failures affected more than 8% of DIMMs, and the average DIMM had 4000 correctable errors and 0.2 uncorrectable errors per year. Measured at the server, about a third experienced DRAM errors each year, with an average of 22,000 correctable errors and 1 uncorrectable error per year. That is, for one-third of the servers, one memory error was corrected every 2.5 h. Note that these systems used the more powerful Chipkill codes rather than the simpler SECDED codes. If the easier scheme had been used, the uncorrectable error rates would have been 4–10 times higher.

In a WSC that had only parity error protection, the servers would have to reboot for each memory parity error. If the reboot time were 5 min, one-third of the machines would spend 20% of their time rebooting! Such behavior would lower the performance of the expensive facility by about 6%. Moreover, these systems would suffer many uncorrectable errors without operators being notified that they occurred.

In the early years, Google used DRAM that did not even have parity protection. In 2000, during testing before shipping the next release of the search index, it started suggesting random documents in response to test queries (Barroso and Hölzle, 2009). The reason was a stuck-at-zero fault in some DRAMs, which corrupted the new index. Google added consistency checks to detect such errors in the future. As WSC grew in size and as ECC DIMMs became more affordable, ECC became the standard in Google WSCs. ECC has the added benefit of making it much easier to find broken DIMMs during repair.

Such data suggest why the Fermi GPU (Chapter 4) adds ECC to its memory where its predecessors didn't even have parity protection. Moreover, these FIT rates cast doubts on efforts to use the Intel Atom processor in a WSC—because of its improved power efficiency—since the chip set did not support ECC DRAM.

Pitfall *Coping effectively with microsecond delays as opposed to nanosecond or millisecond delays.*

Barroso et al. (2017) point out that modern computer systems make it easy for programmers to mitigate latencies in the nanosecond and millisecond timescales (such as cache and DRAM accesses at tens of nanoseconds and disk accesses at a few milliseconds) but that such systems significantly lack support for microsecond-scale events. Programmers get a synchronous interface to the memory hierarchy, with hardware doing heroic work so that such accesses appear consistent and coherent (Chapter 2). Operating systems offer programmers a similar synchronous interface for a disk read, with many lines of OS code enabling the safe switching to another process while waiting for the disk and then returning again to the original process when the data is ready. We need new mechanisms to cope with the microsecond delays of memory technologies like Flash or the fast network interfaces like 100 Gbit/s Ethernet.

Fallacy *Turning off hardware during periods of low activity improves cost-performance of a WSC.*

Figure 6.14 on page 454 shows that the cost of amortizing the power distribution and cooling infrastructure is 50% higher than the entire monthly power bill. Thus, although it certainly would save some money to compact workloads and turn off idle machines, even if half the power were saved, the monthly operational bill would be reduced only by 7%. There would also be practical problems to overcome because the extensive WSC monitoring infrastructure depends on being able to poke equipment and see it respond. Another advantage of energy proportionality and active low power modes is that they are compatible with the WSC monitoring infrastructure, which allows a single operator to be responsible for more than 1000

servers. Note also that preventive maintenance is one of the important tasks that take place during idle time.

The conventional WSC wisdom is to run other valuable tasks during periods of little activity to recoup the investment in power distribution and cooling. A prime example is the batch MapReduce jobs that create indices for search. Another example of getting value from meager utilization is spot pricing on AWS, which the example in Figure 6.17 on page 461 illustrates. AWS users who are flexible about when their tasks are run can save up to a factor of four for computation by letting AWS schedule the tasks more flexibly using spot instances, such as when the WSC would otherwise have low utilization.

6.9 Concluding Remarks

Inheriting the title of building the world's biggest computers, computer architects of WSCs are designing the large part of the future IT that supports the mobile client and IoT devices. Many of us use WSCs many times a day, and the number of times per day and the number of people using WSCs will surely increase in the next decade. Already more than six billion of the seven billion people on the planet have cell phone subscriptions. As these devices become Internet-ready, many more people from around the world will be able to benefit from WSCs.

Moreover, the economies of scale uncovered by WSC have realized the long-dreamed-of goal of computing as a utility. Cloud computing means anyone anywhere with good ideas and business models can tap thousands of servers to deliver their vision almost instantly. Of course, there are important obstacles that could limit the growth of cloud computing around standards, privacy, the rate of growth of Internet bandwidth, and the pitfalls we mention in Section 6.8, but we foresee them being addressed so that cloud computing can continue to flourish.

Among the many attractive features of cloud computing is that it offers economic incentives for conservation. Whereas it is hard to convince cloud computing *providers* to turn off unused equipment to save energy given the cost of the infrastructure investment, it is easy to convince cloud computing *users* to give up idle instances since they are paying for them, whether or not they are doing anything useful. Similarly, charging by use encourages programmers to use computation, communication, and storage efficiently, which can be difficult to encourage without an understandable pricing scheme. The explicit pricing also makes it possible for researchers to evaluate innovations in cost-performance instead of just performance, because costs are now easily measured and believable. Finally, cloud computing means that researchers can evaluate their ideas at the scale of thousands of computers, which in the past only large companies could afford.

We believe that WSCs are changing the goals and principles of server design, just as the needs of mobile clients and IoT are changing the goals and principles of microprocessor design. Both are revolutionizing the software industry, as well. Performance per dollar and performance per joule drive both mobile client hardware and the WSC hardware, and parallelism and domain-specific accelerators are key to delivering on those sets of goals. Architects will play a vital role in both halves of this exciting future world.

Looking forward, the end of Moore's Law and Dennard scaling (Chapter 1) means that the single-thread performance of the newest processors is not that much faster than their predecessors, which will likely stretch the lifetimes of the servers in the WSCs. Thus, the money formerly spent replacing older servers will instead be used to expand to the cloud, which could mean that the cloud will be even more economically attractive in the next decade than it is today. The Moore's Law era combined with innovations in the design and operation of WSCs caused the performance-cost-energy curve of WSCs to improve continuously. With the end of that glorious era, plus the removal of the largest causes of inefficiency in WSCs, the field will likely need to look to for innovations in computer architecture of the chips that populate the WSC for sustained improvement, which is the topic of the next chapter.

6.10 Historical Perspectives and References

Section M.8 (available online) covers the development of clusters that were the foundation of WSC and of utility computing. (Readers interested in learning more should start with Barroso et al. (2013) and the blog postings of James Hamilton at *http://perspectives.mvdirona.com* plus his talks at the annual Amazon Re-Invent conference.)

Case Studies and Exercises by Parthasarathy Ranganathan

Case Study 1: Total Cost of Ownership Influencing Warehouse-Scale Computer Design Decisions

Concepts illustrated by this case study

- Total Cost of Ownership (TCO)
- Influence of Server Cost and Power on the Entire WSC
- Benefits and Drawbacks of Low-Power Servers

Total cost of ownership is an important metric for measuring the effectiveness of a warehouse-scale computer (WSC). TCO includes both the CAPEX and OPEX described in Section 6.4, and reflects the ownership cost of the entire datacenter to achieve a certain level of performance. In considering different servers, networks, and storage architectures, TCO is often the most important comparison metric used by datacenter owners to decide which options are best; however, TCO is a multidimensional computation that takes into account many different factors. The goal of this case study is to take a detailed look into WSCs, to see how different architectures influence TCO, and to understand how TCO drives operator decisions. This case study will use the numbers from Figures 6.13 and 6.14 and Section 6.4, and assumes that the described WSC achieves the operator's target level of

performance. TCO is often used to compare different server options that have multiple dimensions. The exercises in this case study examine how such comparisons are made in the context of WSCs and the complexity involved in making the decisions.

6.1 [5/5/10] <6.2, 6.4> In this chapter, data-level parallelism has been discussed as a way for WSCs to achieve high performance on large problems. Conceivably, even greater performance can be obtained by using high-end servers; however, higher performance servers often come with a nonlinear price increase.

a. [5] <6.4> Assuming servers that are 10% faster at the same utilization, but are 20% more expensive, what is the CAPEX for the WSC?

b. [5] <6.4> If those servers also use 15% more power, what is the OPEX of the warehouse-scale computer?

c. [10] <6.2, 6.4> Given the speed improvement and power increase, what must the cost of the new servers be to be comparable to the original cluster? (*Hint:* Based on this TCO model, you may have to change the critical load of the facility.)

6.2 [5/10] <6.4, 6.6, 6.8> To achieve a lower OPEX, one appealing alternative is to use low-power versions of servers to reduce the total electricity required to run the servers; however, similar to high-end servers, low-power versions of high-end components also have nonlinear trade-offs.

a. [5] <6.4, 6.6, 6.8> If low-power server options offered 15% lower power at the same performance but are 20% more expensive, are they a good trade-off?

b. [10] <6.4, 6.6, 6.8> At what cost do the servers become comparable to the original cluster? What if the price of electricity doubles?

6.3 [5/10/15] <6.4, 6.6> Servers that have different operating modes offer opportunities for dynamically running different configurations in the cluster to match workload usage. Use the data in Figure 6.35 for the power/performance modes for a given low-power server.

a. [5] <6.4, 6.6> If a server operator decided to save power costs by running all servers at medium performance, how many servers would be needed to achieve the same level of performance?

b. [10] <6.4, 6.6> What are the CAPEX and OPEX of such a configuration?

Mode	Performance	Power
High	100%	100%
Medium	75%	60%
Low	59%	38%

Figure 6.35 Power–performance modes for low-power servers.

c. [15] <6.4, 6.6> If there was an alternative where you could purchase a server that is 20% cheaper but *x*% slower and uses *y*% less power, find the performance–power curve that provides a TCO comparable to the baseline server.

6.4 [Discussion] <6.4> Discuss the trade-offs and benefits of the two options in Exercise 6.3, assuming a constant workload being run on the servers.

6.5 [Discussion] <6.2, 6.4> Unlike high-performance computing (HPC) clusters, WSCs often experience significant workload fluctuation throughout the day. Discuss the trade-offs and benefits of the two options in Exercise 6.3, this time assuming a workload that varies.

6.6 [Discussion] <6.4, 6.7> The TCO model presented so far abstracts away a significant amount of lower level details. Discuss the impact of these abstractions to the overall accuracy of the TCO model. When are these abstractions safe to make? In what cases would greater detail provide significantly different answers?

Case Study 2: Resource Allocation in WSCs and TCO

Concepts illustrated by this case study

- Server and Power Provisioning within a WSC
- Time Variance of Workloads
- Effects of Variance on TCO

Some of the key challenges to deploying efficient WSCs are provisioning resources properly and utilizing them to their fullest capacity. This problem is complex due to the size of WSCs as well as the potential variance of the workloads being run. The exercises in this case study show how different uses of resources can affect TCO. Assume data from Figures 6.13 and 6.14 as appropriate.

6.7 [5/5/10] <6.4> One of the challenges in provisioning a WSC is determining the proper power load, given the facility size. As described in the chapter, nameplate power is often a peak value that is rarely encountered.

a. [5] <6.4> Estimate how the per-server TCO changes if the nameplate server power is 200 W and the cost is $3000.

b. [5] <6.4> Also consider a higher power, but cheaper server option whose power is 300 W and costs $2000.

c. [10] <6.4> How does the per-server TCO change if the actual average power usage of the servers is only 70% of the nameplate power?

6.8 [15/10] <6.2, 6.4> One assumption in the TCO model is that the critical load of the facility is fixed, and the amount of servers fits that critical load. In reality, due to the variations of server power based on load, the critical power used by a facility can vary at any given time. Operators must initially provision the datacenter based on its critical power resources and an estimate of how much power is used by the datacenter components.

a. [15] <6.2, 6.4> Extend the TCO model to initially provision a WSC based on a server with a nameplate power of 300 W, but also calculate the actual monthly critical power used and TCO assuming the server averages 40% utilization and so consumes only 225 W. How much capacity is left unused?

b. [10] <6.2, 6.4> Repeat this exercise with a 500-W server that averages 20% utilization and consumes only 300 W.

6.9 [10] <6.4, 6.5> WSCs are often used in an interactive manner with end users, as mentioned in Section 6.5. This interactive usage often leads to time-of-day fluctuations, with peaks correlating to specific time periods. For example, for Netflix rentals there is a peak during the evening periods of 8–10 p.m.; the entirety of these time-of-day effects is significant. Compare the per-server TCO of a datacenter with a capacity to match the utilization at 4 a.m. compared to 9 p.m.

6.10 [Discussion/15] <6.4, 6.5> Discuss some options to better utilize the excess servers during the off-peak hours or find ways to save costs. Given the interactive nature of WSCs, what are some of the challenges to aggressively reducing power usage?

6.11 [Discussion/25] <6.4, 6.6, 6.8> Propose one possible way to improve TCO by focusing on reducing server power. What are the challenges to evaluating your proposal? Estimate the TCO improvements based on your proposal. What are some advantages and drawbacks?

Exercises

6.12 [10/10/10] <6.1, 6.2> One of the important enablers of WSC is ample request-level parallelism, in contrast to instruction- or thread-level parallelism. This question explores the implication of different types of parallelism on computer architecture and system design.

a. [10] <6.1> Discuss scenarios where improving the instruction- or thread-level parallelism would provide greater benefits than those achievable through request-level parallelism.

b. [10] <6.1, 6.2> What are the software design implications of increasing request-level parallelism?

c. [10] <6.1, 6.2> What are potential drawbacks of increasing request-level parallelism?

6.13 [Discussion/15/15] <6.2, 6.3> When a cloud computing service provider receives jobs consisting of multiple Virtual Machines (VMs) (e.g., a MapReduce job), many scheduling options exist. The VMs can be scheduled in a round-robin manner to spread across all available processors and servers, or they can be consolidated to use as few processors as possible. Using these scheduling options, if a job with 24 VMs was submitted and 30 processors were available in the cloud (each able to run up to 3 VMs), round-robin would use 24 processors, while consolidated scheduling would use 8 processors. The scheduler can also find available processor cores at different scopes: socket, server, rack, and an array of racks.

a. [Discussion] <6.2, 6.3> Assuming that the submitted jobs are all compute-heavy workloads, possibly with different memory bandwidth requirements, what are the pros and cons of round-robin versus consolidated scheduling in terms of power and cooling costs, performance, and reliability?

b. [15] <6.2, 6.3> Assuming that the submitted jobs are all I/O-heavy workloads, what are the pros and cons of round-robin versus consolidated scheduling, at different scopes?

c. [15] <6.2, 6.3> Assuming that the submitted jobs are network-heavy workloads, what are the pros and cons of round-robin versus consolidated scheduling, at different scopes?

6.14 [15/15/10/10] <6.2, 6.3> MapReduce enables large amounts of parallelism by having data-independent tasks run on multiple nodes, often using commodity hardware; however, there are limits to the level of parallelism. For example, for redundancy MapReduce will write data blocks to multiple nodes, consuming disk and, potentially, network bandwidth. Assume a total dataset size of 300 GB, a network bandwidth of 1 Gb/s, a 10 s/GB map rate, and a 20 s/GB reduce rate. Also assume that 30% of the data must be read from remote nodes, and each output file is written to two other nodes for redundancy. Use Figure 6.6 for all other parameters.

a. [15] <6.2, 6.3> Assume that all nodes are in the same rack. What is the expected runtime with 5 nodes? 10 nodes? 100 nodes? 1000 nodes? Discuss the bottlenecks at each node size.

b. [15] <6.2, 6.3> Assume that there are 40 nodes per rack and that any remote read/write has an equal chance of going to any node. What is the expected runtime at 100 nodes? 1000 nodes?

c. [10] <6.2, 6.3> An important consideration is minimizing data movement as much as possible. Given the significant slowdown of going from local to rack to array accesses, software must be strongly optimized to maximize locality. Assume that there are 40 nodes per rack, and 1000 nodes are used in the MapReduce job. What is the runtime if remote accesses are within the same rack 20% of the time? 50% of the time? 80% of the time?

d. [10] <6.2, 6.3> Given the simple MapReduce program in Section 6.2, discuss some possible optimizations to maximize the locality of the workload.

6.15 [20/20/10/20/20/20] <6.2, 6.3> WSC programmers often use data replication to overcome failures in the software. Hadoop HDFS, for example, employs three-way replication (one local copy, one remote copy in the rack, and one remote copy in a separate rack), but it's worth examining when such replication is needed.

a. [20] <6.2> Let us assume that Hadoop clusters are relatively small, with 10 nodes or less, and with dataset sizes of 10 TB or less. Using the failure frequency data in Figure 6.1, what kind of availability does a 10-node Hadoop cluster have with one-, two-, and three-way replications?

b. [20] <6.2> Assuming the failure data in Figure 6.1 and a 1000-node Hadoop cluster, what kind of availability does it have with one-, two-, and three-way replications? What can you reason about the benefits of replication, at scale?

c. [10] <6.2, 6.3> The relative overhead of replication varies with the amount of data written per local compute hour. Calculate the amount of extra I/O traffic and network traffic (within and across rack) for a 1000-node Hadoop job that sorts 1 PB of data, where the intermediate results for data shuffling are written to the HDFS.

d. [20] <6.2, 6.3> Using Figure 6.6, calculate the time overhead for two- and three-way replications. Using the failure rates shown in Figure 6.1, compare the expected execution times for no replication versus two- and three-way replications.

e. [20] <6.2, 6.3> Now consider a database system applying replication on logs, assuming each transaction on average accesses the hard disk once and generates 1 KB of log data. Calculate the time overhead for two- and three-way replications. What if the transaction is executed in-memory and takes 10 µs?

f. [20] <6.2, 6.3> Now consider a database system with ACID consistency that requires two network round trips for two-phase commitment. What is the time overhead for maintaining consistency as well as replications?

6.16 [15/15/20/Discussion] <6.1, 6.2, 6.8> Although request-level parallelism allows many machines to work on a single problem in parallel, thereby achieving greater overall performance, one of the challenges is how to avoid dividing the problem too finely. If we look at this problem in the context of service level agreements (SLAs), using smaller problem sizes through greater partitioning can require increased effort to achieve the target SLA. Assume an SLA of 95% of queries respond at 0.5 s or faster, and a parallel architecture similar to MapReduce that can launch multiple redundant jobs to achieve the same result. For the following questions, assume the query–response time curve shown in Figure 6.36. The curve shows the latency of response, based on the number of queries per second, for a baseline server as well as a "small" server that uses a slower processor model.

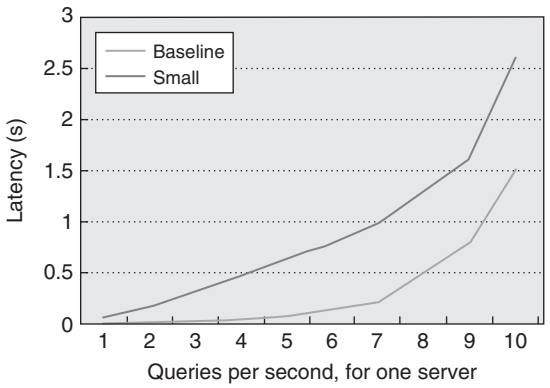


Figure 6.36 Query–response time curve.

a. [15] <6.1, 6.2, 6.8> How many servers are required to achieve this SLA, assuming the query-response time curve shown in Figure 6.36 and the WSC receiving 30,000 queries per second? How many "small" servers are required to achieve this SLA, given this response-time probability curve? Looking only at server costs, how much cheaper must the "small" servers be than the normal servers to achieve a cost advantage for the target SLA?

b. [15] <6.1, 6.2, 6.8> Often, "small" servers are also less reliable due to cheaper components. Using the numbers from Figure 6.1, assume that the number of events due to flaky machines and bad memories increases by 30%. How many "small" servers are required now? How much cheaper must those servers be than the standard servers?

c. [20] <6.1, 6.2, 6.8> Now assume a batch processing environment. The "small" servers provide 30% of the overall performance of the regular servers. Still assuming the reliability numbers from Exercise 6.15 part (b), how many "small" nodes are required to provide the same expected throughput of a 2400-node array of standard servers, assuming perfect linear scaling of performance to node size and an average task length of 10 min per node? What if the scaling is 85%? 60%?

d. [Discussion] <6.1, 6.2, 6.8> Often the scaling is not a linear function, but instead a logarithmic function. A natural response may be instead to purchase larger nodes that have more computational power per node to minimize the array size. Discuss some of the trade-offs with this architecture.

6.17 [10/10/15/Discussion] <6.3, 6.8> One trend in high-end servers is toward the inclusion of nonvolatile flash memory in the memory hierarchy, either through solid-state disks (SSDs) or PCI Express-attached cards. Typical SSDs have a bandwidth of 250 MB/s and latency of 75 μs, whereas PCIe cards have a bandwidth of 600 MB/s and latency of 35 μs.

a. [10] Take Figure 6.7 and include these points in the local server hierarchy. Assuming that identical performance scaling factors like DRAM are accessed at different hierarchy levels, how do these flash memory devices compare when accessed across the rack? Across the array?

b. [10] Discuss some software-based optimizations that can utilize the new level of the memory hierarchy.

c. [15] As discussed in "Fallacies and Pitfalls" (Section 6.8), replacing all disks with SSDs is not necessarily a cost-effective strategy. Consider a WSC operator that uses it to provide cloud services. Discuss some scenarios where using SSDs or other flash memory would make sense.

d. [Discussion] Recently, some vendors have discussed new memory technologies that are much faster than flash. As an example, look up the specifications for Intel 3D X-point memory and discuss how it would factor in Figure 6.7.

6.18 [20/20/Discussion] <6.3> *Memory Hierarchy*: Caching is heavily used in some WSC designs to reduce latency, and there are multiple caching options to satisfy varying access patterns and requirements.

a. [20] Let's consider the design options for streaming rich media from the Web (e.g., Netflix). First we need to estimate the number of videos, number of encode formats per video, and concurrent viewing users. Assume a streaming video provider that has 12,000 titles for online streaming, each title having at least four encode formats (at 500, 1000, 1600, and 2200 kbps). Let's also assume that there are 100,000 concurrent viewers for the entire site, and an average video is 75 min long (accounting for both 30-min shows and 2-h videos). Estimate the total storage capacity, I/O and network bandwidths, and video-streaming-related computation requirements.

b. [20] What are the access patterns and reference locality characteristics per user, per video, and across all videos? (*Hint*: Random versus sequential, good versus poor temporal and spatial locality, relatively small versus large working set size.)

c. [Discussion] What movie storage options exist by using DRAM, SSD, and hard drives? Compare them in performance and TCO. Would new memory technologies like those in Problem 6.17(d) be useful?

6.19 [Discussion/20/Discussion/Discussion] <6.3> Consider a social networking website with 100 million active users posting updates about themselves (in text and pictures) as well as browsing and interacting with updates in their social networks. To provide low latency, Facebook and many other websites use memcached as a caching layer before the backend storage/database tiers. Assume that at any given time the average user is browsing megabytes of content, and on any given day the average user uploads megabytes of content.

a. [20] For the social networking website discussed here, how much DRAM is needed to host its working set? Using servers each having 96 GB DRAM, estimate how many local versus remote memory accesses are needed to generate a user's home page?

b. [Discussion] Now consider two candidate memcached server designs, one using conventional Xeon processors and the other using smaller cores, such as Atom processors. Given that memcached requires large physical memory but has low CPU utilization, what are the pros and cons of these two designs?

c. [Discussion] Today's tight coupling between memory modules and processors often requires an increase in CPU socket count in order to provide large memory support. List other designs to provide large physical memory without proportionally increasing the number of sockets in a server. Compare them based on performance, power, costs, and reliability.

d. [Discussion] The same user's information can be stored in both the memcached and storage servers, and such servers can be physically hosted in different ways. Discuss the pros and cons of the following server layout in the WSC: (1)

memcached collocated on the same storage server, (2) memcached and storage servers on separate nodes in the same rack, or (3) memcached servers on the same racks and storage servers collocated on separate racks.

6.20 [5/5/10/10/Discussion/Discussion/Discussion] <6.3, 6.5, 6.6> *Datacenter Networking*: MapReduce and WSC are a powerful combination to tackle large-scale data processing. For this problem, we will assume we sort one petabyte (1 PB) of records in 6 h using 4000 servers and 48,000 hard drives (Google discussed doing this in 2008).

a. [5] Derive disk bandwidth from Figure 6.6 and associated text. How many seconds does it take to read the data into main memory and write the sorted results back?

b. [5] Assuming each server has two 1 Gb/s Ethernet network interface cards (NICs) and the WSC switch infrastructure is oversubscribed by a factor of 4, how many seconds does it take to shuffle the entire dataset across 4000 servers?

c. [10] Assuming network transfer is the performance bottleneck for petabyte sort, can you estimate what oversubscription ratio Google has in its datacenter?

d. [10] Now let's examine the benefits of having 10 Gb/s Ethernet without oversubscription—for example, using a 48-port 10 Gb/s Ethernet (this was used by the 2010 Indy sort benchmark winner TritonSort). How long does it take to shuffle 1 PB of data?

e. [Discussion] Compare the two approaches here: (1) the massive scale-out approach with high network oversubscription ratio, and (2) a relatively small-scale system with a high-bandwidth network. What are their potential bottlenecks? What are their advantages and disadvantages, in terms of scalability and TCO?

f. [Discussion] Sort and many important scientific computing workloads are communication-heavy, while many other workloads are not. List three example workloads that do not benefit from high-speed networking. What EC2 instances would you recommend to use for these two classes of workloads?

g. [Discussion] Look up the various benchmarks in www.sortbenchmark.org and recent winners in each category. How do these results match the insights from the discussion in part (e) above? How does the cloud instance used for the most recent winner of CloudSort compare with your answer in part (f) above?

6.21 [10/25/Discussion] <6.4, 6.6> Because of the massive scale of WSCs, it is very important to properly allocate network resources based on the workloads that are expected to be run. Different allocations can have significant impacts on both the performance and total cost of ownership.

a. [10] Using the numbers in the spreadsheet detailed in Figure 6.13, what is the oversubscription ratio at each access-layer switch? What is the impact on TCO if the oversubscription ratio is cut in half? What if it is doubled?

b. [25] Reducing the oversubscription ratio can potentially improve performance if a workload is network-limited. Assume a MapReduce job that uses 120

servers and reads 5 TB of data. Assume the same ratio of read/intermediate/output data as in Figure 6.2, Sep-09, and use Figure 6.6 to define the bandwidths of the memory hierarchy. When reading data, assume that 50% of data is read from remote disks; of that, 80% is read from within the rack and 20% is read from within the array. For intermediate data and output data, assume that 30% of the data uses remote disks; of that, 90% is within the rack and 10% is within the array. What is the overall performance improvement when reducing the oversubscription ratio by half? What is the performance if the oversubscription ratio is doubled? Calculate the TCO in each case.

c. [Discussion] We are seeing the trend to more cores per system. We are also seeing the increasing adoption of optical communication (with potentially higher bandwidth and improved energy efficiency). How do you think these and other emerging technology trends will affect the design of future WSCs?

6.22 [5/15/15/20/25/Discussion] <6.5> *Realizing the Capability of the Cloud*: Imagine you are the site operation and infrastructure manager of an Alexa.com top site and are considering using Amazon Web Services (AWS). What factors do you need to consider in determining whether to migrate to AWS? What services and instance types could you use, and how much cost could you save? You can use Alexa and site traffic information (e.g., Wikipedia provides page view stats) to estimate the amount of traffic received by a top site, or you can take concrete examples from the Web, such as the following example: *http://2bits.com/sites/2bits.com/files/drupal-single-server-2.8-million-page-views-a-day.pdf*. The slides describe an Alexa #3400 site that receives 2.8 million page views per day, using a single server. The server has two quad-core Xeon 2.5 GHz processors with 8 GB DRAM and three 15 K RPM SAS hard drives in a RAID1 configuration, and it costs about $400 per month. The site uses caching heavily, and the CPU utilization ranges from 50% to 250% (roughly 0.5–2.5 cores busy).

a. [5] Looking at the available EC2 instances *(http://aws.amazon.com/ec2/instance-types/)*, what instance types match or exceed the current server configuration?

b. [15] Looking at the EC2 pricing information *(http://aws.amazon.com/ec2/pricing/)*, select the most cost-efficient EC2 instances (combinations allowed) to host the site on AWS. What is the monthly cost for EC2?

c. [15] Now add the costs for IP address and network traffic to the equation, and suppose the site transfers 100 GB/day in and out on the Internet. What is the monthly cost for the site now?

d. [20] AWS also offers a micro instance for free for 1 year to new customers and 15 GB bandwidth each for traffic going in and out across AWS. Based on your estimation of peak and average traffic from your department Web server, can you host it for free on AWS?

e. [25] Based on the service characteristics, if a much larger site like Netflix.com migrates its streaming and encoding infrastructure to AWS, what AWS services could be used by Netflix and for what purposes?

f. [Discussion] Look at similar offerings from other cloud providers (Google, Microsoft, Alibaba, etc.). How do the answers to parts (a)–(e) change?

g. [Discussion] "Serverless computing" allows you to build and run higher-level applications and services without thinking about specific servers. Examples include AWS Lambda, Google Cloud Functions, Microsoft Azure Functions, etc. Continuing to wear your site operation and infrastructure manager hat, when would you consider serverless computing?

6.23 [Discussion/Discussion/20/20/Discussion] <6.4, 6.8>Figure 6.12 shows the impact of user perceived response time on revenue, and motivates the need to achieve high-throughput while maintaining low latency.

a. [Discussion] Taking Web search as an example, what are the possible ways of reducing query latency?

b. [Discussion] What monitoring statistics can you collect to help understand where time is spent? How do you plan to implement such a monitoring tool?

c. [20] Assuming that the number of disk accesses per query follows a normal distribution, with an average of 2 and standard deviation of 3, what kind of disk access latency is needed to satisfy a latency SLA of 0.1 s for 95% of the queries?

d. [20] In-memory caching can reduce the frequencies of long-latency events (e.g., accessing hard drives). Assuming a steady-state hit rate of 40%, hit latency of 0.05 s, and miss latency of 0.2 s, does caching help meet a latency SLA of 0.1 s for 95% of the queries?

e. [Discussion] When can cached content become stale or even inconsistent? How often can this happen? How can you detect and invalidate such content?

6.24 [15/15/20/Discussion] <6.4, 6.6>The efficiency of typical power supply units (PSUs) varies as the load changes; for example, PSU efficiency can be about 80% at 40% load (e.g., output 40 W from a 100-W PSU), 75% when the load is between 20% and 40%, and 65% when the load is below 20%.

a. [15] Assume a power-proportional server whose actual power is proportional to CPU utilization, with a utilization curve as shown in Figure 6.3. What is the average PSU efficiency?

b. [15] Suppose the server employs 2 *N* redundancy for PSUs (i.e., doubles the number of PSUs) to ensure stable power when one PSU fails. What is the average PSU efficiency?

c. [20] Blade server vendors use a shared pool of PSUs not only to provide redundancy but also to dynamically match the number of PSUs to the server's actual power consumption. The HP c7000 enclosure uses up to six PSUs for a total of 16 servers. In this case, what is the average PSU efficiency for the enclosure of server with the same utilization curve?

d. [Discussion] Consider the impact of the different efficiency numbers in the context of the broader TCO discussions in Figures 6.13 and 6.14: how do the different design impact the total TCO? Given these, how would you optimize designs for future warehouse-scale computers?

6.25 [5/Discussion/10/15/Discussion/Discussion/Discussion] <6.4, 6.8> *Power stranding* is a term used to refer to power capacity that is provisioned but not used in a datacenter. Consider the data presented in Figure 6.37 [Fan, Weber, and Barroso, 2007] for different groups of machines. (Note that what this paper calls a "cluster" is what we have referred to as an "array" in this chapter.)

a. [5] What is the stranded power at (1) the rack level, (2) the power distribution unit level, and (3) the array (cluster) level? What are the trends with oversubscription of power capacity at larger groups of machines?

b. [Discussion] What do you think causes the differences between power stranding at different groups of machines?

c. [10] Consider an array-level collection of machines where the total machines never use more than 72% of the aggregate power (this is sometimes also referred to as the ratio between the peak-of-sum and sum-of-peaks usage). Using the cost model in the case study, compute the cost savings from comparing a datacenter provisioned for peak capacity and one provisioned for actual use.

d. [15] Assume that the datacenter designer chose to include additional servers at the array level to take advantage of the stranded power. Using the example configuration and assumptions in part (a), compute how many more servers can now be included in the warehouse-scale computer for the same total power provisioning.

e. [Discussion] What is needed to make the optimization of part (d) work in a real-world deployment? (*Hint*: Think about what needs to happen to cap power in the rare case when all the servers in the array are used at peak power.)

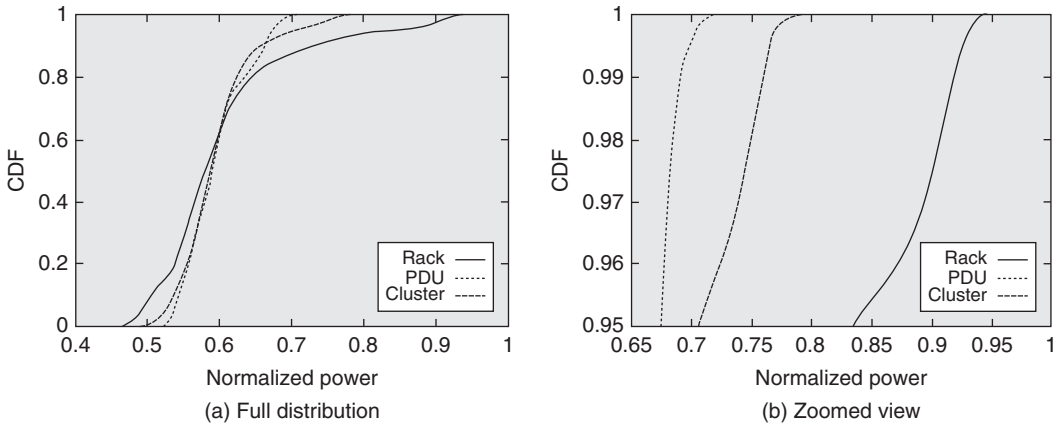


Figure 6.37 Cumulative distribution function (CDF) of a real datacenter.

f. [Discussion] Two kinds of policies can be envisioned to manage power caps [Ranganathan et al., 2006]: (1) preemptive policies where power budgets are predetermined ("don't assume you can use more power; ask before you do!") or (2) reactive policies where power budgets are throttled in the event of a power budget violation ("use as much power as needed until told you can't!"). Discuss the trade-offs between these approaches and when you would use each type.

g. [Discussion] What happens to the total stranded power if systems become more energy-proportional (assume workloads similar to that of Figure 6.4)?

6.26 [5/20/Discussion] <6.4, 6.7> Section 6.7 discussed the use of per-server battery sources in the Google design. Let us examine the consequences of this design.

a. [5] Assume that the use of a battery as a mini-server-level UPS is 99.99% efficient and eliminates the need for a facility-wide UPS that is only 92% efficient. Assume that substation switching is 99.7% efficient and that the efficiency for the PDU, step-down stages, and other electrical breakers are 98%, 98%, and 99%, respectively. Calculate the overall power infrastructure efficiency improvements from using a per-server battery backup.

b. [20] Assume that the UPS is 10% of the cost of the IT equipment. Using the rest of the assumptions from the cost model in the case study, what is the break-even point for the costs of the battery (as a fraction of the cost of a single server) at which the total cost of ownership for a battery-based solution is better than that for a facility-wide UPS?

c. [Discussion] What are the other trade-offs between these two approaches? In particular, how do you think the manageability and failure model will change across these two different designs?

6.27 [5/5/Discussion] <6.4> For this exercise, consider a simplified equation for the total operational power of a WSC as follows:

$$\text{Total operational power} = (1 + \text{Cooling inefficiency multiplier}) * \text{IT equipment power}.$$

a. [5] Assume an 8 MW datacenter at 80% power usage, electricity costs of $0.10 per kilowatt-hour, and a cooling-inefficiency multiplier of 0.8. Compare the cost savings from (1) an optimization that improves cooling efficiency by 20%, and (2) an optimization that improves the energy efficiency of the IT equipment by 20%.

b. [5] What is the percentage improvement in IT equipment energy efficiency needed to match the cost savings from a 20% improvement in cooling efficiency?

c. [Discussion/10] What conclusions can you draw about the relative importance of optimizations that focus on server energy efficiency and cooling energy efficiency?

6.28 [5/5/Discussion] <6.4> As discussed in this chapter, the cooling equipment in WSCs can themselves consume a lot of energy. Cooling costs can be lowered

by proactively managing temperature. Temperature-aware workload placement is one optimization that has been proposed to manage temperature to reduce cooling costs. The idea is to identify the cooling profile of a given room and map the hotter systems to the cooler spots, so that at the WSC level the requirements for overall cooling are reduced.

a. [5] The coefficient of performance (COP) of a computer room air conditioning unit (CRAC) is a measure of its efficiency, and is defined as the ratio of heat removed (Q) to the amount of work necessary (W) to remove that heat. The COP of a CRAC unit increases with the temperature of the air the CRAC unit pushes into the plenum. If air returns to the CRAC unit at 20°C and we remove 10 KW of heat with a COP of 1.9, how much energy do we expend in the CRAC unit? If it takes a COP of 3.1 to cool the same volume of air, but the air is returned at 25°C, how much energy do we now expend in the CRAC unit?

b. [5] Assume a workload distribution algorithm is able to match the hot workloads well with the cool spots to allow the computer room air-conditioning (CRAC) unit to be run at a higher temperature to improve cooling efficiencies like in the exercise above. What is the power savings between the two cases described above?

c. [Discussion] Given the scale of WSC systems, power management can be a complex, multifaceted problem. Optimizations to improve energy efficiency can be implemented in hardware and in software, at the system level, and at the cluster level for the IT equipment or the cooling equipment, etc. It is important to consider these interactions when designing an overall energy-efficiency solution for the WSC. Consider a consolidation algorithm that looks at server utilization and consolidates different workload classes on the same server to increase server utilization (this can potentially have the server operating at a higher energy efficiency if the system is not energy-proportional). How would this optimization interact with a concurrent algorithm that tried to use different power states (see ACPI, Advanced Configuration Power Interface, for some examples)? What other examples can you think of where multiple optimizations can potentially conflict with one another in a WSC? How would you solve this problem?

6.29 [5/10/15/20] <6.2, 6.6> Energy proportionality (sometimes also referred to as energy scale-down) is the attribute of the system to consume no power when idle, but more importantly gradually consume more power *in proportion* to the activity level and work done. In this exercise, we will examine the sensitivity of energy consumption to different energy proportionality models. In the exercises below, unless otherwise mentioned, use the data in Figure 6.4 as the default.

a. [5] A simple way to reason about energy proportionality is to assume linearity between activity and power usage. Using just the peak power and idle power data from Figure 6.4 and a linear interpolation, plot the energy-efficiency trends across varying utilizations. (Energy efficiency is expressed as performance per watt.) What happens if idle power (at 0% activity) is half of what is assumed in Figure 6.4? What happens if idle power is zero?

b. [10] Plot the energy-efficiency trends across varying activities, but use the data from column 3 of Figure 6.4 for power variation. Plot the energy efficiency assuming that the idle power (alone) is half of what is assumed in Figure 6.4. Compare these plots with the linear model in the previous exercise. What conclusions can you draw about the consequences of focusing purely on idle power alone?

c. [15] Assume the system utilization mix in column 7 of Figure 6.4. For simplicity, assume a discrete distribution across 1000 servers, with 109 servers at 0% utilization, 80 servers at 10% utilization, etc. Compute the total performance and total energy for this workload mix using the assumptions in part (a) and part (b).

d. [20] One could potentially design a system that has a sublinear power versus load relationship in the region of load levels between 0% and 50%. This would have an energy-efficiency curve that peaks at lower utilizations (at the expense of higher utilizations). Create a new version of column 3 from Figure 6.4 that shows such an energy-efficiency curve. Assume the system utilization mix in column 7 of Figure 6.4. For simplicity, assume a discrete distribution across 1000 servers, with 109 servers at 0% utilization, 80 servers at 10% utilizations, etc. Compute the total performance and total energy for this workload mix.

6.30 [15/20/20] <6.2, 6.6> This exercise illustrates the interactions of energy proportionality models with optimizations such as server consolidation and energy-efficient server designs. Consider the scenarios shown in Figures 6.38 and 6.39.

a. [15] Consider two servers with the power distributions shown in Figure 6.38: case A (the server considered in Figure 6.4) and case B (a less energy-proportional but more energy-efficient server than case A). Assume the system utilization mix in column 7 of Figure 6.4. For simplicity, assume a discrete distribution across 1000 servers, with 109 servers at 0% utilization, 80 servers at 10% utilizations, etc., as

Activity (%)	0	10	20	30	40	50	60	70	80	90	100
Power, case A (W)	181	308	351	382	416	451	490	533	576	617	662
Power, case B (W)	250	275	325	340	395	405	415	425	440	445	450

Figure 6.38 Power distribution for two servers.

Activity (%)	0	10	20	30	40	50	60	70	80	90	100
No. servers, case A and B	109	80	153	246	191	115	51	21	15	12	8
No. servers, case C	504	6	8	11	26	57	95	123	76	40	54

Figure 6.39 Utilization distributions across cluster, without and with consolidation.

shown in row 1 of Figure 6.39. Assume performance variation based on column 2 of Figure 6.4. Compare the total performance and total energy for this workload mix for the two server types.

b. [20] Consider a cluster of 1000 servers with data similar to the data shown in Figure 6.4 (and summarized in the first rows of Figures 6.38 and 6.39). What are the total performance and total energy for the workload mix with these assumptions? Now assume that we were able to consolidate the workloads to model the distribution shown in case C (second row of Figure 6.39). What are the total performance and total energy now? How does the total energy compare with a system that has a linear energy-proportional model with idle power of zero watts and peak power of 662 W?

c. [20] Repeat part (b), but with the power model of server B, and compare with the results of part (a).

6.31 [10/Discussion] <6.2, 6.4, 6.6> *System-Level Energy Proportionality Trends*: Consider the following breakdowns of the power consumption of a server:

CPU, 50%; memory, 23%; disks, 11%; networking/other, 16%
CPU, 33%; memory, 30%; disks, 10%; networking/other, 27%

a. [10] Assume a dynamic power range of 3.0× for the CPU (i.e., the power consumption of the CPU at idle is one-third that of its power consumption at peak). Assume that the dynamic range of the memory systems, disks, and the networking/other categories above are, respectively, 2.0×, 1.3×, and 1.2×. What is the overall dynamic range for the total system for the two cases?

b. [Discussion/10] What can you learn from the results of part (a)? How would we achieve better energy proportionality at the system level? (*Hint*: Energy proportionality at a system level cannot be achieved through CPU optimizations alone, but instead requires improvement across all components.)

6.32 [30] <6.4> Pitt Turner IV et al. [2008] presented a good overview of datacenter tier classifications. Tier classifications define site infrastructure performance. For simplicity, consider the key differences as shown in Figure 6.40 (adapted from Pitt Turner IV et al. [2008]). Using the TCO model in the case study as a guiding framework, compare the cost implications of the different tiers shown.

6.33 [Discussion] <6.4> Based on the observations in Figures 6.12 and 6.13, what can you say qualitatively about the trade-offs between revenue loss from downtime and costs incurred for uptime?

6.34 [15/Discussion] <6.4> Some recent studies have defined a metric called TPUE, which stands for "true PUE" or "total PUE." TPUE is defined as PUE * SPUE. PUE, the power utilization effectiveness, is defined in Section 6.4 as the ratio of the total facility power over the total IT equipment power. SPUE, or server PUE, is a new metric analogous to PUE, but instead applied to computing equipment. SPUE is defined as the ratio of total server input power to its useful power, where useful power is defined as the power consumed by the electronic components

Tier 1	Single path for power and cooling distributions, without redundant components	99.0%
Tier 2	(N + 1) redundancy = two power and cooling distribution paths	99.7%
Tier 3	(N + 2) redundancy = three power and cooling distribution paths for uptime even during maintenance	99.98%
Tier 4	Two active power and cooling distribution paths, with redundant components in each path, to tolerate any single equipment failure without impacting the load	99.995%

Figure 6.40 Overview of data center tier classifications. (Adapted from Pitt Turner IV et al. [2008].).

directly involved in the computation: motherboard, disks, CPUs, DRAM, I/O cards, and so on. In other words, the SPUE metric captures inefficiencies associated with the power supplies, voltage regulators, and fans housed on a server.

a. [15] <6.4> Consider a design that uses a higher supply temperature for the computer room air conditioning (CRAC) units. The efficiency of the CRAC unit is approximately a quadratic function of the temperature, and this design therefore improves the overall PUE, let's assume by 7%. (Assume baseline PUE of 1.7.) However, the higher temperature at the server level triggers the on-board fan controller to operate the fan at much higher speeds. The fan power is a cubic function of speed, and the increased fan speed leads to a degradation of SPUE. Assume a fan power model:

$$\text{Fan power} = 284*ns*ns*ns - 75*ns*ns,$$

where *ns* is the normalized fan speed = fan speed in rpm/18,000 and a baseline server power of 350 W. Compute the SPUE if the fan speed increases from (1) 10,000–12,500 rpm and (2) 10,000–18,000 rpm. Compare the PUE and TPUE in both these cases. (For simplicity, ignore the inefficiencies with power delivery in the SPUE model.)

b. [Discussion] Part (a) illustrates that, while PUE is an excellent metric to capture the overhead of the facility, it does not capture the inefficiencies within the IT equipment itself. Can you identify another design where changes to the TPUE are potentially lower than the changes to traditional PUE? (*Hint*: See Exercise 6.26.)

6.35 [Discussion/30/Discussion] <6.2> Two benchmarks provide a good starting point for energy-efficiency accounting in servers—the SPECpower_ssj2008 benchmark (available at *http://www.spec.org/power_ssj2008/*) and the JouleSort metric (available at *http://sortbenchmark.org/*).

a. [Discussion] <6.2> Look up the descriptions of the two benchmarks. How are they similar? How are they different? What would you do to improve these benchmarks to better address the goal of improving WSC energy efficiency?

b. [30] <6.2> JouleSort measures the total system energy to perform an out-of-core sort and attempts to derive a metric that enables the comparison of systems ranging from embedded devices to supercomputers. Look up the description of the JouleSort metric at *http://sortbenchmark.org*. Download a publicly available version of the sort algorithm and run it on different classes of machines—a laptop, a PC, a mobile phone, etc.—or with different configurations. What can you learn from the JouleSort ratings for different setups?

c. [Discussion] <6.2> Consider the system with the best JouleSort rating from your experiments above. How would you improve the energy efficiency? For example, try rewriting the sort code to improve the JouleSort rating. What does running sort in the cloud do to energy efficiency?

6.36 [10/10/15] <6.1, 6.2> Figure 6.1 is a listing of outages in an array of servers. When dealing with the large scale of WSCs, it is important to balance cluster design and software architectures to achieve the required uptime without incurring significant costs. This question explores the implications of achieving availability through hardware only.

a. [10] <6.1, 6.2> Assuming that an operator wishes to achieve 95% availability through server hardware improvements alone, how many events of each type would have to be reduced? For now, assume that individual server crashes are completely handled through redundant machines.

b. [10] <6.1, 6.2> How does the answer to part (a) change if the individual server crashes are handled by redundancy 50% of the time? 20% of the time? None of the time?

c. [15] <6.1, 6.2> Discuss the importance of software redundancy to achieving a high level of availability. If a WSC operator considered buying machines that were cheaper but 10% less reliable, what implications would this have on the software architecture? What are the challenges associated with software redundancy?

d. [Discussion] <6.1> Discuss the importance of eventual consistency in how warehouse-scale computers can scale.

6.37 [15] <6.1, 6.8> Look up the current prices of standard DDR4 DRAM versus DDR4 DRAM that has error-correcting code (ECC). What is the increase in price per bit for achieving the higher reliability that ECC provides? Using the DRAM prices alone, and the data provided in Section 6.8, what is the uptime per dollar of a WSC with non-ECC versus ECC DRAM?

6.38 [5/Discussion] <6.1, 6.8> *WSC Reliability and Manageability Concerns*:

a. [5] Consider a cluster of servers costing $2000 each. Assuming an annual failure rate of 5%, an average of an hour of service time per repair, and replacement parts requiring 10% of the system cost per failure, what is the annual maintenance cost per server? Assume an hourly rate of $100 per hour for a service technician.

b. [Discussion] Comment on the differences between this manageability model versus that in a traditional enterprise datacenter with a large number of small- or medium-sized applications each running on its own dedicated hardware infrastructure.

c. [Discussion] Discuss the trade-offs in having heterogeneous machines in a warehouse-scale computer.

6.39 [Discussion] <6.4, 6.7, 6.8> The OpenCompute project at www.opencompute.org provides a community to design and share efficient designs for warehouse-scale computers. Look at some of the recently proposed designs. How do they compare with the design trade-offs discussed in this chapter? How do the designs differ from the Google case study discussed in Section 6.7?

6.40 [15/15] <6.3, 6.4, 6.5> Assume that the MapReduce job from Page #438 in Section 6.2 is executing a task with 2^40 bytes of input data, 2^37 bytes of intermediate data, and 2^30 bytes of output data. This job is entirely memory/storage bound, so its performance can be quantified by the DRAM/Disk bandwidth of Figure 6.6.

a. How much does the job cost to run on m4.16xlarge and m4.large in Figure 6.15? Which EC2 instance provides better performance? Which EC2 instance provides better cost?

b. How much would the job cost if an SSD was added to the system, as in m3.medium? How do the performance and cost of m3.medium compare with the best instance from part (a) above?

6.41 <6.1, 6.4> [5/5/10/Discussion] Imagine you have created a web service that runs very well (responds within 100 ms latency) 99% of the time, and has performance issues 1% of the time (maybe the CPU went into a lower power state and the response took 1000 ms, etc.).

a. [5] Your service grows popular, and you now have 100 servers and your computation has to touch all these servers to handle the user request. What is the percentage of time your query is likely to have a slow response time, across 100 servers?

b. [5] Instead of "two nines" (99%) single server latency SLA, how many "nines" do we need to have for the single server latency SLA so that the cluster latency SLA has bad latencies only 10% of the time or lower?

c. [10] How do the answers to parts (a) and (b) change if we have 2000 servers?

d. [Discussion] Section 6.4 (page 452) discusses "tail-tolerant" designs. What kind of design optimizations would you need to make in your web service (*Hint*: Look at the "Tail at Scale" paper from Dean and Barroso [2013]).

7.1	Introduction	540
7.2	Guidelines for DSAs	543
7.3	Example Domain: Deep Neural Networks	544
7.4	Google's Tensor Processing Unit, an Inference Data Center Accelerator	557
7.5	Microsoft Catapult, a Flexible Data Center Accelerator	567
7.6	Intel Crest, a Data Center Accelerator for Training	579
7.7	Pixel Visual Core, a Personal Mobile Device Image Processing Unit	579
7.8	Cross-Cutting Issues	592
7.9	Putting It All Together: CPUs Versus GPUs Versus DNN Accelerators	595
7.10	Fallacies and Pitfalls	602
7.11	Concluding Remarks	604
7.12	Historical Perspectives and References	606
	Case Studies and Exercises by Cliff Young	606

7

Domain-Specific Architectures

Moore's Law can't continue forever … We have another 10 to 20 years before we reach a fundamental limit

Gordon Moore,
Intel Co-Founder (2005)

Computer Architecture. https://doi.org/10.1016/B978-0-12-811905-1.00007-9
© 2019 Elsevier Inc. All rights reserved.

7.1 Introduction

Gordon Moore not only predicted the amazing growth of transistors per chip in 1965, but the opening chapter quote shows that he also predicted its demise 50 years later. As evidence, Figure 7.1 shows that even the company he founded—which for decades proudly used Moore's Law as a guideline for capital investment—is slowing its development of new semiconductor processes.

During the semiconductor boom time, architects rode Moore's Law to create novel mechanisms that could turn the cornucopia of transistors into higher performance. The resources for a five-stage pipeline, 32-bit RISC processor—which needed as little as 25,000 transistors in the 1980s—grew by a factor of 100,000 to enable features that accelerated general-purpose code on general-purpose processors, as earlier chapters document:

- 1st-level, 2nd-level, 3rd-level, and even 4th-level caches
- 512-bit SIMD floating-point units
- 15+ stage pipelines
- Branch prediction
- Out-of-order execution
- Speculative prefetching
- Multithreading
- Multiprocessing

These sophisticated architectures targeted million-line programs written in efficient languages like C++. Architects treated such code as black boxes, generally

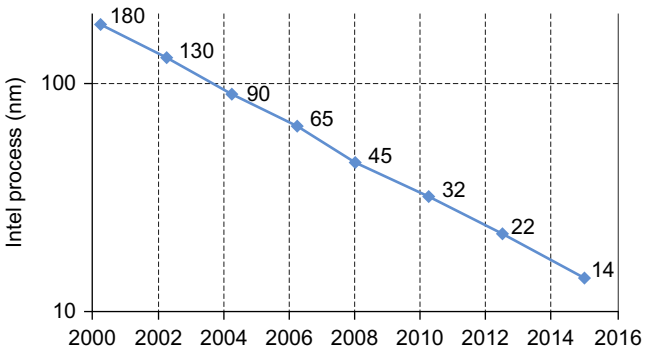


Figure 7.1 Time before new Intel semiconductor process technology measured in nm. The *y*-axis is log scale. Note that the time stretched previously from about 24 months per new process step to about 30 months since 2010.

without understanding either the internal structure of the programs or even what they were trying to do. Benchmark programs like those in SPEC2017 were just artifacts to measure and accelerate. Compiler writers were the people at the hardware-software interface, which dates back to the RISC revolution in the 1980s, but they have limited understanding of the high-level application behavior; that's why compilers cannot even bridge the semantic gap between C or C++ and the architecture of GPUs.

As Chapter 1 described, Dennard scaling ended much earlier than Moore's Law. Thus more transistors switching now means more power. The energy budget is not increasing, and we've already replaced the single inefficient processor with multiple efficient cores. Hence, we have nothing left up our sleeves to continue major improvements in cost-performance and energy efficiency for general-purpose architectures. Because the energy budget is limited (because of electromigration, mechanical and thermal limits of chips), if we want higher performance (higher operations/second), we need to lower the energy per operation.

Figure 7.2 is another take on the relative energy costs of memory and logic mentioned in Chapter 1, this time calculated as overhead for an arithmetic instruction. Given this overhead, minor twists to existing cores may get us 10% improvements, but if we want order-of-magnitude improvements while offering programmability, we need to increase the number of arithmetic operations per instruction from one to hundreds. To achieve that level of efficiency, we need a drastic change in computer architecture from general-purpose cores to *domain-specific architectures (DSAs).*

Thus, just as the field switched from uniprocessors to multiprocessors in the past decade out of necessity, desperation is the reason architects are now working on DSAs. The new normal is that a computer will consist of standard processors to run conventional large programs such as operating systems along with domain-specific processors that do only a narrow range of tasks, but they do them extremely well. Thus such computers will be much more heterogeneous than the homogeneous multicore chips of the past.

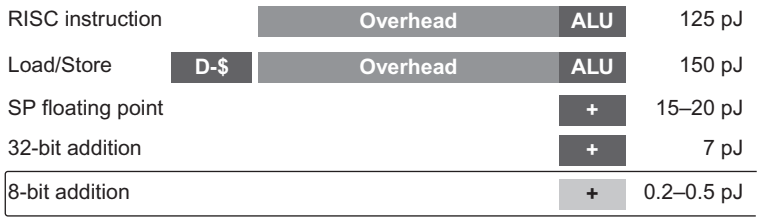


Figure 7.2 Energy costs in picoJoules for a 90 nm process to fetch instructions or access a data cache compared to the energy cost of arithmetic operations (Qadeer et al., 2015).

Part of the argument is that the preceding architecture innovations from the past few decades that leveraged Moore's Law (caches, out-of-order execution, etc.) may not be a good match to some domains—especially in terms of energy usage—so their resources can be recycled to make the chip a better match to the domain. For example, caches are excellent for general-purpose architectures, but not necessarily for DSAs; for applications with easily predictable memory access patterns or huge data sets like video that have little data reuse, multilevel caches are overkill, hording area and energy that could be put to better use. Therefore the promise of DSAs is both improved silicon efficiency and better energy efficiency, with the latter typically being the more important attribute today.

Architects probably won't create a DSA for a large C++ program like a compiler as found in the SPEC2017 benchmark. Domain-specific algorithms are almost always for small compute-intensive kernels of larger systems, such as for object recognition or speech understanding. DSAs should focus on the subset and not plan to run the entire program. In addition, changing the code of the benchmark is no longer breaking the rules; it is a perfectly valid source of speedup for DSAs. Consequently, if they are going to make useful contributions, architects interested in DSA must now shed their blinders and learn application domains and algorithms.

In addition to needing to expand their areas of expertise, a challenge for domain-specific architects is to find a target whose demand is large enough to justify allocating dedicated silicon on an SOC or even a custom chip. The *nonrecurring engineering (NRE)* costs of a custom chip and supporting software are amortized over the number of chips manufactured, so it is unlikely to make economic sense if you need only 1000 chips.

One way to accommodate smaller volume applications is to use reconfigurable chips such as FPGAs because they have lower NRE than custom chips *and* because several different applications may be able to reuse the same reconfigurable hardware to amortize its costs (see Section 7.5). However, since the hardware is less efficient than custom chips, the gains from FPGAs are more modest.

Another DSA challenge is how to port software to it. Familiar programming environments like the C++ programming language and compiler are rarely the right vehicles for a DSA.

The rest of this chapter provides five guidelines for the design of DSAs and then a tutorial on our example domain, which is *deep neural networks* (*DNNs*). We chose DNNs because they are revolutionizing many areas of computing today. Unlike some hardware targets, DNNs are applicable to a wide range of problems, so we can reuse a DNN-specific architecture for solutions in speech, vision, language, translation, search ranking, and many more areas.

We follow with four examples of DSAs: two custom chips for the data center that accelerate DNNs, an FPGA for the data center that accelerates many domains, and an image-processing unit designed for *personal mobile devices* (*PMDs*). We then compare the cost-performance of the DSAs along with CPUs and GPUs using DNN benchmarks, and conclude with a prediction of an upcoming renaissance for computer architecture.

7.2 Guidelines for DSAs

Here are five principles that generally guided the designs of the four DSAs we'll see in Sections 7.4–7.7. Not only do these five guidelines lead to increased area and energy efficiency, they also provide two valuable bonus effects. First, they lead to simpler designs, which reduce the cost of NRE of DSAs (see the fallacy in Section 7.10). Second, for user-facing applications that are commonplace with DSAs, accelerators that follow these principles are a better match to the 99th-percentile response-time deadlines than the time-varying performance optimizations of traditional processors, as we will see in Section 7.9. Figure 7.3 shows how the four DSAs followed these guidelines.

1. *Use dedicated memories to minimize the distance over which data is moved.* The many levels of caches in general-purpose microprocessors use a great deal of area and energy trying to move data optimally for a program. For example, a two-way set associative cache uses 2.5 times as much energy as an equivalent software-controlled scratchpad memory. By definition, the compiler writers and programmers of DSAs understand their domain, so there is no need for the hardware to try to move data for them. Instead, data movement is reduced with software-controlled memories that are dedicated to and tailored for specific functions within the domain.
2. *Invest the resources saved from dropping advanced microarchitectural optimizations into more arithmetic units or bigger memories.* As Section 7.1 describes, architects turned the bounty from Moore's Law into the resource-intensive optimizations for CPUs and GPUs (out-of-order execution, multithreading, multiprocessing, prefetching, address coalescing, etc.).

Guideline	TPU	Catapult	Crest	Pixel Visual Core
Design target	Data center ASIC	Data center FPGA	Data center ASIC	PMD ASIC/SOC IP
1. Dedicated memories	24 MiB Unified Buffer, 4 MiB Accumulators	Varies	N.A.	Per core: 128 KiB line buffer, 64 KiB P.E. memory
2. Larger arithmetic unit	65,536 Multiply-accumulators	Varies	N.A.	Per core: 256 Multiply-accumulators (512 ALUs)
3. Easy parallelism	Single-threaded, SIMD, in-order	SIMD, MISD	N.A.	MPMD, SIMD, VLIW
4. Smaller data size	8-Bit, 16-bit integer	8-Bit, 16-bit integer 32-bit Fl. Pt.	21-bit Fl. Pt.	8-bit, 16-bit, 32-bit integer
5. Domain-specific lang.	TensorFlow	Verilog	TensorFlow	Halide/TensorFlow

Figure 7.3 The four DSAs in this chapter and how closely they followed the five guidelines. Pixel Visual Core typically has 2–16 cores. The first implementation of Pixel Visual Core does not support 8-bit arithmetic.

Given the superior understanding of the execution of programs in these narrower domains, these resources are much better spent on more processing units or larger on-chip memory.

3. *Use the easiest form of parallelism that matches the domain.*
   Target domains for DSAs almost always have inherent parallelism. The key decisions for a DSA are how to take advantage of that parallelism and how to expose it to the software. Design the DSA around the natural granularity of the parallelism of the domain and expose that parallelism simply in the programming model. For example, with respect to data-level parallelism, if SIMD works in the domain, it's certainly easier for the programmer and the compiler writer than MIMD. Similarly, if VLIW can express the instruction-level parallelism for the domain, the design can be smaller and more energy-efficient than out-of-order execution.
4. *Reduce data size and type to the simplest needed for the domain.*
   As we will see, applications in many domains are typically memory-bound, so you can increase the effective memory bandwidth and on-chip memory utilization by using narrower data types. Narrower and simpler data also let's you pack more arithmetic units into the same chip area.
5. *Use a domain-specific programming language to port code to the DSA.*
   As Section 7.1 mentions, a classic challenge for DSAs is getting applications to run on your novel architecture. A long-standing fallacy is assuming that your new computer is so attractive that programmers will rewrite their code just for your hardware. Fortunately, domain-specific programming languages were becoming popular even before architects were forced to switch their attention to DSAs. Examples are Halide for vision processing and TensorFlow for DNNs (Ragan-Kelley et al., 2013; Abadi et al., 2016). Such languages make porting applications to your DSA much more feasible. As previously mentioned, only a small, compute-intensive portion of the application needs to run on the DSA in some domains, which also simplifies porting.

DSAs introduce many new terms, mostly from the new domains but also from novel architecture mechanisms not seen in conventional processors. As we did in Chapter 4, Figure 7.4 lists the new acronyms, terms, and short explanations to aid the reader.

7.3 Example Domain: Deep Neural Networks

Artificial intelligence (AI) is not only the next big wave in computing—it's the next major turning point in human history… the Intelligence Revolution will be driven by data, neural networks and computing power. Intel is committed to AI [thus]… we've added a set of leading-edge accelerants required for the growth and widespread adoption of AI.

Brian Krzanich,
Intel CEO (2016)

Area	Term	Acronym	Short explanation
General	Domain-specific architectures	DSA	A special-purpose processor designed for a particular domain. It relies on other processors to handle processing outside that domain
	Intellectual property block	IP	A portable design block that can be integrated into an SOC. They enable a marketplace where organizations offer IP blocks to others who compose them into SOCs
	System on a chip	SOC	A chip that integrates all the components of a computer; commonly found in PMDs
Deep neural networks	Activation	—	Result of "activating" the artificial neuron; the output of the nonlinear functions
	Batch	—	A collection of datasets processed together to lower the cost of fetching weights
	Convolutional neural network	CNN	A DNN that takes as inputs a set of nonlinear functions of spatially nearby regions of outputs from the prior layer, which are multiplied by the weights
	Deep neural network	DNN	A sequence of layers that are collections of artificial neurons, which consist of a nonlinear function applied to products of weights times the outputs of the prior layer
	Inference	—	The production phase of DNNs; also called *prediction*
	Long short-term memory	LSTM	An RNN well suited to classify, process, and predict time series. It is a hierarchical design consisting of modules called *cells*
	MultiLayer perceptron	MLP	A DNN that takes as inputs a set of nonlinear functions of all outputs from the prior layer multiplied by the weights. These layers are called *fully connected*
	Rectified linear unit	*ReLU*	A nonlinear function that performs $f(x) = \max(x, 0)$. Other popular nonlinear functions are sigmoid and hyperbolic tangent (tanh)
	Recurrent neural network	RNN	A DNN whose inputs are from the prior layer *and* the previous state
	Training	—	The development phase of DNNs; also called *learning*
	Weights	—	The values learned during training that are applied to inputs; also called *parameters*
TPU	Accumulators	—	The 4096 256×32-bit registers (4 MiB) that collect the output of the MMU and are input to the Activation Unit
	Activation unit	—	Performs the nonlinear functions (ReLU, sigmoid, hyperbolic tangent, max pool, and average pool). Its input comes from the Accumulators and its output goes to the Unified Buffer
	Matrix multiply unit	MMU	A systolic array of 256×256 8-bit arithmetic units that perform multiply-add. Its inputs are the Weight Memory and the Unified Buffer, and its output is the Accumulators
	Systolic array	—	An array of processing units that in lockstep input data from upstream neighbors, compute partial results, and pass some inputs and results to downstream neighbors
	Unified buffer	UB	A 24 MiB on-chip memory that holds the activations. It was sized to try to avoid spilling activations to DRAM when running a DNN
	Weight memory	—	An 8 MiB external DRAM chip containing the weights for the MMU. Weights are transferred to a *Weight FIFO* before entering the MMU

Figure 7.4 A handy guide to DSA terms used in Sections 7.3–7.6. Figure 7.29 on page 472 has a guide for Section 7.7.

Artificial intelligence (AI) has made a dramatic comeback since the turn of the century. Instead of *building* artificial intelligence as a large set of logical rules, the focus switched to *machine learning* from example data as the path to artificial intelligence. The amount of data needed to learn was much greater than thought. The warehouse scale computers (WSCs) of this century, which harvest and store petabytes of information found on the Internet from the billions of users and their smartphones, supply the ample data. We also underestimated the amount of computation needed to learn from the massive data, but GPUs—which have excellent single-precision floating-point cost-performance—embedded in the thousands of servers of WSCs deliver sufficient computing.

One part of machine learning, called DNNs, has been the AI star for the past five years. Example DNN breakthroughs are in language translation, which DNNs improved more in a single leap than all the advances from the prior decade (Tung, 2016; Lewis-Kraus, 2016); the switch to DNNs in the past five years reduced the error rate in an image recognition competition from 26% to 3.5% (Krizhevsky et al., 2012; Szegedy et al., 2015; He et al., 2016); and in 2016, DNNs enabled a computer program for the first time to beat a human champion at Go (Silver et al., 2016). Although many of these run in the cloud, they have also enabled Google Translate on smartphones, which we described in Chapter 1. In 2017 new, significant DNN results appear nearly every week.

Readers interested in learning more about DNNs than found in this section should download and try the tutorials in TensorFlow (TensorFlow Tutorials, 2016), or for the less adventurous, consult a free online textbook on DNNs (Nielsen, 2016).

The Neurons of DNNs

DNNs were inspired by the neuron of the brain. The artificial neuron used for neural networks simply computes the sum over a set of products of *weights* or *parameters* and data values that is then put through a nonlinear function to determine its output. As we will see, each artificial neuron has a large fan-in and a large fan-out.

For an image-processing DNN, the input data would be the pixels of a photo, with the pixel values multiplied by the weights. Although many nonlinear functions have been tried, a popular one today is simply `f(x)=max(x,0)`, which returns 0 if the `x` is negative or the original value if positive or zero. (This simple function goes by the complicated name *rectified linear unit* or *ReLU*.) The output of a nonlinear function is called an *activation*, in that it is the output of the artificial neuron that has been "activated."

A cluster of artificial neurons might process different portions of the input, and the output of that cluster becomes the input to the next layer of artificial neurons. The layers between the input layer and the output layer are called *hidden layers*. For image processing, you can think of each layer as looking for different types of features, going from lower-level ones like edges and angles to higher-level ones like eyes and ears. If the image-processing application was trying to decide if the image

Name	DNN layers	Weights	Operations/Weight
MLP0	5	20M	200
MLP1	4	5M	168
LSTM0	58	52M	64
LSTM1	56	34M	96
CNN0	16	8M	2888
CNN1	89	100M	1750

Figure 7.5 Six DNN applications that represent 95% of DNN workloads for inference at Google in 2016, which we use in Section 7.9. The columns are the DNN name, the number of layers in the DNN, the number of weights, and operations per weight (operational intensity). Figure 7.41 on page 595 goes into more detail on these DNNs.

contained a dog, the output of the last layer could be a probability number between 0 and 1 or perhaps a list of probabilities corresponding to a list of dog breeds.

The number of layers gave DNNs their name. The original lack of data and computing horsepower kept most neural networks relatively shallow. Figure 7.5 shows the number of layers for a variety of recent DNNs, the number of weights, and the number of operations per weight fetched. In 2017 some DNNs have 150 layers.

Training Versus Inference

The preceding discussion concerns DNNs that are in production. DNN development starts by defining the neural network architecture, picking the number and type of layers, the dimensions of each layer, and the size of the data. Although experts may develop new neural network architectures, most practitioners will choose among the many existing designs (e.g., Figure 7.5) that have been shown to perform well on problems similar to theirs.

Once the neural architecture has been selected, the next step is to learn the weights associated with each edge in the neural network graph. The weights determine the behavior of the model. Depending on the choice of neural architecture, there can be anywhere from thousands to hundreds of millions of weights in a single model (see Figure 7.5). Training is the costly process of tuning these weights so that the DNN approximates the complex function (e.g., mapping from pictures to the objects in that picture) described by the training data.

This development phase is universally called *training* or *learning*, whereas the production phase has many names: *inference*, *prediction*, *scoring*, *implementation*, *evaluation*, *running*, or *testing*. Most DNNs use *supervised learning* in that they are given a training set to learn from where the data is preprocessed in order to have the correct labels. Thus, in the ImageNet DNN competition (Russakovsky et al., 2015), the training set consists of 1.2 million photos, and each photo has been labeled as one of 1000 categories. Several of these categories

are quite detailed, such as specific breeds of dogs and cats. The winner is determined by evaluating a separate secret set of 50,000 photos to see which DNN has the lowest error rate.

Setting the weights is an iterative process that goes *backward* through the neural network using the training set. This process is called *backpropagation*. For example, because you know the breed of a dog image in the training set, you see what your DNN says about the image, and then you adjust the weights to improve the answer. Amazingly, the weights at the start of the training process should be set to random data, and you just keep iterating until you're satisfied with the DNN accuracy using the training set.

For the mathematically inclined, the goal of learning is to find a function that maps the inputs to the correct outputs over the multilayer neural network architecture. Backpropagation stands for "back propagation of errors." It calculates a gradient over all the weights as input to an optimization algorithm that tries to minimize the errors by updating the weights. The most popular optimization algorithm for DNNs is *stochastic gradient descent*. It adjusts the weights proportionally to maximize the descent of the gradient obtained from backpropagation. Readers interested in learning more should see Nielsen (2016) or TensorFlow Tutorials (2016).

Training can take weeks of computation, as Figure 7.6 shows. The inference phase is often below 100 ms per data sample, which is a million times less. Although training takes much longer than a single inference, the total compute time for inference is a product of the number of customers of the DNN and how frequently they invoke it.

After training, you deploy your DNN, hoping that your training set is representative of the real world, and that your DNN will be so popular that your users will spend much more time employing it than you've put into developing it!

Type of data	Problem area	Size of benchmark's training set	DNN architecture	Hardware	Training time
text [1]	Word prediction (word2vec)	100 billion words (Wikipedia)	2-layer skip gram	1 NVIDIA Titan X GPU	6.2 hours
audio [2]	Speech recognition	2000 hours (Fisher Corpus)	11-layer RNN	1 NVIDIA K1200 GPU	3.5 days
images [3]	Image classification	1 million images (ImageNet)	22-layer CNN	1 NVIDIA K20 GPU	3 weeks
video [4]	activity recognition	1 million videos (Sports-1M)	8-layer CNN	10 NVIDIA GPUs	1 month

Figure 7.6 Training set sizes and training time for several DNNs (Iandola, 2016).

There are tasks that don't have training datasets, such as when trying to predict the future of some real-world event. Although we won't cover it here, *reinforcement learning* (*RL*) is a popular algorithm for such learning in 2017. Instead of a training set to learn from, RL acts on the real world and then gets a signal from a reward function, depending on whether that action made the situation better or worse.

Although it's hard to imagine a faster changing field, only three types of DNNs reign as most popular in 2017: *MultiLayer Perceptrons* (*MLPs*), *Convolutional Neural Networks* (*CNNs*), and *Recurrent Neural Networks* (*RNNs*). They are all examples of supervised learning, which rely on training sets.

Multilayer Perceptron

MLPs were the original DNNs. Each new layer is a set of nonlinear functions F of weighted sum of all outputs from a prior one $y_n = F(W \times y_{n-1})$. The weighted sum consists of a vector-matrix multiply of the outputs with the weights (see Figure 7.7). Such a layer is called *fully connected* because each output neuron result depends on *all* input neurons of the prior layer.

We can calculate the number of neurons, operations, and weights per layer for each of the DNN types. The easiest is MLP because it is just a vector-matrix

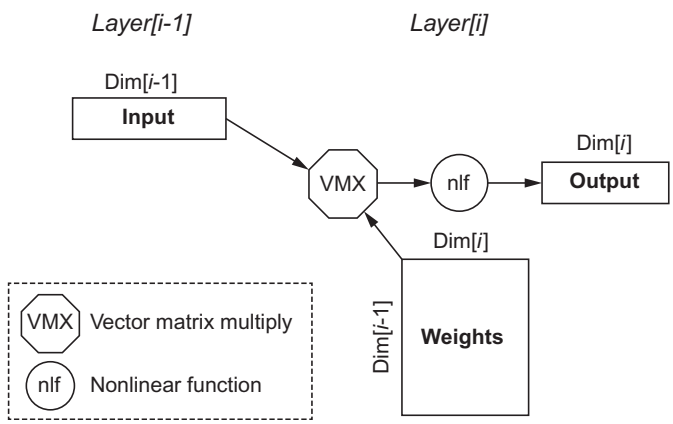


Figure 7.7 MLP showing the input Layer[*i*−1] on the left and the output Layer[*i*] on the right. ReLU is a popular nonlinear function for MLPs. The dimensions of the input and output layers are often different. Such a layer is called fully connected because it depends on all the inputs from the prior layer, even if many of them are zeros. One study suggested that 44% were zeros, which presumably is in part because ReLU turns negative numbers into zeros.

multiply of the input vector times the weights array. Here are the parameters and the equations to determine weights and operations for inference (we count multiply and add as two operations):

- Dim[i]: Dimension of the output vector, which is the number of neurons
- Dim[$i-1$]: Dimension of the input vector
- Number of weights: Dim[$i-1$] $\times$ Dim[i]
- Operations: $2 \times$ Number of weights
- Operations/Weight: 2

This final term is the *operational intensity* from the Roofline model discussed in Chapter 4. We use operations per *weight* because there can be millions of weights, which usually don't fit on the chip. For example, the dimensions of one stage of an MLP in Section 7.9 has Dim[$i-1$] = 4096 and Dim[i] = 2048, so for that layer, the number of neurons is 2048, number of weights is 8,388,608, the number of operations is 16,777,216, and the operational intensity is 2. As we recall from the Roofline model, low operational intensity makes it harder to deliver high performance.

Convolutional Neural Network

CNNs are widely used for computer vision applications. As images have a two-dimensional structure, neighboring pixels are the natural place to look to find relationships. CNNs take as inputs a set of nonlinear functions from spatially nearby regions of outputs from the prior layer and then multiplies by the weights, which reuses the weights many times.

The idea behind CNNs is that each layer raises the level of abstraction of the image. For example, the first layer might identify only horizontal lines and vertical lines. The second layer might combine them to identify corners. The next step might be rectangles and circles. The following layer could use that input to detect portions of a dog, like eyes or ears. The higher layers would be trying to identify characteristics of different breeds of dogs.

Each neural layer produces a set of two-dimensional *feature maps*, where each cell of the two-dimensional feature map is trying to identify one feature in the corresponding area of the input.

Figure 7.8 shows the starting point where a 2×2 stencil computation from the input image creates the elements of the first feature map. A *stencil computation* uses neighboring cells in a fixed pattern to update all the elements of an array. The number of output feature maps will depend on how many different features you are trying to capture from the image and the stride used to apply the stencil.

The process is actually more complicated because the image is usually not just a single, flat two-dimensional layer. Typically, a color image will have three levels for red, green, and blue. For example, a 2×2 stencil will access 12 elements: 2×2

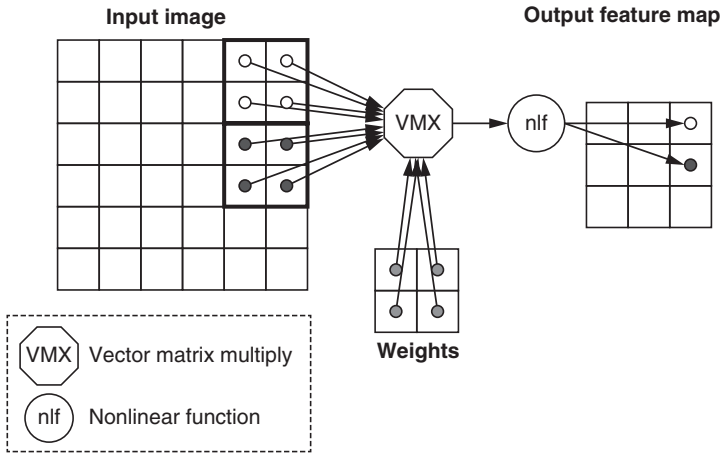


Figure 7.8 Simplified first step of a CNN. In this example, every group of four pixels of the input image are multiplied by the same four weights to create the cells of the output feature map. The pattern depicted shows a stride of two between the groups of input pixels, but other strides are possible. To relate this figure to MLP, you can think of each 2×2 convolution as a tiny fully connected operation to produce one point of the output feature map. Figure 7.9 shows how multiple feature maps turn the points into a vector in the third dimension.

of red pixels, 2×2 of green pixels, and 2×2 of blue pixels. In this case, you need 12 weights per output feature map for a 2×2 stencil on three input levels of an image.

Figure 7.9 shows the general case of an arbitrary number of input and output feature maps, which occurs after that first layer. The calculation is a three-dimensional stencil over all the input feature maps with a set of weights to produce one output feature map.

For the mathematically oriented, if the number of input feature maps and output feature maps both equal 1 and the stride is 1, then a single layer of a two-dimensional CNN is the same calculation as a two-dimensional discrete convolution.

As we see in Figure 7.9, CNNs are more complicated than MLPs. Here are the parameter and the equations to calculate the weights and operations:

- DimFM[$i-1$]: Dimension of the (square) input Feature Map
- DimFM[i]: Dimension of the (square) output Feature Map
- DimSten[i]: Dimension of the (square) stencil
- NumFM[$i-1$]: Number of input Feature Maps
- NumFM[i]: Number of output Feature Maps
- Number of neurons: NumFM[i] $\times$ DimFM[i]^2

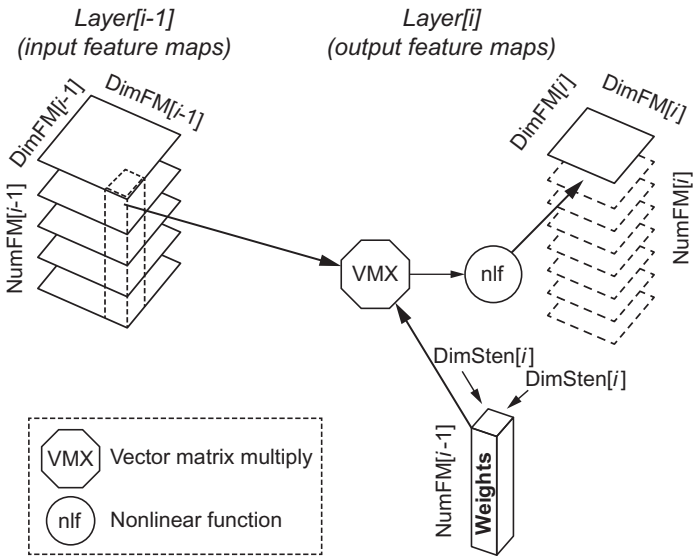


Figure 7.9 CNN general step showing input feature maps of Layer[$i-1$] on the left, the output feature maps of Layer[i] on the right, and a three-dimensional stencil over input feature maps to produce a single output feature map. Each output feature map has its own unique set of weights, and the vector-matrix multiply happens for every one. The dotted lines show future output feature maps in this figure. As this figure illustrates, the dimensions and number of the input and output feature maps are often different. As with MLPs, ReLU is a popular nonlinear function for CNNs.

- Number of weights per output Feature Map: $\text{NumFM}[i-1] \times \text{DimSten}[i]^2$
- Total number of weights per layer: $\text{NumFM}[i] \times$ Number of weights per output Feature Map
- Number of operations per output Feature Map: $2 \times \text{DimFM}[i]^2 \times$ Number of weights per output Feature Map
- Total number of operations per layer: $\text{NumFM}[i] \times$ Number of operations per output Feature Map $= 2 \times \text{DimFM}[i]^2 \times \text{NumFM}[i] \times$ Number of weights per output Feature Map $= 2 \times \text{DimFM}[i]^2 \times$ Total number of weights per layer
- Operations/Weight: $2 \times \text{DimFM}[i]^2$

A CNN in Section 7.9 has a layer with $\text{DimFM}[i-1] = 28$, $\text{DimFM}[i] = 14$, $\text{DimSten}[i] = 3$, $\text{NumFM}[i-1] = 64$ (number of input feature maps), and $\text{NumFM}[i] = 128$ (number of output feature maps). That layer has 25,088 neurons, 73,728 weights, does 28,901,376 operations, and has an operational intensity of 392. As our example indicates, CNN layers generally have fewer weights and greater operational intensity than the fully connected layers found in MLPs.

Recurrent Neural Network

The third type of DNN is RNNs, which are popular for speech recognition or language translation. RNNs add the ability to explicitly model sequential inputs by adding state to the DNN model so that RNNs can remember facts. It's analogous to the difference in hardware between combinational logic and a state machine. For example, you might learn the gender of the person, which you would want to pass along to remember later when translating words. Each layer of an RNN is a collection of weighted sums of inputs from the prior layer and the previous state. The weights are reused across time steps.

Long short-term memory (*LSTM*) is by far the most popular RNN today. LSTMs mitigate a problem that previous RNNs had with their inability to remember important long-term information.

Unlike the other two DNNs, LSTM is a hierarchical design. LSTM consists of modules called *cells*. You can think of cells as templates or macros that are linked together to create the full DNN model, similar to how layers of an MLP line up to form a complete DNN model.

Figure 7.10 shows how the LSTM cells are linked together. They are hooked up from left to right, connecting the output of one cell to the input of the next. They are also unrolled in time, which runs top down in Figure 7.10. Thus a sentence is input a word at a time per iteration of the unrolled loop. The long-term and short-term memory information that gives the LSTM its name is also passed top-down from one iteration to the next.

Figure 7.11 shows the contents of an LSTM cell. As we would expect from Figure 7.10, the input is on the left, the output is on the right, the two memory inputs are at the top, and the two memory outputs are at the bottom.

Each cell does five vector-matrix multiplies using five unique sets of weights. The matrix multiply on the input is just like the MLP in Figure 7.7. Three others are called *gates* in that they gate or limit how much information from one source is passed along to the standard output or the memory output. The amount of information sent per gate is set by their weights. If the weights are mostly zeros or small values, then little gets through; conversely, if they are mostly large, then the gate lets most information flow. The three gates are called the *input gate*, the *output gate*, and the *forget gate*. The first two filter the input and output, and the last one determines what to forget along the long-term memory path.

The short-term memory output is a vector-matrix multiply using the Short Term Weights and the output of this cell. The short-term label is applied because it does not directly use any of the inputs to the cell.

Because the LSTM cell inputs and outputs are all connected together, the size of the three input-output pairs must be the same. Looking inside the cell, there are enough dependencies that all of the inputs and outputs are often the same size. Let's assume they are all the same size, called *Dim*.

Even so, the vector-matrix multiplies are not all the same size. The vectors for the three gate multiplies are $3 \times \text{Dim}$, because the LSTM concatenates all three inputs. The vector for the input multiply is $2 \times \text{Dim}$, because the LSTM

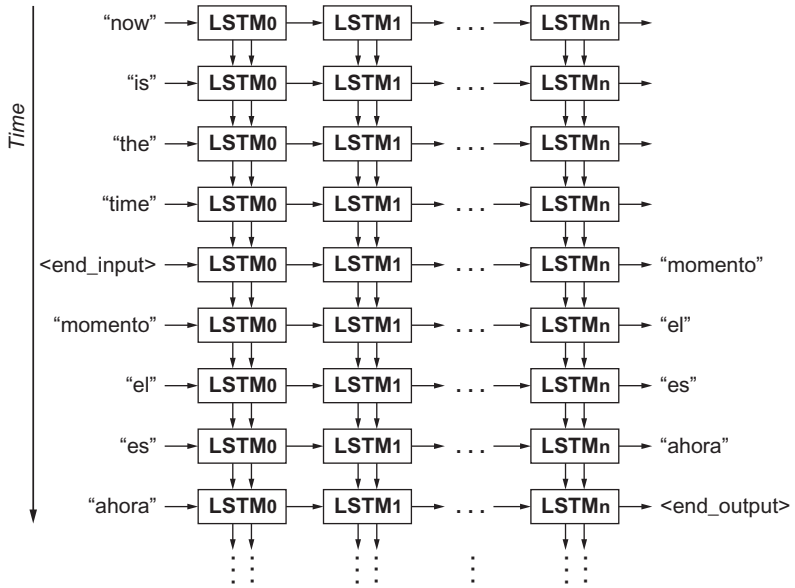


Figure 7.10 LSTM cells connected together. The inputs are on the left (English words), and the outputs are on the right (the translated Spanish words). The cells can be thought of as being unrolled over time, from top to bottom. Thus the short-term and long-term memory of LSTM is implemented by passing information top-down between unrolled cells. They are unrolled enough to translate whole sentences or even paragraphs. Such sequence-to-sequence translation models delay their output until they get to the end of the input (Wu et al., 2016). They produce the translation in *reverse order*, using the most recent translated word as input to the next step, so "now is the time" becomes "ahora es el momento." (This figure and the next are often shown turned 90 degrees in LSTM literature, but we've rotated them to be consistent with Figures 7.7 and 7.8.)

concatenates the input with the short-term memory input as the vector. The vector for the last multiply is just $1 \times \text{Dim}$, because it is just the output.

Now we can finally calculate the weights and operations:

- Number of weights per cell: $3 \times (3 \times \text{Dim} \times \text{Dim}) + (2 \times \text{Dim} \times \text{Dim}) + (1 \times \text{Dim} \times \text{Dim}) = 12 \times \text{Dim}^2$
- Number of operations for the 5 vector-matrix multiplies per cell: $2 \times$ Number of weights per cell $= 24 \times \text{Dim}^2$
- Number of operations for the 3 element-wise multiplies and 1 addition (vectors are all the size of the output): $4 \times \text{Dim}$
- Total number of operations per cell (5 vector-matrix multiplies and the 4 element-wise operations): $24 \times \text{Dim}^2 + 4 \times \text{Dim}$
- Operations/Weight: ~ 2

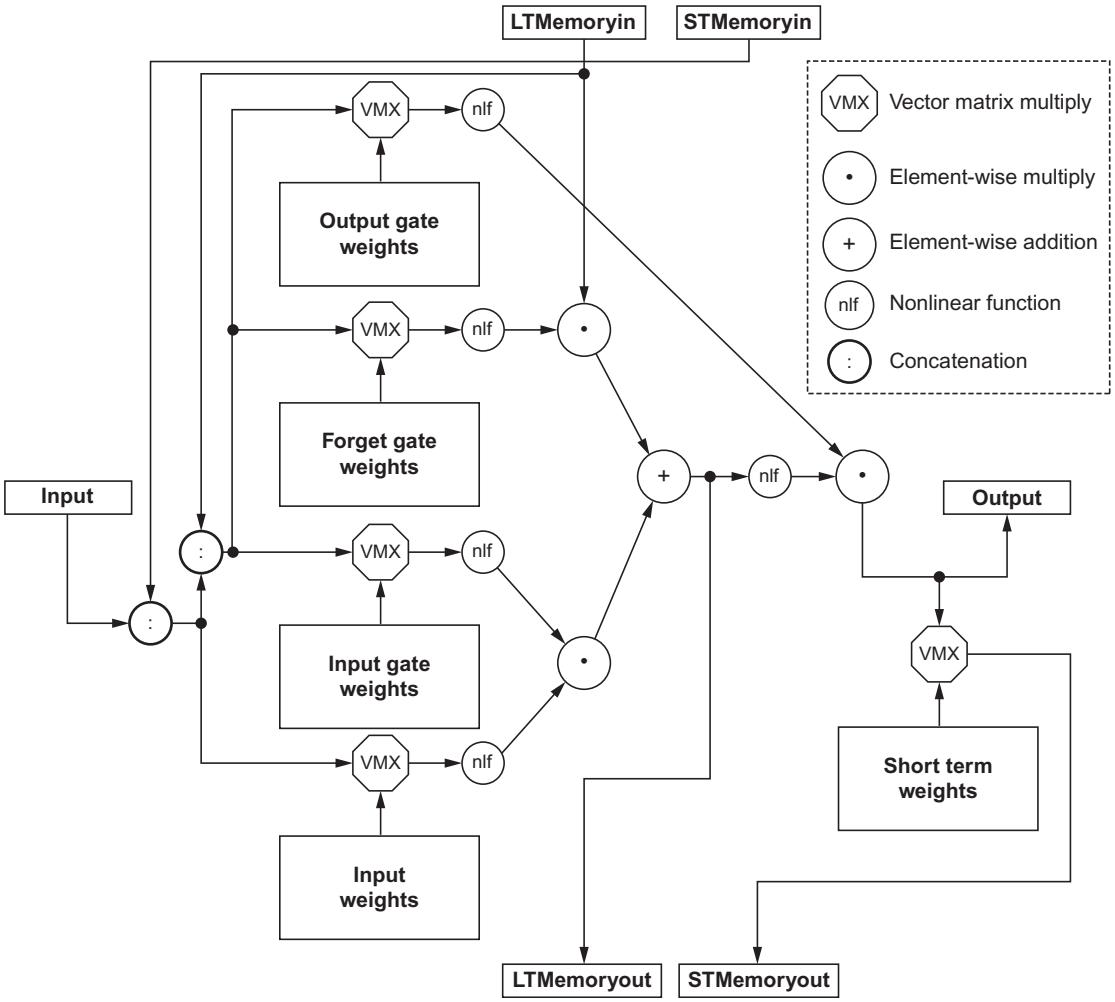


Figure 7.11 This LSTM cell contains 5 vector-matrix multiplies, 3 element-wise multiplies, 1 element-wise add, and 6 nonlinear functions. The standard input and short-term memory input are concatenated to form the vector operand for the input vector-matrix multiply. The standard input, long-term memory input, and short-term memory input are concatenated to form the vector that is used in three of the other four vector-matrix multiplies. The nonlinear functions for the three gates are Sigmoids $f(x) = 1/(1+\exp(-x))$; the others are hyperbolic tangents. (This figure and the previous one are often shown turned 90 degrees in LSTM literature, but we've rotated them to be consistent with Figures 7.7 and 7.8.)

Dim is 1024 for one of the six cells of an LSTM in Section 7.9. Its number of weights is 12,582,912, its number of operations is 25,169,920, and its operational intensity is 2.0003. Thus LSTMs are like MLPs in that they typically have more weights and a lower operational intensity than CNNs.

Batches

Because DNNs can have many weights, a performance optimization is to reuse the weights once they have been fetched from memory across a set of inputs, thereby increasing effective operational intensity. For example, an image-processing DNN might work on a set of 32 images at a time to reduce the effective cost of fetching weights by a factor of 32. Such datasets are called *batches* or *minibatches*. In addition to improving the performance of inference, backpropagation needs a batch of examples instead of one at a time in order to train well.

Looking at an MLP in Figure 7.7, a batch can be seen as a sequence of input row vectors, which you can think of as a matrix with a height dimension that matches the batch size. A sequence of row vector inputs to the five matrix multiplies of LSTMs in Figure 7.11 can also be considered a matrix. In both cases, computing them as matrices instead of sequentially as independent vectors improves computing efficiency.

Quantization

Numerical precision is less important for DNNs than for many applications. For example, there is no need for double-precision floating-point arithmetic, which is the standard bearer of high-performance computing. It's even unclear that you need the full accuracy of the IEEE 754 floating-point standard, which aims to be accurate within one-half of a unit in the last place of the floating-point significand.

To take advantage of the flexibility in numerical precision, some developers use fixed point instead of floating point for the inference phase. (Training is almost always done in floating-point arithmetic.) This conversion is called *quantization*, and such a transformed application is said to be *quantized* (Vanhoucke et al., 2011). The fixed-point data width is usually 8 or 16 bits, with the standard multiply-add operation accumulating at twice the width of the multiplies. This transformation typically occurs after training, and it can reduce DNN accuracy by a few percentage points (Bhattacharya and Lane, 2016).

Summary of DNNs

Even this quick overview suggests that DSAs for DNNs will need to perform at least these matrix-oriented operations well: vector-matrix multiply, matrix-matrix multiply, and stencil computations. They will also need support for the nonlinear functions, which include at a minimum ReLU, Sigmoid, and tanh. These modest requirements still leave open a very large design space, which the next four sections explore.

7.4 Google's Tensor Processing Unit, an Inference Data Center Accelerator

The Tensor Processing Unit (*TPU*)[1] is Google's first custom ASIC DSA for WSCs. Its domain is the inference phase of DNNs, and it is programmed using the TensorFlow framework, which was designed for DNNs. The first TPU was been deployed in Google data centers in 2015.

The heart of the TPU is a 65,536 (256×256) 8-bit ALU Matrix Multiply Unit and a large software-managed on-chip memory. The TPU's single-threaded, deterministic execution model is a good match to the 99th-percentile response-time requirement of the typical DNN inference application.

TPU Origin

Starting as far back as 2006, Google engineers had discussions about deploying GPUs, FPGAs, or custom ASICs in their data centers. They concluded that the few applications that could run on special hardware could be done virtually for free using the excess capacity of the large data centers, and it's hard to improve on free. The conversation changed in 2013 when it was projected that if people used voice search for three minutes a day using speech recognition DNNs, it would have required Google's data centers to double in order to meet computation demands. That would be very expensive to satisfy with conventional CPUs. Google then started a high-priority project to quickly produce a custom ASIC for inference (and bought off-the-shelf GPUs for training). The goal was to improve cost-performance by $10\times$ over GPUs. Given this mandate, the TPU was designed, verified (Steinberg, 2015), built, and deployed in data centers in just 15 months.

TPU Architecture

To reduce the chances of delaying deployment, the TPU was designed to be a coprocessor on the PCIe I/O bus, which allows it to be plugged into existing servers. Moreover, to simplify hardware design and debugging, the host server sends instructions over the PCIe bus directly to the TPU for it to execute, rather than having the TPU fetch the instructions. Thus the TPU is closer in spirit to an FPU (floating-point unit) coprocessor than it is to a GPU, which fetches instructions from its memory.

Figure 7.12 shows the block diagram of the TPU. The host CPU sends TPU instructions over the PCIe bus into an instruction buffer. The internal blocks are typically connected together by 256-byte-wide (2048-bits) paths. Starting in the upper-right corner, the *Matrix Multiply Unit* is the heart of the TPU. It contains

[1]This section is based on the paper "In-Datacenter Performance Analysis of a Tensor Processing Unit" Jouppi et al., 2017, of which one of your book authors was a coauthor.

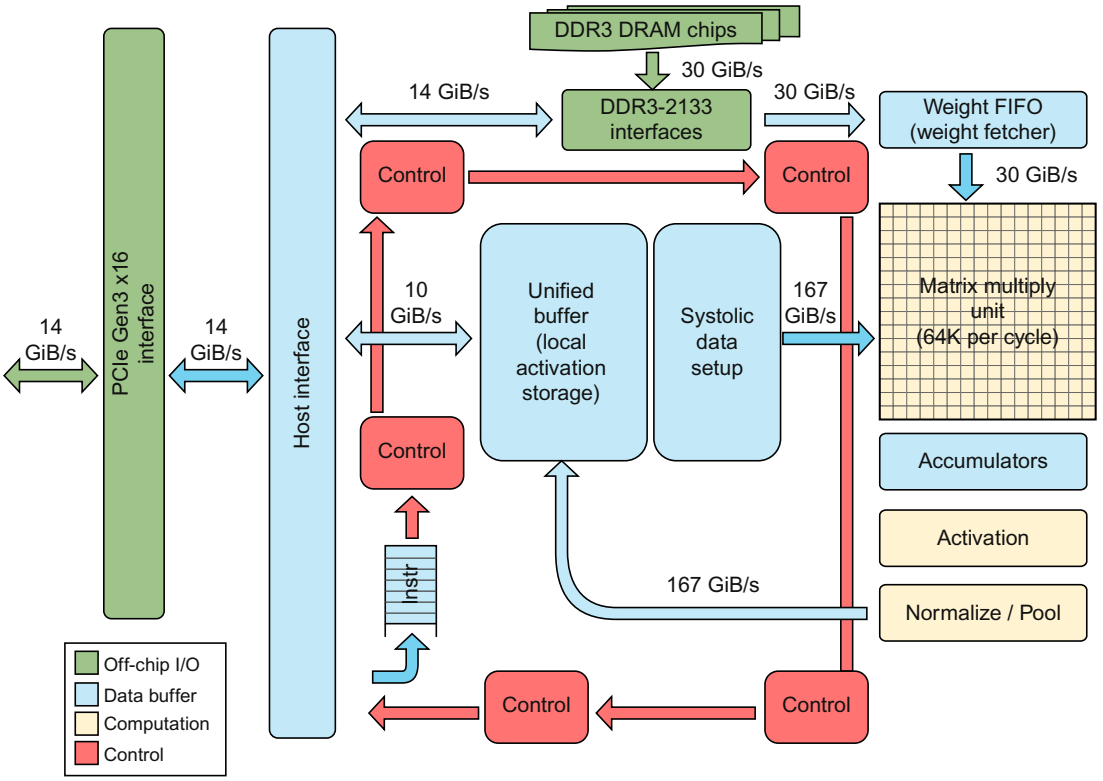


Figure 7.12 TPU Block Diagram. The PCIe bus is Gen3 ×16. The main computation part is the light-shaded Matrix Multiply Unit in the upper-right corner. Its inputs are the medium-shaded Weight FIFO and the medium-shaded Unified Buffer and its output is the medium-shaded Accumulators. The light-shaded Activation Unit performs the nonlinear functions on the Accumulators, which go to the Unified Buffer.

256 × 256 ALUs that can perform 8-bit multiply-and-adds on signed or unsigned integers. The 16-bit products are collected in the 4 MiB of 32-bit *Accumulators* below the matrix unit. When using a mix of 8-bit weights and 16-bit activations (or vice versa), the Matrix Unit computes at half-speed, and it computes at a quarter-speed when both are 16 bits. It reads and writes 256 values per clock cycle and can perform either a matrix multiply or a convolution. The nonlinear functions are calculated by the *Activation* hardware.

The weights for the matrix unit are staged through an on-chip *Weight FIFO* that reads from an off-chip 8 GiB DRAM called *Weight Memory* (for inference, weights are read-only; 8 GiB supports many simultaneously active models). The intermediate results are held in the 24 MiB on-chip *Unified Buffer*, which can serve as inputs to the Matrix Multiply Unit. A programmable DMA controller transfers data to or from CPU Host memory and the Unified Buffer.

TPU Instruction Set Architecture

As instructions are sent over the relatively slow PCIe bus, TPU instructions follow the CISC tradition, including a repeat field. The TPU does not have a program counter, and it has no branch instructions; instructions are sent from the host CPU. The clock cycles per instruction (CPI) of these CISC instructions are typically 10–20. It has about a dozen instructions overall, but these five are the key ones:

1. `Read_Host_Memory` reads data from the CPU host memory into the Unified Buffer.
2. `Read_Weights` reads weights from Weight Memory into the Weight FIFO as input to the Matrix Unit.
3. `MatrixMultiply/Convolve` causes the Matrix Multiply Unit to perform a matrix-matrix multiply, a vector-matrix multiply, an element-wise matrix multiply, an element-wise vector multiply, or a convolution from the Unified Buffer into the Accumulators. A matrix operation takes a variable-sized B*256 input, multiplies it by a 256×256 constant input, and produces a B*256 output, taking B pipelined cycles to complete. For example, if the input were 4 vectors of 256 elements, B would be 4, so it would take 4 clock cycles to complete.
4. `Activate` performs the nonlinear function of the artificial neuron, with options for ReLU, Sigmoid, tanh, and so on. Its inputs are the Accumulators, and its output is the Unified Buffer.
5. `Write_Host_Memory` writes data from the Unified Buffer into the CPU host memory.

The other instructions are alternate host memory read/write, set configuration, two versions of synchronization, interrupt host, debug-tag, nop, and halt. The CISC MatrixMultiply instruction is 12 bytes, of which 3 are Unified Buffer address; 2 are accumulator address; 4 are length (sometimes 2 dimensions for convolutions); and the rest are opcode and flags.

The goal is to run whole inference models in the TPU to reduce interactions with the host CPU and to be flexible enough to match the DNN needs of 2015 and beyond, instead of just what was required for 2013 DNNs.

TPU Microarchitecture

The microarchitecture philosophy of the TPU is to keep the Matrix Multiply Unit busy. The plan is to hide the execution of the other instructions by overlapping their execution with the `MatrixMultiply` instruction. Thus each of the preceding four general categories of instructions have separate execution hardware (with read and write host memory combined into the same unit). To increase instruction parallelism further, the `Read_Weights` instruction follows the decoupled access/

execute philosophy (Smith, 1982b) in that they can complete after sending their addresses but before the weights are fetched from Weight Memory. The matrix unit has not-ready signals from the Unified Buffer and the Weight FIFO that will cause the matrix unit to stall if their data are not yet available.

Note that a TPU instruction can execute for many clock cycles, unlike the traditional RISC pipeline with one clock cycle per stage.

Because reading a large SRAM is much more expensive than arithmetic, the Matrix Multiply Unit uses systolic execution to save energy by reducing reads and writes of the Unified Buffer (Kung and Leiserson, 1980; Ramacher et al., 1991; Ovtcharov et al., 2015b). A *systolic array* is a two-dimensional collection of arithmetic units that each independently compute a partial result as a function of inputs from other arithmetic units that are considered upstream to each unit. It relies on data from different directions arriving at cells in an array at regular intervals where they are combined. Because the data flows through the array as an advancing wave front, it is similar to blood being pumped through the human circulatory system by the heart, which is the origin of the systolic name.

Figure 7.13 demonstrates how a systolic array works. The six circles at the bottom are the multiply-accumulate units that are initialized with the weights *wi*. The staggered input data *xi* are shown coming into the array from above. The 10 steps of the figure represent 10 clock cycles moving down from top to bottom of the page. The systolic array passes the inputs down and the products and sums to the right. The desired sum of products emerges as the data completes its path through the systolic array. Note that in a systolic array, the input data is read only once from memory, and the output data is written only once to memory.

In the TPU, the systolic array is rotated. Figure 7.14 shows that the weights are loaded from the top and the input data flows into the array in from the left. A given 256-element multiply-accumulate operation moves through the matrix as a diagonal wave front. The weights are preloaded and take effect with the advancing wave alongside the first data of a new block. Control and data are pipelined to give the illusion that the 256 inputs are read at once, and after a feed delay, they update one location of each of 256 accumulator memories. From a correctness perspective, software is unaware of the systolic nature of the matrix unit, but for performance, it does worry about the latency of the unit.

TPU Implementation

The TPU chip was fabricated using the 28-nm process. The clock rate is 700 MHz. Figure 7.15 shows the floor plan of the TPU. Although the exact die size is not revealed, it is less than half the size of an Intel Haswell server microprocessor, which is 662 mm^2.

The 24 MiB Unified Buffer is almost a third of the die, and the Matrix Multiply Unit is a quarter, so the datapath is nearly two-thirds of the die. The 24 MiB size was picked in part to match the pitch of the Matrix Unit on the die and, given the

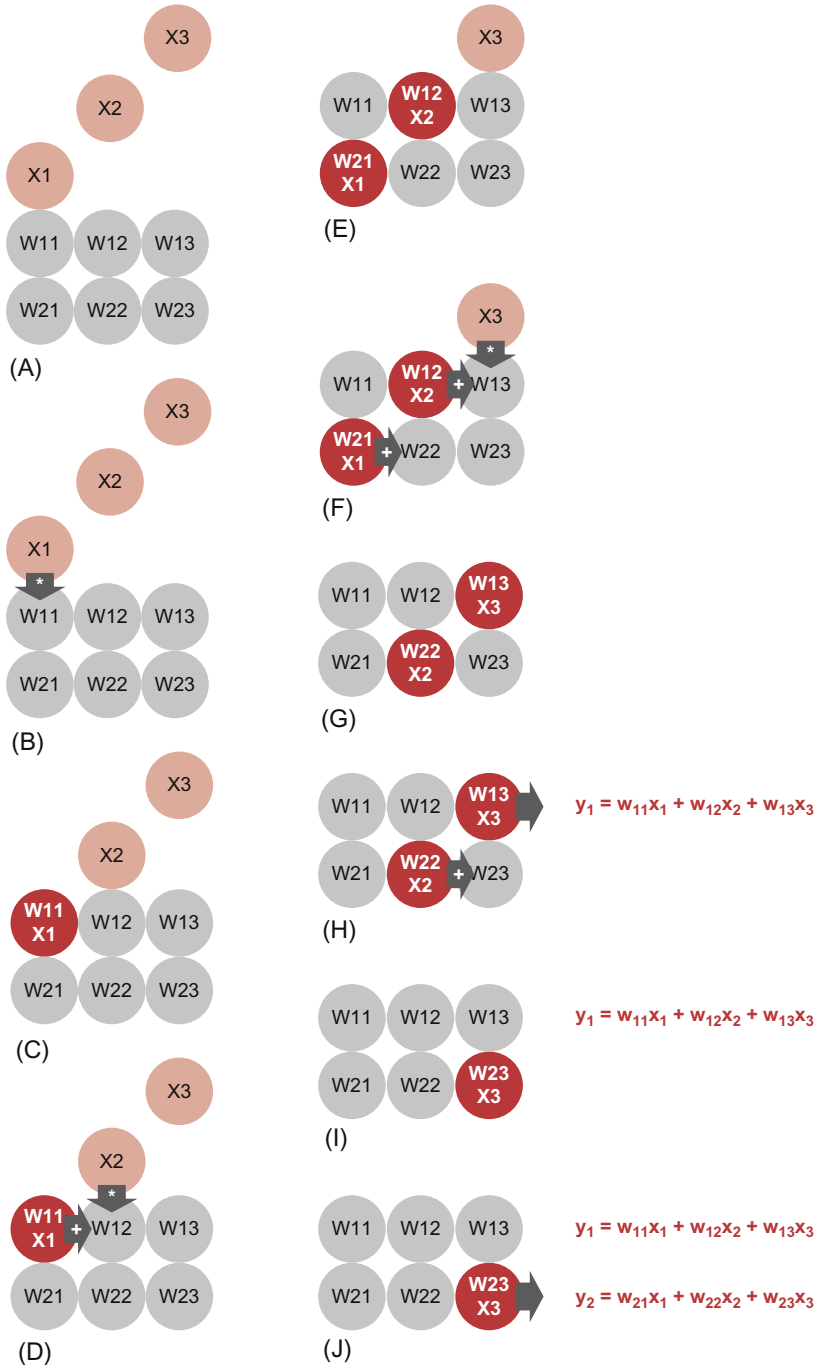


Figure 7.13 Example of systolic array in action, from top to bottom on the page. In this example, the six weights are already inside the multiply-accumulate units, as is the norm for the TPU. The three inputs are staggered in time to get the desired effect, and in this example are shown coming in from the top. (In the TPU, the data actually comes in from the left.) The array passes the data down to the next element and the result of the computation to the right to the next element. At the end of the process, the sum of products is found to the right. Drawings courtesy of Yaz Sato.

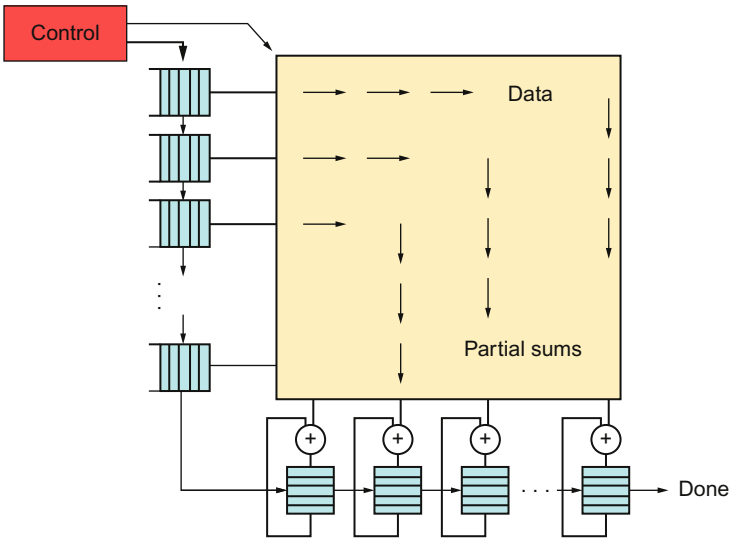


Figure 7.14 Systolic data flow of the Matrix Multiply Unit.

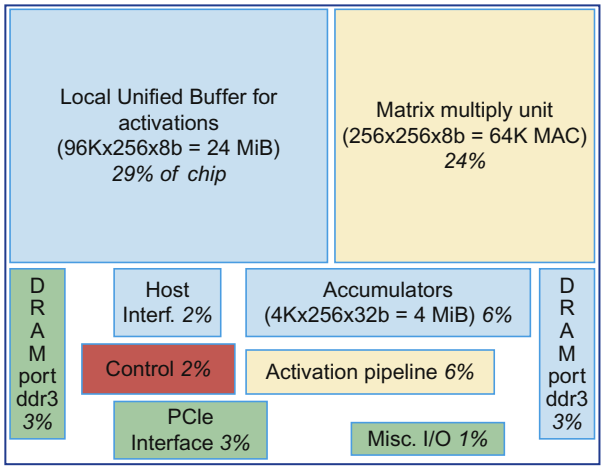


Figure 7.15 Floor plan of TPU die. The shading follows Figure 7.14. The light data buffers are 37%, the light computation units are 30%, the medium I/O is 10%, and the dark control is just 2% of the die. Control is much larger (and much more difficult to design) in a CPU or GPU. The unused white space is a consequence of the emphasis on time to tape-out for the TPU.

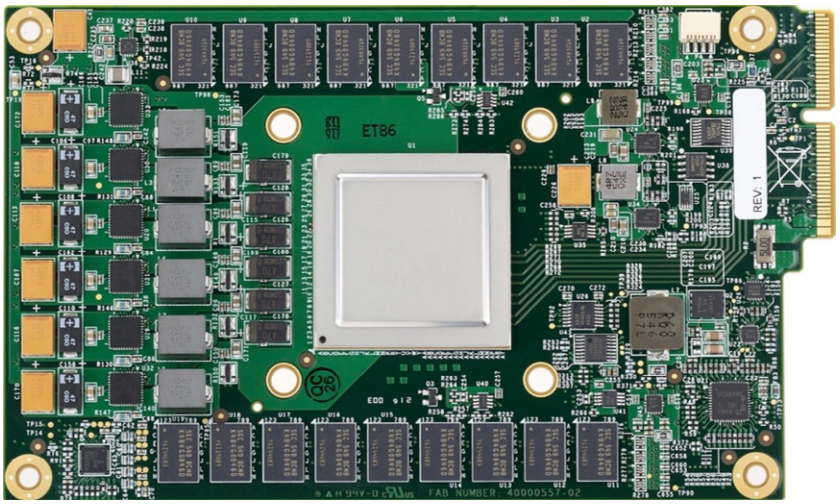


Figure 7.16 TPU printed circuit board. It can be inserted into the slot for an SATA disk in a server, but the card uses the PCIe bus.

short development schedule, in part to simplify the compiler. Control is just 2%. Figure 7.16 shows the TPU on its printed circuit card, which inserts into existing servers in a SATA disk slot.

TPU Software

The TPU software stack had to be compatible with that developed for CPUs and GPUs so that applications could be ported quickly. The portion of the application run on the TPU is typically written using TensorFlow and is compiled into an API that can run on GPUs or TPUs (Larabel, 2016). Figure 7.17 shows TensorFlow code for a portion of an MLP.

Like GPUs, the TPU stack is split into a User Space Driver and a Kernel Driver. The Kernel Driver is lightweight and handles only memory management and interrupts. It is designed for long-term stability. The User Space driver changes frequently. It sets up and controls TPU execution, reformats data into TPU order, and translates API calls into TPU instructions and turns them into an application binary. The User Space driver compiles a model the first time it is evaluated, caching the program image and writing the weight image into the TPU Weight Memory; the second and following evaluations run at full speed. The TPU runs most models completely from inputs to outputs, maximizing the ratio of TPU compute time to I/O time. Computation is often done one layer at a time, with overlapped execution allowing the matrix unit to hide most noncritical path operations.

```
# Network Parameters
n_hidden_1 = 256 # 1st layer number of features
n_hidden_2 = 256 # 2nd layer number of features
n_input = 784 # MNIST data input (img shape: 28*28)
n_classes = 10 # MNIST total classes (0-9 digits)

# tf Graph input
x = tf.placeholder("float", [None, n_input])
y = tf.placeholder("float", [None, n_classes])

# Create model
def multilayer_perceptron(x, weights, biases):
    # Hidden layer with ReLU activation
    layer_1 = tf.add(tf.matmul(x, weights['h1']), biases['b1'])
    layer_1 = tf.nn.relu(layer_1)
    # Hidden layer with ReLU activation
    layer_2 = tf.add(tf.matmul(layer_1, weights['h2']), biases['b2'])
    layer_2 = tf.nn.relu(layer_2)
    # Output layer with linear activation
    out_layer = tf.matmul(layer_2, weights['out']) + biases['out']
    return out_layer

# Store layers weight & bias
weights = {
    'h1': tf.Variable(tf.random_normal([n_input, n_hidden_1])),
    'h2': tf.Variable(tf.random_normal([n_hidden_1, n_hidden_2])),
    'out': tf.Variable(tf.random_normal([n_hidden_2, n_classes]))
}
biases = {
    'b1': tf.Variable(tf.random_normal([n_hidden_1])),
    'b2': tf.Variable(tf.random_normal([n_hidden_2])),
    'out': tf.Variable(tf.random_normal([n_classes]))
}
```

Figure 7.17 Portion of the TensorFlow program for the MNIST MLP. It has two hidden 256 × 256 layers, with each layer using a ReLU as its nonlinear function.

Improving the TPU

The TPU architects looked at variations of the microarchitecture to see whether they could have improved the TPU.

Like an FPU, the TPU coprocessor has a relatively easy microarchitecture to evaluate, so the TPU architects created a performance model and estimated performance as the memory bandwidth, the matrix unit size, and the clock rate and number of accumulators varied. Measurements using TPU hardware counters found that the modeled performance was on average within 8% of the hardware.

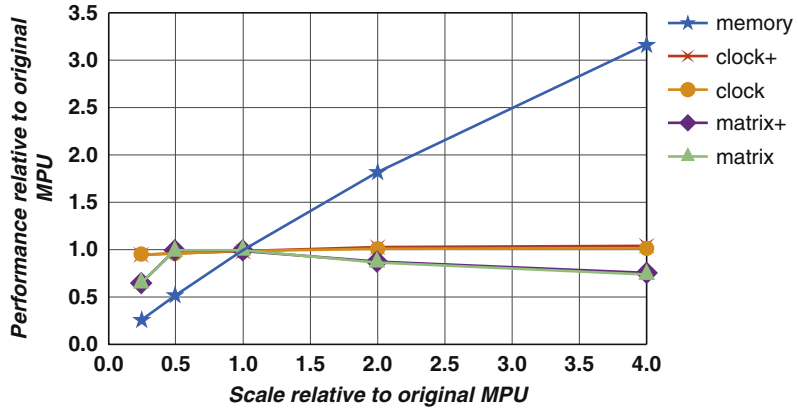


Figure 7.18 Performance as metrics scale from 0.25× to 4×: memory bandwidth, clock rate+accumulators, clock rate, matrix unit dimension+accumulators, and one dimension of the square matrix unit This is the average performance calculated from six DNN applications in Section 7.9. The CNNs tend to be computation-bound, but the MLPs and LSTMs are memory-bound. Most applications benefit from a faster memory, but a faster clock makes little difference, and a bigger matrix unit actually hurts performance. This performance model is only for code running inside the TPU and does not factor in the CPU host overhead.

Figure 7.18 shows the performance sensitivity of the TPU as these parameters scale over the range for 0.25 × to 4 ×. (Section 7.9 lists the benchmarks used.) In addition to evaluating the impact of only raising clock rates (*clock* in Figure 7.18), Figure 7.18 also plots a design (*clock+*) that increases the clock rate and scales the number of accumulators correspondingly so that the compiler can keep more memory references in flight. Likewise, Figure 7.18 plots matrix unit expansion if the number of accumulators increase with the square of the rise in one dimension (*matrix+*), because the matrix grows in both dimensions, as well as only increasing the matrix unit (*matrix*).

First, increasing memory bandwidth (*memory*) has the biggest impact: performance improves 3 × on average when memory bandwidth increases 4 ×, because it reduces the time waiting for weight memory. Second, clock rate has little benefit on average with or without more accumulators. Third, the average performance in Figure 7.18 slightly *degrades* when the matrix unit expands from 256 × 256 to 512 × 512 for all applications, whether or not they get more accumulators. The issue is analogous to internal fragmentation of large pages, only worse because it's in two dimensions.

Consider the 600 × 600 matrix used in LSTM1. With a 256 × 256 matrix unit, it takes nine steps to tile 600 × 600, for a total of 18 µs of time. The larger 512 × 512 unit requires only four steps, but each step takes four times longer, or 32 µs of time. The TPU's CISC instructions are long, so decode is insignificant and does not hide the overhead of loading from the DRAM.

Given these insights from the performance model, the TPU architects next evaluated an alternative and hypothetical TPU that they might have designed in the same process technology if they'd had more than 15 months to do so. More aggressive logic synthesis and block design might have increased the clock rate by 50%. The architects found that designing an interface circuit for GDDR5 memory, as used by the K80, would improve Weight Memory bandwidth by more than a factor of five. As Figure 7.18 shows, increasing clock rate to 1050 MHz, but not helping memory, made almost no change in performance. If the clock is left at 700 MHz, but it uses GDDR5 instead for Weight Memory, performance is increased by 3.2 ×, even accounting for the host CPU overhead of invoking the DNN on the revised TPU. Doing both does not improve average performance further.

Summary: How TPU Follows the Guidelines

Despite living on an I/O bus and having relatively little memory bandwidth that limits full utilization of the TPU, a small fraction of a big number can, nonetheless, be relatively large. As we will see in Section 7.9, the TPU delivered on its goal of a tenfold improvement in cost-performance over the GPU when running DNN inference applications. Moreover, a redesigned TPU with the only change being a switch to the same memory technology as in the GPU would be three times faster.

One way to explain the TPU's success is to see how it followed the guidelines in Section 7.2.

1. *Use dedicated memories to minimize the distance over which data is moved.* The TPU has the 24 MiB Unified Buffer that holds the intermediate matrices and vectors of MLPs and LSTMs and the feature maps of CNNs. It is optimized for accesses of 256 bytes at a time. It also has the 4 MiB Accumulators, each 32-bits wide, that collect the output of the Matrix Unit and act as input to the hardware that calculates the nonlinear functions. The 8-bit weights are stored in a separate off-chip weight memory DRAM and are accessed via an on-chip weight FIFO. In contrast, all these types and sizes of data would exist in redundant copies at several levels of the inclusive memory hierarchy of a general-purpose CPU.
2. *Invest the resources saved from dropping advanced microarchitectural optimizations into more arithmetic units or bigger memories.* The TPU offers 28 MiB of dedicated memory and 65,536 8-bit ALUs, which means it has about 60% of the memory and 250 times as many ALUs as a server-class CPU, despite being half its size and power (see Section 7.9). Compared to a server-class GPU, the TPU has 3.5 times the on-chip memory and 25 times as many ALUs.
3. *Use the easiest form of parallelism that matches the domain.* The TPU delivers its performance via a two-dimensional SIMD parallelism with its 256 × 256 Matrix Multiply Unit, which is internally pipelined with a systolic organization, plus a simple overlapped execution pipeline of its

instructions. GPUs rely instead on multiprocessing, multithreading, and one-dimensional SIMD, and CPUs rely on multiprocessing, out-of-order execution, and one-dimensional SIMD.

4. *Reduce data size and type to the simplest needed for the domain.*
   The TPU computes primarily on 8-bit integers, although it supports 16-bit integers and accumulates in 32-bit integers. CPUs and GPUs also support 64-bit integers and 32-bit and 64-bit floating point.
5. *Use a domain-specific programming language to port code to the DSA.*
   The TPU is programmed using the TensorFlow programming framework, whereas GPUs rely on CUDA and OpenCL and CPUs must run virtually everything.

7.5 Microsoft Catapult, a Flexible Data Center Accelerator

At the same time that Google was thinking about deploying a custom ASIC in its data centers, Microsoft was considering accelerators for theirs. The Microsoft perspective was that any solution had to follow these guidelines:

- It had to preserve homogeneity of servers to enable rapid redeployment of machines and to avoid making maintenance and scheduling even more complicated, even if that notion is a bit at odds with the concept of DSAs.
- It had to scale to applications that might need more resources than could fit into a single accelerator without burdening all applications with multiple accelerators.
- It needed to be power-efficient.
- It couldn't become a dependability problem by being a single point of failure.
- It had to fit within the available spare space and power in existing servers.
- It could not hurt data center network performance or reliability.
- The accelerator had to improve the cost-performance of the server.

The first rule prevented deploying an ASIC that helped only some applications on some servers, which was the decision that Google made.

Microsoft started a project called Catapult that placed an FPGA on a PCIe bus board into data center servers. These boards have a dedicated network for applications that need more than one FPGA. The plan was to use the flexibility of the FPGA to tailor its use for varying applications both on different servers and to reprogram the same server to accelerate distinct applications over time. This plan increased the return on its investment of the accelerator. Another advantage of FPGAs is that they should have lower NRE than ASICs, which could again improve return on investment. We discuss two generations of Catapult, showing how the design evolved to meet the needs of WSCs.

One interesting upside of FPGAs is that each application—or even each phase of an application—can be thought of as its own DSA, so in this section, we get to see many examples of novel architectures in one hardware platform.

Catapult Implementation and Architecture

Figure 7.19 shows a PCIe board that Microsoft designed to fit within its servers, which limited power and cooling to 25 W. This constraint led to the selection of the 28-nm Altera Stratix V D5 FPGA for its first implementation of Catapult. The board also has 32 MiB of flash memory and includes two banks of DDR3-1600 DRAM with a total capacity of 8 GiB. The FPGA has 3926 18-bit ALUs, 5 MiB of on-chip memory, and 11 GB/s bandwidth to DDR3 DRAMs.

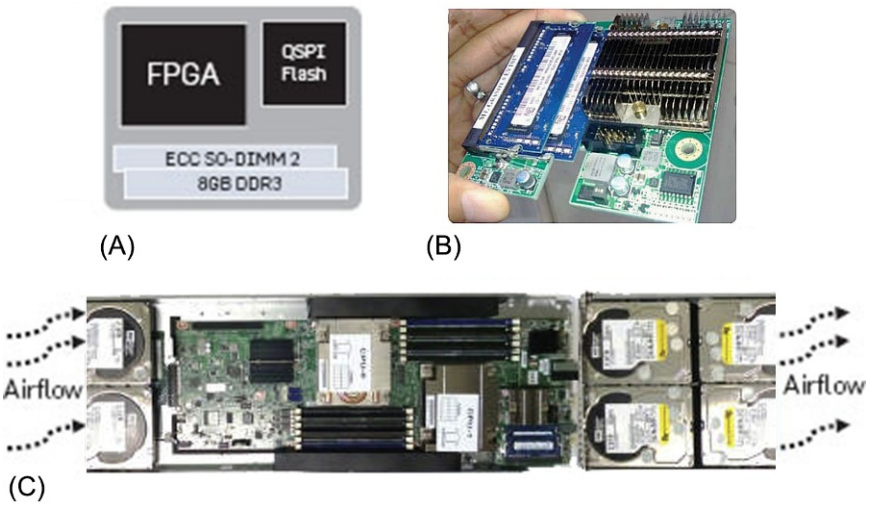


Figure 7.19 The Catapult board design. (A) shows the block diagram, and (B) is a photograph of both sides of the board, which is 10 cm × 9 cm × 16 mm. The PCIe and inter-FPGA network are wired to a connector on the bottom of the board that plugs directly into the motherboard. (C) is a photograph of the server, which is 1U (4.45 cm) high and half a standard rack wide. Each server has two 12-core Intel Sandy Bridge Xeon CPUs, 64 GiB of DRAM, 2 solid-state drives, 4 hard-disk drives, and a 10-Gbit Ethernet network card. The highlighted rectangle on the right in (C) shows the location of the Catapult FPGA board on the server. The cool air is sucked in from the left in (C), and the hot air exhausts to the right, which passes over the Catapult board. This hot spot and the amount of the power that the connector could deliver mean that the Catapult board is limited to 25 watts. Forty-eight servers share an Ethernet switch that connects to the data center network, and they occupy half of a data center rack.

Each of the 48 servers in half of a data center rack contains a Catapult board. Catapult follows the preceding guidelines about supporting applications that need more than a single FPGA without affecting the performance of the data center network. It adds a separate low-latency 20 Gbit/s network that connects 48 FPGAs. The network topology is a two-dimensional 6×8 torus network.

To follow the guideline about not being a single point of failure, this network can be reconfigured to operate even if one of the FPGAs fails. The board also has SECDED protection on all memories outside the FPGA, which is required for large-scale deployment in a data center.

Because FPGAs use a great deal of memory on the chip to deliver programmability, they are more vulnerable than ASICs to *single-event upsets (SEUs)* because of radiation as the process geometries shrink. The Altera FPGA in Catapult boards includes mechanisms to detect and correct SEUs inside the FPGA and reduces the chances of SEUs by periodically scrubbing the FPGA configuration state.

The separate network has an added benefit of reducing the variability of communication performance as compared to a data center network. Network unpredictability increases tail latency—which is especially detrimental for applications that face end users—so a separate network makes it easier to successfully offload work from the CPU to the accelerator. This FPGA network can run a much simpler protocol than in the data center because the error rates are considerably lower and the network topology is well defined.

Note that resiliency requires care when reconfiguring FPGAs so that they neither appear as failed nodes nor crash the host server or corrupt their neighbors. Microsoft developed a high-level protocol for ensuring safety when reconfiguring one or more FPGAs.

Catapult Software

Possibly the biggest difference between Catapult and the TPU is having to program in a hardware-description language such as Verilog or VHDL. As the Catapult authors write (Putnam et al., 2016):

> Going forward, the biggest obstacle to widespread adoption of FPGAs in the datacenter is likely to be programmability. FPGA development still requires extensive hand-coding in Register Transfer Level and manual tuning.

To reduce the burden of programming Catapult FPGAs, the Register Transfer Level (RTL) code is divided into the *shell* and the *role*, as Figure 7.20 shows. The shell code is like the system library on an embedded CPU. It contains the RTL code that will be reused across applications on the same FPGA board, such as data marshaling, CPU-to-FPGA communication, FPGA-to-FPGA communication, data movement, reconfiguration, and health monitoring. The shell RTL code is 23% of the Altera FPGA. The role code is the application logic, which the Catapult programmer writes using the remaining 77% of the FPGA resources. Having a shell has the added benefit of offering a standard API and standard behavior across applications.

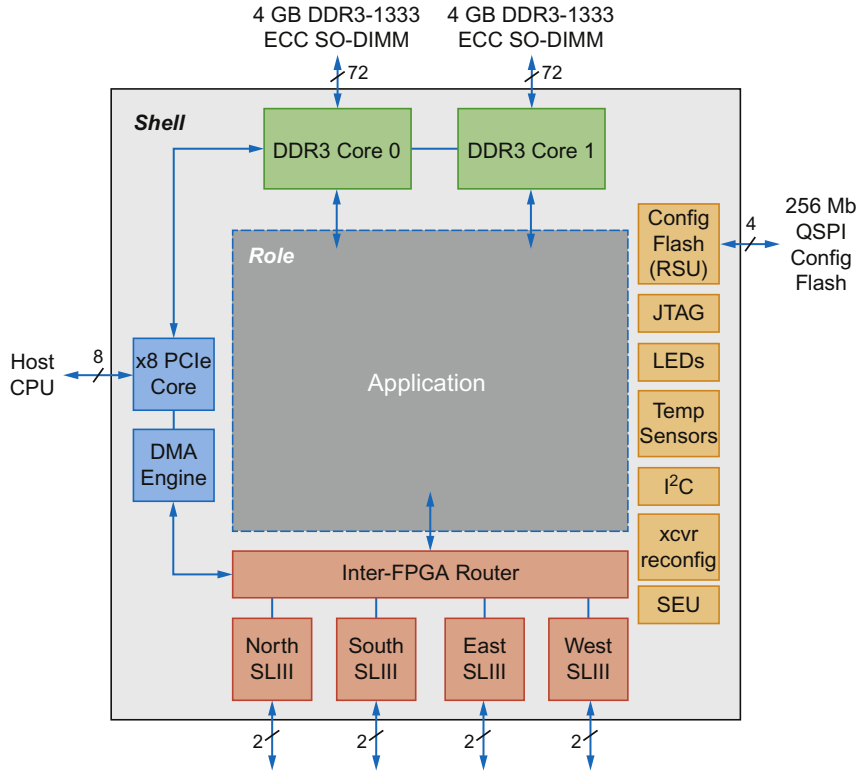


Figure 7.20 Components of Catapult shell and role split of the RTL code.

CNNs on Catapult

Microsoft developed a configurable CNN accelerator as an application for Catapult. Configuration parameters include the number of neural network layers, the dimension of those layers, and even the numerical precision to be used. Figure 7.21 shows the block diagram of the CNN accelerator. Its key features are:

- Run-time configurable design, without requiring recompilation using the FPGA tools.
- To minimize memory accesses, it offers efficient buffering of CNN data structures (see Figure 7.21).
- A two-dimensional array of Processing Elements (PEs) that can scale up to thousands of units.

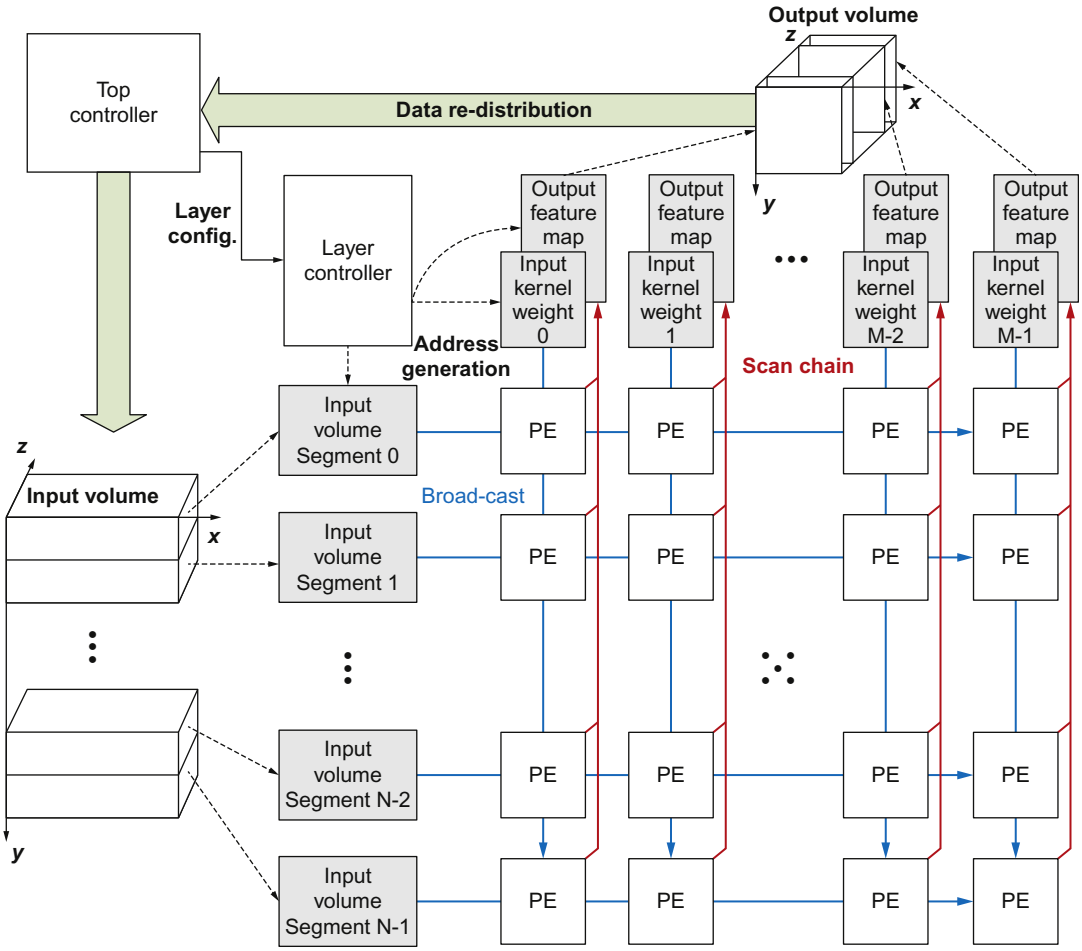


Figure 7.21 CNN Accelerator for Catapult. The Input Volume of the left correspond to Layer[$i-1$] on the left of Figure 7.20, with NumFM[$i-1$] corresponding to y and DimFM[$i-1$] corresponding to *z*. Output Volume at the top maps to Layer[i], with z mapping to NumFM[i] and DimFM[i] mapping to *x*. The next figure shows the inside of the Processing Element (PE).

Images are sent to DRAM and then input into a multibank buffer in the FPGA. The inputs are sent to multiple PEs to perform the stencil computations that produce the output feature maps. A controller (upper left in Figure 7.21) orchestrates the flow of data to each PE. The final results are then recirculated to the input buffers to compute the next layer of the CNN.

Like the TPU, the PEs are designed to be used as a systolic array. Figure 7.22 shows the details of the PE design.

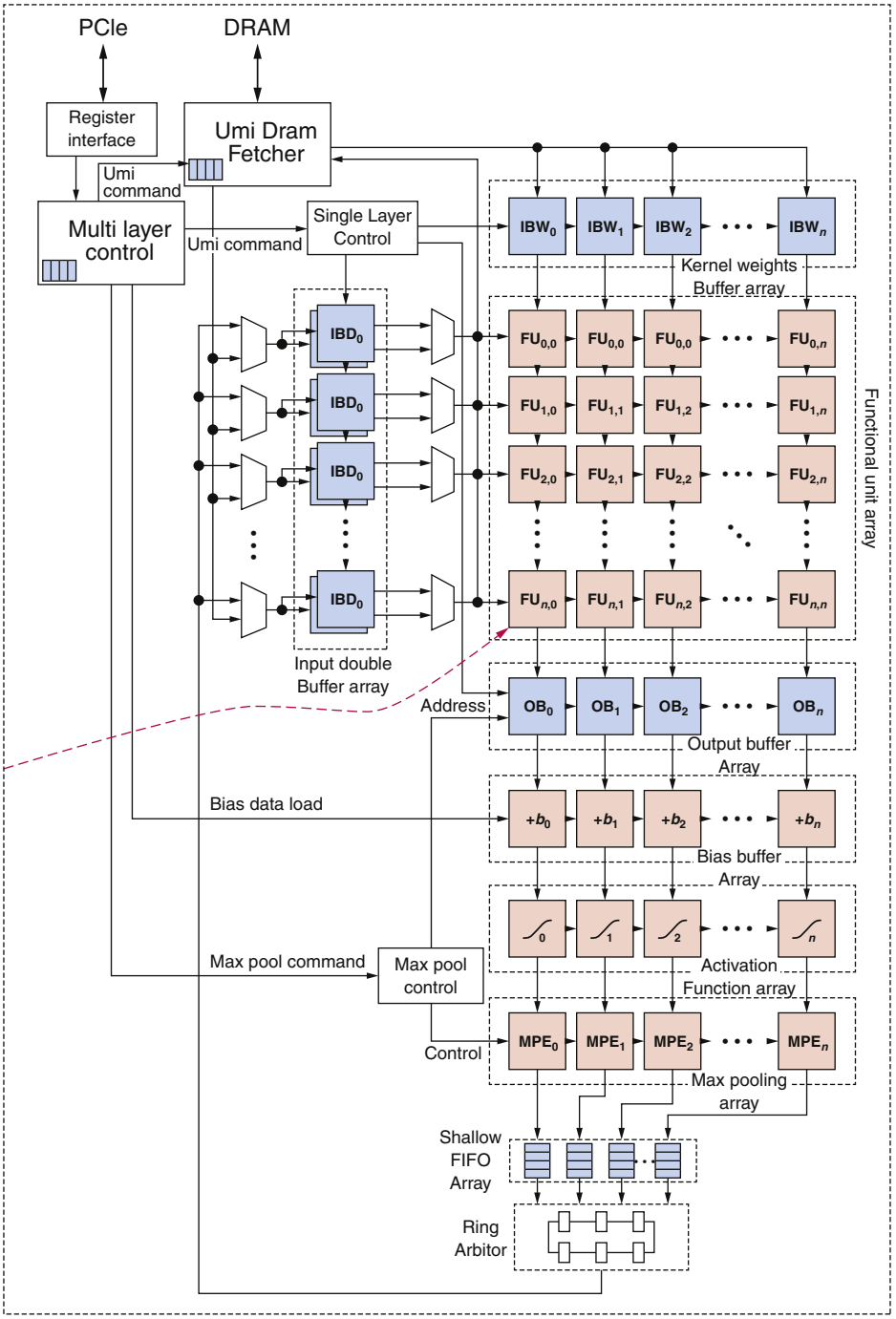


Figure 7.22 The Processing Element (PE) of the CNN Accelerator for Catapult in Figure 7.21. The two-dimension Functional Units (FU) consist of just an ALU and a few registers.

Search Acceleration on Catapult

The primary application to test the return on investment of Catapult was a critical function of the Microsoft Bing search engine called *ranking*. It ranks the order of the results from a search. The output is a document score, which determines the position of the document on the web page that is presented to the user. The algorithm has three stages:

1. *Feature Extraction* extracts thousands of interesting features from a document based on the search query, such as the frequency that the query phrase appears in a document.
2. *Free-Form Expressions* calculates thousands of combinations of features from the prior stage.
3. *Machine-Learned Scoring* uses machine-learning algorithms to evaluate the features from the first two stages to calculate a floating-point score of a document that is returned to the host search software.

The Catapult implementation of ranking produces identical results to equivalent Bing software, even reproducing known bugs!

Taking advantage of one of the preceding guidelines, the ranking function does not have to fit within a single FPGA. Here is how the ranking stages are split across eight FPGAs:

- One FPGA does Feature Extraction.
- Two FPGAs do Free-Form Expressions.
- One FPGA does a compression stage that increases scoring engine efficiency.
- Three FPGA do Machine-Learned Scoring.

The remaining FPGA is a spare used to tolerate faults. Using multiple FPGAs for one application works well because of the dedicated FPGA network.

Figure 7.23 shows the Feature Extraction stage organization. It uses 43 feature-extraction state machines to compute in parallel 4500 features per document-query pair.

Next is the following Free-Form Expressions stage. Rather than implement the functions directly in gates or in state machines, Microsoft developed a 60-core processor that overcomes long-latency operations with multithreading. Unlike a GPU, Microsoft's processor does not require SIMD execution. It has three features that let it match the latency target:

1. Each core supports four simultaneous threads where one can stall on a long operation but the others can continue. All functional units are pipelined, so they can accept a new operation every clock cycle.
2. Threads are statically prioritized using a priority encoder. Expressions with the longest latency use thread slot 0 on all cores, then the next slowest is in slot 1 on all cores, and so on.

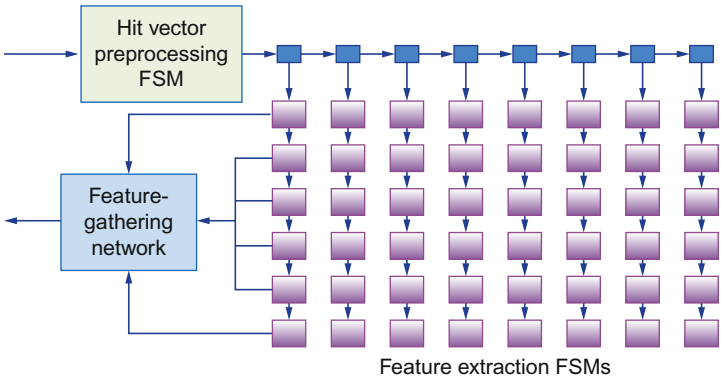


Figure 7.23 The architecture of FPGA implementation of the Feature Extraction stage. A hit vector, which describes the locations of query words in each document, is streamed into the hit vector preprocessing state machine and then split into control and data tokens. These tokens are issued in parallel to the 43 unique feature state machines. The feature-gathering network collects generated feature and value pairs and forwards them to the following Free-Form Expressions stage.

3. Expressions that are too large to fit in the time allocated for a single FPGA can be split across the two FPGAs used for free-form expressions.

One cost of the reprogrammability in an FPGA is a slower clock rate than custom chips. Machine-Learned Scoring uses two forms of parallelism to try to overcome that disadvantage. The first is to have a pipeline that matches the available pipeline parallelism in the application. For ranking, the limit is 8 µs per stage. The second version of parallelism is the rarely seen *multiple instruction streams, single data stream (MISD)* parallelism, where a large number of independent instruction streams operate in parallel on a single document.

Figure 7.24 shows the performance of the ranking function on Catapult. As we will see in Section 7.9, user-facing applications often have rigid response times; it doesn't matter how high the throughput is if the application misses the deadline. The *x*-axis shows the response-time limit, with 1.0 as the cutoff. At this maximum latency, Catapult is 1.95 times as fast as the host Intel server.

Catapult Version 1 Deployment

Before populating a whole warehouse-scale computer with tens of thousands of servers, Microsoft did a test deployment of 17 full racks, which contained $17 \times 48 \times 2$ or 1632 Intel servers. The Catapult cards and network links were tested at manufacture and system integration, but at deployment, seven of the 1632 cards failed (0.43%), and one of the 3264 FPGA network links (0.03%) was defective. After several months of deployment, nothing else failed.

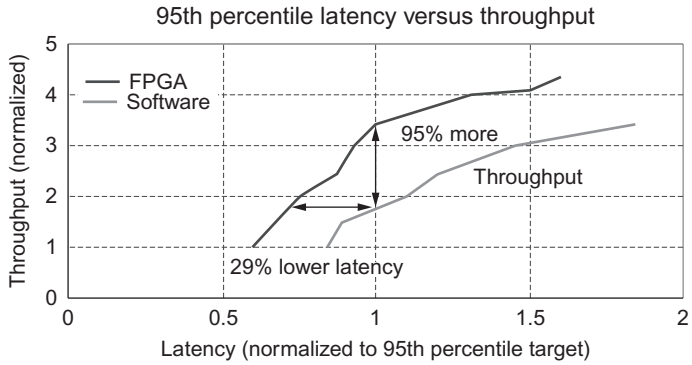


Figure 7.24 Performance for the ranking function on Catapult for a given latency bound. The *x*-axis shows the response time for the Bing ranking function. The maximum response time at the 95th percentile for the Bing application on the *x*-axis is 1.0, so data points to the right may have a higher throughput but arrive too late to be useful. The *y*-axis shows the 95% throughputs on Catapult and pure software for a given response time. At a normalized response time of 1.0, Catapult has 1.95 the throughput of Intel server running in pure software mode. Stated alternatively, if Catapult matches the throughput that the Intel server has at 1.0 normalized response time, Catapult's response time is 29% less.

Catapult Version 2

Although the test deployment was successful, Microsoft changed the architecture for the real deployment to enable both Bing and Azure Networking to use the same boards and architecture (Caulfield et al., 2016). The main problem with the V1 architecture was that the independent FPGA network did not enable the FPGA to see and process standard Ethernet/IP packets, which prevented it from being used to accelerate the data center network infrastructure. In addition, the cabling was expensive and complicated, it was limited to 48 FPGAs, and the rerouting of traffic during certain failure patterns reduced performance and could isolate nodes.

The solution was to place the FPGA logically between the CPU and NIC, so that all network traffic goes through the FPGA. This "bump-on-a-wire" placement removes many weaknesses of the FPGA network in Catapult V1. Moreover, it enables the FPGAs to run their own low-latency network protocol that allows them to be treated as a global pool of all the FPGAs in the data center and even across data centers.

Three changes occurred between V1 and V2 to overcome the original concerns of Catapult applications interfering with data center network traffic. First, the data center network was upgraded from 10 Gbit/s to 40 Gbit/s, increasing the headroom. Second, Catapult V2 added a rate limiter for FPGA logic, ensuring that an FPGA application could not overwhelm the network. The final and perhaps most

important change was that the networking engineers would now had their own use cases for the FPGA, given its bump-in-the-wire placement. That placement transformed these former interested bystanders into enthusiastic collaborators.

By deploying Catapult V2 in the majority of its new servers, Microsoft essentially has a second supercomputer composed of distributed FPGAs that shares the same network wires as the CPU servers and is at the same scale, as there is one FPGA per server. Figures 7.25 and 7.26 show the block diagram and the board for Catapult V2.

Catapult V2 follows the same shell and role split of the RTL to simplify programming, but at the time of publication, the shell uses almost half of the FPGA resources (44%) because of the more complicated network protocol that shares the data center network wires.

Catapult V2 is used for both Ranking acceleration and function network acceleration. In Ranking acceleration, rather than perform nearly all of the ranking function inside the FPGA, Microsoft implemented only the most compute-intensive portions and left the rest to the host CPU:

- The *feature functional unit (FFU)* is a collection of finite state machines that measure standard features in search, such as counting the frequency of a particular search term. It is similar in concept to the Feature Extraction stage of Catapult V1.

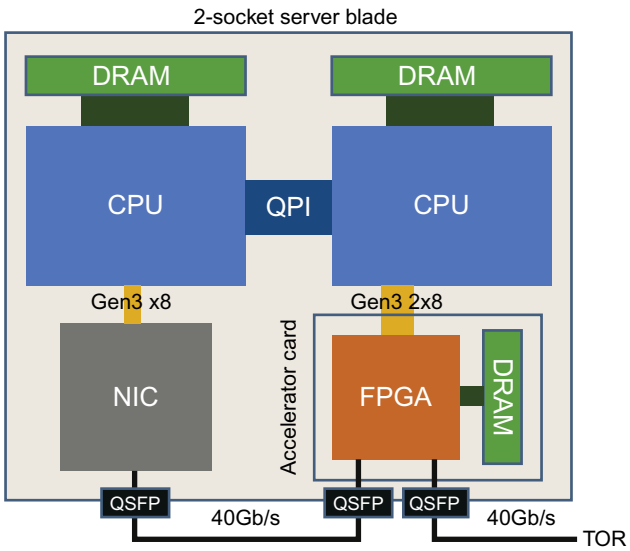


Figure 7.25 The Catapult V2 block diagram. All network traffic is routed through the FPGA to the NIC. There is also a PCIe connector to the CPUs, which allows the FPGA to be used as a local compute accelerator, as in Catapult V1.

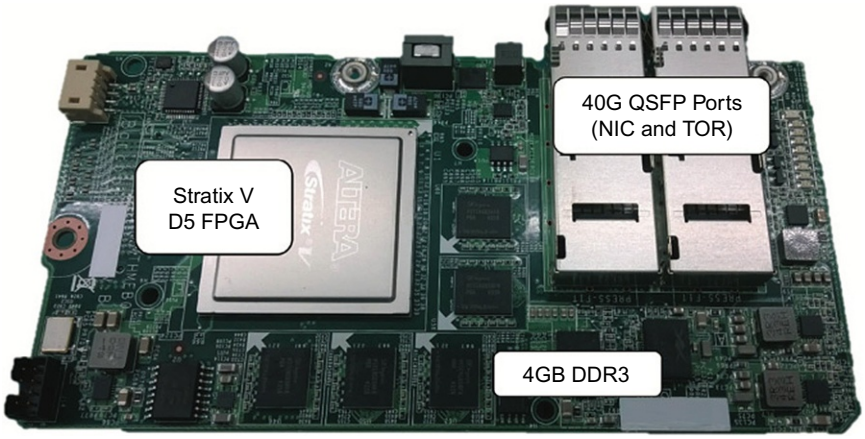


Figure 7.26 The Catapult V2 board uses a PCIe slot. It uses the same FPGA as Catapult V1 and has a TDP of 32 W. A 256-MB Flash chip holds the *golden image* for the FPGA that is loaded at power on, as well as one application image.

- The *dynamic programming feature (DPF)* creates a Microsoft proprietary set of features using dynamic programming and bears some similarity to the Free-Form Expressions stage of Catapult V1.

Both are designed so that they can use non-local FPGAs for these tasks, which simplifies scheduling.

Figure 7.27 shows the performance of Catapult V2 compared to software in a format similar to Figure 7.24. The throughput can now be increased 2.25× without endangering latency, whereas the speedup was previously 1.95×. When ranking was deployed and measured in production, Catapult V2 had better tail latencies than software; that is, the FPGA latencies never exceeded the software latencies at any given demand despite being able to absorb twice the workload.

Summary: How Catapult Follows the Guidelines

Microsoft reported that adding Catapult V1 to the servers in the pilot phase increased the total cost of ownership (TCO) by less than 30%. Thus, for this application, the net gain in cost-performance for Ranking was at least 1.95/1.30, or a return on investment of about 1.5. Although no comment was made about TCO concerning Catapult V2, the board has a similar number of the same type of chips, so we might guess that the TCO is no higher. If so, the cost-performance of Catapult V2 is about 2.25/1.30, or 1.75 for Ranking.

Here is how Catapult followed the guidelines from Section 7.2.

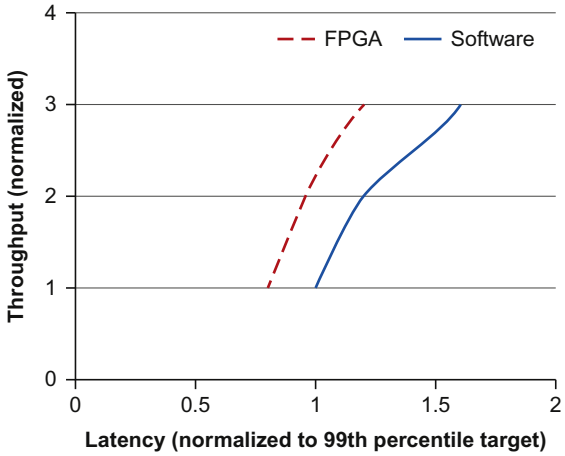


Figure 7.27 Performance for the ranking function on Catapult V2 in the same format as Figure 7.24. Note that this version measures 99th percentile while the earlier figure plots 95th percentile.

1. *Use dedicated memories to minimize the distance over which data is moved.* The Altera V FPGA has 5 MiB of memory on-chip, which an application can customize for its use. For example, for CNNs, it is used for the input and output feature maps of Figure 7.21.
2. *Invest the resources saved from dropping advanced microarchitectural optimizations into more arithmetic units or bigger memories.* The Altera V FPGA also has 3926 18-bit ALUs that are tailored to the application. For CNNs, they are used to create the systolic array that drives the Processing Elements in Figure 7.22, and they form the datapaths of the 60-core multiprocessor used by Free Form Expression stage of ranking.
3. *Use the easiest form of parallelism that matches the domain.* Catapult picks the form of parallelism that matches the application. For example, Catapult uses two-dimensional SIMD parallelism for the CNN application and MISD parallelism in the Machine Scoring phase stream Ranking.
4. *Reduce data size and type to the simplest needed for the domain.* Catapult can use whatever size and type of data that the application wants, from an 8-bit integer to a 64-bit floating point.
5. *Use a domain-specific programming language to port code to the DSA.* In this case, programming is done in the hardware register-transfer language (RTL) Verilog, which is an even less productive language than C or C++. Microsoft did not (and possibly could not) follow this guideline given its use of FPGAs.

Although this guideline concerns the one-time porting of an application from software to FPGA, applications are not frozen in time. Almost by definition, software evolves to add features or fix bugs, especially for something as important as web search.

Maintenance of successful programs can be most of software's development costs. Moreover, when programming in an RTL, software maintenance is even more burdensome. The Microsoft developers, like all others who use FPGAs as accelerators, hope that future advances in domain-specific languages and systems for hardware-software co-design will reduce the difficulty of programming FPGAs.

7.6 Intel Crest, a Data Center Accelerator for Training

The quotation by the Intel CEO that opens Section 7.3 came from the press release announcing that Intel was going to start shipping DSAs ("accelerants") for DNN. The first example was Crest, which was announced while we were writing this edition. Despite the limited details, we include it here because of the significance of a traditional microprocessor manufacturer like Intel taking this bold step of embracing DSAs.

Crest is aimed at DNN training. The Intel CEO said the goal is to accelerate DNN training a hundredfold over the next three years. Figure 7.6 shows that training can take a month. There is likely to be a demand to decrease the DNN training to just eight hours, which would be 100 times quicker than the CEO predicted. DNNs will surely become even more complex over the next 3 years, which will require a much greater training effort. Thus there seems little danger that a 100× improvement in training is overkill.

Crest instructions operate on blocks of 32 × 32 matrices. Crest uses a number format called *flex point*, which is a scaled fixed-point representation: 32 × 32 matrices of 16-bit data share a single 5-bit exponent that is provided as part of the instruction set.

Figure 7.28 shows the block diagram of the Lake Crest chip. To compute on these matrices, Crest uses the12 processing clusters of Figure 7.28. Each cluster includes a large SRAM, a big linear algebra processing unit, and a small amount of logic for on- and off-chip routing. The four 8 GiB HBM2 DRAM modules offer 1 TB/s of memory bandwidth, which should lead to an attractive Roofline model for the Crest chip. In addition to high-bandwidth paths to main memory, Lake Crest supports high bandwidth interconnects directly between compute cores inside the processing clusters, which facilitates quick core-to-core communication without passing through shared memory. Lake Crest's goal is a factor of 10 improvement in training over GPUs.

Figure 7.28 shows 12 Inter-Chip Links (ICLs) and 2 Inter-Chip Controllers (ICCs), so Crest is clearly designed to allow many Crest chips to collaborate, similar in spirit to the dedicated network connecting the 48 FPGAs in Catapult. It's likely that the 100× improvement in training will require ganging together several Crest chips.

7.7 Pixel Visual Core, a Personal Mobile Device Image Processing Unit

Pixel Visual Core is a programmable, scalable DSA intended for image processing and computer vision from Google, initially for cell phones and tablets running the

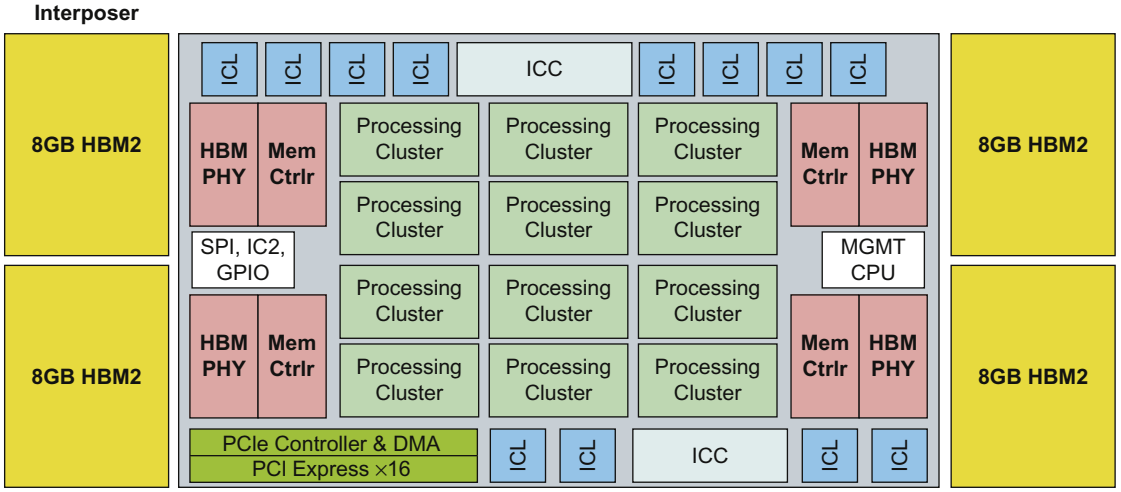


Figure 7.28 Block diagram of the Intel Lake Crest processor. Before being acquired by Intel, Crest said that the chip is almost a full reticle in TSMC 28 nm, which would make the die size 600–700 mm^2. This chip should be available in 2017. Intel is also building Knights Crest, which is a hybrid chip containing Xeon x86 cores and Crest accelerators.

Android operating system, and then potentially for Internet of Things (IoT) devices. It is a multicore design, supporting between 2 and 16 cores to deliver a desired cost-performance. It is designed either to be its own chip or to be part of a *system on a chip* (*SOC*). It has a much smaller area and energy budget than its TPU cousin. Figure 7.29 lists terms and acronyms found in this section.

Pixel Visual Core is an example of a new class of domain specific architectures for vision processing that we call *image processing units* (*IPUs*). IPUs solve the inverse problem of GPUs: they analyze and modify an input image in contrast to generating an output image. We call them IPUs to signal that, as a DSA, they do not need to do everything well because there will also be CPUs (and GPUs) in the system to perform non-input-vision tasks. IPUs rely on stencil computations mentioned above for CNNs.

The innovations of Pixel Visual Core include replacing the one-dimensional SIMD unit of CPUs with a two-dimensional array of processing elements (PEs). They provide a two-dimensional shifting network for the PEs that is aware of the two-dimensional spatial relationship between the elements, and a two-dimensional version of buffers that reduces accesses to off-chip memory. This novel hardware makes it easy to perform stencil computations that are central to both vision processing and CNN algorithms.

ISPs, the Hardwired Predecessors of IPUs

Most portable mobile devices (PMDs) have multiple cameras for input, which has led to hardwired accelerators called *image signal processors* (*ISPs*) for enhancing

Term	Acronym	Short explanation
Core	–	A processor. Pixel Visual Core can have 2–16 cores. The first implementation has 8; also called *stencil processor (STP)*
Halide	–	A domain-specific programming language for image processing that separates the algorithm from its execution schedule
Halo	–	An extended region around the 16 × 16 computation array to handle stencil computation near the borders of the array. It holds values, but doesn't compute
Image signal processors	ISP	A fixed function ASIC that improves the visual quality of an image; found in virtually all PMDs with cameras
Image processing unit	IPU	A DSA that solves the inverse problem of a GPU: it analyzes and modifies an *input* image in contrast to generating an *output* image
Line buffer pool	LB	A line buffer is designed to capture a sufficient number of full lines of an intermediate image to keep the next stage busy. Pixel Visual Core uses two-dimensional line buffers, each Change 64 to 128 KiB. The *Line Buffer Pool* contains one LB per core plus one LB for DMA
Network on chip	NOC	The network that connects the cores in Pixel Visual Core
Physical ISA	pISA	The Pixel Visual Core instruction set architecture (ISA) that is executed by the hardware
Processing element array	–	The 16 × 16 array of Processing Elements plus the halo that performs the 16-bit multiply-add operations. Each Processing Element includes a Vector Lane and local memory. It can shift data en mass to neighbors in any of four directions
Sheet generator	SHG	Does memory accesses of blocks of 1 × 1 to 31 × 31 pixels, which are called *sheets*. The different sizes allow the option of including the space for the halo or not
Scalar lane	SCL	Same operations as the Vector Lane except it adds instructions that handle jumps, branches, and interrupts, controls instruction flow to the vector array, and schedules all the loads and stores for the sheet generator. It also has a small instruction memory. It plays the same role as the scalar processor in a vector architecture
Vector lane	VL	Portion of the Processing Element that performs the computer arithmetic
Virtual ISA	vISA	The Pixel Visual Core ISA generated by the compiler. It is mapped to pISA before execution

Figure 7.29 A handy guide to Pixel Visual Core terms in Section 7.7. Figure 7.4 on page 437 has a guide for Sections 7.3–7.6.

input images. The ISP is usually a fixed function ASIC. Virtually every PMD today includes an ISP.

Figure 7.30 shows a typical organization of an image-processing system, including the lens, sensor, ISP, CPU, DRAM, and display. The ISP receives images, removes artifacts in images from the lens and the sensor, interpolates missing colors, and significantly improves the overall visual quality of the image. PMDs tend to have small lens and thus tiny noisy pixels, so this step is critical to producing high-quality photos and videos.

An ISP processes the input image in raster scan order by calculating a series of cascading algorithms via software configurable hardware building blocks, typically organized as a pipeline to minimize memory traffic. At each stage of the pipeline and for each clock cycle, a few pixels are input, and a few are output. Computation is typically performed over small neighborhoods of pixels (*stencils*). Stages are connected by buffers called *line buffers*. The line buffers help

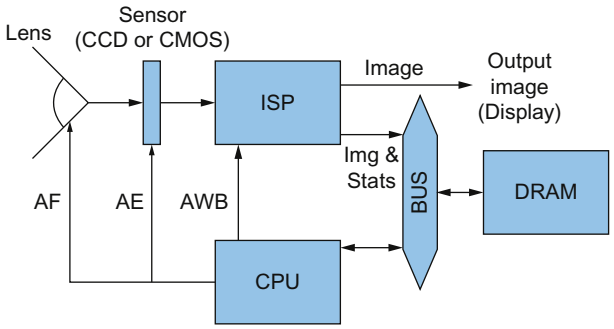


Figure 7.30 Diagram showing interconnection of the Image Signal Processor (ISP), CPU, DRAM, lens, and sensor. The ISP sends statistics to the CPU as well as the improved image either to the display or to DRAM for storage or later processing. The CPU then processes the image statistics and sends information to let the system adapt: *Auto White Balance* (AWB) to the ISP, *Auto Exposure* (AE) to the sensor, and *Auto Focus* (AF) to the lens, known as the *3As*.

keep the processing stages utilized via spatial locality by capturing just enough full lines of an intermediate image to facilitate the computation required by the next stage.

The enhanced image is either sent to a display or to DRAM for storage or for later processing. The ISP also sends statistics about the image (e.g., color and luma histograms, sharpness, and so on) to the CPU, which in turn it processes and sends information to help the system adapt.

Although efficient, ISPs have two major downsides. Given the increasing demand for improved image quality in handheld devices, the first is the inflexibility of an ISP, especially as it takes years to design and manufacture a new ISP within an SOC. The second is that these computing resources can be used only for the image-enhancing function, no matter what is needed at the time on the PMD. Current generation ISPs handle workloads at up to 2 Tera-operations per second on a PMD power budget, so a DSA replacement has to achieve similar performance and efficiency.

Pixel Visual Core Software

Pixel Visual Core generalized the typical hardwired pipeline organization of kernels of an ISP into a *directed acyclic graph* (*DAG*) of kernels. Pixel Visual Core image-processing programs are typically written in Halide, which is a domain-specific functional programming language for image processing. Figure 7.31 is a Halide example that blurs an image. Halide has a functional section to express the function being programmed and a separate schedule section to specify how to optimize that function to the underlying hardware.

```
Func buildBlur(Func input) {
      // Functional portion (independent of target processor)
    Func blur_x("blur_x"), blur_y("blur_y");
    blur_x(x,y) = (input(x−1,y) + input(x,y)*2 + input(x+1,y)) / 4;
    blur_y(x,y) = (blur_x(x,y−1) + blur_x(x,y)*2 + blur_x(x,y+1)) / 4;

    if (has_ipu) {
      // Schedule portion (directs how to optimize for target processor)
      blur_x.ipu(x,y);
      blur_y.ipu(x,y);
}
    return blur_y;
}
```

Figure 7.31 Portion of a Halide example to blur an image. The `ipu(x,y)` suffix schedules the function to Pixel Visual Core. A blur has the effect of looking at the image through a translucent screen, which reduces noise and detail. A Gaussian function is often used to blur the image.

Pixel Visual Core Architecture Philosophy

The power budget of PMDs is 6–8 W for bursts of 10–20 seconds, dropping down to tens of milliwatts when the screen is off. Given the challenging energy goals of a PMD chip, the Pixel Visual Core architecture was strongly shaped by the relative energy costs for the primitive operations mentioned in Chapter 1 and made explicit in Figure 7.32. Strikingly, a single 8-bit DRAM access takes as much energy as 12,500 8-bit additions or 7–100 8-bit SRAM accesses, depending on the organization of the SRAM. The 22× to 150× higher cost of IEEE 754 floating-point operations over 8-bit integer operations, plus the die size and energy benefits of storing narrower data, strongly favor using narrow integers whenever algorithms can accommodate them.

In addition to the guidelines from Section 7.2, these observations led to other themes that guided the Pixel Visual Core design:

- *Two-dimensional is better than one-dimensional*: Two-dimensional organizations can be beneficial for processing images as it minimizes communication distance and because the two- and three-dimensional nature of image data can utilize such organizations.
- *Closer is better than farther*: Moving data is expensive. Moreover, the relative cost of moving data to an ALU operation is increasing. And of course DRAM time and energy costs far exceed any local data storage or movement.

A primary goal in going from an ISP to an IPU is to get more reuse of the hardware via programmability. Here are the three main features of the Pixel Visual Core:

Operation	*Energy (pJ)*	Operation	Energy (pJ)	Operation	Energy (pJ)
8b DRAM LPDDR3	125.00	8b SRAM	1.2–17.1	16b SRAM	2.4–34.2
32b Fl. Pt. muladd	2.70	8b int muladd	0.12	16b int muladd	0.43
32b Fl. Pt. add	1.50	8b int add	0.01	16b int add	0.02

Figure 7.32 Relative energy costs per operation in picoJoules assuming TSMC 28-nm HPM process, which was the process Pixel Visual Core used [17][18][19][20]. The absolute energy cost are less than in Figure 7.2 because of using 28 nm instead of 90 nm, but the relative energy costs are similarly high.

1. Following the theme that two-dimensional is better than one-dimensional, Pixel Visual Core uses a two-dimensional SIMD architecture instead of one-dimensional SIMD architecture. Thus it has a two-dimensional array of independent *processing elements* (*PEs*), each of which contains 2 16-bit ALUs, 1 16-bit MAC unit, 10 16-bit registers, and 10 1-bit predicate registers. The 16-bit arithmetic follows the guideline of providing only the precision needed by the domain.
2. Pixel Visual Core needs temporary storage at each PE. Following the guideline from Section 7.2 of avoiding caches, this PE memory is a compiler-managed scratchpad memory. The logical size of each PE memory is 128 entries of 16 bits, or just 256 bytes. Because it would be inefficient to implement a separate small SRAM in each PE, Pixel Visual Core instead groups the PE memory of 8 PEs together in a single wide SRAM block. Because the PEs operate in SIMD fashion, Pixel Visual Core can bind all the individual reads and writes together to form a "squarer" SRAM, which is more efficient than narrow and deep or wide and shallow SRAMs. Figure 7.33 shows four PEs.
3. To be able to perform simultaneous stencil computations in all PEs, Pixel Visual Core needs to collect inputs from nearest neighbors. This communication pattern requires a "NSEW" (North, South, East, West) shift network: it can shift data en masse between the PEs in any compass direction. So that it doesn't lose pixels along the edges as it shifts images, Pixel Visual Core connects the endpoints of the network together to form a torus.

Note that the shift network is in contrast with the systolic array of processing element arrays in the TPU and Catapult. In this case, software explicitly moves the data in the desired direction across the array, whereas the systolic approach is a hardware-controlled, two-dimensional pipeline that moves data as a wavefront that is invisible to the software.

The Pixel Visual Core Halo

A 3×3, 5×5, or 7×7 stencil is going to get inputs from 1, 2, or 3 external pixels at the edges of the two-dimensional subset being computed (half of the dimension of the stencil minus one-half). That leaves two choices. Either Pixel Visual Core

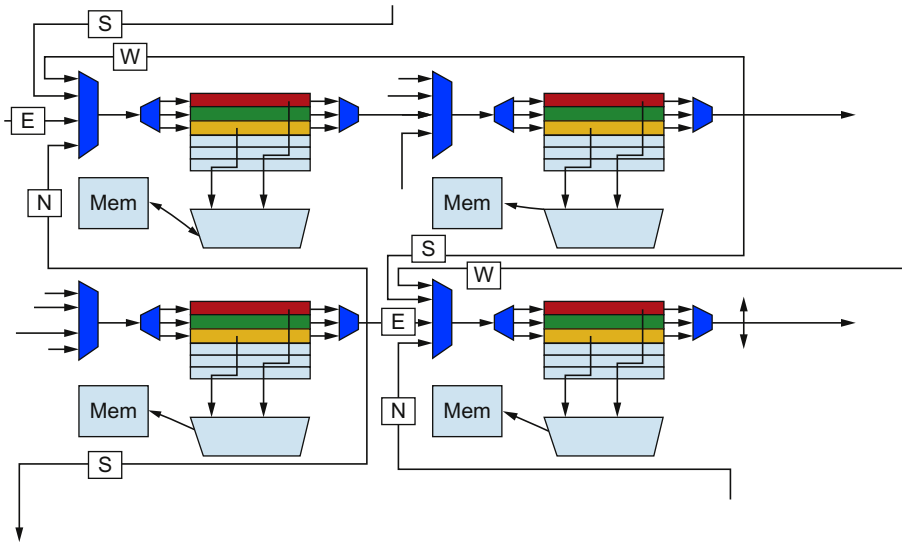


Figure 7.33 The two-dimensional SIMD includes two-dimensional shifting "N," "S," "E," "W," show the direction of the shift (North, South, East, West). Each PE has a software-controlled scratchpad memory.

under utilizes the hardware in the elements near the border, because they only pass input values, or Pixel Visual Core slightly extends the two-dimensional PEs with simplified PEs that leave out the ALUs. Because the difference in size between a standard PE and a simplified PE is about 2.2 ×, Pixel Visual Core has an extended array. This extended region is called the *halo*. Figure 7.34 shows two rows of a halo surrounding an 8x8 PE array and illustrates how an example 5 × 5 stencil computation in the upper-left corner relies on the halo.

A Processor of the Pixel Visual Core

The collection of 16 × 16 PEs and 4 halo lanes in each dimension, called the *PE array* or *vector array*, is the main computation unit of the Pixel Visual Core. It also has a load-store unit called a *Sheet Generator* (*SHG*). SHG refers to memory accesses of blocks of 1 × 1 to 256 × 256 pixels, which are called *sheets*. This happens during downsampling, and typical values are 16 × 16 or 20 × 20.

An implementation of Pixel Visual Core can have any even number of 2 or more cores, depending on the resources available. Thus it needs a network to connect them together, so every core also has an interface to the Network on Chip (NOC). A typical NOC implementation for Pixel Visual Core will not be an expensive cross switch, however, because those require data to travel a long distance, which is expensive. Leveraging the pipeline nature of the application, the NOC typically needs to communicate only to neighboring cores. It is implemented as a two-dimensional mesh, which allows power gating of pairs of cores under software control.

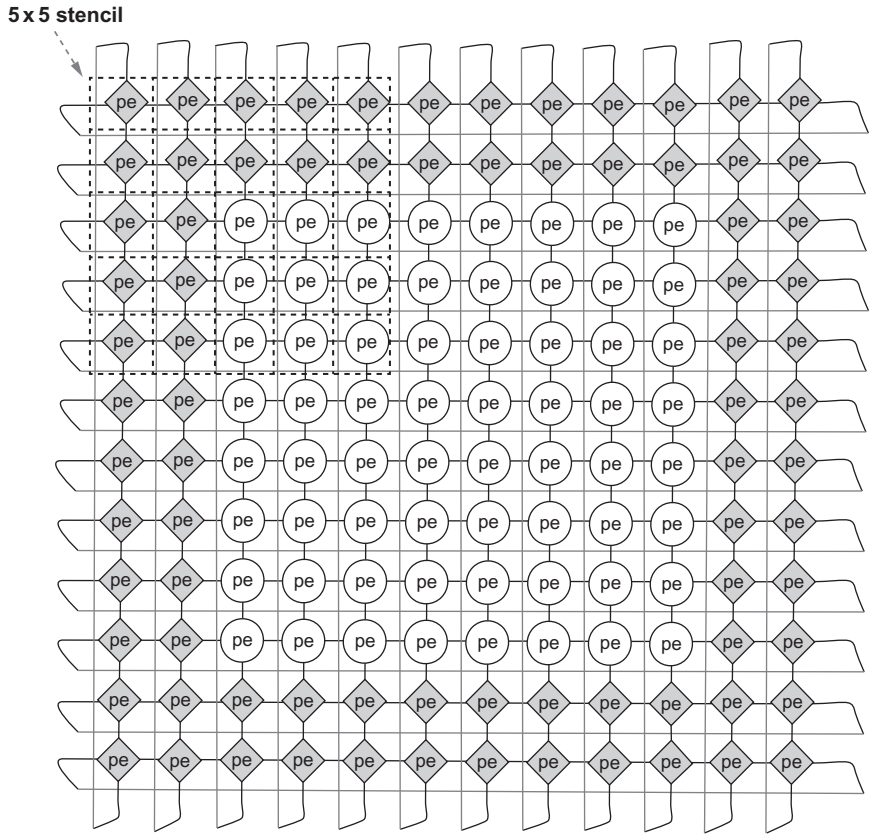


Figure 7.34 The two-dimensional array of full processing elements *(shown as unshaded circles)* surrounded by two layers of simplified processing elements *(shaded diamonds)* called a *halo*. In this figure, there are 8×8 or 64 full PEs with 80 simplified PEs in the halo. (Pixel Visual Core actually has 16×16 or 256 full PEs and two layers in its halo and thus 144 simplified PEs.) The edges of the halo are connected *(shown as gray lines)* to form a torus. Pixel Visual Core does a series of two-dimensional shifts across all processing elements to move the neighbor portions of each stencil computation into the center PE of the stencil. An example 5×5 stencil is shown in the upper-left corner. Note that 16 of the 25 pieces of data for this 5×5 stencil location come from halo processing elements.

Finally, the Pixel Visual Core also includes a scalar processor that is called a *scalar lane* (*SCL*). It is identical to the vector lane, except it adds instructions that handle jumps, branches, and interrupts, controls instruction flow to the vector array, and schedules all the loads and stores for the sheet generator. It also has a small instruction memory. Note that Pixel Visual Core has a single instruction stream that controls the scalar and vector units, similar to how a CPU core has a single instruction stream for its scalar and SIMD units.

In addition to cores, there is also a DMA engine to transfer data between DRAM and the line buffers while efficiently converting between image memory layout formats (e.g., packing/unpacking). As well as sequential DRAM accesses, the DMA engines perform vector-like gather reads of DRAM as well as sequential and strided reads and writes.

Pixel Visual Core Instruction Set Architecture

Like GPUs, Pixel Visual Core uses a two-step compilation process. The first step is compiling the programs from the target language (e.g., Halide) into *vISA* instructions. The Pixel Visual Core *vISA* (*virtual Instruction Set Architecture*) is inspired in part by the RISC-V instruction set, but it uses an image-specific memory model and extends the instruction set to handle image processing, and in particular, the two-dimensional notion of images. In vISA, the two-dimensional array of a core is infinite, the number of register is unbounded, and memory size is similarly unlimited. vISA instructions contain pure functions that don't directly access DRAM (see Figure 7.36), which greatly simplifies mapping them onto the hardware.

The next step is to compile the vISA program into a *pISA* (*physical Instruction Set Architecture*) program. Using vISA as the target of compilers allows the processor to be software-compatible with past programs and yet accept changes to the pISA instruction set, so vISA plays the same role that PTX does for GPUs (see Chapter 4).

Lowering from vISA to pISA takes two steps: compilation and mapping with early-bound parameters, and then patching the code with late-bound parameters. The parameters that must be bound include STP size, halo size, number of STPs, mapping of line buffers, mapping of kernels to processors, as well as register and local memory allocations.

Figure 7.35 shows that pISA is a very long instruction word (VLIW) instruction set with 119-bit-wide instructions. The first 43-bit field is for the Scalar Lane, the next 38-bit field specifies the computation by the two-dimensional PE array, and the third 12-bit field specifies the memory accesses by the two-dimensional PE array. The last two fields are immediates for computation or addressing. The operations for all the VLIW fields are what you'd expect: two's complement integer arithmetic, saturating integer arithmetic, logical operations, shifts, data transfers, and a few special ones like divide iteration and count leading zeros. The Scalar Lane supports a superset of the operations in the two-dimensional PE array, plus it adds instructions for control-flow and sheet-generator control. The 1-bit Predicate registers mentioned above enables conditional moves to registers (e.g., $A = B$ if C).

Field	Scalar	Math	Memory	Imm	MemImm
# Bits	43	38	12	16	10

Figure 7.35 VLIW format of the 119-bit pISA instruction.

Although the pISA VLIW instruction is very wide, Halide kernels are short, often just 200–600 instructions. Recall that as an IPU, it only needs to execute the compute-intensive portion of an application, leaving the rest of the functionality to CPUs and GPUs. Thus the instruction memory of a Pixel Visual Core holds just 2048 pISA instructions (28.5 KiB).

The Scalar Lane issues sheet generator instructions that access line buffers. Unlike other memory accesses within Pixel Visual Core, the latency can be more than 1 clock cycle, so they have a DMA-like interface. The lane first sets up the addresses and transfer size in special registers.

Pixel Visual Core Example

Figure 7.36 shows the vISA code that is output from the Halide compiler for the blur example in Figure 7.31, with comments added for clarity. It calculates a blur first in the *x* direction and then in the y direction using 16-bit arithmetic. The vISA code matches the functional part of the Halide program. This code can be thought of as executing across all the pixels of an image.

Pixel Visual Core Processing Element

One of the architectural decisions was how big to build the halo. Pixel Visual Core uses 16×16 PEs, and it adds a halo of 2 extra elements, so it can support 5×5

```
// vISA inner loop blur in x dimension
input.b16  t1 <- _input[x*1+(-1)][y*1+0][0]; // t1 = input[x-1,y]
input.b16  t2 <- _input[x*1+0][y*1+0][0];  // t2 = input[x,y]
mov.b16    st3 <- 2;
mul.b16    t4 <- t2, st3;  //t4 = input[x,y] * 2
add.b16    t5 <- t1, t4;  //t5 = input[x-1,y] + input[x,y]*2
input.b16  t6 <- _input[x*1+1][y*1+0][0];  // t6 = input[x+1,y]
add.b16    t7 <- t5, t6;  //t7 = input[x+1,y]+input[x,y]+input[x-1,y]*2
mov.b16    st8 <- 4;
div.b16    t9 <- t7, st8;  //t9 = t7/4
output.b16 _blur_x[x*1+0][y*1+0][0] <- t9; // blur_x[x,y] = t7/4
// vISA inner loop blur in y dimension
input.b16  t1 <- _blur_x[x*1+0][y*1+(-1)][0]; // t1 = blur_x[x,y-1]
input.b16  t2 <- _blur_x[x*1+0][y*1+0][0];  // t2 = blur_x[x,y]
mov.b16    st3 <- 2;
mul.b16    t4 <- t2, st3;  //t4 = blur_x[x,y] * 2
add.b16    t5 <- t1, t4;  //t5 = blur_x[x,y-1] + blur_x[x,y]*2
input.b16  t6 <- _blur_x[x*1+0][y*1+1][0];  // t6 = blur_x[x,y+1]
add.b16    t7 <- t5, t6;  //t7 = blurx[x,y+1]+blurx[x,y-1]+blurx[x,y]*2
mov.b16    st8 <- 4;
div.b16    t9 <- t7, st8;  //t9 = t7/4
output.b16 _blur_y[x*1+0][y*1+0][0] <- t9; // blur_y[x,y] = t7/4
```

Figure 7.36 Portion of the vISA instructions compiled from the Halide Blur code in Figure 7.31. This vISA code corresponds to the functional part of the Halide code.

stencils directly. Note that the bigger the array of PEs, the less the halo overhead to support a given stencil size.

For Pixel Visual Core, the smaller size of the halo PEs and the 16×16 arrays means it only costs 20% more area for the halo. For a 5×5 stencil, Pixel Visual Core can calculate 1.8 times as many results per clock cycle ($16^2/12^2$), and the ratio is 1.3 for a 3×3 stencil ($16^2/14^2$).

The design of the arithmetic unit of the PE is driven by multiply-accumulate (MAC), which is a primitive of stencil computation. Pixel Visual Core native MACs are 16-bits wide for the multiplies, but they can accumulate at a 32-bit width. Pipelining MAC would use energy unnecessarily because of the reading and writing of the added pipeline register. Thus the multiply-add hardware determines the clock cycle. The other operations, previously mentioned, are the traditional logical and arithmetic operations along with saturating versions of the arithmetic operations and a few specialized instructions.

The PE has two 16-bit ALUs that can operate in a variety of ways within a single clock cycle:

- Independently, producing two 16-bit results: A op B, C op D.
- Fused, producing just one 16-bit result: A op (C op D).
- Joined, producing one 32-bit result: A:C op B:D.

Two-Dimensional Line Buffers and Their Controller

Because DRAM accesses use so much energy (see Figure 7.32), the Pixel Visual Core memory system was carefully designed to minimize the number of DRAM accesses. The key innovation is the *two-dimensional line buffer*.

Kernels are logically running on separate cores, and they are connected in a DAG with input from the sensor or DRAM and output to DRAM. The line buffers hold portions of the image being calculated between kernels. Figure 7.37 shows the logical use of line buffers in Pixel Visual Core.

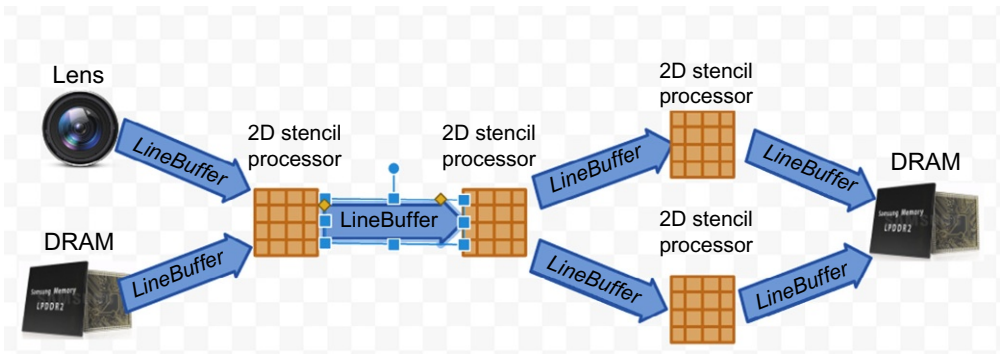


Figure 7.37 Programmer view of Pixel Visual Core: a directed-acyclic graph of kernels.

Here are four features that the two-dimensional line buffer must support:

1. It must support two-dimensional stencil computations of various sizes, which are unknown at design time.
2. Because of the halo, for the 16×16 PE array in Pixel Visual Core, the STPs will want to read 20×20 blocks of pixels from the line buffer and write 16×16 blocks of pixels to the line buffer. (As previously mentioned, they call these blocks of pixels *sheets*.)
3. Because the DAG is programmable, we need line buffers that can be allocated by software between any two cores.
4. Several cores may need to read data from the same line buffer. Thus a line buffer should support multiple consumers, although it needs just one producer.

Line buffers in Pixel Visual Core are really a multi-reader, two-dimensional FIFO abstraction on top of a relatively large amount of SRAM: 128 KiB per instance. It contains temporary "images" that are used just once, so a small, dedicated local FIFO is much more efficient than a cache for data in distant memory.

To accommodate the size mismatch between reading 20×20 blocks of pixels and writing 16×16 blocks, the fundamental unit of allocation in the FIFO is a group of 4×4 pixels. Per stencil processor, there is one *Line Buffer Pool* (*LBP*) that can have eight logical line buffers (*LB*), plus one LBP for DMA of I/O. The LBP has three levels of abstraction:

1. At the top, the LBP controller supports eight LBs as logical instances. Each LB has one FIFO producer and up to eight FIFO consumers per LB.
2. The controller keeps track of a set of head and tail pointers for each FIFO. Note that the sizes of the line buffers inside the LBP are flexible and up to the controller.
3. At the bottom are many physical memory banks to support the bandwidth requirements. Pixel Visual Core has eight physical memory banks, each having a 128-bit interface and 16 KiB of capacity.

The controller for the LBP is challenging because it must fulfill the bandwidth demands of the STPs and I/O DMAs as well as schedule all their reads and writes to the banks of physical SRAM memory. The LBP controller is one of the most complicated pieces of Pixel Visual Core.

Pixel Visual Core Implementation

The first implementation of Pixel Visual Core was as a separate chip. Figure 7.38 shows the floorplan of the chip, which has 8 cores. It was fabricated in a TSMC 28 nm technology in 2016. The chip dimensions are 6×7.2 mm, it runs at 426 MHz, it is stacked with 512 MB DRAM as Silicon in Package, and consumes

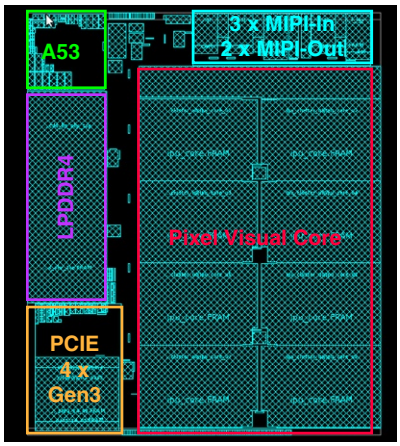


Figure 7.38 Floor plan of the 8-core Pixel Visual Core chip. A53 is an ARMv7 core. LPDDR4 is a DRAM controller. PCIE and MIPI are I/O buses.

(including the DRAM) 187–4500 mW depending on the workload. About 30% of the power for the chip is for an ARMv7 A53 core for control, the MIPI, the PCIe, the PCIe, and the LPDDR interfaces, interface is just over half this die at 23 mm^2. Power for Pixel Visual Core running a worst case "power virus" can go as high as 3200 mW. Figure 7.39 shows the floor plan of a core.

Summary: How Pixel Visual Core Follows the Guidelines

Pixel Visual Core is a multicore DSA for image and vision processing intended as a stand-alone chip or as an IP block for mobile device SOCs. As we will see in Section 7.9, its performance per watt for CNNs are factors of 25–100 better than CPUs and GPUs. Here is how the Pixel Visual core followed the guidelines in Section 7.2.

1. *Use dedicated memories to minimize the distance over which data is moved.* Perhaps the most distinguishing architecture feature of Pixel Visual Core is the software-controlled, two-dimensional line buffers. At 128 KiB per core, they are a significant fraction of the area. Each core also has 64 KiB of software-controlled PE memory for temporary storage.
2. *Invest the resources saved from dropping advanced microarchitectural optimizations into more arithmetic units or bigger memories.* Two other key features of Pixel Visual Core are a 16×16 two-dimensional array of processing elements per core and a two-dimensional shifting network between the processing elements. It offers a halo region that acts as a buffer to allow full utilization of its 256 arithmetic units.

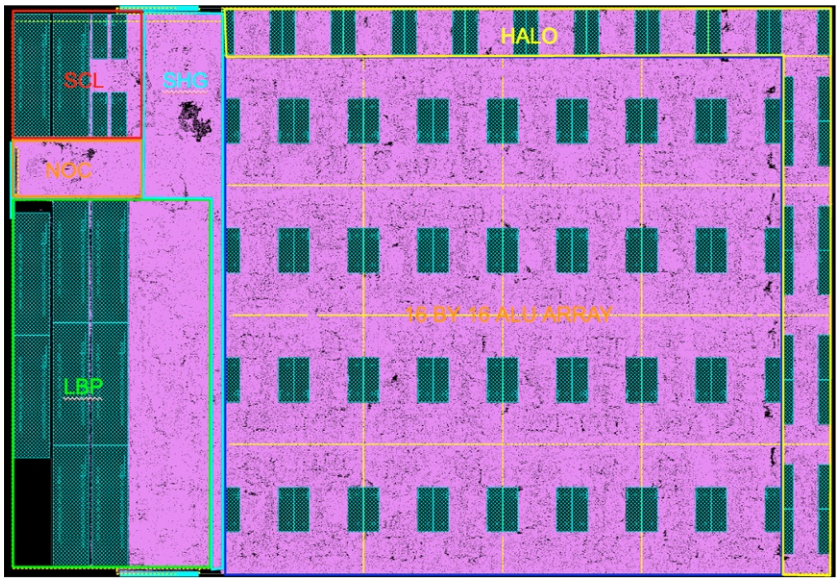


Figure 7.39 Floor plan of a Pixel Visual Core. From left to right, and top down: the scalar lane (SCL) is 4% of the core area, NOC is 2%, the line buffer pool (LBP) is 15%, the sheet generator (SHG) is 5%, the halo is 11%, and the processing element array is 62%. The torus connection of the halo makes each of the four edges of the array logical neighbors. It is more area-efficient to collapse the halo to just two sides, which preserves the topology.

3. *Use the easiest form of parallelism that matches the domain.*
   Pixel Visual Core relies on two-dimensional SIMD parallelism using its PE array, VLIW to express instruction-level parallelism, and multiple program multiple data (MPMD) parallelism to utilize multiple cores.
4. *Reduce data size and type to the simplest needed for the domain.*
   Pixel Visual Core relies primarily on 8-bit and 16-bit integers, but it also works with 32-bit integers, albeit more slowly.
5. *Use a domain-specific programming language to port code to the DSA.*
   Pixel Visual Core is programmed in the domain-specific language Halide for image processing and in TensorFlow for CNNs.

7.8 Cross-Cutting Issues

Heterogeneity and System on a Chip (SOC)

The easy way to incorporate DSAs into a system is over the I/O bus, which is the approach of the data center accelerators in this chapter. To avoid fetching memory operands over the slow I/O bus, these accelerators have local DRAM.

Amdahl's Law reminds us that the performance of an accelerator is limited by the frequency of shipping data between the host memory and the accelerator memory. There will surely be applications that would benefit from the host CPU and the accelerators to be integrated into the same *system on a chip* (*SOC*), which is one of the goals of Pixel Visual Core and eventually the Intel Crest.

Such a design is called an *IP block*, standing for *Intellectual Property*, but a more descriptive name might be portable design block. IP blocks are typically specified in a hardware description language like Verilog or VHDL to be integrated into the SOC. IP blocks enable a marketplace where many companies make IP blocks that other companies can buy to build the SOCs for their applications without having to design everything themselves. Figure 7.40 indicates the importance of IP blocks by plotting the number of IP blocks across generations of Apple PMD SOCs; they tripled in just four years. Another indication of the importance of IP blocks is that the CPU and GPU get only one-third of the area of the Apple SOCs, with IP blocks occupying the remainder (Shao and Brooks, 2015).

Designing an SOC is like city planning, where independent groups lobby for limited resources, and finding that the right compromise is difficult. CPUs, GPUs, caches, video encoders, and so on have adjustable designs that can shrink or expand to use more or less area and energy to deliver more or less performance. Budgets will differ depending on whether the SOC is for tablets or for IoT. Thus an IP block must be scalable in area, energy, and performance. Moreover, it is especially important for a new IP block to offer a small resource version because it may not already have a well-established foothold in the SOC ecosystem; adoption is much easier if the initial resource request can be modest. The Pixel Visual Core approach is a multicore design, allowing the SOC engineer to choose between 2 and 16 cores to match the area and power budget and desired performance.

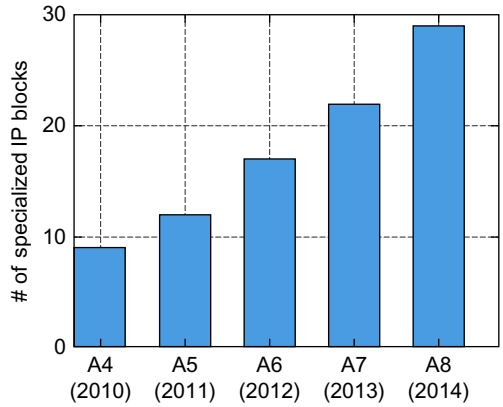


Figure 7.40 Number of IP blocks in Apple SOCs for the iPhone and iPad between 2010 and 2014 (Shao and Brooks, 2015).

It will be interesting to see whether the attractiveness of integration leads to most data center processors coming from traditional CPU companies with IP accelerators integrated into the CPU die, or whether systems companies will continue designing their own accelerators and include IP CPUs in their ASICs.

An Open Instruction Set

One challenge for designers of DSAs is determining how to collaborate with a CPU to run the rest of the application. If it's going to be on the same SOC, then a major decision is which CPU instruction set to choose, because until recently virtually every instruction set belonged to a single company. Previously, the practical first step of an SOC was to sign a contract with a company to lock in the instruction set.

The alternative was to design your own custom RISC processor and to port a compiler and libraries to it. The cost and hassle of licensing IP cores led to a surprisingly large number of do-it-yourself simple RISC processors in SOCs. One AMD engineer estimated that there were 12 instruction sets in a modern microprocessor!

RISC-V offers a third choice: a viable free and open instruction set with plenty of opcode space reserved for adding instructions for domain-specific coprocessors, which enables the previously mentioned tighter integration between CPUs and DSAs. SOC designers can now select a standard instruction set that comes with a large base of support software without having to sign a contract.

They still have to pick the instruction set early in the design, but they don't have to pick one company and sign a contract. They can design a RISC-V core themselves, they can buy one from the several companies that sell RISC-V IP blocks, or they can download one of the free open-source RISC-V IP blocks developed by others. The last case is analogous to open-source software, which offers web browsers, compilers, operating systems, and so on that volunteers maintain for users to download and use for free.

As a bonus, the open nature of the instruction set improves the business case for small companies offering RISC-V technology because customers don't have to worry about the long-term viability of a company with its own unique instruction set.

Another attraction of RISC-V for DSAs is that the instruction set is not as important as it is for general-purpose processors. If DSAs are programmed at higher levels using abstractions like DAGs or parallel patterns, as is the case for Halide and TensorFlow, then there is less to do at the instruction set level. Moreover, in a world where performance-cost and energy-cost advances come from adding DSAs, binary compatibility may not play as important a role as in the past.

At the time of this writing, the future of the open RISC-V instruction set appears promising. (We wish we could peer into the future and learn the status of RISC-V from now to the next edition of this book!)

7.9 Putting It All Together: CPUs Versus GPUs Versus DNN Accelerators

We now use the DNN domain to compare the cost-performance of the accelerators in this chapter.[2] We start with a thorough comparison of the TPU to standard CPUs and GPUs and then add brief comparisons to Catapult and Pixel Visual Core.

Figure 7.41 shows the six benchmarks we use in this comparison. They consist of two examples of each of the three types of DNNs in Section 7.3. These six benchmarks represent 95% of TPU inference workload in Google data centers in 2016. Typically written in TensorFlow, they are surprisingly short: just 100–1500 lines of code. They are small pieces of larger applications that run on the host server, which can be thousands to millions of lines of C++ code. The applications are typically user-facing, which leads to rigid response-time limits, as we will see.

Figures 7.42 and 7.43 show the chips and servers being compared. They are server-class computers deployed in Google data centers at the same time that TPUs were deployed. To be deployed in Google data centers, they must at least check for internal memory errors, which excluded some choices, such as the Nvidia Maxwell GPU. For Google to purchase and deploy them, the machines had to be sensibly configured, and not awkward artifacts assembled solely to win benchmarks.

The traditional CPU server is represented by an 18-core, dual-socket Haswell processor from Intel. This platform is also the host server for GPUs or TPUs.

		DNN layers							
Name	LOC	FC	Conv	Element	Pool	Total	Weights	TPU Ops/Weight	% deployed TPUs 2016
MLP0	100	5				5	20M	200	61%
MLP1	1000	4				4	5M	168	
LSTM0	1000	24		34		58	52M	64	29%
LSTM1	1500	37		19		56	34M	96	
CNN0	1000		16			16	8M	2888	5%
CNN1	1000	4	72		13	89	100M	1750	

Figure 7.41 Six DNN applications (two per DNN type) that represent 95% of the TPU's workload. The 10 columns are the DNN name; the number of lines of code; the types and number of layers in the DNN (FC is fully connected; Conv is convolution; Element is element-wise operation of LSTM, see Section 7.3; and Pool is pooling, which is a downsizing stage that replaces a group of elements with its average or maximum); the number of weights; TPU operational intensity; and TPU application popularity in 2016. The operational intensity varies between TPU, CPU, and GPU because the batch sizes vary. The TPU can have larger batch sizes while still staying under the response time limit. One DNN is RankBrain (Clark, 2015), one LSTM is GNM Translate (Wu et al., 2016), and one CNN is DeepMind AlphaGo (Silver et al., 2016; Jouppi, 2016).

[2]This section is also largely based upon the paper "In-Datacenter Performance Analysis of a Tensor Processing Unit" Jouppi et al., 2017, of which one of your book authors was a coauthor.

					Measured		TOPS/s			
Chip model	mm^2	nm	MHz	TDP	Idle	Busy	8b	FP	GB/s	On-chip memory
Intel Haswell	662	22	2300	145W	41W	145W	2.6	1.3	51	51 MiB
NVIDIA K80	561	28	560	150W	25W	98W	–	2.8	160	8 MiB
TPU	<331*	28	700	75W	28W	40W	92	–	34	28 MiB

*The TPU die size is less than half of the Haswell die size.

Figure 7.42 The chips used by the benchmarked servers are Haswell CPUs, K80 GPUs, and TPUs. Haswell has 18 cores, and the K80 has 13 SMX processors.

				Measured power	
Server	Dies/Server	DRAM	TDP	Idle	Busy
Intel Haswell	2	256 GiB	504W	159W	455W
NVIDIA K80 (2 dies/card)	8	256 GiB (host) + 12 GiB × 8	1838W	357W	991W
TPU	4	256 GiB (host) + 8 GiB × 4	861W	290W	384W

Figure 7.43 Benchmarked servers that use the chips in Figure 7.42. The low-power TPU allows for better rack-level density than the high-power GPU. The 8 GiB DRAM per TPU is Weight Memory.

Haswell is fabricated in an Intel 22-nm process. Both the CPU and GPU are very large dies: about 600 mm^2!

The GPU accelerator is the Nvidia K80. Each K80 card contains two dies and offers SECDED on internal memory and DRAM. Nvidia states that (Nvidia, 2016)

> the K80 Accelerator dramatically lowers datacenter cost by delivering application performance with fewer, more powerful servers.

DNN researchers frequently used K80s in 2015, which is when they were deployed at Google. Note that K80s were also chosen for new cloud-based GPUs by Amazon Web Services and by Microsoft Azure in late 2016.

Because the number of dies per benchmarked server varies between 2 and 8, the following figures show results normalized per die, except for Figure 7.50, which compares the performance/watt of whole servers.

Performance: Rooflines, Response Time, and Throughput

To illustrate the performance of the six benchmarks on the three processors, we adapt the Roofline performance model in Chapter 4. To use the Roofline model for the TPU, when DNN applications are quantized, we first replace floating-point operations with integer multiply-accumulate operations. As weights do not normally fit in on-chip memory for DNN applications, the second change is to redefine operational intensity to be integer operations per byte of weights read (Figure 7.41).

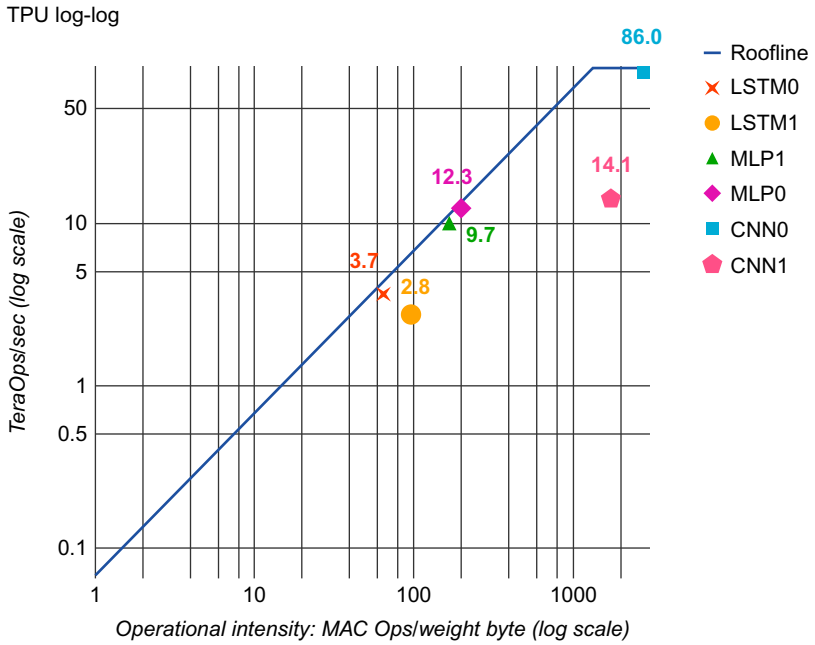


Figure 7.44 TPU Roofline. Its ridge point is far to the right at 1350 multiply-accumulate operations per byte of weight memory. CNN1 is much further below its Roofline than the other DNNs because it spends about a third of the time waiting for weights to be loaded into the matrix unit and because the shallow depth of some layers in the CNN results in only half of the elements within the matrix unit holding useful values (Jouppi et al., 2017).

Figure 7.44 shows the Roofline model for a single TPU on log-log scales. The TPU has a long "slanted" part of its Roofline, where operational intensity means that performance is limited by memory bandwidth rather than by peak compute. Five of the six applications are happily bumping their heads against the ceiling: the MLPs and LSTMs are memory-bound, and the CNNs are computation-bound. The single DNN that is not bumping its head against the ceiling is CNN1. Despite CNNs having very high operational intensity, CNN1 is running at only 14.1 Tera Operations Per Second (TOPS), while CNN0 runs at a satisfying 86 TOPS.

For readers interested into a deep dive into what happened with CNN1, Figure 7.45 uses performance counters to give partial visibility into the utilization of the TPU. The TPU spends less than half of its cycles performing matrix operations for CNN1 (column 7, row 1). On each of those active cycles, only about half of the 65,536 MACs hold useful weights because some layers in CNN1 have shallow feature depths. About 35% of cycles are spent waiting for weights to load from memory into the matrix unit, which occurs during the four fully connected layers that run at an operational intensity of just 32. This leaves roughly 19% of cycles not

Application	MLP0	MLP1	LSTM0	LSTM1	CNN0	CNN1	Mean	Row
Array active cycles	12.7%	10.6%	8.2%	10.5%	78.2%	46.2%	28%	1
Useful MACs in 64K matrix (% peak)	12.5%	9.4%	8.2%	6.3%	78.2%	22.5%	23%	2
Unused MACs	0.3%	1.2%	0.0%	4.2%	0.0%	23.7%	5%	3
Weight stall cycles	53.9%	44.2%	58.1%	62.1%	0.0%	28.1%	43%	4
Weight shift cycles	15.9%	13.4%	15.8%	17.1%	0.0%	7.0%	12%	5
Non-matrix cycles	17.5%	31.9%	17.9%	10.3%	21.8%	18.7%	20%	6
RAW stalls	3.3%	8.4%	14.6%	10.6%	3.5%	22.8%	11%	7
Input data stalls	6.1%	8.8%	5.1%	2.4%	3.4%	0.6%	4%	8
TeraOp/s (92 Peak)	12.3	9.7	3.7	2.8	86.0	14.1	21.4	9

Figure 7.45 Factors limiting TPU performance of the NN workload based on hardware performance counters. Rows 1, 4, 5, and 6 total 100% and are based on measurements of activity of the matrix unit. Rows 2 and 3 further break down the fraction of 64K weights in the matrix unit that hold useful weights on active cycles. Our counters cannot exactly explain the time when the matrix unit is idle in row 6; rows 7 and 8 show counters for two possible reasons, including RAW pipeline hazards and PCIe input stalls. Row 9 (TOPS) is based on measurements of production code while the other rows are based on performance-counter measurements, so they are not perfectly consistent. Host server overhead is excluded here. The MLPs and LSTMs are memory-bandwidth limited, but CNNs are not. CNN1 results are explained in the text.

explained by the matrix-related counters. Because of overlapped execution on the TPU, we do not have exact accounting for those cycles, but we can see that 23% of cycles have stalls for RAW dependences in the pipeline and that 1% are spent stalled for input over the PCIe bus.

Figures 7.46 and 7.47 show Rooflines for Haswell and the K80. The six NN applications are generally further below their ceilings than the TPU in Figure 7.44. Response-time limits are the reason. Many of these DNN applications are parts of services that are part of end-user-facing services. Researchers have demonstrated that small increases in response time cause customers to use a service less (see Chapter 6). Thus, although training may not have hard response-time deadlines, inference usually does. That is, inference cares about throughput only while it is maintaining the latency bound.

Figure 7.48 illustrates the impact of the 99th percentile response-time limit of 7 ms for MLP0 on Haswell and the K80, which was required by the application developer. (The inferences per second and 7-ms latency include the server host time as well as the accelerator time.) They can operate at 42% and 37%, respectively, with the highest throughput achievable for MLP0, if the response-time limit is relaxed. Thus, although CPUs and GPUs have potentially much higher throughput, it's wasted if they don't meet the response-time limit. These bounds affect the TPU as well, but at 80% in Figure 7.48, it is operating much closer to its highest MLP0 throughput. As compared with CPUs and GPUs, the single-threaded TPU has none of the sophisticated microarchitectural features discussed in Section 7.1 that consume transistors and energy to improve the average case but not the 99th-percentile case.

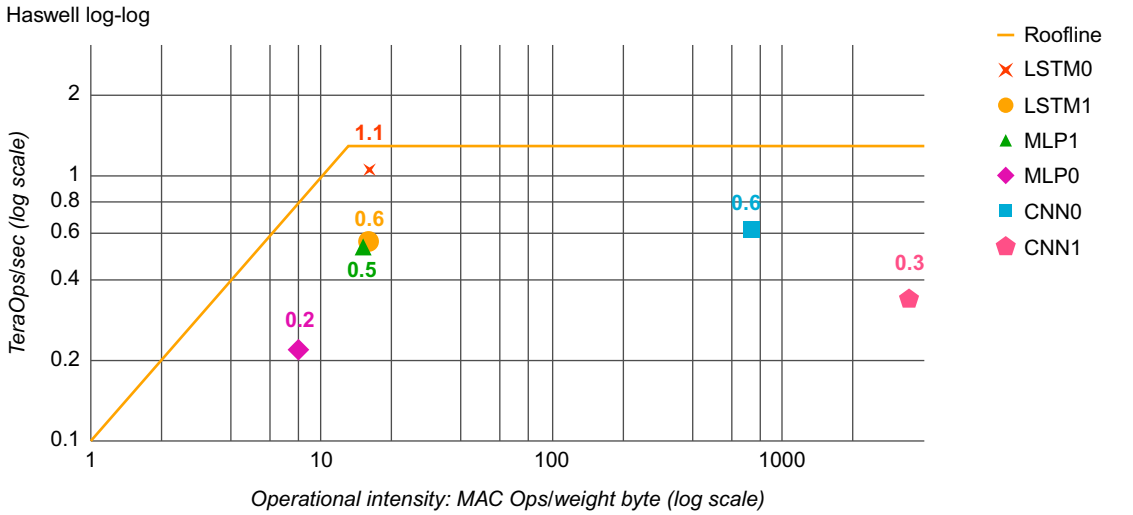


Figure 7.46 Intel Haswell CPU Roofline with its ridge point at 13 multiply-accumulate operations/byte, which is much further to the left than in Figure 7.44.

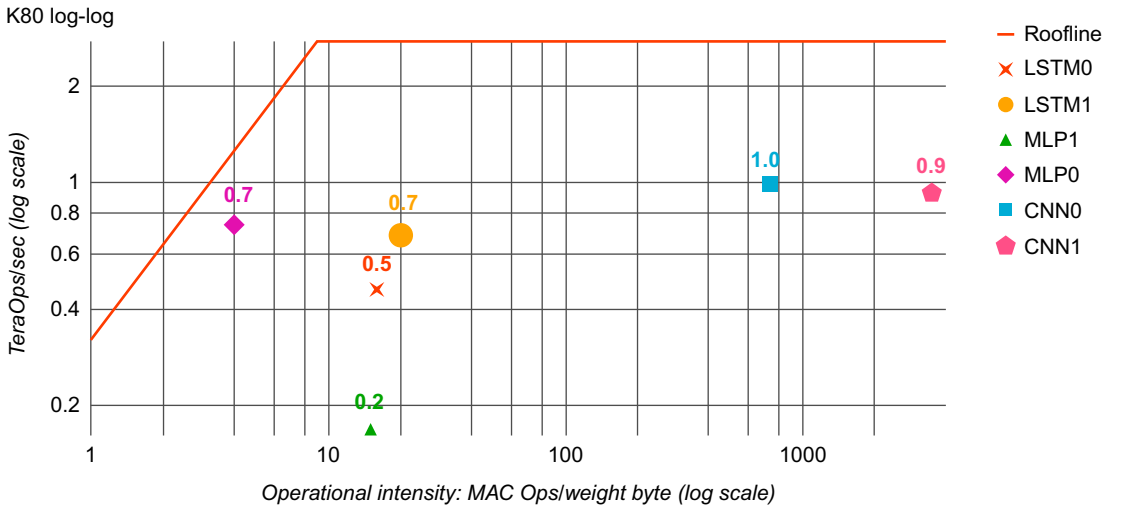


Figure 7.47 NVIDIA K80 GPU die Roofline. The much higher memory bandwidth moves the ridge point to 9 multiply-accumulate operations per weight byte, which is even further to the left than in Figure 7.46.

Figure 7.49 gives the bottom line of relative inference performance per die, including the host server overhead for the two accelerators. Recall that architects use the geometric mean when they don't know the actual mix of programs that will be run. For this comparison, however, we *do* know the mix (Figure 7.41). The

Type	Batch	99th% response	Inf/s (IPS)	% max IPS
CPU	16	7.2 ms	5482	42%
CPU	64	21.3 ms	13,194	100%
GPU	16	6.7 ms	13,461	37%
GPU	64	8.3 ms	36,465	100%
TPU	200	7.0 ms	225,000	80%
TPU	250	10.0 ms	280,000	100%

Figure 7.48 99th% response time and per die throughput (IPS) for MLP0 as batch size varies. The longest allowable latency is 7 ms. For the GPU and TPU, the maximum MLP0 throughput is limited by the host server overhead.

Type	MLP0	MLP1	LSTM0	LSTM1	CNN0	CNN1	Mean
GPU	2.5	0.3	0.4	1.2	1.6	2.7	1.9
TPU	41.0	18.5	3.5	1.2	40.3	71.0	29.2
Ratio	16.7	60.0	8.0	1.0	25.4	26.3	15.3

Figure 7.49 K80 GPU and TPU performance relative to CPU for the DNN workload. The mean uses the actual mix of the six applications in Figure 7.41. Relative performance for the GPU and TPU includes host server overhead. Figure 7.48 corresponds to the second column of this table (MLP0), showing relative IPS that meet the 7-ms latency threshold.

weighted mean in the last column of Figure 7.49 using the actual mix makes the GPU up to 1.9 times, and the TPU is 29.2 times as fast as the CPU, so the TPU is 15.3 times as fast as the GPU.

Cost-Performance, TCO, and Performance/Watt

When buying computers by the thousands, cost-performance trumps general performance. The best cost metric in a data center is total cost of ownership (TCO). The actual price Google pays for thousands of chips depends on negotiations between the companies involved. For business reasons, Google can't publish such price information or data that might let them be deduced. However, power is correlated with TCO, and Google can publish watts per server, so we use performance/watt as our proxy for performance/TCO. In this section, we compare servers (Figure 7.43) rather than single dies (Figure 7.42).

Figure 7.50 shows the weighted mean performance/watt for the K80 GPU and TPU relative to the Haswell CPU. We present two different calculations of performance/watt. The first ("total") includes the power consumed by the host CPU server when calculating performance/watt for the GPU and TPU. The second ("incremental") subtracts the host CPU server power from the total for the GPU and TPU beforehand.

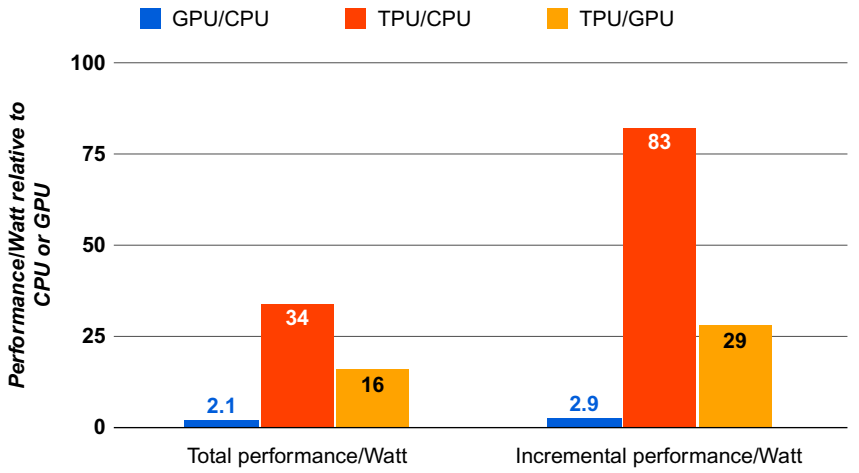


Figure 7.50 Relative performance/watt of GPU and TPU servers to CPU or GPU servers. Total performance/watt includes host server power, but incremental doesn't. It is a widely quoted metric, but we use it as a proxy for performance/TCO in the data center.

For total-performance/watt, the K80 server is 2.1 × Haswell. For incremental-performance/watt, when Haswell power is omitted, the K80 server is 2.9 ×.

The TPU server has 34 times better total-performance/watt than Haswell, which makes the TPU server 16 times the performance/watt of the K80 server. The relative incremental-performance/watt—which was Google's justification for a custom ASIC—is 83 for the TPU, which lifts the TPU to 29 times the performance/watt of the GPU.

Evaluating Catapult and Pixel Visual Core

Catapult V1 runs CNNs 2.3 × as fast as a 2.1 GHz, 16-core, dual-socket server (Ovtcharov et al., 2015a). Using the next generation of FPGAs (14-nm Arria 10), performance goes up 7 ×, and perhaps even 17 × with more careful floorplanning and scaling up of the Processing Elements (Ovtcharov et al., 2015b). In both cases, Catapult increases power by less than 1.2 ×. Although it's apples versus oranges, the TPU runs its CNNs 40 × to 70 × versus a somewhat faster server (see Figures 7.42, 7.43, and 7.49).

Because Pixel Visual Core and the TPU are both made by Google, the good news is that we can directly compare performance for CNN1, which is a common DNN, although it had to be translated from TensorFlow. It runs with batch size of 1 instead of 32 as in the TPU. The TPU runs CNN1 about 50 times as fast as Pixel Visual Core, which makes Pixel Visual Core about half as fast as the GPU and a little faster than Haswell. Incremental performance/watt for CNN1 raises Pixel Visual Core to about half the TPU, 25 times the GPU, and 100 times the CPU.

Because the Intel Crest is designed for training rather than inference, it wouldn't be fair to include it in this section, even if it were available to measure.

7.10 Fallacies and Pitfalls

In these early days of both DSAs and DNNs, fallacies abound.

Fallacy *It costs $100 million to design a custom chip.*

Figure 7.51 shows a graph from an article that debunks the widely quoted $100-million myth that it was "only" $50 million, with most of the cost being salaries (Olofsson, 2011). Note that the author's estimate is for sophisticated processors that include features that DSAs by definition omit, so even if there were no improvement to the development process, you would expect the cost of a DSA design to be less.

Why are we more optimistic six years later, when, if anything, mask costs are even higher for the smaller process technologies?

First, software is the largest category, at almost a third of the cost. The availability of applications written in domain-specific languages allows the compilers to do most of the work of porting the applications to your DSA, as we saw for the TPU and Pixel Visual Core. The open RISC-V instruction set will also help reduce the cost of getting system software as well as cut the large IP costs.

Mask and fabrication costs can be saved by having multiple projects share a single reticle. As long as you have a small chip, amazingly enough, for $30,000 anyone can get 100 untested parts in 28-nm TSMC technology (Patterson and Nikolić, 2015).

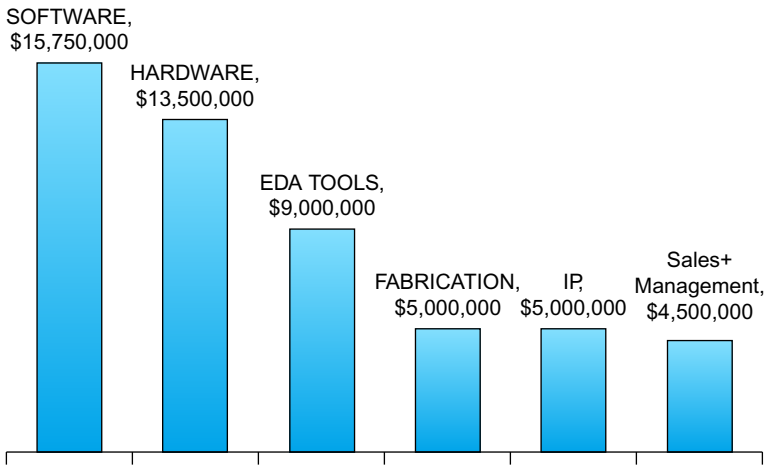


Figure 7.51 The breakdown of the $50 million cost of a custom ASIC that came from surveying others (Olofsson, 2011). The author wrote that his company spent just $2 million for its ASIC.

Perhaps the biggest change is to hardware engineering, which is more than a quarter of the cost. Hardware engineers have begun to follow their software colleagues to use agile development. The traditional hardware process not only has separate phases for design requirements, architecture, logical design, layout, verification, and so on, but also it uses different job titles for the people who perform each of the phases. This process is heavy on planning, documentation, and scheduling in part because of the change in personnel each phase.

Software used to follow this "waterfall" model as well, but projects were so commonly late, over budget, and even canceled that it led to a radically different approach. The Agile Manifesto in 2001 basically said that it was much more likely that a small team that iterated on an incomplete but working prototype shown regularly to customers would produce useful software on schedule and on budget more than the traditional plan-and-document approach of the waterfall process would.

Small hardware teams now do agile iterations (Lee et al., 2016). To ameliorate the long latency of a chip fabrication, engineers do some iterations using FPGAs because modern design systems can produce both the EDIF for FPGAs and chip layout from a single design. FPGA prototypes run 10–20 times slower than chips, but that is still much faster than simulators. They also do "tape-in" iterations, where you do all the work of a tape-out for your working but incomplete prototype, but you don't pay the costs of fabricating a chip.

In addition to an improved development process, more modern hardware design languages to support them (Bachrach et al., 2012), and advances in automatic generation of hardware from high-level domain-specific languages (Canis et al., 2013; Huang et al., 2016; Prabhakar et al., 2016). Open source cores that you can download for free and modify should also lower the cost of hardware design.

Pitfall *Performance counters added as an afterthought for DSA hardware.*

The TPU has 106 performance counters, and the designers wanted even more (see Figure 7.45). The *raison d'être* for *DSAs* is performance, and it is way too early in their evolution to have a good idea about what is going on.

Fallacy *Architects are tackling the right DNN tasks.*

The architecture community is paying attention to deep learning: 15% of the papers at ISCA 2016 were on hardware accelerators for DNNs! Alas, all nine papers looked at CNNs, and only two mentioned other DNNs. CNNs are more complex than MLPs and are prominent in DNN competitions (Russakovsky et al., 2015), which might explain their allure, but they are only about 5% of the Google data center NN workload. It seems wise try to accelerate MLPs and LSTMs with at least as much gusto.

Fallacy *For DNN hardware, inferences per second (IPS) is a fair summary performance metric.*

IPS is not appropriate as a single, overall performance summary for DNN hardware because it's simply the inverse of the complexity of the typical inference in the application (e.g., the number, size, and type of NN layers). For example, the

TPU runs the 4-layer MLP1 at 360,000 IPS but the 89-layer CNN1 at only 4700 IPS; thus TPU IPS varies by 75X! Therefore using IPS as the single-speed summary is much more misleading for NN accelerators than MIPS or FLOPS is for traditional processors, so IPS should be even more disparaged. To compare DNN machines better, we need a benchmark suite written at a high level to port it to the wide variety of DNN architectures. Fathom is a promising new attempt at such a benchmark suite (Adolf et al., 2016).

Pitfall *Being ignorant of architecture history when designing a DSA.*

Ideas that didn't fly for general-purpose computing may be ideal for DSAs, thus history-aware architects could have a competitive edge. For the TPU, three important architectural features date back to the early 1980s: systolic arrays (Kung and Leiserson, 1980), decoupled-access/execute (Smith, 1982b), and CISC instructions (Patterson and Ditzel, 1980). The first reduced the area and power of the large Matrix Multiply Unit, the second fetched weights concurrently during operation of the Matrix Multiply Unit, and the third better utilized the limited bandwidth of the PCIe bus for delivering instructions. We advise mining the historical perspectives sections at the end of every chapter of this book to discover jewels that could embellish DSAs that you design.

7.11 Concluding Remarks

In this chapter, we've seen several commercial examples of the recent shift from the traditional goal of improving general-purpose computers so that all programs benefit to accelerating a subset of programs with DSAs.

Both versions of Catapult preserved data-center homogeneity by designing a small, low-power FPGA board that could fit inside a server. The hope is that the flexibility of FPGAs will allow Catapult to be useful to many current applications and the new ones that appeared after deployment. Catapult runs search rank and CNNs faster than GPUs, offering a 1.5–1.75 gain in performance/TCO for ranking over CPUs.

The TPU project actually began with FPGAs but abandoned them when the designers concluded that the FPGAs of that time were not competitive in performance compared to the GPUs. They also believed the TPU would use much less power than GPUs, while being as fast or faster, potentially making the TPU much better than FPGAs and GPUs. Finally, the TPU was not the device that broke data center homogeneity at Google because some servers in its data centers already had GPUs. The TPU basically followed in the footsteps of the GPU and was just another type of accelerator.

The nonrecurring engineering costs were likely much higher for the TPU than for Catapult, but the rewards were also greater: both performance and performance/watt were much higher for an ASIC than for an FPGA. The risk was that the TPU was appropriate only for DNN inference, but as we mentioned, DNNs are an attractive target because they can potentially be used for many applications. In 2013

Google's management took a leap of faith by trusting that the DNN requirements in 2015 and beyond would justify investment in the TPU.

The deterministic execution model of both Catapult and the TPU is a better match to the response-time deadline of user-facing applications than are the time-varying optimizations of CPUs and GPUs (caches, out-of-order execution, multithreading, multiprocessing, prefetching, etc.) that help average throughput more than latency. The lack of such features helps explain why, despite having myriad ALUs and a big memory, the TPU is relatively small and low powered. This achievement suggests a "Cornucopia Corollary" to Amdahl's Law: *low utilization of a huge, cheap resource can still deliver high, cost-effective performance*.

In summary, the TPU succeeded for DNNs because of the large matrix unit; the substantial software-controlled on-chip memory; the ability to run whole inference models to reduce dependence on host CPU; a single-threaded, deterministic execution model that proved to be a good match to 99th-percentile response-time limits; enough flexibility to match the DNNs of 2017 as well as of 2013; the omission of general-purpose features that enabled a small and low-power die despite the larger datapath and memory; the use of 8-bit integers by the quantized applications; and the fact that applications were written using TensorFlow, which made it easy to port them to the DSA at high-performance rather than having to rewrite them in order for them to run well on the very different hardware.

Pixel Visual Core demonstrated the constraints of designing a DSA for a PMD in terms of die size and power. Unlike the TPU, it is a separate processor from the host that fetches its own instructions. Despite being aimed primarily at computer vision, Pixel Visual Core can run CNNs one to two orders of magnitude better in performance/watt than the K80 GPU and the Haswell CPU.

It's too early to render judgment on the Intel Crest, although its enthusiastic announcement by the Intel CEO signals a shift in the computing landscape.

An Architecture Renaissance

For at least the past decade, architecture researchers have been publishing innovations based on simulations using limited benchmarks claiming improvements for general-purpose processors of *10% or less* while companies are now reporting gains for DSA hardware products of *10 times or more*.

We think that is a sign that the field is undergoing a transformation, and we expect to see a renaissance in architecture innovation in the next decade because of

- the historic end of both Dennard scaling and Moore's Law, which means improving cost-energy-performance will require innovation in computer architecture;
- the productivity advances in building hardware from both Agile hardware development and new hardware design languages that leverage advances in modern programming languages;

- the reduced cost of hardware development because of free and open instruction sets, open-source IP blocks, and commercial IP blocks (which so far is where most DSAs are found);
- the improvements mentioned above in productivity and cost of development means researchers can afford to demonstrate their ideas by building them in FPGAs or even in custom chips, instead of trying to convince skeptics with simulators; and
- the potential upside of DSAs and their synergy with domain-specific programming languages.

We believe that many architecture researchers will build DSAs that will raise the bar still higher than those discussed in this chapter. And we can't wait to see what the computer architecture world will look like by the next edition of this book!

7.12 Historical Perspectives and References

Section M.9 (available online) covers the development of DSAs.

Case Studies and Exercises by Cliff Young

Case Study: Google's Tensor Processing Unit and Acceleration of Deep Neural Networks

Concepts illustrated by this case study

- Structure of matrix multiplication operations
- Capacities of memories and rates of computations ("speeds and feeds") for a simple neural network model
- Construction of a special-purpose ISA
- Inefficiencies in mapping convolutions to TPU hardware
- Fixed-point arithmetic
- Function approximation

7.1 [10/20/10/25/25] <7.3,7.4> Matrix multiplication is a key operation supported in hardware by the TPU. Before going into details of the TPU hardware, it's worth analyzing the matrix multiplication calculation itself. One common way to depict matrix multiplication is with the following triply nested loop:

```
float a[M][K], b[K][N], c[M][N];
// M, N, and K are constants.
for (int i = 0; i < M; ++i)
        for (int j = 0; j < N; ++j)
                for (int k = 0; k < K; ++k)
                        c[i][j] += a[i][k] * b[k][j];
```

a. [10] Suppose that M, N, and K are all equal. What is the asymptotic complexity in time of this algorithm? What is the asymptotic complexity in space of the arguments? What does this mean for the operational intensity of matrix multiplication as M, N, and K grow large?

b. [20] Suppose that M = 3, N = 4, and K = 5, so that each of the dimensions are relatively prime. Write out the order of accesses to memory locations in each of the three matrices A, B, and C (you might start with two-dimensional indices, then translate those to memory addresses or offsets from the start of each matrix). For which matrices are the elements accessed sequentially? Which are not? Assume row-major (C-language) memory ordering.

c. [10] Suppose that you transpose matrix B, swapping its indices so that they are B[N][K] instead. So, now the innermost statement of the loop looks like:

```
c[i][j] += a[i][k] * b[j][k];
```

Now, for which matrices are the elements accessed sequentially?

d. [25] The innermost (k-indexed) loop of our original routine performs a dot-product operation. Suppose that you are a given a hardware unit that can perform an 8-element dot-product more efficiently than the raw C code, behaving effectively like this C function:

```
void hardware_dot(float *accumulator,
   const float *a_slice, const float *b_slice) {
       float total = 0.;
       for (int k = 0; k < 8; ++k) {
              total += a_slice[k] * b_slice[k];
       }
       *accumulator += total;
}
```

How would you rewrite the routine with the transposed B matrix from part (c) to use this function?

e. [25] Suppose that instead, you are given a hardware unit that performs an 8-element "saxpy" operation, which behaves like this C function:

```
void hardware_saxpy(float *accumulator,
   float a, const float *input) {
        for (int k = 0; k < 8; ++k) {
                accumulator[k] += a * input[k];
        }
}
```

Write another routine that uses the saxpy primitive to deliver equivalent results to the original loop, without the transposed memory ordering for the B matrix.

7.2 [15/10/10/20/15/15/20/20] <7.3,7.4> Consider the neural network model MLP0 from Figure 7.5. That model has 20 M weights in five fully connected layers (neural network researchers count the input layer as if it were a layer in the stack, but it has no

weights associated with it). For simplicity, let's assume that those layers are each the same size, so each layer holds 4 M weights. Then assume that each layer has identical geometry, so each group of 4 M weights represents a 2 K*2 K matrix. Because the TPU typically uses 8-bit numerical values, 20 M weights take up 20 MB.

a. [15] For batch sizes of 128, 256, 512, 1024, and 2048, how big are the input activations for each layer of the model (which, except for the input layer, are also the output activations of the previous layer)? Now considering the whole mode (i.e., there's just the input to the first layer and the output from the last layer), for each batch size, what is the transfer time for input and output over PCIe Gen3 x16, which has a transfer speed of about 100 Gibit/s?

b. [10] Given the memory system speed of 30 GiB/s, give a lower bound for the time the TPU takes to read the weights of MLP0 from memory. How much time does it take for the TPU to read a 256×256 "tile" of weights from memory?

c. [10] Show how to calculate the TPU's 92 T operations/second, given that we know that the systolic array matrix multiplier has 256×256 elements, each of which performs an 8-bit multiply-accumulate operation (MAC) each cycle. In high-performance-computing marketing terms, a MAC counts as two operations.

d. [20] Once a weight tile has been loaded into the matrix unit of the TPU, it can be reused to multiply a 256-element input vector by the 256×256 weight matrix represented by the tile to produce a 256-element output vector every cycle. How many cycles pass during the time it takes to load a weight tile? This is the "break-even" batch size, where compute and memory-load times are equal, also known as the "ridge" of the roofline.

e. [15] The compute peak for the Intel Haswell x86 server is about 1 T FLOPS, while the compute peak for the NVIDIA K80 GPU is about 3 T FLOPS. Supposing that they hit these peak numbers, calculate their best-case compute time for batch size 128. How do these times compare to the time the TPU takes to load all 20 M weights from memory?

f. [15] Assuming that the TPU program does not overlap computation with I/O over PCIe, calculate the time elapsed from when the CPU starts to send the first byte of data to the TPU until the time that the last byte of output is returned. What fraction of PCIe bandwidth is used?

g. [20] Suppose that we deployed a configuration where one CPU was connected to five TPUs across a single PCIe Gen3 x16 bus (with appropriate PCIe switches). Assume that we parallelize by placing one layer of MLP0 on each TPU, and that the TPUs can communicate directly with each other over PCIe. At batch = 128, what is the best-case latency for calculating a single inference, and what throughput, in terms of inferences per second, would such a configuration deliver? How does this compare to a single TPU?

h. [20] Suppose that each example in a batch of inferences requires 50 core-microseconds of processing time on the CPU. How many cores on the host CPU will be required to drive a single-TPU configuration at batch = 128?

7.3 [20/25/25/25/Discussion] <7.3,7.4> Consider a pseudo-assembly language for the TPU, and consider the program that handles a batch of size 2048 for a tiny fully

connected layer with a 256×256 weight matrix. If there were no constraints on the sizes or alignments of computations in each instruction, the entire program for that layer might look like the following:

```
read_host u#0, 256*2048
read_weights w#0, 256*256
// matmul weights are implicitly read from the FIFO.
activate u#256*2048, a#0, 256*2048
write_host, u#256*2048, 256*2048
```

In this pseudo-assembly language, a prefix of "u#" refers to a memory address in the unified buffer; a prefix of "w#" refers to a memory address in the off-chip weight DRAM, and a prefix of "a#" refers to an accumulator address. The last argument of each assembly instruction describes the number of bytes to be operated upon.

Let's walk through the program instruction by instruction:

- The read_host instruction reads 512 KB of data from host memory, storing it at the very beginning of the unified buffer (u#0).
- The read_weights instruction tells the weight fetching unit to read 64 KB of weights, loading them into the on-chip weight FIFO. These 64 KB of weights represent a single, 256×256 matrix of weights, which we will call a "weight tile."
- The matmul instruction reads the 512 KB of input data from address 0 in the unified buffer, performs a matrix multiplication with the tile of weights, and stores the resulting $256*2048 = 524{,}288$, 32-bit activations at accumulator address 0 (a#0). We have intentionally glossed over the details of the ordering of weights; the exercise will expand on these details.
- The activate instruction takes those 524,288 32-bit accumulators at a#0, applies an activation function to them, and stores the resulting 524,288, 8-bit output values at the next free location in the unified buffer, u#524288.
- The write_host instruction writes the 512 KB of output activations, starting at u#524288, back to the host CPU.

We will progressively add realistic details to the pseudo-assembly language to explore some aspects of TPU design.

a. [20] While we have written our pseudo-code in terms of bytes and byte addresses (or in the case of the accumulators, in terms of addresses to 32-bit values), the TPU operates on a natural vector length of 256. This means that the unified buffer is typically addressed at 256-byte boundaries, the accumulators are addressed in groups of 256 32-bit values (or at 1 KB boundaries), and weights are loaded in groups of 65,536 8-bit values. Rewrite the program's addresses and transfer sizes to take these vector and weight-tile lengths into account. How many 256-element vectors of input activations will be read by the program? How many bytes of accumulated values will be used while computing the results? How many 256-element vectors of output activations will be written back to the host?

b. [25] Suppose that the application requirements change, and instead of a multiplication by a 256×256 weight matrix, the shape of the weight matrix now becomes 1024×256. Think of the matmul instruction as putting the weights as the right argument of the matrix multiplication operator, so 1024 corresponds to K, the dimension in which the matrix multiplication adds up values. Suppose that there are now two variants of the accumulate instruction, one of which overwrites the accumulators with its results, and the other of which adds the matrix multiplication results to the specified accumulator. How would you change the program to handle this 1024×256 matrix? Do you need more accumulators? The size of the matrix unit remains the same at 256×256; how many 256×256 weight tiles does your program need?

c. [25] Now write the program to handle a multiplication by a weight matrix of size 256×512. Does your program need more accumulators? Can you rewrite your program so that it uses only 2048, 256-entry accumulators? How many weight tiles does your program need? In what order should they be stored in the weight DRAM?

d. [25] Next, write the program to handle a multiplication by a weight matrix of size 1024×768. How many weight tiles does your program need? Write your program so that it uses only 2048, 256-entry accumulators. In what order should the weight tiles be stored in the weight DRAM? For this calculation, how many times did each input activation get read?

e. [Discussion] What would it take to build an architecture that reads each 256-element set of input activations just once? How many accumulators would that require? If you did it that way, how big would the accumulator memory have to be? Contrast this approach with the TPU, which uses 4096 accumulators, so that one set of 2048 accumulators can be written by the matrix unit while another is being used for activations.

7.4 [15/15/15] <7.3,7.4> Consider the first convolutional layer of AlexNet, which uses a 7×7 convolutional kernel, with an input feature depth of 3 and an output feature depth of 48. The original image width is 220×220.

a. [15] Ignore the 7×7 convolutional kernel for the moment, and consider just the center element of that kernel. A 1×1 convolutional kernel is mathematically equivalent to a matrix multiplication, using a weight matrix that is input_depth $\times$ output_depth in dimensions. With these depths, and using a standard matrix multiplication, what fraction of the TPU's 65,536 ALUs can be used?

b. [15] For convolutional neural networks, the spatial dimensions are also sources of weight reuse, since the convolutional kernel gets applied to many different (x,y) coordinate positions. Suppose that the TPU reaches balanced compute and memory at a batch size of 1400 (as you might have computed in exercise 1d). What is the smallest square image size that the TPU can process efficiently at a batch size of 1?

c. [15] The first convolutional layer of AlexNet implements a *kernel stride* of 4, which means that rather than moving by one X or Y pixel at each application,

the 7×7 kernel moves by 4 pixels at a time. This striding means that we can permute the input data from $220 \times 220 \times 3$ to be $55 \times 55 \times 48$ (dividing the X and Y dimensions by 4 and multiplying the input depth by 16), and simultaneously we can restack the $7 \times 7 \times 3 \times 48$ convolutional weights to be $2 \times 2 \times 48 \times 48$ (just as the input data gets restacked by 4 in X and Y, we do the same to the 7×7 elements of the convolutional kernel, ending up with ceiling(7/4) = 2 elements in each of the X and Y dimensions). Because the kernel is now 2×2, we need to perform only four matrix multiplication operations, using weight matrices of size 48×48. What is the fraction of the 65,536 ALUs that can be used now?

7.5 [15/10/20/20/20/25] <7.3> The TPU uses *fixed-point arithmetic* (sometimes also called *quantized arithmetic*, with overlapping and conflicting definitions), where integers are used to represent values on the real number line. There are a number of different schemes for fixed-point arithmetic, but they share the common theme that there is an affine projection from the integer used by hardware to the real number that the integer represents. An affine projection has the form r = i*s + b, where i is the integer, r is the represented real value, and s and b are a scale and bias. You can of course write the projection in either direction, from integers to reals or vice versa (although you need to round when converting from reals to integers).

a. [15] The simplest activation function supported by the TPU is "ReLUX," which is a rectified linear unit with a maximum of X. For example, ReLU6 is defined by Relu6(x) = { 0, when x < 0; x, when 0 < = x < = 6; and 6, when x > 6 }. So 0.0 and 6.0 on the real number line are the minimum and maximum values that Relu6 might produce. Assume that you use an 8-bit unsigned integer in hardware, and that you want to make 0 map to 0.0 and 255 map to 6.0. Solve for s and b.

b. [10] How many values on the real number line are exactly representable by an 8-bit quantized representation of ReLU6 output? What is the real-number spacing between them?

c. [20] The difference between representable values is sometimes called a "unit in the least place," or *ulp*, when performing numerical analysis. If you map a real number to its fixed-point representation, then map back, you only rarely get back the original real number. The difference between the original number and its representation is called the *quantization error*. When mapping a real number in the range [0.0,6.0] to an 8-bit integer, show that the worst-case quantization error is one-half of an ulp (make sure you round to the nearest representable value). You might consider graphing the errors as a function of the original real number.

d. [20] Keep the real-number range [0.0,6.0] for an 8-bit integer from the last step. What 8-bit unsigned integer represents 1.0? What is the quantization error for 1.0? Suppose that you ask the TPU to add 1.0 to 1.0. What answer do you get back, and what is the error in that result?

e. [20] If you pick a random number uniformly in the range [0.0, 6.0], then quantize it to an 8-bit unsigned integer, what distribution would you expect to see for the 256 integer values?

f. [25] The hyperbolic tangent function, *tanh*, is another commonly used activation function in deep learning: $\tanh(x) = \frac{1 - e^{-2x}}{1 + e^{-2x}}$

Tanh also has a bounded range, mapping the entire real number line to the interval $(-1.0, 1.0)$. Solve for s and b for this range, using an 8-bit unsigned representation. Then solve for s and b using an 8-bit two's complement representation. For both cases, what real number does the integer 0 represent? Which integer represents the real number 0.0? Can you imagine any issues that might result from the quantization error incurred when representing 0.0?

7.6 [20/25/15/15/30/30/30/40/40/25/20/Discussion] <7.3> In addition to tanh, another s-shaped smooth function, the logistic sigmoid function $y = 1 / (1 + \exp(-x))$,

$$logistic_sigmoid(x) = \frac{1}{1 + e^{-x}}$$

is commonly used as an activation function in neural networks. A common way to implement them in fixed-point arithmetic uses a piecewise quadratic approximation, where the most significant bits of the input value select which table entry to use. Then the least significant bits of the input value are sent to a degree-2 polynomial that describes a parabola that is fit to the subrange of the approximated function.

a. [20] Using a graphing tool (we like www.desmos.com/calculator), draw the graphs for the logistic sigmoid and tanh functions.

b. [25] Now draw the graph of $y = \tanh(x/2)/2$. Compare that graph with the logistic sigmoid function. How much do they differ by? Build an equation that shows how to transform one into the other. Prove that your equation is correct.

c. [15] Given this algebraic identity, do you need to use two different sets of coefficients to approximate logistic sigmoid and tanh?

d. [15] Tanh is an odd function, meaning that $f(-x) = -f(x)$. Can you exploit this fact to save table space?

e. [30] Let's focus our attention on approximating *tanh* over the interval $x \in [0.0, 6.4]$ on the number line. Using floating-point arithmetic, write a program that divides the interval into 64 subintervals (each of length 0.1), and then approximates the value of *tanh* over each subinterval using a single constant floating-point value (so you'll need to pick 64 different floating-point values, one for each subinterval). If you spot-check 100 different values (randomly chosen is fine) within each subinterval, what is the worst-case approximation error you see over all subintervals? Can you choose your constant to minimize the approximation error for each subinterval?

f. [30] Now consider building a floating-point linear approximation for each subinterval. In this case, you want to pick a pair of floating-point values m and b, for the traditional line equation $y = mx + b$, to approximate each of the 64 subintervals. Come up with a strategy that you think is reasonable to build this linear interpolation over 64 subintervals for tanh. Measure the worst-case approximation error over the 64 intervals. Is your approximation monotonic when it reaches a boundary between subintervals?

g. [40] Next, build a quadratic approximation, using the standard formula $y = ax^2 + bx + c$. Experiment with a number of different ways to fit the formula. Try fitting the parabola to the endpoints and midpoint of the bucket, or using a Taylor approximation around a single point in the bucket. What worst-case error do you get?

h. [40] (extra credit) Let's combine the numerical approximations of this exercise with the fixed-point arithmetic of the previous exercise. Suppose that the input $x \in [0.0, 6.4]$ is represented by a 15-bit unsigned value, with 0x0000 representing 0.0 and 0x7FFF representing 6.4. For the output, similarly use a 15-bit unsigned value, with 0x0000 representing 0.0 and 0x7FFF representing 1.0. For each of your constant, linear, and quadratic approximations, calculate the combined effect of approximation and quantization errors. Since there are so few input values, you can write a program to check them exhaustively.

i. [25] For the quadratic, quantized approximation, is your approximation monotonic within each subinterval?

j. [20] A difference of one ulp in the output scale should correspond to an error of 1.0 / 32767. How many ulps of error are you seeing in each case?

k. [Discussion] By choosing to approximate the interval [0.0, 6.4], we effectively clipped the "tail" of the hyperbolic tangent function, for values of $x > 6.4$. It's not an unreasonable approximation to set the output value for all of the tail to 1.0. What's the worst-case error, in terms of both real numbers and ulps, of treating the tail this way? Is there a better place we might have clipped the tail to improve our accuracy?

Exercises

7.7 [10/20/10/15] <7.2,7.5> One popular family of FPGAs, the Virtex-7 series, is built by Xilinx. A Virtex-7 XC7VX690T FPGA contains 3,600 25x18-bit integer multiply-add "DSP slices." Consider building a TPU-style design on such an FPGA.

a. [10] Using one 25×18 integer multiplier per systolic array cell, what's the largest matrix multiplication unit one could construct? Assume that the matrix multiplication unit must be square.

b. [20] Suppose that you could build a rectangular, nonsquare matrix multiplication unit. What implications would such a design have for hardware and software? (Hint: think about the vector length that software must handle.)

c. [10] Many FPGA designs are lucky to reach 500 MHz operation. At that speed, calculate the peak 8-bit operations per second that such a device might achieve. How does that compare to the 3 T FLOPS of a K80 GPU?

d. [15] Assume that you can make up the difference between 3600 and 4096 DSP slices using LUTs, but that doing so will reduce your clock rate to 350 MHz. Is this a worthwhile trade-off to make?

7.8 [15/15/15] <7.9> Amazon Web Services (AWS) offers a wide variety of "computing instances," which are machines configured to target different applications and scales. AWS prices tell us useful data about the Total Cost of Ownership (TCO) of various computing devices, particularly as computer equipment is often depreciated[1] on a 3-year schedule. As of July 2017, a dedicated, compute-oriented "c4" computing instance includes two x86 chips with 20 physical cores in total. It rents on-demand for $1.75/hour, or $17,962 for 3 years. In contrast, a dedicated "p2" computing instance also has two x86 chips but with 36 cores in total, and adds 16 NVIDIA K80 GPUs. A p2 rents on-demand for $15.84/hour, or $184,780 for 3 years.

a. [15] The c4 instance uses Intel Xeon E5-2666 v3 (Haswell) processors. The p2 instance uses Intel Xeon E5-2686 v4 (Broadwell) processors. Neither part number is listed officially on Intel's product website, which suggests that these parts are specially built for Amazon by Intel. The E5-2660 v3 part has a similar core count to the E5-2666 v3 and has a street price of around $1500. The E5-2697 v4 part has a similar core count to the E5-2686 v4 and has a street price of around $3000. Assume that the non-GPU portion of the p2 instance would have a price proportional to the ratio of street prices. What is the TCO, over 3 years, for a single K80 GPU?

b. [15] Suppose that you have a compute- and throughput-dominated workload that runs at rate 1 on the c4 instance and at rate T on the GPU-accelerated p2 instance. How large must T be for the GPU-based solution to be more cost-effective? Suppose that each general-purpose CPU core can compute at a rate of about 30G single-precision FLOPS. Ignoring the CPUs of the p2 instance, what fraction of peak K80 FLOPs would be required to reach the same rate of computation as the c4 instance?

c. [15] AWS also offers "f1" instances that include 8 Xilinx Ultrascale + VU9P FPGAs. They rent at $13.20/hour, or $165,758 for 3 years. Each VU9P device includes 6840 DSP slices, which can perform 27×18-bit integer multiply-accumulate operations (recall that one multiply-accumulate counts as two "operations"). At 500 MHz, what is the peak multiply-accumulate operations/cycle that an f1-based system might achieve, counting all 8 FPGAs toward the computation total? Assuming that the integer operations on the FPGAs can substitute for floating-point operations, how does this compare to the peak single-precision multiply-accumulate operations/cycle of the GPUs of the p2 instance? How do they compare in terms of cost-effectiveness?

7.9 [20/20/25] <7.7> As shown in Figure 7.34 (but simplified to fewer PEs), each Pixel Visual Core includes a 16×16 set of full processing elements, surrounded

[1] Capital expenses are accounted for over the lifetime of an asset, using a "depreciation schedule." Rather than taking a one-time charge at the point where an asset is acquired, standard accounting practice spreads out the capital cost over the lifetime of the asset. So one might account for a $30,000 device that has a useful life of 3 years by assigning $10,000 in depreciation to each year.

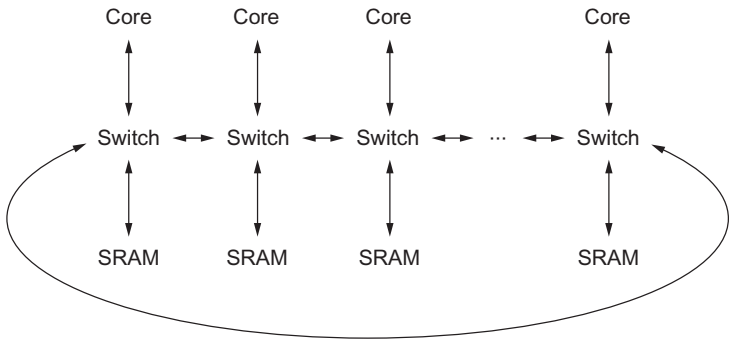
by an additional two layers of "simplified" processing elements. Simplified PEs can store and communicate data but omit the computation hardware of full PEs. Simplified PEs store copies of data that might be the "home data" of a neighboring core, so there are $(16+2+2)^2=400$ PEs in total, 256 full and 144 simplified.

a. [20] Suppose that you wanted to process a 64×32 grayscale image with a 5×5 stencil using 8 Pixel Visual Cores. For now, assume that the image is laid out in raster-scan order (pixels that are adjacent in X are adjacent in memory, while pixels that are adjacent in Y are 64 memory locations apart). For each of the 8 cores, describe the memory region that the core should import to handle its part of the image. Make sure to include the halo region. Which parts of the halo region should be zeroed by software to ensure correct operation? You may find it convenient to refer to subregions of the image using a 2D slice notation, where for example image[2:5][6:13] refers to the set of pixels whose x component is $2 <= x < 5$ and whose y component is $6 <= y < 13$ (the slices are half-open following Python slicing practice).

b. [20] If we change to a 3×3 stencil, how do the regions imported from memory change? How many halo-simplified PEs go unused?

c. [25] Now consider how to support a 7×7 stencil. In this case, we don't have as many hardware-supported simplified PEs as we need to cover the three pixels worth of halo data that "belong to" neighboring cores. To handle this, we use the outermost ring of full PEs as if they were simplified PEs. How many pixels can we handle in a single core using this strategy? How many "tiles" are now required to handle our 64×32 input image? What is the utilization of our full PEs over the complete processing time for the 7×7 stencil over the 64×32 image?

7.10 [20/20/20/25/25] <7.7> Consider a case in which each of the eight cores on a Pixel Visual Core device is connected through a four-port switch to a 2D SRAM, forming a core + memory unit. The remaining two ports on the switch link these units in a ring, so that each core is able to access any of the eight SRAMs. However, this ring-based network-on-chip topology makes some data access patterns more efficient than others.



a. [20] Suppose that each link in the NOC has the same bandwidth B, and that each link is full-duplex, so it can simultaneously transfer bandwidth B in each direction. Links connect the core to the switch, the switch to SRAM, and pairs of switches in the ring. Assume that each local memory has at least B bandwidth, so it can saturate its link. Consider a memory access pattern where each of the eight PEs access only the closest memory (the one connected via the switch of the core + memory unit). What is the maximum memory bandwidth that the core will be able to achieve?

b. [20] Now consider an off-by-one access pattern, where core i accesses memory $i+1$, going through three links to reach that memory (core 7 will access memory 0, because of the ring topology). What is the maximum memory bandwidth that the core will be able to achieve in this case? To achieve that bandwidth, do you need to make any assumptions about the capabilities of the 4-port switch? What if the switch can only move data at rate B?

c. [20] Consider an off-by-two access pattern, where core i access memory $i+2$. Once again, what is the maximum memory bandwidth that the core will be able to achieve in this case? Where are the bottleneck links in the network-on-chip?

d. [25] Consider a uniform random memory access pattern, where each core uses each of the SRAMs for ⅛ of its memory requests. Assuming this traffic pattern, how much traffic traverses a switch-to-switch link, compared to the amount of traffic between a core and its associated switch or between an SRAM and its associated switch?

e. [25] (advanced) Can you conceive of a case (workload) where this network can deadlock? From the standpoint of software-only solutions, what should the compiler do to avoid such a scenario? If you can make changes to hardware, what changes in routing topology (and routing scheme) would guarantee no deadlocks?

7.11 <7.2> The first Anton molecular dynamics supercomputer typically simulated a box of water that was 64 Å on a side. The computer itself might be approximated as a box with 1 m side length. A single simulation step represented 2.5 fs of simulation time, and took about 10 μs of wall-clock time. The physics models used in molecular dynamics act as if every particle in the system exerts a force on every other particle in the system on each ("outer") time step, requiring what amounts to a global synchronization across the entire computer.

a. Calculate the spatial expansion factor from simulation space to hardware in real space.

b. Calculate the temporal slowdown factor from simulated time to wall-clock time.

c. These two numbers come out surprisingly close. Is this just a coincidence, or is there some other limit that constrains them in some way? (Hint: the speed of light applies to both the simulated chemical system and the hardware that does the simulation.)

d. Given these limits, what would it take to use a warehouse-scale supercomputer to perform molecular dynamics simulations at Anton rates? That is, what's the fastest simulation step time that might be achieved with a machine 10^2 or 10^3 m on a side? What about simulating on a world-spanning Cloud service?

7.12 <7.2> The Anton communication network is a 3D, $8 \times 8 \times 8$ torus, where each node in the system has six links to neighboring nodes. Latency for a packet to transit single link is about 50 ns. Ignore on-chip switching time between links for this exercise.

a. What is the diameter (maximum number of hops between a pair of nodes) of the communication network? Given that diameter, what is the shortest latency required to broadcast a single value from one node of the machine to all 512 nodes of the machine?

b. Assuming that adding up two values takes zero time, what is the shortest latency to add up a sum over 512 values to a single node, where each value starts on a different node of the machine?

c. Once again assume that you want to perform the sum over 512 values, but you want each of the 512 nodes of the system to end up with a copy of the sum. Of course you could perform a global reduction followed by a broadcast. Can you do the combined operation in less time? This pattern is called an *all-reduce*. Compare the times of your all-reduce pattern to the time of a broadcast from a single node or a global sum to a single node. Compare the bandwidth used by the all-reduce pattern with the other patterns.

A.1	Introduction	A-2
A.2	Classifying Instruction Set Architectures	A-3
A.3	Memory Addressing	A-7
A.4	Type and Size of Operands	A-13
A.5	Operations in the Instruction Set	A-15
A.6	Instructions for Control Flow	A-16
A.7	Encoding an Instruction Set	A-21
A.8	Cross-Cutting Issues: The Role of Compilers	A-24
A.9	Putting It All Together: The RISC-V Architecture	A-33
A.10	Fallacies and Pitfalls	A-42
A.11	Concluding Remarks	A-46
A.12	Historical Perspective and References	A-47
	Exercises by Gregory D. Peterson	A-47

A

Instruction Set Principles

A *n* Add the number in storage location *n* into the accumulator.

E *n* If the number in the accumulator is greater than or equal to zero execute next the order which stands in storage location *n*; otherwise proceed serially.

Z Stop the machine and ring the warning bell.

Wilkes and Renwick,
Selection from the List of 18 Machine Instructions for the EDSAC (1949)

A.1 Introduction

In this appendix we concentrate on instruction set architecture—the portion of the computer visible to the programmer or compiler writer. Most of this material should be review for readers of this book; we include it here for background. This appendix introduces the wide variety of design alternatives available to the instruction set architect. In particular, we focus on four topics. First, we present a taxonomy of instruction set alternatives and give some qualitative assessment of the advantages and disadvantages of various approaches. Second, we present and analyze some instruction set measurements that are largely independent of a specific instruction set. Third, we address the issue of languages and compilers and their bearing on instruction set architecture. Finally, the "Putting It All Together" section shows how these ideas are reflected in the RISC-V instruction set, which is typical of RISC architectures. We conclude with fallacies and pitfalls of instruction set design.

To illustrate the principles further and to provide a comparison with RISC-V, Appendix K also gives four examples of other general-purpose RISC architectures (MIPS, Power ISA, SPARC, and Armv8), four embedded RISC processors (ARM Thumb2, RISC-V Compressed, microMIPS), and three older architectures (80x86, IBM 360/370, and VAX). Before we discuss how to classify architectures, we need to say something about instruction set measurement.

Throughout this appendix, we examine a wide variety of architectural measurements. Clearly, these measurements depend on the programs measured and on the compilers used in making the measurements. The results should not be interpreted as absolute, and you might see different data if you did the measurement with a different compiler or a different set of programs. We believe that the measurements in this appendix are reasonably indicative of a class of typical applications. Many of the measurements are presented using a small set of benchmarks, so that the data can be reasonably displayed and the differences among programs can be seen. An architect for a new computer would want to analyze a much larger collection of programs before making architectural decisions. The measurements shown are usually *dynamic*—that is, the frequency of a measured event is weighed by the number of times that event occurs during execution of the measured program.

Before starting with the general principles, let's review the three application areas from Chapter 1. *Desktop computing* emphasizes the performance of programs with integer and floating-point data types, with little regard for program size. For example, code size has never been reported in the five generations of SPEC benchmarks. *Servers* today are used primarily for database, file server, and Web applications, plus some time-sharing applications for many users. Hence, floating-point performance is much less important for performance than integers and character strings, yet virtually every server processor still includes floating-point instructions. *Personal mobile devices* and *embedded applications* value cost and energy, so code size is important because less memory is both cheaper and lower energy, and some classes of instructions (such as floating point) may be optional to reduce chip costs, and a compressed version of the instructions set designed to save memory space may be used.

Thus, instruction sets for all three applications are very similar. In fact, architectures similar to RISC-V, which we focus on here, have been used successfully in desktops, servers, and embedded applications.

One successful architecture very different from RISC is the 80x86 (see Appendix K). Surprisingly, its success does not necessarily belie the advantages of a RISC instruction set. The commercial importance of binary compatibility with PC software combined with the abundance of transistors provided by Moore's Law led Intel to use a RISC instruction set internally while supporting an 80x86 instruction set externally. Recent 80x86 microprocessors, including all the Intel Core microprocessors built in the past decade, use hardware to translate from 80x86 instructions to RISC-like instructions and then execute the translated operations inside the chip. They maintain the illusion of 80x86 architecture to the programmer while allowing the computer designer to implement a RISC-style processor for performance. There remain, however, serious disadvantages for a complex instruction set like the 80x86, and we discuss these further in the conclusions.

Now that the background is set, we begin by exploring how instruction set architectures can be classified.

A.2 Classifying Instruction Set Architectures

The type of internal storage in a processor is the most basic differentiation, so in this section we will focus on the alternatives for this portion of the architecture. The major choices are a stack, an accumulator, or a set of registers. Operands may be named explicitly or implicitly: The operands in a *stack architecture* are implicitly on the top of the stack, and in an *accumulator architecture* one operand is implicitly the accumulator. The *general-purpose register architectures* have only explicit operands—either registers or memory locations. Figure A.1 shows a block diagram of such architectures, and Figure A.2 shows how the code sequence C = A + B would typically appear in these three classes of instruction sets. The explicit operands may be accessed directly from memory or may need to be first loaded into temporary storage, depending on the class of architecture and choice of specific instruction.

As the figures show, there are really two classes of register computers. One class can access memory as part of any instruction, called *register-memory* architecture, and the other can access memory only with load and store instructions, called *load-store* architecture. A third class, not found in computers shipping today, keeps all operands in memory and is called a *memory-memory* architecture. Some instruction set architectures have more registers than a single accumulator but place restrictions on uses of these special registers. Such an architecture is sometimes called an *extended accumulator* or *special-purpose register* computer.

Although most early computers used stack or accumulator-style architectures, virtually every new architecture designed after 1980 uses a load-store register architecture. The major reasons for the emergence of general-purpose register (GPR) computers are twofold. First, registers—like other forms of storage internal to the processor—are faster than memory. Second, registers are more efficient for a

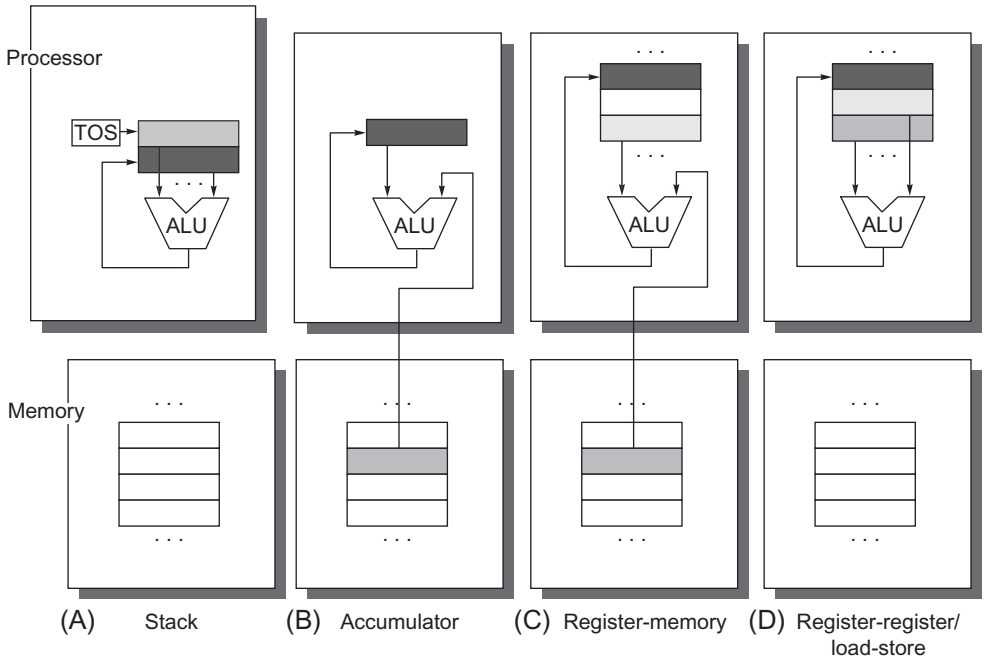


Figure A.1 Operand locations for four instruction set architecture classes. The arrows indicate whether the operand is an input or the result of the arithmetic-logical unit (ALU) operation, or both an input and result. Lighter shades indicate inputs, and the dark shade indicates the result. In (A), a top of stack (TOS) register points to the top input operand, which is combined with the operand below. The first operand is removed from the stack, the result takes the place of the second operand, and TOS is updated to point to the result. All operands are implicit. In (B), the accumulator is both an implicit input operand and a result. In (C), one input operand is a register, one is in memory, and the result goes to a register. All operands are registers in (D) and, like the stack architecture, can be transferred to memory only via separate instructions: push or pop for (A) and load or store for (D).

Stack	Accumulator	Register (register-memory)	Register (load-store)
`Push A`	`Load A`	`Load R1,A`	`Load R1,A`
`Push B`	`Add B`	`Add R3,R1,B`	`Load R2,B`
`Add`	`Store C`	`Store R3,C`	`Add R3,R1,R2`
`Pop C`			`Store R3,C`

Figure A.2 The code sequence for `C = A + B` for four classes of instruction sets. Note that the `Add` instruction has implicit operands for stack and accumulator architectures and explicit operands for register architectures. It is assumed that A, B, and C all belong in memory and that the values of A and B cannot be destroyed. Figure A.1 shows the `Add` operation for each class of architecture.

compiler to use than other forms of internal storage. For example, on a register computer the expression (A * B) + (B * C) - (A * D) may be evaluated by doing the multiplications in any order, which may be more efficient because of the location of the operands or because of pipelining concerns (see Chapter 3). Nevertheless, on a stack computer the hardware must evaluate the expression in only one order, because operands are hidden on the stack, and it may have to load an operand multiple times.

More importantly, registers can be used to hold variables. When variables are allocated to registers, the memory traffic reduces, the program speeds up (because registers are faster than memory), and the code density improves (because a register can be named with fewer bits than can a memory location).

As explained in Section A.8, compiler writers would prefer that all registers be equivalent and unreserved. Older computers compromise this desire by dedicating registers to special uses, effectively decreasing the number of general-purpose registers. If the number of truly general-purpose registers is too small, trying to allocate variables to registers will not be profitable. Instead, the compiler will reserve all the uncommitted registers for use in expression evaluation.

How many registers are sufficient? The answer, of course, depends on the effectiveness of the compiler. Most compilers reserve some registers for expression evaluation, use some for parameter passing, and allow the remainder to be allocated to hold variables. Modern compiler technology and its ability to effectively use larger numbers of registers has led to an increase in register counts in more recent architectures.

Two major instruction set characteristics divide GPR architectures. Both characteristics concern the nature of operands for a typical arithmetic or logical instruction (ALU instruction). The first concerns whether an ALU instruction has two or three operands. In the three-operand format, the instruction contains one result operand and two source operands. In the two-operand format, one of the operands is both a source and a result for the operation. The second distinction among GPR architectures concerns how many of the operands may be memory addresses in ALU instructions. The number of memory operands supported by a typical ALU instruction may vary from none to three. Figure A.3 shows combinations of these two attributes with examples of computers. Although there are seven

Number of memory addresses	Maximum number of operands allowed	Type of architecture	Examples
0	3	Load-store	ARM, MIPS, PowerPC, SPARC, RISC-V
1	2	Register-memory	IBM 360/370, Intel 80x86, Motorola 68000, TI TMS320C54x
2	2	Memory-memory	VAX (also has three-operand formats)
3	3	Memory-memory	VAX (also has two-operand formats)

Figure A.3 Typical combinations of memory operands and total operands per typical ALU instruction with examples of computers. Computers with no memory reference per ALU instruction are called load-store or register-register computers. Instructions with multiple memory operands per typical ALU instruction are called register-memory or memory-memory, according to whether they have one or more than one memory operand.

Type	Advantages	Disadvantages
Register-register (0, 3)	Simple, fixed-length instruction encoding. Simple code generation model. Instructions take similar numbers of clocks to execute (see Appendix C)	Higher instruction count than architectures with memory references in instructions. More instructions and lower instruction density lead to larger programs, which may have some instruction cache effects
Register-memory (1, 2)	Data can be accessed without a separate load instruction first. Instruction format tends to be easy to encode and yields good density	Operands are not equivalent because a source operand in a binary operation is destroyed. Encoding a register number and a memory address in each instruction may restrict the number of registers. Clocks per instruction vary by operand location
Memory-memory (2, 2) or (3, 3)	Most compact. Doesn't waste registers for temporaries	Large variation in instruction size, especially for three-operand instructions. In addition, large variation in work per instruction. Memory accesses create memory bottleneck. (Not used today.)

Figure A.4 Advantages and disadvantages of the three most common types of general-purpose register computers. The notation (*m*, *n*) means *m* memory operands and *n* total operands. In general, computers with fewer alternatives simplify the compiler's task because there are fewer decisions for the compiler to make (see Section A.8). Computers with a wide variety of flexible instruction formats reduce the number of bits required to encode the program. The number of registers also affects the instruction size because you need $\log_2$ (number of registers) for each register specifier in an instruction. Thus, doubling the number of registers takes three extra bits for a register-register architecture, or about 10% of a 32-bit instruction.

possible combinations, three serve to classify nearly all existing computers. As we mentioned earlier, these three are load-store (also called register-register), register-memory, and memory-memory.

Figure A.4 shows the advantages and disadvantages of each of these alternatives. Of course, these advantages and disadvantages are not absolutes: they are qualitative and their actual impact depends on the compiler and implementation strategy. A GPR computer with memory-memory operations could easily be ignored by the compiler and used as a load-store computer. One of the most pervasive architectural impacts is on instruction encoding and the number of instructions needed to perform a task. We see the impact of these architectural alternatives on implementation approaches in Appendix C and Chapter 3.

Summary: Classifying Instruction Set Architectures

Here and at the end of Sections A.3–A.8 we summarize those characteristics we would expect to find in a new instruction set architecture, building the foundation for the RISC-V architecture introduced in Section A.9. From this section we should clearly expect the use of general-purpose registers. Figure A.4, combined with Appendix C on pipelining, leads to the expectation of a load-store version of a general-purpose register architecture.

With the class of architecture covered, the next topic is addressing operands.

A.3 Memory Addressing

Independent of whether the architecture is load-store or allows any operand to be a memory reference, it must define how memory addresses are interpreted and how they are specified. The measurements presented here are largely, but not completely, computer independent. In some cases the measurements are significantly affected by the compiler technology. These measurements have been made using an optimizing compiler, because compiler technology plays a critical role.

Interpreting Memory Addresses

How is a memory address interpreted? That is, what object is accessed as a function of the address and the length? All the instruction sets discussed in this book are byte addressed and provide access for bytes (8 bits), half words (16 bits), and words (32 bits). Most of the computers also provide access for double words (64 bits).

There are two different conventions for ordering the bytes within a larger object. *Little Endian* byte order puts the byte whose address is "x … x000" at the least-significant position in the double word (the little end). The bytes are numbered:

7	6	5	4	3	2	1	0

Big Endian byte order puts the byte whose address is "x … x000" at the most-significant position in the double word (the big end). The bytes are numbered:

0	1	2	3	4	5	6	7

When operating within one computer, the byte order is often unnoticeable—only programs that access the same locations as both, say, words and bytes, can notice the difference. Byte order is a problem when exchanging data among computers with different orderings, however. Little Endian ordering also fails to match the normal ordering of words when strings are compared. Strings appear "SDRAWKCAB" (backwards) in the registers.

A second memory issue is that in many computers, accesses to objects larger than a byte must be *aligned*. An access to an object of size s bytes at byte address A is aligned if $A \bmod s = 0$. Figure A.5 shows the addresses at which an access is aligned or misaligned.

Why would someone design a computer with alignment restrictions? Misalignment causes hardware complications, because the memory is typically aligned on a multiple of a word or double-word boundary. A misaligned memory access may, therefore, take multiple aligned memory references. Thus, even in computers that allow misaligned access, programs with aligned accesses run faster.

	Value of three low-order bits of byte address							
Width of object	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**
1 byte (byte)	Aligned	Aligned	Aligned	Aligned	Aligned	Aligned	Aligned	Aligned
2 bytes (half word)	Aligned		Aligned		Aligned		Aligned	
2 bytes (half word)		Misaligned		Misaligned		Misaligned		Misaligned
4 bytes (word)	Aligned				Aligned			
4 bytes (word)		Misaligned				Misaligned		
4 bytes (word)			Misaligned				Misaligned	
4 bytes (word)				Misaligned				Misaligned
8 bytes (double word)	Aligned							
8 bytes (double word)		Misaligned						
8 bytes (double word)			Misaligned					
8 bytes (double word)				Misaligned				
8 bytes (double word)					Misaligned			
8 bytes (double word)						Misaligned		
8 bytes (double word)							Misaligned	
8 bytes (double word)								Misaligned

Figure A.5 Aligned and misaligned addresses of byte, half-word, word, and double-word objects for byte-addressed computers. For each misaligned example some objects require two memory accesses to complete. Every aligned object can always complete in one memory access, as long as the memory is as wide as the object. The figure shows the memory organized as 8 bytes wide. The byte offsets that label the columns specify the low-order three bits of the address.

Even if data are aligned, supporting byte, half-word, and word accesses requires an alignment network to align bytes, half words, and words in 64-bit registers. For example, in Figure A.5, suppose we read a byte from an address with its 3 low-order bits having the value 4. We will need to shift right 3 bytes to align the byte to the proper place in a 64-bit register. Depending on the instruction, the computer may also need to sign-extend the quantity. Stores are easy: only the addressed bytes in memory may be altered. On some computers a byte, half-word, and word operation does not affect the upper portion of a register. Although all the computers discussed in this book permit byte, half-word, and word accesses to memory, only the IBM 360/370, Intel 80x86, and VAX support ALU operations on register operands narrower than the full width.

Now that we have discussed alternative interpretations of memory addresses, we can discuss the ways addresses are specified by instructions, called *addressing modes*.

Addressing Modes

Given an address, we now know what bytes to access in memory. In this subsection we will look at addressing modes—how architectures specify the address

of an object they will access. Addressing modes specify constants and registers in addition to locations in memory. When a memory location is used, the actual memory address specified by the addressing mode is called the *effective address*.

Figure A.6 shows all the data addressing modes that have been used in recent computers. Immediates or literals are usually considered memory addressing modes (even though the value they access is in the instruction stream), although registers are often separated because they don't usually have memory addresses. We have kept addressing modes that depend on the program counter, called *PC-relative addressing*, separate. PC-relative addressing is used primarily for specifying code addresses in control transfer instructions, discussed in Section A.6.

Addressing mode	Example instruction	Meaning	When used
Register	`Add R4,R3`	`Regs[R4]←Regs[R4] + Regs[R3]`	When a value is in a register
Immediate	`Add R4,3`	`Regs[R4]←Regs[R4]+3`	For constants
Displacement	`Add R4,100(R1)`	`Regs[R4]←Regs[R4] + Mem[100+Regs[R1]]`	Accessing local variables (+ simulates register indirect, direct addressing modes)
Register indirect	`Add R4,(R1)`	`Regs[R4]←Regs[R4] + Mem[Regs[R1]]`	Accessing using a pointer or a computed address
Indexed	`Add R3,(R1+R2)`	`Regs[R3]←Regs[R3] + Mem[Regs[R1]+Regs [R2]]`	Sometimes useful in array addressing: `R1` = base of array; `R2` = index amount
Direct or absolute	`Add R1,(1001)`	`Regs[R1]←Regs[R1] + Mem[1001]`	Sometimes useful for accessing static data; address constant may need to be large
Memory indirect	`Add R1,@(R3)`	`Regs[R1]←Regs[R1] + Mem[Mem[Regs[R3]]]`	If `R3` is the address of a pointer *p*, then mode yields **p*
Autoincrement	`Add R1,(R2)+`	`Regs[R1]←Regs[R1] + Mem[Regs[R2]]` `Regs[R2]←Regs[R2]+d`	Useful for stepping through arrays within a loop. `R2` points to start of array; each reference increments `R2` by size of an element, *d*
Autodecrement	`Add R1, -(R2)`	`Regs[R2]←Regs[R2] - d` `Regs[R1]←Regs[R1] + Mem[Regs[R2]]`	Same use as autoincrement. Autodecrement/-increment can also act as push/pop to implement a stack.
Scaled	`Add R1,100(R2)[R3]`	`Regs[R1]←Regs[R1] + Mem[100+Regs[R2] + Regs[R3] * d]`	Used to index arrays. May be applied to any indexed addressing mode in some computers

Figure A.6 Selection of addressing modes with examples, meaning, and usage. In autoincrement/-decrement and scaled addressing modes, the variable *d* designates the size of the data item being accessed (i.e., whether the instruction is accessing 1, 2, 4, or 8 bytes). These addressing modes are only useful when the elements being accessed are adjacent in memory. RISC computers use displacement addressing to simulate register indirect with 0 for the address and to simulate direct addressing using 0 in the base register. In our measurements, we use the first name shown for each mode. The extensions to C used as hardware descriptions are defined on page A.38.

Figure A.6 shows the most common names for the addressing modes, though the names differ among architectures. In this figure and throughout the book, we will use an extension of the C programming language as a hardware description notation. In this figure, only one non-C feature is used: the left arrow (←) is used for assignment. We also use the array `Mem` as the name for main memory and the array `Regs` for registers. Thus, `Mem[Regs[R1]]` refers to the contents of the memory location whose address is given by the contents of register 1 (`R1`). Later, we will introduce extensions for accessing and transferring data smaller than a word.

Addressing modes have the ability to significantly reduce instruction counts; they also add to the complexity of building a computer and may increase the average clock cycles per instruction (CPI) of computers that implement those modes. Thus, the usage of various addressing modes is quite important in helping the architect choose what to include.

Figure A.7 shows the results of measuring addressing mode usage patterns in three programs on the VAX architecture. We use the old VAX architecture for a few measurements in this appendix because it has the richest set of addressing

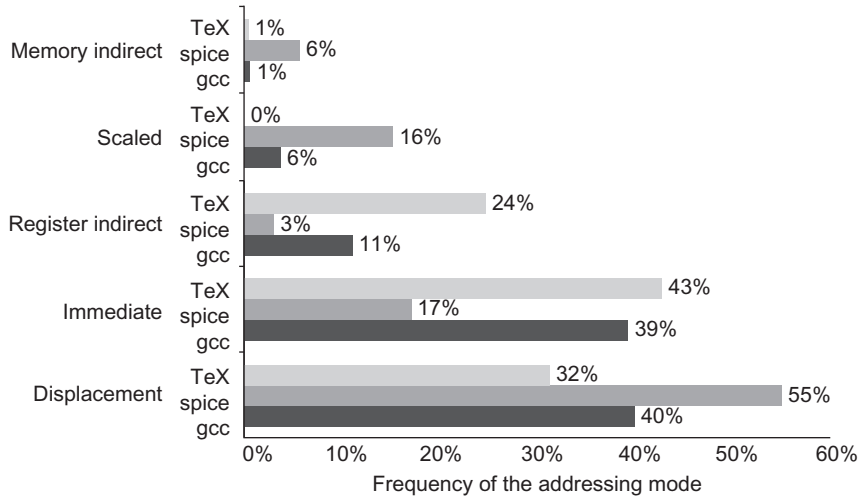


Figure A.7 Summary of use of memory addressing modes (including immediates). These major addressing modes account for all but a few percent (0%–3%) of the memory accesses. Register modes, which are not counted, account for one-half of the operand references, while memory addressing modes (including immediate) account for the other half. Of course, the compiler affects what addressing modes are used; see Section A.8. The memory indirect mode on the VAX can use displacement, autoincrement, or autodecrement to form the initial memory address; in these programs, almost all the memory indirect references use displacement mode as the base. Displacement mode includes all displacement lengths (8, 16, and 32 bits). The PC-relative addressing modes, used almost exclusively for branches, are not included. Only the addressing modes with an average frequency of over 1% are shown.

modes and the fewest restrictions on memory addressing. For example, Figure A.6 on page A.9 shows all the modes the VAX supports. Most measurements in this appendix, however, will use the more recent register-register architectures to show how programs use instruction sets of current computers.

As Figure A.7 shows, displacement and immediate addressing dominate addressing mode usage. Let's look at some properties of these two heavily used modes.

Displacement Addressing Mode

The major question that arises for a displacement-style addressing mode is that of the range of displacements used. Based on the use of various displacement sizes, a decision of what sizes to support can be made. Choosing the displacement field sizes is important because they directly affect the instruction length. Figure A.8

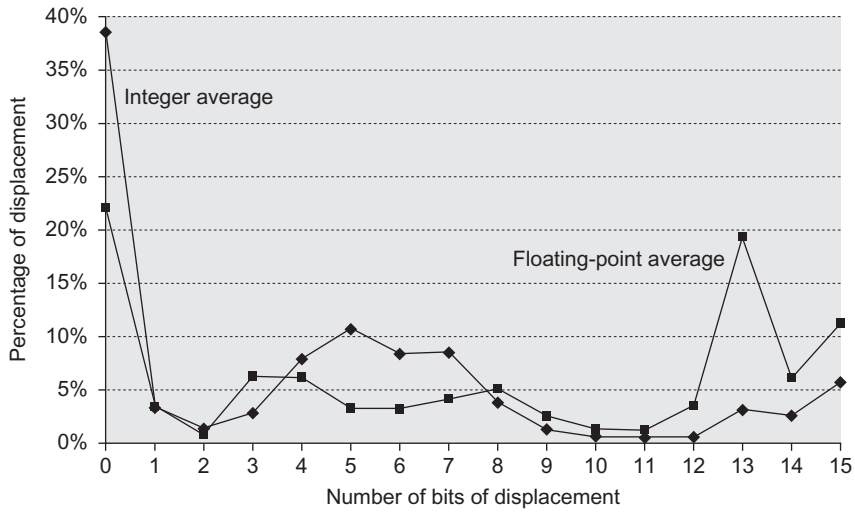


Figure A.8 Displacement values are widely distributed. There are both a large number of small values and a fair number of large values. The wide distribution of displacement values is due to multiple storage areas for variables and different displacements to access them (see Section A.8) as well as the overall addressing scheme the compiler uses. The *x*-axis is $\log_2$ of the displacement, that is, the size of a field needed to represent the magnitude of the displacement. Zero on the *x*-axis shows the percentage of displacements of value 0. The graph does not include the sign bit, which is heavily affected by the storage layout. Most displacements are positive, but a majority of the largest displacements (14+ bits) are negative. Because these data were collected on a computer with 16-bit displacements, they cannot tell us about longer displacements. These data were taken on the Alpha architecture with full optimization (see Section A.8) for SPEC CPU2000, showing the average of integer programs (CINT2000) and the average of floating-point programs (CFP2000).

shows the measurements taken on the data access on a load-store architecture using our benchmark programs. We look at branch offsets in Section A.6—data accessing patterns and branches are different; little is gained by combining them, although in practice the immediate sizes are made the same for simplicity.

Immediate or Literal Addressing Mode

Immediates can be used in arithmetic operations, in comparisons (primarily for branches), and in moves where a constant is wanted in a register. The last case occurs for constants written in the code—which tend to be small—and for address constants, which tend to be large. For the use of immediates it is important to know whether they need to be supported for all operations or for only a subset. Figure A.9 shows the frequency of immediates for the general classes of integer and floating-point operations in an instruction set.

Another important instruction set measurement is the range of values for immediates. Like displacement values, the size of immediate values affects instruction length. As Figure A.10 shows, small immediate values are most heavily used. Large immediates are sometimes used, however, most likely in addressing calculations.

Summary: Memory Addressing

First, because of their popularity, we would expect a new architecture to support at least the following addressing modes: displacement, immediate, and register indirect. Figure A.7 shows that they represent 75%–99% of the addressing modes used

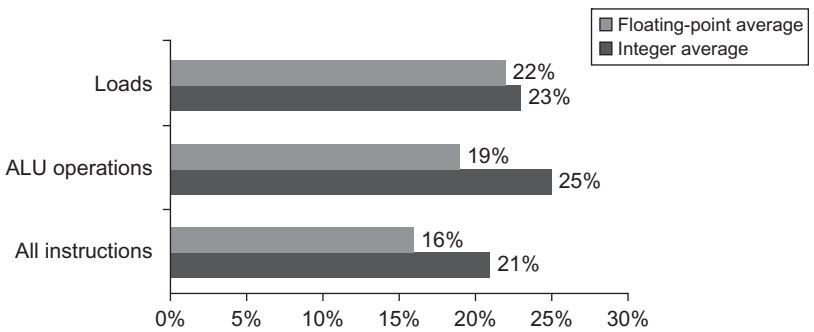


Figure A.9 About one-quarter of data transfers and ALU operations have an immediate operand. The bottom bars show that integer programs use immediates in about one-fifth of the instructions, while floating-point programs use immediates in about one-sixth of the instructions. For loads, the load immediate instruction loads 16 bits into either half of a 32-bit register. Load immediates are not loads in a strict sense because they do not access memory. Occasionally a pair of load immediates is used to load a 32-bit constant, but this is rare. (For ALU operations, shifts by a constant amount are included as operations with immediate operands.) The programs and computer used to collect these statistics are the same as in Figure A.8.

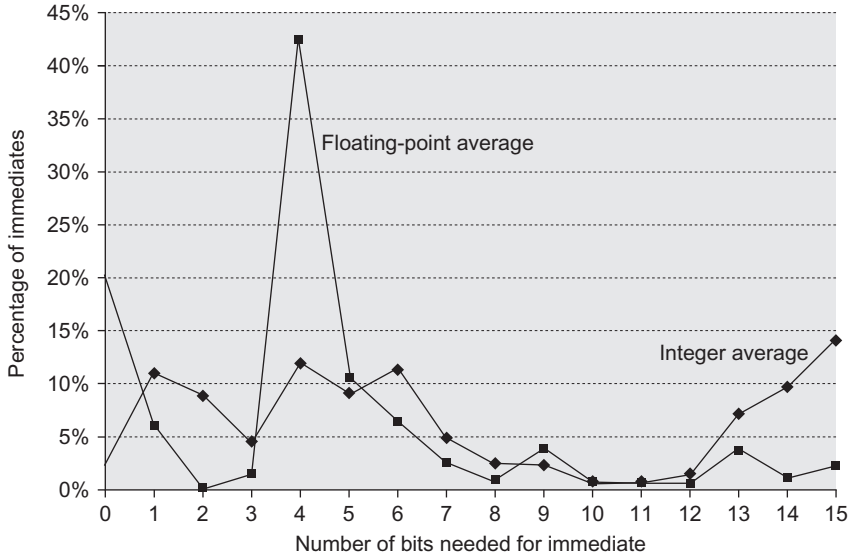


Figure A.10 The distribution of immediate values. The *x*-axis shows the number of bits needed to represent the magnitude of an immediate value—0 means the immediate field value was 0. The majority of the immediate values are positive. About 20% were negative for CINT2000, and about 30% were negative for CFP2000. These measurements were taken on an Alpha, where the maximum immediate is 16 bits, for the same programs as in Figure A.8. A similar measurement on the VAX, which supported 32-bit immediates, showed that about 20%–25% of immediates were longer than 16 bits. Thus, 16 bits would capture about 80% and 8 bits about 50%.

in our measurements. Second, we would expect the size of the address for displacement mode to be at least 12–16 bits, because the caption in Figure A.8 suggests these sizes would capture 75%–99% of the displacements. Third, we would expect the size of the immediate field to be at least 8–16 bits. This claim is not substantiated by the caption of the figure to which it refers.

Having covered instruction set classes and decided on register-register architectures, plus the previous recommendations on data addressing modes, we next cover the sizes and meanings of data.

A.4 Type and Size of Operands

How is the type of an operand designated? Usually, encoding in the opcode designates the type of an operand—this is the method used most often. Alternatively, the data can be annotated with tags that are interpreted by the hardware. These tags specify the type of the operand, and the operation is chosen accordingly. Computers with tagged data, however, can only be found in computer museums.

Let's start with desktop and server architectures. Usually the type of an operand—integer, single-precision floating point, character, and so on—effectively

gives its size. Common operand types include character (8 bits), half word (16 bits), word (32 bits), single-precision floating point (also 1 word), and double-precision floating point (2 words). Integers are almost universally represented as two's complement binary numbers. Characters are usually in ASCII, but the 16-bit Unicode (used in Java) is gaining popularity with the internationalization of computers. Until the early 1980s, most computer manufacturers chose their own floating-point representation. Almost all computers since that time follow the same standard for floating point, the IEEE standard 754, although this level of accuracy has recently been abandoned in application-specific processors. The IEEE floating-point standard is discussed in detail in Appendix J.

Some architectures provide operations on character strings, although such operations are usually quite limited and treat each byte in the string as a single character. Typical operations supported on character strings are comparisons and moves.

For business applications, some architectures support a decimal format, usually called *packed decimal* or *binary-coded decimal*—4 bits are used to encode the values 0–9, and 2 decimal digits are packed into each byte. Numeric character strings are sometimes called *unpacked decimal*, and operations—called *packing* and *unpacking*—are usually provided for converting back and forth between them.

One reason to use decimal operands is to get results that exactly match decimal numbers, as some decimal fractions do not have an exact representation in binary. For example, 0.10_{10} is a simple fraction in decimal, but in binary it requires an infinite set of repeating digits: $0.0001100110\overline{0011}_{\ldots 2}$. Thus, calculations that are exact in decimal can be close but inexact in binary, which can be a problem for financial transactions. (See Appendix J to learn more about precise arithmetic.)

The SPEC benchmarks use byte or character, half-word (short integer), word (integer and single precision floating point), double-word (long integer), and floating-point data types. Figure A.11 shows the dynamic distribution of the sizes of objects referenced from memory for these programs. The frequency of access to

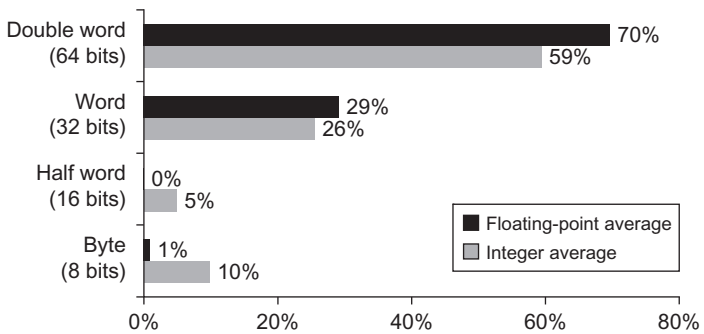


Figure A.11 Distribution of data accesses by size for the benchmark programs. The double-word data type is used for double-precision floating point in floating-point programs and for addresses, because the computer uses 64-bit addresses. On a 32-bit address computer the 64-bit addresses would be replaced by 32-bit addresses, and so almost all double-word accesses in integer programs would become single-word accesses.

different data types helps in deciding what types are most important to support efficiently. Should the computer have a 64-bit access path, or would taking two cycles to access a double word be satisfactory? As we saw earlier, byte accesses require an alignment network: how important is it to support bytes as primitives? Figure A.11 uses memory references to examine the types of data being accessed.

In some architectures, objects in registers may be accessed as bytes or half words. However, such access is very infrequent—on the VAX, it accounts for no more than 12% of register references, or roughly 6% of all operand accesses in these programs.

A.5 Operations in the Instruction Set

The operators supported by most instruction set architectures can be categorized as in Figure A.12. One rule of thumb across all architectures is that the most widely executed instructions are the simple operations of an instruction set. For example, Figure A.13 shows 10 simple instructions that account for 96% of instructions executed for a collection of integer programs running on the popular Intel 80x86. Hence, the implementor of these instructions should be sure to make these fast, as they are the common case.

Operator type	Examples
Arithmetic and logical	Integer arithmetic and logical operations: add, subtract, and, or, multiply, divide
Data transfer	Loads-stores (move instructions on computers with memory addressing)
Control	Branch, jump, procedure call and return, traps
System	Operating system call, virtual memory management instructions
Floating point	Floating-point operations: add, multiply, divide, compare
Decimal	Decimal add, decimal multiply, decimal-to-character conversions
String	String move, string compare, string search
Graphics	Pixel and vertex operations, compression/decompression operations

Figure A.12 Categories of instruction operators and examples of each. All computers generally provide a full set of operations for the first three categories. The support for system functions in the instruction set varies widely among architectures, but all computers must have some instruction support for basic system functions. The amount of support in the instruction set for the last four categories may vary from none to an extensive set of special instructions. Floating-point instructions will be provided in any computer that is intended for use in an application that makes much use of floating point. These instructions are sometimes part of an optional instruction set. Decimal and string instructions are sometimes primitives, as in the VAX or the IBM 360, or may be synthesized by the compiler from simpler instructions. Graphics instructions typically operate on many smaller data items in parallel—for example, performing eight 8-bit additions on two 64-bit operands.

Rank	80x86 instruction	Integer average % total executed)
1	Load	22%
2	Conditional branch	20%
3	Compare	16%
4	Store	12%
5	Add	8%
6	And	6%
7	Sub	5%
8	Move register-register	4%
9	Call	1%
10	Return	1%
Total		**96%**

Figure A.13 The top 10 instructions for the 80x86. Simple instructions dominate this list and are responsible for 96% of the instructions executed. These percentages are the average of the five SPECint92 programs.

As mentioned before, the instructions in Figure A.13 are found in every computer for every application—desktop, server, embedded—with the variations of operations in Figure A.12 largely depending on which data types the instruction set includes.

A.6 Instructions for Control Flow

Because the measurements of branch and jump behavior are fairly independent of other measurements and applications, we now examine the use of control flow instructions, which have little in common with the operations of the previous sections.

There is no consistent terminology for instructions that change the flow of control. In the 1950s they were typically called *transfers*. Beginning in 1960 the name *branch* began to be used. Later, computers introduced additional names. Throughout this book we will use *jump* when the change in control is unconditional and *branch* when the change is conditional.

We can distinguish four different types of control flow change:

- Conditional branches
- Jumps
- Procedure calls
- Procedure returns

We want to know the relative frequency of these events, as each event is different, may use different instructions, and may have different behavior. Figure A.14 shows the frequencies of these control flow instructions for a load-store computer running our benchmarks.

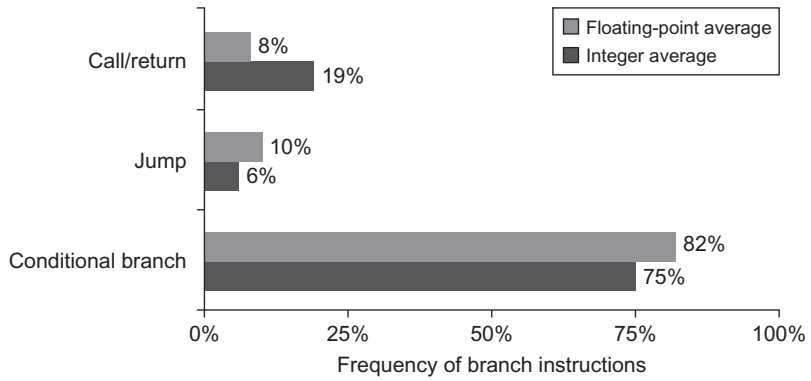


Figure A.14 Breakdown of control flow instructions into three classes: calls or returns, jumps, and conditional branches. Conditional branches clearly dominate. Each type is counted in one of three bars. The programs and computer used to collect these statistics are the same as those in Figure A.8.

Addressing Modes for Control Flow Instructions

The destination address of a control flow instruction must always be specified. This destination is specified explicitly in the instruction in the vast majority of cases—procedure return being the major exception, because for return the target is not known at compile time. The most common way to specify the destination is to supply a displacement that is added to the *program counter* (PC). Control flow instructions of this sort are called *PC-relative*. PC-relative branches or jumps are advantageous because the target is often near the current instruction, and specifying the position relative to the current PC requires fewer bits. Using PC-relative addressing also permits the code to run independently of where it is loaded. This property, called *position independence*, can eliminate some work when the program is linked and is also useful in programs linked dynamically during execution.

To implement returns and indirect jumps when the target is not known at compile time, a method other than PC-relative addressing is required. Here, there must be a way to specify the target dynamically, so that it can change at runtime. This dynamic address may be as simple as naming a register that contains the target address; alternatively, the jump may permit any addressing mode to be used to supply the target address.

These register indirect jumps are also useful for four other important features:

- *Case* or *switch* statements, found in most programming languages (which select among one of several alternatives).
- *Virtual functions* or *methods* in object-oriented languages like C++ or Java (which allow different routines to be called depending on the type of the argument).

- *High-order functions* or *function pointers* in languages like C or C++ (which allow functions to be passed as arguments, giving some of the flavor of object-oriented programming).
- *Dynamically shared libraries* (which allow a library to be loaded and linked at runtime only when it is actually invoked by the program rather than loaded and linked statically before the program is run).

In all four cases the target address is not known at compile time, and hence is usually loaded from memory into a register before the register indirect jump.

As branches generally use PC-relative addressing to specify their targets, an important question concerns how far branch targets are from branches. Knowing the distribution of these displacements will help in choosing what branch offsets to support, and thus will affect the instruction length and encoding. Figure A.15 shows the distribution of displacements for PC-relative branches in instructions. About 75% of the branches are in the forward direction.

Conditional Branch Options

Because most changes in control flow are branches, deciding how to specify the branch condition is important. Figure A.16 shows the three primary techniques in use today and their advantages and disadvantages.

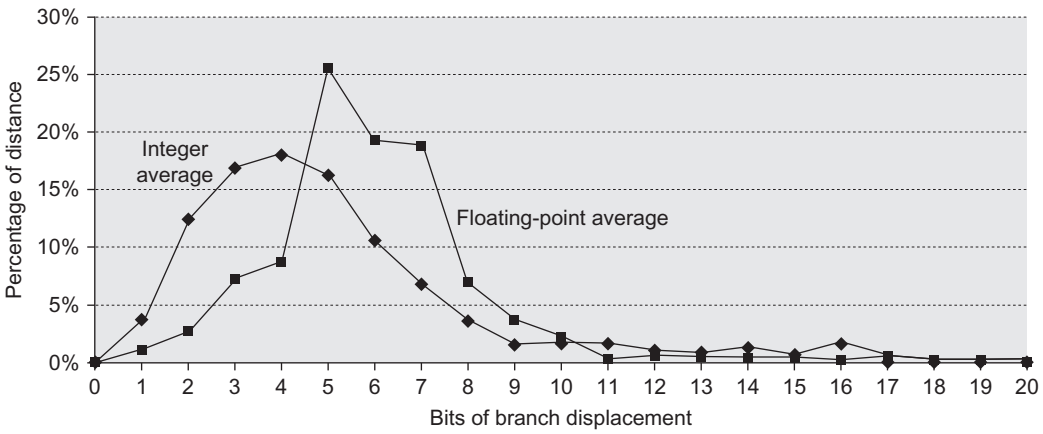


Figure A.15 Branch distances in terms of number of instructions between the target and the branch instruction. The most frequent branches in the integer programs are to targets that can be encoded in 4–8 bits. This result tells us that short displacement fields often suffice for branches and that the designer can gain some encoding density by having a shorter instruction with a smaller branch displacement. These measurements were taken on a load-store computer (Alpha architecture) with all instructions aligned on word boundaries. An architecture that requires fewer instructions for the same program, such as a VAX, would have shorter branch distances. However, the number of bits needed for the displacement may increase if the computer has variable-length instructions to be aligned on any byte boundary. The programs and computer used to collect these statistics are the same as those in Figure A.8.

Name	Examples	How condition is tested	Advantages	Disadvantages
Condition code (CC)	80x86, ARM, PowerPC, SPARC, SuperH	Tests special bits set by ALU operations, possibly under program control	Sometimes condition is set for free.	CC is extra state. Condition codes constrain the ordering of instructionsbecause they pass information from one instruction to a branch
Condition register/ limited comparison	Alpha, MIPS	Tests arbitrary register with the result of a simple comparison (equality or zero tests)	Simple	Limited compare may affect critical path or require extra comparison for general condition
Compare and branch	PA-RISC, VAX, RISC-V	Compare is part of the branch. Fairly general compares are allowed (greater then, less then)	One instruction rather than two for a branch	May set critical path for branch instructions

Figure A.16 The major methods for evaluating branch conditions, their advantages, and their disadvantages. Although condition codes can be set by ALU operations that are needed for other purposes, measurements on programs show that this rarely happens. The major implementation problems with condition codes arise when the condition code is set by a large or haphazardly chosen subset of the instructions, rather than being controlled by a bit in the instruction. Computers with compare and branch often limit the set of compares and use a separate operation and register for more complex compares. Often, different techniques are used for branches based on floating-point comparison versus those based on integer comparison. This dichotomy is reasonable because the number of branches that depend on floating-point comparisons is much smaller than the number depending on integer comparisons.

One of the most noticeable properties of branches is that a large number of the comparisons are simple tests, and a large number are comparisons with zero. Thus, some architectures choose to treat these comparisons as special cases, especially if a *compare and branch* instruction is being used. Figure A.17 shows the frequency of different comparisons used for conditional branching.

Procedure Invocation Options

Procedure calls and returns include control transfer and possibly some state saving; at a minimum the return address must be saved somewhere, sometimes in a special link register or just a GPR. Some older architectures provide a mechanism to save many registers, while newer architectures require the compiler to generate stores and loads for each register saved and restored.

There are two basic conventions in use to save registers: either at the call site or inside the procedure being called. *Caller saving* means that the calling procedure must save the registers that it wants preserved for access after the call, and thus the called procedure need not worry about registers. *Callee saving* is the opposite: the called procedure must save the registers it wants to use, leaving the caller unrestrained. There are times when caller save must be used because of access patterns to globally visible variables in two different procedures. For example, suppose we have a procedure P1 that calls procedure P2, and both procedures manipulate the

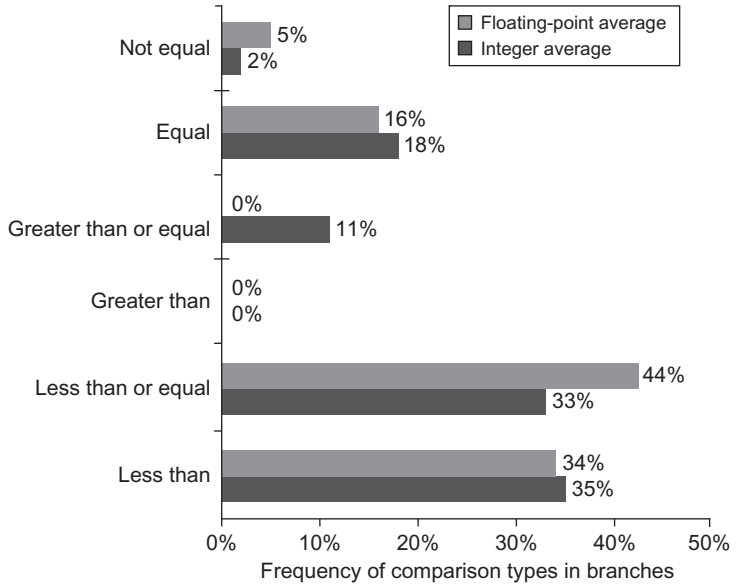


Figure A.17 Frequency of different types of compares in conditional branches. Less than (or equal) branches dominate this combination of compiler and architecture. These measurements include both the integer and floating-point compares in branches. The programs and computer used to collect these statistics are the same as those in Figure A.8.

global variable *x*. If P1 had allocated *x* to a register, it must be sure to save *x* to a location known by P2 before the call to P2. A compiler's ability to discover when a called procedure may access register-allocated quantities is complicated by the possibility of separate compilation. Suppose P2 may not touch *x* but can call another procedure, P3, that may access *x*, yet P2 and P3 are compiled separately. Because of these complications, most compilers will conservatively caller save *any* variable that may be accessed during a call.

In the cases where either convention could be used, some programs will be more optimal with callee save and some will be more optimal with caller save. As a result, most real systems today use a combination of the two mechanisms. This convention is specified in an application binary interface (ABI) that sets down the basic rules as to which registers should be caller saved and which should be callee saved. Later in this appendix we will examine the mismatch between sophisticated instructions for automatically saving registers and the needs of the compiler.

Summary: Instructions for Control Flow

Control flow instructions are some of the most frequently executed instructions. Although there are many options for conditional branches, we would expect

branch addressing in a new architecture to be able to jump to hundreds of instructions either above or below the branch. This requirement suggests a PC-relative branch displacement of at least 8 bits. We would also expect to see register indirect and PC-relative addressing for jump instructions to support returns as well as many other features of current systems.

We have now completed our instruction architecture tour at the level seen by an assembly language programmer or compiler writer. We are leaning toward a load-store architecture with displacement, immediate, and register indirect addressing modes. These data are 8-, 16-, 32-, and 64-bit integers and 32- and 64-bit floating-point data. The instructions include simple operations, PC-relative conditional branches, jump and link instructions for procedure call, and register indirect jumps for procedure return (plus a few other uses).

Now we need to select how to represent this architecture in a form that makes it easy for the hardware to execute.

A.7 Encoding an Instruction Set

Clearly, the choices mentioned herein will affect how the instructions are encoded into a binary representation for execution by the processor. This representation affects not only the size of the compiled program but also the implementation of the processor, which must decode this representation to quickly find the operation and its operands. The operation is typically specified in one field, called the *opcode*. As we shall see, the important decision is how to encode the addressing modes with the operations.

This decision depends on the range of addressing modes and the degree of independence between opcodes and modes. Some older computers have one to five operands with 10 addressing modes for each operand (see Figure A.6). For such a large number of combinations, typically a separate *address specifier* is needed for each operand: the address specifier tells what addressing mode is used to access the operand. At the other extreme are load-store computers with only one memory operand and only one or two addressing modes; obviously, in this case, the addressing mode can be encoded as part of the opcode.

When encoding the instructions, the number of registers and the number of addressing modes both have a significant impact on the size of instructions, as the register field and addressing mode field may appear many times in a single instruction. In fact, for most instructions many more bits are consumed in encoding addressing modes and register fields than in specifying the opcode. The architect must balance several competing forces when encoding the instruction set:

1. The desire to have as many registers and addressing modes as possible.
2. The impact of the size of the register and addressing mode fields on the average instruction size and hence on the average program size.

3. A desire to have instructions encoded into lengths that will be easy to handle in a pipelined implementation. (The value of easily decoded instructions is discussed in Appendix C and Chapter 3.) As a minimum, the architect wants instructions to be in multiples of bytes, rather than an arbitrary bit length. Many desktop and server architects have chosen to use a fixed-length instruction to gain implementation benefits while sacrificing average code size.

Figure A.18 shows three popular choices for encoding the instruction set. The first we call *variable*, because it allows virtually all addressing modes to be with all operations. This style is best when there are many addressing modes and operations. The second choice we call *fixed*, because it combines the operation and the addressing mode into the opcode. Often fixed encoding will have only a single

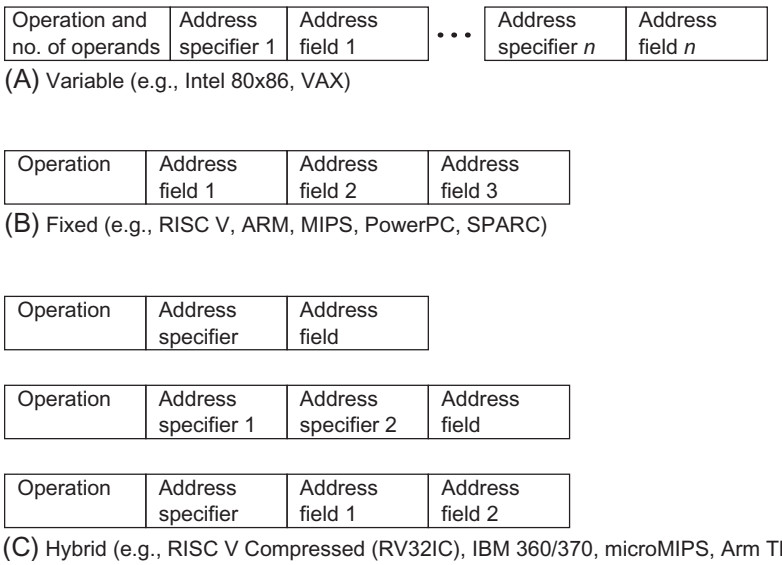


Figure A.18 Three basic variations in instruction encoding: variable length, fixed length, and hybrid. The variable format can support any number of operands, with each address specifier determining the addressing mode and the length of the specifier for that operand. It generally enables the smallest code representation, because unused fields need not be included. The fixed format always has the same number of operands, with the addressing modes (if options exist) specified as part of the opcode. It generally results in the largest code size. Although the fields tend not to vary in their location, they will be used for different purposes by different instructions. The hybrid approach has multiple formats specified by the opcode, adding one or two fields to specify the addressing mode and one or two fields to specify the operand address.

size for all instructions; it works best when there are few addressing modes and operations. The trade-off between variable encoding and fixed encoding is size of programs versus ease of decoding in the processor. Variable tries to use as few bits as possible to represent the program, but individual instructions can vary widely in both size and the amount of work to be performed.

Let's look at an 80x86 instruction to see an example of the variable encoding:

```
add EAX,1000(EBX)
```

The name `add` means a 32-bit integer add instruction with two operands, and this opcode takes 1 byte. An 80x86 address specifier is 1 or 2 bytes, specifying the source/destination register (`EAX`) and the addressing mode (displacement in this case) and base register (`EBX`) for the second operand. This combination takes 1 byte to specify the operands. When in 32-bit mode (see Appendix K), the size of the address field is either 1 byte or 4 bytes. Because1000 is bigger than 2^8, the total length of the instruction is

$$1 + 1 + 4 = 6 \text{ bytes}$$

The length of 80x86 instructions varies between 1 and 17 bytes. 80x86 programs are generally smaller than the RISC architectures, which use fixed formats (see Appendix K).

Given these two poles of instruction set design of variable and fixed, the third alternative immediately springs to mind: reduce the variability in size and work of the variable architecture but provide multiple instruction lengths to reduce code size. This *hybrid* approach is the third encoding alternative, and we'll see examples shortly.

Reduced Code Size in RISCs

As RISC computers started being used in embedded applications, the 32-bit fixed format became a liability because cost, and hence smaller code, are important. In response, several manufacturers offered a new hybrid version of their RISC instruction sets, with both 16-bit and 32-bit instructions. The narrow instructions support fewer operations, smaller address and immediate fields, fewer registers, and the two-address format rather than the classic three-address format of RISC computers. RISC-V offers such an extension, called RV32IC, the C standing for compressed. Common instruction occurrences, such as intermediates with small values and common ALU operations with the source and destination register being identical, are encoded in 16-bit formats. Appendix K gives two other examples, the ARM Thumb and microMIPS, which both claim a code size reduction of up to 40%.

In contrast to these instruction set extensions, IBM simply compresses its standard instruction set and then adds hardware to decompress instructions as they are fetched from memory on an instruction cache miss. Thus, the instruction cache contains full 32-bit instructions, but compressed code is kept in main memory, ROMs, and the disk. The advantage of a compressed format, such as RV32IC,

microMIPS and Thumb2 is that instruction caches act as if they are about 25% larger, while IBM's CodePack means that compilers need not be changed to handle different instruction sets and instruction decoding can remain simple.

CodePack starts with run-length encoding compression on any PowerPC program and then loads the resulting compression tables in a 2 KB table on chip. Hence, every program has its own unique encoding. To handle branches, which are no longer to an aligned word boundary, the PowerPC creates a hash table in memory that maps between compressed and uncompressed addresses. Like a TLB (see Chapter 2), it caches the most recently used address maps to reduce the number of memory accesses. IBM claims an overall performance cost of 10%, resulting in a code size reduction of 35%–40%.

Summary: Encoding an Instruction Set

Decisions made in the components of instruction set design discussed in previous sections determine whether the architect has the choice between variable and fixed instruction encodings. Given the choice, the architect more interested in code size than performance will pick variable encoding, and the one more interested in performance than code size will pick fixed encoding. RISC-V, MIPS, and ARM all have an instruction set extension that uses 16-bit instruction, as well as 32-bit; applications with serious code size constraints can opt to use the 16-bit variant to decrease code size. Appendix E gives 13 examples of the results of architects' choices. In Appendix C and Chapter 3, the impact of variability on performance of the processor will be discussed further.

We have almost finished laying the groundwork for the RISC-V instruction set architecture that will be introduced in Section A.9. Before we do that, however, it will be helpful to take a brief look at compiler technology and its effect on program properties.

A.8 Cross-Cutting Issues: The Role of Compilers

Today almost all programming is done in high-level languages for desktop and server applications. This development means that because most instructions executed are the output of a compiler, an instruction set architecture is essentially a compiler target. In earlier times for these applications, architectural decisions were often made to ease assembly language programming or for a specific kernel. Because the compiler will significantly affect the performance of a computer, understanding compiler technology today is critical to designing and efficiently implementing an instruction set.

Once it was popular to try to isolate the compiler technology and its effect on hardware performance from the architecture and its performance, just as it was popular to try to separate architecture from its implementation. This separation is essentially impossible with today's desktop compilers and computers. Architectural choices affect the quality of the code that can be generated for a computer and the complexity of building a good compiler for it, for better or for worse.

In this section, we discuss the critical goals in the instruction set primarily from the compiler viewpoint. It starts with a review of the anatomy of current compilers. Next we discuss how compiler technology affects the decisions of the architect, and how the architect can make it hard or easy for the compiler to produce good code. We conclude with a review of compilers and multimedia operations, which unfortunately is a bad example of cooperation between compiler writers and architects.

The Structure of Recent Compilers

To begin, let's look at what optimizing compilers are like today. Figure A.19 shows the structure of recent compilers.

A compiler writer's first goal is correctness—all valid programs must be compiled correctly. The second goal is usually speed of the compiled code. Typically, a whole set of other goals follows these two, including fast compilation, debugging support, and interoperability among languages. Normally, the passes

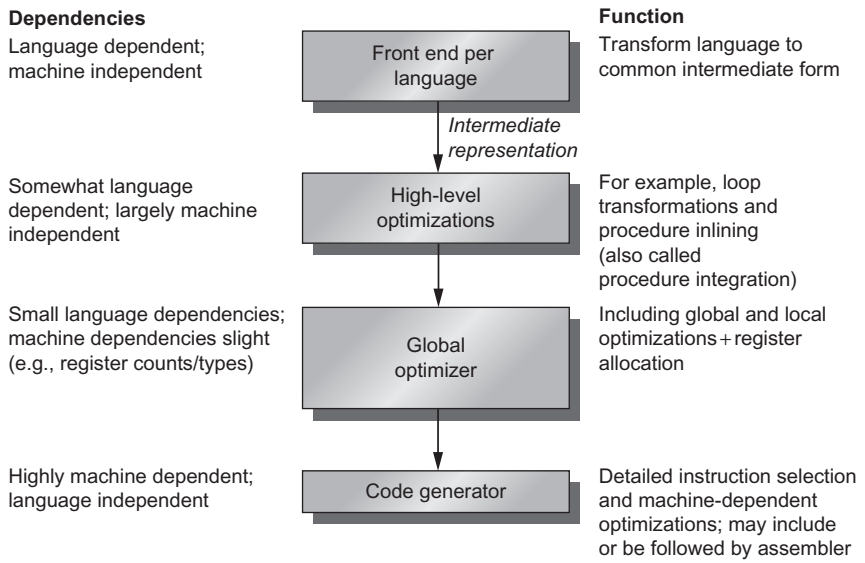


Figure A.19 Compilers typically consist of two to four passes, with more highly optimizing compilers having more passes. This structure maximizes the probability that a program compiled at various levels of optimization will produce the same output when given the same input. The optimizing passes are designed to be optional and may be skipped when faster compilation is the goal and lower-quality code is acceptable. A *pass* is simply one phase in which the compiler reads and transforms the entire program. (The term *phase* is often used interchangeably with *pass*.) Because the optimizing passes are separated, multiple languages can use the same optimizing and code generation passes. Only a new front end is required for a new language.

in the compiler transform higher-level, more abstract representations into progressively lower-level representations. Eventually it reaches the instruction set. This structure helps manage the complexity of the transformations and makes writing a bug-free compiler easier.

The complexity of writing a correct compiler is a major limitation on the amount of optimization that can be done. Although the multiple-pass structure helps reduce compiler complexity, it also means that the compiler must order and perform some transformations before others. In the diagram of the optimizing compiler in Figure A.19, we can see that certain high-level optimizations are performed long before it is known what the resulting code will look like. Once such a transformation is made, the compiler can't afford to go back and revisit all steps, possibly undoing transformations. Such iteration would be prohibitive, both in compilation time and in complexity. Thus, compilers make assumptions about the ability of later steps to deal with certain problems. For example, compilers usually have to choose which procedure calls to expand inline before they know the exact size of the procedure being called. Compiler writers call this problem the *phase-ordering problem*.

How does this ordering of transformations interact with the instruction set architecture? A good example occurs with the optimization called *global common subexpression elimination*. This optimization finds two instances of an expression that compute the same value and saves the value of the first computation in a temporary. It then uses the temporary value, eliminating the second computation of the common expression.

For this optimization to be significant, the temporary must be allocated to a register. Otherwise, the cost of storing the temporary in memory and later reloading it may negate the savings gained by not recomputing the expression. There are, in fact, cases where this optimization actually slows down code when the temporary is not register allocated. Phase ordering complicates this problem because register allocation is typically done near the end of the global optimization pass, just before code generation. Thus, an optimizer that performs this optimization must *assume* that the register allocator will allocate the temporary to a register.

Optimizations performed by modern compilers can be classified by the style of the transformation, as follows:

- *High-level optimizations* are often done on the source with output fed to later optimization passes.
- *Local optimizations* optimize code only within a straight-line code fragment (called a *basic block* by compiler people).
- *Global optimizations* extend the local optimizations across branches and introduce a set of transformations aimed at optimizing loops.
- *Register allocation* associates registers with operands.
- *Processor-dependent optimizations* attempt to take advantage of specific architectural knowledge.

Register Allocation

Because of the central role that register allocation plays, both in speeding up the code and in making other optimizations useful, it is one of the most important—if not the most important—of the optimizations. Register allocation algorithms today are based on a technique called *graph coloring*. The basic idea behind graph coloring is to construct a graph representing the possible candidates for allocation to a register and then to use the graph to allocate registers. Roughly speaking, the problem is how to use a limited set of colors so that no two adjacent nodes in a dependency graph have the same color. The emphasis in the approach is to achieve 100% register allocation of active variables. The problem of coloring a graph in general can take exponential time as a function of the size of the graph (NP-complete). There are heuristic algorithms, however, that work well in practice, yielding close allocations that run in near-linear time.

Graph coloring works best when there are at least 16 (and preferably more) general-purpose registers available for global allocation for integer variables and additional registers for floating point. Unfortunately, graph coloring does not work very well when the number of registers is small because the heuristic algorithms for coloring the graph are likely to fail.

Impact of Optimizations on Performance

It is sometimes difficult to separate some of the simpler optimizations—local and processor-dependent optimizations—from transformations done in the code generator. Examples of typical optimizations are given in Figure A.20. The last column of Figure A.20 indicates the frequency with which the listed optimizing transforms were applied to the source program.

Figure A.21 shows the effect of various optimizations on instructions executed for two programs. In this case, optimized programs executed roughly 25%–90% fewer instructions than unoptimized programs. The figure illustrates the importance of looking at optimized code before suggesting new instruction set features, because a compiler might completely remove the instructions the architect was trying to improve.

The Impact of Compiler Technology on the Architect's Decisions

The interaction of compilers and high-level languages significantly affects how programs use an instruction set architecture. There are two important questions: how are variables allocated and addressed? How many registers are needed to allocate variables appropriately? To address these questions, we must look at the three separate areas in which current high-level languages allocate their data:

Optimization name	Explanation	Percentage of the total number of optimizing transforms
High-level	*At or near the source level; processor-independent*	
Procedure integration	Replace procedure call by procedure body	N.M.
Local	*Within straight-line code*	
Common subexpression elimination	Replace two instances of the same computation by single copy	18%
Constant propagation	Replace all instances of a variable that is assigned a constant with the constant	22%
Stack height reduction	Rearrange expression tree to minimize resources needed for expression evaluation	N.M.
Global	*Across a branch*	
Global common subexpression elimination	Same as local, but this version crosses branches	13%
Copy propagation	Replace all instances of a variable A that has been assigned X (i.e., $A=X$) with X	11%
Code motion	Remove code from a loop that computes same value each iteration of the loop	16%
Induction variable elimination	Simplify/eliminate array addressing calculations within loops	2%
Processor-dependent	*Depends on processor knowledge*	
Strength reduction	Many examples, such as replace multiply by a constant with adds and shifts	N.M.
Pipeline scheduling	Reorder instructions to improve pipeline performance	N.M.
Branch offset optimization	Choose the shortest branch displacement that reaches target	N.M.

Figure A.20 Major types of optimizations and examples in each class. These data tell us about the relative frequency of occurrence of various optimizations. The third column lists the static frequency with which some of the common optimizations are applied in a set of 12 small Fortran and Pascal programs. There are nine local and global optimizations done by the compiler included in the measurement. Six of these optimizations are covered in the figure, and the remaining three account for 18% of the total static occurrences. The abbreviation *N.M.* means that the number of occurrences of that optimization was not measured. Processor-dependent optimizations are usually done in a code generator, and none of those was measured in this experiment. The percentage is the portion of the static optimizations that are of the specified type. Data from Chow, F.C., 1983. A Portable Machine-Independent Global Optimizer—Design and Measurements (Ph.D. thesis). Stanford University, Palo Alto, CA (collected using the Stanford UCODE compiler).

- The *stack* is used to allocate local variables. The stack is grown or shrunk on procedure call or return, respectively. Objects on the stack are addressed relative to the stack pointer and are primarily scalars (single variables) rather than arrays. The stack is used for activation records, *not* as a stack for evaluating expressions. Hence, values are almost never pushed or popped on the stack.

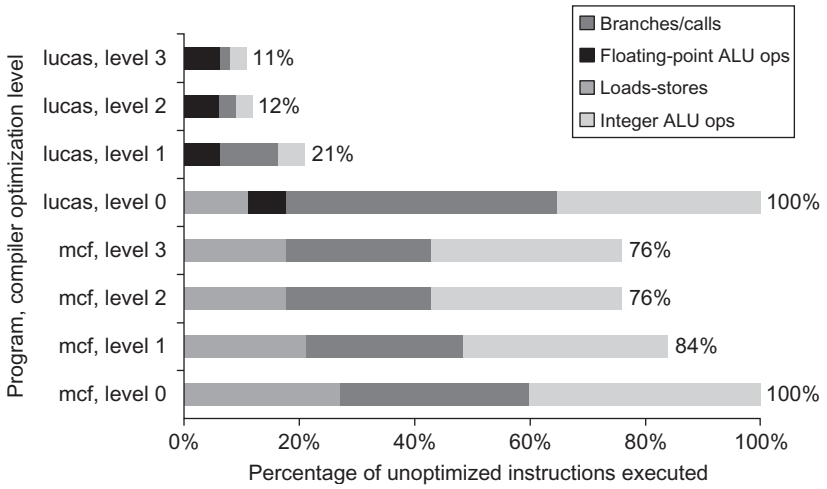


Figure A.21 Change in instruction count for the programs lucas and mcf from the SPEC2000 as compiler optimization levels vary. Level 0 is the same as unoptimized code. Level 1 includes local optimizations, code scheduling, and local register allocation. Level 2 includes global optimizations, loop transformations (software pipelining), and global register allocation. Level 3 adds procedure integration. These experiments were performed on Alpha compilers.

- The *global data area* is used to allocate statically declared objects, such as global variables and constants. A large percentage of these objects are arrays or other aggregate data structures.
- The *heap* is used to allocate dynamic objects that do not adhere to a stack discipline. Objects in the heap are accessed with pointers and are typically not scalars.

Register allocation is much more effective for stack-allocated objects than for global variables, and register allocation is essentially impossible for heap-allocated objects because they are accessed with pointers. Global variables and some stack variables are impossible to allocate because they are *aliased*—there are multiple ways to refer to the address of a variable, making it illegal to put it into a register. (Most heap variables are effectively aliased for today's compiler technology.)

For example, consider the following code sequence, where `&` returns the address of a variable and `*` dereferences a pointer:

```
p =&a  - gets address of a in p
a =...  - assigns to a directly
*p =...  - uses p to assign to a
...a...  - accesses a
```

The variable `a` could not be register allocated across the assignment to `*p` without generating incorrect code. Aliasing causes a substantial problem because it is

often difficult or impossible to decide what objects a pointer may refer to. A compiler must be conservative; some compilers will not allocate *any* local variables of a procedure in a register when there is a pointer that may refer to *one* of the local variables.

How the Architect Can Help the Compiler Writer

Today, the complexity of a compiler does not come from translating simple statements like A = B + C. Most programs are *locally simple*, and simple translations work fine. Rather, complexity arises because programs are large and globally complex in their interactions, and because the structure of compilers means decisions are made one step at a time about which code sequence is best.

Compiler writers often are working under their own corollary of a basic principle in architecture: *make the frequent cases fast and the rare case correct*. That is, if we know which cases are frequent and which are rare, and if generating code for both is straightforward, then the quality of the code for the rare case may not be very important—but it must be correct!

Some instruction set properties help the compiler writer. These properties should not be thought of as hard-and-fast rules, but rather as guidelines that will make it easier to write a compiler that will generate efficient and correct code.

- *Provide regularity*—Whenever it makes sense, the three primary components of an instruction set—the operations, the data types, and the addressing modes—should be *orthogonal*. Two aspects of an architecture are said to be orthogonal if they are independent. For example, the operations and addressing modes are orthogonal if, for every operation to which one addressing mode can be applied, all addressing modes are applicable. This regularity helps simplify code generation and is particularly important when the decision about what code to generate is split into two passes in the compiler. A good counterexample of this property is restricting what registers can be used for a certain class of instructions. Compilers for special-purpose register architectures typically get stuck in this dilemma. This restriction can result in the compiler finding itself with lots of available registers, but none of the right kind!
- *Provide primitives, not solutions*—Special features that "match" a language construct or a kernel function are often unusable. Attempts to support high-level languages may work only with one language or do more or less than is required for a correct and efficient implementation of the language. An example of how such attempts have failed is given in Section A.10.
- *Simplify trade-offs among alternatives*—One of the toughest jobs a compiler writer has is figuring out what instruction sequence will be best for every segment of code that arises. In earlier days, instruction counts or total code size might have been good metrics, but—as we saw in Chapter 1—this is no longer

true. With caches and pipelining, the trade-offs have become very complex. Anything the designer can do to help the compiler writer understand the costs of alternative code sequences would help improve the code. One of the most difficult instances of complex trade-offs occurs in a register-memory architecture in deciding how many times a variable should be referenced before it is cheaper to load it into a register. This threshold is hard to compute and, in fact, may vary among models of the same architecture.

- *Provide instructions that bind the quantities known at compile time as constants*—A compiler writer hates the thought of the processor interpreting at runtime a value that was known at compile time. Good counterexamples of this principle include instructions that interpret values that were fixed at compile time. For instance, the VAX procedure call instruction (`calls`) dynamically interprets a mask saying what registers to save on a call, but the mask is fixed at compile time (see Section A.10).

Compiler Support (or Lack Thereof) for Multimedia Instructions

Alas, the designers of the SIMD instructions (see Section 4.3 in Chapter 4) basically ignored the previous subsection. These instructions tend to be solutions, not primitives; they are short of registers; and the data types do not match existing programming languages. Architects hoped to find an inexpensive solution that would help some users, but often only a few low-level graphics library routines use them.

The SIMD instructions are really an abbreviated version of an elegant architecture style that has its own compiler technology. As explained in Section 4.2, *vector architectures* operate on vectors of data. Invented originally for scientific codes, multimedia kernels are often vectorizable as well, albeit often with shorter vectors. As Section 4.3 suggests, we can think of Intel's MMX and SSE or PowerPC's AltiVec, or the RISC-V P extension, as simply short vector computers: MMX with vectors of eight 8-bit elements, four 16-bit elements, or two 32-bit elements, and AltiVec with vectors twice that length. They are implemented as simply adjacent, narrow elements in wide registers.

These microprocessor architectures build the vector register size into the architecture: the sum of the sizes of the elements is limited to 64 bits for MMX and 128 bits for AltiVec. When Intel decided to expand to 128-bit vectors, it added a whole new set of instructions, called streaming SIMD extension (SSE).

A major advantage of vector computers is hiding latency of memory access by loading many elements at once and then overlapping execution with data transfer. The goal of vector addressing modes is to collect data scattered about memory, place them in a compact form so that they can be operated on efficiently, and then place the results back where they belong.

Vector computers include *strided addressing* and *gather/scatter addressing* (see Section 4.2) to increase the number of programs that can be vectorized. Strided addressing skips a fixed number of words between each access, so sequential addressing is often called *unit stride addressing*. Gather and scatter find their

addresses in another vector register: think of it as register indirect addressing for vector computers. From a vector perspective, in contrast, these short-vector SIMD computers support only unit strided accesses: memory accesses load or store all elements at once from a single wide memory location. Because the data for multimedia applications are often streams that start and end in memory, strided and gather/scatter addressing modes are essential to successful vectorization (see Section 4.7).

Example As an example, compare a vector computer with MMX for color representation conversion of pixels from RGB (red, green, blue) to YUV (luminosity chrominance), with each pixel represented by 3 bytes. The conversion is just three lines of C code placed in a loop:

```
Y = (9798*R + 19235*G + 3736*B) / 32768;
U = (-4784*R - 9437*G + 4221*B) / 32768 + 128;
V = (20218*R - 16941*G - 3277*B) / 32768 + 128;
```

A 64-bit-wide vector computer can calculate 8 pixels simultaneously. One vector computer for media with strided addresses takes

- 3 vector loads (to get RGB)
- 3 vector multiplies (to convert R)
- 6 vector multiply adds (to convert G and B)
- 3 vector shifts (to divide by 32,768)
- 2 vector adds (to add 128)
- 3 vector stores (to store YUV)

The total is 20 instructions to perform the 20 operations in the previous C code to convert 8 pixels (Kozyrakis, 2000). (Because a vector might have 32 64-bit elements, this code actually converts up to 32×8 or 256 pixels.)

In contrast, Intel's website shows that a library routine to perform the same calculation on 8 pixels takes 116 MMX instructions plus 6 80x86 instructions (Intel, 2001). This six-fold increase in instructions is due to the large number of instructions to load and unpack RGB pixels and to pack and store YUV pixels, because there are no strided memory accesses.

Having short, architecture-limited vectors with few registers and simple memory addressing modes makes it more difficult to use vectorizing compiler technology. Hence, these SIMD instructions are more likely to be found in hand-coded libraries than in compiled code.

Summary: The Role of Compilers

This section leads to several recommendations. First, we expect a new instruction set architecture to have at least 16 general-purpose registers—not counting

separate registers for floating-point numbers—to simplify allocation of registers using graph coloring. The advice on orthogonality suggests that all supported addressing modes apply to all instructions that transfer data. Finally, the last three pieces of advice—provide primitives instead of solutions, simplify trade-offs between alternatives, don't bind constants at runtime—all suggest that it is better to err on the side of simplicity. In other words, understand that less is more in the design of an instruction set. Alas, SIMD extensions are more an example of good marketing than of outstanding achievement of hardware–software co-design.

A.9 Putting It All Together: The RISC-V Architecture

In this section we describe the load-store architecture called RISC-V. RISC-V is a freely licensed open standard, similar to many of the RISC architectures, and based on observations similar to those covered in the last sections. (In Section M.3 we discuss how and why these architectures became popular.) RISC-V builds on 30 years of experience with RISC architectures and "cleans up" most of the short-term inclusions and omissions, leading to an architecture that is easier and more efficient to implement. RISC-V provides a both a 32-bit and a 64-bit instruction set, as well as a variety of extensions for features like floating point; these extensions can be added to either the 32-bit or 64-bit base instruction set. We discuss a 64-bit version of RISC-V, RV64, which is a superset of the 32-bit version RV32.

Reviewing our expectations from each section, for desktop and server applications:

- Section A.2—Use general-purpose registers with a load-store architecture.
- Section A.3—Support these addressing modes: displacement (with an address offset size of 12–16 bits), immediate (size 8–16 bits), and register indirect.
- Section A.4—Support these data sizes and types: 8-, 16-, 32-, and 64-bit integers and 64-bit IEEE 754 floating-point numbers.
- Section A.5—Support these simple instructions, because they will dominate the number of instructions executed: load, store, add, subtract, move register-register, and shift.
- Section A.6—Compare equal, compare not equal, compare less, branch (with a PC-relative address at least 8 bits long), jump, call, and return.
- Section A.7—Use fixed instruction encoding if interested in performance, and use variable instruction encoding if interested in code size. In some low-end, embedded applications, with small or only one-level caches, larger code size may have significant performance implications. ISAs that provide a compressed instruction set extension provide a way of addressing this difference.
- Section A.8—Provide at least 16, and preferably 32, general-purpose registers, be sure all addressing modes apply to all data transfer instructions, and aim for

a minimalist instruction set. This section didn't cover floating-point programs, but they often use separate floating-point registers. The justification is to increase the total number of registers without raising problems in the instruction format or in the speed of the general-purpose register file. This compromise, however, is not orthogonal.

We introduce RISC-V by showing how it follows these recommendations. Like its RISC predecessors, RISC-V emphasizes

- A simple load-store instruction set.
- Design for pipelining efficiency (discussed in Appendix C), including a fixed instruction set encoding.
- Efficiency as a compiler target.

RISC-V provides a good architectural model for study, not only because of the popularity of this type of processor, but also because it is an easy architecture to understand. We will use this architecture again in Appendix C and in Chapter 3, and it forms the basis for a number of exercises and programming projects.

RISC-V Instruction Set Organization

The RISC-V instruction set is organized as three base instruction sets that support 32-bit or 64-bit integers, and a variety of optional extensions to one of the base instruction sets. This allows RISC-V to be implemented for a wide range of potential applications from a small embedded processor with a minimal budget for logic and memory that likely costs $1 or less, to high-end processor configurations with full support for floating point, vectors, and multiprocessor configurations. Figure A.22 summarizes the three base instruction sets and the instruction set extensions with their basic functionality. For purposes of this text, we use RV64IMAFD (also known as RV64G, for short) in examples. RV32G is the 32-bit subset of the 64-bit architecture RV64G.

Registers for RISC-V

RV64G has 32 64-bit general-purpose registers (GPRs), named x0, x1, … , x31. GPRs are also sometimes known as *integer registers*. Additionally, with the F and D extensions for floating point that are part of RV64G, come a set of 32 floating-point registers (FPRs), named f0, f1, … , f31, which can hold 32 single-precision (32-bit) values or 32 double-precision (64-bit) values. (When holding one single-precision number, the other half of the FPR is unused.) Both single- and double-precision floating-point operations (32-bit and 64-bit) are provided.

The value of x0 is always 0. We shall see later how we can use this register to synthesize a variety of useful operations from a simple instruction set.

A few special registers can be transferred to and from the general-purpose registers. An example is the floating-point status register, used to hold information

Name of base or extension	Functionality
RV32I	Base 32-bit integer instruction set with 32 registers
RV32E	Base 32-bit instruction set but with only 16 registers; intended for very low-end embedded applications
RV64I	Base 64-bit instruction set; all registers are 64-bits, and instructions to move 64-bit from/to the registers (LD and SD) are added
M	Adds integer multiply and divide instructions
A	Adds atomic instructions needed for concurrent processing; see Chapter 5
F	Adds single precision (32-bit) IEEE floating point, includes 32 32-bit floating point registers, instructions to load and store those registers and operate on them
D	Extends floating point to double precision, 64-bit, making the registers 64-bits, adding instructions to load, store, and operate on the registers
Q	Further extends floating point to add support for quad precision, adding 128-bit operations
L	Adds support for 64- and 128-bit decimal floating point for the IEEE standard
C	Defines a compressed version of the instruction set intended for small-memory-sized embedded applications. Defines 16-bit versions of common RV32I instructions
V	A future extension to support vector operations (see Chapter 4)
B	A future extension to support operations on bit fields
T	A future extension to support transactional memory
P	An extension to support packed SIMD instructions: see Chapter 4
RV128I	A future base instruction set providing a 128-bit address space

Figure A.22 RISC-V has three base instructions sets (and a reserved spot for a future fourth); all the extensions extend one of the base instruction sets. An instruction set is thus named by the base name followed by the extensions. For example, RISC-V64IMAFD refers to the base 64-bit instruction set with extensions M, A, F, and D. For consistency of naming and software, this combination is given the abbreviated name: RV64G, and we use RV64G through most of this text.

about the results of floating-point operations. There are also instructions for moving between an FPR and a GPR.

Data Types for RISC-V

The data types are 8-bit bytes, 16-bit half words, 32-bit words, and 64-bit double-words for integer data and 32-bit single precision and 64-bit double precision for floating point. Half words were added because they are found in languages like C

and are popular in some programs, such as the operating systems, concerned about size of data structures. They will also become more popular if Unicode becomes widely used.

The RV64G operations work on 64-bit integers and 32- or 64-bit floating point. Bytes, half words, and words are loaded into the general-purpose registers with either zeros or the sign bit replicated to fill the 64 bits of the GPRs. Once loaded, they are operated on with the 64-bit integer operations.

Addressing Modes for RISC-V Data Transfers

The only data addressing modes are immediate and displacement, both with 12-bit fields. Register indirect is accomplished simply by placing 0 in the 12-bit displacement field, and limited absolute addressing with a 12-bit field is accomplished by using register 0 as the base register. Embracing zero gives us four effective modes, although only two are supported in the architecture.

RV64G memory is byte addressable with a 64-bit address and uses Little Endian byte numbering. As it is a load-store architecture, all references between memory and either GPRs or FPRs are through loads or stores. Supporting the data types mentioned herein, memory accesses involving GPRs can be to a byte, half word, word, or double word. The FPRs may be loaded and stored with single-precision or double-precision numbers. Memory accesses need not be aligned; however, it may be that unaligned accesses run extremely slow. In practice, programmers and compilers would be stupid to use unaligned accesses.

RISC-V Instruction Format

Because RISC-V has just two addressing modes, these can be encoded into the opcode. Following the advice on making the processor easy to pipeline and decode, all instructions are 32 bits with a 7-bit primary opcode. Figure A.23 shows the instruction layout of the four major instruction types. These formats are simple while providing 12-bit fields for displacement addressing, immediate constants, or PC-relative branch addresses.

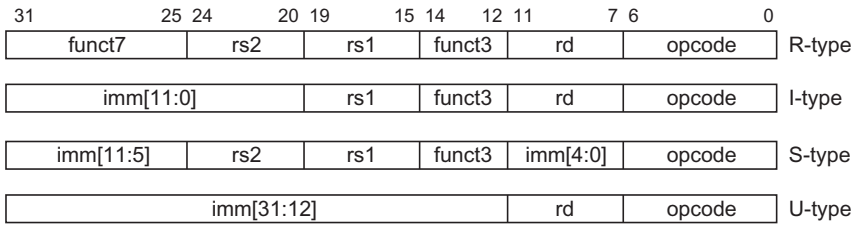


Figure A.23 The RISC-V instruction layout. There are two variations on these formata, called the SB and UJ formats; they deal with a slightly different treatment for immediate fields.

Instruction format	Primary use	rd	rs1	rs2	Immediate
R-type	Register-register ALU instructions	Destination	First source	Second source	
I-type	ALU immediates Load	Destination	First source base register		Value displacement
S-type	Store Compare and branch		Base register first source	Data source to store second source	Displacement offset
U-type	Jump and link Jump and link register	Register destination for return PC	Target address for jump and link register		Target address for jump and link

Figure A.24 The use of instruction fields for each instruction type. Primary use shows the major instructions that use the format. A blank indicates that the corresponding field is not present in this instruction type. The I-format is used for both loads and ALU immediates, with the 12-bit immediate holding either the value for an immediate or the displacement for a load. Similarly, the S-format encodes both store instructions (where the first source register is the base register and the second contains the register source for the value to store) and compare and branch instructions (where the register fields contain the sources to compare and the immediate field specifies the offset of the branch target). There are actually two other formats: SB and UJ that follow the same basic organization as S and J, but slightly modify the interpretation of the immediate fields.

The instruction formats and the use of the instruction fields is described in Figure A.24. The opcode specifies the general instruction type (ALU instruction, ALU immediate, load, store, branch, or jump), while the funct fields are used for specific operations. For example, an ALU instruction is encoded with a single opcode with the funct field dictating the exact operation: add, subtract, and, etc. Notice that several formats encode multiple types of instructions, including the use of the I-format for both ALU immediates and loads, and the use of the S-format for stores and conditional branches.

RISC-V Operations

RISC-V (or more properly RV64G) supports the list of simple operations recommended herein plus a few others. There are four broad classes of instructions: loads and stores, ALU operations, branches and jumps, and floating-point operations.

Any of the general-purpose or floating-point registers may be loaded or stored, except that loading x0 has no effect. Figure A.25 gives examples of the load and store instructions. Single-precision floating-point numbers occupy half a floating-point register. Conversions between single and double precision must be done explicitly. The floating-point format is IEEE 754 (see Appendix J). A list of all the RV64G instructions appears in Figure A.28 (page A.42).

Example instruction	Instruction name	Meaning
`ld x1,80(x2)`	Load doubleword	Regs[x1] ← Mem[80+Regs[x2]]
`lw x1,60(x2)`	Load word	Regs[x1] $\leftarrow_{64}$ Mem[60+Regs[x2]]$_0$)^{32} ## Mem[60+Regs[x2]]
`lwu x1,60(x2)`	Load word unsigned	Regs[x1] $\leftarrow_{64}$ 0^{32} ## Mem[60+Regs[x2]]
`lb x1,40(x3)`	Load byte	Regs[x1] $\leftarrow_{64}$ (Mem[40+Regs[x3]]$_0$)^{56} ## Mem[40+Regs[x3]]
`lbu x1,40(x3)`	Load byte unsigned	Regs[x1] $\leftarrow_{64}$ 0^{56} ## Mem[40+Regs[x3]]
`lh x1,40(x3)`	Load half word	Regs[x1] $\leftarrow_{64}$ (Mem[40+Regs[x3]]$_0$)^{48} ## Mem[40+Regs[x3]]
`flw f0,50(x3)`	Load FP single	Regs[f0] $\leftarrow_{64}$ Mem[50+Regs[x3]] ## 0^{32}
`fld f0,50(x2)`	Load FP double	Regs[f0] $\leftarrow_{64}$ Mem[50+Regs[x2]]
`sd x2,400(x3)`	Store double	Mem[400+Regs[x3]] $\leftarrow_{64}$ Regs[x2]
`sw x3,500(x4)`	Store word	Mem[500+Regs[x4]] $\leftarrow_{32}$ Regs[x3]$_{32..63}$
`fsw f0,40(x3)`	Store FP single	Mem[40+Regs[x3]] $\leftarrow_{32}$ Regs[f0]$_{0..31}$
`fsd f0,40(x3)`	Store FP double	Mem[40+Regs[x3]] $\leftarrow_{64}$ Regs[f0]
`sh x3,502(x2)`	Store half	Mem[502+Regs[x2]] $\leftarrow_{16}$ Regs[x3]$_{48..63}$
`sb x2,41(x3)`	Store byte	Mem[41+Regs[x3]] $\leftarrow_{8}$ Regs[x2]$_{56..63}$

Figure A.25 The load and store instructions in RISC-V. Loads shorter than 64 bits are available in both sign-extended and zero-extended forms. All memory references use a single addressing mode. Of course, both loads and stores are available for all the data types shown. Because RV64G supports double precision floating point, all single precision floating point loads must be aligned in the FP register, which are 64-bits wide.

To understand these figures we need to introduce a few additional extensions to our C description language used initially on page A-9:

- A subscript is appended to the symbol ← whenever the length of the datum being transferred might not be clear. Thus, $\leftarrow_n$ means transfer an n-bit quantity. We use $x, y \leftarrow z$ to indicate that z should be transferred to x and y.
- A subscript is used to indicate selection of a bit from a field. Bits are labeled from the most-significant bit starting at 0. The subscript may be a single digit (e.g., Regs[x4]$_0$ yields the sign bit of x4) or a subrange (e.g., Regs[x3]$_{56..63}$ yields the least-significant byte of x3).
- The variable Mem, used as an array that stands for main memory, is indexed by a byte address and may transfer any number of bytes.
- A superscript is used to replicate a field (e.g., 0^{48} yields a field of zeros of length 48 bits).
- The symbol ## is used to concatenate two fields and may appear on either side of a data transfer, and the symbols ≪ and ≫ shift the first operand left or right by the amount of the second operand.

Example instrucmtion	Instruction name	Meaning
`add x1,x2,x3`	Add	`Regs[x1]←Regs[x2]+Regs[x3]`
`addi x1,x2,3`	Add immediate unsigned	`Regs[x1]←Regs[x2]+3`
`lui x1,42`	Load upper immediate	$\texttt{Regs[x1]} \leftarrow 0^{32} \#\# 42 \#\# 0^{12}$
`sll x1,x2,5`	Shift left logical	`Regs[x1]←Regs[x2]<<5`
`slt x1,x2,x3`	Set less than	`if (Regs[x2]<Regs[x3]) Regs[x1]←1 else Regs[x1]←0`

Figure A.26 The basic ALU instructions in RISC-V are available both with register-register operands and with one immediate operand. LUI uses the U-format that employs the rs1 field as part of the immediate, yielding a 20-bit immediate.

As an example, assuming that x8 and x10 are 32-bit registers:

$$\texttt{Regs[x10]} \leftarrow_{64} (\texttt{Mem[Regs[x8]]}_0)^{32} \#\# \texttt{Mem[Regs[R8]]}$$

means that the word at the memory location addressed by the contents of register x8 is sign-extended to form a 64-bit quantity that is stored into register x10.

All ALU instructions are register-register instructions. Figure A.26 gives some examples of the arithmetic/logical instructions. The operations include simple arithmetic and logical operations: add, subtract, AND, OR, XOR, and shifts. Immediate forms of all these instructions are provided using a 12-bit sign-extended immediate. The operation `LUI` (load upper immediate) loads bits 12–31 of a register, sign-extends the immediate field to the upper 32-bits, and sets the low-order 12-bits of the register to 0. `LUI` allows a 32-bit constant to be built in two instructions, or a data transfer using any constant 32-bit address in one extra instruction.

As mentioned herein, x0 is used to synthesize popular operations. Loading a constant is simply an add immediate where the source operand is x0, and a register-register move is simply an add (or an or) where one of the sources is x0. (We sometimes use the mnemonic li, standing for load immediate, to represent the former, and the mnemonic mv for the latter.)

RISC-V Control Flow Instructions

Control is handled through a set of jumps and a set of branches, and Figure A.27 gives some typical branch and jump instructions. The two jump instructions (jump and link and jump and link register) are unconditional transfers and always store the "link," which is the address of the instruction sequentially following the jump instruction, in the register specified by the rd field. In the event that the link address is not needed, the rd field can simply be set to x0, which results in a typical unconditional jump. The two jump instructions are differentiated by whether the address is computed by adding an immediate field to the PC or by adding the immediate

Example instruction	Instruction name	Meaning
`jal x1,offset`	Jump and link	`Regs[x1]←PC+4; PC←PC+(offset<<1)`
`jalr x1,x2,offset`	Jump and link register	`Regs[x1]←PC+4; PC←Regs[x2]+offset`
`beq x3,x4,offset`	Branch equal zero	`if (Regs[x3]==Regs[x4]) PC←PC+(offset<<1)`
`bgt x3,x4,name`	Branch not equal zero	`if (Regs[x3]>Regs[x4]) PC←PC+(offset<<1)`

Figure A.27 Typical control flow instructions in RISC-V. All control instructions, except jumps to an address in a register, are PC-relative.

field to the contents of a register. The offset is interpreted as a half word offset for compatibility with the compressed instruction set, R64C, which includes 16-bit instructions.

All branches are conditional. The branch condition is specified by the instruction, and any arithmetic comparison (equal, greater than, less than, and their inverses) is permitted. The branch-target address is specified with a 12-bit signed offset that is shifted left one place (to get 16-bit alignment) and then added to the current program counter. Branches based on the contents of the floating point registers are implemented by executing a floating point comparison (e.g., feq.d or fle.d), which sets an integer register to 0 or 1 based on the comparison, and then executing a beq or bne with x0 as an operand.

The observant reader will have noticed that there are very few 64-bit only instructions in RV64G. Primarily, these are the 64-bit loads and stores and versions of 32-bit, 16-bit, and 8-bit loads that do not sign extend (the default is to sign-extend). To support 32-bit modular arithmetic without additional instructions, there are versions of the instructions that ignore the upper 32 bits of a 64-bit register, such as add and subtract word (addw, subw). Amazingly, everything else just works.

RISC-V Floating-Point Operations

Floating-point instructions manipulate the floating-point registers and indicate whether the operation to be performed is single or double precision. The floating-point operations are add, subtract, multiply, divide, square root, as well as fused multiply-add and multiply-subtract. All floating point instructions begin with the letter `f` and use the suffix `d` for double precision and `s` for single precision (e.g., `fadd.d`, `fadd.s`, `fmul.d`, `fmul.s`, `fmadd.d fmadd.s`). Floating-point compares set an integer register based on the comparison, similarly to the integer instruction set-less-than and set-great-than.

In addition to floating-point loads and stores (`flw, fsw, fld, fsd`), instructions are provided for converting between different FP precisions, for moving between integer and FP registers (fmv), and for converting between floating point and integer (fcvt, which uses the integer registers for source or destination as appropriate).

Figure A.28 contains a list of nearly all the RV64G instructions and a summary of their meaning.

Instruction type/opcode	Instruction meaning
Data transfers	*Move data between registers and memory, or between the integer and FP; only memory address mode is 12-bit displacement + contents of a GPR*
`lb, lbu, sb`	Load byte, load byte unsigned, store byte (to/from integer registers)
`lh, lhu, sh`	Load half word, load half word unsigned, store half word (to/from integer registers)
`lw, lwu, sw`	Load word, store word (to/from integer registers)
`ld, sd`	Load doubleword, store doubleword
Arithmetic/logical	*Operations on data in GPRs. Word versions ignore upper 32 bits*
`add, addi, addw, addiw, sub, subi, subw, subiw`	Add and subtract, with both word and immediate versions
`slt, sltu, slti, sltiu`	set-less-than with signed and unsigned, and immediate
`and, or, xor, andi, ori, xori`	and, or, xor, both register-register and register-immediate
`lui`	Load upper immediate: loads bits 31..12 of a register with the immediate value. Upper 32 bits are set to 0
`auipc`	Sums an immediate and the upper 20-bits of the PC into a register; used for building a branch to any 32-bit address
`sll, srl, sra, slli, srli, srai, sllw,slliw, srli, srliw, srai, sraiw`	Shifts: logical shift left and right and arithmetic shift right, both immediate and word versions (word versions leave the upper 32 bit untouched)
`mul, mulw, mulh, mulhsu, mulhu, div,divw, divu, rem, remu, remw, remuw`	Integer multiply, divide, and remainder, signed and unsigned with support for 64-bit products in two instructions. Also word versions
Control	*Conditional branches and jumps; PC-relative or through register*
`beq, bne, blt, bge, bltu, bgeu`	Branch based on compare of two registers, equal, not equal, less than, greater or equal, signed and unsigned
`jal,jalr`	Jump and link address relative to a register or the PC
Floating point	*All FP operation appear in double precision (.d) and single (.s)*
`flw, fld, fsw, fsd`	Load, store, word (single precision), doubleword (double precision)
`fadd, fsub, fmult, fiv, fsqrt, fmadd, fmsub, fnmadd, fnmsub, fmin, fmax, fsgn, fsgnj, fsjnx`	Add, subtract, multiply, divide, square root, multiply-add, multiply-subtract, negate multiply-add, negate multiply-subtract, maximum, minimum, and instructions to replace the sign bit. For single precision, the opcode is followed by: .s, for double precision: .d. Thus fadd.s, fadd.d
`feq, flt, fle`	Compare two floating point registers; result is 0 or 1 stored into a GPR
`fmv.x.*, fmv.*.x`	Move between the FP register abd GPR, "*" is s or d
`fcvt.*.l, fcvt.l.*, fcvt.*.lu, fcvt.lu.*, fcvt.*.w, fcvt.w.*, fcvt.*.wu, fcvt.wu.*`	Converts between a FP register and integer register, where "*" is S or D for single or double precision. Signed and unsigned versions and word, doubleword versions

Figure A.28 A list of the vast majority of instructions in RV64G. This list can also be found on the back inside cover. This table omits system instructions, synchronization and atomic instructions, configuration instructions, instructions to reset and access performance counters, about 10 instructions in total.

Program	Loads	Stores	Branches	Jumps	ALU operations
astar	28%	6%	18%	2%	46%
bzip	20%	7%	11%	1%	54%
gcc	17%	23%	20%	4%	36%
gobmk	21%	12%	14%	2%	50%
h264ref	33%	14%	5%	2%	45%
hmmer	28%	9%	17%	0%	46%
libquantum	16%	6%	29%	0%	48%
mcf	35%	11%	24%	1%	29%
omnetpp	23%	15%	17%	7%	31%
perlbench	25%	14%	15%	7%	39%
sjeng	19%	7%	15%	3%	56%
xalancbmk	30%	8%	27%	3%	31%

Figure A.29 RISC-V dynamic instruction mix for the SPECint2006 programs. Omnetpp includes 7% of the instructions that are floating point loads, stores, operations, or compares; no other program includes even 1% of other instruction types. A change in gcc in SPECint2006, creates an anomaly in behavior. Typical integer programs have load frequencies that are 1/5 to 3x the store frequency. In gcc, the store frequency is actually higher than the load frequency! This arises because a large fraction of the execution time is spent in a loop that clears memory by storing x0 (not where a compiler like gcc would usually spend most of its execution time!). A store instruction that stores a register pair, which some other RISC ISAs have included, would address this issue.

RISC-V Instruction Set Usage

To give an idea of which instructions are popular, Figure A.29 shows the frequency of instructions and instruction classes for the SPECint2006 programs, using RV32G.

A.10 Fallacies and Pitfalls

Architects have repeatedly tripped on common, but erroneous, beliefs. In this section we look at a few of them.

Pitfall *Designing a "high-level" instruction set feature specifically oriented to supporting a high-level language structure.*

Attempts to incorporate high-level language features in the instruction set have led architects to provide powerful instructions with a wide range of flexibility. However, often these instructions do more work than is required in the frequent case, or they don't exactly match the requirements of some languages. Many such efforts have been aimed at eliminating what in the 1970s was called the *semantic gap*. Although the idea is to supplement the instruction set with additions that bring

the hardware up to the level of the language, the additions can generate what Wulf et al. (1981) have called a *semantic clash*:

> … by giving too much semantic content to the instruction, the computer designer made it possible to use the instruction only in limited contexts. [p. 43]

More often the instructions are simply overkill—they are too general for the most frequent case, resulting in unneeded work and a slower instruction. Again, the VAX `CALLS` is a good example. `CALLS` uses a callee save strategy (the registers to be saved are specified by the callee), *but* the saving is done by the call instruction in the caller. The `CALLS` instruction begins with the arguments pushed on the stack, and then takes the following steps:

1. Align the stack if needed.
2. Push the argument count on the stack.
3. Save the registers indicated by the procedure call mask on the stack (as mentioned in Section A.8). The mask is kept in the called procedure's code—this permits the callee to specify the registers to be saved by the caller even with separate compilation.
4. Push the return address on the stack, and then push the top and base of stack pointers (for the activation record).
5. Clear the condition codes, which sets the trap enable to a known state.
6. Push a word for status information and a zero word on the stack.
7. Update the two stack pointers.
8. Branch to the first instruction of the procedure.

The vast majority of calls in real programs do not require this amount of overhead. Most procedures know their argument counts, and a much faster linkage convention can be established using registers to pass arguments rather than the stack in memory. Furthermore, the `CALLS` instruction forces two registers to be used for linkage, while many languages require only one linkage register. Many attempts to support procedure call and activation stack management have failed to be useful, either because they do not match the language needs or because they are too general and hence too expensive to use.

The VAX designers provided a simpler instruction, `JSB`, that is much faster because it only pushes the return PC on the stack and jumps to the procedure. However, most VAX compilers use the more costly `CALLS` instructions. The call instructions were included in the architecture to standardize the procedure linkage convention. Other computers have standardized their calling convention by agreement among compiler writers and without requiring the overhead of a complex, very general procedure call instruction.

Fallacy *There is such a thing as a typical program.*

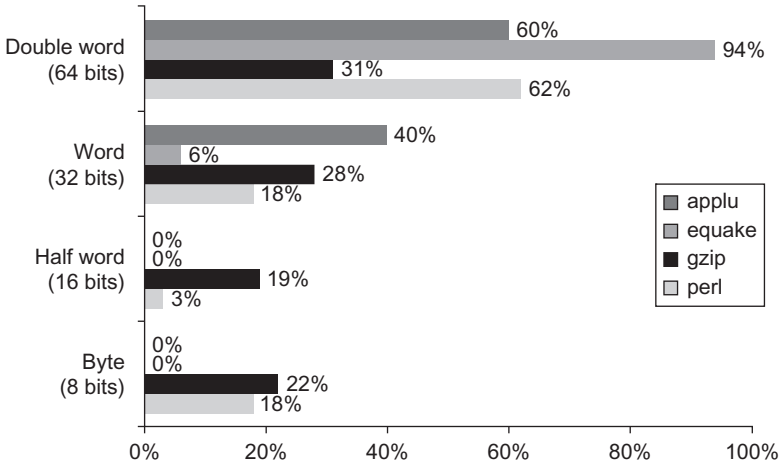


Figure A.30 Data reference size of four programs from SPEC2000. Although you can calculate an average size, it would be hard to claim the average is typical of programs.

Many people would like to believe that there is a single "typical" program that could be used to design an optimal instruction set. For example, see the synthetic benchmarks discussed in Chapter 1. The data in this appendix clearly show that programs can vary significantly in how they use an instruction set. For example, Figure A.30 shows the mix of data transfer sizes for four of the SPEC2000 programs: It would be hard to say what is typical from these four programs. The variations are even larger on an instruction set that supports a class of applications, such as decimal instructions, that are unused by other applications.

Pitfall *Innovating at the instruction set architecture to reduce code size without accounting for the compiler.*

Figure A.31 shows the relative code sizes for four compilers for the MIPS instruction set. Whereas architects struggle to reduce code size by 30%–40%, different compiler strategies can change code size by much larger factors. Similar to performance optimization techniques, the architect should start with the tightest code the compilers can produce before proposing hardware innovations to save space.

Fallacy *An architecture with flaws cannot be successful.*

The 80x86 provides a dramatic example: the instruction set architecture is one only its creators could love (see Appendix K). Succeeding generations of Intel engineers have tried to correct unpopular architectural decisions made in designing the 80x86. For example, the 80x86 supports segmentation, whereas all others picked paging; it uses extended accumulators for integer data, but other processors use general-purpose registers; and it uses a stack for floating-point data, when everyone else abandoned execution stacks long before.

Compiler	Apogee software version 4.1	Green Hills Multi2000 Version 2.0	Algorithmics SDE4.0B	IDT/c 7.2.1
Architecture	MIPS IV	MIPS IV	MIPS 32	MIPS 32
Processor	NEC VR5432	NEC VR5000	IDT 32334	IDT 79RC32364
Autocorrelation kernel	1.0	2.1	1.1	2.7
Convolutional encoder kernel	1.0	1.9	1.2	2.4
Fixed-point bit allocation kernel	1.0	2.0	1.2	2.3
Fixed-point complex FFT kernel	1.0	1.1	2.7	1.8
Viterbi GSM decoder kernel	1.0	1.7	0.8	1.1
Geometric mean of five kernels	1.0	1.7	1.4	2.0

Figure A.31 Code size relative to Apogee Software Version 4.1 C compiler for Telecom application of EEMBC benchmarks. The instruction set architectures are virtually identical, yet the code sizes vary by factors of 2. These results were reported February–June 2000.

Despite these major difficulties, the 80x86 architecture has been enormously successful. The reasons are threefold: first, its selection as the microprocessor in the initial IBM PC makes 80x86 binary compatibility extremely valuable. Second, Moore's Law provided sufficient resources for 80x86 microprocessors to translate to an internal RISC instruction set and then execute RISC-like instructions. This mix enables binary compatibility with the valuable PC software base and performance on par with RISC processors. Third, the very high volumes of PC microprocessors mean Intel can easily pay for the increased design cost of hardware translation. In addition, the high volumes allow the manufacturer to go up the learning curve, which lowers the cost of the product.

The larger die size and increased power for translation may be a liability for embedded applications, but it makes tremendous economic sense for the desktop. And its cost-performance in the desktop also makes it attractive for servers, with its main weakness for servers being 32-bit addresses, which was resolved with a 64-bit address extension.

Fallacy *You can design a flawless architecture.*

All architecture design involves trade-offs made in the context of a set of hardware and software technologies. Over time those technologies are likely to change, and decisions that may have been correct at the time they were made look like mistakes. For example, in 1975 the VAX designers overemphasized the importance of code size efficiency, underestimating how important ease of decoding and pipelining would be five years later. An example in the RISC camp is delayed branch (see Appendix K). It was a simple matter to control pipeline hazards with five-stage pipelines, but a challenge for processors with longer pipelines that issue multiple instructions per clock cycle. In addition, almost all architectures eventually succumb to the lack of sufficient address space. This is one reason that RISC-V

has planned for the possibility of 128-bit addresses, although it may be decades before such capability is needed.

In general, avoiding such flaws in the long run would probably mean compromising the efficiency of the architecture in the short run, which is dangerous, since a new instruction set architecture must struggle to survive its first few years.

A.11 Concluding Remarks

The earliest architectures were limited in their instruction sets by the hardware technology of that time. As soon as the hardware technology permitted, computer architects began looking for ways to support high-level languages. This search led to three distinct periods of thought about how to support programs efficiently. In the 1960s, stack architectures became popular. They were viewed as being a good match for high-level languages—and they probably were, given the compiler technology of the day. In the 1970s, the main concern of architects was how to reduce software costs. This concern was met primarily by replacing software with hardware, or by providing high-level architectures that could simplify the task of software designers. The result was both the high-level language computer architecture movement and powerful architectures like the VAX, which has a large number of addressing modes, multiple data types, and a highly orthogonal architecture. In the 1980s, more sophisticated compiler technology and a renewed emphasis on processor performance saw a return to simpler architectures, based mainly on the load-store style of computer.

The following instruction set architecture changes occurred in the 1990s:

- *Address size doubles*—The 32-bit address instruction sets for most desktop and server processors were extended to 64-bit addresses, expanding the width of the registers (among other things) to 64 bits. Appendix K gives three examples of architectures that have gone from 32 bits to 64 bits.
- *Optimization of conditional branches via conditional execution*—In Chapter 3, we see that conditional branches can limit the performance of aggressive computer designs. Hence, there was interest in replacing conditional branches with conditional completion of operations, such as conditional move (see Appendix H), which was added to most instruction sets.
- *Optimization of cache performance via prefetch*—Chapter 2 explains the increasing role of memory hierarchy in the performance of computers, with a cache miss on some computers taking as many instruction times as page faults took on earlier computers. Hence, prefetch instructions were added to try to hide the cost of cache misses by prefetching (see Chapter 2).
- *Support for multimedia*—Most desktop and embedded instruction sets were extended with support for multimedia applications.

- *Faster floating-point operations*—Appendix J describes operations added to enhance floating-point performance, such as operations that perform a multiply and an add and paired single execution, which are part of RISC-V.

Between 1970 and 1985 many thought the primary job of the computer architect was the design of instruction sets. As a result, textbooks of that era emphasize instruction set design, much as computer architecture textbooks of the 1950s and 1960s emphasized computer arithmetic. The educated architect was expected to have strong opinions about the strengths and especially the weaknesses of the popular computers. The importance of binary compatibility in quashing innovations in instruction set design was unappreciated by many researchers and textbook writers, giving the impression that many architects would get a chance to design an instruction set.

The definition of computer architecture today has been expanded to include design and evaluation of the full computer system—not just the definition of the instruction set and not just the processor—and hence there are plenty of topics for the architect to study. In fact, the material in this appendix was a central point of the book in its first edition in 1990, but now is included in an appendix primarily as reference material!

Appendix K may satisfy readers interested in instruction set architecture; it describes a variety of instruction sets, which are either important in the marketplace today or historically important, and it compares nine popular load-store computers with RISC-V.

A.12 Historical Perspective and References

Section M.4 (available online) features a discussion on the evolution of instruction sets and includes references for further reading and exploration of related topics.

Exercises by Gregory D. Peterson

A.1 [10] <A.9> Compute the effective CPI for an implementation of an embedded RISC-V CPU using Figure A.29. Assume we have made the following measurements of average CPI for each of the instruction types:

Instruction	Clock cycles
All ALU operations	1.0
Loads	5.0
Stores	3.0
Branches	
Taken	5.0
Not taken	3.0
Jumps	3.0

Average the instruction frequencies of astar and gcc to obtain the instruction mix.

A.2 [10] < A.9 > Compute the effective CPI for RISC-V using Figure A.29 and the table above. Average the instruction frequencies of bzip and hmmer to obtain the instruction mix. You may assume that all other instructions (for instructions not accounted for by the types in Table A.29) require 3.0 clock cycles each.

A.3 [10] < A.9 > Compute the effective CPI for an implementation of a RISC-V CPU using Figure A.29. Assume we have made the following measurements of average CPI for each of the instruction types:

Instruction	Clock cycles
All ALU operations	1.0
Loads	3.5
Stores	2.8
Branches	
Taken	4.0
Not taken	2.0
Jumps	2.4

Average the instruction frequencies of gobmk and mcf to obtain the instruction mix. You may assume that all other instructions (for instructions not accounted for by the types in Table A.29) require 3.0 clock cycles each.

A.4 [10] < A.9 > Compute the effective CPI for RISC-V using Figure A.29 and the table above. Average the instruction frequencies of perlbench and sjeng to obtain the instruction mix.

A.5 [10] < A.8 > Consider this high-level code sequence of three statements:

```
A = B + C;
B = A + C;
D = A - B;
```

Use the technique of copy propagation (see Figure A.20) to transform the code sequence to the point where no operand is a computed value. Note the instances in which the transformation has reduced the computational work of a statement and those cases where the work has increased. What does this suggest about the technical challenge faced in trying to satisfy the desire for optimizing compilers?

A.6 [30] < A.8 > Compiler optimizations may result in improvements to code size and/or performance. Consider one or more of the benchmark programs from the SPEC CPU2017 or the EEMBC benchmark suites. Use the RISC-V processor or a processor available to you along with the GNU C compiler to optimize the benchmark program(s) using no optimization, –O1, –O2, and –O3. Compare the performance and size of the resulting programs. Also compare your results to Figure A.21.

A.7 [20/20/20/25/10] <A.2, A.9> Consider the following fragment of C code:

```
for (i=0; i<100; i++) {
      A[i]=B[i]+C;
}
```

Assume that A and B are arrays of 64-bit integers, and C and i are 64-bit integers. Assume that all data values and their addresses are kept in memory (at addresses 1000, 3000, 5000, and 7000 for A, B, C, and i, respectively) except when they are operated on. Assume that values in registers are lost between iterations of the loop. Assume all addresses and words are 64 bits.

a. [20] <A.2, A.9> Write the code for RISC-V. How many instructions are required dynamically? How many memory-data references will be executed? What is the code size in bytes?

b. [20] <A.2> Write the code for x86. How many instructions are required dynamically? How many memory-data references will be executed? What is the code size in bytes?

c. [20] <A.2> Write the code for a stack machine. Assume all operations occur on top of the stack. Push and pop are the only instructions that access memory; all others remove their operands from the stack and replace them with the result. The implementation uses a hardwired stack for only the top two stack entries, which keeps the processor circuit very small and low in cost. Additional stack positions are kept in memory locations, and accesses to these stack positions require memory references. How many instructions are required dynamically? How many memory-data references will be executed?

d. [25] <A.2, A.9> Instead of the code fragment above, write a routine for computing a matrix multiplication for dense, single precision matrices, also known as SGEMM. For input matrices of size 100×100, how many instructions are required dynamically? How many memory-data references will be executed?

e. [10] <A.2, A.9> As the matrix size increases, how does this affect the number of instructions executed dynamically or the number of memory-data references?

A.8 [25/25] <A.2, A.8, A.9> Consider the following fragment of C code:

```
for(p=0; p<8; p++) {
     Y[p]=(9798*R[p]+19235*G[p]+3736*B[p])/32768;
     U[p]=(-4784*R[p]-9437*G[p]+4221*B[p])/32768+128;
     V[p]=(20218*R[p]-16941*G[p]-3277*B[p])/32768+128;
}
```

Assume that R, G, B, Y, U, and V are arrays of 64-bit integers. Assume that all data values and their addresses are kept in memory (at addresses 1000, 2000, 3000, 4000, 5000, and 6000 for R, G, B, Y, U, and V, respectively) except when they are operated on. Assume that values in registers are lost between iterations of the loop. Assume all addresses and words are 64 bits.

a. [25] < A.2, A.9 > Write the code for RISC-V. How many instructions are required dynamically? How many memory-data references will be executed? What is the code size in bytes?

b. [25] < A.2 > Write the code for x86. How many instructions are required dynamically? How many memory-data references will be executed? What is the code size in bytes? Compare your results to the multimedia instructions (MMX) and vector implementations discussed in the A.8.

A.9 [10/10/10/10] < A.2, A.7 > For the following, we consider instruction encoding for instruction set architectures.

a. [10] < A.2, A.7 > Consider the case of a processor with an instruction length of 14 bits and with 64 general-purpose registers so the size of the address fields is 6 bits. Is it possible to have instruction encodings for the following?
   - 3 two-address instructions
   - 63 one-address instructions
   - 45 zero-address instructions

b. [10] < A.2, A.7 > Assuming the same instruction length and address field sizes as above, determine if it is possible to have
   - 3 two-address instructions
   - 65 one-address instructions
   - 35 zero-address instructions

   Explain your answer.

c. [10] < A.2, A.7 > Assume the same instruction length and address field sizes as above. Further assume there are already 3 two-address and 24 zero-address instructions. What is the maximum number of one-address instructions that can be encoded for this processor?

d. [10] < A.2, A.7 > Assume the same instruction length and address field sizes as above. Further assume there are already 3 two-address and 65 zero-address instructions. What is the maximum number of one-address instructions that can be encoded for this processor?

A.10 [10/15] < A.2 > For the following assume that integer values A, B, C, D, E, and F reside in memory. Also assume that instruction operation codes are represented in 8 bits, memory addresses are 64 bits, and register addresses are 6 bits.

a. [10] < A.2 > For each instruction set architecture shown in Figure A.2, how many addresses, or names, appear in each instruction for the code to compute C = A + B, and what is the total code size?

b. [15] < A.2 > Some of the instruction set architectures in Figure A.2 destroy operands in the course of computation. This loss of data values from processor internal storage has performance consequences. For each architecture in Figure A.2, write the code sequence to compute:

```
C = A + B
D = A – E
F = C + D
```

In your code, mark each operand that is destroyed during execution and mark each "overhead" instruction that is included just to overcome this loss of data from processor internal storage. What is the total code size, the number of bytes of instructions and data moved to or from memory, the number of overhead instructions, and the number of overhead data bytes for each of your code sequences?

A.11 [15] < A.2, A.7, A.9 > The design of RISC-V provides for 32 general-purpose registers and 32 floating-point registers. If registers are good, are more registers better? List and discuss as many trade-offs as you can that should be considered by instruction set architecture designers examining whether to, and how much to, increase the number of RISC-V registers.

A.12 [5] < A.3 > Consider a C struct that includes the following members:

```
struct foo {
    char a;
    bool b;
    int c;
    double d;
    short e;
    float f;
    double g;
    char *cptr;
    float *fptr;
    int x;
};
```

Note that for C, the compiler must keep the elements of the struct in the same order as given in the struct definition. For a 32-bit machine, what is the size of the foo struct? What is the minimum size required for this struct, assuming you may arrange the order of the struct members as you wish? What about for a 64-bit machine?

A.13 [30] < A.7 > Many computer manufacturers now include tools or simulators that allow you to measure the instruction set usage of a user program. Among the methods in use are machine simulation, hardware-supported trapping, and techniques that instrument the object code module by inserting counters in software or using built-in hardware counters. Pick a processor and tools to instrument user programs. (The open source RISC-V architecture supports a collection of tools. Tools such as the Performance API (PAPI) work with x86 processors.) Use the processor and tools to measure the instruction set mix for one of the SPEC CPU2017 benchmarks. Compare the results to those shown in this chapter.

A.14 [30] < A.8 > Newer processors such as Intel's i7 Kaby Lake include support for AVX2 vector/multimedia instructions. Write a dense matrix multiply function using single-precision values and compile it with different compilers and optimization flags. Linear algebra codes using Basic Linear Algebra Subroutine (BLAS) routines such as SGEMM include optimized versions of dense matrix multiply. Compare the code size and performance of your code to that of BLAS SGEMM. Explore what happens when using double-precision values and DGEMM.

A.15 [30] < A.8 > For the SGEMM code developed above for the i7 processor, include the use of AVX2 intrinsics to improve the performance. In particular, try to vectorize your code to better utilize the AVX hardware. Compare the code size and performance to the original code. Compare your results to Intel's Math Kernel Library (MKL) implementation for SGEMM.

A.16 [30] < A.7, A.9 > The RISC-V processor is open source and boasts an impressive collection of implementations, simulators, compilers, and other tools. See riscv.org for an overview of tools, including spike, a simulator for RISC-V processors. Use spike or another simulator to measure the instruction set mix for some SPEC CPU2017 benchmark programs.

A.17 [35/35/35/35] < A.2–A.8 > gcc targets most modern instruction set architectures (see www.gnu.org/software/gcc/). Create a version of gcc for several architectures that you have access to, such as x86, RISC-V, PowerPC, and ARM.

a. [35] < A.2–A.8 > Compile a subset of SPEC CPU2017 integer benchmarks and create a table of code sizes. Which architecture is best for each program?

b. [35] < A.2–A.8 > Compile a subset of SPEC CPU2017 floating-point benchmarks and create a table of code sizes. Which architecture is best for each program?

c. [35] < A.2–A.8 > Compile a subset of EEMBC AutoBench benchmarks (see www.eembc.org/home.php) and create a table of code sizes. Which architecture is best for each program?

d. [35] < A.2–A.8 > Compile a subset of EEMBC FPBench floating-point benchmarks and create a table of code sizes. Which architecture is best for each program?

A.18 [40] < A.2–A.8 > Power efficiency has become very important for modern processors, particularly for embedded systems. Create a version of gcc for two architectures that you have access to, such as x86, RISC-V, PowerPC, Atom, and ARM. (Note that the different versions of RISC-V can also be explored and compared.) Compile a subset of EEMBC benchmarks while using EnergyBench to measure energy usage during execution. Compare code size, performance, and energy usage for the processors. Which is best for each program?

A.19 [20/15/15/20] Your task is to compare the memory efficiency of four different styles of instruction set architectures. The architecture styles are:

- *Accumulator*—All operations occur between a single register and a memory location.
- *Memory-memory*—All instruction addresses reference only memory locations.
- *Stack*—All operations occur on top of the stack. Push and pop are the only instructions that access memory; all others remove their operands from the stack and replace them with the result. The implementation uses a hardwired stack for only the top two stack entries, which keeps the processor circuit very small and low in cost. Additional stack positions are kept in memory locations, and accesses to these stack positions require memory references.

- *Load-store*—All operations occur in registers, and register-to-register instructions have three register names per instruction.

To measure memory efficiency, make the following assumptions about all four instruction sets:

- All instructions are an integral number of bytes in length.
- The opcode is always one byte (8 bits).
- Memory accesses use direct, or absolute, addressing.
- The variables `A`, `B`, `C`, and `D` are initially in memory.

a. [20] <A.2, A.3> Invent your own assembly language mnemonics (Figure A.2 provides a useful sample to generalize), and for each architecture write the best equivalent assembly language code for this high-level language code sequence:

```
A = B + C;
B = A + C;
D = A – B;
```

b. [15] <A.3> Label each instance in your assembly codes for part (a) where a value is loaded from memory after having been loaded once. Also label each instance in your code where the result of one instruction is passed to another instruction as an operand, and further classify these events as involving storage within the processor or storage in memory.

c. [15] <A.7> Assume that the given code sequence is from a small, embedded computer application that uses a 16-bit memory address and data operands. If a load-store architecture is used, assume it has 16 general-purpose registers. For each architecture answer the following questions: How many instruction bytes are fetched? How many bytes of data are transferred from/to memory? Which architecture is most efficient as measured by total memory traffic (code + data)?

d. [20] <A.7> Now assume a processor with 64-bit memory addresses and data operands. For each architecture answer the questions of part (c). How have the relative merits of the architectures changed for the chosen metrics?

A.20 [30] <A.2, A.3> Use the four different instruction set architecture styles from above, but assume that the memory operations supported include register indirect as well as direct addressing. Invent your own assembly language mnemonics (Figure A.2 provides a useful sample to generalize), and for each architecture, write the best equivalent assembly language code for this fragment of C code:

```
for (i = 0; i <= 100; i++) {
     A[i] = B[i] * C + D ;
}
```

Assume that `A` and `B` are arrays of 64-bit integers, and `C`, `D`, and `i` are 64-bit integers.

A.21 [20/20] <A.3, A.6, A.9> The size of displacement values needed for the displacement addressing mode or for PC-relative addressing can be extracted from compiled applications. Use a disassembler with one or more of the SPEC CPU2017 or EEMBC benchmarks compiled for the RISC-V processor.

a. [20] <A.3, A.9> For each instruction using displacement addressing, record the displacement value used. Create a histogram of displacement values. Compare the results to those shown in this appendix in Figure A.8.

b. [20] <A.6, A.9> For each branch instruction using PC-relative addressing, record the offset value used. Create a histogram of offset values. Compare the results to those shown in this chapter in Figure A.15.

A.22 [15/15/10/10/10/10] <A.3> The value represented by the hexadecimal number 5249 5343 5643 5055 is to be stored in an aligned 64-bit double word.

a. [15] <A.3> Using the physical arrangement of the first row in Figure A.5, write the value to be stored using Big Endian byte order. Next, interpret each byte as an ASCII character and below each byte write the corresponding character, forming the character string as it would be stored in Big Endian order.

b. [15] <A.3> Using the same physical arrangement as in part (a), write the value to be stored using Little Endian byte order, and below each byte write the corresponding ASCII character.

c. [10] <A.3> What are the hexadecimal values of all misaligned 2-byte words that can be read from the given 64-bit double word when stored in Big Endian byte order?

d. [10] <A.3> What are the hexadecimal values of all misaligned 2-byte words that can be read from the given 64-bit double word when stored in Big Endian byte order?

e. [10] <A.3> What are the hexadecimal values of all misaligned 2-byte words that can be read from the given 64-bit double word when stored in Little Endian byte order?

f. [10] <A.3> What are the hexadecimal values of all misaligned 4-byte words that can be read from the given 64-bit double word when stored in Little Endian byte order?

A.23 [25,25] <A.3, A.9> The relative frequency of different addressing modes impacts the choices of addressing modes support for an instruction set architecture. Figure A.7 illustrates the relative frequency of addressing modes for three applications on the VAX.

a. [25] <A.3> Compile one or more programs from the SPEC CPU2017 or EEMBC benchmark suites to target the x86 architecture. Using a disassembler, inspect the instructions and the relative frequency of various addressing modes. Create a histogram to illustrate the relative frequency of the addressing modes. How do your results compare to Figure A.7?

b. [25] < A.3, A.9 > Compile one or more programs from the SPEC CPU2017 or EEMBC benchmark suites to target the RISC-V architecture. Using a disassembler, inspect the instructions and the relative frequency of various addressing modes. Create a histogram to illustrate the relative frequency of the addressing modes. How do your results compare to Figure A.7?

A.24 [Discussion] < A.2–A.12 > Consider typical applications for desktop, server, cloud, and embedded computing. How would instruction set architecture be impacted for machines targeting each of these markets?

B.1	Introduction	B-2
B.2	Cache Performance	B-15
B.3	Six Basic Cache Optimizations	B-22
B.4	Virtual Memory	B-40
B.5	Protection and Examples of Virtual Memory	B-49
B.6	Fallacies and Pitfalls	B-57
B.7	Concluding Remarks	B-59
B.8	Historical Perspective and References	B-59
	Exercises by Amr Zaky	B-60

B

Review of Memory Hierarchy

Cache: a safe place for hiding or storing things.

Webster's New World Dictionary of
the American Language,
Second College Edition (1976)

B.1 Introduction

This appendix is a quick refresher of the memory hierarchy, including the basics of cache and virtual memory, performance equations, and simple optimizations. This first section reviews the following 36 terms:

cache	*fully associative*	*write allocate*
virtual memory	*dirty bit*	*unified cache*
memory stall cycles	*block offset*	*misses per instruction*
direct mapped	*write back*	*block*
valid bit	*data cache*	*locality*
block address	*hit time*	*address trace*
write through	*cache miss*	*set*
instruction cache	*page fault*	*random replacement*
average memory access time	*miss rate*	*index field*
cache hit	*n-way set associative*	*no-write allocate*
page	*least recently used*	*write buffer*
miss penalty	*tag field*	*write stall*

If this review goes too quickly, you might want to look at Chapter 7 in *Computer Organization and Design*, which we wrote for readers with less experience.

Cache is the name given to the highest or first level of the memory hierarchy encountered once the address leaves the processor. Because the principle of locality applies at many levels, and taking advantage of locality to improve performance is popular, the term *cache* is now applied whenever buffering is employed to reuse commonly occurring items. Examples include *file caches*, *name caches*, and so on.

When the processor finds a requested data item in the cache, it is called a *cache hit*. When the processor does not find a data item it needs in the cache, a *cache miss* occurs. A fixed-size collection of data containing the requested word, called a *block* or line run, is retrieved from the main memory and placed into the cache. *Temporal locality* tells us that we are likely to need this word again in the near future, so it is useful to place it in the cache where it can be accessed quickly. Because of *spatial locality*, there is a high probability that the other data in the block will be needed soon.

The time required for the cache miss depends on both the latency and bandwidth of the memory. Latency determines the time to retrieve the first word of the block, and bandwidth determines the time to retrieve the rest of this block. A cache miss is handled by hardware and causes processors using in-order execution to pause, or stall, until the data are available. With out-of-order execution, an instruction using the result must still wait, but other instructions may proceed during the miss.

Similarly, not all objects referenced by a program need to reside in main memory. *Virtual memory* means some objects may reside on disk. The address space is

Level	1	2	3	4
Name	Registers	Cache	Main memory	Disk storage
Typical size	<4 KiB	32 KiB to 8 MiB	<1 TB	>1 TB
Implementation technology	Custom memory with multiple ports, CMOS	On-chip CMOS SRAM	CMOS DRAM	Magnetic disk or FLASH
Access time (ns)	0.1–0.2	0.5–10	30–150	5,000,000
Bandwidth (MiB/sec)	1,000,000–10,000,000	20,000–50,000	10,000–30,000	100–1000
Managed by	Compiler	Hardware	Operating system	Operating system
Backed by	Cache	Main memory	Disk or FLASH	Other disks and DVD

Figure B.1 The typical levels in the hierarchy slow down and get larger as we move away from the processor for a large workstation or small server. Embedded computers might have no disk storage and much smaller memories and caches. Increasingly, FLASH is replacing magnetic disks, at least for first level file storage. The access times increase as we move to lower levels of the hierarchy, which makes it feasible to manage the transfer less responsively. The implementation technology shows the typical technology used for these functions. The access time is given in nanoseconds for typical values in 2017; these times will decrease over time. Bandwidth is given in megabytes per second between levels in the memory hierarchy. Bandwidth for disk/FLASH storage includes both the media and the buffered interfaces.

usually broken into fixed-size blocks, called *pages*. At any time, each page resides either in main memory or on disk. When the processor references an item within a page that is not present in the cache or main memory, a *palt* occurs, and the entire page is moved from the disk to main memory. Because page faults take so long, they are handled in software and the processor is not stalled. The processor usually switches to some other task while the disk access occurs. From a high-level perspective, the reliance on locality of references and the relative relationships in size and relative cost per bit of cache versus main memory are similar to those of main memory versus disk.

Figure B.1 shows the range of sizes and access times of each level in the memory hierarchy for computers ranging from high-end desktops to low-end servers.

Cache Performance Review

Because of locality and the higher speed of smaller memories, a memory hierarchy can substantially improve performance. One method to evaluate cache performance is to expand our processor execution time equation from Chapter 1. We now account for the number of cycles during which the processor is stalled waiting for a memory access, which we call the *memory stall cycles*. The performance is then the product of the clock cycle time and the sum of the processor cycles and the memory stall cycles:

$$\text{CPU execution time} = (\text{CPU clock cycles} + \text{Memory stall cycles}) \times \text{Clock cycle time}$$

This equation assumes that the CPU clock cycles include the time to handle a cache hit and that the processor is stalled during a cache miss. Section B.2 reexamines this simplifying assumption.

The number of memory stall cycles depends on both the number of misses and the cost per miss, which is called the *miss penalty:*

$$\begin{aligned}\text{Memory stall cycles} &= \text{Number of misses} \times \text{Miss penalty}\\ &= \text{IC} \times \frac{\text{Misses}}{\text{Instruction}} \times \text{Miss penalty}\\ &= \text{IC} \times \frac{\text{Memory accesses}}{\text{Instruction}} \times \text{Miss rate} \times \text{Miss penalty}\end{aligned}$$

The advantage of the last form is that the components can be easily measured. We already know how to measure instruction count (IC). (For speculative processors, we only count instructions that commit.) Measuring the number of memory references per instruction can be done in the same fashion; every instruction requires an instruction access, and it is easy to decide if it also requires a data access.

Note that we calculated miss penalty as an average, but we will use it herein as if it were a constant. The memory behind the cache may be busy at the time of the miss because of prior memory requests or memory refresh. The number of clock cycles also varies at interfaces between different clocks of the processor, bus, and memory. Thus, please remember that using a single number for miss penalty is a simplification.

The component *miss rate* is simply the fraction of cache accesses that result in a miss (i.e., number of accesses that miss divided by number of accesses). Miss rates can be measured with cache simulators that take an *address trace* of the instruction and data references, simulate the cache behavior to determine which references hit and which miss, and then report the hit and miss totals. Many microprocessors today provide hardware to count the number of misses and memory references, which is a much easier and faster way to measure miss rate.

The preceding formula is an approximation because the miss rates and miss penalties are often different for reads and writes. Memory stall clock cycles could then be defined in terms of the number of memory accesses per instruction, miss penalty (in clock cycles) for reads and writes, and miss rate for reads and writes:

$$\begin{aligned}\text{Memory stall clock cycles} = {} & \text{IC} \times \text{Reads per instruction} \times \text{Read miss rate} \times \text{Read miss penalty}\\ & + \text{IC} \times \text{Writes per instruction} \times \text{Write miss rate} \times \text{Write miss penalty}\end{aligned}$$

We usually simplify the complete formula by combining the reads and writes and finding the average miss rates and miss penalty for reads *and* writes:

$$\text{Memory stall clock cycles} = \text{IC} \times \frac{\text{Memory accesses}}{\text{Instruction}} \times \text{Miss rate} \times \text{Miss penalty}$$

The miss rate is one of the most important measures of cache design, but, as we will see in later sections, not the only measure.

Example Assume we have a computer where the cycles per instruction (CPI) is 1.0 when all memory accesses hit in the cache. The only data accesses are loads and stores, and these total 50% of the instructions. If the miss penalty is 50 clock cycles and the miss rate is 1%, how much faster would the computer be if all instructions were cache hits?

Answer First compute the performance for the computer that always hits:

$$\begin{aligned}\text{CPU execution time} &= (\text{CPU clock cycles} + \text{Memory stall cycles}) \times \text{Clock cycle}\\ &= (\text{IC} \times \text{CPI} + 0) \times \text{Clock cycle}\\ &= \text{IC} \times 1.0 \times \text{Clock cycle}\end{aligned}$$

Now for the computer with the real cache, first we compute memory stall cycles:

$$\begin{aligned}\text{Memory stall cycles} &= \text{IC} \times \frac{\text{Memory accesses}}{\text{Instruction}} \times \text{Miss rate} \times \text{Miss penalty}\\ &= \text{IC} \times (1 + 0.5) \times 0.01 \times 50\\ &= \text{IC} \times 0.75\end{aligned}$$

where the middle term $(1 + 0.5)$ represents one instruction access and 0.5 data accesses per instruction. The total performance is thus

$$\begin{aligned}\text{CPU execution time}_{\text{cache}} &= (\text{IC} \times 1.0 + \text{IC} \times 0.75) \times \text{Clock cycle}\\ &= 1.75 \times \text{IC} \times \text{Clock cycle}\end{aligned}$$

The performance ratio is the inverse of the execution times:

$$\begin{aligned}\frac{\text{CPU execution time}_{\text{cache}}}{\text{CPU execution time}} &= \frac{1.75 \times \text{IC} \times \text{Clock cycle}}{1.0 \times \text{IC} \times \text{Clock cycle}}\\ &= 1.75\end{aligned}$$

The computer with no cache misses is 1.75 times faster.

Some designers prefer measuring miss rate as *misses per instruction* rather than misses per memory reference. These two are related:

$$\frac{\text{Misses}}{\text{Instruction}} = \frac{\text{Miss rate} \times \text{Memory accesses}}{\text{Instruction count}} = \text{Miss rate} \times \frac{\text{Memory accesses}}{\text{Instruction}}$$

The latter formula is useful when you know the average number of memory accesses per instruction because it allows you to convert miss rate into misses per instruction, and vice versa. For example, we can turn the miss rate per memory reference in the previous example into misses per instruction:

$$\frac{\text{Misses}}{\text{Instruction}} = \text{Miss rate} \times \frac{\text{Memory accesses}}{\text{Instruction}} = 0.02 \times (1.5) = 0.030$$

By the way, misses per instruction are often reported as misses per 1000 instructions to show integers instead of fractions. Thus, the preceding answer could also be expressed as 30 misses per 1000 instructions.

The advantage of misses per instruction is that it is independent of the hardware implementation. For example, speculative processors fetch about twice as many instructions as are actually committed, which can artificially reduce the miss rate if measured as misses per memory reference rather than per instruction. The drawback is that misses per instruction is architecture dependent; for example, the average number of memory accesses per instruction may be very different for an 80x86 versus RISC V. Thus, misses per instruction are most popular with architects working with a single computer family, although the similarity of RISC architectures allows one to give insights into others.

Example To show equivalency between the two miss rate equations, let's redo the preceding example, this time assuming a miss rate per 1000 instructions of 30. What is memory stall time in terms of instruction count?

Answer Recomputing the memory stall cycles:

$$\begin{aligned}\text{Memory stall cycles} &= \text{Number of misses} \times \text{Miss penalty}\\ &= \text{IC} \times \frac{\text{Misses}}{\text{Instruction}} \times \text{Miss penalty}\\ &= \text{IC}/1000 \times \frac{\text{Misses}}{\text{Intruction} \times 1000} \times \text{Miss penalty}\\ &= \text{IC}/1000 \times 30 \times 25\\ &= \text{IC}/1000 \times 750\\ &= \text{IC} \times 0.75\end{aligned}$$

We get the same answer as on page B-5, showing equivalence of the two equations.

Four Memory Hierarchy Questions

We continue our introduction to caches by answering the four common questions for the first level of the memory hierarchy:

Q1: Where can a block be placed in the upper level? (*block placement*)

Q2: How is a block found if it is in the upper level? (*block identification*)

Q3: Which block should be replaced on a miss? (*block replacement*)

Q4: What happens on a write? (*write strategy*)

The answers to these questions help us understand the different trade-offs of memories at different levels of a hierarchy; hence, we ask these four questions on every example.

Q1: Where Can a Block be Placed in a Cache?

Figure B.2 shows that the restrictions on where a block is placed create three categories of cache organization:

- If each block has only one place it can appear in the cache, the cache is said to be *direct mapped*. The mapping is usually

  (*Block address*) MOD (*Number of blocks in cache*)

- If a block can be placed anywhere in the cache, the cache is said to be *fully associative*.

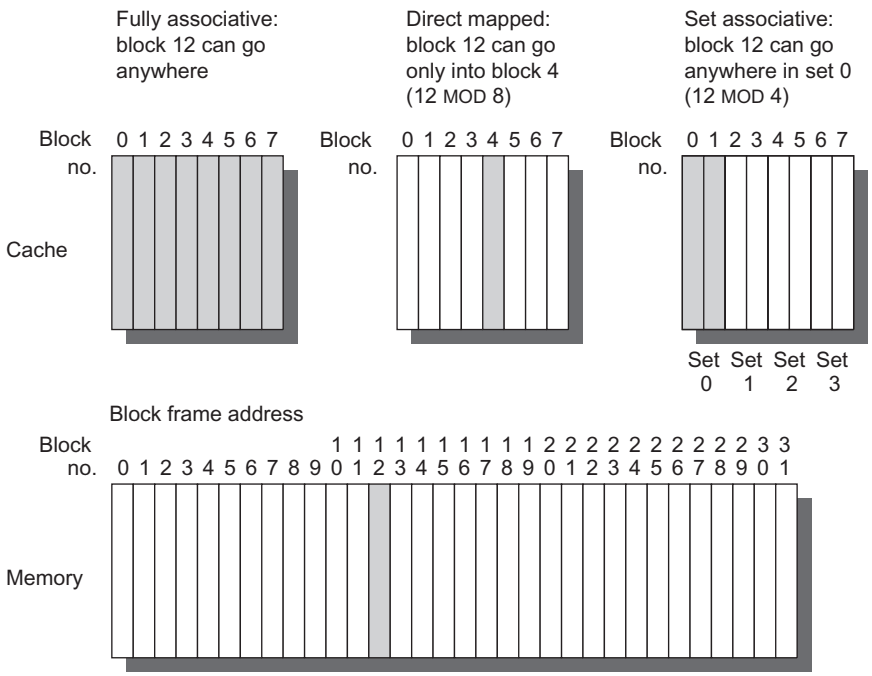


Figure B.2 This example cache has eight block frames and memory has 32 blocks. The three options for caches are shown left to right. In fully associative, block 12 from the lower level can go into any of the eight block frames of the cache. With direct mapped, block 12 can only be placed into block frame 4 (12 modulo 8). Set associative, which has some of both features, allows the block to be placed anywhere in set 0 (12 modulo 4). With two blocks per set, this means block 12 can be placed either in block 0 or in block 1 of the cache. Real caches contain thousands of block frames, and real memories contain millions of blocks. The set associative organization has four sets with two blocks per set, called *two-way set associative*. Assume that there is nothing in the cache and that the block address in question identifies lower-level block 12.

- If a block can be placed in a restricted set of places in the cache, the cache is *set associative*. A *set* is a group of blocks in the cache. A block is first mapped onto a set, and then the block can be placed anywhere within that set. The set is usually chosen by *bit selection*; that is,

  (*Block address*) MOD (*Number of sets in cache*)

  If there are n blocks in a set, the cache placement is called *n-way set associative*.

The range of caches from direct mapped to fully associative is really a continuum of levels of set associativity. Direct mapped is simply one-way set associative, and a fully associative cache with m blocks could be called "m-way set associative." Equivalently, direct mapped can be thought of as having m sets, and fully associative as having one set.

The vast majority of processor caches today are direct mapped, two-way set associative, or four-way set associative, for reasons we will see shortly.

Q2: How Is a Block Found If It Is in the Cache?

Caches have an address tag on each block frame that gives the block address. The tag of every cache block that might contain the desired information is checked to see if it matches the block address from the processor. As a rule, all possible tags are searched in parallel because speed is critical.

There must be a way to know that a cache block does not have valid information. The most common procedure is to add a *valid bit* to the tag to say whether or not this entry contains a valid address. If the bit is not set, there cannot be a match on this address.

Before proceeding to the next question, let's explore the relationship of a processor address to the cache. Figure B.3 shows how an address is divided. The first division is between the *block address* and the *block offset.* The block frame address can be further divided into the *tag field* and the *index field.* The block offset field selects the desired data from the block, the index field selects the set, and the tag field is compared against it for a hit. Although the comparison could be made on more of the address than the tag, there is no need because of the following:

- The offset should not be used in the comparison, because the entire block is present or not, and hence all block offsets result in a match by definition.

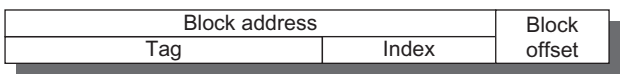


Figure B.3 The three portions of an address in a set associative or direct-mapped cache. The tag is used to check all the blocks in the set, and the index is used to select the set. The block offset is the address of the desired data within the block. Fully associative caches have no index field.

- Checking the index is redundant, because it was used to select the set to be checked. An address stored in set 0, for example, must have 0 in the index field or it couldn't be stored in set 0; set 1 must have an index value of 1; and so on. This optimization saves hardware and power by reducing the width of memory size for the cache tag.

If the total cache size is kept the same, increasing associativity increases the number of blocks per set, thereby decreasing the size of the index and increasing the size of the tag. That is, the tag-index boundary in Figure B.3 moves to the right with increasing associativity, with the end point of fully associative caches having no index field.

Q3: Which Block Should be Replaced on a Cache Miss?

When a miss occurs, the cache controller must select a block to be replaced with the desired data. A benefit of direct-mapped placement is that hardware decisions are simplified—in fact, so simple that there is no choice: only one block frame is checked for a hit, and only that block can be replaced. With fully associative or set associative placement, there are many blocks to choose from on a miss. There are three primary strategies employed for selecting which block to replace:

- *Random*—To spread allocation uniformly, candidate blocks are randomly selected. Some systems generate pseudorandom block numbers to get reproducible behavior, which is particularly useful when debugging hardware.
- *Least recently used* (LRU)—To reduce the chance of throwing out information that will be needed soon, accesses to blocks are recorded. Relying on the past to predict the future, the block replaced is the one that has been unused for the longest time. LRU relies on a corollary of locality: if recently used blocks are likely to be used again, then a good candidate for disposal is the least recently used block.
- *First in, first out* (FIFO)—Because LRU can be complicated to calculate, this approximates LRU by determining the *oldest* block rather than the LRU.

A virtue of random replacement is that it is simple to build in hardware. As the number of blocks to keep track of increases, LRU becomes increasingly expensive and is usually only approximated. A common approximation (often called pseudo-LRU) has a set of bits for each set in the cache with each bit corresponding to a single way (a *way* is bank in a set associative cache; there are four ways in four-way set associative cache) in the cache. When a set is accessed, the bit corresponding to the way containing the desired block is turned on; if all the bits associated with a set are turned on, they are reset with the exception of the most recently turned on bit. When a block must be replaced, the processor chooses a block from the way whose bit is turned off, often randomly if more than one choice is available. This approximates LRU, because the block that is replaced will not have

	Associativity								
	Two-way			Four-way			Eight-way		
Size	LRU	Random	FIFO	LRU	Random	FIFO	LRU	Random	FIFO
16 KiB	114.1	117.3	115.5	111.7	115.1	113.3	109.0	111.8	110.4
64 KiB	103.4	104.3	103.9	102.4	102.3	103.1	99.7	100.5	100.3
256 KiB	92.2	92.1	92.5	92.1	92.1	92.5	92.1	92.1	92.5

Figure B.4 Data cache misses per 1000 instructions comparing least recently used, random, and first in, first out replacement for several sizes and associativities. There is little difference between LRU and random for the largest size cache, with LRU outperforming the others for smaller caches. FIFO generally outperforms random in the smaller cache sizes. These data were collected for a block size of 64 bytes for the Alpha architecture using 10 SPEC2000 benchmarks. Five are from SPECint2000 (gap, gcc, gzip, mcf, and perl) and five are from SPECfp2000 (applu, art, equake, lucas, and swim). We will use this computer and these benchmarks in most figures in this appendix.

been accessed since the last time that all the blocks in the set were accessed. Figure B.4 shows the difference in miss rates between LRU, random, and FIFO replacement.

Q4: What Happens on a Write?

Reads dominate processor cache accesses. All instruction accesses are reads, and most instructions don't write to memory. Figures A.32 and A.33 in Appendix A suggest a mix of 10% stores and 26% loads for RISC V programs, making writes 10%/(100% + 26% + 10%) or about 7% of the overall memory traffic. Of the *data cache* traffic, writes are 10%/(26% + 10%) or about 28%. Making the common case fast means optimizing caches for reads, especially because processors traditionally wait for reads to complete but need not wait for writes. Amdahl's Law (Section 1.9) reminds us, however, that high-performance designs cannot neglect the speed of writes.

Fortunately, the common case is also the easy case to make fast. The block can be read from the cache at the same time that the tag is read and compared, so the block read begins as soon as the block address is available. If the read is a hit, the requested part of the block is passed on to the processor immediately. If it is a miss, there is no benefit—but also no harm except more power in desktop and server computers; just ignore the value read.

Such optimism is not allowed for writes. Modifying a block cannot begin until the tag is checked to see if the address is a hit. Because tag checking cannot occur in parallel, writes usually take longer than reads. Another complexity is that the processor also specifies the size of the write, usually between 1 and 8 bytes; only that portion of a block can be changed. In contrast, reads can access more bytes than necessary without fear.

The write policies often distinguish cache designs. There are two basic options when writing to the cache:

- *Write through*—The information is written to both the block in the cache *and* to the block in the lower-level memory.
- *Write back*—The information is written only to the block in the cache. The modified cache block is written to main memory only when it is replaced.

To reduce the frequency of writing back blocks on replacement, a feature called the *dirty bit* is commonly used. This status bit indicates whether the block is *dirty* (modified while in the cache) or *clean* (not modified). If it is clean, the block is not written back on a miss, because identical information to the cache is found in lower levels.

Both write back and write through have their advantages. With write back, writes occur at the speed of the cache memory, and multiple writes within a block require only one write to the lower-level memory. Because some writes don't go to memory, write back uses less memory bandwidth, making write back attractive in multiprocessors. Since write back uses the rest of the memory hierarchy and memory interconnect less than write through, it also saves power, making it attractive for embedded applications.

Write through is easier to implement than write back. The cache is always clean, so unlike write back read misses never result in writes to the lower level. Write through also has the advantage that the next lower level has the most current copy of the data, which simplifies data coherency. Data coherency is important for multiprocessors and for I/O, which we examine in Chapter 4 and Appendix D. Multilevel caches make write through more viable for the upper-level caches, as the writes need only propagate to the next lower level rather than all the way to main memory.

As we will see, I/O and multiprocessors are fickle: they want write back for processor caches to reduce the memory traffic and write through to keep the cache consistent with lower levels of the memory hierarchy.

When the processor must wait for writes to complete during write through, the processor is said to *write stall.* A common optimization to reduce write stalls is a *write buffer,* which allows the processor to continue as soon as the data are written to the buffer, thereby overlapping processor execution with memory updating. As we will see shortly, write stalls can occur even with write buffers.

Because the data are not needed on a write, there are two options on a write miss:

- *Write allocate*—The block is allocated on a write miss, followed by the preceding write hit actions. In this natural option, write misses act like read misses.
- *No-write allocate*—This apparently unusual alternative is write misses do *not* affect the cache. Instead, the block is modified only in the lower-level memory.

Thus, blocks stay out of the cache in no-write allocate until the program tries to read the blocks, but even blocks that are only written will still be in the cache with write allocate. Let's look at an example.

Example Assume a fully associative write-back cache with many cache entries that starts empty. Following is a sequence of five memory operations (the address is in square brackets):

```
Write Mem[100];
Write Mem[100];
Read  Mem[200];
Write Mem[200];
Write Mem[100].
```

What are the number of hits and misses when using no-write allocate versus write allocate?

Answer For no-write allocate, the address 100 is not in the cache, and there is no allocation on write, so the first two writes will result in misses. Address 200 is also not in the cache, so the read is also a miss. The subsequent write to address 200 is a hit. The last write to 100 is still a miss. The result for no-write allocate is four misses and one hit.

For write allocate, the first accesses to 100 and 200 are misses, and the rest are hits because 100 and 200 are both found in the cache. Thus, the result for write allocate is two misses and three hits.

Either write miss policy could be used with write through or write back. Usually, write-back caches use write allocate, hoping that subsequent writes to that block will be captured by the cache. Write-through caches often use no-write allocate. The reasoning is that even if there are subsequent writes to that block, the writes must still go to the lower-level memory, so what's to be gained?

An Example: The Opteron Data Cache

To give substance to these ideas, Figure B.5 shows the organization of the data cache in the AMD Opteron microprocessor. The cache contains 65,536 (64 K) bytes of data in 64-byte blocks with two-way set associative placement, least-recently used replacement, write back, and write allocate on a write miss.

Let's trace a cache hit through the steps of a hit as labeled in Figure B.5. (The four steps are shown as circled numbers.) As described in Section B.5, the Opteron presents a 48-bit virtual address to the cache for tag comparison, which is simultaneously translated into a 40-bit physical address.

The reason Opteron doesn't use all 64 bits of virtual address is that its designers don't think anyone needs that much virtual address space yet, and the smaller size simplifies the Opteron virtual address mapping. The designers plan to grow the virtual address in future microprocessors.

The physical address coming into the cache is divided into two fields: the 34-bit block address and the 6-bit block offset ($64 = 2^6$ and $34 + 6 = 40$). The block

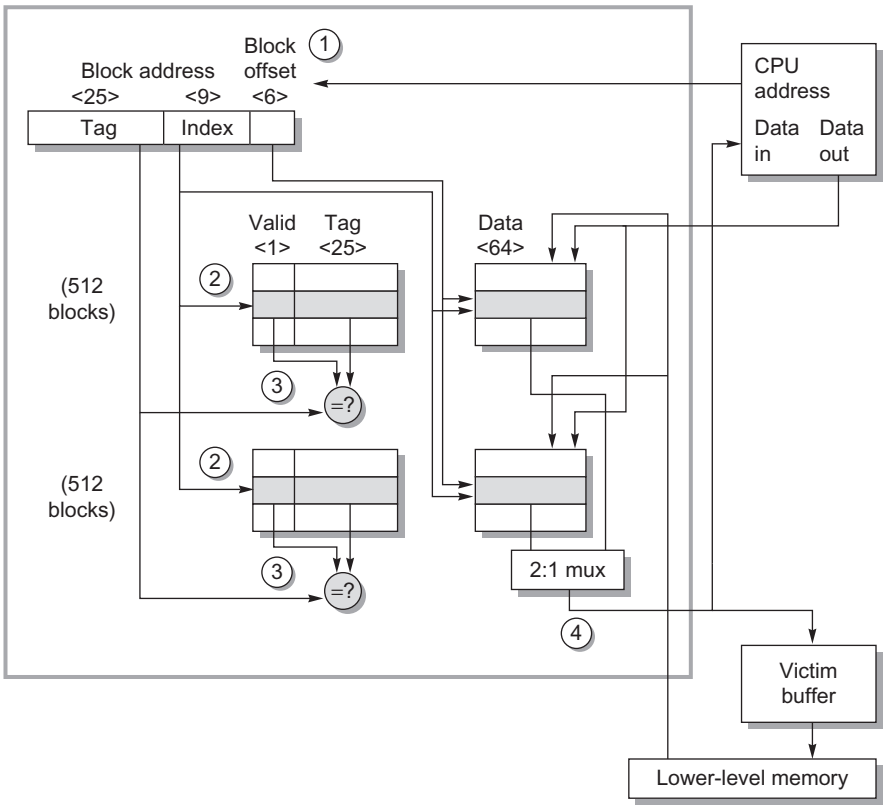


Figure B.5 The organization of the data cache in the Opteron microprocessor. The 64 KiB cache is two-way set associative with 64-byte blocks. The 9-bit index selects among 512 sets. The four steps of a read hit, shown as circled numbers in order of occurrence, label this organization. Three bits of the block offset join the index to supply the RAM address to select the proper 8 bytes. Thus, the cache holds two groups of 4096 64-bit words, with each group containing half of the 512 sets. Although not exercised in this example, the line from lower-level memory to the cache is used on a miss to load the cache. The size of address leaving the processor is 40 bits because it is a physical address and not a virtual address. Figure B.24 on page B-47 explains how the Opteron maps from virtual to physical for a cache access.

address is further divided into an address tag and cache index. Step 1 shows this division.

The cache index selects the tag to be tested to see if the desired block is in the cache. The size of the index depends on cache size, block size, and set associativity. For the Opteron cache the set associativity is set to two, and we calculate the index as follows:

$$2^{\text{Index}} = \frac{\text{Cache size}}{\text{Block size} \times \text{Set associativity}} = \frac{65,536}{64 \times 2} = 512 = 2^9$$

Hence, the index is 9 bits wide, and the tag is 34 – 9 or 25 bits wide. Although that is the index needed to select the proper block, 64 bytes is much more than the processor wants to consume at once. Hence, it makes more sense to organize the data portion of the cache memory 8 bytes wide, which is the natural data word of the 64-bit Opteron processor. Thus, in addition to 9 bits to index the proper cache block, 3 more bits from the block offset are used to index the proper 8 bytes. Index selection is step 2 in Figure B.5.

After reading the two tags from the cache, they are compared with the tag portion of the block address from the processor. This comparison is step 3 in the figure. To be sure the tag contains valid information, the valid bit must be set or else the results of the comparison are ignored.

Assuming one tag does match, the final step is to signal the processor to load the proper data from the cache by using the winning input from a 2:1 multiplexor. The Opteron allows 2 clock cycles for these four steps, so the instructions in the following 2 clock cycles would wait if they tried to use the result of the load.

Handling writes is more complicated than handling reads in the Opteron, as it is in any cache. If the word to be written is in the cache, the first three steps are the same. Because the Opteron executes out of order, only after it signals that the instruction has committed and the cache tag comparison indicates a hit are the data written to the cache.

So far we have assumed the common case of a cache hit. What happens on a miss? On a read miss, the cache sends a signal to the processor telling it the data are not yet available, and 64 bytes are read from the next level of the hierarchy. The latency is 7 clock cycles to the first 8 bytes of the block, and then 2 clock cycles per 8 bytes for the rest of the block. Because the data cache is set associative, there is a choice on which block to replace. Opteron uses LRU, which selects the block that was referenced longest ago, so every access must update the LRU bit. Replacing a block means updating the data, the address tag, the valid bit, and the LRU bit.

Because the Opteron uses write back, the old data block could have been modified, and hence it cannot simply be discarded. The Opteron keeps 1 dirty bit per block to record if the block was written. If the "victim" was modified, its data and address are sent to the victim buffer. (This structure is similar to a *write buffer* in other computers.) The Opteron has space for eight victim blocks. In parallel with other cache actions, it writes victim blocks to the next level of the hierarchy. If the victim buffer is full, the cache must wait.

A write miss is very similar to a read miss, because the Opteron allocates a block on a read or a write miss.

We have seen how it works, but the *data* cache cannot supply all the memory needs of the processor: the processor also needs instructions. Although a single cache could try to supply both, it can be a bottleneck. For example, when a load or store instruction is executed, the pipelined processor will simultaneously request both a data word *and* an instruction word. Hence, a single cache would present a structural hazard for loads and stores, leading to stalls. One simple way to conquer

Size (KiB)	Instruction cache	Data cache	Unified cache
8	8.16	44.0	63.0
16	3.82	40.9	51.0
32	1.36	38.4	43.3
64	0.61	36.9	39.4
128	0.30	35.3	36.2
256	0.02	32.6	32.9

Figure B.6 Miss per 1000 instructions for instruction, data, and unified caches of different sizes. The percentage of instruction references is about 74%. The data are for two-way associative caches with 64-byte blocks for the same computer and benchmarks as Figure B.4.

this problem is to divide it: one cache is dedicated to instructions and another to data. Separate caches are found in most recent processors, including the Opteron. Hence, it has a 64 KiB instruction cache as well as the 64 KiB data cache.

The processor knows whether it is issuing an instruction address or a data address, so there can be separate ports for both, thereby doubling the bandwidth between the memory hierarchy and the processor. Separate caches also offer the opportunity of optimizing each cache separately: different capacities, block sizes, and associativities may lead to better performance. (In contrast to the instruction caches and data caches of the Opteron, the terms *unified* or *mixed* are applied to caches that can contain either instructions or data.)

Figure B.6 shows that instruction caches have lower miss rates than data caches. Separating instructions and data removes misses due to conflicts between instruction blocks and data blocks, but the split also fixes the cache space devoted to each type. Which is more important to miss rates? A fair comparison of separate instruction and data caches to unified caches requires the total cache size to be the same. For example, a separate 16 KiB instruction cache and 16 KiB data cache should be compared with a 32 KiB unified cache. Calculating the average miss rate with separate instruction and data caches necessitates knowing the percentage of memory references to each cache. From the data in Appendix A we find the split is 100%/(100% + 26% + 10%) or about 74% instruction references to (26% + 10%)/(100% + 26% + 10%) or about 26% data references. Splitting affects performance beyond what is indicated by the change in miss rates, as we will see shortly.

B.2 Cache Performance

Because instruction count is independent of the hardware, it is tempting to evaluate processor performance using that number. Such indirect performance measures have waylaid many a computer designer. The corresponding temptation for evaluating memory hierarchy performance is to concentrate on miss rate because it,

too, is independent of the speed of the hardware. As we will see, miss rate can be just as misleading as instruction count. A better measure of memory hierarchy performance is the *average memory access time*:

$$\text{Average memory access time} = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$$

where *hit time* is the time to hit in the cache; we have seen the other two terms before. The components of average access time can be measured either in absolute time—say, 0.25–1.0 ns on a hit—or in the number of clock cycles that the processor waits for the memory—such as a miss penalty of 150–200 clock cycles. Remember that average memory access time is still an indirect measure of performance; although it is a better measure than miss rate, it is not a substitute for execution time.

This formula can help us decide between split caches and a unified cache.

Example Which has the lower miss rate: a 16 KiB instruction cache with a 16 KiB data cache or a 32 KiB unified cache? Use the miss rates in Figure B.6 to help calculate the correct answer, assuming 36% of the instructions are data transfer instructions. Assume a hit takes 1 clock cycle and the miss penalty is 100 clock cycles. A load or store hit takes 1 extra clock cycle on a unified cache if there is only one cache port to satisfy two simultaneous requests. Using the pipelining terminology of Chapter 3, the unified cache leads to a structural hazard. What is the average memory access time in each case? Assume write-through caches with a write buffer and ignore stalls due to the write buffer.

Answer First let's convert misses per 1000 instructions into miss rates. Solving the preceding general formula, the miss rate is

$$\text{Miss rate} = \frac{\dfrac{\text{Misses}}{\dfrac{1000\ \text{Instructions}}{}}/1000}{\dfrac{\text{Memory accesses}}{\text{Instruction}}}$$

Because every instruction access has exactly one memory access to fetch the instruction, the instruction miss rate is

$$\text{Miss rate}_{16\,\text{KB instruction}} = \frac{3.82/1000}{1.00} = 0.004$$

Because 36% of the instructions are data transfers, the data miss rate is

$$\text{Miss rate}_{16\,\text{KB data}} = \frac{40.9/1000}{0.36} = 0.114$$

The unified miss rate needs to account for instruction and data accesses:

$$\text{Miss rate}_{32\,\text{KB unified}} = \frac{43.3/1000}{1.00 + 0.36} = 0.0318$$

As stated herein, about 74% of the memory accesses are instruction references. Thus, the overall miss rate for the split caches is

$$(74\% \times 0.004) + (26\% \times 0.114) = 0.0326$$

Thus, a 32 KiB unified cache has a slightly lower effective miss rate than two 16 KiB caches.

The average memory access time formula can be divided into instruction and data accesses:

$$\begin{aligned}&\text{Average memory access time}\\&= \%\,\text{instructions} \times (\text{Hit time} + \text{Instruction miss rate} \times \text{Miss penalty})\\&\quad + \%\,\text{data} \times (\text{Hit time} + \text{Data miss rate} \times \text{Miss penalty})\end{aligned}$$

Therefore, the time for each organization is

$$\begin{aligned}&\text{Average memory access time}_{\text{split}}\\&= 74\% \times (1 + 0.004 \times 200) + 26\% \times (1 + 0.114 \times 200)\\&= (74\% \times 1.80) + (26\% \times 23.80) = 1.332 + 6.188 = 7.52\\&\text{Average memory access time}_{\text{unified}}\\&= 74\% \times (1 + 0.0318 \times 200) + 26\% \times (1 + 1 + 0.0318 \times 200)\\&= (74\% \times 7.36) + (26\% \times 8.36) = 5.446 + 2.174 = 7.62\end{aligned}$$

Hence, the split caches in this example—which offer two memory ports per clock cycle, thereby avoiding the structural hazard—have a better average memory access time than the single-ported unified cache despite having a worse effective miss rate.

Average Memory Access Time and Processor Performance

An obvious question is whether average memory access time due to cache misses predicts processor performance.

First, there are other reasons for stalls, such as contention due to I/O devices using memory. Designers often assume that all memory stalls are due to cache misses, because the memory hierarchy typically dominates other reasons for stalls. We use this simplifying assumption here, but be sure to account for *all* memory stalls when calculating final performance.

Second, the answer also depends on the processor. If we have an in-order execution processor (see Chapter 3), then the answer is basically yes. The processor stalls during misses, and the memory stall time is strongly correlated to average memory access time. Let's make that assumption for now, but we'll return to out-of-order processors in the next subsection.

As stated in the previous section, we can model CPU time as:

$$\text{CPU time} = (\text{CPU execution clock cycles} + \text{Memory stall clock cycles}) \times \text{Clock cycle time}$$

This formula raises the question of whether the clock cycles for a cache hit should be considered part of CPU execution clock cycles or part of memory stall clock cycles. Although either convention is defensible, the most widely accepted is to include hit clock cycles in CPU execution clock cycles.

We can now explore the impact of caches on performance.

Example Let's use an in-order execution computer for the first example. Assume that the cache miss penalty is 200 clock cycles, and all instructions usually take 1.0 clock cycles (ignoring memory stalls). Assume that the average miss rate is 2%, there is an average of 1.5 memory references per instruction, and the average number of cache misses per 1000 instructions is 30. What is the impact on performance when behavior of the cache is included? Calculate the impact using both misses per instruction and miss rate.

Answer

$$\text{CPU time} = \text{IC} \times \left(\text{CPI}_{\text{execution}} + \frac{\text{Memory stall clock cycles}}{\text{Instruction}} \right) \times \text{Clock cycle time}$$

The performance, including cache misses, is

$$\begin{aligned} \text{CPU time}_{\text{with cache}} &= \text{IC} \times [1.0 + (30/1000 \times 200)] \times \text{Clock cycle time} \\ &= \text{IC} \times 7.00 \times \text{Clock cycle time} \end{aligned}$$

Now calculating performance using miss rate:

$$\text{CPU time} = \text{IC} \times \left(\text{CPI}_{\text{execution}} + \text{Miss rate} \times \frac{\text{Memory accesses}}{\text{Instruction}} \times \text{Miss penalty} \right) \times \text{Clock cycle time}$$

$$\begin{aligned} \text{CPU time}_{\text{with cache}} &= \text{IC} \times [1.0 + (1.5 \times 2\% \times 200)] \times \text{Clock cycle time} \\ &= \text{IC} \times 7.00 \times \text{Clock cycle time} \end{aligned}$$

The clock cycle time and instruction count are the same, with or without a cache. Thus, CPU time increases sevenfold, with CPI from 1.00 for a "perfect cache" to 7.00 with a cache that can miss. Without any memory hierarchy at all the CPI would increase again to $1.0 + 200 \times 1.5$ or 301—a factor of more than 40 times longer than a system with a cache!

As this example illustrates, cache behavior can have enormous impact on performance. Furthermore, cache misses have a double-barreled impact on a processor with a low CPI and a fast clock:

1. The lower the $\text{CPI}_{\text{execution}}$, the higher the *relative* impact of a fixed number of cache miss clock cycles.
2. When calculating CPI, the cache miss penalty is measured in processor clock cycles for a miss. Therefore, even if memory hierarchies for two computers are

identical, the processor with the higher clock rate has a larger number of clock cycles per miss and hence a higher memory portion of CPI.

The importance of the cache for processors with low CPI and high clock rates is thus greater, and, consequently, greater is the danger of neglecting cache behavior in assessing performance of such computers. Amdahl's Law strikes again!

Although minimizing average memory access time is a reasonable goal—and we will use it in much of this appendix—keep in mind that the final goal is to reduce processor execution time. The next example shows how these two can differ.

Example What is the impact of two different cache organizations on the performance of a processor? Assume that the CPI with a perfect cache is 1.0, the clock cycle time is 0.35 ns, there are 1.4 memory references per instruction, the size of both caches is 128 KiB, and both have a block size of 64 bytes. One cache is direct mapped and the other is two-way set associative. Figure B.5 shows that for set associative caches we must add a multiplexor to select between the blocks in the set depending on the tag match. Because the speed of the processor can be tied directly to the speed of a cache hit, assume the processor clock cycle time must be stretched 1.35 times to accommodate the selection multiplexor of the set associative cache. To the first approximation, the cache miss penalty is 65 ns for either cache organization. (In practice, it is normally rounded up or down to an integer number of clock cycles.) First, calculate the average memory access time and then processor performance. Assume the hit time is 1 clock cycle, the miss rate of a direct-mapped 128 KiB cache is 2.1%, and the miss rate for a two-way set associative cache of the same size is 1.9%.

Answer Average memory access time is

$$\text{Average memory access time} = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$$

Thus, the time for each organization is

$$\text{Average memory access time}_{\text{1-way}} = 0.35 + (.021 \times 65) = 1.72\,\text{ns}$$

$$\text{Average memory access time}_{\text{2-way}} = 0.35 \times 1.35 + (.019 \times 65) = 1.71\,\text{ns}$$

The average memory access time is better for the two-way set-associative cache.

The processor performance is

$$\begin{aligned}\text{CPU time} &= \text{IC} \times \left(\text{CPI}_{\text{execution}} + \frac{\text{Misses}}{\text{Instruction}} \times \text{Miss penalty}\right) \times \text{Clock cycle time} \\ &= \text{IC} \times \left[(\text{CPI}_{\text{execution}} \times \text{Clock cycle time}) \right. \\ &\quad \left. + \left(\text{Miss rate} \times \frac{\text{Memory accesses}}{\text{Instruction}} \times \text{Miss penalty} \times \text{Clock cycle time}\right)\right]\end{aligned}$$

Substituting 65 ns for (Miss penalty × Clock cycle time), the performance of each cache organization is

$$\text{CPU time}_{\text{1-way}} = \text{IC} \times [1.0 \times 0.35 + (0.021 \times 1.4 \times 65)] = 2.26 \times \text{IC}$$

$$\text{CPU time}_{\text{2-way}} = \text{IC} \times [1.0 \times 0.35 \times 1.35 + (0.019 \times 1.4 \times 65)] = 2.20 \times \text{IC}$$

and relative performance is

$$\frac{\text{CPU time}_{\text{2-way}}}{\text{CPU time}_{\text{1-way}}} = \frac{2.26 \times \text{Instruction count}}{2.20 \times \text{Instruction count}} = 1.03$$

In contrast to the results of average memory access time comparison, the direct-mapped cache leads to slightly better average performance because the clock cycle is stretched for *all* instructions for the two-way set associative case, even if there are fewer misses. Because CPU time is our bottom-line evaluation and because direct mapped is simpler to build, the preferred cache is direct mapped in this example.

Miss Penalty and Out-of-Order Execution Processors

For an out-of-order execution processor, how do you define "miss penalty"? Is it the full latency of the miss to memory, or is it just the "exposed" or nonoverlapped latency when the processor must stall? This question does not arise in processors that stall until the data miss completes.

Let's redefine memory stalls to lead to a new definition of miss penalty as nonoverlapped latency:

$$\frac{\text{Memory stall cycles}}{\text{Instruction}} = \frac{\text{Misses}}{\text{Instruction}} \times (\text{Total miss latency} - \text{Overlapped miss latency})$$

Similarly, as some out-of-order processors stretch the hit time, that portion of the performance equation could be divided by total hit latency less overlapped hit latency. This equation could be further expanded to account for contention for memory resources in an out-of-order processor by dividing total miss latency into latency without contention and latency due to contention. Let's just concentrate on miss latency.

We now have to decide the following:

- *Length of memory latency*—What to consider as the start and the end of a memory operation in an out-of-order processor.
- *Length of latency overlap*—What is the start of overlap with the processor (or, equivalently, when do we say a memory operation is stalling the processor)?

Given the complexity of out-of-order execution processors, there is no single correct definition.

Because only committed operations are seen at the retirement pipeline stage, we say a processor is stalled in a clock cycle if it does not retire the maximum possible number of instructions in that cycle. We attribute that stall to the first instruction that could not be retired. This definition is by no means foolproof. For example, applying an optimization to improve a certain stall time may not always improve execution time because another type of stall—hidden behind the targeted stall—may now be exposed.

For latency, we could start measuring from the time the memory instruction is queued in the instruction window, or when the address is generated, or when the instruction is actually sent to the memory system. Any option works as long as it is used in a consistent fashion.

Example Let's redo the preceding example, but this time we assume the processor with the longer clock cycle time supports out-of-order execution yet still has a direct-mapped cache. Assume 30% of the 65 ns miss penalty can be overlapped; that is, the average CPU memory stall time is now 45.5 ns.

Answer Average memory access time for the out-of-order (OOO) computer is

$$\text{Average memory access time}_{\text{1-way,OOO}} = 0.35 \times 1.35 + (0.021 \times 45.5) = 1.43\,\text{ns}$$

The performance of the OOO cache is

$$\text{CUP time}_{\text{1-way,OOO}} = \text{IC} \times [1.6 \times 0.35 \times 1.35 + (0.021 \times 1.4 \times 45.5)] = 2.09 \times \text{IC}$$

Hence, despite a much slower clock cycle time and the higher miss rate of a direct-mapped cache, the out-of-order computer can be slightly faster if it can hide 30% of the miss penalty.

In summary, although the state of the art in defining and measuring memory stalls for out-of-order processors is complex, be aware of the issues because they significantly affect performance. The complexity arises because out-of-order processors tolerate some latency due to cache misses without hurting performance. Consequently, designers usually use simulators of the out-of-order processor and memory when evaluating trade-offs in the memory hierarchy to be sure that an improvement that helps the average memory latency actually helps program performance.

To help summarize this section and to act as a handy reference, Figure B.7 lists the cache equations in this appendix.

$$2^{\text{index}} = \frac{\text{Cache size}}{\text{Block size} \times \text{Set associativity}}$$

$$\text{CPU execution time} = (\text{CPU clock cycles} + \text{Memory stall cycles}) \times \text{Clock cycle time}$$

$$\text{Memory stall cycles} = \text{Number of misses} \times \text{Miss penalty}$$

$$\text{Memory stall cycles} = \text{IC} \times \frac{\text{Misses}}{\text{Instruction}} \times \text{Miss penalty}$$

$$\frac{\text{Misses}}{\text{Instruction}} = \text{Miss rate} \times \frac{\text{Memory accesses}}{\text{Instruction}}$$

$$\text{Average memory access time} = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$$

$$\text{CPU execution time} = \text{IC} \times \left(\text{CPI}_{\text{execution}} + \frac{\text{Memory stall clock cycles}}{\text{Instruction}} \right) \times \text{Clock cycle time}$$

$$\text{CPU execution time} = \text{IC} \times \left(\text{CPI}_{\text{execution}} + \frac{\text{Misses}}{\text{Instruction}} \times \text{Miss penalty} \right) \times \text{Clock cycle time}$$

$$\text{CPU execution time} = \text{IC} \times \left(\text{CPI}_{\text{execution}} + \text{Miss rate} \times \frac{\text{Memory accesses}}{\text{Instruction}} \times \text{Miss penalty} \right) \times \text{Clock cycle time}$$

$$\frac{\text{Memory stall cycles}}{\text{Instruction}} = \frac{\text{Misses}}{\text{Instruction}} \times (\text{Total miss latency} - \text{Overlapped miss latency})$$

$$\text{Average memory access time} = \text{Hit time}_{\text{L1}} + \text{Miss rate}_{\text{L1}} \times (\text{Hit time}_{\text{L2}} + \text{Miss rate}_{\text{L2}} \times \text{Miss penalty}_{\text{L2}})$$

$$\frac{\text{Memory stall cycles}}{\text{Instruction}} = \frac{\text{Misses}_{\text{L1}}}{\text{Instruction}} \times \text{Hit time}_{\text{L2}} + \frac{\text{Misses}_{\text{L2}}}{\text{Instruction}} \times \text{Miss penalty}_{\text{L2}}$$

Figure B.7 Summary of performance equations in this appendix. The first equation calculates the cache index size, and the rest help evaluate performance. The final two equations deal with multilevel caches, which are explained early in the next section. They are included here to help make the figure a useful reference.

B.3 Six Basic Cache Optimizations

The average memory access time formula gave us a framework to present cache optimizations for improving cache performance:

$$\text{Average memory access time} = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$$

Hence, we organize six cache optimizations into three categories:

- *Reducing the miss rate*—larger block size, larger cache size, and higher associativity
- *Reducing the miss penalty*—multilevel caches and giving reads priority over writes
- *Reducing the time to hit in the cache*—avoiding address translation when indexing the cache

Figure B.18 on page B-40 concludes this section with a summary of the implementation complexity and the performance benefits of these six techniques.

The classical approach to improving cache behavior is to reduce miss rates, and we present three techniques to do so. To gain better insights into the causes of misses, we first start with a model that sorts all misses into three simple categories:

- *Compulsory*—The very first access to a block *cannot* be in the cache, so the block must be brought into the cache. These are also called *cold-start misses* or *first-reference misses.*
- *Capacity*—If the cache cannot contain all the blocks needed during execution of a program, capacity misses (in addition to compulsory misses) will occur because of blocks being discarded and later retrieved.
- *Conflict*—If the block placement strategy is set associative or direct mapped, conflict misses (in addition to compulsory and capacity misses) will occur because a block may be discarded and later retrieved if too many blocks map to its set. These misses are also called *collision misses*. The idea is that hits in a fully associative cache that become misses in an *n*-way set-associative cache are due to more than *n* requests on some popular sets.

(Chapter 5 adds a fourth C, for *coherency* misses due to cache flushes to keep multiple caches coherent in a multiprocessor; we won't consider those here.)

Figure B.8 shows the relative frequency of cache misses, broken down by the three C's. Compulsory misses are those that occur in an infinite cache. Capacity misses are those that occur in a fully associative cache. Conflict misses are those that occur going from fully associative to eight-way associative, four-way associative, and so on. Figure B.9 presents the same data graphically. The top graph shows absolute miss rates; the bottom graph plots the percentage of all the misses by type of miss as a function of cache size.

To show the benefit of associativity, conflict misses are divided into misses caused by each decrease in associativity. Here are the four divisions of conflict misses and how they are calculated:

- *Eight-way*—Conflict misses due to going from fully associative (no conflicts) to eight-way associative
- *Four-way*—Conflict misses due to going from eight-way associative to four-way associative
- *Two-way*—Conflict misses due to going from four-way associative to two-way associative
- *One-way*—Conflict misses due to going from two-way associative to one-way associative (direct mapped)

As we can see from the figures, the compulsory miss rate of the SPEC2000 programs is very small, as it is for many long-running programs.

Having identified the three C's, what can a computer designer do about them? Conceptually, conflicts are the easiest: Fully associative placement avoids all

Cache size (KiB)	Degree associative	Total miss rate	Miss rate components (relative percent) (sum = 100% of total miss rate)					
			Compulsory		Capacity		Conflict	
4	1-way	0.098	0.0001	0.1%	0.070	72%	0.027	28%
4	2-way	0.076	0.0001	0.1%	0.070	93%	0.005	7%
4	4-way	0.071	0.0001	0.1%	0.070	99%	0.001	1%
4	8-way	0.071	0.0001	0.1%	0.070	100%	0.000	0%
8	1-way	0.068	0.0001	0.1%	0.044	65%	0.024	35%
8	2-way	0.049	0.0001	0.1%	0.044	90%	0.005	10%
8	4-way	0.044	0.0001	0.1%	0.044	99%	0.000	1%
8	8-way	0.044	0.0001	0.1%	0.044	100%	0.000	0%
16	1-way	0.049	0.0001	0.1%	0.040	82%	0.009	17%
16	2-way	0.041	0.0001	0.2%	0.040	98%	0.001	2%
16	4-way	0.041	0.0001	0.2%	0.040	99%	0.000	0%
16	8-way	0.041	0.0001	0.2%	0.040	100%	0.000	0%
32	1-way	0.042	0.0001	0.2%	0.037	89%	0.005	11%
32	2-way	0.038	0.0001	0.2%	0.037	99%	0.000	0%
32	4-way	0.037	0.0001	0.2%	0.037	100%	0.000	0%
32	8-way	0.037	0.0001	0.2%	0.037	100%	0.000	0%
64	1-way	0.037	0.0001	0.2%	0.028	77%	0.008	23%
64	2-way	0.031	0.0001	0.2%	0.028	91%	0.003	9%
64	4-way	0.030	0.0001	0.2%	0.028	95%	0.001	4%
64	8-way	0.029	0.0001	0.2%	0.028	97%	0.001	2%
128	1-way	0.021	0.0001	0.3%	0.019	91%	0.002	8%
128	2-way	0.019	0.0001	0.3%	0.019	100%	0.000	0%
128	4-way	0.019	0.0001	0.3%	0.019	100%	0.000	0%
128	8-way	0.019	0.0001	0.3%	0.019	100%	0.000	0%
256	1-way	0.013	0.0001	0.5%	0.012	94%	0.001	6%
256	2-way	0.012	0.0001	0.5%	0.012	99%	0.000	0%
256	4-way	0.012	0.0001	0.5%	0.012	99%	0.000	0%
256	8-way	0.012	0.0001	0.5%	0.012	99%	0.000	0%
512	1-way	0.008	0.0001	0.8%	0.005	66%	0.003	33%
512	2-way	0.007	0.0001	0.9%	0.005	71%	0.002	28%
512	4-way	0.006	0.0001	1.1%	0.005	91%	0.000	8%
512	8-way	0.006	0.0001	1.1%	0.005	95%	0.000	4%

Figure B.8 Total miss rate for each size cache and percentage of each according to the three C's. Compulsory misses are independent of cache size, while capacity misses decrease as capacity increases, and conflict misses decrease as associativity increases. Figure B.9 shows the same information graphically. Note that a direct-mapped cache of size *N* has about the same miss rate as a two-way set-associative cache of size *N*/2 up through 128 K. Caches larger than 128 KiB do not prove that rule. Note that the Capacity column is also the fully associative miss rate. Data were collected as in Figure B.4 using LRU replacement.

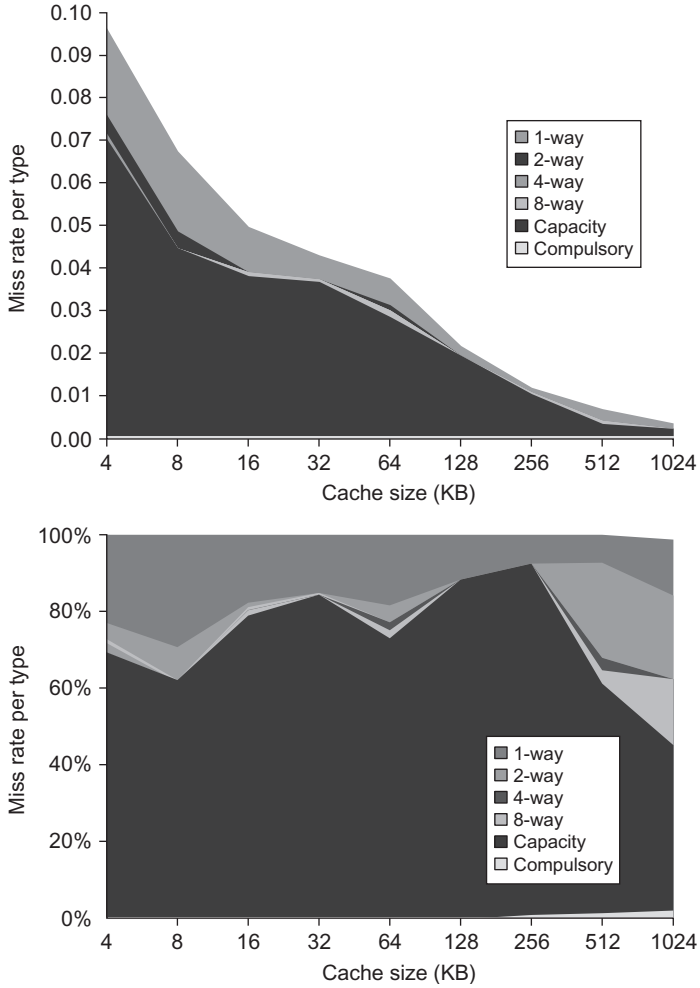


Figure B.9 Total miss rate (top) and distribution of miss rate (bottom) for each size cache according to the three C's for the data in Figure B.8. The top diagram shows the actual data cache miss rates, while the bottom diagram shows the percentage in each category. (*Space allows* the graphs to show one extra cache size than can fit in Figure B.8.)

conflict misses. Full associativity is expensive in hardware, however, and may slow the processor clock rate (see the example on page B-29), leading to lower overall performance.

There is little to be done about capacity except to enlarge the cache. If the upper-level memory is much smaller than what is needed for a program, and a significant percentage of the time is spent moving data between two levels in the

hierarchy, the memory hierarchy is said to *thrash*. Because so many replacements are required, thrashing means the computer runs close to the speed of the lower-level memory, or maybe even slower because of the miss overhead.

Another approach to improving the three C's is to make blocks larger to reduce the number of compulsory misses, but, as we will see shortly, large blocks can increase other kinds of misses.

The three C's give insight into the cause of misses, but this simple model has its limits; it gives you insight into average behavior but may not explain an individual miss. For example, changing cache size changes conflict misses as well as capacity misses, because a larger cache spreads out references to more blocks. Thus, a miss might move from a capacity miss to a conflict miss as cache size changes. Similarly, changing the block size can sometimes reduce capacity misses (in addition to the expected reduction in compusolory misses), as Gupta et al. (2013) show.

Note also that the three C's also ignore replacement policy, because it is difficult to model and because, in general, it is less significant. In specific circumstances the replacement policy can actually lead to anomalous behavior, such as poorer miss rates for larger associativity, which contradicts the three C's model. (Some have proposed using an address trace to determine optimal placement in memory to avoid placement misses from the three C's model; we've not followed that advice here.)

Alas, many of the techniques that reduce miss rates also increase hit time or miss penalty. The desirability of reducing miss rates using the three optimizations must be balanced against the goal of making the whole system fast. This first example shows the importance of a balanced perspective.

First Optimization: Larger Block Size to Reduce Miss Rate

The simplest way to reduce miss rate is to increase the block size. Figure B.10 shows the trade-off of block size versus miss rate for a set of programs and cache sizes. Larger block sizes will reduce also compulsory misses. This reduction occurs because the principle of locality has two components: temporal locality and spatial locality. Larger blocks take advantage of spatial locality.

At the same time, larger blocks increase the miss penalty. Because they reduce the number of blocks in the cache, larger blocks may increase conflict misses and even capacity misses if the cache is small. Clearly, there is little reason to increase the block size to such a size that it *increases* the miss rate. There is also no benefit to reducing miss rate if it increases the average memory access time. The increase in miss penalty may outweigh the decrease in miss rate.

Example Figure B.11 shows the actual miss rates plotted in Figure B.10. Assume the memory system takes 80 clock cycles of overhead and then delivers 16 bytes every 2 clock cycles. Thus, it can supply 16 bytes in 82 clock cycles, 32 bytes in 84 clock cycles, and so on. Which block size has the smallest average memory access time for each cache size in Figure B.11?

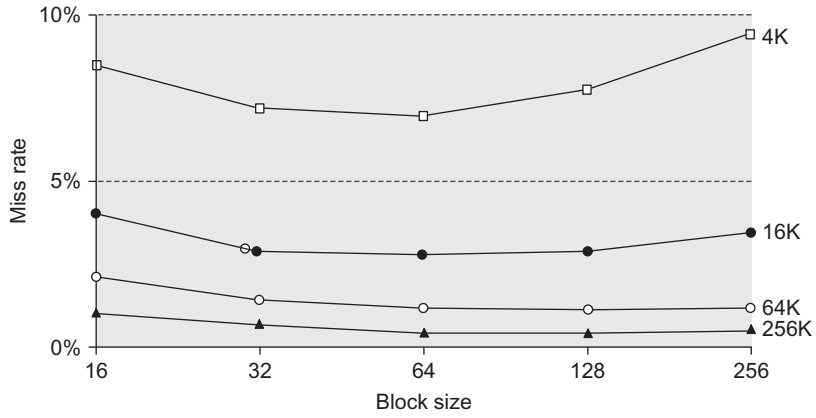


Figure B.10 Miss rate versus block size for five different-sized caches. Note that miss rate actually goes up if the block size is too large relative to the cache size. Each line represents a cache of different size. Figure B.11 shows the data used to plot these lines. Unfortunately, SPEC2000 traces would take too long if block size were included, so these data are based on SPEC92 on a DECstation 5000 (Gee et al. 1993).

Answer Average memory access time is

$$\text{Average memory access time} = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$$

If we assume the hit time is 1 clock cycle independent of block size, then the access time for a 16-byte block in a 4 KiB cache is

$$\text{Average memory access time} = 1 + (8.57\% \times 82) = 8.027 \text{ clock cycles}$$

and for a 256-byte block in a 256 KiB cache the average memory access time is

$$\text{Average memory access time} = 1 + (0.49\% \times 112) = 1.549 \text{ clock cycles}$$

	Cache size			
Block size	**4K**	**16K**	**64K**	**256K**
16	8.57%	3.94%	2.04%	1.09%
32	7.24%	2.87%	1.35%	0.70%
64	7.00%	2.64%	1.06%	0.51%
128	7.78%	2.77%	1.02%	0.49%
256	9.51%	3.29%	1.15%	0.49%

Figure B.11 Actual miss rate versus block size for the five different-sized caches in Figure B.10. Note that for a 4 KiB cache, 256-byte blocks have a higher miss rate than 32-byte blocks. In this example, the cache would have to be 256 KiB in order for a 256-byte block to decrease misses.

		Cache size			
Block size	**Miss penalty**	**4K**	**16K**	**64K**	**256K**
16	82	8.027	4.231	2.673	1.894
32	84	**7.082**	3.411	2.134	1.588
64	88	7.160	**3.323**	**1.933**	**1.449**
128	96	8.469	3.659	1.979	1.470
256	112	11.651	4.685	2.288	1.549

Figure B.12 Average memory access time versus block size for five different-sized caches in Figure B.10. Block sizes of 32 and 64 bytes dominate. The smallest average time per cache size is boldfaced.

Figure B.12 shows the average memory access time for all block and cache sizes between those two extremes. The boldfaced entries show the fastest block size for a given cache size: 32 bytes for 4 KiB and 64 bytes for the larger caches. These sizes are, in fact, popular block sizes for processor caches today.

As in all of these techniques, the cache designer is trying to minimize both the miss rate and the miss penalty. The selection of block size depends on both the latency and bandwidth of the lower-level memory. High latency and high bandwidth encourage large block size because the cache gets many more bytes per miss for a small increase in miss penalty. Conversely, low latency and low bandwidth encourage smaller block sizes because there is little time saved from a larger block. For example, twice the miss penalty of a small block may be close to the penalty of a block twice the size. The larger number of small blocks may also reduce conflict misses. Note that Figures B.10 and B.12 show the difference between selecting a block size based on minimizing miss rate versus minimizing average memory access time.

After seeing the positive and negative impact of larger block size on compulsory and capacity misses, the next two subsections look at the potential of higher capacity and higher associativity.

Second Optimization: Larger Caches to Reduce Miss Rate

The obvious way to reduce capacity misses in Figures B.8 and B.9 is to increase capacity of the cache. The obvious drawback is potentially longer hit time and higher cost and power. This technique has been especially popular in off-chip caches.

Third Optimization: Higher Associativity to Reduce Miss Rate

Figures B.8 and B.9 show how miss rates improve with higher associativity. There are two general rules of thumb that can be gleaned from these figures. The first is

that eight-way set associative is for practical purposes as effective in reducing misses for these sized caches as fully associative. You can see the difference by comparing the eight-way entries to the capacity miss column in Figure B.8, because capacity misses are calculated using fully associative caches.

The second observation, called the *2:1 cache rule of thumb*, is that a direct-mapped cache of size *N* has about the same miss rate as a two-way set associative cache of size *N*/2. This held in three C's figures for cache sizes less than 128 KiB.

Like many of these examples, improving one aspect of the average memory access time comes at the expense of another. Increasing block size reduces miss rate while increasing miss penalty, and greater associativity can come at the cost of increased hit time. Hence, the pressure of a fast processor clock cycle encourages simple cache designs, but the increasing miss penalty rewards associativity, as the following example suggests.

Example Assume that higher associativity would increase the clock cycle time as listed as follows:

$$\text{Clock cycle time}_{\text{2-way}} = 1.36 \times \text{Clock cycle time}_{\text{1-way}}$$
$$\text{Clock cycle time}_{\text{4-way}} = 1.44 \times \text{Clock cycle time}_{\text{1-way}}$$
$$\text{Clock cycle time}_{\text{8-way}} = 1.52 \times \text{Clock cycle time}_{\text{1-way}}$$

Assume that the hit time is 1 clock cycle, that the miss penalty for the direct-mapped case is 25 clock cycles to a level 2 cache (see next subsection) that never misses, and that the miss penalty need not be rounded to an integral number of clock cycles. Using Figure B.8 for miss rates, for which cache sizes are each of these three statements true?

$$\text{Average memory access time}_{\text{8-way}} < \text{Average memory access time}_{\text{4-way}}$$
$$\text{Average memory access time}_{\text{4-way}} < \text{Average memory access time}_{\text{2-way}}$$
$$\text{Average memory access time}_{\text{2-way}} < \text{Average memory access time}_{\text{1-way}}$$

Answer Average memory access time for each associativity is

$$\begin{aligned}
\text{Average memory access time}_{\text{8-way}} &= \text{Hit time}_{\text{8-way}} + \text{Miss rate}_{\text{8-way}} \times \text{Miss penalty}_{\text{8-way}} \\
&= 1.52 + \text{Miss rate}_{\text{8-way}} \times 25 \\
\text{Average memory access time}_{\text{4-way}} &= 1.44 + \text{Miss rate}_{\text{4-way}} \times 25 \\
\text{Average memory access time}_{\text{2-way}} &= 1.36 + \text{Miss rate}_{\text{2-way}} \times 25 \\
\text{Average memory access time}_{\text{1-way}} &= 1.00 + \text{Miss rate}_{\text{1-way}} \times 25
\end{aligned}$$

The miss penalty is the same time in each case, so we leave it as 25 clock cycles. For example, the average memory access time for a 4 KiB direct-mapped cache is

$$\text{Average memory access time}_{\text{1-way}} = 1.00 + (0.098 \times 25) = 3.44$$

and the time for a 512 KiB, eight-way set associative cache is

$$\text{Average memory access time}_{\text{8-way}} = 1.52 + (0.006 \times 25) = 1.66$$

Using these formulas and the miss rates from Figure B.8, Figure B.13 shows the average memory access time for each cache and associativity. The figure shows

	Associativity			
Cache size (KiB)	1-way	2-way	4-way	8-way
4	3.44	3.25	3.22	**3.28**
8	2.69	2.58	2.55	**2.62**
16	2.23	**2.40**	**2.46**	**2.53**
32	2.06	**2.30**	**2.37**	**2.45**
64	1.92	**2.14**	**2.18**	**2.25**
128	1.52	**1.84**	**1.92**	**2.00**
256	1.32	**1.66**	**1.74**	**1.82**
512	1.20	**1.55**	**1.59**	**1.66**

Figure B.13 Average memory access time using miss rates in Figure B.8 for parameters in the example. *Boldface* type means that this time is higher than the number to the left, that is, higher associativity *increases* average memory access time.

that the formulas in this example hold for caches less than or equal to 8 KiB for up to four-way associativity. Starting with 16 KiB, the greater hit time of larger associativity outweighs the time saved due to the reduction in misses.

Note that we did not account for the slower clock rate on the rest of the program in this example, thereby understating the advantage of direct-mapped cache.

Fourth Optimization: Multilevel Caches to Reduce Miss Penalty

Reducing cache misses had been the traditional focus of cache research, but the cache performance formula assures us that improvements in miss penalty can be just as beneficial as improvements in miss rate. Moreover, Figure 2.2 on page 80 shows that technology trends have improved the speed of processors faster than DRAMs, making the relative cost of miss penalties increase over time.

This performance gap between processors and memory leads the architect to this question: Should I make the cache faster to keep pace with the speed of processors, or make the cache larger to overcome the widening gap between the processor and main memory?

One answer is, do both. Adding another level of cache between the original cache and memory simplifies the decision. The first-level cache can be small enough to match the clock cycle time of the fast processor. Yet, the second-level cache can be large enough to capture many accesses that would go to main memory, thereby lessening the effective miss penalty.

Although the concept of adding another level in the hierarchy is straightforward, it complicates performance analysis. Definitions for a second level of cache are not always straightforward. Let's start with the definition of *average memory access time* for a two-level cache. Using the subscripts L1 and L2 to refer, respectively, to a first-level and a second-level cache, the original formula is

$$\text{Average memory access time} = \text{Hit time}_{L1} + \text{Miss rate}_{L1} \times \text{Miss penalty}_{L1}$$

and

$$\text{Miss penalty}_{L1} = \text{Hit time}_{L2} + \text{Miss rate}_{L2} \times \text{Miss penalty}_{L2}$$

so

$$\text{Average memory access time} = \text{Hit time}_{L1} + \text{Miss rate}_{L1} \times (\text{Hit time}_{L2} + \text{Miss rate}_{L2} \times \text{Miss penalty}_{L2})$$

In this formula, the second-level miss rate is measured on the leftovers from the first-level cache. To avoid ambiguity, these terms are adopted here for a two-level cache system:

- *Local miss rate*—This rate is simply the number of misses in a cache divided by the total number of memory accesses to this cache. As you would expect, for the first-level cache it is equal to Miss rate_{L1}, and for the second-level cache it is Miss rate_{L2}.
- *Global miss rate*—The number of misses in the cache divided by the total number of memory accesses generated by the processor. Using the terms above, the global miss rate for the first-level cache is still just Miss rate_{L1}, but for the second-level cache it is $\text{Miss rate}_{L1} \times \text{Miss rate}_{L2}$.

This local miss rate is large for second-level caches because the first-level cache skims the cream of the memory accesses. This is why the global miss rate is the more useful measure: It indicates what fraction of the memory accesses that leave the processor go all the way to memory.

Here is a place where the misses per instruction metric shines. Instead of confusion about local or global miss rates, we just expand memory stalls per instruction to add the impact of a second-level cache.

$$\text{Average memory stalls per instruction} = \text{Misses per instruction}_{L1} \times \text{Hit time}_{L2} + \text{Misses per instruction}_{L2} \times \text{Miss penalty}_{L2}$$

Example Suppose that in 1000 memory references there are 40 misses in the first-level cache and 20 misses in the second-level cache. What are the various miss rates? Assume the miss penalty from the L2 cache to memory is 200 clock cycles, the hit time of the L2 cache is 10 clock cycles, the hit time of L1 is 1 clock cycle, and there are 1.5 memory references per instruction. What is the average memory access time and average stall cycles per instruction? Ignore the impact of writes.

Answer The miss rate (either local or global) for the first-level cache is 40/1000 or 4%. The local miss rate for the second-level cache is 20/40 or 50%. The global miss rate of the second-level cache is 20/1000 or 2%. Then

$$\begin{aligned}\text{Average memory access time} &= \text{Hit time}_{L1} + \text{Miss rate}_{L1} \times (\text{Hit time}_{L2} + \text{Miss rate}_{L2} \times \text{Miss penalty}_{L2}) \\ &= 1 + 4\% \times (10 + 50\% \times 200) = 1 + 4\% \times 110 = 5.4 \text{ clock cycles}\end{aligned}$$

To see how many misses we get per instruction, we divide 1000 memory references by 1.5 memory references per instruction, which yields 667 instructions. Thus, we need to multiply the misses by 1.5 to get the number of misses per 1000 instructions. We have 40×1.5 or 60 L1 misses, and 20×1.5 or 30 L2 misses, per 1000 instructions. For average memory stalls per instruction, assuming the misses are distributed uniformly between instructions and data:

$$\begin{aligned}\text{Average memory stalls per instruction} &= \text{Misses per instruction}_{L1} \times \text{Hit time}_{L2} + \text{Misses per instruction}_{L2} \\ &\quad \times \text{Miss penalty}_{L2} \\ &= (60/1000) \times 10 + (30/1000) \times 200 \\ &= 0.060 \times 10 + 0.030 \times 200 = 6.6 \text{ clock cycles}\end{aligned}$$

If we subtract the L1 hit time from the average memory access time (AMAT) and then multiply by the average number of memory references per instruction, we get the same average memory stalls per instruction:

$$(5.4 - 1.0) \times 1.5 = 4.4 \times 1.5 = 6.6 \text{ clock cycles}$$

As this example shows, there may be less confusion with multilevel caches when calculating using misses per instruction versus miss rates.

Note that these formulas are for combined reads and writes, assuming a write-back first-level cache. Obviously, a write-through first-level cache will send *all* writes to the second level, not just the misses, and a write buffer might be used.

Figures B.14 and B.15 show how miss rates and relative execution time change with the size of a second-level cache for one design. From these figures we can gain two insights. The first is that the global cache miss rate is very similar to the single cache miss rate of the second-level cache, provided that the second-level cache is much larger than the first-level cache. Hence, our intuition and knowledge about the first-level caches apply. The second insight is that the local cache miss rate is *not* a good measure of secondary caches; it is a function of the miss rate of the first-level cache, and hence can vary by changing the first-level cache. Thus, the global cache miss rate should be used when evaluating second-level caches.

With these definitions in place, we can consider the parameters of second-level caches. The foremost difference between the two levels is that the speed of the first-level cache affects the clock rate of the processor, while the speed of the second-level cache only affects the miss penalty of the first-level cache. Thus, we can consider many alternatives in the second-level cache that would be ill chosen for the first-level cache. There are two major questions for the design of the second-level cache: Will it lower the average memory access time portion of the CPI, and how much does it cost?

The initial decision is the size of a second-level cache. Since everything in the first-level cache is likely to be in the second-level cache, the second-level cache should be much bigger than the first. If second-level caches are just a little bigger, the local miss rate will be high. This observation inspires the design of huge second-level caches—the size of main memory in older computers!

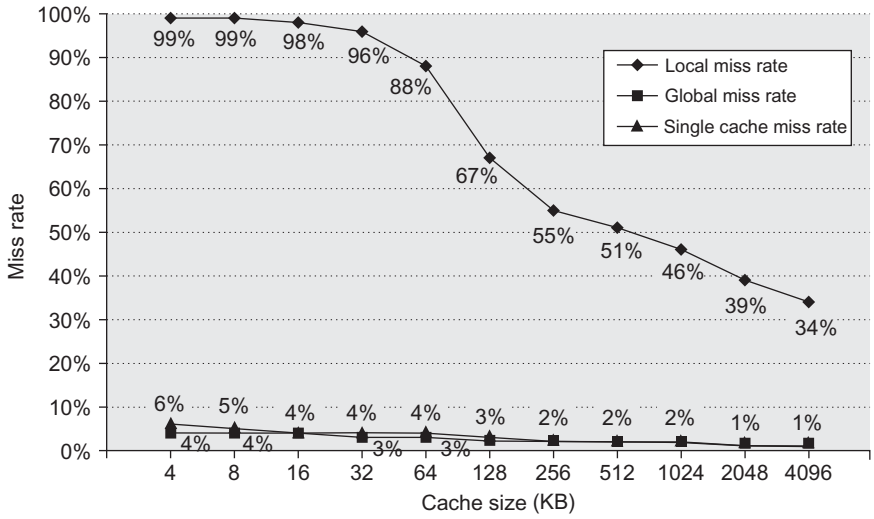


Figure B.14 Miss rates versus cache size for multilevel caches. Second-level caches *smaller* than the sum of the two 64 KiB first-level caches make little sense, as reflected in the high miss rates. After 256 KiB the single cache is within 10% of the global miss rates. The miss rate of a single-level cache versus size is plotted against the local miss rate and global miss rate of a second-level cache using a 32 KiB first-level cache. The L2 caches (unified) were two-way set associative with replacement. Each had split L1 instruction and data caches that were 64 KiB two-way set associative with LRU replacement. The block size for both L1 and L2 caches was 64 bytes. Data were collected as in Figure B.4.

One question is whether set associativity makes more sense for second-level caches.

Example Given the following data, what is the impact of second-level cache associativity on its miss penalty?

- Hit time$_{L2}$ for direct mapped = 10 clock cycles.
- Two-way set associativity increases hit time by 0.1 clock cycle to 10.1 clock cycles.
- Local miss rate$_{L2}$ for direct mapped = 25%.
- Local miss rate$_{L2}$ for two-way set associative = 20%.
- Miss penalty$_{L2}$ = 200 clock cycles.

Answer For a direct-mapped second-level cache, the first-level cache miss penalty is

$$\text{Miss penalty}_{\text{1-way L2}} = 10 + 25\% \times 200 = 60.0 \text{ clock cycles}$$

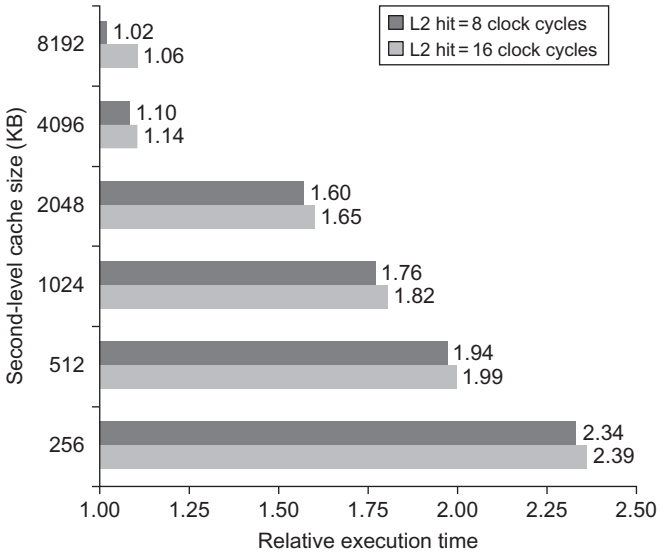


Figure B.15 Relative execution time by second-level cache size. The two bars are for different clock cycles for an L2 cache hit. The reference execution time of 1.00 is for an 8192 KiB second-level cache with a 1-clock-cycle latency on a second-level hit. These data were collected the same way as in Figure B.14, using a simulator to imitate the Alpha 21264.

Adding the cost of associativity increases the hit cost only 0.1 clock cycle, making the new first-level cache miss penalty:

$$\text{Miss penalty}_{\text{2-way L2}} = 10.1 + 20\% \times 200 = 50.1 \text{ clock cycles}$$

In reality, second-level caches are almost always synchronized with the first-level cache and processor. Accordingly, the second-level hit time must be an integral number of clock cycles. If we are lucky, we shave the second-level hit time to 10 cycles; if not, we round up to 11 cycles. Either choice is an improvement over the direct-mapped second-level cache:

$$\text{Miss penalty}_{\text{2-way L2}} = 10 + 20\% \times 200 = 50.0 \text{ clock cycles}$$
$$\text{Miss penalty}_{\text{2-way L2}} = 11 + 20\% \times 200 = 51.0 \text{ clock cycles}$$

Now we can reduce the miss penalty by reducing the *miss rate* of the second-level caches.

Another consideration concerns whether data in the first-level cache are in the second-level cache. *Multilevel inclusion* is the natural policy for memory hierarchies: L1 data are always present in L2. Inclusion is desirable because consistency between I/O and caches (or among caches in a multiprocessor) can be determined just by checking the second-level cache.

One drawback to inclusion is that measurements can suggest smaller blocks for the smaller first-level cache and larger blocks for the larger second-level cache. For example, the Pentium 4 has 64-byte blocks in its L1 caches and 128-byte blocks in its L2 cache. Inclusion can still be maintained with more work on a second-level miss. The second-level cache must invalidate all first-level blocks that map onto the second-level block to be replaced, causing a slightly higher first-level miss rate. To avoid such problems, many cache designers keep the block size the same in all levels of caches.

However, what if the designer can only afford an L2 cache that is slightly bigger than the L1 cache? Should a significant portion of its space be used as a redundant copy of the L1 cache? In such cases a sensible opposite policy is *multilevel exclusion:* L1 data are *never* found in an L2 cache. Typically, with exclusion a cache miss in L1 results in a swap of blocks between L1 and L2 instead of a replacement of an L1 block with an L2 block. This policy prevents wasting space in the L2 cache. For example, the AMD Opteron chip obeys the exclusion property using two 64 KiB L1 caches and 1 MiB L2 cache.

As these issues illustrate, although a novice might design the first- and second-level caches independently, the designer of the first-level cache has a simpler job given a compatible second-level cache. It is less of a gamble to use a write through, for example, if there is a write-back cache at the next level to act as a backstop for repeated writes and it uses multilevel inclusion.

The essence of all cache designs is balancing fast hits and few misses. For second-level caches, there are far fewer hits than in the first-level cache, so the emphasis shifts to fewer misses. This insight leads to much larger caches and techniques to lower the miss rate, such as higher associativity and larger blocks.

Fifth Optimization: Giving Priority to Read Misses over Writes to Reduce Miss Penalty

This optimization serves reads before writes have been completed. We start with looking at the complexities of a write buffer.

With a write-through cache the most important improvement is a write buffer of the proper size. Write buffers, however, do complicate memory accesses because they might hold the updated value of a location needed on a read miss.

Example Look at this code sequence:

```
sd x3, 512(x0);M[512] ¬ R3 (cache index 0)
ld x1, 1024(x0);x1 ¬ M[1024](cache index 0)
ld x2, 512(x0);x2 ¬ M[512] (cache index 0)
```

Assume a direct-mapped, write-through cache that maps 512 and 1024 to the same block, and a four-word write buffer that is not checked on a read miss. Will the value in x2 always be equal to the value in x3?

Answer Using the terminology from Chapter 2, this is a read-after-write data hazard in memory. Let's follow a cache access to see the danger. The data in x3 are placed into the write buffer after the store. The following load uses the same cache index and is therefore a miss. The second load instruction tries to put the value in location 512 into register x2; this also results in a miss. If the write buffer hasn't completed writing to location 512 in memory, the read of location 512 will put the old, wrong value into the cache block, and then into x2. Without proper precautions, x3x1 would not be equal to x2!

The simplest way out of this dilemma is for the read miss to wait until the write buffer is empty. The alternative is to check the contents of the write buffer on a read miss, and if there are no conflicts and the memory system is available, let the read miss continue. Virtually all desktop and server processors use the latter approach, giving reads priority over writes.

The cost of writes by the processor in a write-back cache can also be reduced. Suppose a read miss will replace a dirty memory block. Instead of writing the dirty block to memory, and then reading memory, we could copy the dirty block to a buffer, then read memory, and *then* write memory. This way the processor read, for which the processor is probably waiting, will finish sooner. Similar to the previous situation, if a read miss occurs, the processor can either stall until the buffer is empty or check the addresses of the words in the buffer for conflicts.

Now that we have five optimizations that reduce cache miss penalties or miss rates, it is time to look at reducing the final component of average memory access time. Hit time is critical because it can affect the clock rate of the processor; in many processors today the cache access time limits the clock cycle rate, even for processors that take multiple clock cycles to access the cache. Hence, a fast hit time is multiplied in importance beyond the average memory access time formula because it helps everything.

Sixth Optimization: Avoiding Address Translation During Indexing of the Cache to Reduce Hit Time

Even a small and simple cache must cope with the translation of a virtual address from the processor to a physical address to access memory. As described in Section B.4, processors treat main memory as just another level of the memory hierarchy, and thus the address of the virtual memory that exists on disk must be mapped onto the main memory.

The guideline of making the common case fast suggests that we use virtual addresses for the cache, because hits are much more common than misses. Such caches are termed *virtual caches*, with *physical cache* used to identify the traditional cache that uses physical addresses. As we will shortly see, it is important to distinguish two tasks: indexing the cache and comparing addresses. Thus, the issues are whether a virtual or physical address is used to index the cache and

whether a virtual or physical address is used in the tag comparison. Full virtual addressing for both indices and tags eliminates address translation time from a cache hit. Then why doesn't everyone build virtually addressed caches?

One reason is protection. Page-level protection is checked as part of the virtual to physical address translation, and it must be enforced no matter what. One solution is to copy the protection information from the TLB on a miss, add a field to hold it, and check it on every access to the virtually addressed cache.

Another reason is that every time a process is switched, the virtual addresses refer to different physical addresses, requiring the cache to be flushed. Figure B.16 shows the impact on miss rates of this flushing. One solution is to increase the width of the cache address tag with a *process-identifier tag* (PID). If the operating system assigns these tags to processes, it only need flush the cache when a PID is

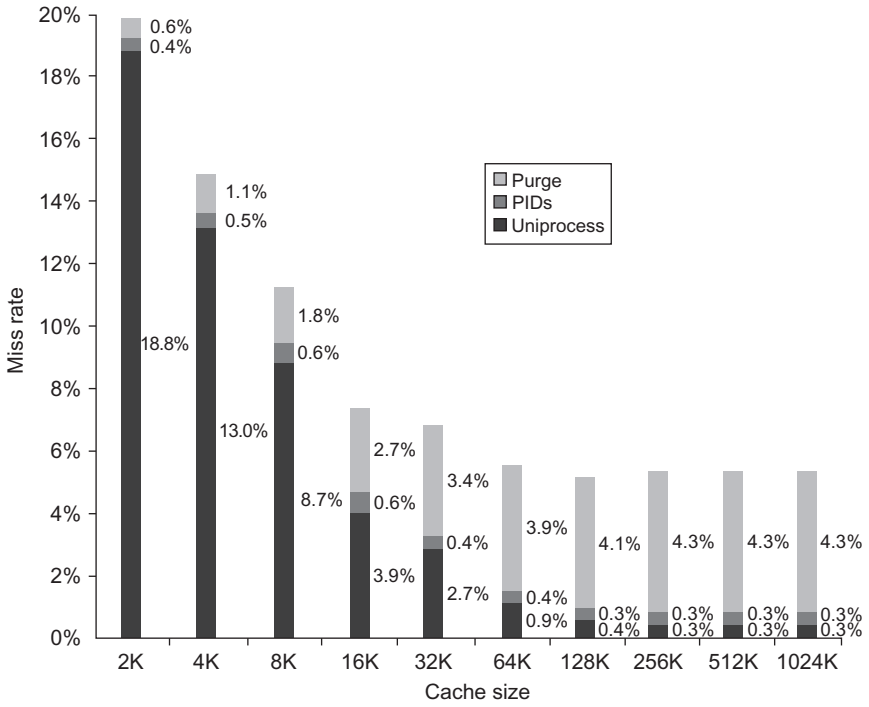


Figure B.16 Miss rate versus virtually addressed cache size of a program measured three ways: without process switches (uniprocess), with process switches using a process-identifier tag (PID), and with process switches but without PIDs (purge). PIDs increase the uniprocess absolute miss rate by 0.3%–0.6% and save 0.6%–4.3% over purging. Agarwal (1987) collected these statistics for the Ultrix operating system running on a VAX, assuming direct-mapped caches with a block size of 16 bytes. Note that the miss rate goes up from 128 to 256 K. Such nonintuitive behavior can occur in caches because changing size changes the mapping of memory blocks onto cache blocks, which can change the conflict miss rate.

recycled; that is, the PID distinguishes whether or not the data in the cache are for this program. Figure B.16 shows the improvement in miss rates by using PIDs to avoid cache flushes.

A third reason why virtual caches are not more popular is that operating systems and user programs may use two different virtual addresses for the same physical address. These duplicate addresses, called *synonyms* or *aliases*, could result in two copies of the same data in a virtual cache; if one is modified, the other will have the wrong value. With a physical cache this wouldn't happen, because the accesses would first be translated to the same physical cache block.

Hardware solutions to the synonym problem, called *antialiasing*, guarantee every cache block a unique physical address. For example, the AMD Opteron uses a 64 KiB instruction cache with a 4 KiB page and two-way set associativity; hence, the hardware must handle aliases involved with the three virtual address bits in the set index. It avoids aliases by simply checking all eight possible locations on a miss—two blocks in each of four sets—to be sure that none matches the physical address of the data being fetched. If one is found, it is invalidated, so when the new data are loaded into the cache their physical address is guaranteed to be unique.

Software can make this problem much easier by forcing aliases to share some address bits. An older version of UNIX from Sun Microsystems, for example, required all aliases to be identical in the last 18 bits of their addresses; this restriction is called *page coloring*. Note that page coloring is simply set associative mapping applied to virtual memory: the 4 KiB (2^{12}) pages are mapped using 64 (2^6) sets to ensure that the physical and virtual addresses match in the last 18 bits. This restriction means a direct-mapped cache that is 2^{18} (256 K) bytes or smaller can never have duplicate physical addresses for blocks. From the perspective of the cache, page coloring effectively increases the page offset, as software guarantees that the last few bits of the virtual and physical page address are identical.

The final area of concern with virtual addresses is I/O. I/O typically uses physical addresses and thus would require mapping to virtual addresses to interact with a virtual cache. (The impact of I/O on caches is further discussed in Appendix D.)

One alternative to get the best of both virtual and physical caches is to use part of the page offset—the part that is identical in both virtual and physical addresses—to index the cache. At the same time as the cache is being read using that index, the virtual part of the address is translated, and the tag match uses physical addresses.

This alternative allows the cache read to begin immediately, and yet the tag comparison is still with physical addresses. The limitation of this *virtually indexed, physically tagged* alternative is that a direct-mapped cache can be no bigger than the page size. For example, in the data cache in Figure B.5 on page B-13, the index is 9 bits and the cache block offset is 6 bits. To use this trick, the virtual page size would have to be at least $2^{(9+6)}$ bytes or 32 KiB. If not, a portion of the index must be translated from virtual to physical address. Figure B.17 shows the organization of the caches, translation lookaside buffers (TLBs), and virtual memory when this technique is used.

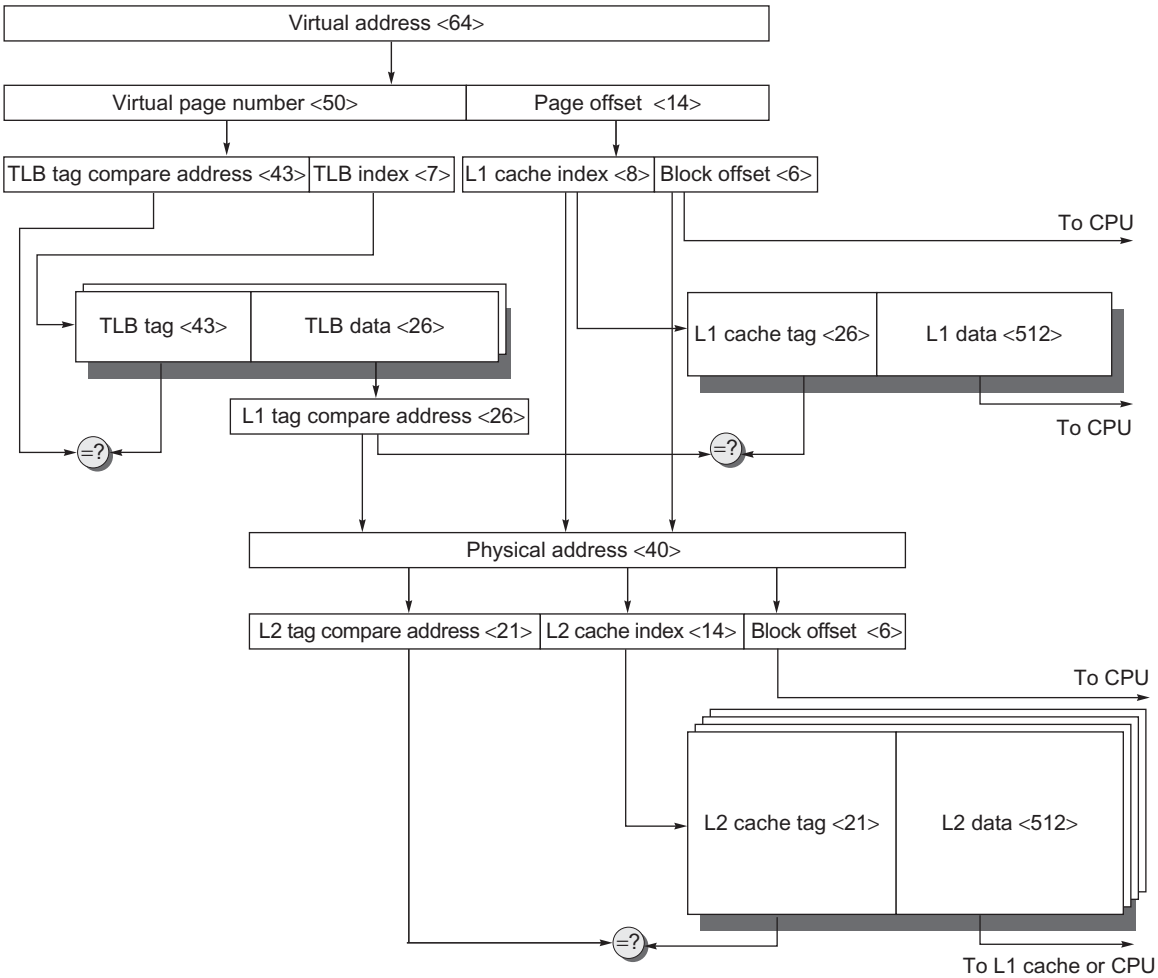


Figure B.17 The overall picture of a hypothetical memory hierarchy going from virtual address to L2 cache access. The page size is 16 KiB. The TLB is two-way set associative with 256 entries. The L1 cache is a direct-mapped 16 KiB, and the L2 cache is a four-way set associative with a total of 4 MiB. Both use 64-byte blocks. The virtual address is 64 bits and the physical address is 40 bits.

Associativity can keep the index in the physical part of the address and yet still support a large cache. Recall that the size of the index is controlled by this formula:

$$2^{\text{Index}} = \frac{\text{Cache size}}{\text{Block size} \times \text{Set associativity}}$$

For example, doubling associativity and doubling the cache size does not change the size of the index. The IBM 3033 cache, as an extreme example, is 16-way set associative, even though studies show there is little benefit to miss rates above

Technique	Hit time	Miss penalty	Miss rate	Hardware complexity	Comment
Larger block size		–	+	0	Trivial; Pentium 4L2 uses 128 bytes
Larger cache size	–		+	1	Widely used, especially for L2 caches
Higher associativity	–		+	1	Widely used
Multilevel caches		+		2	Costly hardware; harder if L1 block size ≠ L2 block size; widely used
Read priority over writes		+		1	Widely used
Avoiding address translation during cache indexing	+			1	Widely used

Figure B.18 Summary of basic cache optimizations showing impact on cache performance and complexity for the techniques in this appendix. Generally a technique helps only one factor. + means that the technique improves the factor, – means it hurts that factor, and blank means it has no impact. The complexity measure is subjective, with 0 being the easiest and 3 being a challenge.

8-way set associativity. This high associativity allows a 64 KiB cache to be addressed with a physical index, despite the handicap of 4 KiB pages in the IBM architecture.

Summary of Basic Cache Optimization

The techniques in this section to improve miss rate, miss penalty, and hit time generally impact the other components of the average memory access equation as well as the complexity of the memory hierarchy. Figure B.18 summarizes these techniques and estimates the impact on complexity, with + meaning that the technique improves the factor, – meaning it hurts that factor, and blank meaning it has no impact. No optimization in this figure helps more than one category.

B.4 Virtual Memory

… a system has been devised to make the core drum combination appear to the programmer as a single level store, the requisite transfers taking place automatically.

Kilburn et al. (1962)

At any instant in time computers are running multiple processes, each with its own address space. (Processes are described in the next section.) It would be too expensive to dedicate a full address space worth of memory for each process, especially because many processes use only a small part of their address space. Hence, there must be a means of sharing a smaller amount of physical memory among many processes.

One way to do this, *virtual memory*, divides physical memory into blocks and allocates them to different processes. Inherent in such an approach must be a *protection* scheme that restricts a process to the blocks belonging only to that process. Most forms of virtual memory also reduce the time to start a program, because not all code and data need be in physical memory before a program can begin.

Although protection provided by virtual memory is essential for current computers, sharing is not the reason that virtual memory was invented. If a program became too large for physical memory, it was the programmer's job to make it fit. Programmers divided programs into pieces, then identified the pieces that were mutually exclusive, and loaded or unloaded these *overlays* under user program control during execution. The programmer ensured that the program never tried to access more physical main memory than was in the computer, and that the proper overlay was loaded at the proper time. As you can well imagine, this responsibility eroded programmer productivity.

Virtual memory was invented to relieve programmers of this burden; it automatically manages the two levels of the memory hierarchy represented by main memory and secondary storage. Figure B.19 shows the mapping of virtual memory to physical memory for a program with four pages.

In addition to sharing protected memory space and automatically managing the memory hierarchy, virtual memory also simplifies loading the program for execution. Called *relocation*, this mechanism allows the same program to run in any location in physical memory. The program in Figure B.19 can be placed anywhere

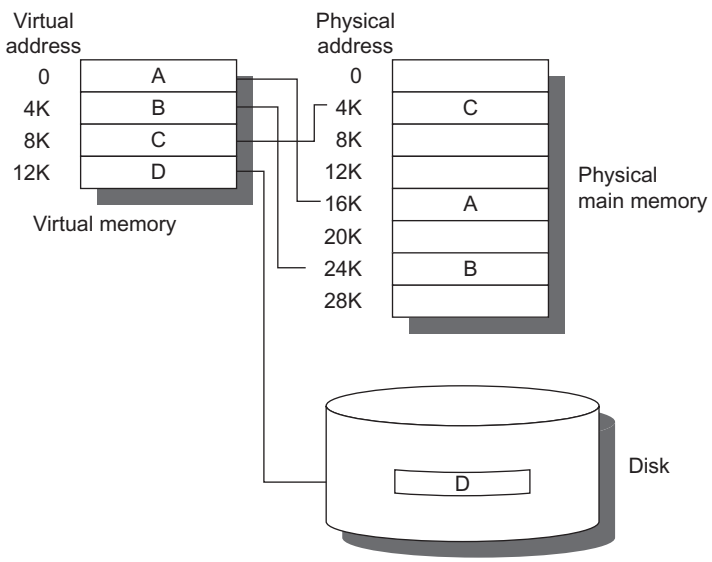


Figure B.19 The logical program in its contiguous virtual address space is shown on the left. It consists of four pages, A, B, C, and D. The actual location of three of the blocks is in physical main memory and the other is located on the disk.

in physical memory or disk just by changing the mapping between them. (Prior to the popularity of virtual memory, processors would include a relocation register just for that purpose.) An alternative to a hardware solution would be software that changed all addresses in a program each time it was run.

Several general memory hierarchy ideas from Chapter 1 about caches are analogous to virtual memory, although many of the terms are different. *Page* or *segment* is used for block, and *page fault* or *address fault* is used for miss. With virtual memory, the processor produces *virtual addresses* that are translated by a combination of hardware and software to *physical addresses*, which access main memory. This process is called *memory mapping* or *address translation*. Today, the two memory hierarchy levels controlled by virtual memory are DRAMs and magnetic disks. Figure B.20 shows a typical range of memory hierarchy parameters for virtual memory.

There are further differences between caches and virtual memory beyond those quantitative ones mentioned in Figure B.20:

- Replacement on cache misses is primarily controlled by hardware, while virtual memory replacement is primarily controlled by the operating system. The longer miss penalty means it's more important to make a good decision, so the operating system can be involved and take time deciding what to replace.
- The size of the processor address determines the size of virtual memory, but the cache size is independent of the processor address size.
- In addition to acting as the lower-level backing store for main memory in the hierarchy, secondary storage is also used for the file system. In fact, the file system occupies most of secondary storage. It is not usually in the address space.

Parameter	First-level cache	Virtual memory
Block (page) size	16–128 bytes	4096–65,536 bytes
Hit time	1–3 clock cycles	100–200 clock cycles
Miss penalty	8–200 clock cycles	1,000,000–10,000,000 clock cycles
(access time)	(6–160 clock cycles)	(800,000–8,000,000 clock cycles)
(transfer time)	(2–40 clock cycles)	(200,000–2,000,000 clock cycles)
Miss rate	0.1%–10%	0.00001%–0.001%
Address mapping	25–45-bit physical address to 14–20-bit cache address	32–64-bit virtual address to 25–45-bit physical address

Figure B.20 Typical ranges of parameters for caches and virtual memory. Virtual memory parameters represent increases of 10–1,000,000 times over cache parameters. Usually, first-level caches contain at most 1 MiB of data, whereas physical memory contains 256 MiB to 1 TB.

Virtual memory also encompasses several related techniques. Virtual memory systems can be categorized into two classes: those with fixed-size blocks, called *pages*, and those with variable-size blocks, called *segments*. Pages are typically fixed at 4096–8192 bytes, while segment size varies. The largest segment supported on any processor ranges from 2^{16} bytes up to 2^{32} bytes; the smallest segment is 1 byte. Figure B.21 shows how the two approaches might divide code and data.

The decision to use paged virtual memory versus segmented virtual memory affects the processor. Paged addressing has a single fixed-size address divided into page number and offset within a page, analogous to cache addressing. A single address does not work for segmented addresses; the variable size of segments requires 1 word for a segment number and 1 word for an offset within a segment, for a total of 2 words. An unsegmented address space is simpler for the compiler.

The pros and cons of these two approaches have been well documented in operating systems textbooks; Figure B.22 summarizes the arguments. Because of the

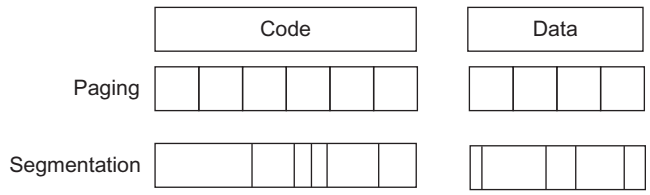


Figure B.21 Example of how paging and segmentation divide a program.

	Page	Segment
Words per address	One	Two (segment and offset)
Programmer visible?	Invisible to application programmer	May be visible to application programmer
Replacing a block	Trivial (all blocks are the same size)	Difficult (must find contiguous, variable-size, unused portion of main memory)
Memory use inefficiency	Internal fragmentation (unused portion of page)	External fragmentation (unused pieces of main memory)
Efficient disk traffic	Yes (adjust page size to balance access time and transfer time)	Not always (small segments may transfer just a few bytes)

Figure B.22 Paging versus segmentation. Both can waste memory, depending on the block size and how well the segments fit together in main memory. Programming languages with unrestricted pointers require both the segment and the address to be passed. A hybrid approach, called *paged segments*, shoots for the best of both worlds: segments are composed of pages, so replacing a block is easy, yet a segment may be treated as a logical unit.

replacement problem (the third line of the figure), few computers today use pure segmentation. Some computers use a hybrid approach, called *paged segments*, in which a segment is an integral number of pages. This simplifies replacement because memory need not be contiguous, and the full segments need not be in main memory. A more recent hybrid is for a computer to offer multiple page sizes, with the larger sizes being powers of 2 times the smallest page size. The IBM 405CR embedded processor, for example, allows 1 KiB, 4 KiB ($2^2 \times 1$ KiB), 16 KiB ($2^4 \times 1$ KiB), 64 KiB ($2^6 \times 1$ KiB), 256 KiB ($2^8 \times 1$ KiB), 1024 KiB ($2^{10} \times 1$ KiB), and 4096 KiB ($2^{12} \times 1$ KiB) to act as a single page.

Four Memory Hierarchy Questions Revisited

We are now ready to answer the four memory hierarchy questions for virtual memory.

Q1: Where Can a Block be Placed in Main Memory?

The miss penalty for virtual memory involves access to a rotating magnetic storage device and is therefore quite high. Given the choice of lower miss rates or a simpler placement algorithm, operating systems designers usually pick lower miss rates because of the exorbitant miss penalty. Thus, operating systems allow blocks to be placed anywhere in main memory. According to the terminology in Figure B.2 on page B-8, this strategy would be labeled fully associative.

Q2: How Is a Block Found If It Is in Main Memory?

Both paging and segmentation rely on a data structure that is indexed by the page or segment number. This data structure contains the physical address of the block. For segmentation, the offset is added to the segment's physical address to obtain the final physical address. For paging, the offset is simply concatenated to this physical page address (see Figure B.23).

This data structure, containing the physical page addresses, usually takes the form of a *page table.* Indexed by the virtual page number, the size of the table is the number of pages in the virtual address space. Given a 32-bit virtual address, 4 KiB pages, and 4 bytes per page table entry (PTE), the size of the page table would be $(2^{32}/2^{12}) \times 2^2 = 2^{22}$ or 4 MiB.

To reduce the size of this data structure, some computers apply a hashing function to the virtual address. The hash allows the data structure to be the length of the number of *physical* pages in main memory. This number could be much smaller than the number of virtual pages. Such a structure is called an *inverted page table*. Using the previous example, a 512 MiB physical memory would only need 1 MiB (8×512 MiB/4 KiB) for an inverted page table; the extra 4 bytes per page table entry are for the virtual address. The HP/Intel IA-64 covers both bases by offering

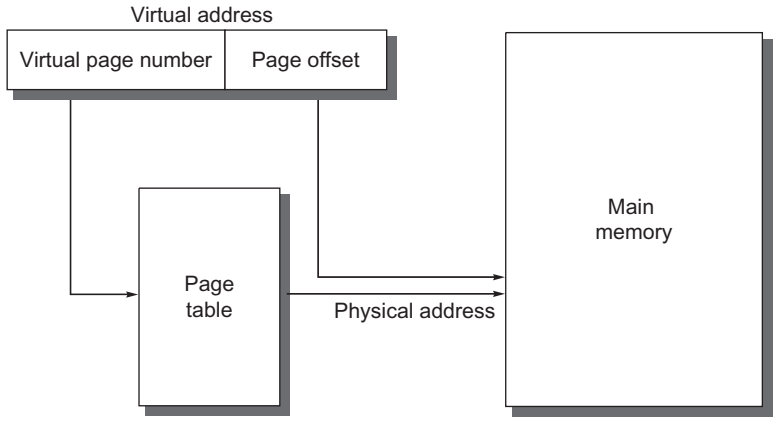


Figure B.23 The mapping of a virtual address to a physical address via a page table.

both traditional pages tables *and* inverted page tables, leaving the choice of mechanism to the operating system programmer.

To reduce address translation time, computers use a cache dedicated to these address translations, called a *translation lookaside buffer*, or simply *translation buffer*, described in more detail shortly.

Q3: Which Block Should be Replaced on a Virtual Memory Miss?

As mentioned earlier, the overriding operating system guideline is minimizing page faults. Consistent with this guideline, almost all operating systems try to replace the least recently used (LRU) block because if the past predicts the future, that is the one less likely to be needed.

To help the operating system estimate LRU, many processors provide a *use bit* or *reference bit*, which is logically set whenever a page is accessed. (To reduce work, it is actually set only on a translation buffer miss, which is described shortly.) The operating system periodically clears the use bits and later records them so it can determine which pages were touched during a particular time period. By keeping track in this way, the operating system can select a page that is among the least recently referenced.

Q4: What Happens on a Write?

The level below main memory contains rotating magnetic disks that take millions of clock cycles to access. Because of the great discrepancy in access time, no one has yet built a virtual memory operating system that writes through main memory to disk on every store by the processor. (This remark should not be interpreted as an opportunity to become famous by being the first to build one!) Thus, the write strategy is always write-back.

Because the cost of an unnecessary access to the next-lower level is so high, virtual memory systems usually include a dirty bit. It allows blocks to be written to disk only if they have been altered since being read from the disk.

Techniques for Fast Address Translation

Page tables are usually so large that they are stored in main memory and are sometimes paged themselves. Paging means that every memory access logically takes at least twice as long, with one memory access to obtain the physical address and a second access to get the data. As mentioned in Chapter 2, we use locality to avoid the extra memory access. By keeping address translations in a special cache, a memory access rarely requires a second access to translate the data. This special address translation cache is referred to as a *translation look aside buffer* (TLB), also called a *translation buffer* (TB).

A TLB entry is like a cache entry where the tag holds portions of the virtual address and the data portion holds a physical page frame number, protection field, valid bit, and usually a use bit and dirty bit. To change the physical page frame number or protection of an entry in the page table, the operating system must make sure the old entry is not in the TLB; otherwise, the system won't behave properly. Note that this dirty bit means the corresponding *page* is dirty, not that the address translation in the TLB is dirty nor that a particular block in the data cache is dirty. The operating system resets these bits by changing the value in the page table and then invalidates the corresponding TLB entry. When the entry is reloaded from the page table, the TLB gets an accurate copy of the bits.

Figure B.24 shows the Opteron data TLB organization, with each step of the translation labeled. This TLB uses fully associative placement; thus, the translation begins (steps 1 and 2) by sending the virtual address to all tags. Of course, the tag must be marked valid to allow a match. At the same time, the type of memory access is checked for a violation (also in step 2) against protection information in the TLB.

For reasons similar to those in the cache case, there is no need to include the 12 bits of the page offset in the TLB. The matching tag sends the corresponding physical address through effectively a 40:1 multiplexor (step 3). The page offset is then combined with the physical page frame to form a full physical address (step 4). The address size is 40 bits.

Address translation can easily be on the critical path determining the clock cycle of the processor, so the Opteron uses virtually addressed, physically tagged L1 caches.

Selecting a Page Size

The most obvious architectural parameter is the page size. Choosing the page is a question of balancing forces that favor a larger page size versus those favoring a smaller size. The following favor a larger size:

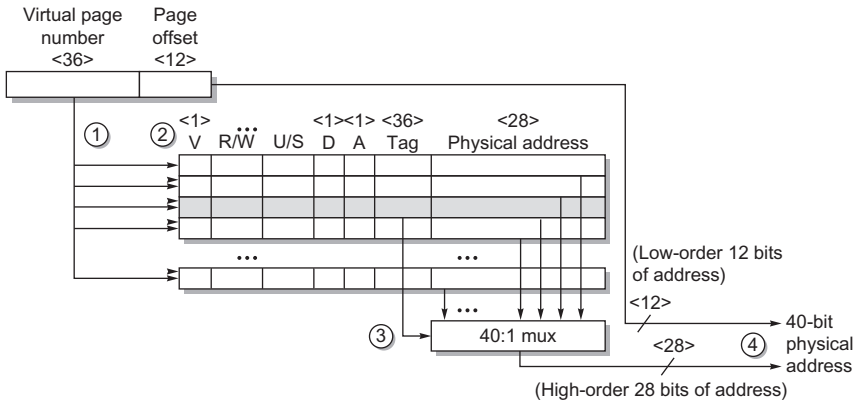


Figure B.24 Operation of the Opteron data TLB during address translation. The four steps of a TLB hit are shown as *circled numbers*. This TLB has 40 entries. Section B.5 describes the various protection and access fields of an Opteron page table entry.

- The size of the page table is inversely proportional to the page size; memory (or other resources used for the memory map) can therefore be saved by making the pages bigger.
- As mentioned in Section B.3, a larger page size can allow larger caches with fast cache hit times.
- Transferring larger pages to or from secondary storage, possibly over a network, is more efficient than transferring smaller pages.
- The number of TLB entries is restricted, so a larger page size means that more memory can be mapped efficiently, thereby reducing the number of TLB misses.

It is for this final reason that recent microprocessors have decided to support multiple page sizes; for some programs, TLB misses can be as significant on CPI as the cache misses.

The main motivation for a smaller page size is conserving storage. A small page size will result in less wasted storage when a contiguous region of virtual memory is not equal in size to a multiple of the page size. The term for this unused memory in a page is *internal fragmentation*. Assuming that each process has three primary segments (text, heap, and stack), the average wasted storage per process will be 1.5 times the page size. This amount is negligible for computers with hundreds of megabytes of memory and page sizes of 4–8 KiB. Of course, when the page sizes become very large (more than 32 KiB), storage (both main and secondary) could be wasted, as well as I/O bandwidth. A final concern is process start-up time; many processes are small, so a large page size would lengthen the time to invoke a process.

Summary of Virtual Memory and Caches

With virtual memory, TLBs, first-level caches, and second-level caches all mapping portions of the virtual and physical address space, it can get confusing what bits go where. Figure B.25 gives a hypothetical example going from a 64-bit virtual address to a 41-bit physical address with two levels of cache. This L1 cache is virtually indexed, and physically tagged because both the cache size and the page size are 8 KiB. The L2 cache is 4 MiB. The block size for both is 64 bytes.

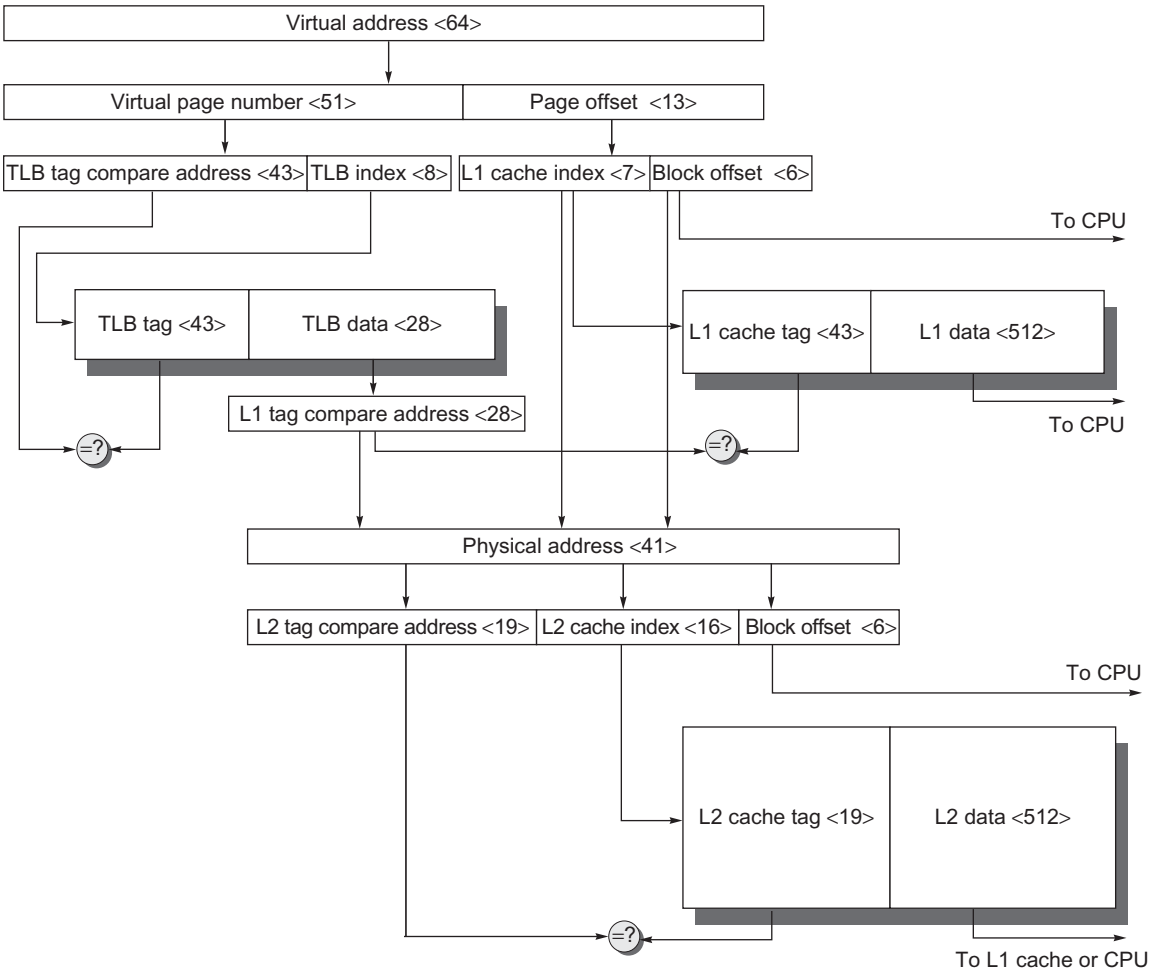


Figure B.25 The overall picture of a hypothetical memory hierarchy going from virtual address to L2 cache access. The page size is 8 KiB. The TLB is direct mapped with 256 entries. The L1 cache is a direct-mapped 8 KiB, and the L2 cache is a direct-mapped 4 MiB. Both use 64-byte blocks. The virtual address is 64 bits and the physical address is 41 bits. The primary difference between this simple figure and a real cache is replication of pieces of this figure.

First, the 64-bit virtual address is logically divided into a virtual page number and page offset. The former is sent to the TLB to be translated into a physical address, and the high bit of the latter is sent to the L1 cache to act as an index. If the TLB match is a hit, then the physical page number is sent to the L1 cache tag to check for a match. If it matches, it's an L1 cache hit. The block offset then selects the word for the processor.

If the L1 cache check results in a miss, the physical address is then used to try the L2 cache. The middle portion of the physical address is used as an index to the 4 MiB L2 cache. The resulting L2 cache tag is compared with the upper part of the physical address to check for a match. If it matches, we have an L2 cache hit, and the data are sent to the processor, which uses the block offset to select the desired word. On an L2 miss, the physical address is then used to get the block from memory.

Although this is a simple example, the major difference between this drawing and a real cache is replication. First, there is only one L1 cache. When there are two L1 caches, the top half of the diagram is duplicated. Note that this would lead to two TLBs, which is typical. Hence, one cache and TLB is for instructions, driven from the PC, and one cache and TLB is for data, driven from the effective address.

The second simplification is that all the caches and TLBs are direct mapped. If any were n-way set associative, then we would replicate each set of tag memory, comparators, and data memory n times and connect data memories with an n:1 multiplexor to select a hit. Of course, if the total cache size remained the same, the cache index would also shrink by $\log 2n$ bits according to the formula in Figure B.7 on page B-22.

B.5 Protection and Examples of Virtual Memory

The invention of multiprogramming, where a computer would be shared by several programs running concurrently, led to new demands for protection and sharing among programs. These demands are closely tied to virtual memory in computers today, and so we cover the topic here along with two examples of virtual memory.

Multiprogramming leads to the concept of a *process*. Metaphorically, a process is a program's breathing air and living space—that is, a running program plus any state needed to continue running it. Time-sharing is a variation of multiprogramming that shares the processor and memory with several interactive users at the same time, giving the illusion that all users have their own computers. Thus, at any instant it must be possible to switch from one process to another. This exchange is called a *process switch* or *context switch*.

A process must operate correctly whether it executes continuously from start to finish, or it is interrupted repeatedly and switched with other processes. The responsibility for maintaining correct process behavior is shared by designers of the computer and the operating system. The computer designer must ensure that

the processor portion of the process state can be saved and restored. The operating system designer must guarantee that processes do not interfere with each others' computations.

The safest way to protect the state of one process from another would be to copy the current information to disk. However, a process switch would then take seconds—far too long for a time-sharing environment.

This problem is solved by operating systems partitioning main memory so that several different processes have their state in memory at the same time. This division means that the operating system designer needs help from the computer designer to provide protection so that one process cannot modify another. Besides protection, the computers also provide for sharing of code and data between processes, to allow communication between processes or to save memory by reducing the number of copies of identical information.

Protecting Processes

Processes can be protected from one another by having their own page tables, each pointing to distinct pages of memory. Obviously, user programs must be prevented from modifying their page tables or protection would be circumvented.

Protection can be escalated, depending on the apprehension of the computer designer or the purchaser. *Rings* added to the processor protection structure expand memory access protection from two levels (user and kernel) to many more. Like a military classification system of top secret, secret, confidential, and unclassified, concentric rings of security levels allow the most trusted to access anything, the second most trusted to access everything except the innermost level, and so on. The "civilian" programs are the least trusted and, hence, have the most limited range of accesses. There may also be restrictions on what pieces of memory can contain code—execute protection—and even on the entrance point between the levels. The Intel 80x86 protection structure, which uses rings, is described later in this section. It is not clear whether rings are an improvement in practice over the simple system of user and kernel modes.

As the designer's apprehension escalates to trepidation, these simple rings may not suffice. Restricting the freedom given a program in the inner sanctum requires a new classification system. Instead of a military model, the analogy of this system is to keys and locks: a program can't unlock access to the data unless it has the key. For these keys, or *capabilities*, to be useful, the hardware and operating system must be able to explicitly pass them from one program to another without allowing a program itself to forge them. Such checking requires a great deal of hardware support if time for checking keys is to be kept low.

The 80x86 architecture has tried several of these alternatives over the years. Because backward compatibility is one of the guidelines of this architecture, the most recent versions of the architecture include all of its experiments in virtual memory. We'll go over two of the options here: first the older segmented address space and then the newer flat, 64-bit address space.

A Segmented Virtual Memory Example: Protection in the Intel Pentium

The second system is the most dangerous system a man ever designs… . The general tendency is to over-design the second system, using all the ideas and frills that were cautiously sidetracked on the first one.

F. P. Brooks, Jr.
The Mythical Man-Month (1975)

The original 8086 used segments for addressing, yet it provided nothing for virtual memory or for protection. Segments had base registers but no bound registers and no access checks, and before a segment register could be loaded the corresponding segment had to be in physical memory. Intel's dedication to virtual memory and protection is evident in the successors to the 8086, with a few fields extended to support larger addresses. This protection scheme is elaborate, with many details carefully designed to try to avoid security loopholes. We'll refer to it as IA-32. The next few pages highlight a few of the Intel safeguards; if you find the reading difficult, imagine the difficulty of implementing them!

The first enhancement is to double the traditional two-level protection model: the IA-32 has four levels of protection. The innermost level (0) corresponds to the traditional kernel mode, and the outermost level (3) is the least privileged mode. The IA-32 has separate stacks for each level to avoid security breaches between the levels. There are also data structures analogous to traditional page tables that contain the physical addresses for segments, as well as a list of checks to be made on translated addresses.

The Intel designers did not stop there. The IA-32 divides the address space, allowing both the operating system and the user access to the full space. The IA-32 user can call an operating system routine in this space and even pass parameters to it while retaining full protection. This safe call is not a trivial action, because the stack for the operating system is different from the user's stack. Moreover, the IA-32 allows the operating system to maintain the protection level of the *called* routine for the parameters that are passed to it. This potential loophole in protection is prevented by not allowing the user process to ask the operating system to access something indirectly that it would not have been able to access itself. (Such security loopholes are called *Trojan horses*.)

The Intel designers were guided by the principle of trusting the operating system as little as possible, while supporting sharing and protection. As an example of the use of such protected sharing, suppose a payroll program writes checks and also updates the year-to-date information on total salary and benefits payments. Thus, we want to give the program the ability to read the salary and year-to-date information and modify the year-to-date information but not the salary. We will see the mechanism to support such features shortly. In the rest of this subsection, we will look at the big picture of the IA-32 protection and examine its motivation.

Adding Bounds Checking and Memory Mapping

The first step in enhancing the Intel processor was getting the segmented addressing to check bounds as well as supply a base. Rather than a base address, the segment registers in the IA-32 contain an index to a virtual memory data structure called a *descriptor table*. Descriptor tables play the role of traditional page tables. On the IA-32 the equivalent of a page table entry is a *segment descriptor*. It contains fields found in PTEs:

- *Present bit*—Equivalent to the PTE valid bit, used to indicate this is a valid translation
- *Base field*—Equivalent to a page frame address, containing the physical address of the first byte of the segment
- *Access bit*—Like the reference bit or use bit in some architectures that is helpful for replacement algorithms
- *Attributes field*—Specifies the valid operations and protection levels for operations that use this segment

There is also a *limit field,* not found in paged systems, which establishes the upper bound of valid offsets for this segment. Figure B.26 shows examples of IA-32 segment descriptors.

IA-32 provides an optional paging system in addition to this segmented addressing. The upper portion of the 32-bit address selects the segment descriptor, and the middle portion is an index into the page table selected by the descriptor. The following section describes the protection system that does not rely on paging.

Adding Sharing and Protection

To provide for protected sharing, half of the address space is shared by all processes and half is unique to each process, called *global address space* and *local address space*, respectively. Each half is given a descriptor table with the appropriate name. A descriptor pointing to a shared segment is placed in the global descriptor table, while a descriptor for a private segment is placed in the local descriptor table.

A program loads an IA-32 segment register with an index to the table *and* a bit saying which table it desires. The operation is checked according to the attributes in the descriptor, the physical address being formed by adding the offset in the processor to the base in the descriptor, provided the offset is less than the limit field. Every segment descriptor has a separate 2-bit field to give the legal access level of this segment. A violation occurs only if the program tries to use a segment with a lower protection level in the segment descriptor.

We can now show how to invoke the payroll program mentioned herein to update the year-to-date information without allowing it to update salaries. The program could be given a descriptor to the information that has the writable field clear, meaning it can read but not write the data. A trusted program can then be supplied that will only write the year-to-date information. It is given a descriptor with the

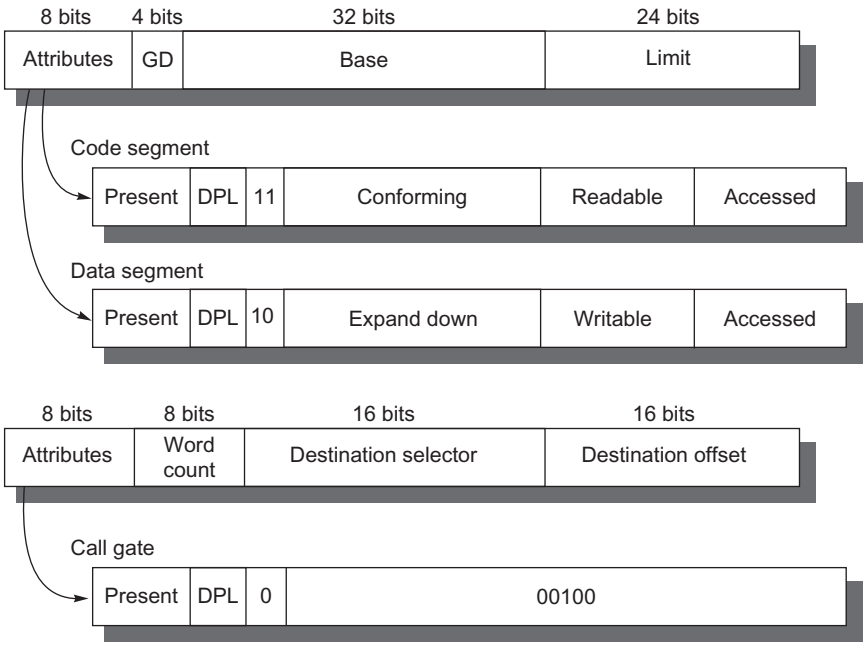


Figure B.26 The IA-32 segment descriptors are distinguished by bits in the attributes field. *Base, limit, present, readable*, and *writable* are all self-explanatory. D gives the default addressing size of the instructions: 16 bits or 32 bits. G gives the granularity of the segment limit: 0 means in bytes and 1 means in 4 KiB pages. G is set to 1 when paging is turned on to set the size of the page tables. DPL means *descriptor privilege level*—this is checked against the code privilege level to see if the access will be allowed. *Conforming* says the code takes on the privilege level of the code being called rather than the privilege level of the caller; it is used for library routines. The *expand-down field* flips the check to let the base field be the high-water mark and the limit field be the low-water mark. As you might expect, this is used for stack segments that grow down. *Word count* controls the number of words copied from the current stack to the new stack on a call gate. The other two fields of the call gate descriptor, *destination selector* and *destination offset,* select the descriptor of the destination of the call and the offset into it, respectively. There are many more than these three segment descriptors in the IA-32 protection model.

writable field set (Figure B.26). The payroll program invokes the trusted code using a code segment descriptor with the conforming field set. This setting means the called program takes on the privilege level of the code being called rather than the privilege level of the caller. Hence, the payroll program can read the salaries and call a trusted program to update the year-to-date totals, yet the payroll program cannot modify the salaries. If a Trojan horse exists in this system, to be effective it must be located in the trusted code whose only job is to update the year-to-date information. The argument for this style of protection is that limiting the scope of the vulnerability enhances security.

Adding Safe Calls from User to OS Gates and Inheriting Protection Level for Parameters

Allowing the user to jump into the operating system is a bold step. How, then, can a hardware designer increase the chances of a safe system without trusting the operating system or any other piece of code? The IA-32 approach is to restrict where the user can enter a piece of code, to safely place parameters on the proper stack, and to make sure the user parameters don't get the protection level of the called code.

To restrict entry into others' code, the IA-32 provides a special segment descriptor, or *call gate*, identified by a bit in the attributes field. Unlike other descriptors, call gates are full physical addresses of an object in memory; the offset supplied by the processor is ignored. As stated previously, their purpose is to prevent the user from randomly jumping anywhere into a protected or more privileged code segment. In our programming example, this means the only place the payroll program can invoke the trusted code is at the proper boundary. This restriction is needed to make conforming segments work as intended.

What happens if caller and callee are "mutually suspicious," so that neither trusts the other? The solution is found in the word count field in the bottom descriptor in Figure B.26. When a call instruction invokes a call gate descriptor, the descriptor copies the number of words specified in the descriptor from the local stack onto the stack corresponding to the level of this segment. This copying allows the user to pass parameters by first pushing them onto the local stack. The hardware then safely transfers them onto the correct stack. A return from a call gate will pop the parameters off both stacks and copy any return values to the proper stack. Note that this model is incompatible with the current practice of passing parameters in registers.

This scheme still leaves open the potential loophole of having the operating system use the user's address, passed as parameters, with the operating system's security level, instead of with the user's level. The IA-32 solves this problem by dedicating 2 bits in every processor segment register to the *requested protection level*. When an operating system routine is invoked, it can execute an instruction that sets this 2-bit field in all address parameters with the protection level of the user that called the routine. Thus, when these address parameters are loaded into the segment registers, they will set the requested protection level to the proper value. The IA-32 hardware then uses the requested protection level to prevent any foolishness: no segment can be accessed from the system routine using those parameters if it has a more privileged protection level than requested.

A Paged Virtual Memory Example: The 64-Bit Opteron Memory Management

AMD engineers found few uses of the elaborate protection model described in the previous section. The popular model is a flat, 32-bit address space, introduced by the 80386, which sets all the base values of the segment registers to zero. Hence,

AMD dispensed with the multiple segments in the 64-bit mode. It assumes that the segment base is zero and ignores the limit field. The page sizes are 4 KiB, 2 MiB, and 4 MiB.

The 64-bit virtual address of the AMD64 architecture is mapped onto 52-bit physical addresses, although implementations can implement fewer bits to simplify hardware. The Opteron, for example, uses 48-bit virtual addresses and 40-bit physical addresses. AMD64 requires that the upper 16 bits of the virtual address be just the sign extension of the lower 48 bits, which it calls *canonical form*.

The size of page tables for the 64-bit address space is alarming. Hence, AMD64 uses a multilevel hierarchical page table to map the address space to keep the size reasonable. The number of levels depends on the size of the virtual address space. Figure B.27 shows the four-level translation of the 48-bit virtual addresses of the Opteron.

The offsets for each of these page tables come from four 9-bit fields. Address translation starts with adding the first offset to the page-map level 4 base register and then reading memory from this location to get the base of the next-level page table. The next address offset is in turn added to this newly fetched address, and

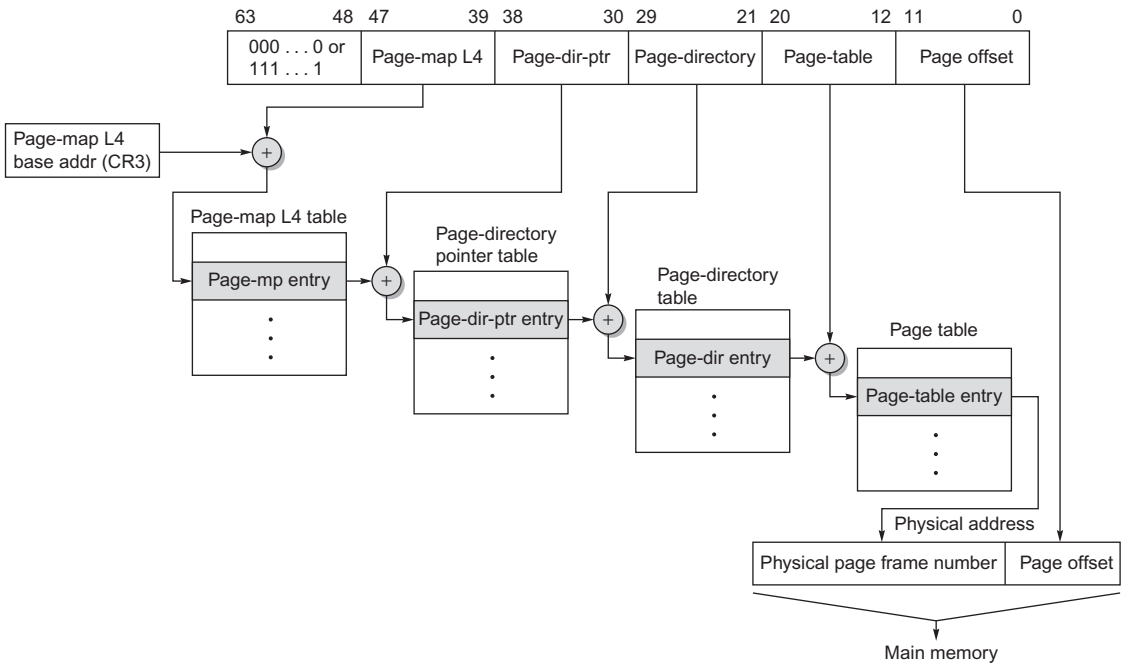


Figure B.27 The mapping of an Opteron virtual address. The Opteron virtual memory implementation with four page table levels supports an effective physical address size of 40 bits. Each page table has 512 entries, so each level field is 9 bits wide. The AMD64 architecture document allows the virtual address size to grow from the current 48 bits to 64 bits, and the physical address size to grow from the current 40 bits to 52 bits.

memory is accessed again to determine the base of the third page table. It happens again in the same fashion. The last address field is added to this final base address, and memory is read using this sum to (finally) get the physical address of the page being referenced. This address is concatenated with the 12-bit page offset to get the full physical address. Note that the page table in the Opteron architecture fits within a single 4 KiB page.

The Opteron uses a 64-bit entry in each of these page tables. The first 12 bits are reserved for future use, the next 52 bits contain the physical page frame number, and the last 12 bits give the protection and use information. Although the fields vary some between the page table levels, here are the basic ones:

- *Presence*—Says that page is present in memory.
- *Read/write*—Says whether page is read-only or read-write.
- *User/supervisor*—Says whether a user can access the page or if it is limited to the upper three privilege levels.
- *Dirty*—Says if page has been modified.
- *Accessed*—Says if page has been read or written since the bit was last cleared.
- *Page size*—Says whether the last level is for 4 KiB pages or 4 MiB pages; if it's the latter, then the Opteron only uses three instead of four levels of pages.
- *No execute*—Not found in the 80386 protection scheme, this bit was added to prevent code from executing in some pages.
- *Page level cache disable*—Says whether the page can be cached or not.
- *Page level write through*—Says whether the page allows write back or write through for data caches.

Because the Opteron usually goes through four levels of tables on a TLB miss, there are three potential places to check protection restrictions. The Opteron obeys only the bottom-level PTE, checking the others only to be sure the valid bit is set.

As the entry is 8 bytes long, each page table has 512 entries, and the Opteron has 4 KiB pages, the page tables are exactly one page long. Each of the four level fields are 9 bits long, and the page offset is 12 bits. This derivation leaves $64 - (4 \times 9 + 12)$ or 16 bits to be sign extended to ensure canonical addresses.

Although we have explained translation of legal addresses, what prevents the user from creating illegal address translations and getting into mischief? The page tables themselves are protected from being written by user programs. Thus, the user can try any virtual address, but by controlling the page table entries the operating system controls what physical memory is accessed. Sharing of memory between processes is accomplished by having a page table entry in each address space point to the same physical memory page.

The Opteron employs four TLBs to reduce address translation time, two for instruction accesses and two for data accesses. Like multilevel caches, the Opteron

Parameter	Description
Block size	1 PTE (8 bytes)
L1 hit time	1 clock cycle
L2 hit time	7 clock cycles
L1 TLB size	Same for instruction and data TLBs: 40 PTEs per TLBs, with 32 4 KiB pages and 8 for 2 MiB or 4 MiB pages
L2 TLB size	Same for instruction and data TLBs: 512 PTEs of 4 KiB pages
Block selection	LRU
Write strategy	(Not applicable)
L1 block placement	Fully associative
L2 block placement	4-way set associative

Figure B.28 Memory hierarchy parameters of the Opteron L1 and L2 instruction and data TLBs.

reduces TLB misses by having two larger L2 TLBs: one for instructions and one for data. Figure B.28 describes the data TLB.

Summary: Protection on the 32-Bit Intel Pentium Versus the 64-Bit AMD Opteron

Memory management in the Opteron is typical of most desktop or server computers today, relying on page-level address translation and correct operation of the operating system to provide safety to multiple processes sharing the computer. Although presented as alternatives, Intel has followed AMD's lead and embraced the AMD64 architecture. Hence, both AMD and Intel support the 64-bit extension of 80x86; yet, for compatibility reasons, both support the elaborate segmented protection scheme.

If the segmented protection model looks harder to build than the AMD64 model, that's because it is. This effort must be especially frustrating for the engineers, because few customers use the elaborate protection mechanism. In addition, the fact that the protection model is a mismatch to the simple paging protection of UNIX-like systems means it will be used only by someone writing an operating system especially for this computer, which hasn't happened yet.

B.6 Fallacies and Pitfalls

Even a review of memory hierarchy has fallacies and pitfalls!

Pitfall *Too small an address space.*

Just five years after DEC and Carnegie Mellon University collaborated to design the new PDP-11 computer family, it was apparent that their creation had a fatal

flaw. An architecture announced by IBM six years *before* the PDP-11 was still thriving, with minor modifications, 25 years later. And the DEC VAX, criticized for including unnecessary functions, sold millions of units after the PDP-11 went out of production. Why?

The fatal flaw of the PDP-11 was the size of its addresses (16 bits) as compared with the address sizes of the IBM 360 (24–31 bits) and the VAX (32 bits). Address size limits the program length, because the size of a program and the amount of data needed by the program must be less than $2^{\text{Address size}}$. The reason the address size is so hard to change is that it determines the minimum width of anything that can contain an address: PC, register, memory word, and effective-address arithmetic. If there is no plan to expand the address from the start, then the chances of successfully changing address size are so slim that it usually means the end of that computer family. Bell and Strecker (1976) put it like this:

> There is only one mistake that can be made in computer design that is difficult to recover from—not having enough address bits for memory addressing and memory management. The PDP-11 followed the unbroken tradition of nearly every known computer. [p. 2]

A partial list of successful computers that eventually starved to death for lack of address bits includes the PDP-8, PDP-10, PDP-11, Intel 8080, Intel 8086, Intel 80186, Intel 80286, Motorola 6800, AMI 6502, Zilog Z80, CRAY-1, and CRAY X-MP.

The venerable 80x86 line bears the distinction of having been extended twice, first to 32 bits with the Intel 80386 in 1985 and recently to 64 bits with the AMD Opteron.

Pitfall *Ignoring the impact of the operating system on the performance of the memory hierarchy.*

Figure B.29 shows the memory stall time due to the operating system spent on three large workloads. About 25% of the stall time is either spent in misses in the operating system or results from misses in the application programs because of interference with the operating system.

Pitfall *Relying on the operating systems to change the page size over time.*

The Alpha architects had an elaborate plan to grow the architecture over time by growing its page size, even building it into the size of its virtual address. When it came time to grow page sizes with later Alphas, the operating system designers balked and the virtual memory system was revised to grow the address space while maintaining the 8 KiB page.

Architects of other computers noticed very high TLB miss rates, and so added multiple, larger page sizes to the TLB. The hope was that operating systems programmers would allocate an object to the largest page that made sense, thereby preserving TLB entries. After a decade of trying, most operating systems use these "superpages" only for handpicked functions: mapping the display memory or other I/O devices, or using very large pages for the database code.

	Misses		Time						
			% Time due to application misses		% Time due directly to OS misses				
Workload	% in applications	% in OS	Inherent application misses	OS conflicts with applications	OS instruction misses	Data misses for migration	Data misses in block operations	Rest of OS misses	% Time OS misses and application conflicts
Pmake	47%	53%	14.1%	4.8%	10.9%	1.0%	6.2%	2.9%	25.8%
Multipgm	53%	47%	21.6%	3.4%	9.2%	4.2%	4.7%	3.4%	24.9%
Oracle	73%	27%	25.7%	10.2%	10.6%	2.6%	0.6%	2.8%	26.8%

Figure B.29 Misses and time spent in misses for applications and operating system. The operating system adds about 25% to the execution time of the application. Each processor has a 64 KiB instruction cache and a two-level data cache with 64 KiB in the first level and 256 KiB in the second level; all caches are direct mapped with 16-byte blocks. Collected on Silicon Graphics POWER station 4D/340, a multiprocessor with four 33 MHz R3000 processors running three application workloads under a UNIX System V—Pmake, a parallel compile of 56 files; Multipgm, the parallel numeric program MP3D running concurrently with Pmake and a five-screen edit session; and Oracle, running a restricted version of the TP-1 benchmark using the Oracle database. Data from Torrellas, J., Gupta, A., Hennessy, J., 1992. Characterizing the caching and synchronization performance of a multiprocessor operating system. In: Proceedings of the Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), October 12–15, 1992, Boston (SIGPLAN Notices 27:9 (September)), pp. 162–174.

B.7 Concluding Remarks

The difficulty of building a memory system to keep pace with faster processors is underscored by the fact that the raw material for main memory is the same as that found in the cheapest computer. It is the principle of locality that helps us here—its soundness is demonstrated at all levels of the memory hierarchy in current computers, from disks to TLBs.

However, the increasing relative latency to memory, taking hundreds of clock cycles in 2016, means that programmers and compiler writers must be aware of the parameters of the caches and TLBs if they want their programs to perform well.

B.8 Historical Perspective and References

In Section M.3 (available online) we examine the history of caches, virtual memory, and virtual machines. (The historical section covers both this appendix and Chapter 3.) IBM plays a prominent role in the history of all three. References for further reading are included.

Additional reference: Gupta, S. Xiang, P., Yang, Y., Zhou, H., Locality principle revisited: a probability-based quantitative approach. J. Parallel Distrib. Comput. 73 (7), 1011–1027.

Exercises by Amr Zaky

B.1 [10/10/10/15] <B.1> You are trying to appreciate how important the principle of locality is in justifying the use of a cache memory, so you experiment with a computer having an L1 data cache and a main memory (you exclusively focus on data accesses). The latencies (in CPU cycles) of the different kinds of accesses are as follows: cache hit, 1 cycle; cache miss, 110 cycles; main memory access with cache disabled, 105 cycles.

a. [10] <B.1> When you run a program with an overall miss rate of 3%, what will the average memory access time (in CPU cycles) be?

b. [10] <B.1> Next, you run a program specifically designed to produce completely random data addresses with no locality. Toward that end, you use an array of size 1 GB (all of which fits in the main memory). Accesses to random elements of this array are continuously made (using a uniform random number generator to generate the elements indices). If your data cache size is 64 KB, what will the average memory access time be?

c. [10] <B.1> If you compare the result obtained in part (b) with the main memory access time when the cache is disabled, what can you conclude about the role of the principle of locality in justifying the use of cache memory?

d. [15] <B.1> You observed that a cache hit produces a gain of 104 cycles (1 cycle vs. 105), but it produces a loss of 5 cycles in the case of a miss (110 cycles vs. 105). In the general case, we can express these two quantities as G (gain) and L (loss). Using these two quantities (G and L), identify the highest miss rate after which the cache use would be disadvantageous.

B.2 [15/15] <B.1> For the purpose of this exercise, we assume that we have a 512-byte cache with 64-byte blocks. We will also assume that the main memory is 2 KB large. We can regard the memory as an array of 64-byte blocks: M0, M1, …, M31. Figure B.30 sketches the memory blocks that can reside in different cache blocks if the cache was direct-mapped.

a. [15] <B.1> Show the contents of the table if the cache is organized as a fully-associative cache.

b. [15] <B.1> Repeat part (a) with the cache organized as a four-way set associative cache.

B.3 [10/10/10/10/15/10/15/20] <B.1> Cache organization is often influenced by the desire to reduce the cache's power consumption. For that purpose we assume that the cache is physically distributed into a data array (holding the data), a tag array (holding the tags), and replacement array (holding information needed by replacement policy). Furthermore, every one of these arrays is physically distributed into multiple subarrays (one per way) that can be individually accessed; for example, a four-way set associative least recently used (LRU) cache would have four data subarrays, four tag subarrays, and four replacement subarrays. We assume that the

Cache block	Set	Way	Possible memory blocks
0	0	0	M0, M8, M16, M24
1	1	0	M1, M9, M17, M25
2	2	0	M2, M10, M18, M26
3	3	0	….
4	4	0	….
5	5	0	….
6	6	0	….
7	7	0	M7, M15, M23, M31

Figure B.30 Memory blocks distributed to direct-mapped cache.

Array	Power consumption weight (per way accessed)
Data array	20 units
Tag	Array 5 units
Miscellaneous array	1 unit
Memory access	200 units

Figure B.31 Power consumption costs of different operations.

replacement subarrays are accessed once per access when the LRU replacement policy is used, and once per miss if the first-in, first-out (FIFO) replacement policy is used. It is not needed when a random replacement policy is used. For a specific cache, it was determined that the accesses to the different arrays have the following power consumption weights (Figure B.31):

a. [10] <B.1> A cache read hit. All arrays are read simultaneously.

b. [10] <B.1> Repeat part (a) for a cache read miss.

c. [10] <B.1> Repeat part (a) assuming that the cache access is split across two cycles. In the first cycle, all the tag subarrays are accessed. In the second cycle, only the subarray whose tag matched will be accessed.

d. [10] <B.1> Repeat part (c) for a cache read miss (no data array accesses in the second cycle).

e. [15] <B.1> Repeat part (c) assuming that logic is added to predict the cache way to be accessed. Only the tag subarray for the predicted way is accessed in cycle one. A way hit (address match in predicted way) implies a cache hit. A way miss dictates examining all the tag subarrays in the second cycle. In case of a way hit, only one data subarray (the one whose tag matched) is accessed in cycle two. Assume the way predictor hits.

f. [10] <B.1> Repeat part (e) assuming that the way predictor misses (the way it choses is wrong). When it fails, the way predictor adds an extra cycle in which it accesses all the tag subarrays. Assume the way predictor miss is followed by a cache read hit.

g. [15] <B.1> Repeat part (f) assuming a cache read miss.

h. [20] <B.1> Use parts (e), (f), and (g) for the general case where the workload has the following statistics: way predictor miss rate = 5% and cache miss rate = 3%. (Consider different replacement policies.)

Estimate the memory system (cache + memory) power usage (in power units) for the following configurations. We assume the cache is four-way set associative. Provide answers for the LRU, FIFO, and random replacement policies.

B.4 [10/10/15/15/15/20] <B.1> We compare the write bandwidth requirements of write-through versus write-back caches using a concrete example. Let us assume that we have a 64 KB cache with a line size of 32 bytes. The cache will allocate a line on a write miss. If configured as a write-back cache, it will write back all of the dirty line if it needs to be replaced. We will also assume that the cache is connected to the lower level in the hierarchy through a 64-bit-wide (8-byte-wide) bus. The number of CPU cycles for a B-bytes write access on this bus is

$10 + 5\left\lceil \frac{B}{8} - 1 \right\rceil$, where the square brackets represent the "ceiling" function. For example, an 8-byte write would take

$10 + 5\left\lceil \frac{8}{8} - 1 \right\rceil = 10$ cycles, whereas using the same formula a 12-byte write would take 15 cycles.

Answer the following questions while referring to the C code snippet below:

```
... #define PORTION 1
    ...
    base = 8*i;
    for (unsigned int j = base; j < base + PORTION; j++)
//assume j is stored in a register
            {
                    data[j] = j;
            }
```

a. [10] <B.1> For a write-through cache, how many CPU cycles are spent on write transfers to the memory for all the combined iterations of the j loop?

b. [10] <B.1> If the cache is configured as a write-back cache, how many CPU cycles are spent on writing back a cache line?

c. [15] <B.1> Change PORTION to 8 and repeat part (a).

d. [15] <B.1> What is the minimum number of array updates to the same cache line (before replacing it) that would render the write-back cache superior?

e. [15] <B.1> Think of a scenario where all the words of the cache line will be written (not necessarily using the above code) and a write-through cache will require fewer total CPU cycles than the write-back cache.

B.5 [10/10/10/10/] <B.2> You are building a system around a processor with in-order execution that runs at 1.1 GHz and has a CPI of 1.35 excluding memory accesses. The only instructions that read or write data from memory are loads (20% of all instructions) and stores (10% of all instructions). The memory system for this computer is composed of a split L1 cache that imposes no penalty on hits. Both the I-cache and D-cache are direct-mapped and hold 32 KB each. The I-cache has a 2% miss rate and 32-byte blocks, and the D-cache is write-through with a 5% miss rate and 16-byte blocks. There is a write buffer on the D-cache that eliminates stalls for 95% of all writes. The 512 KB write-back, unified L2 cache has 64-byte blocks and an access time of 15 ns. It is connected to the L1 cache by a 128-bit data bus that runs at 266 MHz and can transfer one 128-bit word per bus cycle. Of all memory references sent to the L2 cache in this system, 80% are satisfied without going to main memory. Also, 50% of all blocks replaced are dirty. The 128-bit-wide main memory has an access latency of 60 ns, after which any number of bus words may be transferred at the rate of one per cycle on the 128-bit-wide 133 MHz main memory bus.

a. [10] <B.2> What is the average memory access time for instruction accesses?

b. [10] <B.2> What is the average memory access time for data reads?

c. [10] <B.2> What is the average memory access time for data writes?

d. [10] <B.2> What is the overall CPI, including memory accesses?

B.6 [10/15/15] <B.2> Converting miss rate (misses per reference) into misses per instruction relies upon two factors: references per instruction fetched and the fraction of fetched instructions that actually commits.

a. [10] <B.2> The formula for misses per instruction on page B-5 is written first in terms of three factors: miss rate, memory accesses, and instruction count. Each of these factors represents actual events. What is different about writing misses per instruction as *miss rate* times the factor *memory accesses per instruction*?

b. [15] <B.2> Speculative processors will fetch instructions that do not commit. The formula for misses per instruction on page B-5 refers to misses per instruction on the execution path; that is, only the instructions that must actually be executed to carry out the program. Convert the formula for misses per instruction on page B-5 into one that uses only miss rate, references per instruction fetched, and fraction of fetched instructions that commit. Why rely upon these factors rather than those in the formula on page B-5?

c. [15] <B.2> The conversion in part (b) could yield an incorrect value to the extent that the value of the factor references per instruction fetched is not equal to the number of references for any particular instruction. Rewrite the formula of part (b) to correct this deficiency.

B.7 [20] <B.1, B.3> In systems with a write-through L1 cache backed by a write-back L2 cache instead of main memory, a merging write buffer can be simplified. Explain how this can be done. Are there situations where having a full write buffer (instead of the simple version you have just proposed) could be helpful?

B.8 [5/5/5] <B.3> We want to observe the following calculation

$$d_i = a_i + b_i * c_i, \qquad i : (0 : 511)$$

Arrays *a*, *b*, *c*, and *d* memory layout is displayed below (each has 512 4-byte-wide integer elements).

The above calculation employs a for loop that runs through 512 iterations.

Assume a 32 Kbyte 4-way set associative cache with a single cycle access time. The miss penalty is 100 CPU cycles/access, and so is the cost of a write-back. The cache is a write-back on hits write-allocate on misses cache (Figure B.32).

a. [5] <B3> How many cycles will an iteration take if all three loads and single store miss in the data cache?

b. [5] <B3> If the cache line size is 16 bytes, what is the average number of cycles an average iteration will take? (Hint: Spatial locality!)

c. [5] <B3> If the cache line size is 64 bytes, what is the average number of cycles an average iteration will take?

d. If the cache is direct-mapped and its size is reduced to 2048 bytes, what is the average number of cycles an average iteration will take?

B.9 [20] <B.3> Increasing a cache's associativity (with all other parameters kept constant) statistically reduces the miss rate. However, there can be pathological cases where increasing a cache's associativity would increase the miss rate for a particular workload.

Consider the case of direct-mapped compared to a two-way set associative cache of equal size. Assume that the set associative cache uses the LRU replacement policy. To simplify, assume that the block size is one word. Now, construct a trace of word accesses that would produce more misses in the two-way associative cache.

(Hint: Focus on constructing a trace of accesses that are exclusively directed to a single set of the two-way set associative cache, such that the same trace would exclusively access two blocks in the direct-mapped cache.)

B.10 [10/10/15] <B.3> Consider a two-level memory hierarchy made of L1 and L2 data caches. Assume that both caches use write-back policy on write hit and both have the same block size. List the actions taken in response to the following events:

a. [10] <B.3> An L1 cache miss when the caches are organized in an inclusive hierarchy.

Mem. address in bytes	Contents
0–2047	Array *a*
2048–4095	Array *b*
4096–6143	Array *c*
6144–8191	Array *d*

Figure B.32 Arrays layout in memory.

b. [10] <B.3> An L1 cache miss when the caches are organized in an exclusive hierarchy.

c. [15] <B.3> In both parts (a) and (b), consider the possibility that the evicted line might be clean or dirty.

B.11 [15/20] <B.2, B.3> excluding some instructions from entering the cache can reduce conflict misses.

a. [15] <B.3> Sketch a program hierarchy where parts of the program would be better excluded from entering the instruction cache. (Hint: Consider a program with code blocks that are placed in deeper loop nests than other blocks.)

b. [20] <B.2, B.3> Suggest software or hardware techniques to enforce exclusion of certain blocks from the instruction cache.

B.12 [5/15] <B.3> Whereas larger caches have lower miss rates, they also tend to have longer hit times.

Assume a direct-mapped 8 KB cache has 0.22 ns hit time and miss rate m1; also assume a 4-way associative 64 KB cache has 0.52 ns hit time and a miss rate m2.

a. [5] <B.3> If the miss penalty is 100 ns, when would it be advantageous to use the smaller cache to reduce the overall memory access time?

b. [15] <B.3> Repeat part (a) for miss penalties of 10 and 1000 cycles. Conclude when it might be advantageous to use a smaller cache.

B.13 [15] <B.4> A program is running on a computer with a four-entry fully associative (micro) translation lookaside buffer (TLB) (Figure B.33):

The following is a trace of virtual page numbers accessed by a program. For each access indicate whether it produces a TLB hit/miss and, if it accesses the page table, whether it produces a page hit or fault. Put an X under the page table column if it is not accessed (Figures B.34 and B.35).

B.14 [15/15/15/15/] <B.4> Some memory systems handle TLB misses in software (as an exception), while others use hardware for TLB misses.

a. [15] <B.4> What are the trade-offs between these two methods for handling TLB misses?

b. [15] <B.4> Will TLB miss handling in software always be slower than TLB miss handling in hardware? Explain.

VP#	PP#	Entry valid
5	30	1
7	1	0
10	10	1
15	25	1

Figure B.33 TLB contents (problem B.12).

Virtual page index	Physical page #	Present
0	3	Y
1	7	N
2	6	N
3	5	Y
4	14	Y
5	30	Y
6	26	Y
7	11	Y
8	13	N
9	18	N
10	10	Y
11	56	Y
12	110	Y
13	33	Y
14	12	N
15	25	Y

Figure B.34 Page table contents.

Virtual page accessed	TLB (hit or miss)	Page table (hit or fault)
1		
5		
9		
14		
10		
6		
15		
12		
7		
2		

Figure B.35 Page access trace.

c. [15] <B.4> Are there page table structures that would be difficult to handle in hardware but possible in software? Are there any such structures that would be difficult for software to handle but easy for hardware to manage?

d. [15] <B.4> Why are TLB miss rates for floating-point programs generally higher than those for integer programs?

B.15 [20/20] <B.5> It is possible to provide more flexible protection than that in the Intel Pentium architecture by using a protection scheme similar to that used in the Hewlett-Packard Precision Architecture (HP/PA). In such a scheme, each page table entry contains a "protection ID" (key) along with access rights for the page. On each reference, the CPU compares the protection ID in the page table entry with those stored in each of four protection ID registers (access to these registers requires that the CPU be in supervisor mode). If there is no match for the protection ID in the page table entry or if the access is not a permitted access (writing to a read-only page, for example), an exception is generated.

a. [20] <B.5> Explain how this model could be used to facilitate the construction of operating systems from relatively small pieces of code that cannot overwrite each other (microkernels). What advantages might such an operating system have over a monolithic operating system in which any code in the OS can write to any memory location?

b. [20] <B.5> A simple design change to this system would allow two protection IDs for each page table entry, one for read access and the other for either write or execute access (the field is unused if neither the writable nor executable bit is set). What advantages might there be from having different protection IDs for read and write capabilities? (*Hint*: Could this make it easier to share data and code between processes?)

C.1	Introduction	C-2
C.2	The Major Hurdle of Pipelining—Pipeline Hazards	C-10
C.3	How Is Pipelining Implemented?	C-26
C.4	What Makes Pipelining Hard to Implement?	C-37
C.5	Extending the RISC V Integer Pipeline to Handle Multicycle Operations	C-45
C.6	Putting It All Together: The MIPS R4000 Pipeline	C-55
C.7	Cross-Cutting Issues	C-65
C.8	Fallacies and Pitfalls	C-70
C.9	Concluding Remarks	C-71
C.10	Historical Perspective and References	C-71
	Updated Exercises by Diana Franklin	C-71

C

Pipelining: Basic and Intermediate Concepts

It is quite a three-pipe problem.

Sir Arthur Conan Doyle,
The Adventures of Sherlock Holmes

C.1 Introduction

Many readers of this text will have covered the basics of pipelining in another text (such as our more basic text *Computer Organization and Design*) or in another course. Because Chapter 3 builds heavily on this material, readers should ensure that they are familiar with the concepts discussed in this appendix before proceeding. As you read Chapter 3, you may find it helpful to turn to this material for a quick review.

We begin the appendix with the basics of pipelining, including discussing the data path implications, introducing hazards, and examining the performance of pipelines. This section describes the basic five-stage RISC pipeline that is the basis for the rest of the appendix. Section C.2 describes the issue of hazards, why they cause performance problems, and how they can be dealt with. Section C.3 discusses how the simple five-stage pipeline is actually implemented, focusing on control and how hazards are dealt with.

Section C.4 discusses the interaction between pipelining and various aspects of instruction set design, including discussing the important topic of exceptions and their interaction with pipelining. Readers unfamiliar with the concepts of precise and imprecise interrupts and resumption after exceptions will find this material useful, because they are key to understanding the more advanced approaches in Chapter 3.

Section C.5 discusses how the five-stage pipeline can be extended to handle longer-running floating-point instructions. Section C.6 puts these concepts together in a case study of a deeply pipelined processor, the MIPS R4000/4400, including both the eight-stage integer pipeline and the floating-point pipeline. The MIPS R40000 is similar to a single-issue embedded processor, such as the ARM Cortex-A5, which became available in 2010, and was used in several smart phones and tablets.

Section C.7 introduces the concept of dynamic scheduling and the use of scoreboards to implement dynamic scheduling. It is introduced as a cross-cutting issue, because it can be used to serve as an introduction to the core concepts in Chapter 3, which focused on dynamically scheduled approaches. Section C.7 is also a gentle introduction to the more complex Tomasulo's algorithm covered in Chapter 3. Although Tomasulo's algorithm can be covered and understood without introducing scoreboarding, the scoreboarding approach is simpler and easier to comprehend.

What Is Pipelining?

Pipelining is an implementation technique whereby multiple instructions are overlapped in execution; it takes advantage of parallelism that exists among the actions needed to execute an instruction. Today, pipelining is the key implementation technique used to make fast processors, and even processors that cost less than a dollar are pipelined.

A pipeline is like an assembly line. In an automobile assembly line, there are many steps, each contributing something to the construction of the car. Each step operates in parallel with the other steps, although on a different car. In a computer pipeline, each step in the pipeline completes a part of an instruction. Like the assembly line, different steps are completing different parts of different instructions in parallel. Each of these steps is called a *pipe stage* or a *pipe segment*. The stages are connected one to the next to form a pipe—instructions enter at one end, progress through the stages, and exit at the other end, just as cars would in an assembly line.

In an automobile assembly line, *throughput* is defined as the number of cars per hour and is determined by how often a completed car exits the assembly line. Likewise, the throughput of an instruction pipeline is determined by how often an instruction exits the pipeline. Because the pipe stages are hooked together, all the stages must be ready to proceed at the same time, just as we would require in an assembly line. The time required between moving an instruction one step down the pipeline is a *processor cycle*. Because all stages proceed at the same time, the length of a processor cycle is determined by the time required for the slowest pipe stage, just as in an auto assembly line the longest step would determine the time between advancing cars in the line. In a computer, this processor cycle is almost always 1 clock cycle.

The pipeline designer's goal is to balance the length of each pipeline stage, just as the designer of the assembly line tries to balance the time for each step in the process. If the stages are perfectly balanced, then the time per instruction on the pipelined processor—assuming ideal conditions—is equal to

$$\frac{\text{Time per instruction on unpipelined machine}}{\text{Number of pipe stages}}$$

Under these conditions, the speedup from pipelining equals the number of pipe stages, just as an assembly line with n stages can ideally produce cars n times as fast. Usually, however, the stages will not be perfectly balanced; furthermore, pipelining does involve some overhead. Thus, the time per instruction on the pipelined processor will not have its minimum possible value, yet it can be close.

Pipelining yields a reduction in the average execution time per instruction. If the starting point is a processor that takes multiple clock cycles per instruction, then pipelining reduces the CPI. This is the primary view we will take.

Pipelining is an implementation technique that exploits parallelism among the instructions in a sequential instruction stream. It has the substantial advantage that, unlike some speedup techniques (see Chapter 4), it is not visible to the programmer.

The Basics of the RISC V Instruction Set

Throughout this book we use RISC V, a load-store architecture, to illustrate the basic concepts. Nearly all the ideas we introduce in this book are applicable to other

processors, but the implementation may be much more complicated with complex instructions. In this section, we make use of the core of the RISC V architecture; see Chapter 1 for a full description. Although we use RISC V, the concepts are significantly similar in that they will apply to any RISC, including the core architectures of ARM and MIPS. All RISC architectures are characterized by a few key properties:

- All operations on data apply to data in registers and typically change the entire register (32 or 64 bits per register).
- The only operations that affect memory are load and store operations that move data from memory to a register or to memory from a register, respectively. Load and store operations that load or store less than a full register (e.g., a byte, 16 bits, or 32 bits) are often available.
- The instruction formats are few in number, with all instructions typically being one size. In RISC V, the register specifiers: rs1, rs2, and rd are always in the same place simplifying the control.

These simple properties lead to dramatic simplifications in the implementation of pipelining, which is why these instruction sets were designed this way. Chapter 1 contains a full description of the RISC V ISA, and we assume the reader has read Chapter 1.

A Simple Implementation of a RISC Instruction Set

To understand how a RISC instruction set can be implemented in a pipelined fashion, we need to understand how it is implemented *without* pipelining. This section shows a simple implementation where every instruction takes at most 5 clock cycles. We will extend this basic implementation to a pipelined version, resulting in a much lower CPI. Our unpipelined implementation is not the most economical or the highest-performance implementation without pipelining. Instead, it is designed to lead naturally to a pipelined implementation. Implementing the instruction set requires the introduction of several temporary registers that are not part of the architecture; these are introduced in this section to simplify pipelining. Our implementation will focus only on a pipeline for an integer subset of a RISC architecture that consists of load-store word, branch, and integer ALU operations.

Every instruction in this RISC subset can be implemented in, at most, 5 clock cycles. The 5 clock cycles are as follows.

1. *Instruction fetch cycle* (IF):

   Send the program counter (PC) to memory and fetch the current instruction from memory. Update the PC to the next sequential instruction by adding 4 (because each instruction is 4 bytes) to the PC.

2. *Instruction decode/register fetch cycle* (ID):

   Decode the instruction and read the registers corresponding to register source specifiers from the register file. Do the equality test on the registers as they are read, for a possible branch. Sign-extend the offset field of the instruction in case it is needed. Compute the possible branch target address by adding the sign-extended offset to the incremented PC.

   Decoding is done in parallel with reading registers, which is possible because the register specifiers are at a fixed location in a RISC architecture. This technique is known as *fixed-field decoding*. Note that we may read a register we don't use, which doesn't help but also doesn't hurt performance. (It does waste energy to read an unneeded register, and power-sensitive designs might avoid this.) For loads and ALU immediate operations, the immediate field is always in the same place, so we can easily sign extend it. (For a more complete implementation of RISC V, we would need to compute two different sign-extended values, because the immediate field for store is in a different location.)

3. *Execution/effective address cycle* (EX):

   The ALU operates on the operands prepared in the prior cycle, performing one of three functions, depending on the instruction type.

   - Memory reference—The ALU adds the base register and the offset to form the effective address.
   - Register-Register ALU instruction—The ALU performs the operation specified by the ALU opcode on the values read from the register file.
   - Register-Immediate ALU instruction—The ALU performs the operation specified by the ALU opcode on the first value read from the register file and the sign-extended immediate.
   - Conditional branch—Determine whether the condition is true.

   In a load-store architecture the effective address and execution cycles can be combined into a single clock cycle, because no instruction needs to simultaneously calculate a data address and perform an operation on the data.

4. *Memory access* (MEM):

   If the instruction is a load, the memory does a read using the effective address computed in the previous cycle. If it is a store, then the memory writes the data from the second register read from the register file using the effective address.

5. *Write-back cycle* (WB):

   - Register-Register ALU instruction or load instruction:

   Write the result into the register file, whether it comes from the memory system (for a load) or from the ALU (for an ALU instruction).

In this implementation, branch instructions require three cycles, store instructions require four cycles, and all other instructions require five cycles. Assuming a

branch frequency of 12% and a store frequency of 10%, a typical instruction distribution leads to an overall CPI of 4.66. This implementation, however, is not optimal either in achieving the best performance or in using the minimal amount of hardware given the performance level; we leave the improvement of this design as an exercise for you and instead focus on pipelining this version.

The Classic Five-Stage Pipeline for a RISC Processor

We can pipeline the execution described in the previous section with almost no changes by simply starting a new instruction on each clock cycle. (See why we chose this design?) Each of the clock cycles from the previous section becomes a *pipe stage*—a cycle in the pipeline. This results in the execution pattern shown in Figure C.1, which is the typical way a pipeline structure is drawn. Although each instruction takes 5 clock cycles to complete, during each clock cycle the hardware will initiate a new instruction and will be executing some part of the five different instructions.

You may find it hard to believe that pipelining is as simple as this; it's not. In this and the following sections, we will make our RISC pipeline "real" by dealing with problems that pipelining introduces.

To start with, we have to determine what happens on every clock cycle of the processor and make sure we don't try to perform two different operations with the same data path resource on the same clock cycle. For example, a single ALU cannot be asked to compute an effective address and perform a subtract operation at the same time. Thus, we must ensure that the overlap of instructions in the pipeline cannot cause such a conflict. Fortunately, the simplicity of a RISC instruction set makes resource evaluation relatively easy. Figure C.2 shows a simplified version of a RISC data path drawn in pipeline fashion. As you can see, the major functional units are used in different cycles, and hence overlapping the execution of multiple

	Clock number								
Instruction number	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
Instruction i	IF	ID	EX	MEM	WB				
Instruction $i+1$		IF	ID	EX	MEM	WB			
Instruction $i+2$			IF	ID	EX	MEM	WB		
Instruction $i+3$				IF	ID	EX	MEM	WB	
Instruction $i+4$					IF	ID	EX	MEM	WB

Figure C.1 Simple RISC pipeline. On each clock cycle, another instruction is fetched and begins its five-cycle execution. If an instruction is started every clock cycle, the performance will be up to five times that of a processor that is not pipelined. The names for the stages in the pipeline are the same as those used for the cycles in the unpipelined implementation: IF = instruction fetch, ID = instruction decode, EX = execution, MEM = memory access, and WB = write-back.

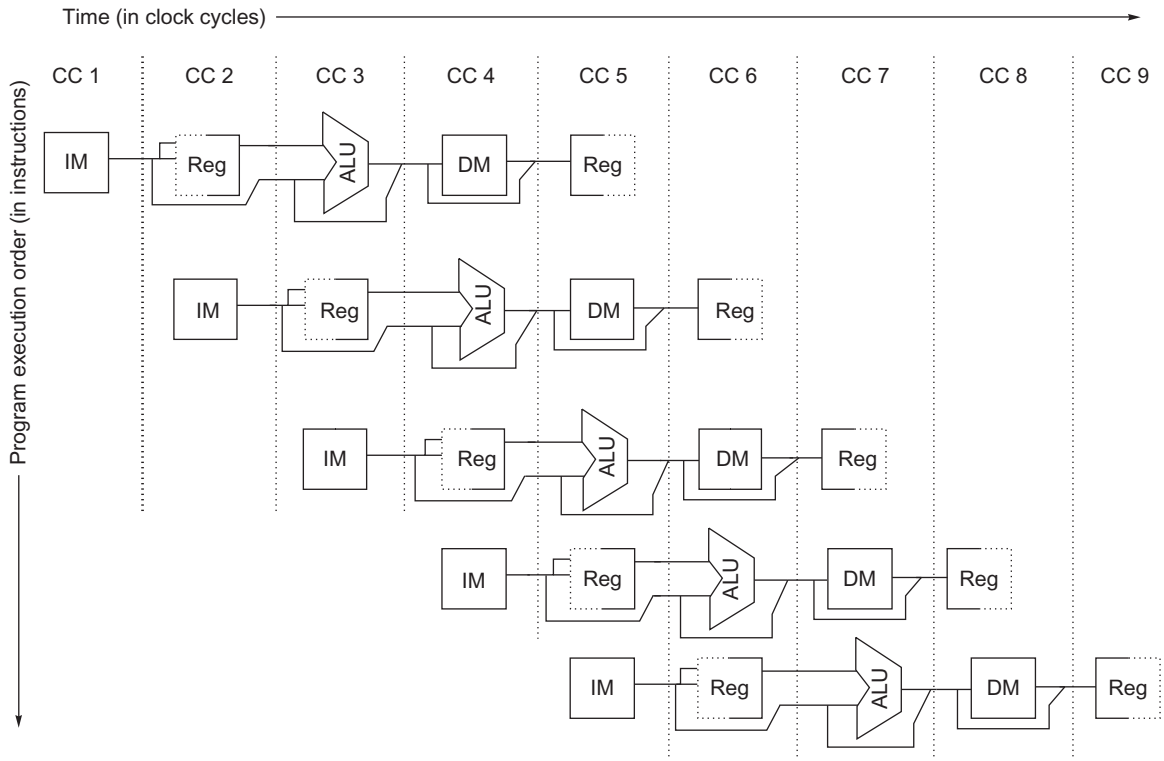


Figure C.2 The pipeline can be thought of as a series of data paths shifted in time. This figure shows the overlap among the parts of the data path, with clock cycle 5 (CC 5) showing the steady-state situation. Because the register file is used as a source in the ID stage and as a destination in the WB stage, it appears twice. We show that it is read in one part of the stage and written in another by using a solid line, on the right or left, respectively, and a dashed line on the other side. The abbreviation IM is used for instruction memory, DM for data memory, and CC for clock cycle.

instructions introduces relatively few conflicts. There are three observations on which this fact rests.

First, we use separate instruction and data memories, which we would typically implement with separate instruction and data caches (discussed in Chapter 2). The use of separate caches eliminates a conflict for a single memory that would arise between instruction fetch and data memory access. Notice that if our pipelined processor has a clock cycle that is equal to that of the unpipelined version, the memory system must deliver five times the bandwidth. This increased demand is one cost of higher performance.

Second, the register file is used in the two stages: one for reading in ID and one for writing in WB. These uses are distinct, so we simply show the register file in two places. Hence, we need to perform two reads and one write every clock cycle.

To handle reads and a write to the same register (and for another reason, which will become obvious shortly), we perform the register write in the first half of the clock cycle and the read in the second half.

Third, Figure C.2 does not deal with the PC. To start a new instruction every clock, we must increment and store the PC every clock, and this must be done during the IF stage in preparation for the next instruction. Furthermore, we must also have an adder to compute the potential branch target address during ID. One further problem is that we need the ALU in the ALU stage to evaluate the branch condition. Actually, we don't really need a full ALU to evaluate the comparison between two registers, but we need enough of the function that it has to occur in this pipestage.

Although it is critical to ensure that instructions in the pipeline do not attempt to use the hardware resources at the same time, we must also ensure that instructions in different stages of the pipeline do not interfere with one another. This separation is done by introducing *pipeline registers* between successive stages of the pipeline, so that at the end of a clock cycle all the results from a given stage are stored into a register that is used as the input to the next stage on the next clock cycle. Figure C.3 shows the pipeline drawn with these pipeline registers.

Although many figures will omit such registers for simplicity, they are required to make the pipeline operate properly and must be present. Of course, similar registers would be needed even in a multicycle data path that had no pipelining (because only values in registers are preserved across clock boundaries). In the case of a pipelined processor, the pipeline registers also play the key role of carrying intermediate results from one stage to another where the source and destination may not be directly adjacent. For example, the register value to be stored during a store instruction is read during ID, but not actually used until MEM; it is passed through two pipeline registers to reach the data memory during the MEM stage. Likewise, the result of an ALU instruction is computed during EX, but not actually stored until WB; it arrives there by passing through two pipeline registers. It is sometimes useful to name the pipeline registers, and we follow the convention of naming them by the pipeline stages they connect, so the registers are called IF/ID, ID/EX, EX/MEM, and MEM/WB.

Basic Performance Issues in Pipelining

Pipelining increases the processor instruction throughput—the number of instructions completed per unit of time—but it does not reduce the execution time of an individual instruction. In fact, it usually slightly increases the execution time of each instruction due to overhead in the control of the pipeline. The increase in instruction throughput means that a program runs faster and has lower total execution time, even though no single instruction runs faster!

The fact that the execution time of each instruction does not decrease puts limits on the practical depth of a pipeline, as we will see in the next section. In addition to limitations arising from pipeline latency, limits arise from imbalance

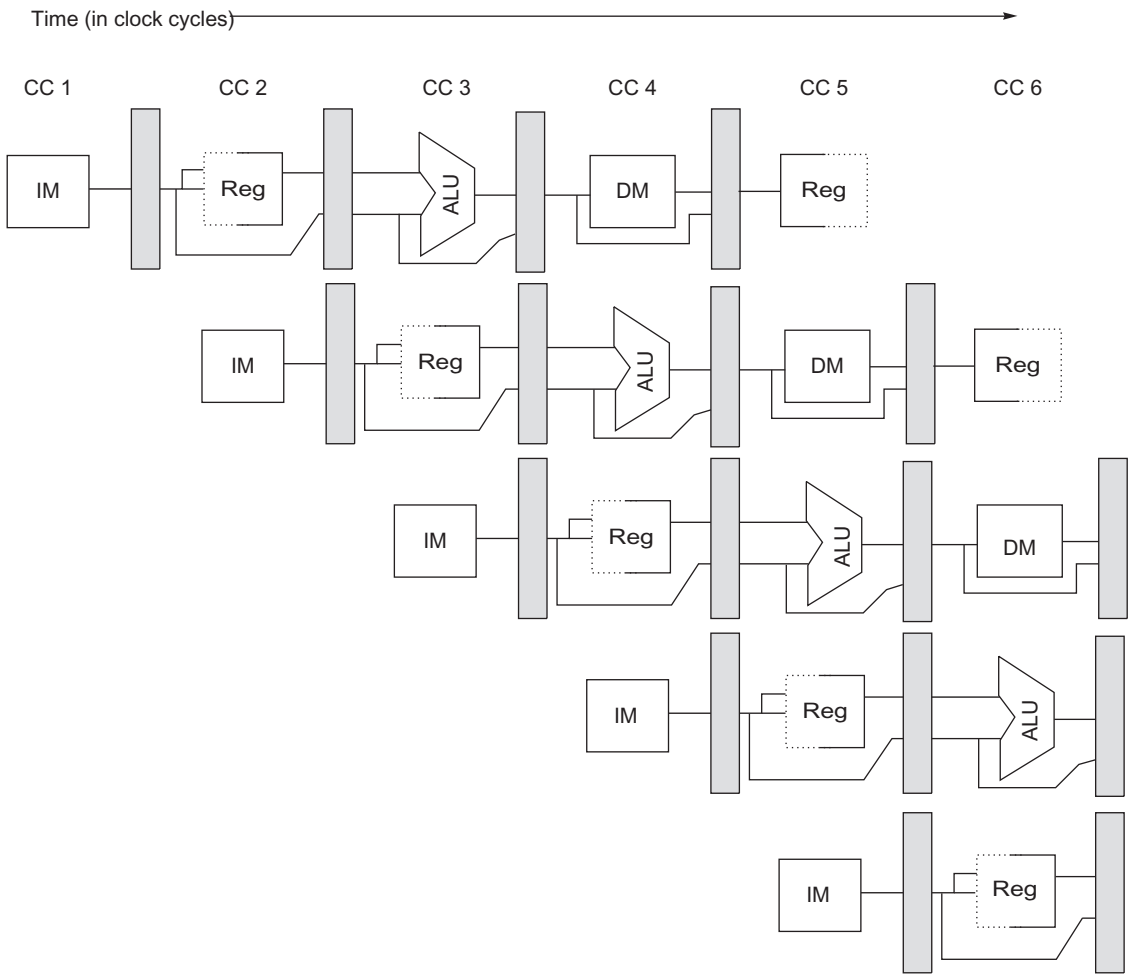


Figure C.3 A pipeline showing the pipeline registers between successive pipeline stages. Notice that the registers prevent interference between two different instructions in adjacent stages in the pipeline. The registers also play the critical role of carrying data for a given instruction from one stage to the other. The edge-triggered property of registers—that is, that the values change instantaneously on a clock edge—is critical. Otherwise, the data from one instruction could interfere with the execution of another!

among the pipe stages and from pipelining overhead. Imbalance among the pipe stages reduces performance because the clock can run no faster than the time needed for the slowest pipeline stage. Pipeline overhead arises from the combination of pipeline register delay and clock skew. The pipeline registers add setup time, which is the time that a register input must be stable before the clock signal that triggers a write occurs, plus propagation delay to the clock cycle. Clock skew, which is the maximum delay between when the clock arrives at any two registers,

also contributes to the lower limit on the clock cycle. Once the clock cycle is as small as the sum of the clock skew and latch overhead, no further pipelining is useful, because there is no time left in the cycle for useful work. The interested reader should see Kunkel and Smith (1986).

Example Consider the unpipelined processor in the previous section. Assume that it has a 4 GHz clock (or a 0.5 ns clock cycle) and that it uses four cycles for ALU operations and branches and five cycles for memory operations. Assume that the relative frequencies of these operations are 40%, 20%, and 40%, respectively. Suppose that due to clock skew and setup, pipelining the processor adds 0.1 ns of overhead to the clock. Ignoring any latency impact, how much speedup in the instruction execution rate will we gain from a pipeline?

Answer The average instruction execution time on the unpipelined processor is

$$\begin{aligned}\text{Average instruction execution time} &= \text{Clock cycle} \times \text{Average CPI}\\ &= 0.5\,\text{ns} \times [(40\% + 20\%) \times 4 + 40\% \times 5]\\ &= 0.5\ \text{ns} \times 4.4\\ &= 2.2\ \text{ns}\end{aligned}$$

In the pipelined implementation, the clock must run at the speed of the slowest stage plus overhead, which will be 0.5 + 0.1 or 0.6 ns; this is the average instruction execution time. Thus, the speedup from pipelining is

$$\begin{aligned}\text{Speedup from pipelining} &= \frac{\text{Average instruction time unpipelined}}{\text{Average instruction time pipelined}}\\ &= \frac{2.2\ \text{ns}}{0.6\ \text{ns}} = 3.7\ \text{times}\end{aligned}$$

The 0.1 ns overhead essentially establishes a limit on the effectiveness of pipelining. If the overhead is not affected by changes in the clock cycle, Amdahl's Law tells us that the overhead limits the speedup.

This simple RISC pipeline would function just fine for integer instructions if every instruction were independent of every other instruction in the pipeline. In reality, instructions in the pipeline can depend on one another; this is the topic of the next section.

C.2 The Major Hurdle of Pipelining—Pipeline Hazards

There are situations, called *hazards*, that prevent the next instruction in the instruction stream from executing during its designated clock cycle. Hazards reduce the performance from the ideal speedup gained by pipelining. There are three classes of hazards:

1. *Structural hazards* arise from resource conflicts when the hardware cannot support all possible combinations of instructions simultaneously in overlapped execution. In modern processors, structural hazards occur primarily in special purpose functional units that are less frequently used (such as floating point divide or other complex long running instructions). They are not a major performance factor, assuming programmers and compiler writers are aware of the lower throughput of these instructions. Instead of spending more time on this infrequent case, we focus on the two other hazards that are much more frequent.
2. *Data hazards* arise when an instruction depends on the results of a previous instruction in a way that is exposed by the overlapping of instructions in the pipeline.
3. *Control hazards* arise from the pipelining of branches and other instructions that change the PC.

Hazards in pipelines can make it necessary to *stall* the pipeline. Avoiding a hazard often requires that some instructions in the pipeline be allowed to proceed while others are delayed. For the pipelines we discuss in this appendix, when an instruction is stalled, all instructions issued *later* than the stalled instruction—and hence not as far along in the pipeline—are also stalled. Instructions issued *earlier* than the stalled instruction—and hence farther along in the pipeline—must continue, because otherwise the hazard will never clear. As a result, no new instructions are fetched during the stall. We will see several examples of how pipeline stalls operate in this section—don't worry, they aren't as complex as they might sound!

Performance of Pipelines With Stalls

A stall causes the pipeline performance to degrade from the ideal performance. Let's look at a simple equation for finding the actual speedup from pipelining, starting with the formula from the previous section:

$$\begin{aligned}\text{Speedup from pipelining} &= \frac{\text{Average instruction time unpipelined}}{\text{Average instruction time pipelined}}\\ &= \frac{\text{CPI unpipelined} \times \text{Clock cycle unpipelined}}{\text{CPI pipelined} \times \text{Clock cycle pipelined}}\\ &= \frac{\text{CPI unpipelined} \times \text{Clock cycle unpipelined}}{\text{CPI pipelined} \times \text{Clock cycle pipelined}}\end{aligned}$$

Pipelining can be thought of as decreasing the CPI or the clock cycle time. Because it is traditional to use the CPI to compare pipelines, let's start with that assumption. The ideal CPI on a pipelined processor is almost always 1. Hence, we can compute the pipelined CPI:

$$\begin{aligned}\text{CPI pipelined} &= \text{Ideal CPI} + \text{Pipeline stall clock cycles per instruction}\\ &= 1 + \text{Pipelines stall clock cycles per instruction}\end{aligned}$$

If we ignore the cycle time overhead of pipelining and assume that the stages are perfectly balanced, then the cycle time of the two processors can be equal, leading to

$$\text{Speedup} = \frac{\text{CPI unpiplined}}{1 + \text{Pipeline stall cycles per instruction}}$$

One important simple case is where all instructions take the same number of cycles, which must also equal the number of pipeline stages (also called the *depth of the pipeline*). In this case, the unpipelined CPI is equal to the depth of the pipeline, leading to

$$\text{Speedup} = \frac{\text{Pipeline depth}}{1 + \text{Pipeline stall cycles per instruction}}$$

If there are no pipeline stalls, this leads to the intuitive result that pipelining can improve performance by the depth of the pipeline.

Data Hazards

A major effect of pipelining is to change the relative timing of instructions by overlapping their execution. This overlap introduces data and control hazards. Data hazards occur when the pipeline changes the order of read/write accesses to operands so that the order differs from the order seen by sequentially executing instructions on an unpipelined processor. Assume instruction i occurs in program order before instruction j and both instructions use register x, then there are three different types of hazards that can occur between i and j:

1. Read After Write (RAW) hazard: the most common, these occur when a read of register x by instruction j occurs before the write of register x by instruction i. If this hazard were not prevented instruction j would use the wrong value of x.
2. Write After Read (WAR) hazard: this hazard occurs when read of register x by instruction i occurs after a write of register x by instruction j. In this case, instruction i would use the wrong value of x. WAR hazards are impossible in the simple five stage, integrer pipeline, but they occur when instructions are reordered, as we will see when we discuss dynamically scheduled pipelines beginning on page C.65.
3. Write After Write (WAW) hazard: this hazard occurs when write of register x by instruction i occurs after a write of register x by instruction j. When this occurs, register x will have the wrong value going forward. WAR hazards are also impossible in the simple five stage, integrer pipeline, but they occur when instructions are reordered or when running times vary, as we will see later.

Chapter 3 explores the issues of data dependence and hazards in much more detail. For now, we focus only on RAW hazards.

Consider the pipelined execution of these instructions:

```
add     x1,x2,x3
sub     x4,x1,x5
and     x6,x1,x7
or      x8,x1,x9
xor     x10,x1,x11
```

All the instructions after the `add` use the result of the `add` instruction. As shown in Figure C.4, the `add` instruction writes the value of x1 in the WB pipe stage, but the `sub` instruction reads the value during its ID stage, which results in a RAW hazard. Unless precautions are taken to prevent it, the `sub` instruction will read the wrong value and try to use it. In fact, the value used by the `sub` instruction is not even deterministic: though we might think it logical to assume that `sub` would always use the value of x1 that was assigned by an instruction prior to `add`, this is not

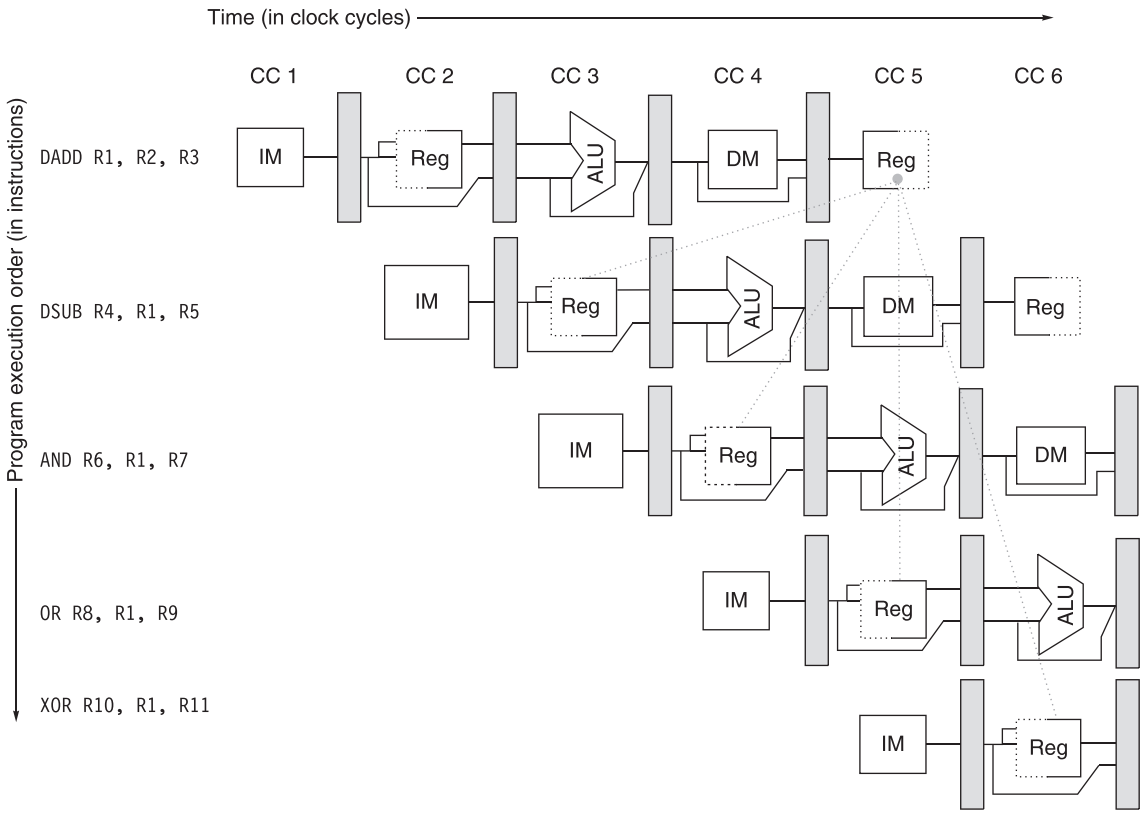


Figure C.4 The use of the result of the `add` instruction in the next three instructions causes a hazard, because the register is not written until after those instructions read it.

always the case. If an interrupt should occur between the `add` and `sub` instructions, the WB stage of the `add` will complete, and the value of x1 at that point will be the result of the `add`. This unpredictable behavior is obviously unacceptable.

The `and` instruction also creates a possible RAW hazard. As we can see from Figure C.4, the write of x1 does not complete until the end of clock cycle 5. Thus, the `and` instruction that reads the registers during clock cycle 4 will receive the wrong results.

The `xor` instruction operates properly because its register read occurs in clock cycle 6, after the register write. The `or` instruction also operates without incurring a hazard because we perform the register file reads in the second half of the cycle and the writes in the first half. Note that the `xor` instruction still depends on the `add`, but it no longer creates a hazard; a topic we explore in more detail in Chapter 3.

The next subsection discusses a technique to eliminate the stalls for the hazard involving the `sub` and `and` instructions.

Minimizing Data Hazard Stalls by Forwarding

The problem posed in Figure C.4 can be solved with a simple hardware technique called *forwarding* (also called *bypassing* and sometimes *short-circuiting*). The key insight in forwarding is that the result is not really needed by the `sub` until after the `add` actually produces it. If the result can be moved from the pipeline register where the `add` stores it to where the `sub` needs it, then the need for a stall can be avoided. Using this observation, forwarding works as follows:

1. The ALU result from both the EX/MEM and MEM/WB pipeline registers is always fed back to the ALU inputs.
2. If the forwarding hardware detects that the previous ALU operation has written the register corresponding to a source for the current ALU operation, control logic selects the forwarded result as the ALU input rather than the value read from the register file.

Notice that with forwarding, if the `sub` is stalled, the `add` will be completed and the bypass will not be activated. This relationship is also true for the case of an interrupt between the two instructions.

As the example in Figure C.4 shows, we need to forward results not only from the immediately previous instruction but also possibly from an instruction that started two cycles earlier. Figure C.5 shows our example with the bypass paths in place and highlighting the timing of the register read and writes. This code sequence can be executed without stalls.

Forwarding can be generalized to include passing a result directly to the functional unit that requires it: a result is forwarded from the pipeline register corresponding to the output of one unit to the input of another, rather than just from

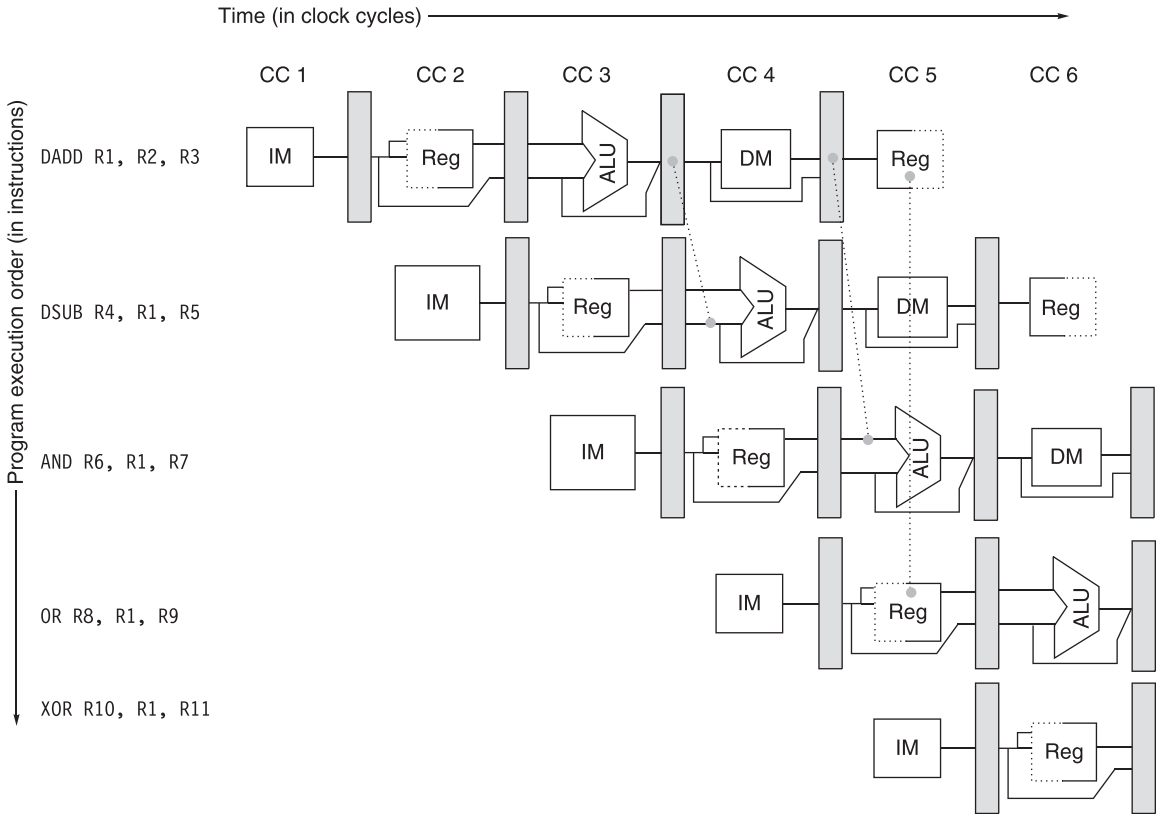


Figure C.5 A set of instructions that depends on the `add` result uses forwarding paths to avoid the data hazard. The inputs for the `sub` and `and` instructions forward from the pipeline registers to the first ALU input. The `or` receives its result by forwarding through the register file, which is easily accomplished by reading the registers in the second half of the cycle and writing in the first half, as the dashed lines on the registers indicate. Notice that the forwarded result can go to either ALU input; in fact, both ALU inputs could use forwarded inputs from either the same pipeline register or from different pipeline registers. This would occur, for example, if the `and` instruction was `and x6,x1,x4`.

the result of a unit to the input of the same unit. Take, for example, the following sequence:

```
add     x1,x2,x3
ld      x4,0(x1)
sd      x4,12(x1)
```

To prevent a stall in this sequence, we would need to forward the values of the ALU output and memory unit output from the pipeline registers to the ALU and data memory inputs. Figure C.6 shows all the forwarding paths for this example.

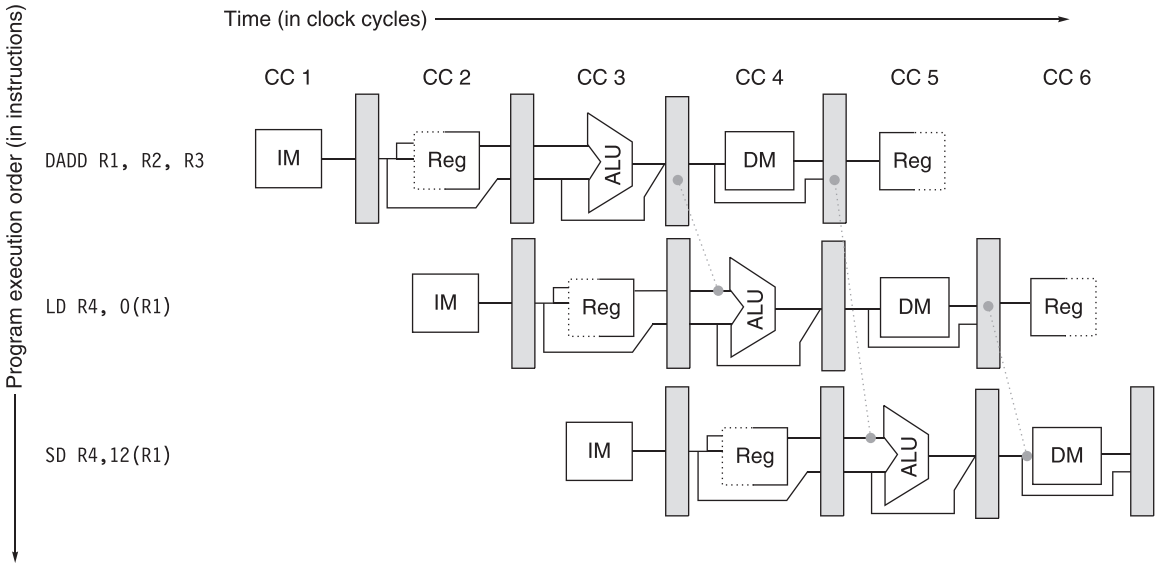


Figure C.6 Forwarding of operand required by stores during MEM. The result of the load is forwarded from the memory output to the memory input to be stored. In addition, the ALU output is forwarded to the ALU input for the address calculation of both the load and the store (this is no different than forwarding to another ALU operation). If the store depended on an immediately preceding ALU operation (not shown herein), the result would need to be forwarded to prevent a stall.

Data Hazards Requiring Stalls

Unfortunately, not all potential data hazards can be handled by bypassing. Consider the following sequence of instructions:

```
ld      x1,0(x2)
sub     x4,x1,x5
and     x6,x1,x7
or      x8,x1,x9
```

The pipelined data path with the bypass paths for this example is shown in Figure C.7. This case is different from the situation with back-to-back ALU operations. The `ld` instruction does not have the data until the end of clock cycle 4 (its MEM cycle), while the `sub` instruction needs to have the data by the beginning of that clock cycle. Thus, the data hazard from using the result of a load instruction cannot be completely eliminated with simple hardware. As Figure C.7 shows, such a forwarding path would have to operate backward in time—a capability not yet available to computer designers! We *can* forward the result immediately to the ALU from the pipeline registers for use in the `and` operation, which begins 2 clock cycles after the load. Likewise, the `or` instruction has no problem, because it receives the value through the register file. For the `sub` instruction, the forwarded

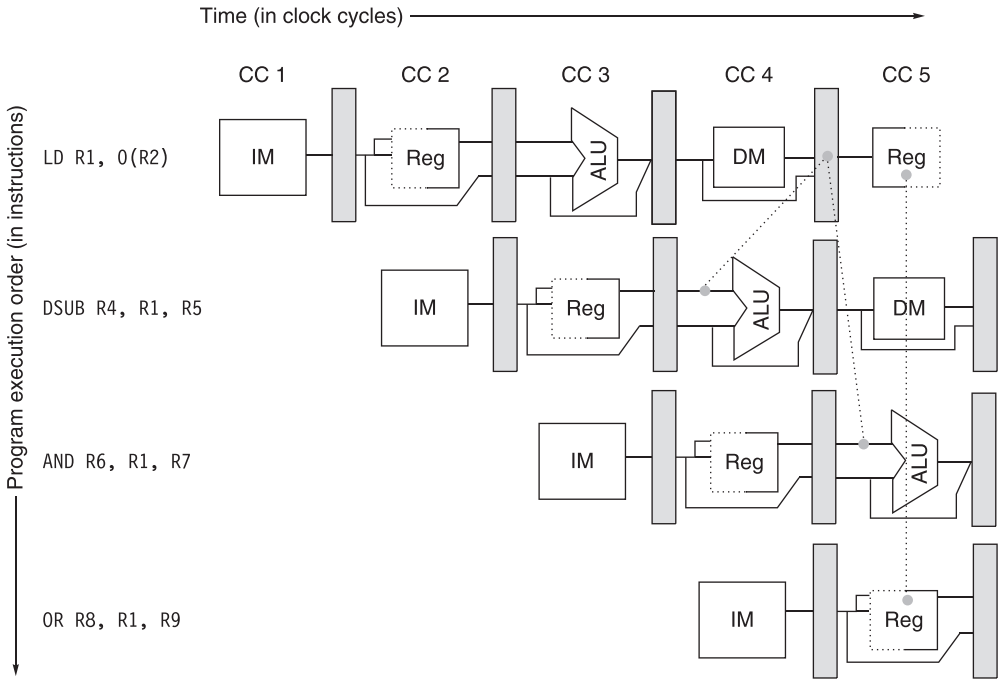


Figure C.7 The load instruction can bypass its results to the `and` and `or` instructions, but not to the `sub`, because that would mean forwarding the result in "negative time."

result arrives too late—at the end of a clock cycle, when it is needed at the beginning.

The load instruction has a delay or latency that cannot be eliminated by forwarding alone. Instead, we need to add hardware, called a *pipeline interlock*, to preserve the correct execution pattern. In general, a pipeline interlock detects a hazard and stalls the pipeline until the hazard is cleared. In this case, the interlock stalls the pipeline, beginning with the instruction that wants to use the data until the source instruction produces it. This pipeline interlock introduces a stall or bubble, just as it did for the structural hazard. The CPI for the stalled instruction increases by the length of the stall (1 clock cycle in this case).

Figure C.8 shows the pipeline before and after the stall using the names of the pipeline stages. Because the stall causes the instructions starting with the `sub` to move one cycle later in time, the forwarding to the `and` instruction now goes through the register file, and no forwarding at all is needed for the `or` instruction. The insertion of the bubble causes the number of cycles to complete this sequence to grow by one. No instruction is started during clock cycle 4 (and none finishes during cycle 6).

`ld x1,0(x2)`	IF	ID	EX	MEM	WB				
`sub x4,x1,x5`		IF	ID	EX	MEM	WB			
`and x6,x1,x7`			IF	ID	EX	MEM	WB		
`or x8,x1,x9`				IF	ID	EX	MEM	WB	
`ld x1,0(x2)`	IF	ID	EX	MEM	WB				
`sub x4,x1,x5`		IF	ID	Stall	EX	MEM	WB		
`and x6,x1,x7`			IF	Stall	ID	EX	MEM	WB	
`or x8,x1,x9`				Stall	IF	ID	EX	MEM	WB

Figure C.8 In the top half, we can see why a stall is needed: the MEM cycle of the load produces a value that is needed in the EX cycle of the `sub`, which occurs at the same time. This problem is solved by inserting a stall, as shown in the bottom half.

Branch Hazards

Control hazards can cause a greater performance loss for our RISC V pipeline than do data hazards. When a branch is executed, it may or may not change the PC to something other than its current value plus 4. Recall that if a branch changes the PC to its target address, it is a *taken* branch; if it falls through, it is *not taken*, or *untaken*. If instruction *i* is a taken branch, then the PC is usually not changed until the end of ID, after the completion of the address calculation and comparison.

Figure C.9 shows that the simplest method of dealing with branches is to redo the fetch of the instruction following a branch, once we detect the branch during ID (when instructions are decoded). The first IF cycle is essentially a stall, because it never performs useful work. You may have noticed that if the branch is untaken, then the repetition of the IF stage is unnecessary because the correct instruction was indeed fetched. We will develop several schemes to take advantage of this fact shortly.

One stall cycle for every branch will yield a performance loss of 10% to 30% depending on the branch frequency, so we will examine some techniques to deal with this loss.

Branch instruction	IF	ID	EX	MEM	WB		
Branch successor		IF	IF	ID	EX	MEM	WB
Branch successor + 1				IF	ID	EX	MEM
Branch successor + 2					IF	ID	EX

Figure C.9 A branch causes a one-cycle stall in the five-stage pipeline. The instruction after the branch is fetched, but the instruction is ignored, and the fetch is restarted once the branch target is known. It is probably obvious that if the branch is not taken, the second IF for branch successor is redundant. This will be addressed shortly.

Reducing Pipeline Branch Penalties

There are many methods for dealing with the pipeline stalls caused by branch delay; we discuss four simple compile time schemes in this subsection. In these four schemes the actions for a branch are static—they are fixed for each branch during the entire execution. The software can try to minimize the branch penalty using knowledge of the hardware scheme and of branch behavior. We will then look at hardware-based schemes that dynamically predict branch behavior, and Chapter 3 looks at more powerful hardware techniques for dynamic branch prediction.

The simplest scheme to handle branches is to *freeze* or *flush* the pipeline, holding or deleting any instructions after the branch until the branch destination is known. The attractiveness of this solution lies primarily in its simplicity both for hardware and software. It is the solution used earlier in the pipeline shown in Figure C.9. In this case, the branch penalty is fixed and cannot be reduced by software.

A higher-performance, and only slightly more complex, scheme is to treat every branch as not taken, simply allowing the hardware to continue as if the branch were not executed. Here, care must be taken not to change the processor state until the branch outcome is definitely known. The complexity of this scheme arises from having to know when the state might be changed by an instruction and how to "back out" such a change.

In the simple five-stage pipeline, this *predicted-not-taken* or *predicted-untaken* scheme is implemented by continuing to fetch instructions as if the branch were a normal instruction. The pipeline looks as if nothing out of the ordinary is happening. If the branch is taken, however, we need to turn the fetched instruction into a no-op and restart the fetch at the target address. Figure C.10 shows both situations.

An alternative scheme is to treat every branch as taken. As soon as the branch is decoded and the target address is computed, we assume the branch to be taken and

Untaken branch instruction	IF	ID	EX	MEM	WB				
Instruction $i+1$		IF	ID	EX	MEM	WB			
Instruction $i+2$			IF	ID	EX	MEM	WB		
Instruction $i+3$				IF	ID	EX	MEM	WB	
Instruction $i+4$					IF	ID	EX	MEM	WB

Taken branch instruction	IF	ID	EX	MEM	WB				
Instruction $i+1$		IF	**idle**	**idle**	**idle**	**idle**			
Branch target			IF	ID	EX	MEM	WB		
Branch target + 1				IF	ID	EX	MEM	WB	
Branch target + 2					IF	ID	EX	MEM	WB

Figure C.10 The predicted-not-taken scheme and the pipeline sequence when the branch is untaken (top) and taken (bottom). When the branch is untaken, determined during ID, we fetch the fall-through and just continue. If the branch is taken during ID, we restart the fetch at the branch target. This causes all instructions following the branch to stall 1 clock cycle.

begin fetching and executing at the target. This buys us a one-cycle improvement when the branch is actually taken, because we know the target address at the end of ID, one cycle before we know whether the branch condition is satisfied in the ALU stage. In either a predicted-taken or predicted-not-taken scheme, the compiler can improve performance by organizing the code so that the most frequent path matches the hardware's choice.

A fourth scheme, which was heavily used in early RISC processors is called *delayed branch*. In a delayed branch, the execution cycle with a branch delay of one is

```
branch instruction
sequential successor1
branch target if taken
```

The sequential successor is in the *branch delay slot*. This instruction is executed whether or not the branch is taken. The pipeline behavior of the five-stage pipeline with a branch delay is shown in Figure C.11. Although it is possible to have a branch delay longer than one, in practice almost all processors with delayed branch have a single instruction delay; other techniques are used if the pipeline has a longer potential branch penalty.The job of the compiler is to make the successor instructions valid and useful.

Although the delayed branch was useful for short simple pipelines at a time when hardware prediction was too expensive, the technique complicates implementation when there is dynamic branch prediction. For this reason, RISC V appropriately omitted delayed branches.

Untaken branch instruction	IF	ID	EX	MEM	WB				
Branch delay instruction ($i+1$)		IF	ID	EX	MEM	WB			
Instruction $i+2$			IF	ID	EX	MEM	WB		
Instruction $i+3$				IF	ID	EX	MEM	WB	
Instruction $i+4$					IF	ID	EX	MEM	WB

Taken branch instruction	IF	ID	EX	MEM	WB				
Branch delay instruction ($i+1$)		IF	ID	EX	MEM	WB			
Branch target			IF	ID	EX	MEM	WB		
Branch target + 1				IF	ID	EX	MEM	WB	
Branch target + 2					IF	ID	EX	MEM	WB

Figure C.11 The behavior of a delayed branch is the same whether or not the branch is taken. The instructions in the delay slot (there was only one delay slot for most RISC architectures that incorporated them) are executed. If the branch is untaken, execution continues with the instruction after the branch delay instruction; if the branch is taken, execution continues at the branch target. When the instruction in the branch delay slot is also a branch, the meaning is unclear: if the branch is not taken, what should happen to the branch in the branch delay slot? Because of this confusion, architectures with delay branches often disallow putting a branch in the delay slot.

Performance of Branch Schemes

What is the effective performance of each of these schemes? The effective pipeline speedup with branch penalties, assuming an ideal CPI of 1, is

$$\text{Pipeline speedup} = \frac{\text{Pipeline depth}}{1 + \text{Pipeline stall cycles from branches}}$$

Because of the following:

$$\text{Pipeline stall cycles from branches} = \text{Branch frequency} \times \text{Branch penalty}$$

we obtain:

$$\text{Pipeline speedup} = \frac{\text{Pipeline depth}}{1 + \text{Branch frequency} \times \text{Branch penalty}}$$

The branch frequency and branch penalty can have a component from both unconditional and conditional branches. However, the latter dominate because they are more frequent.

Example For a deeper pipeline, such as that in a MIPS R4000 and later RISC processors, it takes at least three pipeline stages before the branch-target address is known and an additional cycle before the branch condition is evaluated, assuming no stalls on the registers in the conditional comparison. A three-stage delay leads to the branch penalties for the three simplest prediction schemes listed in Figure C.12.

Find the effective addition to the CPI arising from branches for this pipeline, assuming the following frequencies:

Unconditional branch	4%
Conditional branch, untaken	6%
Conditional branch, taken	10%

Answer We find the CPIs by multiplying the relative frequency of unconditional, conditional untaken, and conditional taken branches by the respective penalties. The results are shown in Figure C.13.

Branch scheme	Penalty unconditional	Penalty untaken	Penalty taken
Flush pipeline	2	3	3
Predicted taken	2	3	2
Predicted untaken	2	0	3

Figure C.12 Branch penalties for the three simplest prediction schemes for a deeper pipeline.

	Additions to the CPI from branch costs			
Branch scheme	**Unconditional branches**	**Untaken conditional branches**	**Taken conditional branches**	**All branches**
Frequency of event	4%	6%	10%	20%
Stall pipeline	0.08	0.18	0.30	0.56
Predicted taken	0.08	0.18	0.20	0.46
Predicted untaken	0.08	0.00	0.30	0.38

Figure C.13 CPI penalties for three branch-prediction schemes and a deeper pipeline.

The differences among the schemes are substantially increased with this longer delay. If the base CPI were 1 and branches were the only source of stalls, the ideal pipeline would be 1.56 times faster than a pipeline that used the stall-pipeline scheme. The predicted-untaken scheme would be 1.13 times better than the stall-pipeline scheme under the same assumptions.

Reducing the Cost of Branches Through Prediction

As pipelines get deeper and the potential penalty of branches increases, using delayed branches and similar schemes becomes insufficient. Instead, we need to turn to more aggressive means for predicting branches. Such schemes fall into two classes: low-cost static schemes that rely on information available at compile time and strategies that predict branches dynamically based on program behavior. We discuss both approaches here.

Static Branch Prediction

A key way to improve compile-time branch prediction is to use profile information collected from earlier runs. The key observation that makes this worthwhile is that the behavior of branches is often bimodally distributed; that is, an individual branch is often highly biased toward taken or untaken. Figure C.14 shows the success of branch prediction using this strategy. The same input data were used for runs and for collecting the profile; other studies have shown that changing the input so that the profile is for a different run leads to only a small change in the accuracy of profile-based prediction.

The effectiveness of any branch prediction scheme depends both on the accuracy of the scheme and the frequency of conditional branches, which vary in SPEC from 3% to 24%. The fact that the misprediction rate for the integer programs is higher and such programs typically have a higher branch frequency is a major limitation for static branch prediction. In the next section, we consider dynamic branch predictors, which most recent processors have employed.

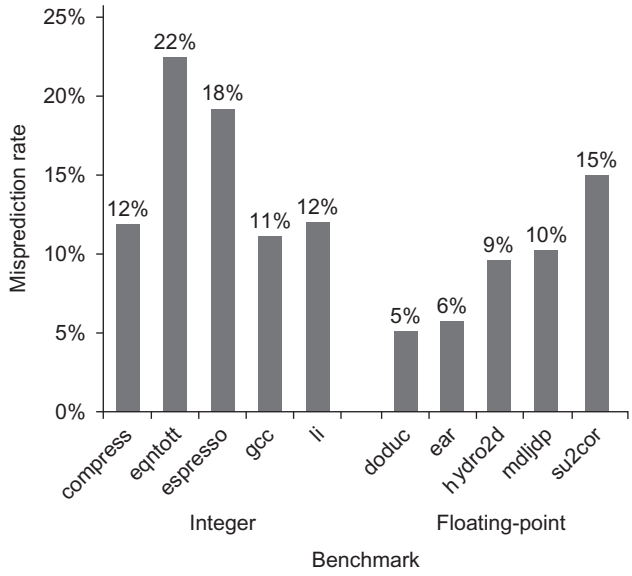


Figure C.14 Misprediction rate on SPEC92 for a profile-based predictor varies widely but is generally better for the floating-point programs, which have an average misprediction rate of 9% with a standard deviation of 4%, than for the integer programs, which have an average misprediction rate of 15% with a standard deviation of 5%. The actual performance depends on both the prediction accuracy and the branch frequency, which vary from 3% to 24%.

Dynamic Branch Prediction and Branch-Prediction Buffers

The simplest dynamic branch-prediction scheme is a *branch-prediction buffer* or *branch history table*. A branch-prediction buffer is a small memory indexed by the lower portion of the address of the branch instruction. The memory contains a bit that says whether the branch was recently taken or not. This scheme is the simplest sort of buffer; it has no tags and is useful only to reduce the branch delay when it is longer than the time to compute the possible target PCs.

With such a buffer, we don't know, in fact, if the prediction is correct—it may have been put there by another branch that has the same low-order address bits. But this doesn't matter. The prediction is a hint that is assumed to be correct, and fetching begins in the predicted direction. If the hint turns out to be wrong, the prediction bit is inverted and stored back.

This buffer is effectively a cache where every access is a hit, and, as we will see, the performance of the buffer depends on both how often the prediction is for the branch of interest and how accurate the prediction is when it matches. Before we analyze the performance, it is useful to make a small, but important, improvement in the accuracy of the branch-prediction scheme.

This simple 1-bit prediction scheme has a performance shortcoming: even if a branch is almost always taken, we will likely predict incorrectly twice, rather than once, when it is not taken, because the misprediction causes the prediction bit to be flipped.

To remedy this weakness, 2-bit prediction schemes are often used. In a 2-bit scheme, a prediction must miss twice before it is changed. Figure C.15 shows the finite-state processor for a 2-bit prediction scheme.

A branch-prediction buffer can be implemented as a small, special "cache" accessed with the instruction address during the IF pipe stage, or as a pair of bits attached to each block in the instruction cache and fetched with the instruction. If the instruction is decoded as a branch and if the branch is predicted as taken, fetching begins from the target as soon as the PC is known. Otherwise, sequential fetching and executing continue. As Figure C.15 shows, if the prediction turns out to be wrong, the prediction bits are changed.

What kind of accuracy can be expected from a branch-prediction buffer using 2 bits per entry on real applications? Figure C.16 shows that for the SPEC89 benchmarks a branch-prediction buffer with 4096 entries results in a prediction accuracy

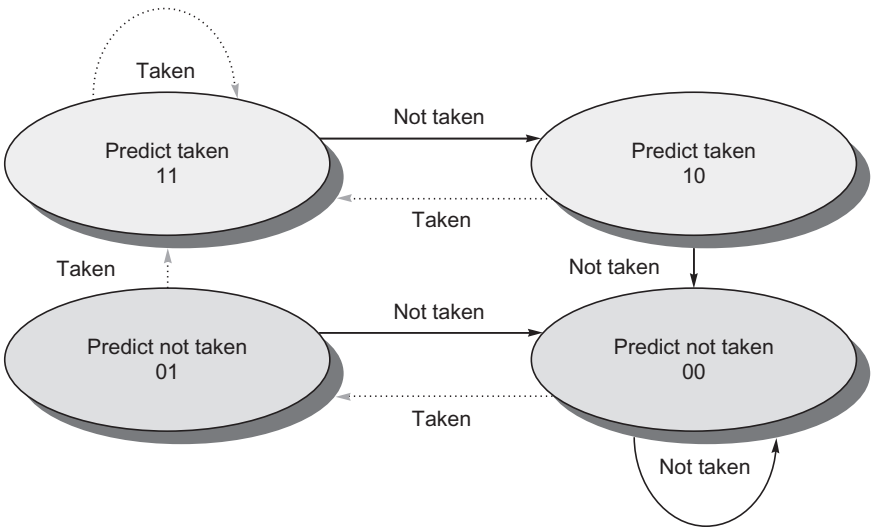


Figure C.15 The states in a 2-bit prediction scheme. By using 2 bits rather than 1, a branch that strongly favors taken or not taken—as many branches do—will be mispredicted less often than with a 1-bit predictor. The 2 bits are used to encode the four states in the system. The 2-bit scheme is actually a specialization of a more general scheme that has an n-bit saturating counter for each entry in the prediction buffer. With an n-bit counter, the counter can take on values between 0 and $2^n - 1$: when the counter is greater than or equal to one-half of its maximum value ($2^n - 1$), the branch is predicted as taken; otherwise, it is predicted as untaken. Studies of n-bit predictors have shown that the 2-bit predictors do almost as well, thus most systems rely on 2-bit branch predictors rather than the more general n-bit predictors.

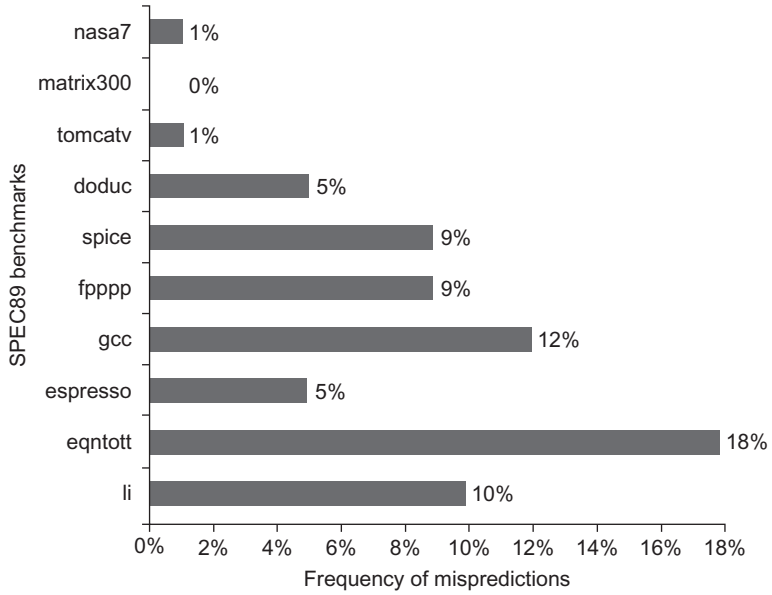


Figure C.16 Prediction accuracy of a 4096-entry 2-bit prediction buffer for the SPEC89 benchmarks. The misprediction rate for the integer benchmarks (gcc, espresso, eqntott, and li) is substantially higher (average of 11%) than that for the floating-point programs (average of 4%). Omitting the floating-point kernels (nasa7, matrix300, and tomcatv) still yields a higher accuracy for the FP benchmarks than for the integer benchmarks. These data, as well as the rest of the data in this section, are taken from a branch-prediction study done using the IBM Power architecture and optimized code for that system. See Pan et al. (1992). Although these data are for an older version of a subset of the SPEC benchmarks, the newer benchmarks are larger and would show slightly worse behavior, especially for the integer benchmarks.

ranging from over 99% to 82%, or a *misprediction rate* of 1%–18%. A 4K entry buffer, like that used for these results, is considered small in 2017, and a larger buffer could produce somewhat better results.

As we try to exploit more ILP, the accuracy of our branch prediction becomes critical. As we can see in Figure C.16, the accuracy of the predictors for integer programs, which typically also have higher branch frequencies, is lower than for the loop-intensive scientific programs. We can attack this problem in two ways: by increasing the size of the buffer and by increasing the accuracy of the scheme we use for each prediction. A buffer with 4K entries, however, as Figure C.17 shows, performs quite comparably to an infinite buffer, at least for benchmarks like those in SPEC. The data in Figure C.17 make it clear that the hit rate of the buffer is not the major limiting factor. As we mentioned, simply increasing the number of bits per predictor without changing the predictor structure also has little impact. Instead, we need to look at how we might increase the accuracy of each predictor, as we will in Chapter 3.

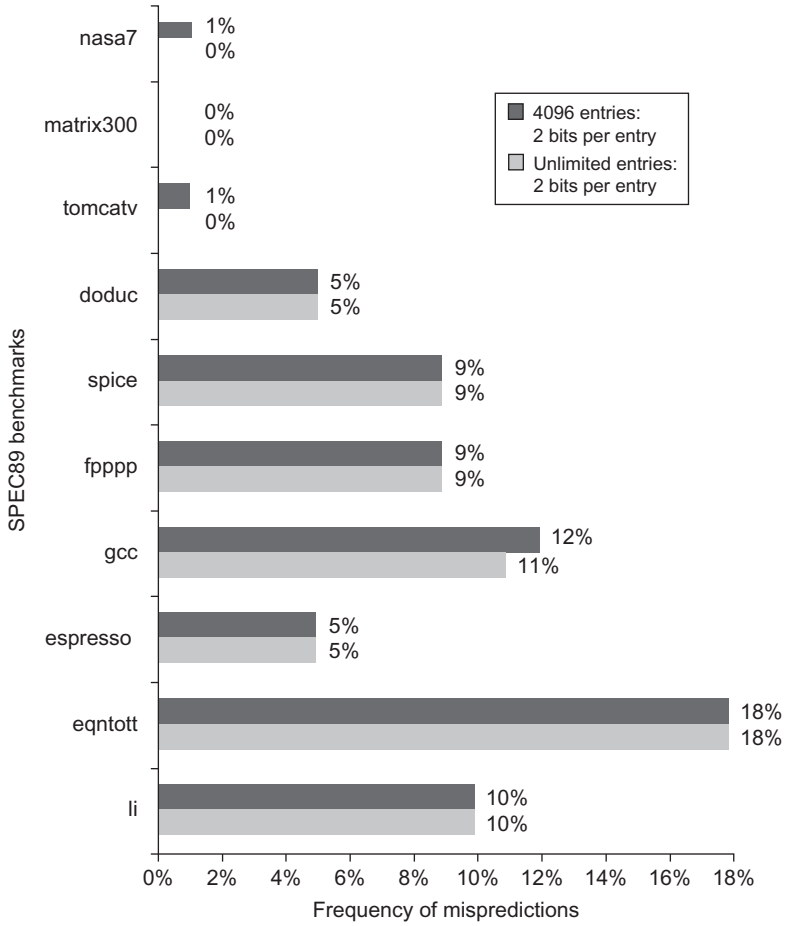


Figure C.17 Prediction accuracy of a 4096-entry 2-bit prediction buffer versus an infinite buffer for the SPEC89 benchmarks. Although these data are for an older version of a subset of the SPEC benchmarks, the results would be comparable for newer versions with perhaps as many as 8K entries needed to match an infinite 2-bit predictor.

C.3 How Is Pipelining Implemented?

Before we proceed to basic pipelining, we need to review a simple implementation of an unpipelined version of RISC V.

A Simple Implementation of RISC V

In this section we follow the style of Section C.1, showing first a simple unpipelined implementation and then the pipelined implementation. This time, however, our example is specific to the RISC V architecture.

In this subsection, we focus on a pipeline for an integer subset of RISC V that consists of load-store word, branch equal, and integer ALU operations. Later in this appendix we will incorporate the basic floating-point operations. Although we discuss only a subset of RISC V, the basic principles can be extended to handle all the instructions; for example, adding store involves some additional computing of the immediate field. We initially used a less aggressive implementation of a branch instruction. We show how to implement the more aggressive version at the end of this section.

Every RISC V instruction can be implemented in, at most, 5 clock cycles. The 5 clock cycles are as follows:

1. *Instruction fetch cycle* (IF):

```
IR ← Mem[PC];
NPC ← PC + 4;
```

Operation—Send out the PC and fetch the instruction from memory into the instruction register (IR); increment the PC by 4 to address the next sequential instruction. The IR is used to hold the instruction that will be needed on subsequent clock cycles; likewise, the register NPC is used to hold the next sequential PC.

2. *Instruction decode/register fetch cycle* (ID):

```
A ← Regs[rs1];
B ← Regs[rs2];
Imm ← sign-extended immediate field of IR;
```

Operation—Decode the instruction and access the register file to read the registers (rs1 and rs2 are the register specifiers). The outputs of the general-purpose registers are read into two temporary registers (A and B) for use in later clock cycles. The lower 16 bits of the IR are also sign extended and stored into the temporary register Imm, for use in the next cycle.

Decoding is done in parallel with reading registers, which is possible because these fields are at a fixed location in the RISC V instruction format. Because the immediate portion of a load and an ALU immediate is located in an identical place in every RISC V instruction, the sign-extended immediate is also calculated during this cycle in case it is needed in the next cycle. For stores, a separate sign-extension is needed, because the immediate field is split in two pieces.

3. *Execution/effective address cycle* (EX):

The ALU operates on the operands prepared in the prior cycle, performing one of four functions depending on the RISC V instruction type:

- Memory reference:

```
ALUOutput ← A + Imm;
```

Operation—The ALU adds the operands to form the effective address and places the result into the register ALUOutput.

- Register-register ALU instruction:

```
ALUOutput ← A func B;
```

Operation—The ALU performs the operation specified by the function code (a combination of the func3 and func7 fields) on the value in register A and on the value in register B. The result is placed in the temporary register ALUOutput.

- Register-Immediate ALU instruction:

```
ALUOutput ← A op Imm;
```

Operation—The ALU performs the operation specified by the opcode on the value in register A and on the value in register Imm. The result is placed in the temporary register ALUOutput.

- Branch:

```
ALUOutput ← NPC + (Imm << 2);
Cond ← (A == B)
```

Operation—The ALU adds the NPC to the sign-extended immediate value in Imm, which is shifted left by 2 bits to create a word offset, to compute the address of the branch target. Register A, which has been read in the prior cycle, is checked to determine whether the branch is taken, by comparison with Register B, because we consider only branch equal.

The load-store architecture of RISC V means that effective address and execution cycles can be combined into a single clock cycle, because no instruction needs to simultaneously calculate a data address, calculate an instruction target address, and perform an operation on the data. The other integer instructions not included herein are jumps of various forms, which are similar to branches.

4. *Memory access/branch completion cycle* (MEM):

The PC is updated for all instructions: `PC ← NPC;`

- Memory reference:

```
LMD ← Mem[ALUOutput] or
Mem[ALUOutput] ← B;
```

Operation—Access memory if needed. If the instruction is a load, data return from memory and are placed in the LMD (load memory data) register; if it is a store, then the data from the B register are written into memory. In either case, the address used is the one computed during the prior cycle and stored in the register ALUOutput.

- Branch:

```
if (cond) PC ← ALUOutput
```

Operation—If the instruction branches, the PC is replaced with the branch destination address in the register ALUOutput.

5. *Write-back cycle* (WB):

- Register-register or Register-immediate ALU instruction:

```
Regs[rd] ← ALUOutput;
```

- Load instruction:

```
Regs[rd] ← LMD;
```

Operation—Write the result into the register file, whether it comes from the memory system (which is in LMD) or from the ALU (which is in ALUOutput) with rd designating the register.

Figure C.18 shows how an instruction flows through the data path. At the end of each clock cycle, every value computed during that clock cycle and required on a later clock cycle (whether for this instruction or the next) is written into a storage device, which may be memory, a general-purpose register, the PC, or a temporary register (i.e., LMD, Imm, A, B, IR, NPC, ALUOutput, or Cond). The temporary registers hold values between clock cycles for one instruction, while the other storage elements are visible parts of the state and hold values between successive instructions.

Although all processors today are pipelined, this multicycle implementation is a reasonable approximation of how most processors would have been implemented in earlier times. A simple finite-state machine could be used to implement the control following the five-cycle structure shown herein. For a much more complex processor, microcode control could be used. In either event, an instruction sequence like the one described in this section would determine the structure of the control.

There are some hardware redundancies that could be eliminated in this multicycle implementation. For example, there are two ALUs: one to increment the PC and one used for effective address and ALU computation. Because they are not needed on the same clock cycle, we could merge them by adding additional multiplexers and sharing the same ALU. Likewise, instructions and data could be stored in the same memory, because the data and instruction accesses happen on different clock cycles.

Rather than optimize this simple implementation, we will leave the design as it is in Figure C.18, because this provides us with a better base for the pipelined implementation.

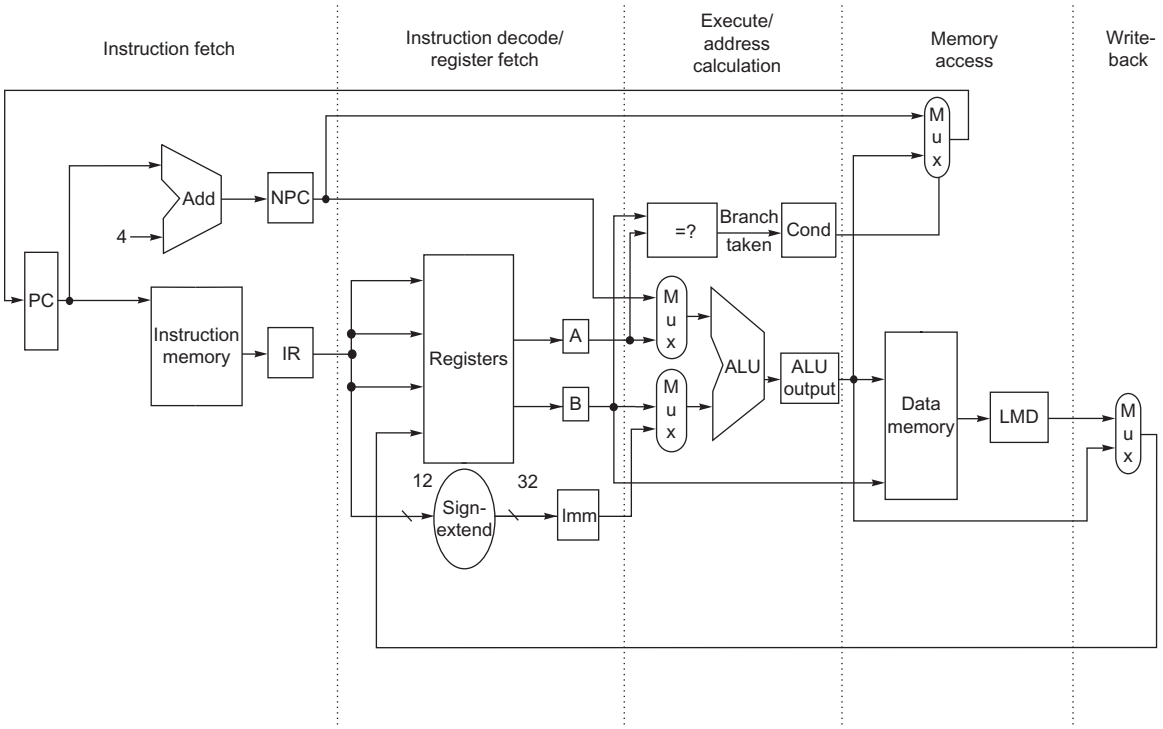


Figure C.18 The implementation of the RISC V data path allows every instruction to be executed in 4 or 5 clock cycles. Although the PC is shown in the portion of the data path that is used in instruction fetch and the registers are shown in the portion of the data path that is used in instruction decode/register fetch, both of these functional units are read as well as written by an instruction. Although we show these functional units in the cycle corresponding to where they are read, the PC is written during the memory access clock cycle and the registers are written during the write-back clock cycle. In both cases, the writes in later pipe stages are indicated by the multiplexer output (in memory access or write-back), which carries a value back to the PC or registers. These backward-flowing signals introduce much of the complexity of pipelining, because they indicate the possibility of hazards.

A Basic Pipeline for RISC V

As before, we can pipeline the data path of Figure C.18 with almost no changes by starting a new instruction on each clock cycle. Because every pipe stage is active on every clock cycle, all operations in a pipe stage must complete in 1 clock cycle and any combination of operations must be able to occur at once. Furthermore, pipelining the data path requires that values passed from one pipe stage to the next must be placed in registers. Figure C.19 shows the RISC V pipeline with the appropriate registers, called *pipeline registers* or *pipeline latches*, between each pipeline stage. The registers are labeled with the names of the stages they connect.

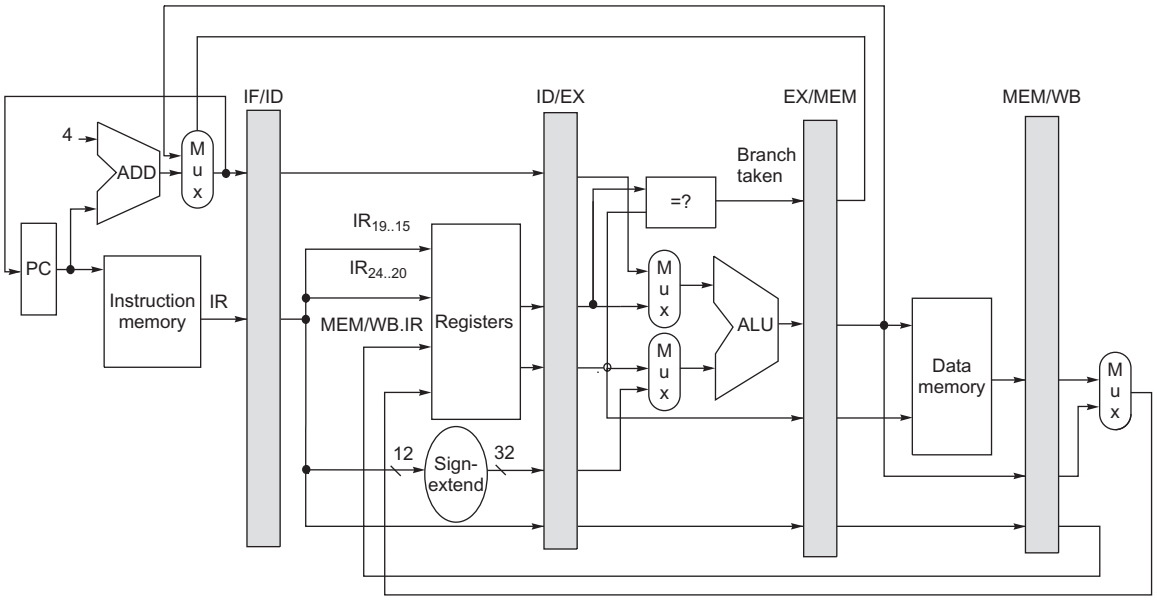


Figure C.19 The data path is pipelined by adding a set of registers, one between each pair of pipe stages. The registers serve to convey values and control information from one stage to the next. We can also think of the PC as a pipeline register, which sits before the IF stage of the pipeline, leading to one pipeline register for each pipe stage. Recall that the PC is an edge-triggered register written at the end of the clock cycle; hence, there is no race condition in writing the PC. The selection multiplexer for the PC has been moved so that the PC is written in exactly one stage (IF). If we didn't move it, there would be a conflict when a branch occurred, because two instructions would try to write different values into the PC. Most of the data paths flow from left to right, which is from earlier in time to later. The paths flowing from right to left (which carry the register write-back information and PC information on a branch) introduce complications into our pipeline.

Figure C.19 is drawn so that connections through the pipeline registers from one stage to another are clear.

All of the registers needed to hold values temporarily between clock cycles within one instruction are subsumed into these pipeline registers. The fields of the instruction register (IR), which is part of the IF/ID register, are labeled when they are used to supply register names. The pipeline registers carry both data and control from one pipeline stage to the next. Any value needed on a later pipeline stage must be placed in such a register and copied from one pipeline register to the next, until it is no longer needed. If we tried to just use the temporary registers we had in our earlier unpipelined data path, values could be overwritten before all uses were completed. For example, the field of a register operand used for a write on a load or ALU operation is supplied from the MEM/WB pipeline register rather than from the IF/ID register. This is because we want a load or ALU operation to write the register designated by that operation, not the register field of the

instruction currently transitioning from IF to ID! This destination register field is simply copied from one pipeline register to the next, until it is needed during the WB stage.

Any instruction is active in exactly one stage of the pipeline at a time; therefore, any actions taken on behalf of an instruction occur between a pair of pipeline registers. Thus, we can also look at the activities of the pipeline by examining what has to happen on any pipeline stage depending on the instruction type. Figure C.20 shows this view. Fields of the pipeline registers are named so as to show the flow

Stage	Any instruction		
IF	`IF/ID.IR←Mem[PC]` `IF/ID.NPC,PC←(if ((EX/MEM.opcode == branch) & EX/MEM.cond){EX/MEM.ALUOutput} else {PC+4});`		

ID	`ID/EX.A←Regs[IF/ID.IR[rs1]]; ID/EX.B←Regs[IF/ID.IR[rs2]];`
	`ID/EX.NPC←IF/ID.NPC; ID/EX.IR←IF/ID.IR;`
	`ID/EX.Imm←sign-extend(IF/ID.IR[immediate field]);`		

	ALU instruction	**Load instruction**	**Branch instruction**

EX	`EX/MEM.IR←ID/EX.IR;`
	`EX/MEM.ALUOutput←`
	`ID/EX.A func ID/EX.B;`
	or
	`EX/MEM.ALUOutput←`
	`ID/EX.A` *op* `ID/EX.Imm;`
		`EX/MEM.IR to ID/EX.IR`
		`EX/MEM.ALUOutput←`
		`ID/EX.A+ID/EX.Imm;`
		
		`EX/MEM.B←ID/EX.B;`
			`EX/MEM.ALUOutput←`
			`ID/EX.NPC +`
			`(ID/EX.Imm<<2);`
			
			`EX/MEM.cond←`
			`(ID/EX.A == ID/EX.B);`

MEM	`MEM/WB.IR←EX/MEM.IR;`
	`MEM/WB.ALUOutput←`
	`EX/MEM.ALUOutput;`
		`MEM/WB.IR←EX/MEM.IR;`
		`MEM/WB.LMD←`
		`Mem[EX/MEM.ALUOutput];`
		or
		`Mem[EX/MEM.ALUOutput]←`
		`EX/MEM.B;`	

WB	`Regs[MEM/WB.IR[rd]]←`
	`MEM/WB.ALUOutput;`
		`For load only:`
		`Regs[MEM/WB.IR[rd]]←`
		`MEM/WB.LMD;`	

Figure C.20 Events on every pipe stage of the RISC V pipeline. Let's review the actions in the stages that are specific to the pipeline organization. In IF, in addition to fetching the instruction and computing the new PC, we store the incremented PC both into the PC and into a pipeline register (NPC) for later use in computing the branch-target address. This structure is the same as the organization in Figure C.19, where the PC is updated in IF from one of two sources. In ID, we fetch the registers, extend the sign of the 12 bits of the IR (the immediate field), and pass along the IR and NPC. During EX, we perform an ALU operation or an address calculation; we pass along the IR and the B register (if the instruction is a store). We also set the value of cond to 1 if the instruction is a taken branch. During the MEM phase, we cycle the memory, write the PC if needed, and pass along values needed in the final pipe stage. Finally, during WB, we update the register field from either the ALU output or the loaded value. For simplicity we always pass the entire IR from one stage to the next, although as an instruction proceeds down the pipeline, less and less of the IR is needed.

of data from one stage to the next. Notice that the actions in the first two stages are independent of the current instruction type; they must be independent because the instruction is not decoded until the end of the ID stage. The IF activity depends on whether the instruction in EX/MEM is a taken branch. If so, then the branch-target address of the branch instruction in EX/MEM is written into the PC at the end of IF; otherwise, the incremented PC will be written back. (As we said earlier, this effect of branches leads to complications in the pipeline that we deal with in the next few sections.) The fixed-position encoding of the register source operands is critical to allowing the registers to be fetched during ID.

To control this simple pipeline we need only determine how to set the control for the four multiplexers in the data path of Figure C.19. The two multiplexers in the ALU stage are set depending on the instruction type, which is dictated by the IR field of the ID/EX register. The top ALU input multiplexer is set by whether the instruction is a branch or not, and the bottom multiplexer is set by whether the instruction is a register-register ALU operation or any other type of operation. The multiplexer in the IF stage chooses whether to use the value of the incremented PC or the value of the EX/MEM.ALUOutput (the branch target) to write into the PC. This multiplexer is controlled by the field EX/MEM.cond. The fourth multiplexer is controlled by whether the instruction in the WB stage is a load or an ALU operation. In addition to these four multiplexers, there is one additional multiplexer needed that is not drawn in Figure C.19, but whose existence is clear from looking at the WB stage of an ALU operation. The destination register field is in one of two different places depending on the instruction type (register-register ALU versus either ALU immediate or load). Thus, we will need a multiplexer to choose the correct portion of the IR in the MEM/WB register to specify the register destination field, assuming the instruction writes a register.

Implementing the Control for the RISC V Pipeline

The process of letting an instruction move from the instruction decode stage (ID) into the execution stage (EX) of this pipeline is usually called *instruction issue*; an instruction that has made this step is said to have *issued*. For the RISC V integer pipeline, all the data hazards can be checked during the ID phase of the pipeline. If a data hazard exists, the instruction is stalled before it is issued. Likewise, we can determine what forwarding will be needed during ID and set the appropriate controls then. Detecting interlocks early in the pipeline reduces the hardware complexity because the hardware never has to suspend an instruction that has updated the state of the processor, unless the entire processor is stalled. Alternatively, we can detect the hazard or forwarding at the beginning of a clock cycle that uses an operand (EX and MEM for this pipeline). To show the differences in these two approaches, we will show how the interlock for a read after write (RAW) hazard with the source coming from a load instruction (called a *load interlock*) can be implemented by a check in ID, while the implementation of forwarding paths to

Situation	Example code sequence	Action

No dependence	`ld x1,45(x2)`
	`add x5,x6,x7`
	`sub x8,x6,x7`
	`or x9,x6,x7`
		No hazard possible because no dependence exists on x1 in the immediately following three instructions

Dependence requiring stall	`ld x1,45(x2)`
	`add x5,x1,x7`
	`sub x8,x6,x7`
	`or x9,x6,x7`
		Comparators detect the use of x1 in the `add` and stall the `add` (and `sub` and `or`) before the `add` begins EX

Dependence overcome by forwarding	`ld x1,45(x2)`
	`add x5,x6,x7`
	`sub x8,x1,x7`
	`or x9,x6,x7`
		Comparators detect use of x1 in `sub` and forward result of load to ALU in time for `sub` to begin EX

Dependence with accesses in order	`ld x1,45(x2)`
	`add x5,x6,x7`
	`sub x8,x6,x7`
	`or x9,x1,x7`
		No action required because the read of x1 by `or` occurs in the second half of the ID phase, while the write of the loaded data occurred in the first half

Figure C.21 Situations that the pipeline hazard detection hardware can see by comparing the destination and sources of adjacent instructions. This table indicates that the only comparison needed is between the destination and the sources on the two instructions following the instruction that wrote the destination. In the case of a stall, the pipeline dependences will look like the third case once execution continues (dependence overcome by forwarding). Of course, hazards that involve x0 can be ignored because the register always contains 0, and the preceding test could be extended to do this.

the ALU inputs can be done during EX. Figure C.21 lists the variety of circumstances that we must handle.

Let's start with implementing the load interlock. If there is a RAW hazard with the source instruction being a load, the load instruction will be in the EX stage when an instruction that needs the load data will be in the ID stage. Thus, we can describe all the possible hazard situations with a small table, which can be directly translated to an implementation. Figure C.22 shows a table that detects all load interlocks when the instruction using the load result is in the ID stage.

Once a hazard has been detected, the control unit must insert the pipeline stall and prevent the instructions in the IF and ID stages from advancing. As we said earlier, all the control information is carried in the pipeline registers. (Carrying the instruction along is enough, because all control is derived from it.) Thus, when we detect a hazard we need only change the control portion of the ID/EX pipeline register to all 0s, which happens to be a no-op (an instruction that does nothing, such as `add x0,x0,x0`). In addition, we simply recirculate the contents of the IF/ID registers to hold the stalled instruction. In a pipeline with more complex hazards, the same ideas would apply: we can detect the hazard by comparing some set of pipeline registers and shift in no-ops to prevent erroneous execution.

Opcode field of ID/EX ($ID/EX.IR_{0..5}$)	Opcode field of IF/ID ($IF/ID.IR_{0..6}$)	Matching operand fields
Load	Register-register ALU, load, store, ALU immediate, or branch	`ID/EX.IR[rd] == IF/ID.IR[rs1]`
Load	Register-register ALU, or branch	`ID/EX.IR[rd] ==` IF/`ID.IR[rs2]`

Figure C.22 The logic to detect the need for load interlocks during the ID stage of an instruction requires two comparisons, one for each possible source. Remember that the IF/ID register holds the state of the instruction in ID, which potentially uses the load result, while ID/EX holds the state of the instruction in EX, which is the load instruction.

Implementing the forwarding logic is similar, although there are more cases to consider. The key observation needed to implement the forwarding logic is that the pipeline registers contain both the data to be forwarded as well as the source and destination register fields. All forwarding logically happens from the ALU or data memory output to the ALU input, the data memory input, or the zero detection unit. Thus, we can implement the forwarding by a comparison of the destination registers of the IR contained in the EX/MEM and MEM/WB stages against the source registers of the IR contained in the ID/EX and EX/MEM registers. Figure C.23 shows the comparisons and possible forwarding operations where the destination of the forwarded result is an ALU input for the instruction currently in EX.

In addition to the comparators and combinational logic that we must determine when a forwarding path needs to be enabled, we also must enlarge the multiplexers at the ALU inputs and add the connections from the pipeline registers that are used to forward the results. Figure C.24 shows the relevant segments of the pipelined data path with the additional multiplexers and connections in place.

For RISC V, the hazard detection and forwarding hardware is reasonably simple; we will see that things become somewhat more complicated when we extend this pipeline to deal with floating point. Before we do that, we need to handle branches.

Dealing With Branches in the Pipeline

In RISC V, conditional branches depend on comparing two register values, which we assume occurs during the EX cycle, and uses the ALU for this function. We will need to also compute the branch target address. Because testing the branch condition and determining the next PC will determine what the branch penalty is, we would like to compute both the possible PCs and choose the correct PC before the end of the EX cycle. We can do this by adding a separate adder that computes the branch target address during ID. Because the instruction is not yet decoded, we will be computing a possible target as if every instruction were a branch. This is

Pipeline register of source instruction	Opcode of source instruction	Pipeline register of destination instruction	Opcode of destination instruction	Destination of the forwarded result	Comparison (if equal then forward)
EX/MEM	Register-register ALU, ALU immediate	ID/EX	Register-register ALU, ALU immediate, load, store, branch	Top ALU input	`EX/MEM.IR[rd] == ID/EX.IR[rs1]`
EX/MEM	Register-register ALU, ALU immediate	ID/EX	Register-register ALU	Bottom ALU input	`EX/MEM.IR[rd] == ID/EX.IR[rs2]`
MEM/WB	Register-register ALU, ALU immediate, Load	ID/EX	Register-register ALU, ALU immediate, load, store, branch	Top ALU input	`MEM/WB.IR[rd] == ID/EX.IR[rs1]`
MEM/WB	Register-register ALU, ALU immediate, Load	ID/EX	Register-register ALU	Bottom ALU input	`MEM/WB.IR[rd] == ID/EX.IR[rs2]`

Figure C.23 Forwarding of data to the two ALU inputs (for the instruction in EX) can occur from the ALU result (in EX/MEM or in MEM/WB) or from the load result in MEM/WB. There are 10 separate comparisons needed to tell whether a forwarding operation should occur. The top and bottom ALU inputs refer to the inputs corresponding to the first and second ALU source operands, respectively, and are shown explicitly in Figure C.18 on page C.30 and in Figure C.24 on page C.36. Remember that the pipeline latch for destination instruction in EX is ID/EX, while the source values come from the ALUOutput portion of EX/MEM or MEM/WB or the LMD portion of MEM/WB. There is one complication not addressed by this logic: dealing with multiple instructions that write the same register. For example, during the code sequence `add x1, x2, x3; addi x1, x1, 2; sub x4, x3, x1`, the logic must ensure that the `sub` instruction uses the result of the `addi` instruction rather than the result of the `add` instruction. The logic shown here can be extended to handle this case by simply testing that forwarding from MEM/WB is enabled only when forwarding from EX/MEM is not enabled for the same input. Because the `addi` result will be in EX/MEM, it will be forwarded, rather than the `add` result in MEM/WB.

likely faster than computing the target and evaluating the condition both in EX, but does use slightly more energy.

Figure C.25 shows a pipelined data path assuming the adder in ID and the evaluation of the branch condition in EX, a minor change of the pipeline structure. This pipeline will incur a two-cycle penalty on branches. In some early RISC processors, such as MIPS, the condition test on branches was restricted to allow the test to occur in ID, reducing the branch delay to one cycle. Of course, that meant that an ALU operation to a register followed by a conditional branch based on that register incurred a data hazard, which does not occur if the branch condition is evaluated in EX.

As pipeline depths increased, the branch delay increased, which made dynamic branch prediction necessary. For example, a processor with separate decode and

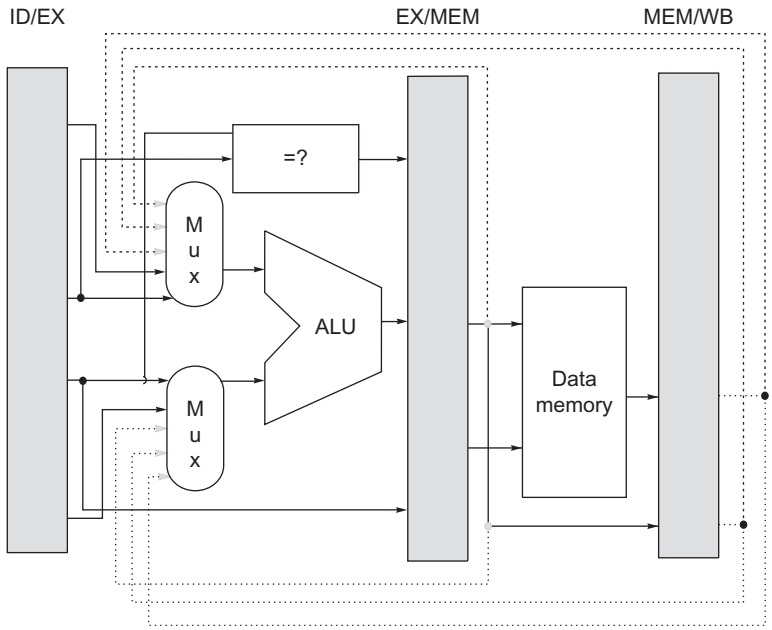


Figure C.24 Forwarding of results to the ALU requires the addition of three extra inputs on each ALU multiplexer and the addition of three paths to the new inputs. The paths correspond to a bypass of: (1) the ALU output at the end of the EX, (2) the ALU output at the end of the MEM stage, and (3) the memory output at the end of the MEM stage.

register fetch stages will probably have a branch delay that is at least 1 clock cycle longer. The branch delay, unless it is dealt with, turns into a branch penalty. Many older processors that implement more complex instruction sets have branch delays of 4 clock cycles or more, and large, deeply pipelined processors often have branch penalties of 6 or 7. Aggressive high-end superscalars, such as the Intel i7 discussed in Chapter 3, may have branch penalties of 10–15 cycles! In general, the deeper the pipeline, the worse the branch penalty in clock cycles, and the more critical that branches be accurately predicted.

C.4 What Makes Pipelining Hard to Implement?

Now that we understand how to detect and resolve hazards, we can deal with some complications that we have avoided so far. The first part of this section considers the challenges of exceptional situations where the instruction execution order is changed in unexpected ways. In the second part of this section, we discuss some of the challenges raised by different instruction sets.

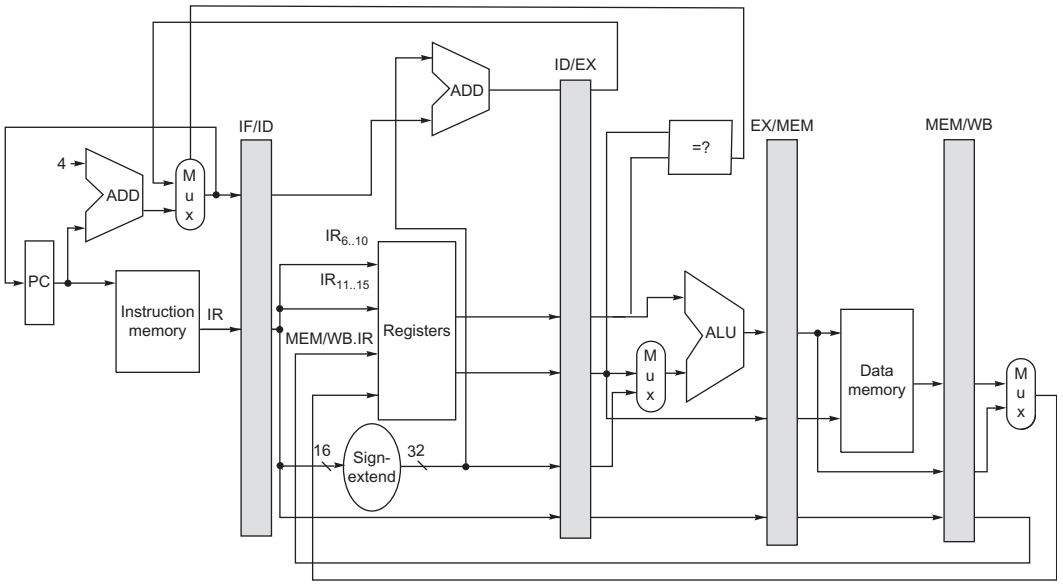


Figure C.25 To minimize the impact of deciding whether a conditional branch is taken, we compute the branch target address in ID while doing the conditional test and final selection of next PC in EX. As mentioned in Figure C.19, the PC can be thought of as a pipeline register (e.g., as part of ID/IF), which is written with the address of the next instruction at the end of each IF cycle.

Dealing With Exceptions

Exceptional situations are harder to handle in a pipelined processor because the overlapping of instructions makes it more difficult to know whether an instruction can safely change the state of the processor. In a pipelined processor, an instruction is executed piece by piece and is not completed for several clock cycles. Unfortunately, other instructions in the pipeline can raise exceptions that may force the processor to abort the instructions in the pipeline before they complete. Before we discuss these problems and their solutions in detail, we need to understand what types of situations can arise and what architectural requirements exist for supporting them.

Types of Exceptions and Requirements

The terminology used to describe exceptional situations where the normal execution order of instruction is changed varies among processors. The terms *interrupt, fault*, and *exception* are used, although not in a consistent fashion. We use the term *exception* to cover all these mechanisms, including the following:

- I/O device request
- Invoking an operating system service from a user program

- Tracing instruction execution
- Breakpoint (programmer-requested interrupt)
- Integer arithmetic overflow
- FP arithmetic anomaly
- Page fault (not in main memory)
- Misaligned memory accesses (if alignment is required)
- Memory protection violation
- Using an undefined or unimplemented instruction
- Hardware malfunctions
- Power failure

When we wish to refer to some particular class of such exceptions, we will use a longer name, such as I/O interrupt, floating-point exception, or page fault.

Although we use the term *exception* to cover all of these events, individual events have important characteristics that determine what action is needed in the hardware. The requirements on exceptions can be characterized on five semi-independent axes:

1. *Synchronous versus asynchronous*—If the event occurs at the same place every time the program is executed with the same data and memory allocation, the event is *synchronous*. With the exception of hardware malfunctions, *asynchronous* events are caused by devices external to the processor and memory. Asynchronous events usually can be handled after the completion of the current instruction, which makes them easier to handle.
2. *User requested versus coerced*—If the user task directly asks for it, it is a *user-requested* event. In some sense, user-requested exceptions are not really exceptions, because they are predictable. They are treated as exceptions, however, because the same mechanisms that are used to save and restore the state are used for these user-requested events. Because the only function of an instruction that triggers this exception is to cause the exception, user-requested exceptions can always be handled after the instruction has completed. *Coerced* exceptions are caused by some hardware event that is not under the control of the user program. Coerced exceptions are harder to implement because they are not predictable.
3. *User maskable versus user nonmaskable*—If an event can be masked or disabled by a user task, it is *user maskable*. This mask simply controls whether the hardware responds to the exception or not.
4. *Within versus between instructions*—This classification depends on whether the event prevents instruction completion by occurring in the middle of execution—no matter how short—or whether it is recognized *between* instructions. Exceptions that occur *within* instructions are usually synchronous, because the instruction triggers the exception. It's harder to implement exceptions that

occur within instructions than those between instructions, because the instruction must be stopped and restarted. Asynchronous exceptions that occur within instructions arise from catastrophic situations (e.g., hardware malfunction) and always cause program termination.

5. *Resume versus terminate*—If the program's execution always stops after the interrupt, it is a *terminating* event. If the program's execution continues after the interrupt, it is a *resuming* event. It is easier to implement exceptions that terminate execution, because the processor need not be able to restart execution of the same program after handling the exception.

Figure C.26 classifies the preceding examples according to these five categories. The difficult task is implementing interrupts occurring within instructions where the instruction must be resumed. Implementing such exceptions requires that another program must be invoked to save the state of the executing program, correct the cause of the exception, and then restore the state of the program before the

Exception type	Synchronous vs. asynchronous	User request vs. coerced	User maskable vs. nonmaskable	Within vs. between instructions	Resume vs. terminate
I/O device request	Asynchronous	Coerced	Nonmaskable	Between	Resume
Invoke operating system	Synchronous	User request	Nonmaskable	Between	Resume
Tracing instruction execution	Synchronous	User request	User maskable	Between	Resume
Breakpoint	Synchronous	User request	User maskable	Between	Resume
Integer arithmetic overflow	Synchronous	Coerced	User maskable	Within	Resume
Floating-point arithmetic overflow or underflow	Synchronous	Coerced	User maskable	Within	Resume
Page fault	Synchronous	Coerced	Nonmaskable	Within	Resume
Misaligned memory accesses	Synchronous	Coerced	User maskable	Within	Resume
Memory protection violations	Synchronous	Coerced	Nonmaskable	Within	Resume
Using undefined instructions	Synchronous	Coerced	Nonmaskable	Within	Terminate
Hardware malfunctions	Asynchronous	Coerced	Nonmaskable	Within	Terminate
Power failure	Asynchronous	Coerced	Nonmaskable	Within	Terminate

Figure C.26 Five categories are used to define what actions are needed for the different exception types. Exceptions that must allow resumption are marked as resume, although the software may often choose to terminate the program. Synchronous, coerced exceptions occurring within instructions that can be resumed are the most difficult to implement. We might expect that memory protection access violations would always result in termination; however, modern operating systems use memory protection to detect events such as the first attempt to use a page or the first write to a page. Thus, processors should be able to resume after such exceptions.

instruction that caused the exception can be tried again. This process must be effectively invisible to the executing program. If a pipeline provides the ability for the processor to handle the exception, save the state, and restart without affecting the execution of the program, the pipeline or processor is said to be *restartable*. While early supercomputers and microprocessors often lacked this property, almost all processors today support it, at least for the integer pipeline, because it is needed to implement virtual memory (see Chapter 2).

Stopping and Restarting Execution

As in unpipelined implementations, the most difficult exceptions have two properties: (1) they occur within instructions (that is, in the middle of the instruction execution corresponding to EX or MEM pipe stages), and (2) they must be restartable. In our RISC V pipeline, for example, a virtual memory page fault resulting from a data fetch cannot occur until sometime in the MEM stage of the instruction. By the time that fault is seen, several other instructions will be in execution. A page fault must be restartable and requires the intervention of another process, such as the operating system. Thus, the pipeline must be safely shut down and the state saved so that the instruction can be restarted in the correct state. Restarting is usually implemented by saving the PC of the instruction at which to restart. If the restarted instruction is not a branch, then we will continue to fetch the sequential successors and begin their execution in the normal fashion. If the restarted instruction is a branch, then we will reevaluate the branch condition and begin fetching from either the target or the fall-through. When an exception occurs, the pipeline control can take the following steps to save the pipeline state safely:

1. Force a trap instruction into the pipeline on the next IF.
2. Until the trap is taken, turn off all writes for the faulting instruction and for all instructions that follow in the pipeline; this can be done by placing zeros into the pipeline latches of all instructions in the pipeline, starting with the instruction that generates the exception, but not those that precede that instruction. This prevents any state changes for instructions that will not be completed before the exception is handled.
3. After the exception-handling routine in the operating system receives control, it immediately saves the PC of the faulting instruction. This value will be used to return from the exception later.

After the exception has been handled, special instructions return the processor from the exception by reloading the PCs and restarting the instruction stream (using the exception return in RISC V). If the pipeline can be stopped so that the instructions just before the faulting instruction are completed and those after it can be restarted from scratch, the pipeline is said to have *precise exceptions*. Ideally, the faulting instruction would not have changed the state, and correctly handling some exceptions requires that the faulting instruction have no effects. For other exceptions,

such as floating-point exceptions, the faulting instruction on some processors writes its result before the exception can be handled. In such cases, the hardware must be prepared to retrieve the source operands, even if the destination is identical to one of the source operands. Because floating-point operations may run for many cycles, it is highly likely that some other instruction may have written the source operands (as we will see in the next section, floating-point operations often complete out of order). To overcome this, many recent high-performance processors have introduced two modes of operation. One mode has precise exceptions and the other (fast or performance mode) does not. Of course, the precise exception mode is slower, since it allows less overlap among floating-point instructions.

Supporting precise exceptions is a requirement in many systems, while in others it is "just" valuable because it simplifies the operating system interface. At a minimum, any processor with demand paging or IEEE arithmetic trap handlers must make its exceptions precise, either in the hardware or with some software support. For integer pipelines, the task of creating precise exceptions is easier, and accommodating virtual memory strongly motivates the support of precise exceptions for memory references. In practice, these reasons have led designers and architects to always provide precise exceptions for the integer pipeline. In this section we describe how to implement precise exceptions for the RISC V integer pipeline. We will describe techniques for handling the more complex challenges arising in the floating-point pipeline in Section C.5.

Exceptions in RISC V

Figure C.27 shows the RISC V pipeline stages and which problem exceptions might occur in each stage. With pipelining, multiple exceptions may occur in the same clock cycle because there are multiple instructions in execution. For example, consider this instruction sequence:

`ld`	IF	ID	EX	MEM	WB	
`add`		IF	ID	EX	MEM	WB

Pipeline stage	Problem exceptions occurring
IF	Page fault on instruction fetch; misaligned memory access; memory protection violation
ID	Undefined or illegal opcode
EX	Arithmetic exception
MEM	Page fault on data fetch; misaligned memory access; memory protection violation
WB	None

Figure C.27 Exceptions that may occur in the RISC V pipeline. Exceptions raised from instruction or data memory access account for six out of eight cases.

This pair of instructions can cause a data page fault and an arithmetic exception at the same time, because the `ld` is in the MEM stage while the `add` is in the EX stage. This case can be handled by dealing with only the data page fault and then restarting the execution. The second exception will reoccur (but not the first, if the software is correct), and when the second exception occurs it can be handled independently.

In reality, the situation is not as straightforward as this simple example. Exceptions may occur out of order; that is, an instruction may cause an exception before an earlier instruction causes one. Consider again the preceding sequence of instructions, `ld` followed by `add`. The `ld` can get a data page fault, seen when the instruction is in MEM, and the `add` can get an instruction page fault, seen when the `add` instruction is in IF. The instruction page fault will actually occur first, even though it is caused by a later instruction!

Because we are implementing precise exceptions, the pipeline is required to handle the exception caused by the `ld` instruction first. To explain how this works, let's call the instruction in the position of the `ld` instruction *i*, and the instruction in the position of the `add` instruction $i+1$. The pipeline cannot simply handle an exception when it occurs in time, because that will lead to exceptions occurring out of the unpipelined order. Instead, the hardware posts all exceptions caused by a given instruction in a status vector associated with that instruction. The exception status vector is carried along as the instruction goes down the pipeline. Once an exception indication is set in the exception status vector, any control signal that may cause a data value to be written is turned off (this includes both register writes and memory writes). Because a store can cause an exception during MEM, the hardware must be prepared to prevent the store from completing if it raises an exception.

When an instruction enters WB (or is about to leave MEM), the exception status vector is checked. If any exceptions are posted, they are handled in the order in which they would occur in time on an unpipelined processor—the exception corresponding to the earliest instruction (and usually the earliest pipe stage for that instruction) is handled first. This guarantees that all exceptions will be seen on instruction *i* before any are seen on $i+1$. Of course, any action taken in earlier pipe stages on behalf of instruction *i* may be invalid, but because writes to the register file and memory were disabled, no state could have been changed. As we will see in Section C.5, maintaining this precise model for FP operations is much harder.

In the next subsection we describe problems that arise in implementing exceptions in the pipelines of processors with more powerful, longer-running instructions.

Instruction Set Complications

No RISC V instruction has more than one result, and our RISC V pipeline writes that result only at the end of an instruction's execution. When an instruction is guaranteed to complete, it is called *committed*. In the RISC V integer pipeline, all instructions are committed when they reach the end of the MEM stage (or beginning of WB) and no instruction updates the state before that stage. Thus, precise exceptions

are straightforward. Some processors have instructions that change the state in the middle of the instruction execution, before the instruction and its predecessors are guaranteed to complete. For example, autoincrement addressing modes in the IA-32 architecture cause the update of registers in the middle of an instruction execution. In such a case, if the instruction is aborted because of an exception, it will leave the processor state altered. Although we know which instruction caused the exception, without additional hardware support the exception will be imprecise because the instruction will be half finished. Restarting the instruction stream after such an imprecise exception is difficult. Alternatively, we could avoid updating the state before the instruction commits, but this may be difficult or costly, because there may be dependences on the updated state: consider a VAX instruction that autoincrements the same register multiple times. Thus, to maintain a precise exception model, most processors with such instructions have the ability to back out any state changes made before the instruction is committed. If an exception occurs, the processor uses this ability to reset the state of the processor to its value before the interrupted instruction started. In the next section, we will see that a more powerful RISC V floating-point pipeline can introduce similar problems, and Section C.7 introduces techniques that substantially complicate exception handling.

A related source of difficulties arises from instructions that update memory state during execution, such as the string copy operations on the Intel architecture or IBM 360 (see Appendix K). To make it possible to interrupt and restart these instructions, the instructions are defined to use the general-purpose registers as working registers. Thus, the state of the partially completed instruction is always in the registers, which are saved on an exception and restored after the exception, allowing the instruction to continue.

A different set of difficulties arises from odd bits of state that may create additional pipeline hazards or may require extra hardware to save and restore. Condition codes are a good example of this. Many processors set the condition codes implicitly as part of the instruction. This approach has advantages, because condition codes decouple the evaluation of the condition from the actual branch. However, implicitly set condition codes can cause difficulties in scheduling any pipeline delays between setting the condition code and the branch, because most instructions set the condition code and cannot be used in the delay slots between the condition evaluation and the branch.

Additionally, in processors with condition codes, the processor must decide when the branch condition is fixed. This involves finding out when the condition code has been set for the last time before the branch. In most processors with implicitly set condition codes, this is done by delaying the branch condition evaluation until all previous instructions have had a chance to set the condition code.

Of course, architectures with explicitly set condition codes allow the delay between condition test and the branch to be scheduled; however, pipeline control must still track the last instruction that sets the condition code to know when the branch condition is decided. In effect, the condition code must be treated as an operand that requires hazard detection for RAW hazards with branches, just as RISC V must do on the registers.

A final thorny area in pipelining is multicycle operations. Imagine trying to pipeline a sequence of x86 instructions such as this:

```
mov       BX, AX          ; moves between registers
add       42(BX+SI),BX;adds memory contents and register
                          ;  to same memory location
sub       BX,AX           ;subtracts registers
rep movsb                 ;moves a character string of
                          ; length given by register CX
```

Although none of these instructions is particularly long (an x86 instruction can be up to 15 bytes), they do differ radically in the number of clock cycles they will require, from as low as one up to hundreds of clock cycles. These instructions also require different numbers of data memory accesses, from zero to possibly hundreds. The data hazards are very complex and occur both between and within instructions (nothing prevents the movsb from having an overlapping source and destination!). The simple solution of making all instructions execute for the same number of clock cycles is unacceptable because it introduces an enormous number of hazards and bypass conditions and makes an immensely long pipeline. Pipelining the x86 at the instruction level is difficult, but a clever solution was found, similar to one used for the VAX. They pipeline the *microinstruction* execution; a microinstruction is a simple instruction used in sequences to implement a more complex instruction set. Because the microinstructions are simple (they look a lot like RISC V), the pipeline control is much easier. Since 1995, all Intel IA-32 microprocessors have used this strategy of converting the IA-32 instructions into microoperations, and then pipelining the microoperations. In fact, this approach is even used for some of the more complex instructions in the ARM architecture.

In comparison, load-store processors have simple operations with similar amounts of work and pipeline more easily. If architects realize the relationship between instruction set design and pipelining, they can design architectures for more efficient pipelining. In the next section, we will see how the RISC V pipeline deals with long-running instructions, specifically floating-point operations.

For many years, the interaction between instruction sets and implementations was believed to be small, and implementation issues were not a major focus in designing instruction sets. In the 1980s, it became clear that the difficulty and inefficiency of pipelining could both be increased by instruction set complications. In the 1990s, all companies moved to simpler instructions sets with the goal of reducing the complexity of aggressive implementations.

C.5 Extending the RISC V Integer Pipeline to Handle Multicycle Operations

We now want to explore how our RISC V pipeline can be extended to handle floating-point operations. This section concentrates on the basic approach and

the design alternatives, closing with some performance measurements of a RISC V floating-point pipeline.

It is impractical to require that all RISC V FP operations complete in 1 clock cycle, or even in 2. Doing so would mean accepting a slow clock or using enormous amounts of logic in the FP units, or both. Instead, the FP pipeline will allow for a longer latency for operations. This is easier to grasp if we imagine the FP instructions as having the same pipeline as the integer instructions, with two important changes. First, the EX cycle may be repeated as many times as needed to complete the operation—the number of repetitions can vary for different operations. Second, there may be multiple FP functional units. A stall will occur if the instruction to be issued will cause either a structural hazard for the functional unit it uses or a data hazard.

For this section, let's assume that there are four separate functional units in our RISC V implementation:

1. The main integer unit that handles loads and stores, integer ALU operations, and branches
2. FP and integer multiplier
3. FP adder that handles FP add, subtract, and conversion
4. FP and integer divider

If we also assume that the execution stages of these functional units are not pipelined, then Figure C.28 shows the resulting pipeline structure. Because EX is not pipelined, no other instruction using that functional unit may issue until the previous instruction leaves EX. Moreover, if an instruction cannot proceed to the EX stage, the entire pipeline behind that instruction will be stalled.

In reality, the intermediate results are probably not cycled around the EX unit as Figure C.28 suggests; instead, the EX pipeline stage has some number of clock delays larger than 1. We can generalize the structure of the FP pipeline shown in Figure C.28 to allow pipelining of some stages and multiple ongoing operations. To describe such a pipeline, we must define both the latency of the functional units and also the *initiation interval* or *repeat interval*. We define latency the same way we defined it earlier: the number of intervening cycles between an instruction that produces a result and an instruction that uses the result. The initiation or repeat interval is the number of cycles that must elapse between issuing two operations of a given type. For example, we will use the latencies and initiation intervals shown in Figure C.29.

With this definition of latency, integer ALU operations have a latency of 0, because the results can be used on the next clock cycle, and loads have a latency of 1, because their results can be used after one intervening cycle. Because most operations consume their operands at the beginning of EX, the latency is usually the number of stages after EX that an instruction produces a result—for example, zero stages for ALU operations and one stage for loads. The primary exception is stores, which consume the value being stored one cycle later. Hence, the latency to

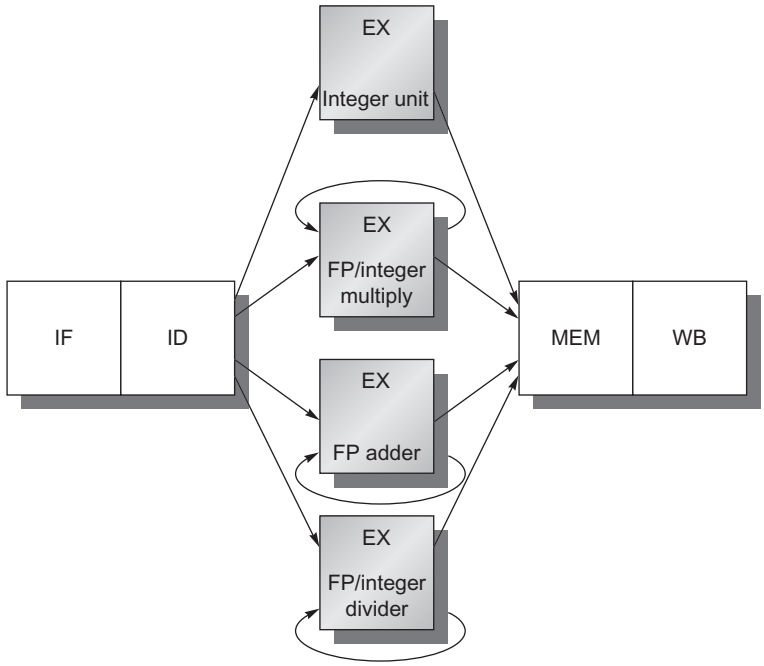


Figure C.28 The RISC V pipeline with three additional unpipelined, floating-point, functional units. Because only one instruction issues on every clock cycle, all instructions go through the standard pipeline for integer operations. The FP operations simply loop when they reach the EX stage. After they have finished the EX stage, they proceed to MEM and WB to complete execution.

Functional unit	Latency	Initiation interval
Integer ALU	0	1
Data memory (integer and FP loads)	1	1
FP add	3	1
FP multiply (also integer multiply)	6	1
FP divide (also integer divide)	24	25

Figure C.29 Latencies and initiation intervals for functional units.

a store for the value being stored, but not for the base address register, will be one cycle less. Pipeline latency is essentially equal to one cycle less than the depth of the execution pipeline, which is the number of stages from the EX stage to the stage that produces the result. Thus, for the preceding example pipeline, the number of stages in an FP add is four, while the number of stages in an FP multiply is seven. To achieve a higher clock rate, designers need to put fewer logic levels

in each pipe stage, which makes the number of pipe stages required for more complex operations larger. The penalty for the faster clock rate is thus longer latency for operations.

The example pipeline structure in Figure C.29 allows up to four outstanding FP adds, seven outstanding FP/integer multiplies, and one FP divide. Figure C.30 shows how this pipeline can be drawn by extending Figure C.28. The repeat interval is implemented in Figure C.30 by adding additional pipeline stages, which will be separated by additional pipeline registers. Because the units are independent, we name the stages differently. The pipeline stages that take multiple clock cycles, such as the divide unit, are further subdivided to show the latency of those stages. Because they are not complete stages, only one operation may be active. The pipeline structure can also be shown using the familiar diagrams from earlier in the appendix, as Figure C.31 shows for a set of independent FP operations and FP loads and stores. Naturally, the longer latency of the FP operations increases the frequency of RAW hazards and resultant stalls, as we will see later in this section.

The structure of the pipeline in Figure C.30 requires the introduction of the additional pipeline registers (e.g.,A1/A2, A2/A3, A3/A4) and the modification of the connections to those registers. The ID/EX register must be expanded to

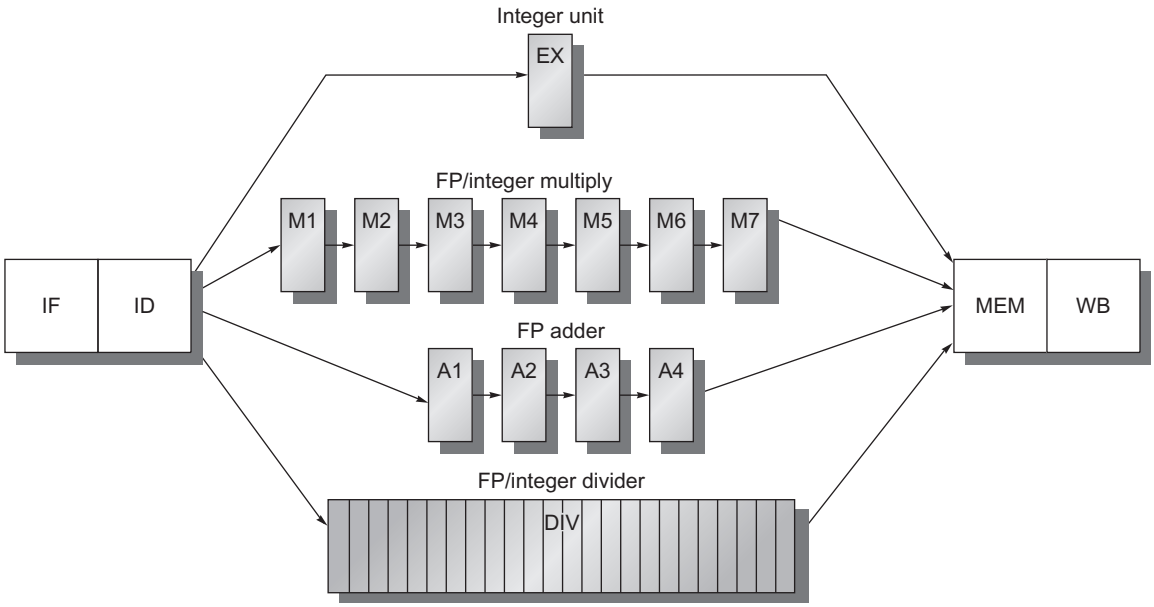


Figure C.30 A pipeline that supports multiple outstanding FP operations. The FP multiplier and adder are fully pipelined and have a depth of seven and four stages, respectively. The FP divider is not pipelined, but requires 24 clock cycles to complete. The latency in instructions between the issue of an FP operation and the use of the result of that operation without incurring a RAW stall is determined by the number of cycles spent in the execution stages. For example, the fourth instruction after an FP add can use the result of the FP add. For integer ALU operations, the depth of the execution pipeline is always one and the next instruction can use the results.

`fmul.d`	IF	ID	*M1*	M2	M3	M4	M5	M6	**M7**	MEM	WB
`fadd.d`		IF	ID	*A1*	A2	A3	**A4**	**MEM**	WB		
`fadd.d`			IF	ID	*EX*	**MEM**	WB				
`fsd`				IF	ID	*EX*	**MEM**	WB			

Figure C.31 The pipeline timing of a set of independent FP operations. The stages in italics show where data are needed, while the stages in bold show where a result is available. FP loads and stores use a 64-bit path to memory so that the pipelining timing is just like an integer load or store.

connect ID to EX, DIV, M1, and A1; we can refer to the portion of the register associated with one of the next stages with the notation ID/EX, ID/DIV, ID/M1, or ID/A1. The pipeline register between ID and all the other stages may be thought of as logically separate registers and may, in fact, be implemented as separate registers. Because only one operation can be in a pipe stage at a time, the control information can be associated with the register at the head of the stage.

Hazards and Forwarding in Longer Latency Pipelines

There are a number of different aspects to the hazard detection and forwarding for a pipeline like that shown in Figure C.30.

1. Because the divide unit is not fully pipelined, structural hazards can occur. These will need to be detected and issuing instructions will need to be stalled.
2. Because the instructions have varying running times, the number of register writes required in a cycle can be larger than 1.
3. Write after write (WAW) hazards are possible, because instructions no longer reach WB in order. Note that write after read (WAR) hazards are not possible, because the register reads always occur in ID.
4. Instructions can complete in a different order than they were issued, causing problems with exceptions; we deal with this in the next subsection.
5. Because of longer latency of operations, stalls for RAW hazards will be more frequent.

The increase in stalls arising from longer operation latencies is fundamentally the same as that for the integer pipeline. Before describing the new problems that arise in this FP pipeline and looking at solutions, let's examine the potential impact of RAW hazards. Figure C.32 shows a typical FP code sequence and the resultant stalls. At the end of this section, we'll examine the performance of this FP pipeline for our SPEC subset.

Now look at the problems arising from writes, described as (2) and (3) in the earlier list. If we assume that the FP register file has one write port, sequences of FP operations, as well as an FP load together with FP operations, can cause conflicts

	Clock cycle number																
Instruction	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**	**16**	**17**
`fld f4,0(x2)`	IF	ID	EX	MEM	WB												
`fmul.d f0,f4,f6`		IF	ID	Stall	M1	M2	M3	M4	M5	M6	M7	MEM	WB				
`fadd.d f2,f0,f8`			IF	Stall	ID	Stall	Stall	Stall	Stall	Stall	Stall	A1	A2	A3	A4	MEM	WB
`fsd f2,0(x2)`					IF	Stall	Stall	Stall	Stall	Stall	Stall	ID	EX	Stall	Stall	Stall	MEM

Figure C.32 A typical FP code sequence showing the stalls arising from RAW hazards. The longer pipeline substantially raises the frequency of stalls versus the shallower integer pipeline. Each instruction in this sequence is dependent on the previous and proceeds as soon as data are available, which assumes the pipeline has full bypassing and forwarding. The `fsd` must be stalled an extra cycle so that its MEM does not conflict with the `fadd.d`. Extra hardware could easily handle this case.

for the register write port. Consider the pipeline sequence shown in Figure C.33. In clock cycle 11, all three instructions will reach WB and want to write the register file. With only a single register file write port, the processor must serialize the instruction completion. This single register port represents a structural hazard. We could increase the number of write ports to solve this, but that solution may be unattractive because the additional write ports would be used only rarely. This is because the maximum steady-state number of write ports needed is 1. Instead, we choose to detect and enforce access to the write port as a structural hazard.

There are two different ways to implement this interlock. The first is to track the use of the write port in the ID stage and to stall an instruction before it issues, just as we would for any other structural hazard. Tracking the use of the write port can be done with a shift register that indicates when already-issued instructions will use the register file. If the instruction in ID needs to use the register file at the same time

	Clock cycle number										
Instruction	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**
`fmul.d f0,f4,f6`	IF	ID	M1	M2	M3	M4	M5	M6	M7	MEM	WB
`...`		IF	ID	EX	MEM	WB					
`...`			IF	ID	EX	MEM	WB				
`fadd.d f2,f4,f6`				IF	ID	A1	A2	A3	A4	MEM	WB
`...`					IF	ID	EX	MEM	WB		
`...`						IF	ID	EX	MEM	WB	
`fld f2,0(x2)`							IF	ID	EX	MEM	WB

Figure C.33 Three instructions want to perform a write-back to the FP register file simultaneously, as shown in clock cycle 11. This is *not* the worst case, because an earlier divide in the FP unit could also finish on the same clock. Note that although the `fmul.d`, `fadd.d`, and `fld` are in the MEM stage in clock cycle 10, only the `fld` actually uses the memory, so no structural hazard exists for MEM.

as an instruction already issued, the instruction in ID is stalled for a cycle. On each clock the reservation register is shifted 1 bit. This implementation has an advantage: It maintains the property that all interlock detection and stall insertion occurs in the ID stage. The cost is the addition of the shift register and write conflict logic. We will assume this scheme throughout this section.

An alternative scheme is to stall a conflicting instruction when it tries to enter either the MEM or WB stage. If we wait to stall the conflicting instructions until they want to enter the MEM or WB stage, we can choose to stall either instruction. A simple, though sometimes suboptimal, heuristic is to give priority to the unit with the longest latency, because that is the one most likely to have caused another instruction to be stalled for a RAW hazard. The advantage of this scheme is that it does not require us to detect the conflict until the entrance of the MEM or WB stage, where it is easy to see. The disadvantage is that it complicates pipeline control, as stalls can now arise from two places. Notice that stalling before entering MEM will cause the EX, A4, or M7 stage to be occupied, possibly forcing the stall to trickle back in the pipeline. Likewise, stalling before WB would cause MEM to back up.

Our other problem is the possibility of WAW hazards. To see that these exist, consider the example in Figure C.33. If the `fadd.d` instruction were issued one cycle earlier and had a destination of `f2`, then it would create a WAW hazard, because it would write `f2` one cycle earlier than the `fadd.d`. Note that this hazard only occurs when the result of the `fadd.d` is overwritten *without* any instruction ever using it! If there were a use of f2 between the `fadd.d` and the `fadd.d`, the pipeline would need to be stalled for a RAW hazard, and the `fadd.d` would not issue until the `fadd.d` was completed. We could argue that, for our pipeline, WAW hazards only occur when a useless instruction is executed, but we must still detect them and make sure that the result of the `fadd.d` appears in `f2` when we are done. (As we will see in Section C.8, such sequences sometimes *do* occur in reasonable code.)

There are two possible ways to handle this WAW hazard. The first approach is to delay the issue of the load instruction until the `fadd.d` enters MEM. The second approach is to stamp out the result of the `fadd.d` by detecting the hazard and changing the control so that the `fadd.d` does not write its result. Then the `fadd.d` can issue right away. Because this hazard is rare, either scheme will work fine—you can pick whatever is simpler to implement. In either case, the hazard can be detected during ID when the `fadd.d` is issuing, and stalling the `fadd.d` or making the `fadd.d` a no-op is easy. The difficult situation is to detect that the `fadd.d` might finish before the `fadd.d`, because that requires knowing the length of the pipeline and the current position of the `fadd.d`. Luckily, this code sequence (two writes with no intervening read) will be very rare, so we can use a simple solution: if an instruction in ID wants to write the same register as an instruction already issued, do not issue the instruction to EX. In Section C.7, we will see how additional hardware can eliminate stalls for such hazards. First, let's put together the pieces for implementing the hazard and issue logic in our FP pipeline.

In detecting the possible hazards, we must consider hazards among FP instructions, as well as hazards between an FP instruction and an integer instruction. Except for FP loads-stores and FP-integer register moves, the FP and integer registers are distinct. All integer instructions operate on the integer registers, while the FP operations operate only on their own registers. Thus, we need only consider FP loads-stores and FP register moves in detecting hazards between FP and integer instructions. This simplification of pipeline control is an additional advantage of having separate register files for integer and floating-point data. (The main advantages are a doubling of the number of registers, without making either set larger, and an increase in bandwidth without adding more ports to either set. The main disadvantage, beyond the need for an extra register file, is the small cost of occasional moves needed between the two register sets.) Assuming that the pipeline does all hazard detection in ID, there are three checks that must be performed before an instruction can issue:

1. *Check for structural hazards*—Wait until the required functional unit is not busy (this is only needed for divides in this pipeline) and make sure the register write port is available when it will be needed.
2. *Check for a RAW data hazard*—Wait until the source registers are not listed as pending destinations in a pipeline register that will not be available when this instruction needs the result. A number of checks must be made here, depending on both the source instruction, which determines when the result will be available, and the destination instruction, which determines when the value is needed. For example, if the instruction in ID is an FP operation with source register f2, then f2 cannot be listed as a destination in ID/A1, A1/A2, or A2/A3, which correspond to FP add instructions that will not be finished when the instruction in ID needs a result. (ID/A1 is the portion of the output register of ID that is sent to A1.) Divide is somewhat more tricky, if we want to allow the last few cycles of a divide to be overlapped, because we need to handle the case when a divide is close to finishing as special. In practice, designers might ignore this optimization in favor of a simpler issue test.
3. *Check for a WAW data hazard*—Determine if any instruction in A1, … , A4, D, M1, … , M7 has the same register destination as this instruction. If so, stall the issue of the instruction in ID.

Although the hazard detection is more complex with the multicycle FP operations, the concepts are the same as for the RISC V integer pipeline. The same is true for the forwarding logic. The forwarding can be implemented by checking if the destination register in any of the EX/MEM, A4/MEM, M7/MEM, D/MEM, or MEM/WB registers is one of the source registers of a floating-point instruction. If so, the appropriate input multiplexer will have to be enabled so as to choose the forwarded data. In the exercises, you will have the opportunity to specify the logic for the RAW and WAW hazard detection as well as for forwarding.

Multicycle FP operations also introduce problems for our exception mechanisms, which we deal with next.

Maintaining Precise Exceptions

Another problem caused by these long-running instructions can be illustrated with the following sequence of code:

```
fdiv.d      f0,f2,f4
fadd.d      f10,f10,f8
fsub.d      f12,f12,f14
```

This code sequence looks straightforward; there are no dependences. A problem arises, however, because an instruction issued early may complete after an instruction issued later. In this example, we can expect `fadd.d` and `fsub.d` to complete *before* the `fdiv.d` completes. This is called *out-of-order completion* and is common in pipelines with long-running operations (see Section C.7). Because hazard detection will prevent any dependence among instructions from being violated, why is out-of-order completion a problem? Suppose that the `fsub.d` causes a floating-point arithmetic exception at a point where the `fadd.d` has completed but the `fdiv.d` has not. The result will be an imprecise exception, something we are trying to avoid. It may appear that this could be handled by letting the floating-point pipeline drain, as we do for the integer pipeline. But the exception may be in a position where this is not possible. For example, if the fdiv.d decided to take a floating-point-arithmetic exception after the add completed, we could not have a precise exception at the hardware level. In fact, because the fadd.d destroys one of its operands, we could not restore the state to what it was before the fdiv.d, even with software help.

This problem arises because instructions are completing in a different order than they were issued. There are four possible approaches to dealing with out-of-order completion. The first is to ignore the problem and settle for imprecise exceptions. This approach was used in the 1960s and early 1970s. It was still used in some supercomputers in thepast fifteen years, where certain classes of exceptions were not allowed or were handled by the hardware without stopping the pipeline. It is difficult to use this approach in most processors built today because of features such as virtual memory and the IEEE floating-point standard that essentially require precise exceptions through a combination of hardware and software. As mentioned earlier, some recent processors have solved this problem by introducing two modes of execution: a fast, but possibly imprecise mode and a slower, precise mode. The slower precise mode is implemented either with a mode switch or by insertion of explicit instructions that test for FP exceptions. In either case, the amount of overlap and reordering permitted in the FP pipeline is significantly restricted so that effectively only one FP instruction is active at a time. This solution was used in the DEC Alpha 21064 and 21164, in the IBM Power1 and Power2, and in the MIPS R8000.

A second approach is to buffer the results of an operation until all the operations that were issued earlier are complete. Some processors actually use this solution, but it becomes expensive when the difference in running times among operations is large, because the number of results to buffer can become large. Furthermore,

results from the queue must be bypassed to continue issuing instructions while waiting for the longer instruction. This requires a large number of comparators and a very large multiplexer.

There are two viable variations on this basic approach. The first is a *history file*, used in the CYBER 180/990. The history file keeps track of the original values of registers. When an exception occurs and the state must be rolled back earlier than some instruction that completed out of order, the original value of the register can be restored from the history file. A similar technique is used for autoincrement and autodecrement addressing on processors such as VAXes. Another approach, the *future file*, proposed by Smith and Pleszkun (1988), keeps the newer value of a register; when all earlier instructions have completed, the main register file is updated from the future file. On an exception, the main register file has the precise values for the interrupted state. In Chapter 3, we will see another approach that is needed to support speculation, a method of executing instructions before we know the outcome of previous branches.

A third technique in use is to allow the exceptions to become somewhat imprecise, but to keep enough information so that the trap-handling routines can create a precise sequence for the exception. This means knowing what operations were in the pipeline and their PCs. Then, after handling the exception, the software finishes any instructions that precede the latest instruction completed, and the sequence can restart. Consider the following worst-case code sequence:

Instruction$_1$—A long-running instruction that eventually interrupts execution.

Instruction$_2$, … , Instruction$_{n-1}$—A series of instructions that are not completed.

Instruction$_n$—An instruction that is finished.

Given the PCs of all the instructions in the pipeline and the exception return PC, the software can find the state of instruction1 and instruction$_n$. Because instruction$_n$ has completed, we will want to restart execution at instruction$_{n+1}$. After handling the exception, the software must simulate the execution of instruction$_1$, … , instruction$_{n-1}$. Then we can return from the exception and restart at instruction$_{n+1}$. The complexity of executing these instructions properly by the handler is the major difficulty of this scheme.

There is an important simplification for simple RISC V-like pipelines: If instruction$_2$, … , instruction$_n$ are all integer instructions, we know that if instruction$_n$ has completed then all of instruction$_2$, … , instruction$_{n-1}$ have also completed. Thus, only FP operations need to be handled. To make this scheme tractable, the number of floating-point instructions that can be overlapped in execution can be limited. For example, if we only overlap two instructions, then only the interrupting instruction need be completed by software. This restriction may reduce the potential throughput if the FP pipelines are deep or if there are a significant number of FP functional units. This approach is used in some SPARC implementations to allow overlap of floating-point and integer operations.

The final technique is a hybrid scheme that allows the instruction issue to continue only if it is certain that all the instructions before the issuing instruction will complete without causing an exception. This guarantees that when an exception occurs, no instructions after the interrupting one will be completed and all of the instructions before the interrupting one can be completed. This sometimes means stalling the processor to maintain precise exceptions. To make this scheme work, the floating-point functional units must determine if an exception is possible early in the EX stage (in the first 3 clock cycles in the RISC V pipeline), so as to prevent further instructions from completing. This scheme is used in the MIPS R2000/3000, the R4000, and the Intel Pentium. It is discussed further in Appendix J.

Performance of a Simple RISC V FP Pipeline

The RISC V FP pipeline of Figure C.30 on page C.48 can generate both structural stalls for the divide unit and stalls for RAW hazards (it also can have WAW hazards, but this rarely occurs in practice). Figure C.34 shows the number of stall cycles for each type of floating-point operation on a per-instance basis (i.e., the first bar for each FP benchmark shows the number of FP result stalls for each FP add, subtract, or convert). As we might expect, the stall cycles per operation track the latency of the FP operations, varying from 46% to 59% of the latency of the functional unit.

Figure C.35 gives the complete breakdown of integer and FP stalls for five SPECfp benchmarks. There are four classes of stalls shown: FP result stalls, FP compare stalls, load and branch delays, and FP structural delays. Branch delay stalls, which would be small with a one cycle delay and even a modest branch predictor, are not included. The total number of stalls per instruction varies from 0.65 to 1.21.

C.6 Putting It All Together: The MIPS R4000 Pipeline

In this section, we look at the pipeline structure and performance of the MIPS R4000 processor family, which includes the 4400. The MIPS architecture and RISC V are very similar, differing only in a few instructions, including a delayed branch in the MIPS ISA. The R4000 implements MIPS64 but uses a deeper pipeline than that of our five-stage design both for integer and FP programs. This deeper pipeline allows it to achieve higher clock rates by decomposing the five-stage integer pipeline into eight stages. Because cache access is particularly time critical, the extra pipeline stages come from decomposing the memory access. This type of deeper pipelining is sometimes called *superpipelining*.

Figure C.36 shows the eight-stage pipeline structure using an abstracted version of the data path. Figure C.37 shows the overlap of successive instructions in the pipeline. Notice that, although the instruction and data memory occupy multiple cycles, they are fully pipelined, so that a new instruction can start on every clock.

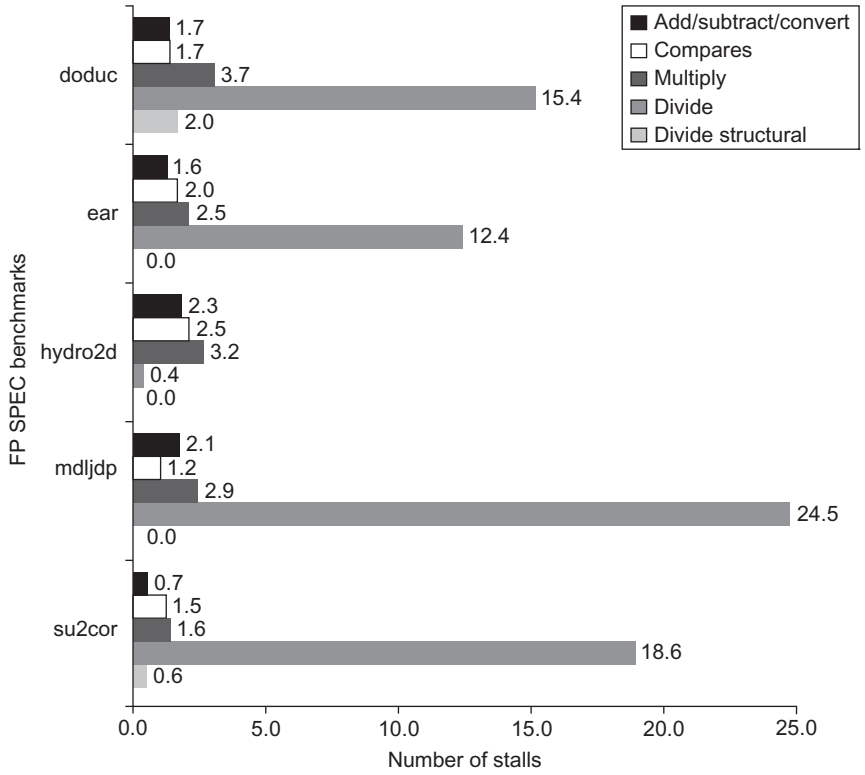


Figure C.34 Stalls per FP operation for each major type of FP operation for the SPEC89 FP benchmarks. Except for the divide structural hazards, these data do not depend on the frequency of an operation, only on its latency and the number of cycles before the result is used. The number of stalls from RAW hazards roughly tracks the latency of the FP unit. For example, the average number of stalls per FP add, subtract, or convert is 1.7 cycles, or 56% of the latency (three cycles). Likewise, the average number of stalls for multiplies and divides are 2.8 and 14.2, respectively, or 46% and 59% of the corresponding latency. Structural hazards for divides are rare, because the divide frequency is low.

In fact, the pipeline uses the data before the cache hit detection is complete; Chapter 3 discusses how this can be done in more detail.

The function of each stage is as follows:

- IF—First half of instruction fetch; PC selection actually happens here, together with initiation of instruction cache access.
- IS—Second half of instruction fetch, complete instruction cache access.
- RF—Instruction decode and register fetch, hazard checking, and instruction cache hit detection.

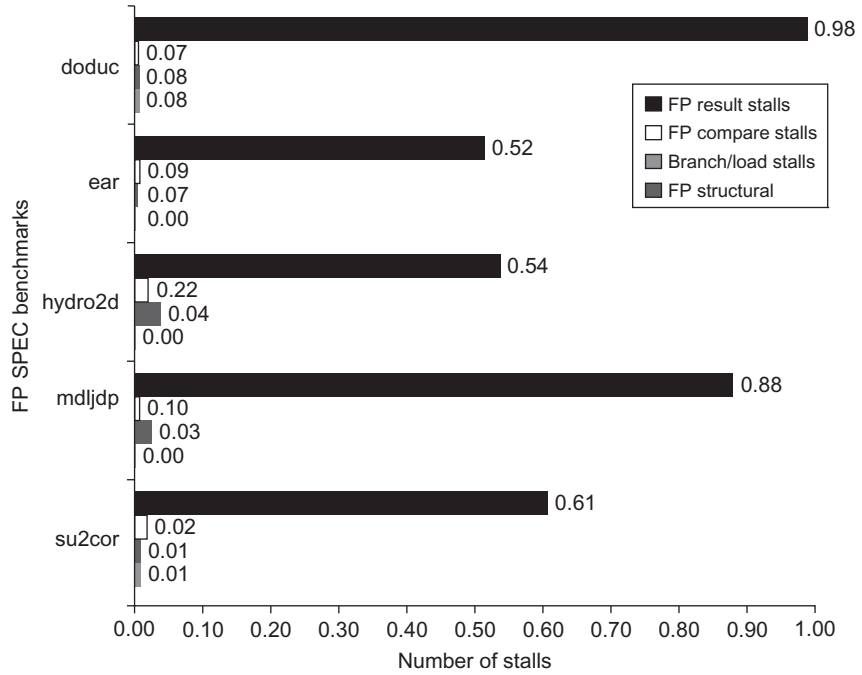


Figure C.35 The stalls occurring for the a simple RISC V FP pipeline for five of the SPEC89 FP benchmarks. The total number of stalls per instruction ranges from 0.65 for su2cor to 1.21 for doduc, with an average of 0.87. FP result stalls dominate in all cases, with an average of 0.71 stalls per instruction, or 82% of the stalled cycles. Compares generate an average of 0.1 stalls per instruction and are the second largest source. The divide structural hazard is only significant for doduc. Branch stalls are not accounted for, but would be small.

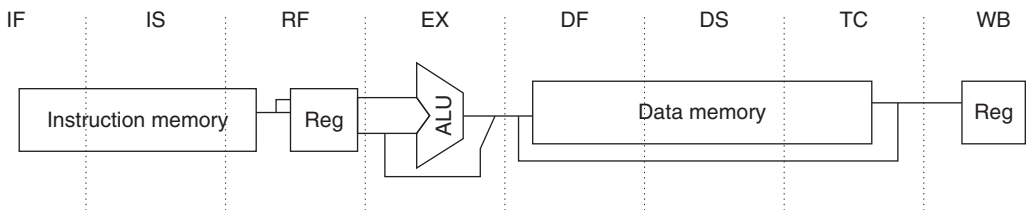


Figure C.36 The eight-stage pipeline structure of the R4000 uses pipelined instruction and data caches. The pipe stages are labeled and their detailed function is described in the text. The vertical dashed lines represent the stage boundaries as well as the location of pipeline latches. The instruction is actually available at the end of IS, but the tag check is done in RF, while the registers are fetched. Thus, we show the instruction memory as operating through RF. The TC stage is needed for data memory access, because we cannot write the data into the register until we know whether the cache access was a hit or not.

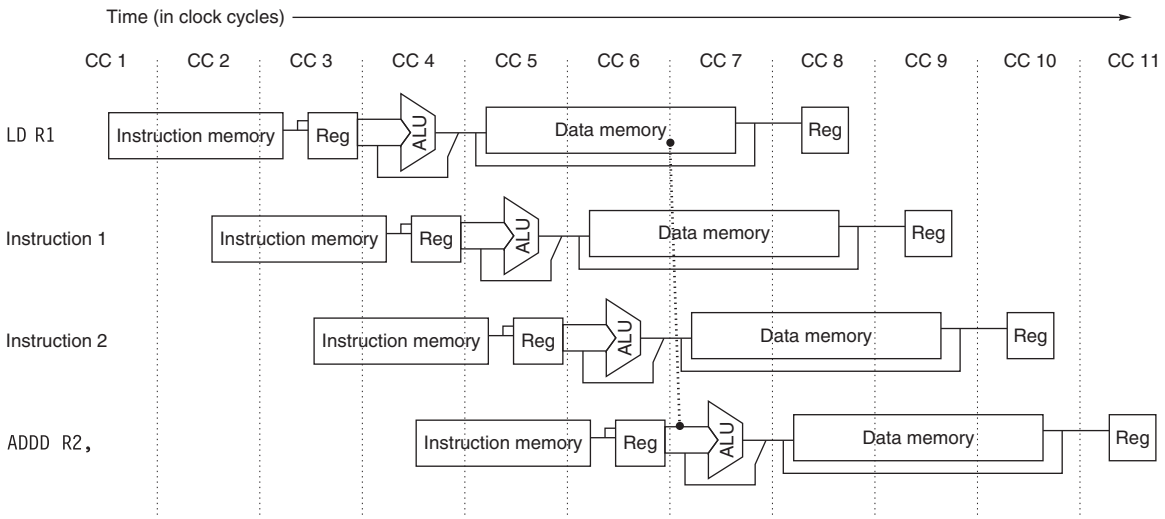


Figure C.37 The structure of the R4000 integer pipeline leads to a x1 load delay. A x1 delay is possible because the data value is available at the end of DS and can be bypassed. If the tag check in TC indicates a miss, the pipeline is backed up a cycle, when the correct data are available.

- EX—Execution, which includes effective address calculation, ALU operation, and branch-target computation and condition evaluation.
- DF—Data fetch, first half of data cache access.
- DS—Second half of data fetch, completion of data cache access.
- TC—Tag check, to determine whether the data cache access hit.
- WB—Write-back for loads and register-register operations.

In addition to substantially increasing the amount of forwarding required, this longer-latency pipeline increases both the load and branch delays. Figure C.37 shows that load delays are two cycles, because the data value is available at the end of DS. Figure C.38 shows the shorthand pipeline schedule when a use immediately follows a load. It shows that forwarding is required for the result of a load instruction to a destination that is three or four cycles later.

Figure C.39 shows that the basic branch delay is three cycles, because the branch condition is computed during EX. The MIPS architecture has a single-cycle delayed branch. The R4000 uses a predicted-not-taken strategy for the remaining two cycles of the branch delay. As Figure C.40 shows, untaken branches are simply one-cycle delayed branches, while taken branches have a one-cycle delay slot followed by two idle cycles. The instruction set provides a branch-likely instruction, which we described earlier and which helps in filling the branch delay slot.

	Clock number								
Instruction number	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
`ld x1,...`	IF	IS	RF	EX	DF	DS	TC	WB	
`add x2,x1,...`		IF	IS	RF	Stall	Stall	EX	DF	DS
`sub x3,x1,...`			IF	IS	Stall	Stall	RF	EX	DF
`or x4,x1,...`				IF	Stall	Stall	IS	RF	EX

Figure C.38 A load instruction followed by an immediate use results in a x1 stall. Normal forwarding paths can be used after two cycles, so the `add` and `sub` get the value by forwarding after the stall. The `or` instruction gets the value from the register file. Because the two instructions after the load could be independent and hence not stall, the bypass can be to instructions that are three or four cycles after the load.

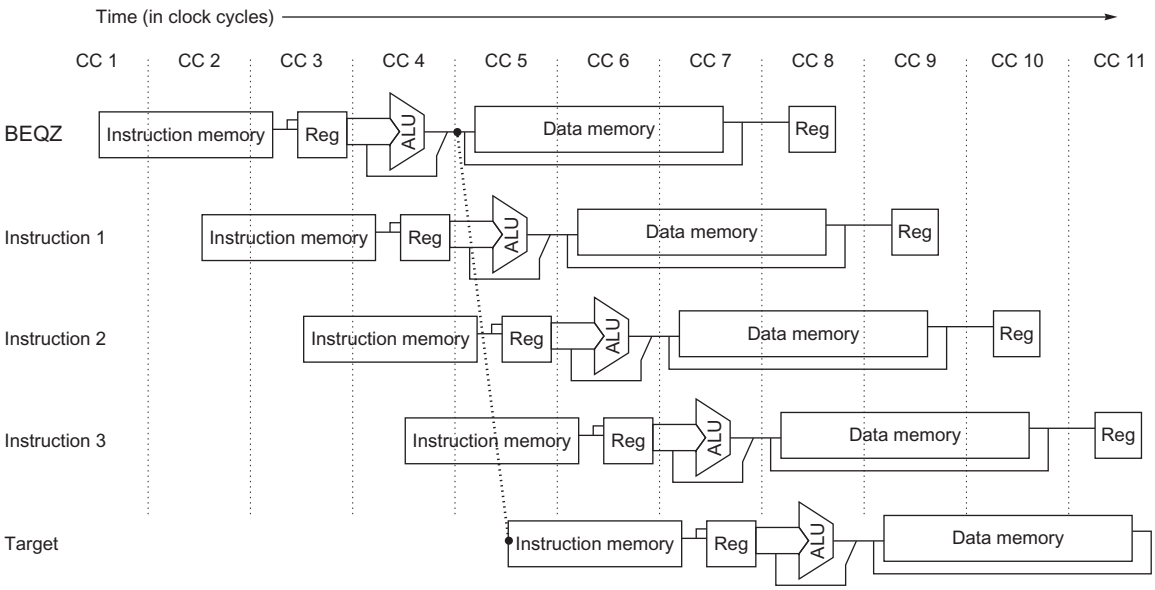


Figure C.39 The basic branch delay is three cycles, because the condition evaluation is performed during EX.

Pipeline interlocks enforce both the x1 branch stall penalty on a taken branch and any data hazard stall that arises from use of a load result. After the R4000, all implementations of MIPS processor made use of dynamic branch prediction.

In addition to the increase in stalls for loads and branches, the deeper pipeline increases the number of levels of forwarding for ALU operations. In our RISC V five-stage pipeline, forwarding between two register-register ALU instructions could happen from the ALU/MEM or the MEM/WB registers. In the R4000 pipeline, there are four possible sources for an ALU bypass: EX/DF, DF/DS, DS/TC, and TC/WB.

	Clock number								
Instruction number	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
Branch instruction	IF	IS	RF	EX	DF	DS	TC	WB	
Delay slot		IF	IS	RF	EX	DF	DS	TC	WB
Stall			Stall	Stall	Stall	Stall	Stall	Stall	Stall
Stall				Stall	Stall	Stall	Stall	Stall	Stall
Branch target					IF	IS	RF	EX	DF
Branch instruction	IF	IS	RF	EX	DF	DS	TC	WB	
Delay slot		IF	IS	RF	EX	DF	DS	TC	WB
Branch instruction + 2			IF	IS	RF	EX	DF	DS	TC
Branch instruction + 3				IF	IS	RF	EX	DF	DS

Figure C.40 A taken branch, shown in the top portion of the figure, has a one-cycle delay slot followed by a x1 stall, while an untaken branch, shown in the bottom portion, has simply a one-cycle delay slot. The branch instruction can be an ordinary delayed branch or a branch-likely, which cancels the effect of the instruction in the delay slot if the branch is untaken.

The Floating-Point Pipeline

The R4000 floating-point unit consists of three functional units: a floating-point divider, a floating-point multiplier, and a floating-point adder. The adder logic is used on the final step of a multiply or divide. Double-precision FP operations can take from 2 cycles (for a negate) up to 112 cycles (for a square root). In addition, the various units have different initiation rates. The FP functional unit can be thought of as having eight different stages, listed in Figure C.41; these stages are combined in different orders to execute various FP operations.

There is a single copy of each of these stages, and various instructions may use a stage zero or more times and in different orders. Figure C.42 shows the latency, initiation rate, and pipeline stages used by the most common double-precision FP operations.

Stage	Functional unit	Description
A	FP adder	Mantissa add stage
D	FP divider	Divide pipeline stage
E	FP multiplier	Exception test stage
M	FP multiplier	First stage of multiplier
N	FP multiplier	Second stage of multiplier
R	FP adder	Rounding stage
S	FP adder	Operand shift stage
U		Unpack FP numbers

Figure C.41 The eight stages used in the R4000 floating-point pipelines.

FP instruction	Latency	Initiation interval	Pipe stages
Add, subtract	4	3	U, S+A, A+R, R+S
Multiply	8	4	U, E+M, M, M, M, N, N+A, R
Divide	36	35	U, A, R, D^{28}, D+A, D+R, D+A, D+R, A, R
Square root	112	111	U, E, $(A+R)^{108}$, A, R
Negate	2	1	U, S
Absolute value	2	1	U, S
FP compare	3	2	U, A, R

Figure C.42 The latencies and initiation intervals for the FP operations initiation intervals for the FP operations both depend on the FP unit stages that a given operation must use. The latency values assume that the destination instruction is an FP operation; the latencies are one cycle less when the destination is a store. The pipe stages are shown in the order in which they are used for any operation. The notation S+A indicates a clock cycle in which both the S and A stages are used. The notation D^{28} indicates that the D stage is used 28 times in a row.

From the information in Figure C.42, we can determine whether a sequence of different, independent FP operations can issue without stalling. If the timing of the sequence is such that a conflict occurs for a shared pipeline stage, then a stall will be needed. Figures C.43–C.46 show four common possible two-instruction sequences: a multiply followed by an add, an add followed by a multiply, a divide followed by an add, and an add followed by a divide. The figures show all the interesting starting positions for the second instruction and whether that second instruction will issue or stall for each position. Of course, there could be three instructions active, in which case the possibilities for stalls are much higher and the figures more complex.

Performance of the R4000 Pipeline

In this section, we examine the stalls that occur for the SPEC92 benchmarks when running on the R4000 pipeline structure. There are four major causes of pipeline stalls or losses:

1. *Load stalls*—Delays arising from the use of a load result one or two cycles after the load
2. *Branch stalls*—Two-cycle stalls on every taken branch plus unfilled or canceled branch delay slots. The version of the MIPS instruction set implemented in the R4000 supports instructions that predict a branch at compile time and cause the instruction in the branch delay slot to be canceled when the branch behavior differs from the prediction. This makes it easier to fill branch delay slots.
3. *FP result stalls*—Stalls because of RAW hazards for an FP operand
4. *FP structural stalls*—Delays because of issue restrictions arising from conflicts for functional units in the FP pipeline

		Clock cycle												
Operation	**Issue/stall**	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
Multiply	Issue	U	E+M	M	M	M	N	**N+A**	**R**					
Add	Issue		U	S+A	A+R	R+S								
	Issue			U	S+A	A+R	R+S							
	Issue				U	S+A	A+R	R+S						
	Stall					U	S+A	**A+R**	**R+S**					
	Stall						U	**S+A**	**A+R**	R+S				
	Issue							U	S+A	A+R	R+S			
	Issue								U	S+A	A+R	R+S		

Figure C.43 An FP multiply issued at clock 0 is followed by a single FP add issued between clocks 1 and 7. The second column indicates whether an instruction of the specified type stalls when it is issued *n* cycles later, where *n* is the clock cycle number in which the U stage of the second instruction occurs. The stage or stages that cause a stall are in bold. Note that this table deals with only the interaction between the multiply and *one* add issued between clocks 1 and 7. In this case, the add will stall if it is issued four or five cycles after the multiply; otherwise, it issues without stalling. Notice that the add will be stalled for two cycles if it issues in cycle 4 because on the next clock cycle it will still conflict with the multiply; if, however, the add issues in cycle 5, it will stall for only 1 clock cycle, because that will eliminate the conflicts.

		Clock cycle												
Operation	**Issue/stall**	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
Add	Issue	U	S+A	A+R	R+S									
Multiply	Issue		U	E+M	M	M	M	N	N+A	R				
	Issue			U	M	M	M	M	N	N+A	R			

Figure C.44 A multiply issuing after an add can always proceed without stalling, because the shorter instruction clears the shared pipeline stages before the longer instruction reaches them.

Figure C.47 shows the pipeline CPI breakdown for the R4000 pipeline for the 10 SPEC92 benchmarks. Figure C.48 shows the same data but in tabular form.

From the data in Figures C.47 and C.48, we can see the penalty of the deeper pipelining. The R4000's pipeline has much longer branch delays than the classic five-stage pipeline. The longer branch delay substantially increases the cycles spent on branches, especially for the integer programs with a higher branch frequency. This is the reason that almost all subsequent processors with moderate to deep pipelines (8–16 stages are typical today) employ dynamic branch predictors.

		Clock cycle											
Operation	**Issue/stall**	**25**	**26**	**27**	**28**	**29**	**30**	**31**	**32**	**33**	**34**	**35**	**36**
Divide	Issued in cycle 0…	D	D	D	D	D	**D+A**	**D+R**	**D+A**	**D+R**	**A**	**R**	
Add	Issue		U	S+A	A+R	R+S							
	Issue			U	S+A	A+R	R+S						
	Stall				U	S+A	**A+R**	**R+S**					
	Stall					U	**S+A**	**A+R**	R+S				
	Stall						U	S+A	**A+R**	**R+S**			
	Stall							U	**S+A**	**A+R**	R+S		
	Stall								U	S+A	**A+R**	**R+S**	
	Stall									U	**S+A**	**A+R**	R+S
	Issue										U	S+A	A+R
	Issue											U	S+A
	Issue												U

Figure C.45 An FP divide can cause a stall for an add that starts near the end of the divide. The divide starts at cycle 0 and completes at cycle 35; the last 10 cycles of the divide are shown. Because the divide makes heavy use of the rounding hardware needed by the add, it stalls an add that starts in any of cycles 28–33. Notice that the add starting in cycle 28 will be stalled until cycle 36. If the add started right after the divide, it would not conflict, because the add could complete before the divide needed the shared stages, just as we saw in Figure C.44 for a multiply and add. As in the earlier figure, this example assumes *exactly* one add that reaches the U stage between clock cycles 26 and 35.

		Clock cycle											
Operation	**Issue/stall**	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
Add	Issue	U	S+A	**A+R**	**R+S**									
Divide	Stall		U	**A**	**R**	D	D	D	D	D	D	D	D	D
	Issue			U	A	R	D	D	D	D	D	D	D	D
	Issue				U	A	R	D	D	D	D	D	D	D

Figure C.46 A double-precision add is followed by a double-precision divide. If the divide starts one cycle after the add, the divide stalls, but after that there is no conflict.

An interesting effect observed in the FP programs is that the latency of the FP functional units leads to more result stalls than the structural hazards, which arise both from the initiation interval limitations and from conflicts for functional units from different FP instructions. Thus, reducing the latency of FP operations should be the first target, rather than more pipelining or replication of the functional units. Of course, reducing the latency would probably increase the structural stalls, because many potential structural stalls are hidden behind data hazards.

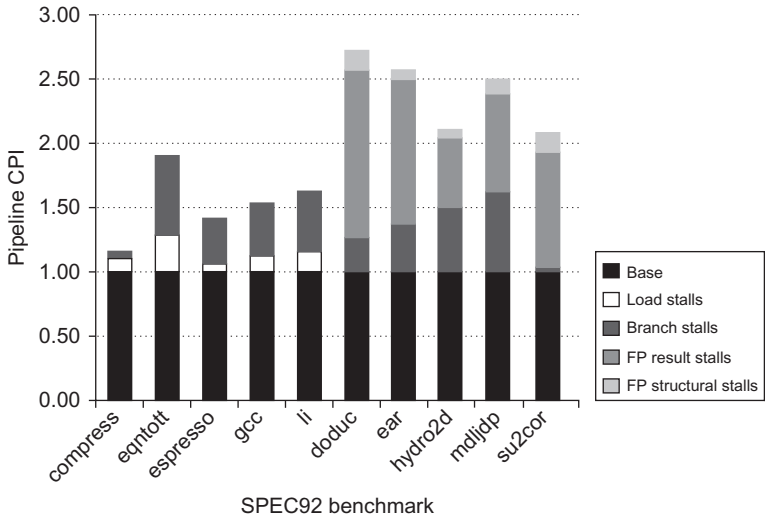


Figure C.47 The pipeline CPI for 10 of the SPEC92 benchmarks, assuming a perfect cache. The pipeline CPI varies from 1.2 to 2.8. The left-most five programs are integer programs, and branch delays are the major CPI contributor for these. The right-most five programs are FP, and FP result stalls are the major contributor for these. Figure C.48 shows the numbers used to construct this plot.

Benchmark	Pipeline CPI	Load stalls	Branch stalls	FP result stalls	FP structural stalls
Compress	1.20	0.14	0.06	0.00	0.00
Eqntott	1.88	0.27	0.61	0.00	0.00
Espresso	1.42	0.07	0.35	0.00	0.00
Gcc	1.56	0.13	0.43	0.00	0.00
Li	1.64	0.18	0.46	0.00	0.00
Integer average	1.54	0.16	0.38	0.00	0.00
Doduc	2.84	0.01	0.22	1.39	0.22
Mdljdp2	2.66	0.01	0.31	1.20	0.15
Ear	2.17	0.00	0.46	0.59	0.12
Hydro2d	2.53	0.00	0.62	0.75	0.17
Su2cor	2.18	0.02	0.07	0.84	0.26
FP average	2.48	0.01	0.33	0.95	0.18
Overall average	2.00	0.10	0.36	0.46	0.09

Figure C.48 The total pipeline CPI and the contributions of the four major sources of stalls are shown. The major contributors are FP result stalls (both for branches and for FP inputs) and branch stalls, with loads and FP structural stalls adding less.

C.7 Cross-Cutting Issues

RISC Instruction Sets and Efficiency of Pipelining

We have already discussed the advantages of instruction set simplicity in building pipelines. Simple instruction sets offer another advantage: they make it easier to schedule code to achieve efficiency of execution in a pipeline. To see this, consider a simple example: suppose we need to add two values in memory and store the result back to memory. In some sophisticated instruction sets this will take only a single instruction; in others, it will take two or three. A typical RISC architecture would require four instructions (two loads, an add, and a store). These instructions cannot be scheduled sequentially in most pipelines without intervening stalls.

With a RISC instruction set, the individual operations are separate instructions and may be individually scheduled either by the compiler (using the techniques we discussed earlier and more powerful techniques discussed in Chapter 3) or using dynamic hardware scheduling techniques (which we discuss next and in further detail in Chapter 3). These efficiency advantages, coupled with the greater ease of implementation, appear to be so significant that almost all recent pipelined implementations of complex instruction sets actually translate their complex instructions into simple RISC-like operations, and then schedule and pipeline those operations. All recent Intel processors use this approach, and it is also used in ARM processors for some of the more complex instructions.

Dynamically Scheduled Pipelines

Simple pipelines fetch an instruction and issue it, unless there is a data dependence between an instruction already in the pipeline and the fetched instruction that cannot be hidden with bypassing or forwarding. Forwarding logic reduces the effective pipeline latency so that certain dependences do not result in hazards. If there is an unavoidable hazard, then the hazard detection hardware stalls the pipeline (starting with the instruction that uses the result). No new instructions are fetched or issued until the dependence is cleared. To overcome these performance losses, the compiler can attempt to schedule instructions to avoid the hazard; this approach is called *compiler* or *static scheduling*.

Several early processors used another approach, called *dynamic scheduling*, whereby the hardware rearranges the instruction execution to reduce the stalls. This section offers a simpler introduction to dynamic scheduling by explaining the scoreboarding technique of the CDC 6600. Some readers will find it easier to read this material before plunging into the more complicated Tomasulo scheme, and the speculation approaches that extend it, both of which are covered in Chapter 3.

All the techniques discussed in this appendix so far use in-order instruction issue, which means that if an instruction is stalled in the pipeline, no later instructions can proceed. With in-order issue, if two instructions have a hazard between

them, the pipeline will stall, even if there are later instructions that are independent and would not stall.

In the RISC V pipeline developed earlier, both structural and data hazards were checked during instruction decode (ID): when an instruction could execute properly, it was issued from ID. To allow an instruction to begin execution as soon as its operands are available, even if a predecessor is stalled, we must separate the issue process into two parts: checking the structural hazards and waiting for the absence of a data hazard. We decode and issue instructions in order; however, we want the instructions to begin execution as soon as their data operands are available. Thus, the pipeline will do *out-of-order execution*, which implies *out-of-order completion*. To implement out-of-order execution, we must split the ID pipe stage into two stages:

1. *Issue*—Decode instructions, check for structural hazards.
2. *Read operands*—Wait until no data hazards, then read operands.

The IF stage proceeds the issue stage, and the EX stage follows the read operands stage, just as in the RISC V pipeline. As in the RISC V floating-point pipeline, execution may take multiple cycles, depending on the operation. Thus, we may need to distinguish when an instruction *begins execution* and when it *completes execution*; between the two times, the instruction is *in execution*. This allows multiple instructions to be in execution at the same time. In addition to these changes to the pipeline structure, we will also change the functional unit design by varying the number of units, the latency of operations, and the functional unit pipelining so as to better explore these more advanced pipelining techniques.

Dynamic Scheduling With a Scoreboard

In a dynamically scheduled pipeline, all instructions pass through the issue stage in order (in-order issue); however, they can be stalled or bypass each other in the second stage (read operands) and thus enter execution out of order. *Scoreboarding* is a technique for allowing instructions to execute out of order when there are sufficient resources and no data dependences; it is named after the CDC 6600 scoreboard, which developed this capability.

Before we see how scoreboarding could be used in the RISC V pipeline, it is important to observe that WAR hazards, which did not exist in the RISC V floating-point or integer pipelines, may arise when instructions execute out of order. For example, consider the following code sequence:

```
fdiv.d      f0,f2,f4
fadd.d      f10,f0,f8
fsub.d      f8,f8,f14
```

There is an potential WAR hazard between the `fadd.d` and the `fsub.d`: If the pipeline executes the `fsub.d` before the `fadd.d`, it will violate yield incorrect execution. Likewise, the pipeline must avoid WAW hazards (e.g.,as would

occur if the destination of the `fsub.d` were `f10`). As we will see, both these hazards are avoided in a scoreboard by stalling the later instruction involved in the hazard.

The goal of a scoreboard is to maintain an execution rate of one instruction per clock cycle (when there are no structural hazards) by executing an instruction as early as possible. Thus, when the next instruction to execute is stalled, other instructions can be issued and executed if they do not depend on any active or stalled instruction. The scoreboard takes full responsibility for instruction issue and execution, including all hazard detection. Taking advantage of out-of-order execution requires multiple instructions to be in their EX stage simultaneously. This can be achieved with multiple functional units, with pipelined functional units, or with both. Because these two capabilities—pipelined functional units and multiple functional units—are essentially equivalent for the purposes of pipeline control, we will assume the processor has multiple functional units.

The CDC 6600 had 16 separate functional units, including 4 floating-point units, 5 units for memory references, and 7 units for integer operations. On a processor for the RISC V architecture, scoreboards make sense primarily on the floating-point unit because the latency of the other functional units is very small. Let's assume that there are two multipliers, one adder, one divide unit, and a single integer unit for all memory references, branches, and integer operations. Although this example is simpler than the CDC 6600, it is sufficiently powerful to demonstrate the principles without having a mass of detail or needing very long examples. Because both RISC V and the CDC 6600 are load-store architectures, the techniques are nearly identical for the two processors. Figure C.49 shows what the processor looks like.

Every instruction goes through the scoreboard, where a record of the data dependences is constructed; this step corresponds to instruction issue and replaces part of the ID step in the RISC V pipeline. The scoreboard then determines when the instruction can read its operands and begin execution. If the scoreboard decides the instruction cannot execute immediately, it monitors every change in the hardware and decides when the instruction *can* execute. The scoreboard also controls when an instruction can write its result into the destination register. Thus, all hazard detection and resolution are centralized in the scoreboard. We will see a picture of the scoreboard later (Figure C.49 on page C.68), but first we need to understand the steps in the issue and execution segment of the pipeline.

Each instruction undergoes four steps in executing. (Because we are concentrating on the FP operations, we will not consider a step for memory access.) Let's first examine the steps informally and then look in detail at how the scoreboard keeps the necessary information that determines when to progress from one step to the next. The four steps, which replace the ID, EX, and WB steps in the standard RISC V pipeline, are as follows:

1. *Issue*—If a functional unit for the instruction is free and no other active instruction has the same destination register, the scoreboard issues the instruction to the functional unit and updates its internal data structure. This step replaces a portion of the ID step in the RISC V pipeline. By ensuring that no other active

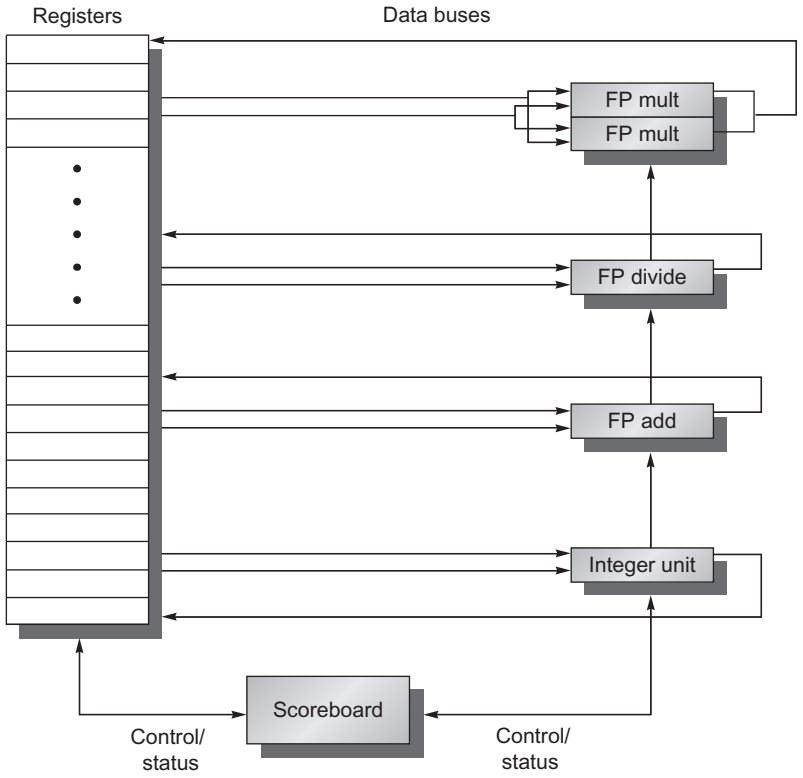


Figure C.49 The basic structure of a RISC V processor with a scoreboard. The scoreboard's function is to control instruction execution (vertical control lines). All of the data flow between the register file and the functional units over the buses (the horizontal lines, called *trunks* in the CDC 6600). There are two FP multipliers, an FP divider, an FP adder, and an integer unit. One set of buses (two inputs and one output) serves a group of functional units. We will explore scoreboarding and its extensions in more detail in Chapter 3.

functional unit wants to write its result into the destination register, we guarantee that WAW hazards cannot be present. If a structural or WAW hazard exists, then the instruction issue stalls, and no further instructions will issue until these hazards are cleared. When the issue stage stalls, it causes the buffer between instruction fetch and issue to fill; if the buffer is a single entry, instruction fetch stalls immediately. If the buffer is a queue with multiple instructions, it stalls when the queue fills.

2. *Read operands*—The scoreboard monitors the availability of the source operands. A source operand is available if no earlier issued active instruction is going to write it. When the source operands are available, the scoreboard tells

the functional unit to proceed to read the operands from the registers and begin execution. The scoreboard resolves RAW hazards dynamically in this step, and instructions may be sent into execution out of order. This step, together with issue, completes the function of the ID step in the simple RISC V pipeline.

3. *Execution*—The functional unit begins execution upon receiving operands. When the result is ready, it notifies the scoreboard that it has completed execution. This step replaces the EX step in the RISC V pipeline and takes multiple cycles in the RISC V FP pipeline.
4. *Write result*—Once the scoreboard is aware that the functional unit has completed execution, the scoreboard checks for WAR hazards and stalls the completing instruction, if necessary.

A WAR hazard exists if there is a code sequence like our earlier example with `fadd.d` and `fsub.d` that both use `f8`. In that example, we had the code

```
fdiv.d      f0,f2,f4
fadd.d      f10,f0,f8
fsub.d      f8,f8,f14
```

`fadd.d` has a source operand `f8`, which is the same register as the destination of `fsub.d`. But `fadd.d` actually depends on an earlier instruction. The scoreboard will still stall the `fsub.d` in its write result stage until `fadd.d` reads its operands. In general, then, a completing instruction cannot be allowed to write its results when:

- There is an instruction that has not read its operands that precedes (i.e., in order of issue) the completing instruction, and
- One of the operands is the same register as the result of the completing instruction.

If this WAR hazard does not exist, or when it clears, the scoreboard tells the functional unit to store its result to the destination register. This step replaces the WB step in the simple RISC V pipeline.

At first glance, it might appear that the scoreboard will have difficulty separating RAW and WAR hazards.

Because the operands for an instruction are read only when both operands are available in the register file, this scoreboard does not take advantage of forwarding. Instead, registers are only read when they are both available. This is not as large a penalty as you might initially think. Unlike our simple pipeline of earlier, instructions will write their result into the register file as soon as they complete execution (assuming no WAR hazards), rather than wait for a statically assigned write slot that may be several cycles away. The effect reduces the pipeline latency and the benefits of forwarding. There is still one additional cycle of latency that arises because the write result and read operand stages cannot overlap. We would need additional buffering to eliminate this overhead.

Based on its own data structure, the scoreboard controls the instruction progression from one step to the next by communicating with the functional units. There is a small complication, however. There are only a limited number of source operand buses and result buses to the register file, which represents a structural hazard. The scoreboard must guarantee that the number of functional units allowed to proceed into steps 2 and 4 does not exceed the number of buses available. We will not go into further detail on this, other than to mention that the CDC 6600 solved this problem by grouping the 16 functional units together into four groups and supplying a set of buses, called *data trunks*, for each group. Only one unit in a group could read its operands or write its result during a clock.

C.8 Fallacies and Pitfalls

Pitfall *Unexpected execution sequences may cause unexpected hazards.*

At first glance, WAW hazards look like they should never occur in a code sequence because no compiler would ever generate two writes to the same register without an intervening read, but they can occur when the sequence is unexpected. For example, consider a long running floating point divide that causes a trap. If the trap routine writes the same register as the divide early on, it may cause a WAW hazard, if it writes the register before the divide completes. Hardware or software must avoid this possibility.

Pitfall *Extensive pipelining can impact other aspects of a design, leading to overall worse cost-performance.*

The best example of this phenomenon comes from two implementations of the VAX, the 8600 and the 8700. When the 8600 was initially delivered, it had a cycle time of 80 ns. Subsequently, a redesigned version, called the 8650, with a 55 ns clock was introduced. The 8700 has a much simpler pipeline that operates at the microinstruction level, yielding a smaller processor with a faster clock cycle of 45 ns. The overall outcome is that the 8650 has a CPI advantage of about 20%, but the 8700 has a clock rate that is about 20% faster. Thus, the 8700 achieved the same performance with much less hardware.

Pitfall *Evaluating dynamic or static scheduling on the basis of unoptimized code.*

Unoptimized code—containing redundant loads, stores, and other operations that might be eliminated by an optimizer—is much easier to schedule than "tight" optimized code. This holds for scheduling both control delays (with delayed branches) and delays arising from RAW hazards. In gcc running on an R3000, which has a pipeline almost identical to that of Section C.1, the frequency of idle clock cycles increases by 18% from the unoptimized and scheduled code to the optimized and scheduled code. Of course, the optimized program is much faster, because it has fewer instructions. To fairly evaluate a compile-time scheduler or runtime dynamic scheduling, you must use optimized

code, because in the real system you will derive good performance from other optimizations in addition to scheduling.

C.9 Concluding Remarks

At the beginning of the 1980s, pipelining was a technique reserved primarily for supercomputers and large multimillion-dollar mainframes. By the mid-1980s, the first pipelined microprocessors appeared and helped transform the world of computing, allowing microprocessors to bypass minicomputers in performance and eventually to take on and outperform mainframes. By the early 1990s, high-end embedded microprocessors embraced pipelining, and desktops were headed toward the use of the sophisticated dynamically scheduled, multiple-issue approaches discussed in Chapter 3. The material in this appendix, which was considered reasonably advanced for graduate students when this text first appeared in 1990, is now considered basic undergraduate material and can be found in processors that cost less than $1!

C.10 Historical Perspective and References

Section M.5 (available online) features a discussion on the development of pipelining and instruction-level parallelism covering both this appendix and the material in Chapter 3. We provide numerous references for further reading and exploration of these topics.

Updated Exercises by Diana Franklin

C.1 [15/15/15/15/25/10/15] <A.2> Use the following code fragment:

```
Loop:  ld     x1,0(x2)    ;load x1 from address 0+x2
       addi   x1,x1,1     ;x1=x1+1
       sd     x1,0,(x2)   ;store x1 at address 0+x2
       addi   x2,x2,4     ;x2=x2+4
       sub    x4,x3,x2    ;x4=x3-x2
       bnez   x4,Loop     ;branch to Loop if x4!= 0
```

Assume that the initial value of x3 is x2 + 396.

a. [15] <C.2> Data hazards are caused by data dependences in the code. Whether a dependency causes a hazard depends on the machine implementation (i.e., number of pipeline stages). List all of the data dependences in the code above. Record the register, source instruction, and destination instruction; for example, there is a data dependency for register x1 from the `ld` to the `addi`.

b. [15] <C.2> Show the timing of this instruction sequence for the 5-stage RISC pipeline without any forwarding or bypassing hardware but assuming that a register read and a write in the same clock cycle "forwards" through the register

file, as between the `add` and `or` shown in Figure C.5. Use a pipeline timing chart like that in Figure C.8. Assume that the branch is handled by flushing the pipeline. If all memory references take 1 cycle, how many cycles does this loop take to execute?

c. [15] <C.2> Show the timing of this instruction sequence for the 5-stage RISC pipeline with full forwarding and bypassing hardware. Use a pipeline timing chart like that shown in Figure C.8. Assume that the branch is handled by predicting it as not taken. If all memory references take 1 cycle, how many cycles does this loop take to execute?

d. [15] <C.2> Show the timing of this instruction sequence for the 5-stage RISC pipeline with full forwarding and bypassing hardware, as shown in Figure C.6. Use a pipeline timing chart like that shown in Figure C.8. Assume that the branch is handled by predicting it as taken. If all memory references take 1 cycle, how many cycles does this loop take to execute?

e. [25] <C.2> High-performance processors have very deep pipelines—more than 15 stages. Imagine that you have a 10-stage pipeline in which every stage of the 5-stage pipeline has been split in two. The only catch is that, for data forwarding, data are forwarded from the end of a *pair of stages* to the beginning of the two stages where they are needed. For example, data are forwarded from the output of the second execute stage to the input of the first execute stage, still causing a 1-cycle delay. Show the timing of this instruction sequence for the 10-stage RISC pipeline with full forwarding and bypassing hardware. Use a pipeline timing chart like that shown in Figure C.8 (but with stages labeled IF1, IF2, ID1, etc.). Assume that the branch is handled by predicting it as taken. If all memory references take 1 cycle, how many cycles does this loop take to execute?

f. [10] <C.2> Assume that in the 5-stage pipeline, the longest stage requires 0.8 ns, and the pipeline register delay is 0.1 ns. What is the clock cycle time of the 5-stage pipeline? If the 10-stage pipeline splits all stages in half, what is the cycle time of the 10-stage machine?

g. [15] <C.2> Using your answers from parts (d) and (e), determine the cycles per instruction (CPI) for the loop on a 5-stage pipeline and a 10-stage pipeline. Make sure you count only from when the first instruction reaches the write-back stage to the end. Do not count the start-up of the first instruction. Using the clock cycle time calculated in part (f), calculate the average instruction execute time for each machine.

C.2 [15/15] <C.2> Suppose the branch frequencies (as percentages of all instructions) are as follows:

Conditional branches	15%
Jumps and calls	1%
Taken conditional branches	60% are taken

a. [15] <C.2> We are examining a four-stage pipeline where the branch is resolved at the end of the second cycle for unconditional branches and at the end of the third cycle for conditional branches. Assuming that only the first pipe stage can always be completed independent of whether the branch is taken and ignoring other pipeline stalls, how much faster would the machine be without any branch hazards?

b. [15] <C.2> Now assume a high-performance processor in which we have a 15-deep pipeline where the branch is resolved at the end of the fifth cycle for unconditional branches and at the end of the tenth cycle for conditional branches. Assuming that only the first pipe stage can always be completed independent of whether the branch is taken and ignoring other pipeline stalls, how much faster would the machine be without any branch hazards?

C.3 [5/15/10/10] <C.2> We begin with a computer implemented in single-cycle implementation. When the stages are split by functionality, the stages do not require exactly the same amount of time. The original machine had a clock cycle time of 7 ns. After the stages were split, the measured times were IF, 1 ns; ID, 1.5 ns; EX, 1 ns; MEM, 2 ns; and WB, 1.5 ns. The pipeline register delay is 0.1 ns.

a. [5] <C.2> What is the clock cycle time of the 5-stage pipelined machine?

b. [15] <C.2> If there is a stall every four instructions, what is the CPI of the new machine?

c. [10] <C.2> What is the speedup of the pipelined machine over the single-cycle machine?

d. [10] <C.2> If the pipelined machine had an infinite number of stages, what would its speedup be over the single-cycle machine?

C.4 [15] <C.1, C.2> A reduced hardware implementation of the classic five-stage RISC pipeline might use the EX stage hardware to perform a branch instruction comparison and then not actually deliver the branch target PC to the IF stage until the clock cycle in which the branch instruction reaches the MEM stage. Control hazard stalls can be reduced by resolving branch instructions in ID, but improving performance in one respect may reduce performance in other circumstances. Write a small snippet of code in which calculating the branch in the ID stage causes a data hazard, even with data forwarding.

C.5 [12/13/20/20/15/15] <C.2, C.3> For these problems, we will explore a pipeline for a register-memory architecture. The architecture has two instruction formats: a register-register format and a register-memory format. There is a single-memory addressing mode (offset + base register). There is a set of ALU operations with the format:

```
ALUop Rdest, Rsrc1, Rsrc2
```

or

```
ALUop Rdest, Rsrc1, MEM
```

where the ALUop is one of the following: add, subtract, AND, OR, load (Rsrc1 ignored), or store. Rsrc or Rdest are registers. MEM is a base register and offset pair. Branches use a full compare of two registers and are PC relative. Assume that this machine is pipelined so that a new instruction is started every clock cycle. The pipeline structure, similar to that used in the VAX 8700 micropipeline (Clark, 1987), is

IF	**RF**	**ALU1**	**MEM**	**ALU2**	**WB**				
	IF	RF	ALU1	MEM	ALU2	WB			
		IF	RF	ALU1	MEM	ALU2	WB		
			IF	RF	ALU1	MEM	ALU2	WB	
				IF	RF	ALU1	MEM	ALU2	WB
					IF	RF	ALU1	MEM	ALU2	WB

The first ALU stage is used for effective address calculation for memory references and branches. The second ALU cycle is used for operations and branch comparison. RF is both a decode and register-fetch cycle. Assume that when a register read and a register write of the same register occur in the same clock, the write data are forwarded.

a. [12] <C.2> Find the number of adders needed, counting any adder or incrementer; show a combination of instructions and pipe stages that justify this answer. You need only give one combination that maximizes the adder count.

b. [13] <C.2> Find the number of register read and write ports and memory read and write ports required. Show that your answer is correct by showing a combination of instructions and pipeline stage indicating the instruction and the number of read ports and write ports required for that instruction.

c. [20] <C.3> Determine any data forwarding for any ALUs that will be needed. Assume that there are separate ALUs for the ALU1 and ALU2 pipe stages. Put in all forwarding among ALUs necessary to avoid or reduce stalls. Show the relationship between the two instructions involved in forwarding using the format of the table in Figure C.23 but ignoring the last two columns. Be careful to consider forwarding across an intervening instruction—for example,

```
add             x1, ...
any instruction
add             ..., x1, ...
```

d. [20] <C.3> Show all of the data forwarding requirements necessary to avoid or reduce stalls when either the source or destination unit is not an ALU. Use the same format as in Figure C.23, again ignoring the last two columns. Remember to forward to and from memory references.

e. [15] <C.3> Show all the remaining hazards that involve at least one unit other than an ALU as the source or destination unit. Use a table like that shown in Figure C.25, but replace the last column with the lengths of the hazards.

f. [15] <C.2> Show all control hazards by example and state the length of the stall. Use a format like that shown in Figure C.11, labeling each example.

C.6 [12/13/13/15/15] <C.1, C.2, C.3> We will now add support for register-memory ALU operations to the classic five-stage RISC pipeline. To offset this increase in complexity, *all* memory addressing will be restricted to register indirect (i.e., all addresses are simply a value held in a register; no offset or displacement may be added to the register value). For example, the register-memory instruction `add x4, x5, (x1)` means add the contents of register x5 to the contents of the memory location with address equal to the value in register x1 and put the sum in register x4. Register-register ALU operations are unchanged. The following items apply to the integer RISC pipeline:

a. [12] <C.1> List a rearranged order of the five traditional stages of the RISC pipeline that will support register-memory operations implemented exclusively by register indirect addressing.

b. [13] <C.2, C.3> Describe what new forwarding paths are needed for the rearranged pipeline by stating the source, destination, and information transferred on each needed new path.

c. [13] <C.2, C.3> For the reordered stages of the RISC pipeline, what new data hazards are created by this addressing mode? Give an instruction sequence illustrating each new hazard.

d. [15] <C.3> List all of the ways that the RISC pipeline with register-memory ALU operations can have a different instruction count for a given program than the original RISC pipeline. Give a pair of specific instruction sequences, one for the original pipeline and one for the rearranged pipeline, to illustrate each way.

e. [15] <C.3> Assume that all instructions take 1 clock cycle per stage. List all of the ways that the register-memory RISC V can have a different CPI for a given program as compared to the original RISC V pipeline.

C.7 [10/10] <C.3> In this problem, we will explore how deepening the pipeline affects performance in two ways: faster clock cycle and increased stalls due to data and control hazards. Assume that the original machine is a 5-stage pipeline with a 1 ns clock cycle. The second machine is a 12-stage pipeline with a 0.6 ns clock cycle. The 5-stage pipeline experiences a stall due to a data hazard every five instructions, whereas the 12-stage pipeline experiences three stalls every eight instructions. In addition, branches constitute 20% of the instructions, and the misprediction rate for both machines is 5%.

a. [10] <C.3> What is the speedup of the 12-stage pipeline over the 5-stage pipeline, taking into account only data hazards?

b. [10] <C.3> If the branch mispredict penalty for the first machine is 2 cycles but the second machine is 5 cycles, what are the CPIs of each, taking into account the stalls due to branch mispredictions?

C.8 [15] <C.5> Construct a table like that shown in Figure C.21 to check for WAW stalls in the RISC V FP pipeline of Figure C.30. Do not consider FP divides.

C.9 [20/22/22] <C.4, C.6> In this exercise, we will look at how a common vector loop runs on statically and dynamically scheduled versions of the RISC V pipeline. The loop is the so-called DAXPY loop (discussed extensively in Appendix G) and the central operation in Gaussian elimination. The loop implements the vector operation $Y = a*X + Y$ for a vector of length 100. Here is the MIPS code for the loop:

```
foo:    fld       f2, 0(x1)    ; load X(i)
        fmul.d    f4, f2, f0   ; multiply a*X(i)
        fld       f6, 0(x2)    ; load Y(i)
        fadd.d    f6, f4, f6   ; add a*X(i) + Y(i)
        fsd       0(x2), f6    ; store Y(i)
        addi      x1, x1, 8    ; increment X index
        addi      x2, x2, 8    ; increment Y index
        sltiu     x3, x1, done ; test if done
        bnez      x3, foo      ; loop if not done
```

For parts (a) to (c), assume that integer operations issue and complete in 1 clock cycle (including loads) and that their results are fully bypassed. You will use the FP latencies (only) shown in Figure C.29, but assume that the FP unit is fully pipelined. For scoreboards below, assume that an instruction waiting for a result from another function unit can pass through read operands at the same time the result is written. Also assume that an instruction in WB completing will allow a currently active instruction that is waiting on the same functional unit to issue in the same clock cycle in which the first instruction completes WB.

a. [20] <C.5> For this problem, use the RISC V pipeline of Section C.5 with the pipeline latencies from Figure C.29, but a fully pipelined FP unit, so the initiation interval is 1. Draw a timing diagram, similar to Figure C.32, showing the timing of each instruction's execution. How many clock cycles does each loop iteration take, counting from when the first instruction enters the WB stage to when the last instruction enters the WB stage?

b. [20] <C.8> Perform *static instruction reordering* to reorder the instructions to minimize the stalls for this loop, renaming registers where necessary. Use all the same assumptions as in (a). Draw a timing diagram, similar to Figure C.32, showing the timing of each instruction's execution. How many clock cycles does each loop iteration take, counting from when the first instruction enters the WB stage to when the last instruction enters the WB stage?

c. [20] <C.8> Using the original code above, consider how the instructions would have executed using scoreboarding, a form of dynamic scheduling. Draw a timing diagram, similar to Figure C.32, showing the timing of the instructions through stages IF, IS (issue), RO (read operands), EX (execution), and WR (write result). How many clock cycles does each loop iteration take, counting from when the first instruction enters the WB stage to when the last instruction enters the WB stage?

C.10 [25] <C.8> It is critical that the scoreboard be able to distinguish RAW and WAR hazards, because a WAR hazard requires stalling the instruction doing the writing until the instruction reading an operand initiates execution, but a RAW hazard requires delaying the reading instruction until the writing instruction finishes—just the opposite. For example, consider the sequence:

```
fmul.d    f0,f6,f4
fsub.d    f8,f0,f2
fadd.d    f2,f10,f2
```

The `fsub.d` depends on the `fmul.d` (a RAW hazard), thus the `fmul.d` must be allowed to complete before the `fsub.d`. If the `fmul.d` were stalled for the `fsub.d` due to the inability to distinguish between RAW and WAR hazards, the processor will deadlock. This sequence contains a WAR hazard between the `fadd.d` and the `fsub.d`, and the `fadd.d` cannot be allowed to complete until the `fsub.d` begins execution. The difficulty lies in distinguishing the RAW hazard between `fmul.d` and `fsub.d`, and the `WAR` hazard between the `fsub.d` and `fadd.d`. To see just why the three-instruction scenario is important, trace the handling of each instruction stage by stage through issue, read operands, execute, and write result. Assume that each scoreboard stage other than execute takes 1 clock cycle. Assume that the `fmul.d` instruction requires 3 clock cycles to execute and that the `fsub.d` and `fadd.d` instructions each take 1 cycle to execute. Finally, assume that the processor has two multiply function units and two add function units. Present the trace as follows.

1. Make a table with the column headings Instruction, Issue, Read Operands, Execute, Write Result, and Comment. In the first column, list the instructions in program order (be generous with space between instructions; larger table cells will better hold the results of your analysis). Start the table by writing a 1 in the Issue column of the `fmul.d` instruction row to show that `fmul.d` completes the issue stage in clock cycle 1. Now, fill in the stage columns of the table through the cycle at which the scoreboard first stalls an instruction.
2. For a stalled instruction write the words "waiting at clock cycle X," where X is the number of the current clock cycle, in the appropriate table column to show that the scoreboard is resolving an RAW or WAR hazard by stalling that stage. In the Comment column, state what type of hazard and what dependent instruction is causing the wait.

3. Adding the words "completes with clock cycle Y" to a "waiting" table entry, fill in the rest of the table through the time when all instructions are complete. For an instruction that stalled, add a description in the Comments column telling why the wait ended when it did and how deadlock was avoided (Hint: Think about how WAW hazards are prevented and what this implies about active instruction sequences.). Note the completion order of the three instructions as compared to their program order.

C.11 [10/10/10] <C.5> For this problem, you will create a series of small snippets that illustrate the issues that arise when using functional units with different latencies. For each one, draw a timing diagram similar to Figure C.32 that illustrates each concept, and clearly indicate the problem.

a. [10] <C.5> Demonstrate, using code different from that used in Figure C.32, the structural hazard of having the hardware for only one MEM and WB stage.

b. [10] <C.5> Demonstrate a WAW hazard requiring a stall.

D.1	Introduction	D-2
D.2	Advanced Topics in Disk Storage	D-2
D.3	Definition and Examples of Real Faults and Failures	D-10
D.4	I/O Performance, Reliability Measures, and Benchmarks	D-15
D.5	A Little Queuing Theory	D-23
D.6	Crosscutting Issues	D-34
D.7	Designing and Evaluating an I/O System—The Internet Archive Cluster	D-36
D.8	Putting It All Together: NetApp FAS6000 Filer	D-41
D.9	Fallacies and Pitfalls	D-43
D.10	Concluding Remarks	D-47
D.11	Historical Perspective and References	D-48
	Case Studies with Exercises by Andrea C. Arpaci-Dusseau and Remzi H. Arpaci-Dusseau	D-48

D

Storage Systems

I think Silicon Valley was misnamed. If you look back at the dollars shipped in products in the last decade, there has been more revenue from magnetic disks than from silicon. They ought to rename the place Iron Oxide Valley.

Al Hoagland
A pioneer of magnetic disks (1982)

Combining bandwidth and storage … enables swift and reliable access to the ever expanding troves of content on the proliferating disks and … repositories of the Internet … the capacity of storage arrays of all kinds is rocketing ahead of the advance of computer performance.

George Gilder
"The End Is Drawing Nigh,"
Forbes ASAP (April 4, 2000)

D.1 Introduction

The popularity of Internet services such as search engines and auctions has enhanced the importance of I/O for computers, since no one would want a desktop computer that couldn't access the Internet. This rise in importance of I/O is reflected by the names of our times. The 1960s to 1980s were called the Computing Revolution; the period since 1990 has been called the Information Age, with concerns focused on advances in information technology versus raw computational power. Internet services depend upon massive storage, which is the focus of this chapter, and networking, which is the focus of Appendix F.

This shift in focus from computation to communication and storage of information emphasizes reliability and scalability as well as cost-performance. Although it is frustrating when a program crashes, people become hysterical if they lose their data; hence, storage systems are typically held to a higher standard of dependability than the rest of the computer. Dependability is the bedrock of storage, yet it also has its own rich performance theory—queuing theory—that balances throughput versus response time. The software that determines which processor features get used is the compiler, but the operating system usurps that role for storage.

Thus, storage has a different, multifaceted culture from processors, yet it is still found within the architecture tent. We start our exploration with advances in magnetic disks, as they are the dominant storage device today in desktop and server computers. We assume that readers are already familiar with the basics of storage devices, some of which were covered in Chapter 1.

D.2 Advanced Topics in Disk Storage

The disk industry historically has concentrated on improving the capacity of disks. Improvement in capacity is customarily expressed as improvement in *areal density*, measured in bits per square inch:

$$\text{Areal density} = \frac{\text{Tracks}}{\text{Inch}} \text{on a disk surface} \times \frac{\text{Bits}}{\text{Inch}} \text{on a track}$$

Through about 1988, the rate of improvement of areal density was 29% per year, thus doubling density every 3 years. Between then and about 1996, the rate improved to 60% per year, quadrupling density every 3 years and matching the traditional rate of DRAMs. From 1997 to about 2003, the rate increased to 100%, doubling every year. After the innovations that allowed this renaissances had largely played out, the rate has dropped recently to about 30% per year. In 2011, the highest density in commercial products is 400 billion bits per square inch. Cost per gigabyte has dropped at least as fast as areal density has increased, with smaller diameter drives playing the larger role in this improvement. Costs per gigabyte improved by almost a factor of 1,000,000 between 1983 and 2011.

Magnetic disks have been challenged many times for supremacy of secondary storage. Figure D.1 shows one reason: the fabled *access time gap* between disks and DRAM. DRAM latency is about 100,000 times less than disk, and that performance advantage costs 30 to 150 times more per gigabyte for DRAM.

The bandwidth gap is more complex. For example, a fast disk in 2011 transfers at 200 MB/sec from the disk media with 600 GB of storage and costs about $400. A 4 GB DRAM module costing about $200 in 2011 could transfer at 16,000 MB/sec (see Chapter 2), giving the DRAM module about 80 times higher bandwidth than the disk. However, the bandwidth per GB is 6000 times higher for DRAM, and the bandwidth per dollar is 160 times higher.

Many have tried to invent a technology cheaper than DRAM but faster than disk to fill that gap, but thus far all have failed. Challengers have never had a product to market at the right time. By the time a new product ships, DRAMs and disks have made advances as predicted earlier, costs have dropped accordingly, and the challenging product is immediately obsolete.

The closest challenger is Flash memory. This semiconductor memory is nonvolatile like disks, and it has about the same bandwidth as disks, but latency is 100 to 1000 times faster than disk. In 2011, the price per gigabyte of Flash was 15 to 20 times cheaper than DRAM. Flash is popular in cell phones because it comes in much smaller capacities and it is more power efficient than disks, despite the cost per gigabyte being 15 to 25 times higher than disks. Unlike disks and DRAM,

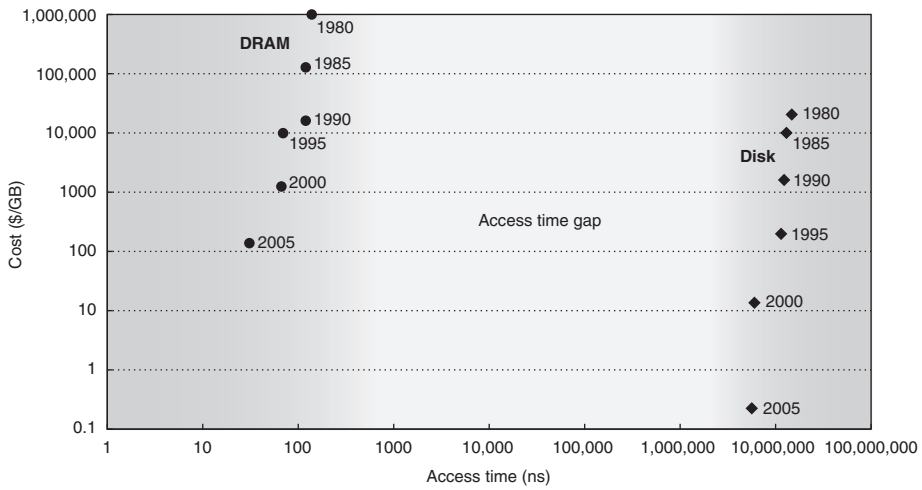


Figure D.1 Cost versus access time for DRAM and magnetic disk in 1980, 1985, 1990, 1995, 2000, and 2005. The two-order-of-magnitude gap in cost and five-order-of-magnitude gap in access times between semiconductor memory and rotating magnetic disks have inspired a host of competing technologies to try to fill them. So far, such attempts have been made obsolete before production by improvements in magnetic disks, DRAMs, or both. Note that between 1990 and 2005 the cost per gigabyte DRAM chips made less improvement, while disk cost made dramatic improvement.

Flash memory bits wear out—typically limited to 1 million writes—and so they are not popular in desktop and server computers.

While disks will remain viable for the foreseeable future, the conventional sector-track-cylinder model did not. The assumptions of the model are that nearby blocks are on the same track, blocks in the same cylinder take less time to access since there is no seek time, and some tracks are closer than others.

First, disks started offering higher-level intelligent interfaces, like ATA and SCSI, when they included a microprocessor inside a disk. To speed up sequential transfers, these higher-level interfaces organize disks more like tapes than like random access devices. The logical blocks are ordered in serpentine fashion across a single surface, trying to capture all the sectors that are recorded at the same bit density. (Disks vary the recording density since it is hard for the electronics to keep up with the blocks spinning much faster on the outer tracks, and lowering linear density simplifies the task.) Hence, sequential blocks may be on different tracks. We will see later in Figure D.22 on page D-45 an illustration of the fallacy of assuming the conventional sector-track model when working with modern disks.

Second, shortly after the microprocessors appeared inside disks, the disks included buffers to hold the data until the computer was ready to accept it, and later caches to avoid read accesses. They were joined by a command queue that allowed the disk to decide in what order to perform the commands to maximize performance while maintaining correct behavior. Figure D.2 shows how a queue depth of 50 can double the number of I/Os per second of random I/Os due to better scheduling of accesses. Although it's unlikely that a system would really have 256 commands in a queue, it would triple the number of I/Os per second. Given buffers, caches, and out-of-order accesses, an accurate performance model of a real disk is much more complicated than sector-track-cylinder.

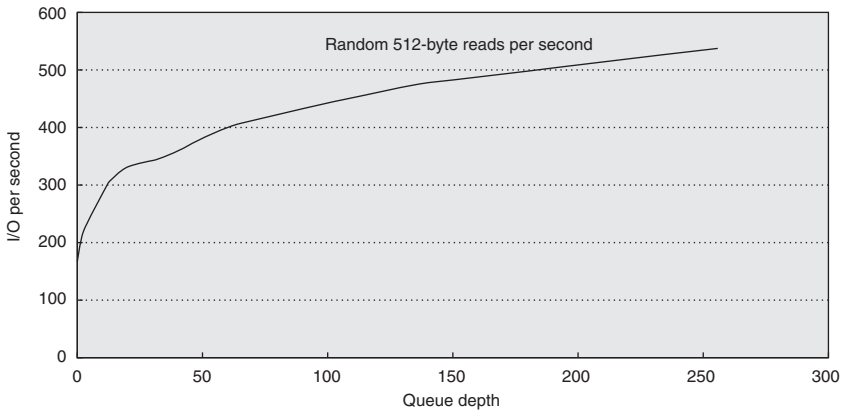


Figure D.2 Throughput versus command queue depth using random 512-byte reads. The disk performs 170 reads per second starting at no command queue and doubles performance at 50 and triples at 256 [Anderson 2003].

Finally, the number of platters shrank from 12 in the past to 4 or even 1 today, so the cylinder has less importance than before because the percentage of data in a cylinder is much less.

Disk Power

Power is an increasing concern for disks as well as for processors. A typical ATA disk in 2011 might use 9 watts when idle, 11 watts when reading or writing, and 13 watts when seeking. Because it is more efficient to spin smaller mass, smaller-diameter disks can save power. One formula that indicates the importance of rotation speed and the size of the platters for the power consumed by the disk motor is the following [Gurumurthi et al. 2005]:

$$\text{Power} \approx \text{Diameter}^{4.6} \times \text{RPM}^{2.8} \times \text{Number of platters}$$

Thus, smaller platters, slower rotation, and fewer platters all help reduce disk motor power, and most of the power is in the motor.

Figure D.3 shows the specifications of two 3.5-inch disks in 2011. The *Serial ATA* (SATA) disks shoot for high capacity and the best cost per gigabyte, so the 2000 GB drives cost less than $0.05 per gigabyte. They use the widest platters that fit the form factor and use four or five of them, but they spin at 5900 RPM and seek relatively slowly to allow a higher areal density and to lower power. The corresponding *Serial Attach SCSI* (SAS) drive aims at performance, so it spins at 15,000 RPM and seeks much faster. It uses a lower areal density to spin at that high rate. To reduce power, the platter is much narrower than the form factor. This combination reduces capacity of the SAS drive to 600 GB.

The cost per gigabyte is about a factor of five better for the SATA drives, and, conversely, the cost per I/O per second or MB transferred per second is about a factor of five better for the SAS drives. Despite using smaller platters and many fewer of them, the SAS disks use twice the power of the SATA drives, due to the much faster RPM and seeks.

	Capacity (GB)	Price	Platters	RPM	Diameter (inches)	Average seek (ms)	Power (watts)	I/O/sec	Disk BW (MB/sec)	Buffer BW (MB/sec)	Buffer size (MB)	MTTF (hrs)
SATA	2000	$85	4	5900	3.7	16	12	47	45–95	300	32	0.6 M
SAS	600	$400	4	15,000	2.6	3–4	16	285	122–204	750	16	1.6 M

Figure D.3 Serial ATA (SATA) versus Serial Attach SCSI (SAS) drives in 3.5-inch form factor in 2011. The I/Os per second were calculated using the average seek plus the time for one-half rotation plus the time to transfer one sector of 512 KB.

Advanced Topics in Disk Arrays

An innovation that improves both dependability and performance of storage systems is *disk arrays*. One argument for arrays is that potential throughput can be increased by having many disk drives and, hence, many disk arms, rather than fewer large drives. Simply spreading data over multiple disks, called *striping*, automatically forces accesses to several disks if the data files are large. (Although arrays improve throughput, latency is not necessarily improved.) As we saw in Chapter 1, the drawback is that with more devices, dependability decreases: N devices generally have $1/N$ the reliability of a single device.

Although a disk array would have more faults than a smaller number of larger disks when each disk has the same reliability, dependability is improved by adding redundant disks to the array to tolerate faults. That is, if a single disk fails, the lost information is reconstructed from redundant information. The only danger is in having another disk fail during the *mean time to repair* (MTTR). Since the *mean time to failure* (MTTF) of disks is tens of years, and the MTTR is measured in hours, redundancy can make the measured reliability of many disks much higher than that of a single disk.

Such redundant disk arrays have become known by the acronym *RAID*, which originally stood for *redundant array of inexpensive disks*, although some prefer the word *independent* for *I* in the acronym. The ability to recover from failures plus the higher throughput, measured as either megabytes per second or I/Os per second, make RAID attractive. When combined with the advantages of smaller size and lower power of small-diameter drives, RAIDs now dominate large-scale storage systems.

Figure D.4 summarizes the five standard RAID levels, showing how eight disks of user data must be supplemented by redundant or check disks at each RAID level, and it lists the pros and cons of each level. The standard RAID levels are well documented, so we will just do a quick review here and discuss advanced levels in more depth.

- *RAID 0*—It has no redundancy and is sometimes nicknamed *JBOD*, for *just a bunch of disks*, although the data may be striped across the disks in the array. This level is generally included to act as a measuring stick for the other RAID levels in terms of cost, performance, and dependability.
- *RAID 1*—Also called *mirroring* or *shadowing*, there are two copies of every piece of data. It is the simplest and oldest disk redundancy scheme, but it also has the highest cost. Some array controllers will optimize read performance by allowing the mirrored disks to act independently for reads, but this optimization means it may take longer for the mirrored writes to complete.
- *RAID 2*—This organization was inspired by applying memory-style error-correcting codes (ECCs) to disks. It was included because there was such a disk array product at the time of the original RAID paper, but none since then as other RAID organizations are more attractive.
- *RAID 3*—Since the higher-level disk interfaces understand the health of a disk, it's easy to figure out which disk failed. Designers realized that if one extra disk

RAID level		Disk failures tolerated, check space overhead for 8 data disks	Pros	Cons	Company products
0	Nonredundant striped	0 failures, 0 check disks	No space overhead	No protection	Widely used
1	Mirrored	1 failure, 8 check disks	No parity calculation; fast recovery; small writes faster than higher RAIDs; fast reads	Highest check storage overhead	EMC, HP (Tandem), IBM
2	Memory-style ECC	1 failure, 4 check disks	Doesn't rely on failed disk to self-diagnose	~ Log 2 check storage overhead	Not used
3	Bit-interleaved parity	1 failure, 1 check disk	Low check overhead; high bandwidth for large reads or writes	No support for small, random reads or writes	Storage Concepts
4	Block-interleaved parity	1 failure, 1 check disk	Low check overhead; more bandwidth for small reads	Parity disk is small write bottleneck	Network Appliance
5	Block-interleaved distributed parity	1 failure, 1 check disk	Low check overhead; more bandwidth for small reads and writes	Small writes → 4 disk accesses	Widely used
6	Row-diagonal parity, EVEN-ODD	2 failures, 2 check disks	Protects against 2 disk failures	Small writes → 6 disk accesses; 2 × check overhead	Network Appliance

Figure D.4 RAID levels, their fault tolerance, and their overhead in redundant disks. The paper that introduced the term *RAID* [Patterson, Gibson, and Katz 1987] used a numerical classification that has become popular. In fact, the nonredundant disk array is often called *RAID 0*, indicating that the data are striped across several disks but without redundancy. Note that mirroring (RAID 1) in this instance can survive up to eight disk failures provided only one disk of each mirrored pair fails; worst case is both disks in a mirrored pair fail. In 2011, there may be no commercial implementations of RAID 2; the rest are found in a wide range of products. RAID 0 + 1, 1 + 0, 01, 10, and 6 are discussed in the text.

contains the parity of the information in the data disks, a single disk allows recovery from a disk failure. The data are organized in stripes, with *N* data blocks and one parity block. When a failure occurs, we just "subtract" the good data from the good blocks, and what remains is the missing data. (This works whether the failed disk is a data disk or the parity disk.) RAID 3 assumes that the data are spread across all disks on reads and writes, which is attractive when reading or writing large amounts of data.

- *RAID 4*—Many applications are dominated by small accesses. Since sectors have their own error checking, you can safely increase the number of reads per second by allowing each disk to perform independent reads. It would seem that writes would still be slow, if you have to read every disk to calculate parity. To increase the number of writes per second, an alternative approach involves only two disks. First, the array reads the old data that are about to be overwritten, and then calculates what bits would change before it writes the new data. It then reads the old value of the parity on the check disks, updates parity according to the list of changes, and then writes the new value of parity to the check

disk. Hence, these so-called "small writes" are still slower than small reads—they involve four disks accesses—but they are faster than if you had to read all disks on every write. RAID 4 has the same low check disk overhead as RAID 3, and it can still do large reads and writes as fast as RAID 3 in addition to small reads and writes, but control is more complex.

- *RAID 5*—Note that a performance flaw for small writes in RAID 4 is that they all must read and write the same check disk, so it is a performance bottleneck. RAID 5 simply distributes the parity information across all disks in the array, thereby removing the bottleneck. The parity block in each stripe is rotated so that parity is spread evenly across all disks. The disk array controller must now calculate which disk has the parity for when it wants to write a given block, but that can be a simple calculation. RAID 5 has the same low check disk overhead as RAID 3 and 4, and it can do the large reads and writes of RAID 3 and the small reads of RAID 4, but it has higher small write bandwidth than RAID 4. Nevertheless, RAID 5 requires the most sophisticated controller of the classic RAID levels.

Having completed our quick review of the classic RAID levels, we can now look at two levels that have become popular since RAID was introduced.

RAID 10 versus 01 (or 1+0 versus RAID 0+1)

One topic not always described in the RAID literature involves how mirroring in RAID 1 interacts with striping. Suppose you had, say, four disks' worth of data to store and eight physical disks to use. Would you create four pairs of disks—each organized as RAID 1—and then stripe data across the four RAID 1 pairs? Alternatively, would you create two sets of four disks—each organized as RAID 0—and then mirror writes to both RAID 0 sets? The RAID terminology has evolved to call the former RAID 1+0 or RAID 10 ("striped mirrors") and the latter RAID 0+1 or RAID 01 ("mirrored stripes").

RAID 6: Beyond a Single Disk Failure

The parity-based schemes of the RAID 1 to 5 protect against a single self-identifying failure; however, if an operator accidentally replaces the wrong disk during a failure, then the disk array will experience two failures, and data will be lost. Another concern is that since disk bandwidth is growing more slowly than disk capacity, the MTTR of a disk in a RAID system is increasing, which in turn increases the chances of a second failure. For example, a 500 GB SATA disk could take about 3 hours to read sequentially assuming no interference. Given that the damaged RAID is likely to continue to serve data, reconstruction could be stretched considerably, thereby increasing MTTR. Besides increasing reconstruction time, another concern is that reading much more data during reconstruction means increasing the chance of an uncorrectable media failure, which would result in data loss. Other arguments for concern about simultaneous multiple failures are

the increasing number of disks in arrays and the use of ATA disks, which are slower and larger than SCSI disks.

Hence, over the years, there has been growing interest in protecting against more than one failure. Network Appliance (NetApp), for example, started by building RAID 4 file servers. As double failures were becoming a danger to customers, they created a more robust scheme to protect data, called *row-diagonal parity* or *RAID-DP* [Corbett et al. 2004]. Like the standard RAID schemes, row-diagonal parity uses redundant space based on a parity calculation on a per-stripe basis. Since it is protecting against a double failure, it adds two check blocks per stripe of data. Let's assume there are $p+1$ disks total, so $p-1$ disks have data. Figure D.5 shows the case when p is 5.

The row parity disk is just like in RAID 4; it contains the even parity across the other four data blocks in its stripe. Each block of the diagonal parity disk contains the even parity of the blocks in the same diagonal. Note that each diagonal does not cover one disk; for example, diagonal 0 does not cover disk 1. Hence, we need just $p-1$ diagonals to protect the p disks, so the disk only has diagonals 0 to 3 in Figure D.5.

Let's see how row-diagonal parity works by assuming that data disks 1 and 3 fail in Figure D.5. We can't perform the standard RAID recovery using the first row using row parity, since it is missing two data blocks from disks 1 and 3. However, we can perform recovery on diagonal 0, since it is only missing the data block associated with disk 3. Thus, row-diagonal parity starts by recovering one of the four blocks on the failed disk in this example using diagonal parity. Since each diagonal misses one disk, and all diagonals miss a different disk, two diagonals are only missing one block. They are diagonals 0 and 2 in this example, so we next restore the block from diagonal 2 from failed disk 1. When the data for those blocks have been recovered, then the standard RAID recovery scheme can be used to

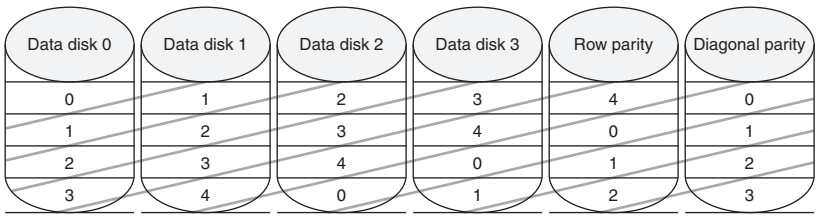


Figure D.5 Row diagonal parity for $p = 5$, which protects four data disks from double failures [Corbett et al. 2004]. This figure shows the diagonal groups for which parity is calculated and stored in the diagonal parity disk. Although this shows all the check data in separate disks for row parity and diagonal parity as in RAID 4, there is a rotated version of row-diagonal parity that is analogous to RAID 5. Parameter p must be prime and greater than 2; however, you can make p larger than the number of data disks by assuming that the missing disks have all zeros and the scheme still works. This trick makes it easy to add disks to an existing system. NetApp picks p to be 257, which allows the system to grow to up to 256 data disks.

recover two more blocks in the standard RAID 4 stripes 0 and 2, which in turn allows us to recover more diagonals. This process continues until two failed disks are completely restored.

The EVEN-ODD scheme developed earlier by researchers at IBM is similar to row diagonal parity, but it has a bit more computation during operation and recovery [Blaum 1995]. Papers that are more recent show how to expand EVEN-ODD to protect against three failures [Blaum, Bruck, and Vardy 1996; Blaum et al. 2001].

D.3 Definition and Examples of Real Faults and Failures

Although people may be willing to live with a computer that occasionally crashes and forces all programs to be restarted, they insist that their information is never lost. The prime directive for storage is then to remember information, no matter what happens.

Chapter 1 covered the basics of dependability, and this section expands that information to give the standard definitions and examples of failures.

The first step is to clarify confusion over terms. The terms *fault*, *error*, and *failure* are often used interchangeably, but they have different meanings in the dependability literature. For example, is a programming mistake a fault, error, or failure? Does it matter whether we are talking about when it was designed or when the program is run? If the running program doesn't exercise the mistake, is it still a fault/error/failure? Try another one. Suppose an alpha particle hits a DRAM memory cell. Is it a fault/error/failure if it doesn't change the value? Is it a fault/error/failure if the memory doesn't access the changed bit? Did a fault/error/failure still occur if the memory had error correction and delivered the corrected value to the CPU? You get the drift of the difficulties. Clearly, we need precise definitions to discuss such events intelligently.

To avoid such imprecision, this subsection is based on the terminology used by Laprie [1985] and Gray and Siewiorek [1991], endorsed by IFIP Working Group 10.4 and the IEEE Computer Society Technical Committee on Fault Tolerance. We talk about a system as a single module, but the terminology applies to submodules recursively. Let's start with a definition of *dependability*:

Computer system dependability *is the quality of delivered service such that reliance can justifiably be placed on this service. The* service *delivered by a system is its observed* actual behavior *as perceived by other system(s) interacting with this system's users. Each module also has an ideal* specified behavior*, where a* service specification *is an agreed description of the expected behavior. A system* failure *occurs when the actual behavior deviates from the specified behavior. The failure occurred because of an* error*, a defect in that module. The cause of an error is a* fault*.*

When a fault occurs, it creates a latent error*, which becomes* effective *when it is activated; when the error actually affects the delivered service, a failure occurs. The*

time between the occurrence of an error and the resulting failure is the error latency. *Thus, an error is the manifestation* in the system *of a fault, and a failure is the manifestation* on the service *of an error. [p. 3]*

Let's go back to our motivating examples above. A programming mistake is a *fault*. The consequence is an *error* (or *latent error*) in the software. Upon activation, the error becomes *effective*. When this effective error produces erroneous data that affect the delivered service, a *failure* occurs.

An alpha particle hitting a DRAM can be considered a fault. If it changes the memory, it creates an error. The error will remain latent until the affected memory word is read. If the effective word error affects the delivered service, a failure occurs. If ECC corrected the error, a failure would not occur.

A mistake by a human operator is a fault. The resulting altered data is an error. It is latent until activated, and so on as before.

To clarify, the relationship among faults, errors, and failures is as follows:

- A fault creates one or more latent errors.
- The properties of errors are (1) a latent error becomes effective once activated; (2) an error may cycle between its latent and effective states; and (3) an effective error often propagates from one component to another, thereby creating new errors. Thus, either an effective error is a formerly latent error in that component or it has propagated from another error in that component or from elsewhere.
- A component failure occurs when the error affects the delivered service.
- These properties are recursive and apply to any component in the system.

Gray and Siewiorek classified faults into four categories according to their cause:

1. *Hardware faults*—Devices that fail, such as perhaps due to an alpha particle hitting a memory cell
2. *Design faults*—Faults in software (usually) and hardware design (occasionally)
3. *Operation faults*—Mistakes by operations and maintenance personnel
4. *Environmental faults*—Fire, flood, earthquake, power failure, and sabotage

Faults are also classified by their duration into transient, intermittent, and permanent [Nelson 1990]. *Transient faults* exist for a limited time and are not recurring. *Intermittent faults* cause a system to oscillate between faulty and fault-free operation. *Permanent faults* do not correct themselves with the passing of time.

Now that we have defined the difference between faults, errors, and failures, we are ready to see some real-world examples. Publications of real error rates are rare for two reasons. First, academics rarely have access to significant hardware resources to measure. Second, industrial researchers are rarely allowed to publish failure information for fear that it would be used against their companies in the marketplace. A few exceptions follow.

Berkeley's Tertiary Disk

The Tertiary Disk project at the University of California created an art image server for the Fine Arts Museums of San Francisco in 2000. This database consisted of high-quality images of over 70,000 artworks [Talagala et al., 2000]. The database was stored on a cluster, which consisted of 20 PCs connected by a switched Ethernet and containing 368 disks. It occupied seven 7-foot-high racks.

Figure D.6 shows the failure rates of the various components of Tertiary Disk. In advance of building the system, the designers assumed that SCSI data disks would be the least reliable part of the system, as they are both mechanical and plentiful. Next would be the IDE disks since there were fewer of them, then the power supplies, followed by integrated circuits. They assumed that passive devices such as cables would scarcely ever fail.

Figure D.6 shatters some of those assumptions. Since the designers followed the manufacturer's advice of making sure the disk enclosures had reduced vibration and good cooling, the data disks were very reliable. In contrast, the PC chassis containing the IDE/ATA disks did not afford the same environmental controls. (The IDE/ATA disks did not store data but helped the application and operating

Component	Total in system	Total failed	Percentage failed
SCSI controller	44	1	2.3%
SCSI cable	39	1	2.6%
SCSI disk	368	7	1.9%
IDE/ATA disk	24	6	25.0%
Disk enclosure—backplane	46	13	28.3%
Disk enclosure—power supply	92	3	3.3%
Ethernet controller	20	1	5.0%
Ethernet switch	2	1	50.0%
Ethernet cable	42	1	2.3%
CPU/motherboard	20	0	0%

Figure D.6 Failures of components in Tertiary Disk over 18 months of operation. For each type of component, the table shows the total number in the system, the number that failed, and the percentage failure rate. Disk enclosures have two entries in the table because they had two types of problems: backplane integrity failures and power supply failures. Since each enclosure had two power supplies, a power supply failure did not affect availability. This cluster of 20 PCs, contained in seven 7-foot-high, 19-inch-wide racks, hosted 368 8.4 GB, 7200 RPM, 3.5-inch IBM disks. The PCs were P6-200 MHz with 96 MB of DRAM each. They ran FreeBSD 3.0, and the hosts were connected via switched 100 Mbit/sec Ethernet. All SCSI disks were connected to two PCs via double-ended SCSI chains to support RAID 1. The primary application was called the Zoom Project, which in 1998 was the world's largest art image database, with 72,000 images. See Talagala et al. [2000b].

system to boot the PCs.) Figure D.6 shows that the SCSI backplane, cables, and Ethernet cables were no more reliable than the data disks themselves!

As Tertiary Disk was a large system with many redundant components, it could survive this wide range of failures. Components were connected and mirrored images were placed so that no single failure could make any image unavailable. This strategy, which initially appeared to be overkill, proved to be vital.

This experience also demonstrated the difference between transient faults and hard faults. Virtually all the failures in Figure D.6 appeared first as transient faults. It was up to the operator to decide if the behavior was so poor that they needed to be replaced or if they could continue. In fact, the word "failure" was not used; instead, the group borrowed terms normally used for dealing with problem employees, with the operator deciding whether a problem component should or should not be "fired."

Tandem

The next example comes from industry. Gray [1990] collected data on faults for Tandem Computers, which was one of the pioneering companies in fault-tolerant computing and used primarily for databases. Figure D.7 graphs the faults that caused system failures between 1985 and 1989 in absolute faults per system and in percentage of faults encountered. The data show a clear improvement in the reliability of hardware and maintenance. Disks in 1985 required yearly service by Tandem, but they were replaced by disks that required no scheduled maintenance. Shrinking numbers of chips and connectors per system plus software's ability to tolerate hardware faults reduced hardware's contribution to only 7% of failures by 1989. Moreover, when hardware was at fault, software embedded in the hardware device (firmware) was often the culprit. The data indicate that software in 1989 was the major source of reported outages (62%), followed by system operations (15%).

The problem with any such statistics is that the data only refer to what is reported; for example, environmental failures due to power outages were not reported to Tandem because they were seen as a local problem. Data on operation faults are very difficult to collect because operators must report personal mistakes, which may affect the opinion of their managers, which in turn can affect job security and pay raises. Gray suggested that both environmental faults and operator faults are underreported. His study concluded that achieving higher availability requires improvement in software quality and software fault tolerance, simpler operations, and tolerance of operational faults.

Other Studies of the Role of Operators in Dependability

While Tertiary Disk and Tandem are storage-oriented dependability studies, we need to look outside storage to find better measurements on the role of humans in failures. Murphy and Gent [1995] tried to improve the accuracy of data on

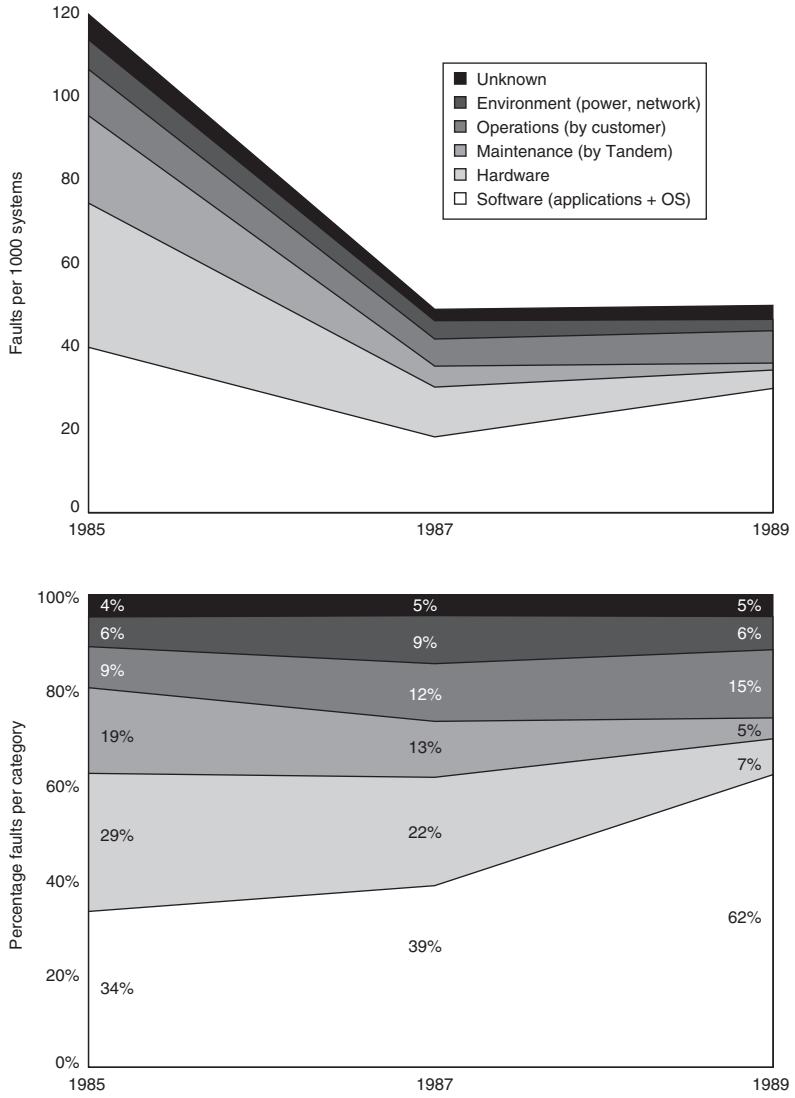


Figure D.7 Faults in Tandem between 1985 and 1989. Gray [1990] collected these data for fault-tolerant Tandem Computers based on reports of component failures by customers.

operator faults by having the system automatically prompt the operator on each boot for the reason for that reboot. They classified consecutive crashes to the same fault as operator fault and included operator actions that directly resulted in crashes, such as giving parameters bad values, bad configurations, and bad application installation. Although they believed that operator error is under-reported,

they did get more accurate information than did Gray, who relied on a form that the operator filled out and then sent up the management chain. The hardware/operating system went from causing 70% of the failures in VAX systems in 1985 to 28% in 1993, and failures due to operators rose from 15% to 52% in that same period. Murphy and Gent expected managing systems to be the primary dependability challenge in the future.

The final set of data comes from the government. The Federal Communications Commission (FCC) requires that all telephone companies submit explanations when they experience an outage that affects at least 30,000 people or lasts 30 minutes. These detailed disruption reports do not suffer from the self-reporting problem of earlier figures, as investigators determine the cause of the outage rather than operators of the equipment. Kuhn [1997] studied the causes of outages between 1992 and 1994, and Enriquez [2001] did a follow-up study for the first half of 2001. Although there was a significant improvement in failures due to overloading of the network over the years, failures due to humans increased, from about one-third to two-thirds of the customer-outage minutes.

These four examples and others suggest that the primary cause of failures in large systems today is faults by human operators. Hardware faults have declined due to a decreasing number of chips in systems and fewer connectors. Hardware dependability has improved through fault tolerance techniques such as memory ECC and RAID. At least some operating systems are considering reliability implications before adding new features, so in 2011 the failures largely occurred elsewhere.

Although failures may be initiated due to faults by operators, it is a poor reflection on the state of the art of systems that the processes of maintenance and upgrading are so error prone. Most storage vendors claim today that customers spend much more on managing storage over its lifetime than they do on purchasing the storage. Thus, the challenge for dependable storage systems of the future is either to tolerate faults by operators or to avoid faults by simplifying the tasks of system administration. Note that RAID 6 allows the storage system to survive even if the operator mistakenly replaces a good disk.

We have now covered the bedrock issue of dependability, giving definitions, case studies, and techniques to improve it. The next step in the storage tour is performance.

D.4 I/O Performance, Reliability Measures, and Benchmarks

I/O performance has measures that have no counterparts in design. One of these is diversity: Which I/O devices can connect to the computer system? Another is capacity: How many I/O devices can connect to a computer system?

In addition to these unique measures, the traditional measures of performance (namely, response time and throughput) also apply to I/O. (I/O throughput is sometimes called *I/O bandwidth* and response time is sometimes called *latency*.) The next two figures offer insight into how response time and throughput trade off

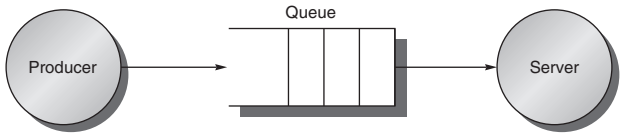


Figure D.8 The traditional producer-server model of response time and throughput. Response time begins when a task is placed in the buffer and ends when it is completed by the server. Throughput is the number of tasks completed by the server in unit time.

against each other. Figure D.8 shows the simple producer-server model. The producer creates tasks to be performed and places them in a buffer; the server takes tasks from the first in, first out buffer and performs them.

Response time is defined as the time a task takes from the moment it is placed in the buffer until the server finishes the task. Throughput is simply the average number of tasks completed by the server over a time period. To get the highest possible throughput, the server should never be idle, thus the buffer should never be empty. Response time, on the other hand, counts time spent in the buffer, so an empty buffer shrinks it.

Another measure of I/O performance is the interference of I/O with processor execution. Transferring data may interfere with the execution of another process. There is also overhead due to handling I/O interrupts. Our concern here is how much longer a process will take because of I/O for another process.

Throughput versus Response Time

Figure D.9 shows throughput versus response time (or latency) for a typical I/O system. The knee of the curve is the area where a little more throughput results in much longer response time or, conversely, a little shorter response time results in much lower throughput.

How does the architect balance these conflicting demands? If the computer is interacting with human beings, Figure D.10 suggests an answer. An interaction, or *transaction*, with a computer is divided into three parts:

1. *Entry time*—The time for the user to enter the command.
2. *System response time*—The time between when the user enters the command and the complete response is displayed.
3. *Think time*—The time from the reception of the response until the user begins to enter the next command.

The sum of these three parts is called the *transaction time*. Several studies report that user productivity is inversely proportional to transaction time. The results in Figure D.10 show that cutting system response time by 0.7 seconds saves 4.9 seconds (34%) from the conventional transaction and 2.0 seconds (70%) from

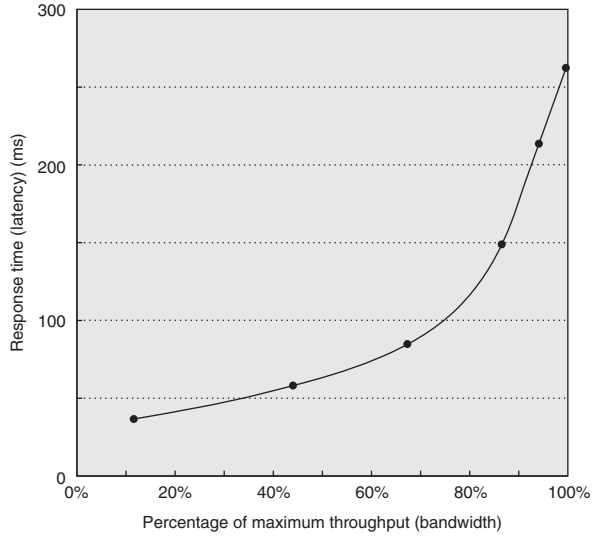


Figure D.9 Throughput versus response time. Latency is normally reported as response time. Note that the minimum response time achieves only 11% of the throughput, while the response time for 100% throughput takes seven times the minimum response time. Note also that the independent variable in this curve is implicit; to trace the curve, you typically vary load (concurrency). Chen et al. [1990] collected these data for an array of magnetic disks.

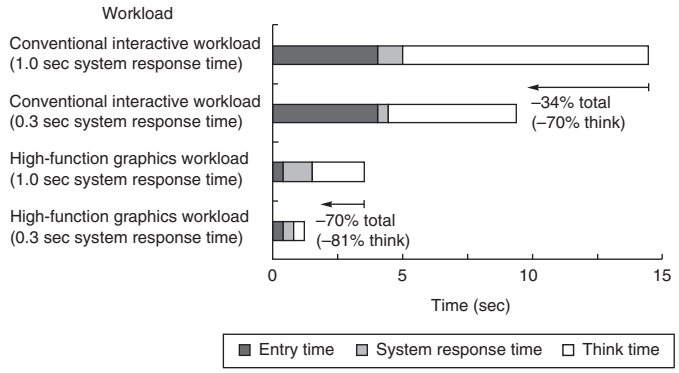


Figure D.10 A user transaction with an interactive computer divided into entry time, system response time, and user think time for a conventional system and graphics system. The entry times are the same, independent of system response time. The entry time was 4 seconds for the conventional system and 0.25 seconds for the graphics system. Reduction in response time actually decreases transaction time by more than just the response time reduction. (From Brady [1986].)

I/O benchmark	Response time restriction	Throughput metric
TPC-C: Complex Query OLTP	≥90% of transaction must meet response time limit; 5 seconds for most types of transactions	New order transactions per minute
TPC-W: Transactional Web benchmark	≥90% of Web interactions must meet response time limit; 3 seconds for most types of Web interactions	Web interactions per second
SPECsfs97	Average response time ≤40 ms	NFS operations per second

Figure D.11 Response time restrictions for three I/O benchmarks.

the graphics transaction. This implausible result is explained by human nature: People need less time to think when given a faster response. Although this study is 20 years old, response times are often still much slower than 1 second, even if processors are 1000 times faster. Examples of long delays include starting an application on a desktop PC due to many disk I/Os, or network delays when clicking on Web links.

To reflect the importance of response time to user productivity, I/O benchmarks also address the response time versus throughput trade-off. Figure D.11 shows the response time bounds for three I/O benchmarks. They report maximum throughput given either that 90% of response times must be less than a limit or that the average response time must be less than a limit.

Let's next look at these benchmarks in more detail.

Transaction-Processing Benchmarks

Transaction processing (TP, or OLTP for online transaction processing) is chiefly concerned with *I/O rate* (the number of disk accesses per second), as opposed to *data rate* (measured as bytes of data per second). TP generally involves changes to a large body of shared information from many terminals, with the TP system guaranteeing proper behavior on a failure. Suppose, for example, that a bank's computer fails when a customer tries to withdraw money from an ATM. The TP system would guarantee that the account is debited if the customer received the money *and* that the account is unchanged if the money was not received. Airline reservations systems as well as banks are traditional customers for TP.

As mentioned in Chapter 1, two dozen members of the TP community conspired to form a benchmark for the industry and, to avoid the wrath of their legal departments, published the report anonymously [Anon. et al. 1985]. This report led to the *Transaction Processing Council*, which in turn has led to eight benchmarks since its founding. Figure D.12 summarizes these benchmarks.

Let's describe TPC-C to give a flavor of these benchmarks. TPC-C uses a database to simulate an order-entry environment of a wholesale supplier, including

Benchmark	Data size (GB)	Performance metric	Date of first results
A: debit credit (retired)	0.1–10	Transactions per second	July 1990
B: batch debit credit (retired)	0.1–10	Transactions per second	July 1991
C: complex query OLTP	100–3000 (minimum 0.07 * TPM)	New order transactions per minute (TPM)	September 1992
D: decision support (retired)	100, 300, 1000	Queries per hour	December 1995
H: ad hoc decision support	100, 300, 1000	Queries per hour	October 1999
R: business reporting decision support (retired)	1000	Queries per hour	August 1999
W: transactional Web benchmark	≈50, 500	Web interactions per second	July 2000
App: application server and Web services benchmark	≈2500	Web service interactions per second (SIPS)	June 2005

Figure D.12 Transaction Processing Council benchmarks. The summary results include both the performance metric and the price-performance of that metric. TPC-A, TPC-B, TPC-D, and TPC-R were retired.

entering and delivering orders, recording payments, checking the status of orders, and monitoring the level of stock at the warehouses. It runs five concurrent transactions of varying complexity, and the database includes nine tables with a scalable range of records and customers. TPC-C is measured in transactions per minute (tpmC) and in price of system, including hardware, software, and three years of maintenance support. Figure 1.17 on page 42 in Chapter 1 describes the top systems in performance and cost-performance for TPC-C.

These TPC benchmarks were the first—and in some cases still the only ones—that have these unusual characteristics:

- *Price is included with the benchmark results.* The cost of hardware, software, and maintenance agreements is included in a submission, which enables evaluations based on price-performance as well as high performance.
- *The dataset generally must scale in size as the throughput increases*. The benchmarks are trying to model real systems, in which the demand on the system and the size of the data stored in it increase together. It makes no sense, for example, to have thousands of people per minute access hundreds of bank accounts.
- *The benchmark results are audited*. Before results can be submitted, they must be approved by a certified TPC auditor, who enforces the TPC rules that try to make sure that only fair results are submitted. Results can be challenged and disputes resolved by going before the TPC.
- *Throughput is the performance metric, but response times are limited*. For example, with TPC-C, 90% of the new order transaction response times must be less than 5 seconds.

- *An independent organization maintains the benchmarks*. Dues collected by TPC pay for an administrative structure including a chief operating office. This organization settles disputes, conducts mail ballots on approval of changes to benchmarks, holds board meetings, and so on.

SPEC System-Level File Server, Mail, and Web Benchmarks

The SPEC benchmarking effort is best known for its characterization of processor performance, but it has created benchmarks for file servers, mail servers, and Web servers.

Seven companies agreed on a synthetic benchmark, called SFS, to evaluate systems running the Sun Microsystems network file service (NFS). This benchmark was upgraded to SFS 3.0 (also called SPEC SFS97_R1) to include support for NFS version 3, using TCP in addition to UDP as the transport protocol, and making the mix of operations more realistic. Measurements on NFS systems led to a synthetic mix of reads, writes, and file operations. SFS supplies default parameters for comparative performance. For example, half of all writes are done in 8 KB blocks and half are done in partial blocks of 1, 2, or 4 KB. For reads, the mix is 85% full blocks and 15% partial blocks.

Like TPC-C, SFS scales the amount of data stored according to the reported throughput: For every 100 NFS operations per second, the capacity must increase by 1 GB. It also limits the average response time, in this case to 40 ms. Figure D.13

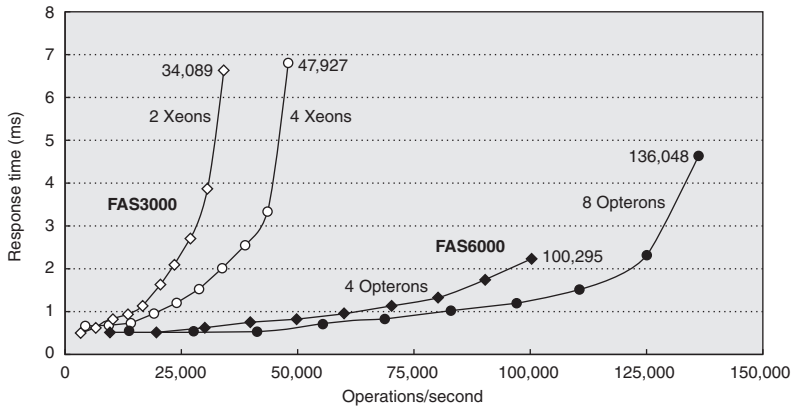


Figure D.13 SPEC SFS97_R1 performance for the NetApp FAS3050c NFS servers in two configurations. Two processors reached 34,089 operations per second and four processors did 47,927. Reported in May 2005, these systems used the Data ONTAP 7.0.1R1 operating system, 2.8 GHz Pentium Xeon microprocessors, 2 GB of DRAM per processor, 1 GB of nonvolatile memory per system, and 168 15 K RPM, 72 GB, Fibre Channel disks. These disks were connected using two or four QLogic ISP-2322 FC disk controllers.

shows average response time versus throughput for two NetApp systems. Unfortunately, unlike the TPC benchmarks, SFS does not normalize for different price configurations.

SPECMail is a benchmark to help evaluate performance of mail servers at an Internet service provider. SPECMail2001 is based on the standard Internet protocols SMTP and POP3, and it measures throughput and user response time while scaling the number of users from 10,000 to 1,000,000.

SPECWeb is a benchmark for evaluating the performance of World Wide Web servers, measuring number of simultaneous user sessions. The SPECWeb2005 workload simulates accesses to a Web service provider, where the server supports home pages for several organizations. It has three workloads: Banking (HTTPS), E-commerce (HTTP and HTTPS), and Support (HTTP).

Examples of Benchmarks of Dependability

The TPC-C benchmark does in fact have a dependability requirement. The benchmarked system must be able to handle a single disk failure, which means in practice that all submitters are running some RAID organization in their storage system.

Efforts that are more recent have focused on the effectiveness of fault tolerance in systems. Brown and Patterson [2000] proposed that availability be measured by examining the variations in system quality-of-service metrics over time as faults are injected into the system. For a Web server, the obvious metrics are performance (measured as requests satisfied per second) and degree of fault tolerance (measured as the number of faults that can be tolerated by the storage subsystem, network connection topology, and so forth).

The initial experiment injected a single fault—such as a write error in disk sector—and recorded the system's behavior as reflected in the quality-of-service metrics. The example compared software RAID implementations provided by Linux, Solaris, and Windows 2000 Server. SPECWeb99 was used to provide a workload and to measure performance. To inject faults, one of the SCSI disks in the software RAID volume was replaced with an emulated disk. It was a PC running software using a SCSI controller that appears to other devices on the SCSI bus as a disk. The disk emulator allowed the injection of faults. The faults injected included a variety of transient disk faults, such as correctable read errors, and permanent faults, such as disk media failures on writes.

Figure D.14 shows the behavior of each system under different faults. The two top graphs show Linux (on the left) and Solaris (on the right). As RAID systems can lose data if a second disk fails before reconstruction completes, the longer the reconstruction (MTTR), the lower the availability. Faster reconstruction implies decreased application performance, however, as reconstruction steals I/O resources from running applications. Thus, there is a policy choice between taking a performance hit during reconstruction or lengthening the window of vulnerability and thus lowering the predicted MTTF.

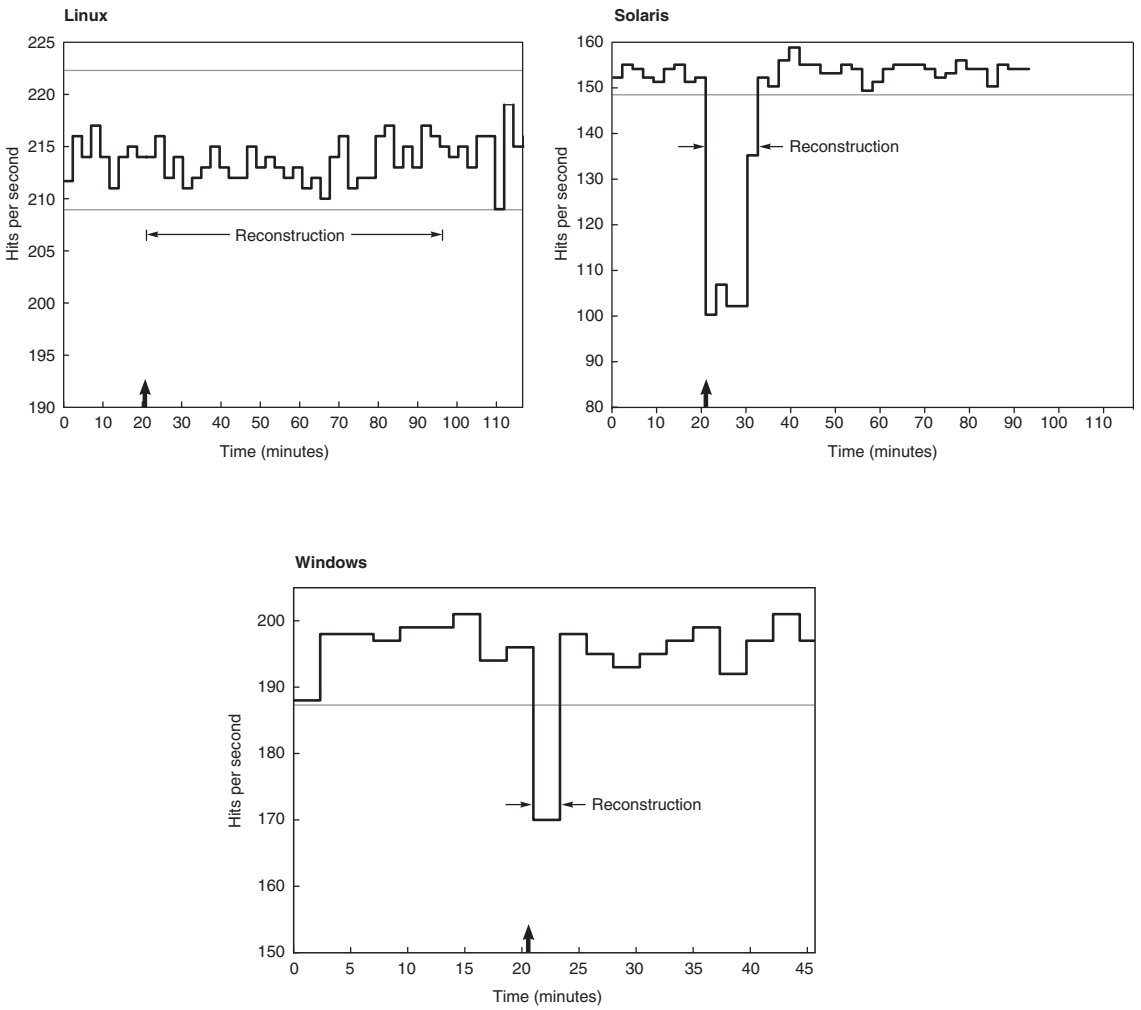


Figure D.14 Availability benchmark for software RAID systems on the same computer running Red Hat 6.0 Linux, Solaris 7, and Windows 2000 operating systems. Note the difference in philosophy on speed of reconstruction of Linux versus Windows and Solaris. The *y*-axis is behavior in hits per second running SPECWeb99. The arrow indicates time of fault insertion. The lines at the top give the 99% confidence interval of performance before the fault is inserted. A 99% confidence interval means that if the variable is outside of this range, the probability is only 1% that this value would appear.

Although none of the tested systems documented their reconstruction policies outside of the source code, even a single fault injection was able to give insight into those policies. The experiments revealed that both Linux and Solaris initiate automatic reconstruction of the RAID volume onto a hot spare when an active disk is taken out of service due to a failure. Although Windows supports RAID

reconstruction, the reconstruction must be initiated manually. Thus, without human intervention, a Windows system that did not rebuild after a first failure remains susceptible to a second failure, which increases the window of vulnerability. It does repair quickly once told to do so.

The fault injection experiments also provided insight into other availability policies of Linux, Solaris, and Windows 2000 concerning automatic spare utilization, reconstruction rates, transient errors, and so on. Again, no system documented their policies.

In terms of managing transient faults, the fault injection experiments revealed that Linux's software RAID implementation takes an opposite approach than do the RAID implementations in Solaris and Windows. The Linux implementation is paranoid—it would rather shut down a disk in a controlled manner at the first error, rather than wait to see if the error is transient. In contrast, Solaris and Windows are more forgiving—they ignore most transient faults with the expectation that they will not recur. Thus, these systems are substantially more robust to transients than the Linux system. Note that both Windows and Solaris do log the transient faults, ensuring that the errors are reported even if not acted upon. When faults were permanent, the systems behaved similarly.

D.5 A Little Queuing Theory

In processor design, we have simple back-of-the-envelope calculations of performance associated with the CPI formula in Chapter 1, or we can use full-scale simulation for greater accuracy at greater cost. In I/O systems, we also have a bestcase analysis as a back-of-the-envelope calculation. Full-scale simulation is also much more accurate and much more work to calculate expected performance.

With I/O systems, however, we also have a mathematical tool to guide I/O design that is a little more work and much more accurate than best-case analysis, but much less work than full-scale simulation. Because of the probabilistic nature of I/O events and because of sharing of I/O resources, we can give a set of simple theorems that will help calculate response time and throughput of an entire I/O system. This helpful field is called *queuing theory*. Since there are many books and courses on the subject, this section serves only as a first introduction to the topic. However, even this small amount can lead to better design of I/O systems.

Let's start with a black-box approach to I/O systems, as shown in Figure D.15. In our example, the processor is making I/O requests that arrive at the I/O device, and the requests "depart" when the I/O device fulfills them.

We are usually interested in the long term, or steady state, of a system rather than in the initial start-up conditions. Suppose we weren't. Although there is a mathematics that helps (Markov chains), except for a few cases, the only way to solve the resulting equations is simulation. Since the purpose of this section is to show something a little harder than back-of-the-envelope calculations but less than simulation, we won't cover such analyses here. (See the references in Appendix M for more details.)



Figure D.15 Treating the I/O system as a black box. This leads to a simple but important observation: If the system is in steady state, then the number of tasks entering the system must equal the number of tasks leaving the system. This *flow-balanced* state is necessary but not sufficient for steady state. If the system has been observed or measured for a sufficiently long time and mean waiting times stabilize, then we say that the system has reached steady state.

Hence, in this section we make the simplifying assumption that we are evaluating systems with multiple independent requests for I/O service that are in equilibrium: The input rate must be equal to the output rate. We also assume there is a steady supply of tasks independent for how long they wait for service. In many real systems, such as TPC-C, the task consumption rate is determined by other system characteristics, such as memory capacity.

This leads us to *Little's law,* which relates the average number of tasks in the system, the average arrival rate of new tasks, and the average time to perform a task:

$$\text{Mean number of tasks in system} = \text{Arrival rate} \times \text{Mean response time}$$

Little's law applies to any system in equilibrium, as long as nothing inside the black box is creating new tasks or destroying them. Note that the arrival rate and the response time must use the same time unit; inconsistency in time units is a common cause of errors.

Let's try to derive Little's law. Assume we observe a system for $\text{Time}_{\text{observe}}$ minutes. During that observation, we record how long it took each task to be serviced, and then sum those times. The number of tasks completed during $\text{Time}_{\text{observe}}$ is $\text{Number}_{\text{task}}$, and the sum of the times each task spends in the system is $\text{Time}_{\text{accumulated}}$. Note that the tasks can overlap in time, so $\text{Time}_{\text{accumulated}} \geq \text{Time}_{\text{observed}}$. Then,

$$\text{Mean number of tasks in system} = \frac{\text{Time}_{\text{accumulated}}}{\text{Time}_{\text{observe}}}$$

$$\text{Mean response time} = \frac{\text{Time}_{\text{accumulated}}}{\text{Number}_{\text{tasks}}}$$

$$\text{Arrival rate} = \frac{\text{Number}_{\text{tasks}}}{\text{Time}_{\text{observe}}}$$

Algebra lets us split the first formula:

$$\frac{\text{Time}_{\text{accumulated}}}{\text{Time}_{\text{observe}}} = \frac{\text{Time}_{\text{accumulated}}}{\text{Number}_{\text{tasks}}} \infty \frac{\text{Number}_{\text{tasks}}}{\text{Time}_{\text{observe}}}$$

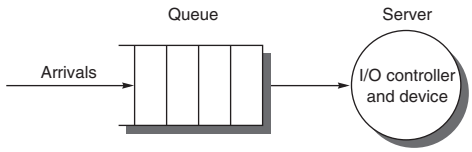


Figure D.16 The single-server model for this section. In this situation, an I/O request "departs" by being completed by the server.

If we substitute the three definitions above into this formula, and swap the resulting two terms on the right-hand side, we get Little's law:

$$\text{Mean number of tasks in system} = \text{Arrival rate} \times \text{Mean response time}$$

This simple equation is surprisingly powerful, as we shall see.

If we open the black box, we see Figure D.16. The area where the tasks accumulate, waiting to be serviced, is called the *queue*, or *waiting line*. The device performing the requested service is called the *server*. Until we get to the last two pages of this section, we assume a single server.

Little's law and a series of definitions lead to several useful equations:

- $\text{Time}_{\text{server}}$—Average time to service a task; average service rate is $1/\text{Time}_{\text{server}}$, traditionally represented by the symbol μ in many queuing texts.
- $\text{Time}_{\text{queue}}$—Average time per task in the queue.
- $\text{Time}_{\text{system}}$—Average time/task in the system, or the response time, which is the sum of $\text{Time}_{\text{queue}}$ and $\text{Time}_{\text{server}}$.
- Arrival rate—Average number of arriving tasks/second, traditionally represented by the symbol λ in many queuing texts.
- $\text{Length}_{\text{server}}$—Average number of tasks in service.
- $\text{Length}_{\text{queue}}$—Average length of queue.
- $\text{Length}_{\text{system}}$—Average number of tasks in system, which is the sum of $\text{Length}_{\text{queue}}$ and $\text{Length}_{\text{server}}$.

One common misunderstanding can be made clearer by these definitions: whether the question is how long a task must wait in the queue before service starts ($\text{Time}_{\text{queue}}$) or how long a task takes until it is completed ($\text{Time}_{\text{system}}$). The latter term is what we mean by response time, and the relationship between the terms is $\text{Time}_{\text{system}} = \text{Time}_{\text{queue}} + \text{Time}_{\text{server}}$.

The mean number of tasks in service ($\text{Length}_{\text{server}}$) is simply Arrival rate $\times$ $\text{Time}_{\text{server}}$, which is Little's law. Server utilization is simply the mean number of tasks being serviced divided by the service rate. For a single server, the service rate is $1/\text{Time}_{\text{server}}$. Hence, server utilization (and, in this case, the mean number of tasks per server) is simply:

$$\text{Server utilization} = \text{Arrival rate} \times \text{Time}_{\text{server}}$$

Service utilization must be between 0 and 1; otherwise, there would be more tasks arriving than could be serviced, violating our assumption that the system is in equilibrium. Note that this formula is just a restatement of Little's law. Utilization is also called *traffic intensity* and is represented by the symbol ρ in many queuing theory texts.

Example Suppose an I/O system with a single disk gets on average 50 I/O requests per second. Assume the average time for a disk to service an I/O request is 10 ms. What is the utilization of the I/O system?

Answer Using the equation above, with 10 ms represented as 0.01 seconds, we get: 50

$$\text{Server utilization} = \text{Arrival rate} \times \text{Time}_{\text{server}} = \frac{50}{\text{sec}} \times 0.01\,\text{sec} = 0.50$$

Therefore, the I/O system utilization is 0.5.

How the queue delivers tasks to the server is called the *queue discipline*. The simplest and most common discipline is *first in, first out* (FIFO). If we assume FIFO, we can relate time waiting in the queue to the mean number of tasks in the queue:

$$\text{Time}_{\text{queue}} = \text{Length}_{\text{queue}} \times \text{Time}_{\text{server}} + \text{Mean time to complete service of task when new task arrives if server is busy}$$

That is, the time in the queue is the number of tasks in the queue times the mean service time plus the time it takes the server to complete whatever task is being serviced when a new task arrives. (There is one more restriction about the arrival of tasks, which we reveal on page D-28.)

The last component of the equation is not as simple as it first appears. A new task can arrive at any instant, so we have no basis to know how long the existing task has been in the server. Although such requests are random events, if we know something about the distribution of events, we can predict performance.

Poisson Distribution of Random Variables

To estimate the last component of the formula we need to know a little about distributions of *random variables*. A variable is random if it takes one of a specified set of values with a specified probability; that is, you cannot know exactly what its next value will be, but you may know the probability of all possible values.

Requests for service from an I/O system can be modeled by a random variable because the operating system is normally switching between several processes that generate independent I/O requests. We also model I/O service times by a random variable given the probabilistic nature of disks in terms of seek and rotational delays.

One way to characterize the distribution of values of a random variable with discrete values is a *histogram*, which divides the range between the minimum and maximum values into subranges called *buckets*. Histograms then plot the number in each bucket as columns.

Histograms work well for distributions that are discrete values—for example, the number of I/O requests. For distributions that are not discrete values, such as time waiting for an I/O request, we have two choices. Either we need a curve to plot the values over the full range, so that we can estimate accurately the value, or we need a very fine time unit so that we get a very large number of buckets to estimate time accurately. For example, a histogram can be built of disk service times measured in intervals of 10 μs although disk service times are truly continuous.

Hence, to be able to solve the last part of the previous equation we need to characterize the distribution of this random variable. The mean time and some measure of the variance are sufficient for that characterization.

For the first term, we use the *weighted arithmetic mean time*. Let's first assume that after measuring the number of occurrences, say, n_i, of tasks, you could compute frequency of occurrence of task i:

$$f_i = \frac{n_i}{\left(\sum_{i=1}^{n} n_i\right)}$$

Then weighted arithmetic mean is

$$\text{Weighted arithmetic mean time} = f_1 \times T_1 + f_2 \times T_2 + \ldots + f_n \times T_n$$

where T_i is the time for task i and f_i is the frequency of occurrence of task i.

To characterize variability about the mean, many people use the standard deviation. Let's use the *variance* instead, which is simply the square of the standard deviation, as it will help us with characterizing the probability distribution. Given the weighted arithmetic mean, the variance can be calculated as

$$\text{Variance} = \left(f_1 \times T_1^2 + f_2 \times T_2^2 + \ldots + f_n \times T_n^2\right) - \text{Weighted arithmetic mean time}^2$$

It is important to remember the units when computing variance. Let's assume the distribution is of time. If time is about 100 milliseconds, then squaring it yields 10,000 square milliseconds. This unit is certainly unusual. It would be more convenient if we had a unitless measure.

To avoid this unit problem, we use the *squared coefficient of variance*, traditionally called C^2:

$$C^2 = \frac{\text{Variance}}{\text{Weighted arithmetic mean time}^2}$$

We can solve for C, the coefficient of variance, as

$$C = \frac{\sqrt{\text{Variance}}}{\text{Weighted arithmetic mean time}} = \frac{\text{Standard deviation}}{\text{Weighted arithmetic mean time}}$$

We are trying to characterize random events, but to be able to predict performance we need a distribution of random events where the mathematics is tractable. The most popular such distribution is the *exponential distribution*, which has a C value of 1.

Note that we are using a constant to characterize variability about the mean. The invariance of C over time reflects the property that the history of events has no impact

on the probability of an event occurring now. This forgetful property is called *memoryless*, and this property is an important assumption used to predict behavior using these models. (Suppose this memoryless property did not exist; then, we would have to worry about the exact arrival times of requests relative to each other, which would make the mathematics considerably less tractable!)

One of the most widely used exponential distributions is called a *Poisson distribution*, named after the mathematician Siméon Poisson. It is used to characterize random events in a given time interval and has several desirable mathematical properties. The Poisson distribution is described by the following equation (called the probability mass function):

$$\text{Probability}(k) = \frac{e^{-a} \times a^{k}}{k!}$$

where a = Rate of events × Elapsed time. If interarrival times are exponentially distributed and we use the arrival rate from above for rate of events, the number of arrivals in a time interval t is a *Poisson process*, which has the Poisson distribution with a = Arrival rate × t. As mentioned on page D-26, the equation for $\text{Time}_{\text{server}}$ has another restriction on task arrival: It holds only for Poisson processes.

Finally, we can answer the question about the length of time a new task must wait for the server to complete a task, called the *average residual service time*, which again assumes Poisson arrivals:

$$\text{Average residual service time} = 1/2 \times \text{Arithemtic mean} \times \left(1 + \text{C}^2\right)$$

Although we won't derive this formula, we can appeal to intuition. When the distribution is not random and all possible values are equal to the average, the standard deviation is 0 and so C is 0. The average residual service time is then just half the average service time, as we would expect. If the distribution is random and it is Poisson, then C is 1 and the average residual service time equals the weighted arithmetic mean time.

Example Using the definitions and formulas above, derive the average time waiting in the queue ($\text{Time}_{\text{queue}}$) in terms of the average service time ($\text{Time}_{\text{server}}$) and server utilization.

Answer All tasks in the queue ($\text{Length}_{\text{queue}}$) ahead of the new task must be completed before the task can be serviced; each takes on average $\text{Time}_{\text{server}}$. If a task is at the server, it takes average residual service time to complete. The chance the server is busy is *server utilization*; hence, the expected time for service is Server utilization × Average residual service time. This leads to our initial formula:

$$\begin{aligned}\text{Time}_{\text{queue}} = {} & \text{Length}_{\text{queue}} \times \text{Time}_{\text{server}} \\ & + \text{Server utilization} \times \text{Average residual service time}\end{aligned}$$

Replacing the average residual service time by its definition and $\text{Length}_{\text{queue}}$ by Arrival rate × $\text{Time}_{\text{queue}}$ yields

$$\text{Time}_{\text{queue}} = \text{Server utilization} \times \left[1/2 \times \text{Time}_{\text{server}} \times \left(1 + C^2\right)\right] + \left(\text{Arrival rate} \times \text{Time}_{\text{queue}}\right) \times \text{Time}_{\text{server}}$$

Since this section is concerned with exponential distributions, C^2 is 1. Thus

$$\text{Time}_{\text{queue}} = \text{Server utilization} \times \text{Time}_{\text{server}} + \left(\text{Arrival rate} \times \text{Time}_{\text{queue}}\right) \times \text{Time}_{\text{server}}$$

Rearranging the last term, let us replace Arrival rate $\times$ $\text{Time}_{\text{server}}$ by Server utilization:

$$\begin{aligned}\text{Time}_{\text{queue}} &= \text{Server utilization} \times \text{Time}_{\text{server}} + (\text{Arrival rate} \times \text{Time}_{\text{server}}) \times \text{Time}_{\text{queue}} \\ &= \text{Server utilization} \times \text{Time}_{\text{server}} + \text{Server utilization} \times \text{Time}_{\text{queue}}\end{aligned}$$

Rearranging terms and simplifying gives us the desired equation:

$$\begin{aligned}\text{Time}_{\text{queue}} &= \text{Server utilization} \times \text{Time}_{\text{server}} + \text{Server utilization} \times \text{Time}_{\text{queue}} \\ \text{Time}_{\text{queue}} - \text{Server utilization} \times \text{Time}_{\text{queue}} &= \text{Server utilization} \times \text{Time}_{\text{server}} \\ \text{Time}_{\text{queue}} \times (1 - \text{Server utilization}) &= \text{Server utilization} \times \text{Time}_{\text{server}} \\ \text{Time}_{\text{queue}} &= \text{Time}_{\text{server}} \times \frac{\text{Server utilization}}{(1 - \text{Server utilization})}\end{aligned}$$

Little's law can be applied to the components of the black box as well, since they must also be in equilibrium:

$$\text{Length}_{\text{queue}} = \text{Arrival rate} \times \text{Time}_{\text{queue}}$$

If we substitute for $\text{Time}_{\text{queue}}$ from above, we get:

$$\text{Length}_{\text{queue}} = \text{Arrival rate} \times \text{Time}_{\text{server}} \times \frac{\text{Server utilization}}{(1 - \text{Server utilization})}$$

Since Arrival rate $\times$ $\text{Time}_{\text{server}}$ = Server utilization, we can simplify further:

$$\text{Length}_{\text{queue}} = \text{Server utilization} \times \frac{\text{Server utilization}}{(1 - \text{Server utilization})} = \frac{\text{Server utilization}^2}{(1 - \text{Server utilization})}$$

This relates number of items in queue to service utilization.

Example For the system in the example on page D-26, which has a server utilization of 0.5, what is the mean number of I/O requests in the queue?

Answer Using the equation above,

$$\text{Length}_{\text{queue}} = \frac{\text{Server uti1ization}^2}{(1 - \text{Server uti1ization})} = \frac{0.5^2}{(1 - 0.5)} = \frac{0.25}{0.50} = 0.5$$

Therefore, there are 0.5 requests on average in the queue.

As mentioned earlier, these equations and this section are based on an area of applied mathematics called *queuing theory*, which offers equations to predict

behavior of such random variables. Real systems are too complex for queuing theory to provide exact analysis, hence queuing theory works best when only approximate answers are needed.

Queuing theory makes a sharp distinction between past events, which can be characterized by measurements using simple arithmetic, and future events, which are predictions requiring more sophisticated mathematics. In computer systems, we commonly predict the future from the past; one example is least recently used block replacement (see Chapter 2). Hence, the distinction between measurements and predicted distributions is often blurred; we use measurements to verify the type of distribution and then rely on the distribution thereafter.

Let's review the assumptions about the queuing model:

- The system is in equilibrium.
- The times between two successive requests arriving, called the *interarrival times*, are exponentially distributed, which characterizes the arrival rate mentioned earlier.
- The number of sources of requests is unlimited. (This is called an *infinite population model* in queuing theory; finite population models are used when arrival rates vary with the number of jobs already in the system.)
- The server can start on the next job immediately after finishing the prior one.
- There is no limit to the length of the queue, and it follows the first in, first out order discipline, so all tasks in line must be completed.
- There is one server.

Such a queue is called *M/M/1:*

M = exponentially random request arrival ($C^2 = 1$), with *M* standing for A. A. Markov, the mathematician who defined and analyzed the memoryless processes mentioned earlier

M = exponentially random service time ($C^2 = 1$), with *M* again for Markov

1 = single server

The M/M/1 model is a simple and widely used model.

The assumption of exponential distribution is commonly used in queuing examples for three reasons—one good, one fair, and one bad. The good reason is that a superposition of many arbitrary distributions acts as an exponential distribution. Many times in computer systems, a particular behavior is the result of many components interacting, so an exponential distribution of interarrival times is the right model. The fair reason is that when variability is unclear, an exponential distribution with intermediate variability ($C = 1$) is a safer guess than low variability ($C \approx 0$) or high variability (large C). The bad reason is that the math is simpler if you assume exponential distributions.

Let's put queuing theory to work in a few examples.

Example Suppose a processor sends 40 disk I/Os per second, these requests are exponentially distributed, and the average service time of an older disk is 20 ms. Answer the following questions:

1. On average, how utilized is the disk?
2. What is the average time spent in the queue?
3. What is the average response time for a disk request, including the queuing time and disk service time?

Answer Let's restate these facts:

Average number of arriving tasks/second is 40.

Average disk time to service a task is 20 ms (0.02 sec).

The server utilization is then

$$\text{Server utilization} = \text{Arrival rate} \times \text{Time}_{\text{server}} = 40 \times 0.02 = 0.8$$

Since the service times are exponentially distributed, we can use the simplified formula for the average time spent waiting in line:

$$\text{Time}_{\text{queue}} = \text{Time}_{\text{server}} \times \frac{\text{Server utilization}}{(1 - \text{Server utilization})}$$

$$= 20 \text{ ms} \times \frac{0.8}{1 - 0.8} = 20 \times \frac{0.8}{0.2} = 20 \times 4 = 80 \text{ ms}$$

The average response time is

$$\text{Time system} = \text{Time}_{\text{queue}} + \text{Time}_{\text{server}} = 80 + 20 \text{ ms} = 100 \text{ ms}$$

Thus, on average we spend 80% of our time waiting in the queue!

Example Suppose we get a new, faster disk. Recalculate the answers to the questions above, assuming the disk service time is 10 ms.

Answer The disk utilization is then

$$\text{Server utilization} = \text{Arrival rate} \times \text{Time}_{\text{server}} = 40 \times 0.01 = 0.4$$

The formula for the average time spent waiting in line:

$$\text{Time}_{\text{queue}} = \text{Time}_{\text{server}} \times \frac{\text{Server utilization}}{(1 - \text{Server utilization})}$$

$$= 10 \text{ ms} \times \frac{0.4}{1 - 0.4} = 10 \times \frac{0.4}{0.6} = 10 \times \frac{2}{3} = 6.7 \text{ ms}$$

The average response time is 10 + 6.7 ms or 16.7 ms, 6.0 times faster than the old response time even though the new service time is only 2.0 times faster.

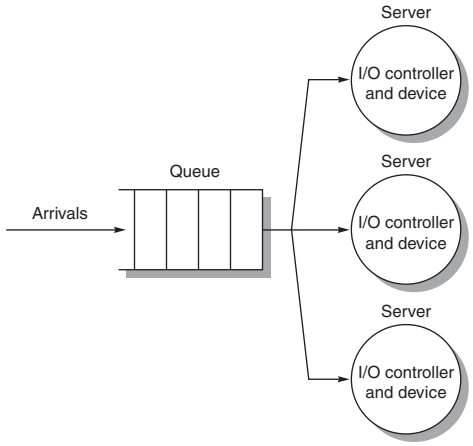


Figure D.17 The M/M/m multiple-server model.

Thus far, we have been assuming a single server, such as a single disk. Many real systems have multiple disks and hence could use multiple servers, as in Figure D.17. Such a system is called an *M/M/m* model in queuing theory.

Let's give the same formulas for the M/M/m queue, using $N_{servers}$ to represent the number of servers. The first two formulas are easy:

$$\text{Utilization} = \frac{\text{Arrival rate} \times \text{Time}_{server}}{N_{servers}}$$

$$\text{Length}_{queue} = \text{Arrival rate} \times \text{Time}_{queue}$$

The time waiting in the queue is

$$\text{Time}_{queue} = \text{Time}_{server} \times \frac{P_{tasks \geq N_{servers}}}{N_{servers} \times (1 - \text{Utilization})}$$

This formula is related to the one for M/M/1, except we replace utilization of a single server with the probability that a task will be queued as opposed to being immediately serviced, and divide the time in queue by the number of servers. Alas, calculating the probability of jobs being in the queue is much more complicated when there are $N_{servers}$. First, the probability that there are no tasks in the system is

$$\text{Prob}_{0\ tasks} = \left[1 + \frac{(N_{servers} \times \text{Utilization})^{N_{servers}}}{N_{servers}! \times (1 - \text{Utilization})} + \sum_{n=1}^{N_{servers}-1} \frac{(N_{servers} \times \text{Utilization})^{n}}{n!} \right]^{-1}$$

Then the probability there are as many or more tasks than we have servers is

$$\text{Prob}_{tasks \geq N_{servers}} = \frac{N_{servers} \times \text{Utilization}^{N_{servers}}}{N_{servers}! \times (1 - \text{Utilization})} \times \text{Prob}_{0\ tasks}$$

Note that if $N_{servers}$ is 1, $Prob_{task \geq N_{servers}}$ simplifies back to Utilization, and we get the same formula as for M/M/1. Let's try an example.

Example Suppose instead of a new, faster disk, we add a second slow disk and duplicate the data so that reads can be serviced by either disk. Let's assume that the requests are all reads. Recalculate the answers to the earlier questions, this time using an M/M/m queue.

Answer The average utilization of the two disks is then

$$\text{Server utilization} = \frac{\text{Arrival rate} \times \text{Time}_{server}}{N_{servers}} = \frac{40 \times 0.02}{2} = 0.4$$

We first calculate the probability of no tasks in the queue:

$$\begin{aligned}
\text{Prob}_{0 \text{ tasks}} &= \left[1 + \frac{(2 \times \text{Utilization})^2}{2! \times (1 - \text{Utilization})} + \sum_{n=1}^{1} \frac{(2 \times \text{Utilization})^n}{n!}\right]^{-1} \\
&= \left[1 + \frac{(2 \times 0.4)^2}{2 \times (1 - 0.4)} + (2 \times 0.4)\right]^{-1} = \left[1 + \frac{0.640}{1.2} + 0.800\right]^{-1} \\
&= [1 + 0.533 + 0.800]^{-1} = 2.333^{-1}
\end{aligned}$$

We use this result to calculate the probability of tasks in the queue:

$$\begin{aligned}
\text{Prob}_{\text{tasks} \geq N_{servers}} &= \frac{2 \times \text{Utilization}^2}{2! \times (1 - \text{Utilization})} \times \text{Prob}_{0 \text{ tasks}} \\
&= \frac{(2 \times 0.4)^2}{2 \times (1 - 0.4)} \times 2.333^{-1} = \frac{0.640}{1.2} \times 2.333^{-1} \\
&= 0.533 / 2.333 = 0.229
\end{aligned}$$

Finally, the time waiting in the queue:

$$\begin{aligned}
\text{Time}_{queue} &= \text{Time}_{server} \times \frac{\text{Prob}_{\text{tasks} \geq N_{servers}}}{N_{servers} \times (1 - \text{Utilization})} \\
&= 0.020 \times \frac{0.229}{2 \times (1 - 0.4)} = 0.020 \times \frac{0.229}{1.2} \\
&= 0.020 \times 0.190 = 0.0038
\end{aligned}$$

The average response time is 20 + 3.8 ms or 23.8 ms. For this workload, two disks cut the queue waiting time by a factor of 21 over a single slow disk and a factor of 1.75 versus a single fast disk. The mean service time of a system with a single fast disk, however, is still 1.4 times faster than one with two disks since the disk service time is 2.0 times faster.

It would be wonderful if we could generalize the M/M/m model to multiple queues and multiple servers, as this step is much more realistic. Alas, these models are very hard to solve and to use, and so we won't cover them here.

D.6 Crosscutting Issues

Point-to-Point Links and Switches Replacing Buses

Point-to-point links and switches are increasing in popularity as Moore's law continues to reduce the cost of components. Combined with the higher I/O bandwidth demands from faster processors, faster disks, and faster local area networks, the decreasing cost advantage of buses means the days of buses in desktop and server computers are numbered. This trend started in high-performance computers in the last edition of the book, and by 2011 has spread itself throughout storage. Figure D.18 shows the old bus-based standards and their replacements.

The number of bits and bandwidth for the new generation is per direction, so they double for both directions. Since these new designs use many fewer wires, a common way to increase bandwidth is to offer versions with several times the number of wires and bandwidth.

Block Servers versus Filers

Thus far, we have largely ignored the role of the operating system in storage. In a manner analogous to the way compilers use an instruction set, operating systems determine what I/O techniques implemented by the hardware will actually be used. The operating system typically provides the file abstraction on top of blocks stored on the disk. The terms *logical units*, *logical volumes*, and *physical volumes* are related terms used in Microsoft and UNIX systems to refer to subset collections of disk blocks.

Standard	Width (bits)	Length (meters)	Clock rate	MB/sec	Max I/O devices
(Parallel) ATA	8	0.5	133 MHz	133	2
Serial ATA	2	2	3 GHz	300	?
SCSI	16	12	80 MHz	320	15
Serial Attach SCSI	1	10	(DDR)	375	16,256
PCI	32/64	0.5	33/66 MHz	533	?
PCI Express	2	0.5	3 GHz	250	?

Figure D.18 Parallel I/O buses and their point-to-point replacements. Note the bandwidth and wires are per direction, so bandwidth doubles when sending both directions.

A logical unit is the element of storage exported from a disk array, usually constructed from a subset of the array's disks. A logical unit appears to the server as a single virtual "disk." In a RAID disk array, the logical unit is configured as a particular RAID layout, such as RAID 5. A physical volume is the device file used by the file system to access a logical unit. A logical volume provides a level of virtualization that enables the file system to split the physical volume across multiple pieces or to stripe data across multiple physical volumes. A logical unit is an abstraction of a disk array that presents a virtual disk to the operating system, while physical and logical volumes are abstractions used by the operating system to divide these virtual disks into smaller, independent file systems.

Having covered some of the terms for collections of blocks, we must now ask: Where should the file illusion be maintained: in the server or at the other end of the storage area network?

The traditional answer is the server. It accesses storage as disk blocks and maintains the metadata. Most file systems use a file cache, so the server must maintain consistency of file accesses. The disks may be *direct attached*—found inside a server connected to an I/O bus—or attached over a storage area network, but the server transmits data blocks to the storage subsystem.

The alternative answer is that the disk subsystem itself maintains the file abstraction, and the server uses a file system protocol to communicate with storage. Example protocols are Network File System (NFS) for UNIX systems and Common Internet File System (CIFS) for Windows systems. Such devices are called *network attached storage* (NAS) devices since it makes no sense for storage to be directly attached to the server. The name is something of a misnomer because a storage area network like FC-AL can also be used to connect to block servers. The term *filer* is often used for NAS devices that only provide file service and file storage. Network Appliance was one of the first companies to make filers.

The driving force behind placing storage on the network is to make it easier for many computers to share information and for operators to maintain the shared system.

Asynchronous I/O and Operating Systems

Disks typically spend much more time in mechanical delays than in transferring data. Thus, a natural path to higher I/O performance is parallelism, trying to get many disks to simultaneously access data for a program.

The straightforward approach to I/O is to request data and then start using it. The operating system then switches to another process until the desired data arrive, and then the operating system switches back to the requesting process. Such a style is called *synchronous I/O*—the process waits until the data have been read from disk.

The alternative model is for the process to continue after making a request, and it is not blocked until it tries to read the requested data. Such *asynchronous I/O*

allows the process to continue making requests so that many I/O requests can be operating simultaneously. Asynchronous I/O shares the same philosophy as caches in out-of-order CPUs, which achieve greater bandwidth by having multiple outstanding events.

D.7 Designing and Evaluating an I/O System—The Internet Archive Cluster

The art of I/O system design is to find a design that meets goals for cost, dependability, and variety of devices while avoiding bottlenecks in I/O performance and dependability. Avoiding bottlenecks means that components must be balanced between main memory and the I/O device, because performance and dependability—and hence effective cost-performance or cost-dependability—can only be as good as the weakest link in the I/O chain. The architect must also plan for expansion so that customers can tailor the I/O to their applications. This expansibility, both in numbers and types of I/O devices, has its costs in longer I/O buses and networks, larger power supplies to support I/O devices, and larger cabinets.

In designing an I/O system, we analyze performance, cost, capacity, and availability using varying I/O connection schemes and different numbers of I/O devices of each type. Here is one series of steps to follow in designing an I/O system. The answers for each step may be dictated by market requirements or simply by cost, performance, and availability goals.

1. List the different types of I/O devices to be connected to the machine, or list the standard buses and networks that the machine will support.
2. List the physical requirements for each I/O device. Requirements include size, power, connectors, bus slots, expansion cabinets, and so on.
3. List the cost of each I/O device, including the portion of cost of any controller needed for this device.
4. List the reliability of each I/O device.
5. Record the processor resource demands of each I/O device. This list should include:
   - Clock cycles for instructions used to initiate an I/O, to support operation of an I/O device (such as handling interrupts), and to complete I/O
   - Processor clock stalls due to waiting for I/O to finish using the memory, bus, or cache
   - Processor clock cycles to recover from an I/O activity, such as a cache flush
6. List the memory and I/O bus resource demands of each I/O device. Even when the processor is not using memory, the bandwidth of main memory and the I/O connection is limited.

7. The final step is assessing the performance and availability of the different ways to organize these I/O devices. When you can afford it, try to avoid single points of failure. Performance can only be properly evaluated with simulation, although it may be estimated using queuing theory. Reliability can be calculated assuming I/O devices fail independently and that the times to failure are exponentially distributed. Availability can be computed from reliability by estimating MTTF for the devices, taking into account the time from failure to repair.

Given your cost, performance, and availability goals, you then select the best organization.

Cost-performance goals affect the selection of the I/O scheme and physical design. Performance can be measured either as megabytes per second or I/Os per second, depending on the needs of the application. For high performance, the only limits should be speed of I/O devices, number of I/O devices, and speed of memory and processor. For low cost, most of the cost should be the I/O devices themselves. Availability goals depend in part on the cost of unavailability to an organization.

Rather than create a paper design, let's evaluate a real system.

The Internet Archive Cluster

To make these ideas clearer, we'll estimate the cost, performance, and availability of a large storage-oriented cluster at the Internet Archive. The Internet Archive began in 1996 with the goal of making a historical record of the Internet as it changed over time. You can use the Wayback Machine interface to the Internet Archive to perform time travel to see what the Web site at a URL looked like sometime in the past. It contains over a petabyte (10^{15} bytes) and is growing by 20 terabytes (10^{12} bytes) of new data per month, so expansible storage is a requirement. In addition to storing the historical record, the same hardware is used to crawl the Web every few months to get snapshots of the Internet.

Clusters of computers connected by local area networks have become a very economical computation engine that works well for some applications. Clusters also play an important role in Internet services such the Google search engine, where the focus is more on storage than it is on computation, as is the case here.

Although it has used a variety of hardware over the years, the Internet Archive is moving to a new cluster to become more efficient in power and in floor space. The basic building block is a 1U storage node called the PetaBox GB2000 from Capricorn Technologies. In 2006, it used four 500 GB Parallel ATA (PATA) disk drives, 512 MB of DDR266 DRAM, one 10/100/1000 Ethernet interface, and a 1 GHz C3 processor from VIA, which executes the 80x86 instruction set. This node dissipates about 80 watts in typical configurations.

Figure D.19 shows the cluster in a standard VME rack. Forty of the GB2000s fit in a standard VME rack, which gives the rack 80 TB of raw capacity. The 40 nodes are connected together with a 48-port 10/100 or 10/100/1000 switch, and it



Figure D.19 The TB-80 VME rack from Capricorn Systems used by the Internet Archive. All cables, switches, and displays are accessible from the front side, and the back side is used only for airflow. This allows two racks to be placed back-to-back, which reduces the floor space demands in machine rooms.

dissipates about 3 KW. The limit is usually 10 KW per rack in computer facilities, so it is well within the guidelines.

A petabyte needs 12 of these racks, connected by a higher-level switch that connects the Gbit links coming from the switches in each of the racks.

Estimating Performance, Dependability, and Cost of the Internet Archive Cluster

To illustrate how to evaluate an I/O system, we'll make some guesses about the cost, performance, and reliability of the components of this cluster. We make the following assumptions about cost and performance:

- The VIA processor, 512 MB of DDR266 DRAM, ATA disk controller, power supply, fans, and enclosure cost $500.
- Each of the four 7200 RPM Parallel ATA drives holds 500 GB, has an average time seek of 8.5 ms, transfers at 50 MB/sec from the disk, and costs $375. The PATA link speed is 133 MB/sec.

- The 48-port 10/100/1000 Ethernet switch and all cables for a rack cost $3000.
- The performance of the VIA processor is 1000 MIPS.
- The ATA controller adds 0.1 ms of overhead to perform a disk I/O.
- The operating system uses 50,000 CPU instructions for a disk I/O.
- The network protocol stacks use 100,000 CPU instructions to transmit a data block between the cluster and the external world.
- The average I/O size is 16 KB for accesses to the historical record via the Wayback interface, and 50 KB when collecting a new snapshot.

Example Evaluate the cost per I/O per second (IOPS) of the 80 TB rack. Assume that every disk I/O requires an average seek and average rotational delay. Assume that the workload is evenly divided among all disks and that all devices can be used at 100% of capacity; that is, the system is limited only by the weakest link, and it can operate that link at 100% utilization. Calculate for both average I/O sizes.

Answer I/O performance is limited by the weakest link in the chain, so we evaluate the maximum performance of each link in the I/O chain for each organization to determine the maximum performance of that organization.

Let's start by calculating the maximum number of IOPS for the CPU, main memory, and I/O bus of one GB2000. The CPU I/O performance is determined by the speed of the CPU and the number of instructions to perform a disk I/O and to send it over the network:

$$\begin{aligned}\text{Maximum IOPS for CPU} &= \frac{1000 \text{ MIPS}}{50{,}000 \text{ instructions per I/O} + 100{,}000 \text{ instructions per message}} \\ &= 6667 \text{ IOPS}\end{aligned}$$

The maximum performance of the memory system is determined by the memory bandwidth and the size of the I/O transfers:

$$\text{Maximum IOPS for main memory} = \frac{266 \times 8}{16 \text{ KB per I/O}} \approx 133{,}000 \text{ IOPS}$$

$$\text{Maximum IOPS for main memory} = \frac{266 \times 8}{50 \text{ KB per I/O}} \approx 42{,}500 \text{ IOPS}$$

The Parallel ATA link performance is limited by the bandwidth and the size of the I/O:

$$\text{Maximum IOPS for the I/O bus} = \frac{133 \text{ MB/sec}}{16 \text{ KB per I/O}} \approx 8300 \text{ IOPS}$$

$$\text{Maximum IOPS for the I/O bus} = \frac{133 \text{ MB/sec}}{50 \text{ KB per I/O}} \approx 2700 \text{ IOPS}$$

Since the box has two buses, the I/O bus limits the maximum performance to no more than 18,600 IOPS for 16 KB blocks and 5400 IOPS for 50 KB blocks.

Now it's time to look at the performance of the next link in the I/O chain, the ATA controllers. The time to transfer a block over the PATA channel is

$$\text{Parallel ATA transfer time} = \frac{16 \text{ KB}}{133 \text{ MB/sec}} \approx 0.1 \text{ ms}$$

$$\text{Parallel ATA transfer time} = \frac{50 \text{ KB}}{133 \text{ MB/sec}} \approx 0.4 \text{ ms}$$

Adding the 0.1 ms ATA controller overhead means 0.2 ms to 0.5 ms per I/O, making the maximum rate per controller

$$\text{Maximum IOPS per ATA controller} = \frac{1}{0.2 \text{ ms}} = 5000 \text{ IOPS}$$

$$\text{Maximum IOPS per ATA controller} = \frac{1}{0.5 \text{ ms}} = 2000 \text{ IOPS}$$

The next link in the chain is the disks themselves. The time for an average disk I/O is

$$\text{I/O time} = 8.5 \text{ ms} + \frac{0.5}{7200 \text{ RPM}} + \frac{16 \text{ KB}}{50 \text{ MB/sec}} = 8.5 + 4.2 + 0.3 = 13.0 \text{ ms}$$

$$\text{I/O time} = 8.5 \text{ ms} + \frac{0.5}{7200 \text{ RPM}} + \frac{50 \text{ KB}}{50 \text{ MB/sec}} = 8.5 + 4.2 + 1.0 = 13.7 \text{ ms}$$

Therefore, disk performance is

$$\text{Maximum IOPS (using average seeks) per disk} = \frac{1}{13.0 \text{ ms}} \approx 77 \text{ IOPS}$$

$$\text{Maximum IOPS (using average seeks) per disk} = \frac{1}{13.7 \text{ ms}} \approx 73 \text{ IOPS}$$

or 292 to 308 IOPS for the four disks.

The final link in the chain is the network that connects the computers to the outside world. The link speed determines the limit:

$$\text{Maximum IOPS per 1000 Mbit Ethernet link} = \frac{1000 \text{ Mbit}}{16 \text{ K} \times 8} = 7812 \text{ IOPS}$$

$$\text{Maximum IOPS per 1000 Mbit Ethernet link} = \frac{1000 \text{ Mbit}}{50 \text{ K} \times 8} = 2500 \text{ IOPS}$$

Clearly, the performance bottleneck of the GB2000 is the disks. The IOPS for the whole rack is 40×308 or 12,320 IOPS to 40×292 or 11,680 IOPS. The network switch would be the bottleneck if it couldn't support $12{,}320 \times 16 \text{ K} \times 8$ or 1.6 Gbits/sec for 16 KB blocks and $11{,}680 \times 50 \text{ K} \times 8$ or 4.7 Gbits/sec for 50 KB blocks. We assume that the extra 8 Gbit ports of the 48-port switch connects the rack to the rest of the world, so it could support the full IOPS of the collective 160 disks in the rack.

Using these assumptions, the cost is $40 \times (\$500 + 4 \times \$375) + \$3000 + \$1500$ or \$84,500 for an 80 TB rack. The disks themselves are almost 60% of the cost. The cost per terabyte is almost \$1000, which is about a factor of 10 to 15 better than storage cluster from the prior edition in 2001. The cost per IOPS is about \$7.

Calculating MTTF of the TB-80 Cluster

Internet services such as Google rely on many copies of the data at the application level to provide dependability, often at different geographic sites to protect against environmental faults as well as hardware faults. Hence, the Internet Archive has two copies of the data in each site and has sites in San Francisco, Amsterdam, and Alexandria, Egypt. Each site maintains a duplicate copy of the high-value content—music, books, film, and video—and a single copy of the historical Web crawls. To keep costs low, there is no redundancy in the 80 TB rack.

Example Let's look at the resulting mean time to fail of the rack. Rather than use the manufacturer's quoted MTTF of 600,000 hours, we'll use data from a recent survey of disk drives [Gray and van Ingen 2005]. As mentioned in Chapter 1, about 3% to 7% of ATA drives fail per year, for an MTTF of about 125,000 to 300,000 hours. Make the following assumptions, again assuming exponential lifetimes:

- CPU/memory/enclosure MTTF is 1,000,000 hours.
- PATA Disk MTTF is 125,000 hours.
- PATA controller MTTF is 500,000 hours.
- Ethernet Switch MTTF is 500,000 hours.
- Power supply MTTF is 200,000 hours.
- Fan MTTF is 200,000 hours.
- PATA cable MTTF is 1,000,000 hours.

Answer Collecting these together, we compute these failure rates:

$$\text{Failure rate} = \frac{40}{1{,}000{,}000} + \frac{160}{125{,}000} + \frac{40}{500{,}000} + \frac{1}{500{,}000} + \frac{40}{200{,}000} + \frac{40}{200{,}000} + \frac{80}{1{,}000{,}000}$$

$$= \frac{40 + 1280 + 80 + 2 + 200 + 200 + 80}{1{,}000{,}000 \text{ hours}} = \frac{1882}{1{,}000{,}000 \text{ hours}}$$

The MTTF for the system is just the inverse of the failure rate:

$$\text{MTTF} = \frac{1}{\text{Failure rate}} = \frac{1{,}000{,}000 \text{ hours}}{1882} = 531 \text{ hours}$$

That is, given these assumptions about the MTTF of components, something in a rack fails on average every 3 weeks. About 70% of the failures would be the disks, and about 20% would be fans or power supplies.

D.8 Putting It All Together: NetApp FAS6000 Filer

Network Appliance entered the storage market in 1992 with a goal of providing an easy-to-operate file server running NSF using their own log-structured file system and a RAID 4 disk array. The company later added support for the Windows CIFS

file system and a RAID 6 scheme called *row-diagonal parity* or *RAID-DP* (see page D-8). To support applications that want access to raw data blocks without the overhead of a file system, such as database systems, NetApp filers can serve data blocks over a standard Fibre Channel interface. NetApp also supports *iSCSI*, which allows SCSI commands to run over a TCP/IP network, thereby allowing the use of standard networking gear to connect servers to storage, such as Ethernet, and hence at a greater distance.

The latest hardware product is the FAS6000. It is a multiprocessor based on the AMD Opteron microprocessor connected using its HyperTransport links. The microprocessors run the NetApp software stack, including NSF, CIFS, RAID-DP, SCSI, and so on. The FAS6000 comes as either a dual processor (FAS6030) or a quad processor (FAS6070). As mentioned in Chapter 5, DRAM is distributed to each microprocessor in the Opteron. The FAS6000 connects 8 GB of DDR2700 to each Opteron, yielding 16 GB for the FAS6030 and 32 GB for the FAS6070. As mentioned in Chapter 4, the DRAM bus is 128 bits wide, plus extra bits for SEC/DED memory. Both models dedicate four HyperTransport links to I/O.

As a filer, the FAS6000 needs a lot of I/O to connect to the disks and to connect to the servers. The integrated I/O consists of:

- 8 Fibre Channel (FC) controllers and ports
- 6 Gigabit Ethernet links
- 6 slots for x8 (2 GB/sec) PCI Express cards
- 3 slots for PCI-X 133 MHz, 64-bit cards
- Standard I/O options such as IDE, USB, and 32-bit PCI

The 8 Fibre Channel controllers can each be attached to 6 shelves containing 14 3.5-inch FC disks. Thus, the maximum number of drives for the integrated I/O is $8 \times 6 \times 14$ or 672 disks. Additional FC controllers can be added to the option slots to connect up to 1008 drives, to reduce the number of drives per FC network so as to reduce contention, and so on. At 500 GB per FC drive, if we assume the RAID RDP group is 14 data disks and 2 check disks, the available data capacity is 294 TB for 672 disks and 441 TB for 1008 disks.

It can also connect to Serial ATA disks via a Fibre Channel to SATA bridge controller, which, as its name suggests, allows FC and SATA to communicate.

The six 1-gigabit Ethernet links connect to servers to make the FAS6000 look like a file server if running NTFS or CIFS or like a block server if running iSCSI.

For greater dependability, FAS6000 filers can be paired so that if one fails, the other can take over. Clustered failover requires that both filers have access to all disks in the pair of filers using the FC interconnect. This interconnect also allows each filer to have a copy of the log data in the NVRAM of the other filer and to keep the clocks of the pair synchronized. The health of the filers is constantly monitored, and failover happens automatically. The healthy filer maintains its own network identity and its own primary functions, but it also assumes the network identity

of the failed filer and handles all its data requests via a virtual filer until an administrator restores the data service to the original state.

D.9 Fallacies and Pitfalls

Fallacy *Components fail fast*

A good deal of the fault-tolerant literature is based on the simplifying assumption that a component operates perfectly until a latent error becomes effective, and then a failure occurs that stops the component.

The Tertiary Disk project had the opposite experience. Many components started acting strangely long before they failed, and it was generally up to the system operator to determine whether to declare a component as failed. The component would generally be willing to continue to act in violation of the service agreement until an operator "terminated" that component.

Figure D.20 shows the history of four drives that were terminated, and the number of hours they started acting strangely before they were replaced.

Fallacy *Computers systems achieve 99.999% availability ("five nines"), as advertised*

Marketing departments of companies making servers started bragging about the availability of their computer hardware; in terms of Figure D.21, they claim availability of 99.999%, nicknamed *five nines*. Even the marketing departments of operating system companies tried to give this impression.

Five minutes of unavailability per year is certainly impressive, but given the failure data collected in surveys, it's hard to believe. For example, Hewlett-Packard claims that the HP-9000 server hardware and HP-UX operating system can deliver

Messages in system log for failed disk	Number of log messages	Duration (hours)
Hardware Failure (Peripheral device write fault [for] Field Replaceable Unit)	1763	186
Not Ready (Diagnostic failure: ASCQ = Component ID [of] Field Replaceable Unit)	1460	90
Recovered Error (Failure Prediction Threshold Exceeded [for] Field Replaceable Unit)	1313	5
Recovered Error (Failure Prediction Threshold Exceeded [for] Field Replaceable Unit)	431	17

Figure D.20 Record in system log for 4 of the 368 disks in Tertiary Disk that were replaced over 18 months. See Talagala and Patterson [1999]. These messages, matching the SCSI specification, were placed into the system log by device drivers. Messages started occurring as much as a week before one drive was replaced by the operator. The third and fourth messages indicate that the drive's failure prediction mechanism detected and predicted imminent failure, yet it was still hours before the drives were replaced by the operator.

Unavailability (minutes per year)	Availability (percent)	Availability class ("number of nines")
50,000	90%	1
5000	99%	2
500	99.9%	3
50	99.99%	4
5	99.999%	5
0.5	99.9999%	6
0.05	99.99999%	7

Figure D.21 Minutes unavailable per year to achieve availability class. (From Gray and Siewiorek [1991].) Note that five nines mean unavailable five minutes per year.

a 99.999% availability guarantee "in certain pre-defined, pre-tested customer environments" (see Hewlett-Packard [1998]). This guarantee does not include failures due to operator faults, application faults, or environmental faults, which are likely the dominant fault categories today. Nor does it include scheduled downtime. It is also unclear what the financial penalty is to a company if a system does not match its guarantee.

Microsoft also promulgated a five nines marketing campaign. In January 2001, *www.microsoft.com* was unavailable for 22 hours. For its Web site to achieve 99.999% availability, it will require a clean slate for 250 years.

In contrast to marketing suggestions, well-managed servers typically achieve 99% to 99.9% availability.

Pitfall *Where a function is implemented affects its reliability*

In theory, it is fine to move the RAID function into software. In practice, it is very difficult to make it work reliably.

The software culture is generally based on eventual correctness via a series of releases and patches. It is also difficult to isolate from other layers of software. For example, proper software behavior is often based on having the proper version and patch release of the operating system. Thus, many customers have lost data due to software bugs or incompatibilities in environment in software RAID systems.

Obviously, hardware systems are not immune to bugs, but the hardware culture tends to place a greater emphasis on testing correctness in the initial release. In addition, the hardware is more likely to be independent of the version of the operating system.

Fallacy *Operating systems are the best place to schedule disk accesses*

Higher-level interfaces such as ATA and SCSI offer logical block addresses to the host operating system. Given this high-level abstraction, the best an OS can do is to try to sort the logical block addresses into increasing order. Since only the disk knows the mapping of the logical addresses onto the physical geometry of sectors, tracks, and surfaces, it can reduce the rotational and seek latencies.

For example, suppose the workload is four reads [Anderson 2003]:

Operation	Starting LBA	Length
Read	724	8
Read	100	16
Read	9987	1
Read	26	128

The host might reorder the four reads into logical block order:

Read	26	128
Read	100	16
Read	724	8
Read	9987	1

Depending on the relative location of the data on the disk, reordering could make it worse, as Figure D.22 shows. The disk-scheduled reads complete in three-quarters of a disk revolution, but the OS-scheduled reads take three revolutions.

Fallacy *The time of an average seek of a disk in a computer system is the time for a seek of one-third the number of cylinders*

This fallacy comes from confusing the way manufacturers market disks with the expected performance, and from the false assumption that seek times are linear in distance. The one-third-distance rule of thumb comes from calculating the distance of a seek from one random location to another random location, not including the current track and assuming there is a large number of tracks. In the past, manufacturers listed the seek of this distance to offer a consistent basis for comparison. (Today, they

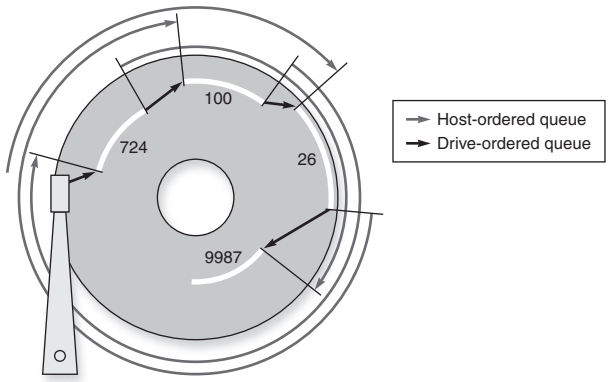


Figure D.22 Example showing OS versus disk schedule accesses, labeled host-ordered versus drive-ordered. The former takes 3 revolutions to complete the 4 reads, while the latter completes them in just 3/4 of a revolution. (From Anderson [2003].)

calculate the "average" by timing all seeks and dividing by the number.) Assuming (incorrectly) that seek time is linear in distance, and using the manufacturer's reported minimum and "average" seek times, a common technique to predict seek time is

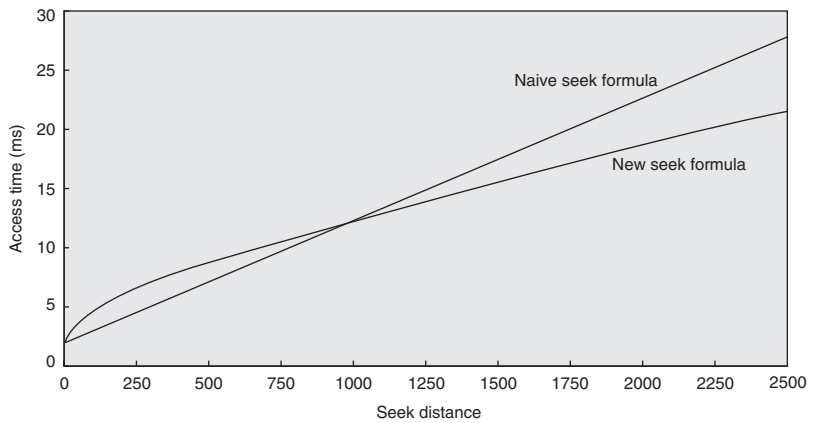
$$\text{Time}_{\text{seek}} = \text{Time}_{\text{minimum}} + \frac{\text{Distance}}{\text{Distance}_{\text{average}}} \times \left(\text{Time}_{\text{average}} - \text{Time}_{\text{minimum}}\right)$$

The fallacy concerning seek time is twofold. First, seek time is *not* linear with distance; the arm must accelerate to overcome inertia, reach its maximum traveling speed, decelerate as it reaches the requested position, and then wait to allow the arm to stop vibrating (*settle time*). Moreover, sometimes the arm must pause to control vibrations. For disks with more than 200 cylinders, Chen and Lee [1995] modeled the seek distance as:

$$\text{Seek time}(\text{Distance}) = a \times \sqrt{\text{Distance} - 1} + b \times (\text{Distance} - 1) + c$$

where a, b, and c are selected for a particular disk so that this formula will match the quoted times for Distance = 1, Distance = max, and Distance = 1/3 max. Figure D.23 plots this equation versus the fallacy equation. Unlike the first equation, the square root of the distance reflects acceleration and deceleration.

The second problem is that the average in the product specification would only be true if there were no locality to disk activity. Fortunately, there is both temporal and spatial locality (see page B-2 in Appendix B). For example, Figure D.24 shows sample measurements of seek distances for two workloads: a UNIX time-sharing workload and a business-processing workload. Notice the high percentage of disk



$$a = \frac{-10 \times \text{Time}_{\text{min}} + 15 \times \text{Time}_{\text{avg}} - 5 \times \text{Time}_{\text{max}}}{3 \times \sqrt{\text{Number of cylinders}}} \quad b = \frac{7 \times \text{Time}_{\text{min}} - 15 \times \text{Time}_{\text{avg}} + 8 \times \text{Time}_{\text{max}}}{3 \times \text{Number of cylinders}} \quad c = \text{Time}_{\text{min}}$$

Figure D.23 Seek time versus seek distance for sophisticated model versus naive model. Chen and Lee [1995] found that the equations shown above for parameters *a*, *b*, and *c* worked well for several disks.

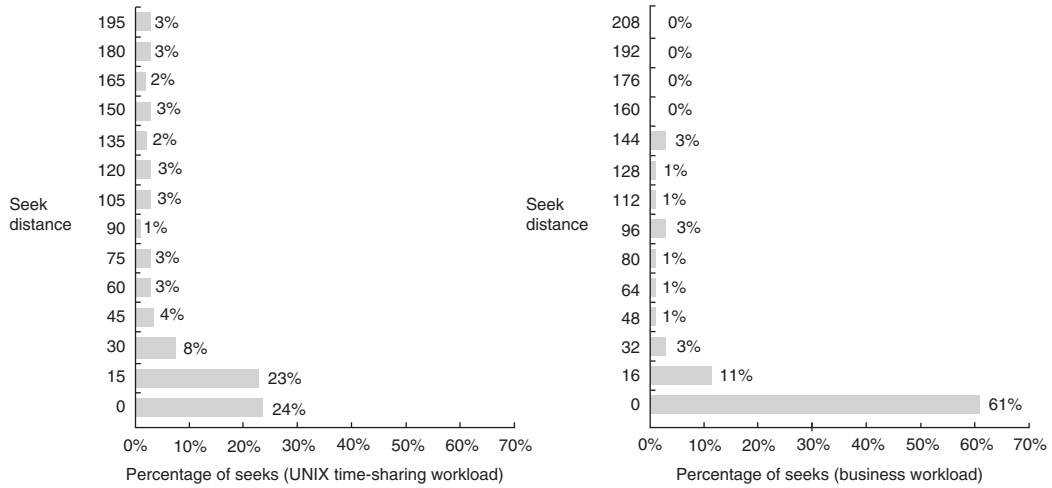


Figure D.24 Sample measurements of seek distances for two systems. The measurements on the left were taken on a UNIX time-sharing system. The measurements on the right were taken from a business-processing application in which the disk seek activity was scheduled to improve throughput. Seek distance of 0 means the access was made to the same cylinder. The rest of the numbers show the collective percentage for distances between numbers on the *y*-axis. For example, 11% for the bar labeled 16 in the business graph means that the percentage of seeks between 1 and 16 cylinders was 11%. The UNIX measurements stopped at 200 of the 1000 cylinders, but this captured 85% of the accesses. The business measurements tracked all 816 cylinders of the disks. The only seek distances with 1% or greater of the seeks that are not in the graph are 224 with 4%, and 304, 336, 512, and 624, each having 1%. This total is 94%, with the difference being small but nonzero distances in other categories. Measurements courtesy of Dave Anderson of Seagate.

accesses to the same cylinder, labeled distance 0 in the graphs, in both workloads. Thus, this fallacy couldn't be more misleading.

D.10 Concluding Remarks

Storage is one of those technologies that we tend to take for granted. And yet, if we look at the true status of things today, storage is king. One can even argue that servers, which have become commodities, are now becoming peripheral to storage devices. Driving that point home are some estimates from IBM, which expects storage sales to surpass server sales in the next two years.

Michael Vizard
Editor-in-chief, *Infoworld* (August 11, 2001)

As their value is becoming increasingly evident, storage systems have become the target of innovation and investment.

The challenges for storage systems today are dependability and maintainability. Not only do users want to be sure their data are never lost (reliability),

applications today increasingly demand that the data are always available to access (availability). Despite improvements in hardware and software reliability and fault tolerance, the awkwardness of maintaining such systems is a problem both for cost and for availability. A widely mentioned statistic is that customers spend $6 to $8 operating a storage system for every $1 of purchase price. When dependability is attacked by having many redundant copies at a higher level of the system—such as for search—then very large systems can be sensitive to the price-performance of the storage components.

Today, challenges in storage dependability and maintainability dominate the challenges of I/O.

D.11 Historical Perspective and References

Section M.9 (available online) covers the development of storage devices and techniques, including who invented disks, the story behind RAID, and the history of operating systems and databases. References for further reading are included.

Case Studies with Exercises by Andrea C. Arpaci-Dusseau and Remzi H. Arpaci-Dusseau

Case Study 1: Deconstructing a Disk

Concepts illustrated by this case study

- Performance Characteristics
- Microbenchmarks

The internals of a storage system tend to be hidden behind a simple interface, that of a linear array of blocks. There are many advantages to having a common interface for all storage systems: An operating system can use any storage system without modification, and yet the storage system is free to innovate behind this interface. For example, a single disk can map its internal < sector, track, surface > geometry to the linear array in whatever way achieves the best performance; similarly, a multidisk RAID system can map the blocks on any number of disks to this same linear array. However, this fixed interface has a number of disadvantages, as well; in particular, the operating system is not able to perform some performance, reliability, and security optimizations without knowing the precise layout of its blocks inside the underlying storage system.

In this case study, we will explore how software can be used to uncover the internal structure of a storage system hidden behind a block-based interface. The basic idea is to *fingerprint* the storage system: by running a well-defined workload on top of the storage system and measuring the amount of time required for different requests, one is able to infer a surprising amount of detail about the underlying system.

The Skippy algorithm, from work by Nisha Talagala and colleagues at the University of California–Berkeley, uncovers the parameters of a single disk. The key is to factor out disk rotational effects by making consecutive seeks to individual sectors with addresses that differ by a linearly increasing amount (increasing by 1, 2, 3, and so forth). Thus, the basic algorithm skips through the disk, increasing the distance of the seek by one sector before every write, and outputs the distance and time for each write. The raw device interface is used to avoid file system optimizations. The `SECTOR SIZE` is set equal to the minimum amount of data that can be read at once from the disk (e.g., 512 bytes). (Skippy is described in more detail in Talagala and Patterson [1999].)

```
fd = open("raw disk device");
for (i = 0; i < measurements; i++) {
    begin_time = gettime();
    lseek(fd, i*SECTOR_SIZE, SEEK_CUR);
    write(fd, buffer, SECTOR_SIZE);
    interval_time = gettime() -begin_time;
    printf("Stride: %d Time: %d\n", i, interval_time);
}
close(fd);
```

By graphing the time required for each write as a function of the seek distance, one can infer the minimal transfer time (with no seek or rotational latency), head switch time, cylinder switch time, rotational latency, and the number of heads in the disk. A typical graph will have four distinct lines, each with the same slope, but with different offsets. The highest and lowest lines correspond to requests that incur different amounts of rotational delay, but no cylinder or head switch costs; the difference between these two lines reveals the rotational latency of the disk. The second lowest line corresponds to requests that incur a head switch (in addition to increasing amounts of rotational delay). Finally, the third line corresponds to requests that incur a cylinder switch (in addition to rotational delay).

D.1 [10/10/10/10/10] < D.2 > The results of running Skippy are shown for a mock disk (Disk Alpha) in Figure D.25.

a. [10] < D.2 > What is the minimal transfer time?

b. [10] < D.2 > What is the rotational latency?

c. [10] < D.2 > What is the head switch time?

d. [10] < D.2 > What is the cylinder switch time?

e. [10] < D.2 > What is the number of disk heads?

D.2 [25] < D.2 > Draw an approximation of the graph that would result from running Skippy on Disk Beta, a disk with the following parameters:

- Minimal transfer time, 2.0 ms
- Rotational latency, 6.0 ms

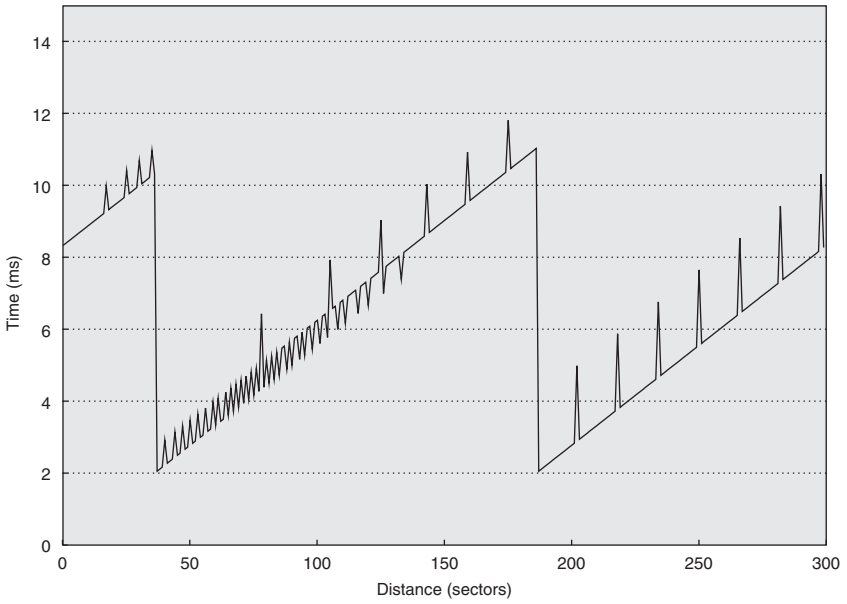


Figure D.25 Results from running Skippy on Disk Alpha.

- Head switch time, 1.0 ms
- Cylinder switch time, 1.5 ms
- Number of disk heads, 4
- Sectors per track, 100

D.3 [10/10/10/10/10/10/10] < D.2 > Implement and run the Skippy algorithm on a disk drive of your choosing.

a. [10] < D.2 > Graph the results of running Skippy. Report the manufacturer and model of your disk.

b. [10] < D.2 > What is the minimal transfer time?

c. [10] < D.2 > What is the rotational latency?

d. [10] < D.2 > What is the head switch time?

e. [10] < D.2 > What is the cylinder switch time?

f. [10] < D.2 > What is the number of disk heads?

g. [10] < D.2 > Do the results of running Skippy on a real disk differ in any qualitative way from that of the mock disk?

Case Study 2: Deconstructing a Disk Array

Concepts illustrated by this case study

- Performance Characteristics
- Microbenchmarks

The Shear algorithm, from work by Timothy Denehy and colleagues at the University of Wisconsin [Denehy et al. 2004], uncovers the parameters of a RAID system. The basic idea is to generate a workload of requests to the RAID array and time those requests; by observing which sets of requests take longer, one can infer which blocks are allocated to the same disk.

We define RAID properties as follows. Data are allocated to disks in the RAID at the block level, where a *block* is the minimal unit of data that the file system reads or writes from the storage system; thus, block size is known by the file system and the fingerprinting software. A *chunk* is a set of blocks that is allocated contiguously within a disk. A *stripe* is a set of chunks across each of D data disks. Finally, a *pattern* is the minimum sequence of data blocks such that block offset i within the pattern is always located on disk j.

D.4 [20/20] <D.2> One can uncover the pattern size with the following code. The code accesses the raw device to avoid file system optimizations. The key to all of the Shear algorithms is to use random requests to avoid triggering any of the prefetch or caching mechanisms within the RAID or within individual disks. The basic idea of this code sequence is to access N random blocks at a fixed interval p within the RAID array and to measure the completion time of each interval.

```
for (p = BLOCKSIZE; p <= testsize; p += BLOCKSIZE) {
    for (i = 0; i < N; i++) {
        request[i] = random()*p;
    }
     begin_time = gettime();

     issues all request[N] to raw device in parallel;

     wait for all request[N] to complete;
     interval_time = gettime() - begin_time;
     printf("PatternSize: %d Time: %d\n", p,
        interval_time);
}
```

If you run this code on a RAID array and plot the measured time for the N requests as a function of p, then you will see that the time is highest when all N requests fall on the same disk; thus, the value of p with the highest time corresponds to the pattern size of the RAID.

a. [20] <D.2> Figure D.26 shows the results of running the pattern size algorithm on an unknown RAID system.

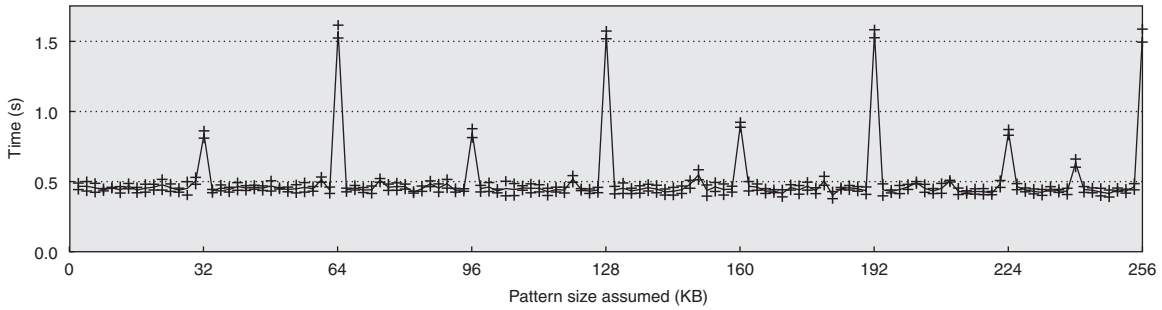


Figure D.26 Results from running the pattern size algorithm of Shear on a mock storage system.

- What is the pattern size of this storage system?
- What do the measured times of 0.4, 0.8, and 1.6 seconds correspond to in this storage system?
- If this is a RAID 0 array, then how many disks are present?
- If this is a RAID 0 array, then what is the chunk size?

b. [20] <D.2> Draw the graph that would result from running this Shear code on a storage system with the following characteristics:

- Number of requests, $N = 1000$
- Time for a random read on disk, 5 ms
- RAID level, RAID 0
- Number of disks, 4
- Chunk size, 8 KB

D.5 [20/20] <D.2> One can uncover the chunk size with the following code. The basic idea is to perform reads from N patterns chosen at random but always at controlled offsets, c and $c-1$, within the pattern.

```
for (c = 0; c < patternsize; c += BLOCKSIZE) {
    for (i = 0; i < N; i++) {
        requestA[i] = random()*patternsize + c;
        requestB[i] = random()*patternsize +
            (c-1)%patternsize;
    }
        begin_time = gettime();

        issue all requestA[N] and requestB[N] to raw device
            in parallel;
        wait for requestA[N] and requestB[N] to complete;

        interval_time = gettime() - begin_time;
        printf("ChunkSize: %d Time: %d\n", c,
            interval_time);
}
```

If you run this code and plot the measured time as a function of *c*, then you will see that the measured time is lowest when the *requestA* and *requestB* reads fall on two different disks. Thus, the values of *c* with low times correspond to the chunk boundaries between disks of the RAID.

a. [20] <D.2> Figure D.27 shows the results of running the chunk size algorithm on an unknown RAID system.
   - What is the chunk size of this storage system?
   - What do the measured times of 0.75 and 1.5 seconds correspond to in this storage system?

b. [20] <D.2> Draw the graph that would result from running this Shear code on a storage system with the following characteristics:
   - Number of requests, $N = 1000$
   - Time for a random read on disk, 5 ms
   - RAID level, RAID 0
   - Number of disks, 8
   - Chunk size, 12 KB

D.6 [10/10/10/10] <D.2> Finally, one can determine the layout of chunks to disks with the following code. The basic idea is to select *N* random patterns and to exhaustively read together all pairwise combinations of the chunks within the pattern.

```
for (a = 0; a < numchunks; a += chunksize) {
    for (b = a; b < numchunks; b += chunksize) {
        for (i = 0; i < N; i++) {
            requestA[i] = random()*patternsize + a;
            requestB[i] = random()*patternsize + b;
    }
    begin_time = gettime();
    issue all requestA[N] and requestB[N] to raw device
      in parallel;
```

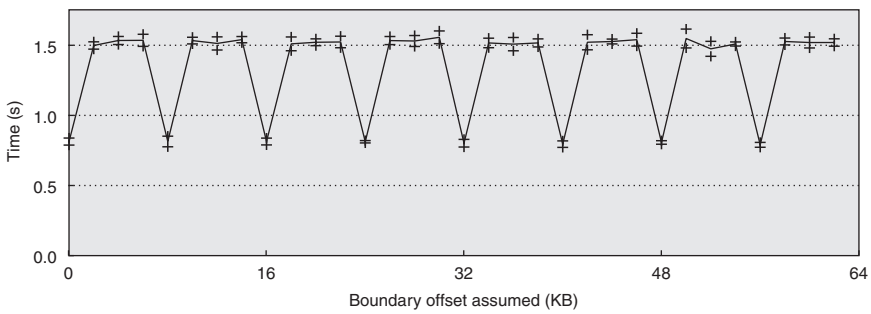


Figure D.27 Results from running the chunk size algorithm of Shear on a mock storage system.

```
            wait for all requestA[N] and requestB[N] to
              complete;
            interval_time = gettime() - begin_time;
            printf("A: %d B: %d Time: %d\n", a, b,
              interval_time);
          }
        }
```

After running this code, you can report the measured time as a function of *a* and *b*. The simplest way to graph this is to create a two-dimensional table with *a* and *b* as the parameters and the time scaled to a shaded value; we use darker shadings for faster times and lighter shadings for slower times. Thus, a light shading indicates that the two offsets of *a* and *b* within the pattern fall on the same disk.

Figure D.28 shows the results of running the layout algorithm on a storage system that is known to have a pattern size of 384 KB and a chunk size of 32 KB.

a. [20] < D.2 > How many chunks are in a pattern?

b. [20] < D.2 > Which chunks of each pattern appear to be allocated on the same disks?

c. [20] < D.2 > How many disks appear to be in this storage system?

d. [20] < D.2 > Draw the likely layout of blocks across the disks.

D.7 [20] < D.2 > Draw the graph that would result from running the layout algorithm on the storage system shown in Figure D.29. This storage system has four disks and a chunk size of four 4 KB blocks (16 KB) and is using a RAID 5 Left-Asymmetric layout.

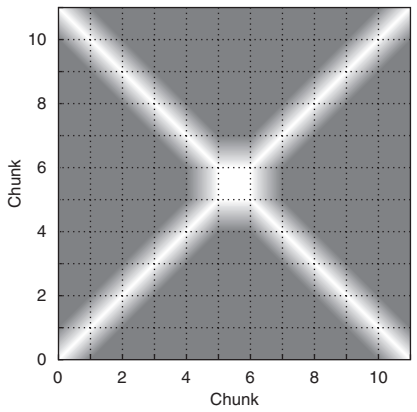


Figure D.28 Results from running the layout algorithm of Shear on a mock storage system.

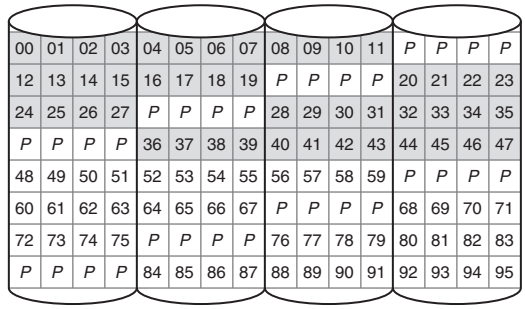


Figure D.29 A storage system with four disks, a chunk size of four 4 KB blocks, and using a RAID 5 Left-Asymmetric layout. Two repetitions of the pattern are shown.

Case Study 3: RAID Reconstruction

Concepts illustrated by this case study

- RAID Systems
- RAID Reconstruction
- Mean Time to Failure (MTTF)
- Mean Time until Data Loss (MTDL)
- Performability
- Double Failures

A RAID system ensures that data are not lost when a disk fails. Thus, one of the key responsibilities of a RAID is to reconstruct the data that were on a disk when it failed; this process is called *reconstruction* and is what you will explore in this case study. You will consider both a RAID system that can tolerate one disk failure and a RAID-DP, which can tolerate two disk failures.

Reconstruction is commonly performed in two different ways. In *offline reconstruction*, the RAID devotes all of its resources to performing reconstruction and does not service any requests from the workload. In *online reconstruction*, the RAID continues to service workload requests while performing the reconstruction; the reconstruction process is often limited to use some fraction of the total bandwidth of the RAID system.

How reconstruction is performed impacts both the *reliability* and the *performability* of the system. In a RAID 5, data are lost if a second disk fails before the data from the first disk can be recovered; therefore, the longer the reconstruction time (MTTR), the lower the reliability or the *mean time until data loss* (MTDL). Performability is a metric meant to combine both the performance of a system and its

availability; it is defined as the performance of the system in a given state multiplied by the probability of that state. For a RAID array, possible states include normal operation with no disk failures, reconstruction with one disk failure, and shutdown due to multiple disk failures.

For these exercises, assume that you have built a RAID system with six disks, plus a sufficient number of hot spares. Assume that each disk is the 37 GB SCSI disk shown in Figure D.3 and that each disk can sequentially read data at a peak of 142 MB/sec and sequentially write data at a peak of 85 MB/sec. Assume that the disks are connected to an Ultra320 SCSI bus that can transfer a total of 320 MB/sec. You can assume that each disk failure is independent and ignore other potential failures in the system. For the reconstruction process, you can assume that the overhead for any XOR computation or memory copying is negligible. During online reconstruction, assume that the reconstruction process is limited to use a total bandwidth of 10 MB/sec from the RAID system.

D.8 [10] <D.2> Assume that you have a RAID 4 system with six disks. Draw a simple diagram showing the layout of blocks across disks for this RAID system.

D.9 [10] <D.2, D.4> When a single disk fails, the RAID 4 system will perform reconstruction. What is the expected time until a reconstruction is needed?

D.10 [10/10/10] <D.2, D.4> Assume that reconstruction of the RAID 4 array begins at time *t*.

a. [10] <D.2, D.4> What read and write operations are required to perform the reconstruction?

b. [10] <D.2, D.4> For offline reconstruction, when will the reconstruction process be complete?

c. [10] <D.2, D.4> For online reconstruction, when will the reconstruction process be complete?

D.11 [10/10/10/10] <D.2, D.4> In this exercise, we will investigate the mean time until data loss (MTDL). In RAID 4, data are lost only if a second disk fails before the first failed disk is repaired.

a. [10] <D.2, D.4> What is the likelihood of having a second failure during offline reconstruction?

b. [10] <D.2, D.4> Given this likelihood of a second failure during reconstruction, what is the MTDL for offline reconstruction?

c. [10] <D.2, D.4> What is the likelihood of having a second failure during online reconstruction?

d. [10] <D.2, D.4> Given this likelihood of a second failure during reconstruction, what is the MTDL for online reconstruction?

D.12 [10] <D.2, D.4> What is performability for the RAID 4 array for offline reconstruction? Calculate the performability using IOPS, assuming a random readonly workload that is evenly distributed across the disks of the RAID 4 array.

D.13 [10] <D.2, D.4> What is the performability for the RAID 4 array for online reconstruction? During online repair, you can assume that the IOPS drop to 70% of their peak rate. Does offline or online reconstruction lead to better performability?

D.14 [10] <D.2, D.4> RAID 6 is used to tolerate up to two simultaneous disk failures. Assume that you have a RAID 6 system based on row-diagonal parity, or RAID-DP; your six-disk RAID-DP system is based on RAID 4, with $p = 5$, as shown in Figure D.5. If data disk 0 and data disk 3 fail, how can those disks be reconstructed? Show the sequence of steps that are required to compute the missing blocks in the first four stripes.

Case Study 4: Performance Prediction for RAIDs

Concepts illustrated by this case study

- RAID Levels
- Queuing Theory
- Impact of Workloads
- Impact of Disk Layout

In this case study, you will explore how simple queuing theory can be used to predict the performance of the I/O system. You will investigate how both storage system configuration and the workload influence service time, disk utilization, and average response time.

The configuration of the storage system has a large impact on performance. Different RAID levels can be modeled using queuing theory in different ways. For example, a RAID 0 array containing N disks can be modeled as N separate systems of M/M/1 queues, assuming that requests are appropriately distributed across the N disks. The behavior of a RAID 1 array depends upon the workload: A read operation can be sent to either mirror, whereas a write operation must be sent to both disks. Therefore, for a read-only workload, a two-disk RAID 1 array can be modeled as an M/M/2 queue, whereas for a write-only workload, it can be modeled as an M/M/1 queue. The behavior of a RAID 4 array containing N disks also depends upon the workload: A read will be sent to a particular data disk, whereas writes must all update the parity disk, which becomes the bottleneck of the system. Therefore, for a read-only workload, RAID 4 can be modeled as $N - 1$ separate systems, whereas for a write-only workload, it can be modeled as one M/M/1 queue.

The layout of blocks within the storage system can have a significant impact on performance. Consider a single disk with a 40 GB capacity. If the workload randomly accesses 40 GB of data, then the layout of those blocks to the disk does not have much of an impact on performance. However, if the workload randomly accesses only half of the disk's capacity (i.e., 20 GB of data on that disk), then layout does matter: To reduce seek time, the 20 GB of data can be compacted within 20 GB of consecutive tracks instead of allocated uniformly distributed over the entire 40 GB capacity.

For this problem, we will use a rather simplistic model to estimate the service time of a disk. In this basic model, the average positioning and transfer time for a small random request is a linear function of the seek distance. For the 40 GB disk in this problem, assume that the service time is 5 *ms* * space utilization. Thus, if the entire 40 GB disk is used, then the average positioning and transfer time for a random request is 5 ms; if only the first 20 GB of the disk is used, then the average positioning and transfer time is 2.5 ms.

Throughout this case study, you can assume that the processor sends 167 small random disk requests per second and that these requests are exponentially distributed. You can assume that the size of the requests is equal to the block size of 8 KB. Each disk in the system has a capacity of 40 GB. Regardless of the storage system configuration, the workload accesses a total of 40 GB of data; you should allocate the 40 GB of data across the disks in the system in the most efficient manner.

D.15 [10/10/10/10/10] < D.5 > Begin by assuming that the storage system consists of a single 40 GB disk.

a. [10] < D.5 > Given this workload and storage system, what is the average service time?

b. [10] < D.5 > On average, what is the utilization of the disk?

c. [10] < D.5 > On average, how much time does each request spend waiting for the disk?

d. [10] < D.5 > What is the mean number of requests in the queue?

e. [10] < D.5 > Finally, what is the average response time for the disk requests?

D.16 [10/10/10/10/10/10] < D.2, D.5 > Imagine that the storage system is now configured to contain two 40 GB disks in a RAID 0 array; that is, the data are striped in blocks of 8 KB equally across the two disks with no redundancy.

a. [10] < D.2, D.5 > How will the 40 GB of data be allocated across the disks? Given a random request workload over a total of 40 GB, what is the expected service time of each request?

b. [10] < D.2, D.5 > How can queuing theory be used to model this storage system?

c. [10] < D.2, D.5 > What is the average utilization of each disk?

d. [10] < D.2, D.5 > On average, how much time does each request spend waiting for the disk?

e. [10] < D.2, D.5 > What is the mean number of requests in each queue?

f. [10] < D.2, D.5 > Finally, what is the average response time for the disk requests?

D.17 [20/20/20/20/20] < D.2, D.5 > Instead imagine that the storage system is configured to contain two 40 GB disks in a RAID 1 array; that is, the data are mirrored

across the two disks. Use queuing theory to model this system for a read-only workload.

a. [20] <D.2, D.5> How will the 40 GB of data be allocated across the disks? Given a random request workload over a total of 40 GB, what is the expected service time of each request?

b. [20] <D.2, D.5> How can queuing theory be used to model this storage system?

c. [20] <D.2, D.5> What is the average utilization of each disk?

d. [20] <D.2, D.5> On average, how much time does each request spend waiting for the disk?

e. [20] <D.2, D.5> Finally, what is the average response time for the disk requests?

D.18 [10/10] <D.2, D.5> Imagine that instead of a read-only workload, you now have a write-only workload on a RAID 1 array.

a. [10] <D.2, D.5> Describe how you can use queuing theory to model this system and workload.

b. [10] <D.2, D.5> Given this system and workload, what are the average utilization, average waiting time, and average response time?

Case Study 5: I/O Subsystem Design

Concepts illustrated by this case study

- RAID Systems
- Mean Time to Failure (MTTF)
- Performance and Reliability Trade-Offs

In this case study, you will design an I/O subsystem, given a monetary budget. Your system will have a minimum required capacity and you will optimize for performance, reliability, or both. You are free to use as many disks and controllers as fit within your budget.

Here are your building blocks:

- A 10,000 MIPS CPU costing $1000. Its MTTF is 1,000,000 hours.
- A 1000 MB/sec I/O bus with room for 20 Ultra320 SCSI buses and controllers.
- Ultra320 SCSI buses that can transfer 320 MB/sec and support up to 15 disks per bus (these are also called *SCSI strings*). The SCSI cable MTTF is 1,000,000 hours.
- An Ultra320 SCSI controller that is capable of 50,000 IOPS, costs $250, and has an MTTF of 500,000 hours.

- A $2000 enclosure supplying power and cooling to up to eight disks. The enclosure MTTF is 1,000,000 hours, the fan MTTF is 200,000 hours, and the power supply MTTF is 200,000 hours.
- The SCSI disks described in Figure D.3.
- Replacing any failed component requires 24 hours.

You may make the following assumptions about your workload:

- The operating system requires 70,000 CPU instructions for each disk I/O.
- The workload consists of many concurrent, random I/Os, with an average size of 16 KB.

All of your constructed systems must have the following properties:

- You have a monetary budget of $28,000.
- You must provide at least 1 TB of capacity.

D.19 [10] < D.2 > You will begin by designing an I/O subsystem that is optimized only for capacity and performance (and not reliability), specifically IOPS. Discuss the RAID level and block size that will deliver the best performance.

D.20 [20/20/20/20] < D.2, D.4, D.7 > What configuration of SCSI disks, controllers, and enclosures results in the best performance given your monetary and capacity constraints?

a. [20] < D.2, D.4, D.7 > How many IOPS do you expect to deliver with your system?

b. [20] < D.2, D.4, D.7 > How much does your system cost?

c. [20] < D.2, D.4, D.7 > What is the capacity of your system?

d. [20] < D.2, D.4, D.7 > What is the MTTF of your system?

D.21 [10] < D.2, D.4, D.7 > You will now redesign your system to optimize for reliability, by creating a RAID 10 or RAID 01 array. Your storage system should be robust not only to disk failures but also to controller, cable, power supply, and fan failures as well; specifically, a single component failure should not prohibit accessing both replicas of a pair. Draw a diagram illustrating how blocks are allocated across disks in the RAID 10 and RAID 01 configurations. Is RAID 10 or RAID 01 more appropriate in this environment?

D.22 [20/20/20/20/20] < D.2, D.4, D.7 > Optimizing your RAID 10 or RAID 01 array only for reliability (but staying within your capacity and monetary constraints), what is your RAID configuration?

a. [20] < D.2, D.4, D.7 > What is the overall MTTF of the components in your system?

b. [20] < D.2, D.4, D.7 > What is the MTDL of your system?

c. [20] < D.2, D.4, D.7 > What is the usable capacity of this system?

d. [20] < D.2, D.4, D.7 > How much does your system cost?

e. [20] < D.2, D.4, D.7 > Assuming a write-only workload, how many IOPS can you expect to deliver?

D.23 [10] < D.2, D.4, D.7 > Assume that you now have access to a disk that has twice the capacity, for the same price. If you continue to design only for reliability, how would you change the configuration of your storage system? Why?

Case Study 6: Dirty Rotten Bits

Concepts illustrated by this case study

- Partial Disk Failure
- Failure Analysis
- Performance Analysis
- Parity Protection
- Checksumming

You are put in charge of avoiding the problem of "bit rot"—bits or blocks in a file going bad over time. This problem is particularly important in archival scenarios, where data are written once and perhaps accessed many years later; without taking extra measures to protect the data, the bits or blocks of a file may slowly change or become unavailable due to media errors or other I/O faults.

Dealing with bit rot requires two specific components: detection and recovery. To detect bit rot efficiently, one can use checksums over each block of the file in question; a checksum is just a function of some kind that takes a (potentially long) string of data as input and outputs a fixed-size string (the checksum) of the data as output. The property you will exploit is that if the data changes then the computed checksum is very likely to change as well.

Once detected, recovering from bit rot requires some form of redundancy. Examples include mirroring (keeping multiple copies of each block) and parity (some extra redundant information, usually more space efficient than mirroring).

In this case study, you will analyze how effective these techniques are given various scenarios. You will also write code to implement data integrity protection over a set of files.

D.24 [20/20/20] < D.2 > Assume that you will use simple parity protection in Exercises D.24 through D.27. Specifically, assume that you will be computing *one* parity block for each file in the file system. Further, assume that you will also use a 20-byte MD5 checksum per 4 KB block of each file.

We first tackle the problem of space overhead. According to studies by Douceur and Bolosky [1999], these file size distributions are what is found in modern PCs:

≤1 KB	2 KB	4 KB	8 KB	16 KB	32 KB	64 KB	128 KB	256 KB	512 KB	≥1 MB
26.6%	11.0%	11.2%	10.9%	9.5%	8.5%	7.1%	5.1%	3.7%	2.4%	4.0%

The study also finds that file systems are usually about half full. Assume that you have a 37 GB disk volume that is roughly half full and follows that same distribution, and answer the following questions:

a. [20] < D.2 > How much extra information (both in bytes and as a percent of the volume) must you keep on disk to be able to detect a single error with checksums?

b. [20] < D.2 > How much extra information (both in bytes and as a percent of the volume) would you need to be able to both detect a single error with checksums as well as correct it?

c. [20] < D.2 > Given this file distribution, is the block size you are using to compute checksums too big, too little, or just right?

D.25 [10/10] < D.2, D.3 > One big problem that arises in data protection is error detection. One approach is to perform error detection *lazily*—that is, wait until a file is accessed, and at that point, check it and make sure the correct data are there. The problem with this approach is that files that are not accessed frequently may slowly rot away and when finally accessed have too many errors to be corrected. Hence, an eager approach is to perform what is sometimes called *disk scrubbing*—periodically go through all data and find errors proactively.

a. [10] < D.2, D.3 > Assume that bit flips occur independently, at a rate of 1 flip per GB of data per month. Assuming the same 20 GB volume that is half full, and assuming that you are using the SCSI disk as specified in Figure D.3 (4 ms seek, roughly 100 MB/sec transfer), how often should you scan through files to check and repair their integrity?

b. [10] < D.2, D.3 > At what bit flip rate does it become impossible to maintain data integrity? Again assume the 20 GB volume and the SCSI disk.

D.26 [10/10/10/10] < D.2, D.4 > Another potential cost of added data protection is found in performance overhead. We now study the performance overhead of this data protection approach.

a. [10] < D.2, D.4 > Assume we write a 40 MB file to the SCSI disk sequentially, and then write out the extra information to implement our data protection scheme to disk once. How much *write traffic* (both in total volume of bytes and as a percentage of total traffic) does our scheme generate?

b. [10] < D.2, D.4 > Assume we now are updating the file randomly, similar to a database table. That is, assume we perform a series of 4 KB random writes to the file, and each time we perform a single write, we must update the on-disk protection information. Assuming that we perform 10,000 random writes, how

much *I/O traffic* (both in total volume of bytes and as a percentage of total traffic) does our scheme generate?

c. [10] <D.2, D.4> Now assume that the data protection information is always kept in a separate portion of the disk, away from the file it is guarding (that is, assume for each file A, there is another file $A_{\text{checksums}}$ that holds all the check-sums for A). Hence, one potential overhead we must incur arises upon reads—that is, upon each read, we will use the checksum to detect data corruption.

Assume you read 10,000 blocks of 4 KB each sequentially from disk. Assuming a 4 ms average seek cost and a 100 MB/sec transfer rate (like the SCSI disk in Figure D.3), how long will it take to read the file (and corresponding checksums) from disk? What is the time penalty due to adding checksums?

d. [10] <D.2, D.4> Again assuming that the data protection information is kept separate as in part (c), now assume you have to read 10,000 random blocks of 4 KB each from a very large file (much bigger than 10,000 blocks, that is). For each read, you must again use the checksum to ensure data integrity. How long will it take to read the 10,000 blocks from disk, again assuming the same disk characteristics? What is the time penalty due to adding checksums?

D.27 [40] <D.2, D.3, D.4> Finally, we put theory into practice by developing a user-level tool to guard against file corruption. Assume you are to write a simple set of tools to detect and repair data integrity. The first tool is used for checksums and parity. It should be called `build` and used like this:

```
build <filename>
```

The `build` program should then store the needed checksum and redundancy information for the file `filename` in a file in the same directory called `.filename.cp` (so it is easy to find later).

A second program is then used to check and potentially repair damaged files. It should be called `repair` and used like this:

```
repair <filename>
```

The `repair` program should consult the `.cp` file for the filename in question and verify that all the stored checksums match the computed checksums for the data. If the checksums don't match for a single block, `repair` should use the redundant information to reconstruct the correct data and fix the file. However, if two or more blocks are bad, `repair` should simply report that the file has been corrupted beyond repair. To test your system, we will provide a tool to corrupt files called `corrupt`. It works as follows:

```
corrupt <filename> <blocknumber>
```

All `corrupt` does is fill the specified block number of the file with random noise. For checksums you will be using MD5. MD5 takes an input string and gives you a

128-bit "fingerprint" or checksum as an output. A great and simple implementation of MD5 is available here:

http://sourceforge.net/project/showfiles.php?group_id=42360

Parity is computed with the XOR operator. In C code, you can compute the parity of two blocks, each of size `BLOCKSIZE`, as follows:

```
unsigned char block1[BLOCKSIZE];
unsigned char block2[BLOCKSIZE];

unsigned char parity[BLOCKSIZE];

// first, clear parity block
for (int i = 0; i < BLOCKSIZE; i++)
     parity[i] = 0;

// then compute parity; carat symbol does XOR in C
for (int i = 0; i < BLOCKSIZE; i++) {
     parity[i] = block1[i] ^block2[i];
}
```

Case Study 7: Sorting Things Out

Concepts illustrated by this case study

- Benchmarking
- Performance Analysis
- Cost/Performance Analysis
- Amortization of Overhead
- Balanced Systems

The database field has a long history of using benchmarks to compare systems. In this question, you will explore one of the benchmarks introduced by Anon. et al. [1985] (see Chapter 1): external, or disk-to-disk, sorting.

Sorting is an exciting benchmark for a number of reasons. First, sorting exercises a computer system across all its components, including disk, memory, and processors. Second, sorting at the highest possible performance requires a great deal of expertise about how the CPU caches, operating systems, and I/O subsystems work. Third, it is simple enough to be implemented by a student (see below!).

Depending on how much data you have, sorting can be done in one or multiple passes. Simply put, if you have enough memory to hold the entire dataset in memory, you can read the entire dataset into memory, sort it, and then write it out; this is called a "one-pass" sort.

If you do not have enough memory, you must sort the data in multiple passes. There are many different approaches possible. One simple approach is to sort each

chunk of the input file and write it to disk; this leaves (input file size)/(memory size) sorted files on disk. Then, you have to merge each sorted temporary file into a final sorted output. This is called a "two-pass" sort. More passes are needed in the unlikely case that you cannot merge all the streams in the second pass.

In this case study, you will analyze various aspects of sorting, determining its effectiveness and cost-effectiveness in different scenarios. You will also write your own version of an external sort, measuring its performance on real hardware.

D.28 [20/20/20] < D.4 > We will start by configuring a system to complete a sort in the least possible time, with no limits on how much we can spend. To get peak bandwidth from the sort, we have to make sure all the paths through the system have sufficient bandwidth.

Assume for simplicity that the time to perform the in-memory sort of keys is linearly proportional to the CPU rate and memory bandwidth of the given machine (e.g., sorting 1 MB of records on a machine with 1 MB/sec of memory bandwidth and a 1 MIPS processor will take 1 second). Assume further that you have carefully written the I/O phases of the sort so as to achieve sequential bandwidth. And, of course, realize that if you don't have enough memory to hold all of the data at once that sort will take two passes.

One problem you may encounter in performing I/O is that systems often perform extra *memory copies*; for example, when the `read()` system call is invoked, data may first be read from disk into a system buffer and then subsequently copied into the specified user buffer. Hence, memory bandwidth during I/O can be an issue.

Finally, for simplicity, assume that there is no overlap of reading, sorting, or writing. That is, when you are reading data from disk, that is all you are doing; when sorting, you are just using the CPU and memory bandwidth; when writing, you are just writing data to disk.

Your job in this task is to configure a system to extract peak performance when sorting 1 GB of data (i.e., roughly 10 million 100-byte records). Use the following table to make choices about which machine, memory, I/O interconnect, and disks to buy.

CPU			**I/O interconnect**		
Slow	1 GIPS	$200	Slow	80 MB/sec	$50
Standard	2 GIPS	$1000	Standard	160 MB/sec	$100
Fast	4 GIPS	$2000	Fast	320 MB/sec	$400
Memory			**Disks**		
Slow	512 MB/sec	$100/GB	Slow	30 MB/sec	$70
Standard	1 GB/sec	$200/GB	Standard	60 MB/sec	$120
Fast	2 GB/sec	$500/GB	Fast	110 MB/sec	$300

Note: Assume that you are buying a single-processor system and that you can have up to two I/O interconnects. However, the amount of memory and number of disks are up to you (assume there is no limit on disks per I/O interconnect).

a. [20] <D.4> What is the total cost of your machine? (Break this down by part, including the cost of the CPU, amount of memory, number of disks, and I/O bus.)

b. [20] <D.4> How much time does it take to complete the sort of 1 GB worth of records? (Break this down into time spent doing reads from disk, writes to disk, and time spent sorting.)

c. [20] <D.4> What is the bottleneck in your system?

D.29 [25/25/25] <D.4> We will now examine cost-performance issues in sorting. After all, it is easy to buy a high-performing machine; it is much harder to buy a costeffective one.

One place where this issue arises is with the PennySort competition (*research.microsoft.com/barc/SortBenchmark/*). PennySort asks that you sort as many records as you can for a single penny. To compute this, you should assume that a system you buy will last for 3 years (94,608,000 seconds), and divide this by the total cost in pennies of the machine. The result is your time budget per penny.

Our task here will be a little simpler. Assume you have a fixed budget of $2000 (or less). What is the fastest sorting machine you can build? Use the same hardware table as in Exercise D.28 to configure the winning machine.

(*Hint:* You might want to write a little computer program to generate all the possible configurations.)

a. [25] <D.4> What is the total cost of your machine? (Break this down by part, including the cost of the CPU, amount of memory, number of disks, and I/O bus.)

b. [25] <D.4> How does the reading, writing, and sorting time break down with this configuration?

c. [25] <D.4> What is the bottleneck in your system?

D.30 [20/20/20] <D.4, D.6> Getting good disk performance often requires *amortization of overhead*. The idea is simple: If you must incur an overhead of some kind, do as much useful work as possible after paying the cost and hence reduce its impact. This idea is quite general and can be applied to many areas of computer systems; with disks, it arises with the seek and rotational costs (overheads) that you must incur before transferring data. You can amortize an expensive seek and rotation by transferring a large amount of data.

In this exercise, we focus on how to amortize seek and rotational costs during the second pass of a two-pass sort. Assume that when the second pass begins, there are N sorted runs on the disk, each of a size that fits within main memory. Our task here is to read in a chunk from each sorted run and merge the results into a final sorted

output. Note that a read from one run will incur a seek and rotation, as it is very likely that the last read was from a different run.

a. [20] <D.4, D.6> Assume that you have a disk that can transfer at 100 MB/sec, with an average seek cost of 7 ms, and a rotational rate of 10,000 RPM. Assume further that every time you read from a run, you read 1 MB of data and that there are 100 runs each of size 1 GB. Also assume that writes (to the final sorted output) take place in large 1 GB chunks. How long will the merge phase take, assuming I/O is the dominant (i.e., only) cost?

b. [20] <D.4, D.6> Now assume that you change the read size from 1 MB to 10 MB. How is the total time to perform the second pass of the sort affected?

c. [20] <D.4, D.6> In both cases, assume that what we wish to maximize is *disk efficiency*. We compute disk efficiency as the ratio of the time spent transferring data over the total time spent accessing the disk. What is the disk efficiency in each of the scenarios mentioned above?

D.31 [40] <D.2, D.4, D.6> In this exercise, you will write your own external sort. To generate the data set, we provide a tool `generate` that works as follows:

```
generate <filename> <size (in MB)>
```

By running `generate`, you create a file named `filename` of size `size` MB. The file consists of 100 byte keys, with 10-byte records (the part that must be sorted).

We also provide a tool called `check` that checks whether a given input file is sorted or not. It is run as follows:

```
check <filename>
```

The basic one-pass sort does the following: reads in the data, sorts the data, and then writes the data out. However, numerous optimizations are available to you: overlapping reading and sorting, separating keys from the rest of the record for better cache behavior and hence faster sorting, overlapping sorting and writing, and so forth.

One important rule is that data must always start on disk (and not in the file system cache). The easiest way to ensure this is to unmount and remount the file system.

One goal: Beat the Datamation sort record. Currently, the record for sorting 1 million 100-byte records is 0.44 seconds, which was obtained on a cluster of 32 machines. If you are careful, you might be able to beat this on a single PC configured with a few disks.

E.1	Introduction	E-2
E.2	Signal Processing and Embedded Applications: The Digital Signal Processor	E-5
E.3	Embedded Benchmarks	E-12
E.4	Embedded Multiprocessors	E-14
E.5	Case Study: The Emotion Engine of the Sony PlayStation 2	E-15
E.6	Case Study: Sanyo VPC-SX500 Digital Camera	E-19
E.7	Case Study: Inside a Cell Phone	E-20
E.8	Concluding Remarks	E-25

E

Embedded Systems

By Thomas M. Conte
North Carolina State University

Where a calculator on the ENIAC is equipped with 18,000 vacuum tubes and weighs 30 tons, computers in the future may have only 1,000 vacuum tubes and perhaps weigh 1 1/2 tons.

Popular Mechanics
March 1949

E.1 Introduction

Embedded computer systems—computers lodged in other devices where the presence of the computers is not immediately obvious—are the fastest-growing portion of the computer market. These devices range from everyday machines (most microwaves, most washing machines, printers, network switches, and automobiles contain simple to very advanced embedded microprocessors) to handheld digital devices (such as PDAs, cell phones, and music players) to video game consoles and digital set-top boxes. Although in some applications (such as PDAs) the computers are programmable, in many embedded applications the only programming occurs in connection with the initial loading of the application code or a later software upgrade of that application. Thus, the application is carefully tuned for the processor and system. This process sometimes includes limited use of assembly language in key loops, although time-to-market pressures and good software engineering practice restrict such assembly language coding to a fraction of the application.

Compared to desktop and server systems, embedded systems have a much wider range of processing power and cost—from systems containing low-end 8-bit and 16-bit processors that may cost less than a dollar, to those containing full 32-bit microprocessors capable of operating in the 500 MIPS range that cost approximately 10 dollars, to those containing high-end embedded processors that cost hundreds of dollars and can execute several billions of instructions per second. Although the range of computing power in the embedded systems market is very large, price is a key factor in the design of computers for this space. Performance requirements do exist, of course, but the primary goal is often meeting the performance need at a minimum price, rather than achieving higher performance at a higher price.

Embedded systems often process information in very different ways from general-purpose processors. Typically these applications include deadline-driven constraints—so-called *real-time constraints.* In these applications, a particular computation must be completed by a certain time or the system fails (there are other constraints considered real time, discussed in the next subsection).

Embedded systems applications typically involve processing information as *signals.* The lay term "signal" often connotes radio transmission, and that is true for some embedded systems (e.g., cell phones). But a signal may be an image, a motion picture composed of a series of images, a control sensor measurement, and so on. Signal processing requires specific computation that many embedded processors are optimized for. We discuss this in depth below. A wide range of benchmark requirements exist, from the ability to run small, limited code segments to the ability to perform well on applications involving tens to hundreds of thousands of lines of code.

Two other key characteristics exist in many embedded applications: the need to minimize memory and the need to minimize power. In many embedded applications, the memory can be a substantial portion of the system cost, and it is important to optimize memory size in such cases. Sometimes the application is expected to fit

entirely in the memory on the processor chip; other times the application needs to fit in its entirety in a small, off-chip memory. In either case, the importance of memory size translates to an emphasis on code size, since data size is dictated by the application. Some architectures have special instruction set capabilities to reduce code size. Larger memories also mean more power, and optimizing power is often critical in embedded applications. Although the emphasis on low power is frequently driven by the use of batteries, the need to use less expensive packaging (plastic versus ceramic) and the absence of a fan for cooling also limit total power consumption. We examine the issue of power in more detail later in this appendix.

Another important trend in embedded systems is the use of processor cores together with application-specific circuitry—so-called "core plus ASIC" or "system on a chip" (SOC), which may also be viewed as special-purpose multiprocessors (see Section E.4). Often an application's functional and performance requirements are met by combining a custom hardware solution together with software running on a standardized embedded processor core, which is designed to interface to such special-purpose hardware. In practice, embedded problems are usually solved by one of three approaches:

1. The designer uses a combined hardware/software solution that includes some custom hardware and an embedded processor core that is integrated with the custom hardware, often on the same chip.
2. The designer uses custom software running on an off-the-shelf embedded processor.
3. The designer uses a digital signal processor and custom software for the processor. *Digital signal processors* are processors specially tailored for signal-processing applications. We discuss some of the important differences between digital signal processors and general-purpose embedded processors below.

Figure E.1 summarizes these three classes of computing environments and their important characteristics.

Real-Time Processing

Often, the performance requirement in an embedded application is a real-time requirement. A *real-time performance requirement* is one where a segment of the application has an absolute maximum execution time that is allowed. For example, in a digital set-top box the time to process each video frame is limited, since the processor must accept and process the frame before the next frame arrives (typically called *hard real-time systems*). In some applications, a more sophisticated requirement exists: The average time for a particular task is constrained as well as is the number of instances when some maximum time is exceeded. Such approaches (typically called *soft real-time*) arise when it is possible to occasionally miss the time constraint on an event, as long as not too many are missed. Real-time

Feature	Desktop	Server	Embedded
Price of system	$1000–$10,000	$10,000–$10,000,000	$10–$100,000 (including network routers at the high end)
Price of microprocessor module	$100–$1000	$200–$2000 (per processor)	$0.20–$200 (per processor)
Microprocessors sold per year (estimates for 2000)	150,000,000	4,000,000	300,000,000 (32-bit and 64-bit processors only)
Critical system design issues	Price-performance, graphics performance	Throughput, availability, scalability	Price, power consumption, application-specific performance

Figure E.1 A summary of the three computing classes and their system characteristics. Note the wide range in system price for servers and embedded systems. For servers, this range arises from the need for very large-scale multiprocessor systems for high-end transaction processing and Web server applications. For embedded systems, one significant high-end application is a network router, which could include multiple processors as well as lots of memory and other electronics. The total number of embedded processors sold in 2000 is estimated to exceed 1 billion, if you include 8-bit and 16-bit microprocessors. In fact, the largest-selling microprocessor of all time is an 8-bit microcontroller sold by Intel! It is difficult to separate the low end of the server market from the desktop market, since low-end servers—especially those costing less than $5000—are essentially no different from desktop PCs. Hence, up to a few million of the PC units may be effectively servers.

performance tends to be highly application dependent. It is usually measured using kernels either from the application or from a standardized benchmark (see Section E.3).

The construction of a hard real-time system involves three key variables. The first is the rate at which a particular task must occur. Coupled to this are the hardware and software required to achieve that real-time rate. Often, structures that are very advantageous on the desktop are the enemy of hard real-time analysis. For example, branch speculation, cache memories, and so on introduce *uncertainty* into code. A particular sequence of code may execute either very efficiently or very inefficiently, depending on whether the hardware branch predictors and caches "do their jobs." Engineers must analyze code assuming the *worst-case execution time* (WCET). In the case of traditional microprocessor hardware, if one assumes that *all branches are mispredicted* and *all caches miss,* the WCET is overly pessimistic. Thus, the system designer may end up overdesigning a system to achieve a given WCET, when a much less expensive system would have sufficed.

In order to address the challenges of hard real-time systems, and yet still exploit such well-known architectural properties as branch behavior and access locality, it is possible to change how a processor is designed. Consider branch prediction: Although dynamic branch prediction is known to perform far more accurately than static "hint bits" added to branch instructions, the behavior of static hints is much more predictable. Furthermore, although caches perform better than software-managed on-chip memories, the latter produces predictable memory latencies. In some embedded processors, caches can be converted into software-managed on-chip memories via *line locking*. In this approach, a cache line can be locked in the cache so that it cannot be replaced until the line is unlocked

E.2 Signal Processing and Embedded Applications: The Digital Signal Processor

A digital signal processor (DSP) is a special-purpose processor optimized for executing digital signal processing algorithms. Most of these algorithms, from time-domain filtering (e.g., infinite impulse response and finite impulse response filtering), to convolution, to transforms (e.g., fast Fourier transform, discrete cosine transform), to even forward error correction (FEC) encodings, all have as their kernel the same operation: a multiply-accumulate operation. For example, the discrete Fourier transform has the form:

$$X(k) = \sum_{n=0}^{N-1} x(n) W_N^{kn} \quad \text{where} \quad W_N^{kn} = e^{j\frac{2\pi kn}{N}} = \cos\left(2\pi\frac{kn}{N}\right) + j\sin\left(2\pi\frac{kn}{N}\right)$$

The discrete cosine transform is often a replacement for this because it does not require complex number operations. Either transform has as its core the *sum* of a *product*. To accelerate this, DSPs typically feature special-purpose hardware to perform *multiply-accumulate* (MAC). A MAC instruction of "MAC A,B,C" has the semantics of "A = A + B * C." In some situations, the performance of this operation is so critical that a DSP is selected for an application based solely upon its MAC operation throughput.

DSPs often employ *fixed-point* arithmetic. If you think of integers as having a binary point to the right of the least-significant bit, fixed point has a binary point just to the right of the sign bit. Hence, fixed-point data are fractions between −1 and +1.

Example Here are three simple 16-bit patterns:

0100 0000 0000 0000

0000 1000 0000 0000

0100 1000 0000 1000

What values do they represent if they are two's complement integers? Fixedpoint numbers?

Answer Number representation tells us that the *i*th digit to the left of the binary point represents 2^{i-1} and the *i*th digit to the right of the binary point represents 2^{-i}. First assume these three patterns are integers. Then the binary point is to the far right, so they represent 2^{14}, 2^{11}, and (2^{14}+ 2^{11}+ 2^{3}), or 16,384, 2048, and 18,440.

Fixed point places the binary point just to the right of the sign bit, so as fixed point these patterns represent 2^{-1}, 2^{-4}, and (2^{-1} + 2^{-4} + 2^{-12}). The fractions are 1/2, 1/16, and (2048 + 256 + 1)/4096 or 2305/4096, which represents about 0.50000, 0.06250, and 0.56274. Alternatively, for an *n*-bit two's complement,

fixed-point number we could just divide the integer presentation by 2^{n-1} to derive the same results:

$$16,384/32,768 = 1/2,\ 2048/32,768 = 1/16,\ \text{and}\ 18,440/32,768 = 2305/4096.$$

Fixed point can be thought of as a low-cost floating point. It doesn't include an exponent in every word and doesn't have hardware that automatically aligns and normalizes operands. Instead, fixed point relies on the DSP programmer to keep the exponent in a separate variable and ensure that each result is shifted left or right to keep the answer aligned to that variable. Since this exponent variable is often shared by a set of fixed-point variables, this style of arithmetic is also called *blocked floating point*, since a block of variables has a common exponent.

To support such manual calculations, DSPs usually have some registers that are wider to guard against round-off error, just as floating-point units internally have extra guard bits. Figure E.2 surveys four generations of DSPs, listing data sizes and width of the accumulating registers. Note that DSP architects are not bound by the powers of 2 for word sizes. Figure E.3 shows the size of data operands for the TI TMS320C55 DSP.

In addition to MAC operations, DSPs often also have operations to accelerate portions of communications algorithms. An important class of these algorithms revolve around encoding and decoding *forward error correction codes*—codes in which extra information is added to the digital bit stream to guard against errors in transmission. A code of rate *m*/*n* has *m* information bits for (*m* + *n*) check bits. So, for example, a 1/2 rate code would have 1 information bit per every 2 bits. Such codes are often called *trellis codes* because one popular graphical flow diagram of

Generation	Year	Example DSP	Data width	Accumulator width
1	1982	TI TMS32010	16 bits	32 bits
2	1987	Motorola DSP56001	24 bits	56 bits
3	1995	Motorola DSP56301	24 bits	56 bits
4	1998	TI TMS320C6201	16 bits	40 bits

Figure E.2 Four generations of DSPs, their data width, and the width of the registers that reduces round-off error.

Data size	Memory operand in operation	Memory operand in data transfer
16 bits	89.3%	89.0%
32 bits	10.7%	11.0%

Figure E.3 Size of data operands for the TMS320C55 DSP. About 90% of operands are 16 bits. This DSP has two 40-bit accumulators. There are no floating-point operations, as is typical of many DSPs, so these data are all fixed-point integers.

their encoding resembles a garden trellis. A common algorithm for decoding trellis codes is due to Viterbi. This algorithm requires a sequence of compares and selects in order to recover a transmitted bit's true value. Thus DSPs often have compare-select operations to support Viterbi decode for FEC codes.

To explain DSPs better, we will take a detailed look at two DSPs, both produced by Texas Instruments. The TMS320C55 series is a DSP family targeted toward battery-powered embedded applications. In stark contrast to this, the TMS VelociTI 320C6x series is a line of powerful, eight-issue VLIW processors targeted toward a broader range of applications that may be less power sensitive.

The TI 320C55

At one end of the DSP spectrum is the TI 320C55 architecture. The C55 is optimized for low-power, embedded applications. Its overall architecture is shown in Figure E.4. At the heart of it, the C55 is a seven-staged pipelined CPU. The stages are outlined below:

- *Fetch stage* reads program data from memory into the instruction buffer queue.
- *Decode stage* decodes instructions and dispatches tasks to the other primary functional units.
- *Address stage* computes addresses for data accesses and branch addresses for program discontinuities.
- *Access 1/Access 2 stages* send data read addresses to memory.
- *Read stage* transfers operand data on the B bus, C bus, and D bus.
- *Execute stage* executes operation in the A unit and D unit and performs writes on the E bus and F bus.

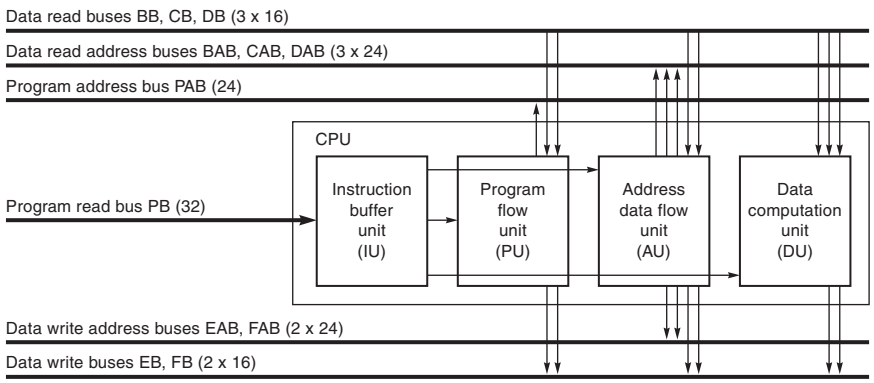


Figure E.4 Architecture of the TMS320C55 DSP. The C55 is a seven-stage pipelined processor with some unique instruction execution facilities. (Courtesy Texas Instruments.)

The C55 pipeline performs pipeline hazard detection and will stall on write after read (WAR) and read after write (RAW) hazards.

The C55 does have a 24 KB instruction cache, but it is configurable to support various workloads. It may be configured to be two-way set associative, direct-mapped, or as a "ramset." This latter mode is a way to support hard realtime applications. In this mode, blocks in the cache cannot be replaced.

The C55 also has advanced power management. It allows dynamic power management through software-programmable "idle domains." Blocks of circuitry on the device are organized into these idle domains. Each domain can operate normally or can be placed in a low-power idle state. A programmer-accessible Idle Control Register (ICR) determines which domains will be placed in the idle state when the execution of the next `IDLE` instruction occurs. The six domains are CPU, direct memory access (DMA), peripherals, clock generator, instruction cache, and external memory interface. When each domain is in the idle state, the functions of that particular domain are not available. However, in the peripheral domain, each peripheral has an Idle Enable bit that controls whether or not the peripheral will respond to the changes in the idle state. Thus, peripherals can be individually configured to idle or remain active when the peripheral domain is idled.

Since the C55 is a DSP, the central feature is its MAC units. The C55 has two MAC units, each comprised of a 17-bit by 17-bit multiplier coupled to a 40-bit dedicated adder. Each MAC unit performs its work in a single cycle; thus, the C55 can execute two MACs per cycle in full pipelined operation. This kind of capability is critical for efficiently performing signal processing applications. The C55 also has a compare, select, and store unit (CSSU) for the add/compare section of the Viterbi decoder.

The TI 320C6x

In stark contrast to the C55 DSP family is the high-end Texas Instruments VelociTI 320C6x family of processors. The C6x processors are closer to traditional very long instruction word (VLIW) processors because they seek to exploit the high levels of instruction-level parallelism (ILP) in many signal processing algorithms. Texas Instruments is not alone in selecting VLIW for exploiting ILP in the embedded space. Other VLIW DSP vendors include Ceva, StarCore, Philips/TriMedia, and STMicroelectronics. Why do these vendors favor VLIW over superscalar? For the embedded space, code compatibility is less of a problem, and so new applications can be either hand tuned or recompiled for the newest generation of processor. The other reason superscalar excels on the desktop is because the compiler cannot predict memory latencies at compile time. In embedded, however, memory latencies are often much more predictable. In fact, hard real-time constraints force memory latencies to be statically predictable. Of course, a superscalar would also perform well in this environment with these constraints, but the extra hardware to dynamically schedule instructions is both wasteful in terms of precious chip area and in terms of power consumption. Thus VLIW is a natural choice for high-performance embedded.

The C6x family employs different pipeline depths depending on the family member. For the C64x, for example, the pipeline has 11 stages. The first four stages of the pipeline perform instruction fetch, followed by two stages for instruction decode, and finally four stages for instruction execution. The overall architecture of the C64x is shown below in Figure E.5.

The C6x family's execution stage is divided into two parts, the left or "1" side and the right or "2" side. The L1 and L2 units perform logical and arithmetic operations. D units in contrast perform a subset of logical and arithmetic operations but also perform memory accesses (loads and stores). The two M units perform multiplication and related operations (e.g., shifts). Finally the S units perform comparisons, branches, and some SIMD operations (see the next subsection for a detailed explanation of SIMD operations). Each side has its own 32-entry, 32-bit register file (the A file for the 1 side, the B file for the 2 side). A side may access the other side's registers, but with a 1- cycle penalty. Thus, an instruction executing on side 1 may access B5, for example, but it will take 1- cycle extra to execute because of this.

VLIWs are traditionally very bad when it comes to code size, which runs contrary to the needs of embedded systems. However, the C6x family's approach "compresses" instructions, allowing the VLIW code to achieve the same density as equivalent RISC (reduced instruction set computer) code. To do so, instruction fetch is carried out on an "instruction packet," shown in Figure E.6. Each instruction has a *p* bit that specifies whether this instruction is a member of the current

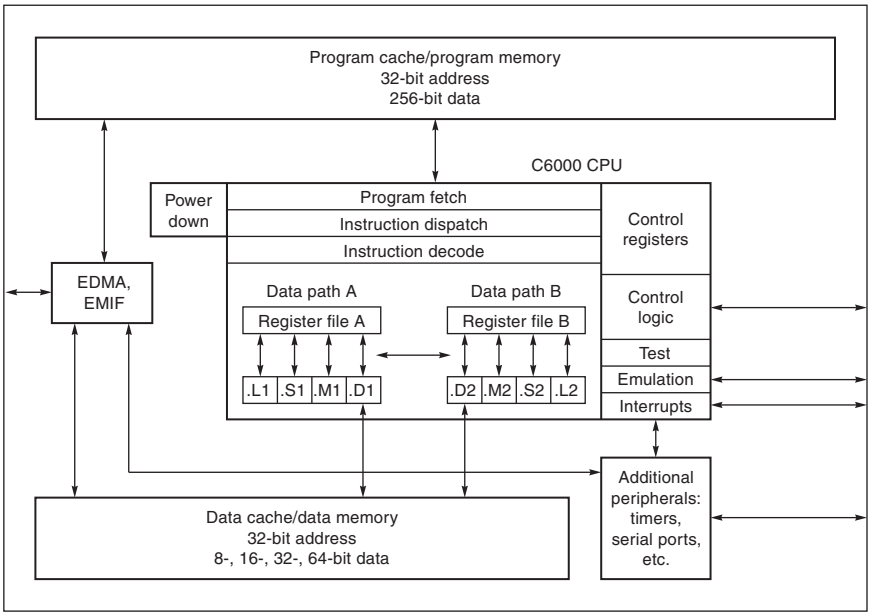


Figure E.5 Architecture of the TMS320C64x family of DSPs. The C6x is an eight-issue traditional VLIW processor. (Courtesy Texas Instruments.)

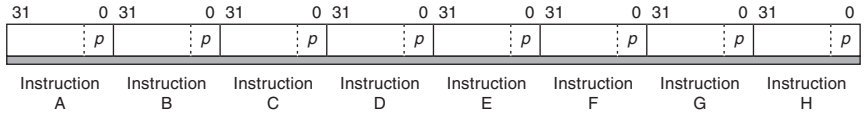


Figure E.6 Instruction packet of the TMS320C6x family of DSPs. The *p* bits determine whether an instruction begins a new VLIW word or not. If the *p* bit of instruction *i* is 1, then instruction *i* + 1 is to be executed in parallel with (in the same cycle as) instruction *i*. If the *p* bit of instruction *i* is 0, then instruction *i* + 1 is executed in the cycle after instruction *i*. (Courtesy Texas Instruments.)

VLIW word or the next VLIW word (see the figure for a detailed explanation). Thus, there are now no NOPs that are needed for VLIW encoding.

Software pipelining is an important technique for achieving high performance in a VLIW. But software pipelining relies on each iteration of the loop having an identical schedule to all other iterations. Because conditional branch instructions disrupt this pattern, the C6x family provides a means to conditionally execute instructions using *predication.* In predication, the instruction performs its work. But when it is done executing, an additional register, for example A1, is checked. If A1 is zero, the instruction does not write its results. If A1 is nonzero, the instruction proceeds normally. This allows simple if-then and if-then-else structures to be collapsed into straight-line code for software pipelining.

Media Extensions

There is a middle ground between DSPs and microcontrollers: *media extensions.* These extensions add DSP-like capabilities to microcontroller architectures at relatively low cost. Because media processing is judged by human perception, the data for multimedia operations are often much narrower than the 64-bit data word of modern desktop and server processors. For example, floating-point operations for graphics are normally in single precision, not double precision, and often at a precision less than is required by IEEE 754. Rather than waste the 64-bit arithmetic-logical units (ALUs) when operating on 32-bit, 16-bit, or even 8-bit integers, multimedia instructions can operate on several narrower data items at the same time. Thus, a *partitioned add* operation on 16-bit data with a 64-bit ALU would perform four 16-bit adds in a single clock cycle. The extra hardware cost is simply to prevent carries between the four 16-bit partitions of the ALU. For example, such instructions might be used for graphical operations on pixels. These operations are commonly called *single-instruction multiple-data* (SIMD) or *vector* instructions.

Most graphics multimedia applications use 32-bit floating-point operations. Some computers double peak performance of single-precision, floating-point operations; they allow a single instruction to launch two 32-bit operations on operands found side by side in a double-precision register. The two partitions must be insulated to prevent operations on one half from affecting the other. Such floating-point operations are called *paired single operations.* For example, such an operation

might be used for graphical transformations of vertices. This doubling in performance is typically accomplished by doubling the number of floating-point units, making it more expensive than just suppressing carries in integer adders.

Figure E.7 summarizes the SIMD multimedia instructions found in several recent computers.

DSPs also provide operations found in the first three rows of Figure E.7, but they change the semantics a bit. First, because they are often used in real-time applications, there is not an option of causing an exception on arithmetic overflow (otherwise it could miss an event); thus, the result will be used no matter what the inputs. To support such an unyielding environment, DSP architectures use *saturating arithmetic*: If the result is too large to be represented, it is set to the largest representable number, depending on the sign of the result. In contrast, two's complement arithmetic can add a small positive number to a large positive.

Instruction category	Alpha MAX	HP PA-RISC MAX2	Intel Pentium MMX	PowerPC AltiVec	SPARC VIS
Add/subtract		4H	8B, 4H, 2W	16B, 8H, 4W	4H, 2W
Saturating add/subtract		4H	8B, 4H	16B, 8H, 4W	
Multiply			4H	16B, 8H	
Compare	8B (>=)		8B, 4H, 2W (=, >)	16B, 8H, 4W (=, >, > =, <, <=)	4H, 2W (=, not =, >, <=)
Shift right/left		4H	4H, 2W	16B, 8H, 4W	
Shift right arithmetic		4H		16B, 8H, 4W	
Multiply and add				8H	
Shift and add (saturating)		4H			
AND/OR/XOR	8B, 4H, 2W	8B, 4H, 2W	8B, 4H, 2W	16B, 8H, 4W	8B, 4H, 2W
Absolute difference	8B			16B, 8H, 4W	8B
Maximum/minimum	8B, 4W			16B, 8H, 4W	
Pack ($2n$ bits → n bits)	2W → 2B, 4H → 4B	2*4H → 8B	4H → 4B, 2W → 2H	4W → 4B, 8H → 8B	2W → 2H, 2W → 2B, 4H → 4B
Unpack/merge	2B → 2W, 4B → 4H		2B → 2W, 4B → 4H	4B → 4W, 8B → 8H	4B → 4H, 2*4B → 8B
Permute/shuffle		4H		16B, 8H, 4W	

Figure E.7 Summary of multimedia support for desktop processors. Note the diversity of support, with little in common across the five architectures. All are fixed-width operations, performing multiple narrow operations on either a 64-bit or 128-bit ALU. B stands for byte (8 bits), H for half word (16 bits), and W for word (32 bits). Thus, 8B means an operation on 8 bytes in a single instruction. Note that AltiVec assumes a 128-bit ALU, and the rest assume 64 bits. Pack and unpack use the notation 2*2W to mean 2 operands each with 2 words. This table is a simplification of the full multimedia architectures, leaving out many details. For example, HP MAX2 includes an instruction to calculate averages, and SPARC VIS includes instructions to set registers to constants. Also, this table does not include the memory alignment operation of AltiVec, MAX, and VIS.

E.3 Embedded Benchmarks

It used to be the case just a couple of years ago that in the embedded market, many manufacturers quoted Dhrystone performance, a benchmark that was criticized and given up by desktop systems more than 20 years ago! As mentioned earlier, the enormous variety in embedded applications, as well as differences in performance requirements (hard real time, soft real time, and overall cost-performance), make the use of a single set of benchmarks unrealistic. In practice, many designers of embedded systems devise benchmarks that reflect their application, either as kernels or as stand-alone versions of the entire application.

For those embedded applications that can be characterized well by kernel performance, the best standardized set of benchmarks appears to be a new benchmark set: the EDN Embedded Microprocessor Benchmark Consortium (or EEMBC, pronounced "embassy"). The EEMBC benchmarks fall into six classes (called "subcommittees" in the parlance of EEMBC): automotive/industrial, consumer, telecommunications, digital entertainment, networking (currently in its second version), and office automation (also the second version of this subcommittee). Figure E.8 shows the six different application classes, which include 50 benchmarks.

Although many embedded applications are sensitive to the performance of small kernels, remember that often the overall performance of the entire application (which may be thousands of lines) is also critical. Thus, for many embedded systems, the EMBCC benchmarks can only be used to partially assess performance.

Benchmark type ("subcommittee")	Number of kernels	Example benchmarks
Automotive/industrial	16	6 microbenchmarks (arithmetic operations, pointer chasing, memory performance, matrix arithmetic, table lookup, bit manipulation), 5 automobile control benchmarks, and 5 filter or FFT benchmarks
Consumer	5	5 multimedia benchmarks (JPEG compress/decompress, filtering, and RGB conversions)
Telecommunications	5	Filtering and DSP benchmarks (autocorrelation, FFT, decoder, encoder)
Digital entertainment	12	MP3 decode, MPEG-2 and MPEG-4 encode and decode (each of which is applied to five different datasets), MPEG Encode Floating Point, 4 benchmark tests for common cryptographic standards and algorithms (AES, DES, RSA, and Huffman decoding for data decompression), and enhanced JPEG and color-space conversion tests
Networking version 2	6	IP Packet Check (borrowed from the RFC1812 standard), IP Reassembly, IP Network Address Translator (NAT), Route Lookup, OSPF, Quality of Service (QOS), and TCP
Office automation version 2	6	Ghostscript, text parsing, image rotation, dithering, Bézier

Figure E.8 The EEMBC benchmark suite, consisting of 50 kernels in six different classes. See *www.eembc.org* for more information on the benchmarks and for scores.

Power Consumption and Efficiency as the Metric

Cost and power are often at least as important as performance in the embedded market. In addition to the cost of the processor module (which includes any required interface chips), memory is often the next most costly part of an embedded system. Unlike a desktop or server system, most embedded systems do not have secondary storage; instead, the entire application must reside in either FLASH or DRAM. Because many embedded systems, such as PDAs and cell phones, are constrained by both cost and physical size, the amount of memory needed for the application is critical. Likewise, power is often a determining factor in choosing a processor, especially for battery-powered systems.

EEMBC EnergyBench provides data on the amount of energy a processor consumes while running EEMBC's performance benchmarks. An EEMBC-certified Energymark score is an optional metric that a device manufacturer may choose to supply in conjunction with certified scores for device performance as a way of indicating a processor's efficient use of power and energy. EEMBC has standardized on the use of National Instruments' LabVIEW graphical development environment and data acquisition hardware to implement EnergyBench.

Figure E.9 shows the relative performance per watt of typical operating power. Compare this figure to Figure E.10, which plots raw performance, and notice how different the results are. The NEC VR 4122 has a clear advantage in performance per watt, but is the second-lowest performing processor! From the viewpoint of power consumption, the NEC VR 4122, which was designed for battery-based systems, is the big winner. The IBM PowerPC displays efficient use of power to achieve its high performance, although at 6 W typical, it is probably not suitable for most battery-based devices.

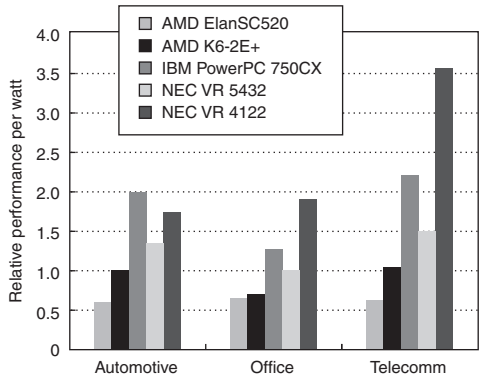


Figure E.9 Relative performance per watt for the five embedded processors. The power is measured as typical operating power for the processor and does not include any interface chips.

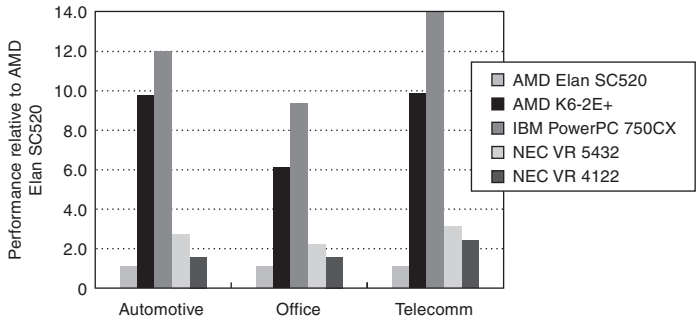


Figure E.10 Raw performance for the five embedded processors. The performance is presented as relative to the performance of the AMD ElanSC520.

E.4 Embedded Multiprocessors

Multiprocessors are now common in server environments, and several desktop multiprocessors are available from vendors, such as Sun, Compaq, and Apple. In the embedded space, a number of special-purpose designs have used customized multiprocessors, including the Sony PlayStation 2 (see Section E.5).

Many special-purpose embedded designs consist of a general-purpose programmable processor or DSP with special-purpose, finite-state machines that are used for stream-oriented I/O. In applications ranging from computer graphics and media processing to telecommunications, this style of special-purpose multiprocessor is becoming common. Although the interprocessor interactions in such designs are highly regimented and relatively simple—consisting primarily of a simple communication channel—because much of the design is committed to silicon, ensuring that the communication protocols among the input/output processors and the general-purpose processor are correct is a major challenge in such designs.

More recently, we have seen the first appearance, in the embedded space, of embedded multiprocessors built from several general-purpose processors. These multiprocessors have been focused primarily on the high-end telecommunications and networking market, where scalability is critical. An example of such a design is the MXP processor designed by empowerTel Networks for use in voice-over-IP systems. The MXP processor consists of four main components:

- An interface to serial voice streams, including support for handling jitter
- Support for fast packet routing and channel lookup
- A complete Ethernet interface, including the MAC layer
- Four MIPS32 R4000-class processors, each with its own cache (a total of 48 KB or 12 KB per processor)

The MIPS processors are used to run the code responsible for maintaining the voice-over-IP channels, including the assurance of quality of service, echo cancellation, simple compression, and packet encoding. Since the goal is to run as many independent voice streams as possible, a multiprocessor is an ideal solution.

Because of the small size of the MIPS cores, the entire chip takes only 13.5 M transistors. Future generations of the chip are expected to handle more voice channels, as well as do more sophisticated echo cancellation, voice activity detection, and more sophisticated compression.

Multiprocessing is becoming widespread in the embedded computing arena for two primary reasons. First, the issues of binary software compatibility, which plague desktop and server systems, are less relevant in the embedded space. Often software in an embedded application is written from scratch for an application or significantly modified (note that this is also the reason VLIW is favored over superscalar in embedded instruction-level parallelism). Second, the applications often have natural parallelism, especially at the high end of the embedded space. Examples of this natural parallelism abound in applications such as a settop box, a network switch, a cell phone (see Section E.7) or a game system (see Section E.5). The lower barriers to use of thread-level parallelism together with the greater sensitivity to die cost (and hence efficient use of silicon) are leading to widespread adoption of multiprocessing in the embedded space, as the application needs grow to demand more performance.

E.5 Case Study: The Emotion Engine of the Sony PlayStation 2

Desktop computers and servers rely on the memory hierarchy to reduce average access time to relatively static data, but there are embedded applications where data are often a continuous stream. In such applications there is still spatial locality, but temporal locality is much more limited.

To give another look at memory performance beyond the desktop, this section examines the microprocessor at the heart of the Sony PlayStation 2. As we will see, the steady stream of graphics and audio demanded by electronic games leads to a different approach to memory design. The style is high bandwidth via many dedicated independent memories.

Figure E.11 shows a block diagram of the Sony PlayStation 2 (PS2). Not surprisingly for a game machine, there are interfaces for video, sound, and a DVD player. Surprisingly, there are two standard computer I/O buses, USB and IEEE 1394, a PCMCIA slot as found in portable PCs, and a modem. These additions show that Sony had greater plans for the PS2 beyond traditional games. Although it appears that the I/O processor (IOP) simply handles the I/O devices and the game console, it includes a 34 MHz MIPS processor that also acts as the emulation computer to run games for earlier Sony PlayStations. It also connects to a standard PC audio card to provide the sound for the games.

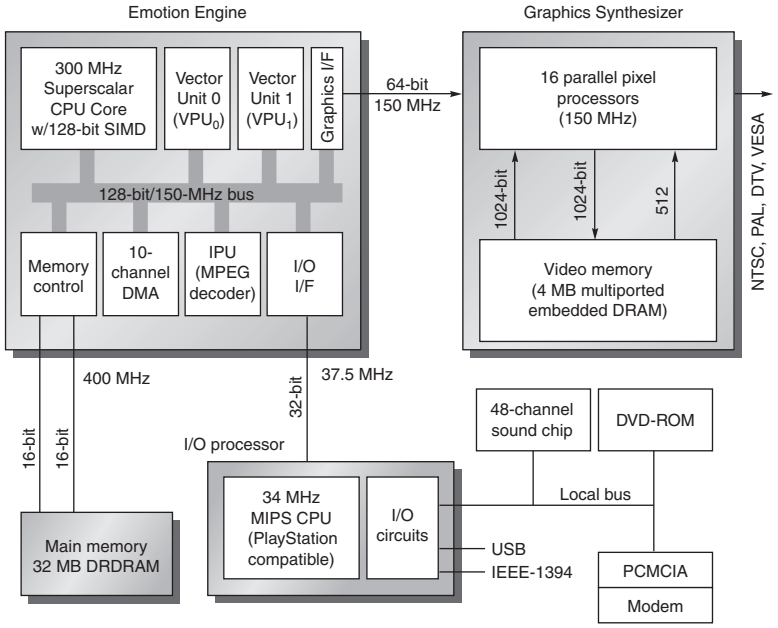


Figure E.11 Block diagram of the Sony PlayStation 2. The 10 DMA channels orchestrate the transfers between all the small memories on the chip, which when completed all head toward the Graphics Interface so as to be rendered by the Graphics Synthesizer. The Graphics Synthesizer uses DRAM on chip to provide an entire frame buffer plus graphics processors to perform the rendering desired based on the display commands given from the Emotion Engine. The embedded DRAM allows 1024-bit transfers between the pixel processors and the display buffer. The Superscalar CPU is a 64-bit MIPS III with two-instruction issue, and comes with a two-way, set associative, 16 KB instruction cache; a two-way, set associative, 8 KB data cache; and 16 KB of scratchpad memory. It has been extended with 128-bit SIMD instructions for multimedia applications (see Section E.2). Vector Unit 0 is primarily a DSP-like coprocessor for the CPU (see Section E.2), which can operate on 128-bit registers in SIMD manner between 8 bits and 32 bits per word. It has 4 KB of instruction memory and 4 KB of data memory. Vector Unit 1 has similar functions to VPU0, but it normally operates independently of the CPU and contains 16 KB of instruction memory and 16 KB of data memory. All three units can communicate over the 128-bit system bus, but there is also a 128-bit dedicated path between the CPU and VPU0 and a 128-bit dedicated path between VPU1 and the Graphics Interface. Although VPU0 and VPU1 have identical microarchitectures, the differences in memory size and units to which they have direct connections affect the roles that they take in a game. At 0.25-micron line widths, the Emotion Engine chip uses 13.5M transistors and is 225 mm^2, and the Graphics Synthesizer is 279 mm^2. To put this in perspective, the Alpha 21264 microprocessor in 0.25-micron technology is about 160 mm^2 and uses 15M transistors. (This figure is based on Figure 1 in "Sony's Emotionally Charged Chip," *Microprocessor Report* 13:5.)

Thus, one challenge for the memory system of this embedded application is to act as source or destination for the extensive number of I/O devices. The PS2 designers met this challenge with two PC800 (400 MHz) DRDRAM chips using two channels, offering 32 MB of storage and a peak memory bandwidth of 3.2 GB/sec.

What's left in the figure are basically two big chips: the Graphics Synthesizer and the Emotion Engine.

The Graphics Synthesizer takes rendering commands from the Emotion Engine in what are commonly called *display lists.* These are lists of 32-bit commands that tell the renderer what shape to use and where to place them, plus what colors and textures to fill them.

This chip also has the highest bandwidth portion of the memory system. By using embedded DRAM on the Graphics Synthesizer, the chip contains the full video buffer *and* has a 2048-bit-wide interface so that pixel filling is not a bottleneck. This embedded DRAM greatly reduces the bandwidth demands on the DRDRAM. It illustrates a common technique found in embedded applications: separate memories dedicated to individual functions to inexpensively achieve greater memory bandwidth for the entire system.

The remaining large chip is the Emotion Engine, and its job is to accept inputs from the IOP and create the display lists of a video game to enable 3D video transformations in real time. A major insight shaped the design of the Emotion Engine: Generally, in a racing car game there are foreground objects that are constantly changing and background objects that change less in reaction to the events, although the background can be most of the screen. This observation led to a split of responsibilities.

The CPU works with VPU0 as a tightly coupled coprocessor, in that every VPU0 instruction is a standard MIPS coprocessor instruction, and the addresses are generated by the MIPS CPU. VPU0 is called a vector processor, but it is similar to 128-bit SIMD extensions for multimedia found in several desktop processors (see Section E.2).

VPU1, in contrast, fetches its own instructions and data and acts in parallel with CPU/VPU0, acting more like a traditional vector unit. With this split, the more flexible CPU/VPU0 handles the foreground action and the VPU1 handles the background. Both deposit their resulting display lists into the Graphics Interface to send the lists to the Graphics Synthesizer.

Thus, the programmers of the Emotion Engine have three processor sets to choose from to implement their programs: the traditional 64-bit MIPS architecture including a floating-point unit, the MIPS architecture extended with multimedia instructions (VPU0), and an independent vector processor (VPU1). To accelerate MPEG decoding, there is another coprocessor (Image Processing Unit) that can act independent of the other two.

With this split of function, the question then is how to connect the units together, how to make the data flow between units, and how to provide the memory bandwidth needed by all these units. As mentioned earlier, the Emotion Engine designers chose many dedicated memories. The CPU has a 16 KB scratch pad memory (SPRAM) in addition to a 16 KB instruction cache and an 8 KB data cache. VPU0 has a 4 KB instruction memory and a 4 KB data memory, and VPU1 has a 16 KB instruction memory and a 16 KB data memory. Note that these are four *memories*, not caches of a larger memory elsewhere. In each memory the latency is just 1 clock cycle. VPU1 has more memory than VPU0 because it creates the bulk of the display lists and because it largely acts independently.

The programmer organizes all memories as two double buffers, one pair for the incoming DMA data and one pair for the outgoing DMA data. The programmer then uses the various processors to transform the data from the input buffer to the output buffer. To keep the data flowing among the units, the programmer next sets up the 10 DMA channels, taking care to meet the real-time deadline for realistic animation of 15 frames per second.

Figure E.12 shows that this organization supports two main operating modes: serial, where CPU/VPU0 acts as a preprocessor on what to give VPU1 for it to create for the Graphics Interface using the scratchpad memory as the buffer, and parallel, where both the CPU/VPU0 and VPU1 create display lists. The display lists and the Graphics Synthesizer have multiple context identifiers to distinguish the parallel display lists to produce a coherent final image.

All units in the Emotion Engine are linked by a common 150 MHz, 128-bit-wide bus. To offer greater bandwidth, there are also two dedicated buses: a 128-bit path between the CPU and VPU0 and a 128-bit path between VPU1 and the Graphics Interface. The programmer also chooses which bus to use when setting up the DMA channels.

Looking at the big picture, if a server-oriented designer had been given the problem, we might see a single common bus with many local caches and cache-coherent mechanisms to keep data consistent. In contrast, the PlayStation 2 followed the tradition of embedded designers and has at least nine distinct memory modules. To keep the data flowing in real time from memory to the display, the PS2 uses dedicated memories, dedicated buses, and DMA channels. Coherency is the responsibility of the programmer, and, given the continuous flow from main memory to the graphics interface and the real-time requirements, programmer-controlled coherency works well for this application.

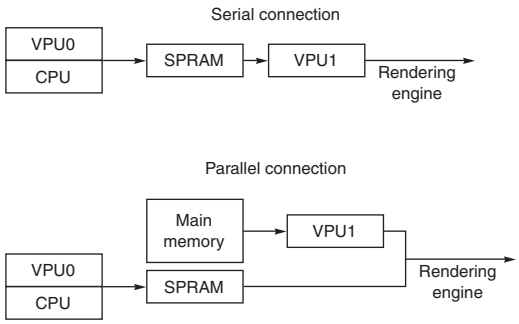


Figure E.12 Two modes of using Emotion Engine organization. The first mode divides the work between the two units and then allows the Graphics Interface to properly merge the display lists. The second mode uses CPU/VPU0 as a filter of what to send to VPU1, which then does all the display lists. It is up to the programmer to choose between serial and parallel data flow. SPRAM is the scratchpad memory.

E.6 Case Study: Sanyo VPC-SX500 Digital Camera

Another very familiar embedded system is a digital camera. Here we consider the Sanyo VPC-SX500. When powered on, the microprocessor of the camera first runs diagnostics on all components and writes any error messages to the liquid crystal display (LCD) on the back of the camera. This camera uses a 1.8-inch low-temperature polysilicon thin-film transistor (TFT) color LCD. When a photographer takes a picture, he first holds the shutter halfway so that the microprocessor can take a light reading. The microprocessor then keeps the shutter open to get the necessary light, which is captured by a charge-coupled device (CCD) as red, green, and blue pixels. The CCD is a 1/2-inch, 1360 × 1024-pixel, progressive-scan chip. The pixels are scanned out row by row; passed through routines for white balance, color, and aliasing correction; and then stored in a 4 MB frame buffer. The next step is to compress the image into a standard format, such as JPEG, and store it in the removable Flash memory. The photographer picks the compression, in this camera called either *fine* or *normal*, with a compression ratio of 10 to 20 times. A 512 MB Flash memory can store at least 1200 fine-quality compressed images or approximately 2000 normal-quality compressed images. The microprocessor then updates the LCD display to show that there is room for one less picture.

Although the previous paragraph covers the basics of a digital camera, there are many more features that are included: showing the recorded images on the color LCD display, sleep mode to save battery life, monitoring battery energy, buffering to allow recording a rapid sequence of uncompressed images, and, in this camera, video recording using MPEG format and audio recording using WAV format.

The electronic brain of this camera is an embedded computer with several special functions embedded on the chip [Okada et al. 1999]. Figure E.13 shows the block diagram of a chip similar to the one in the camera. As mentioned in Section E.1, such chips have been called *systems on a chip* (SOCs) because they essentially integrate into a single chip all the parts that were found on a small printed circuit board of the past. A SOC generally reduces size and lowers power compared to less integrated solutions. Sanyo claims their SOC enables the camera to operate on half the number of batteries and to offer a smaller form factor than competitors' cameras. For higher performance, it has two buses. The 16-bit bus is for the many slower I/O devices: SmartMedia interface, program and data memory, and DMA. The 32-bit bus is for the SDRAM, the signal processor (which is connected to the CCD), the Motion JPEG encoder, and the NTSC/PAL encoder (which is connected to the LCD). Unlike desktop microprocessors, note the large variety of I/O buses that this chip must integrate. The 32-bit RISC MPU is a proprietary design and runs at 28.8 MHz, the same clock rate as the buses. This 700 mW chip contains 1.8M transistors in a 10.5 × 10.5 mm die implemented using a 0.35-micron process.

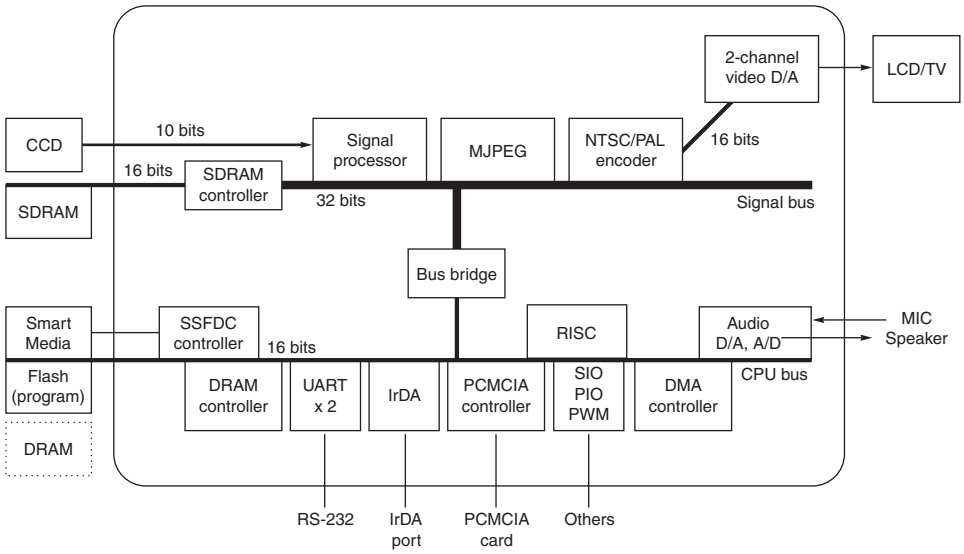


Figure E.13 The system on a chip (SOC) found in Sanyo digital cameras. This block diagram, found in Okada et al. [1999], is for the predecessor of the SOC in the camera described in the text. The successor SOC, called *Super Advanced IC*, uses three buses instead of two, operates at 60 MHz, consumes 800 mW, and fits 3.1M transistors in a 10.2 × 10.2 mm die using a 0.35-micron process. Note that this embedded system has twice as many transistors as the state-of-the-art, high-performance microprocessor in 1990! The SOC in the figure is limited to processing 1024 × 768 pixels, but its successor supports 1360 × 1024 pixels.

E.7 Case Study: Inside a Cell Phone

Although gaming consoles and digital cameras are familiar embedded systems, today the most familiar embedded system is the cell phone. In 1999, there were 76 million cellular subscribers in the United States, a 25% growth rate from the year before. That growth rate is almost 35% per year worldwide, as developing countries find it much cheaper to install cellular towers than copper-wire-based infrastructure. Thus, in many countries, the number of cell phones in use exceeds the number of wired phones in use.

Not surprisingly, the cellular handset market is growing at 35% per year, with about 280 million cellular phone handsets sold worldwide in 1999. To put that in perspective, in the same year sales of personal computers were 120 million. These numbers mean that tremendous engineering resources are available to improve cell phones, and cell phones are probably leaders in engineering innovation per cubic inch [Grice and Kanellos 2000].

Before unveiling the anatomy of a cell phone, let's try a short introduction to wireless technology.

Background on Wireless Networks

Networks can be created out of thin air as well as out of copper and glass, creating *wireless networks.* Much of this section is based on a report from the National Research Council [1997].

A radio wave is an electromagnetic wave propagated by an antenna. Radio waves are modulated, which means that the sound signal is superimposed on the stronger radio wave that carries the sound signal, and hence is called the *carrier signal.* Radio waves have a particular wavelength or frequency: They are measured either as the length of the complete wave or as the number of waves per second. Long waves have low frequencies, and short waves have high frequencies. FM radio stations transmit on the band of 88 MHz to 108 MHz using frequency modulations (FM) to record the sound signal.

By tuning in to different frequencies, a radio receiver can pick up a specific signal. In addition to AM and FM radio, other frequencies are reserved for citizens band radio, television, pagers, air traffic control radar, Global Positioning System, and so on. In the United States, the Federal Communications Commission decides who gets to use which frequencies and for what purpose.

The *bit error rate* (BER) of a wireless link is determined by the received signal power, noise due to interference caused by the receiver hardware, interference from other sources, and characteristics of the channel. Noise is typically proportional to the radio frequency bandwidth, and a key measure is the *signal-to-noise ratio* (SNR) required to achieve a given BER. Figure E.14 lists more challenges for wireless communication.

Typically, wireless communication is selected because the communicating devices are mobile or because wiring is inconvenient, which means the wireless network must rearrange itself dynamically. Such rearrangement makes routing

Challenge	Description	Impact
Path loss	Received power divided by transmitted power; the radio must overcome signal-to-noise ratio (SNR) of noise from interference. Path loss is exponential in distance and depends on interference if it is above 100 meters.	1 W transmit power, 1 GHz transmit frequency, 1 Mbit/sec data rate at 10^{-7} BER, distance between radios can be 728 meters in free space vs. 4 meters in a dense jungle.
Shadow fading	Received signal blocked by objects, buildings outdoors, or walls indoors; increase power to improve received SNR. It depends on the number of objects and their dielectric properties.	If transmitter is moving, need to change transmit power to ensure received SNR in region.
Multipath fading	Interference between multiple versions of signal that arrive at different times, determined by time between fastest signal and slowest signal relative to signal bandwidth.	900 MHz transmit frequency signal power changes every 30 cm.
Interference	Frequency reuse, adjacent channel, narrow band interference.	Requires filters, spread spectrum.

Figure E.14 Challenges for wireless communication.

more challenging. A second challenge is that wireless signals are not protected and hence are subject to mutual interference, especially as devices move. Power is another challenge for wireless communication, both because the devices tend to be battery powered and because antennas radiate power to communicate and little of it reaches the receiver. As a result, raw bit error rates are typically a thousand to a million times higher than copper wire.

There are two primary architectures for wireless networks: *base station* architectures and *peer-to-peer* architectures. Base stations are connected by landlines for longer-distance communication, and the mobile units communicate only with a single local base station. Peer-to-peer architectures allow mobile units to communicate with each other, and messages hop from one unit to the next until delivered to the desired unit. Although peer-to-peer is more reconfigurable, base stations tend to be more reliable since there is only one hop between the device and the station. *Cellular telephony*, the most popular example of wireless networks, relies on radio with base stations.

Cellular systems exploit exponential path loss to reuse the same frequency at spatially separated locations, thereby greatly increasing the number of customers served. Cellular systems will divide a city into nonoverlapping hexagonal cells that use different frequencies if nearby, reusing a frequency only when cells are far enough apart so that mutual interference is acceptable.

At the intersection of three hexagonal cells is a base station with transmitters and antennas that is connected to a switching office that coordinates handoffs when a mobile device leaves one cell and goes into another, as well as accepts and places calls over landlines. Depending on topography, population, and so on, the radius of a typical cell is 2 to 10 miles.

The Cell Phone

Figure E.15 shows the components of a radio, which is the heart of a cell phone. Radio signals are first received by the antenna, amplified, passed through a mixer, then filtered, demodulated, and finally decoded. The antenna acts as the interface between the medium through which radio waves travel and the electronics of the transmitter or receiver. Antennas can be designed to work best in particular directions, giving both transmission and reception directional properties. Modulation encodes information in the amplitude, phase, or frequency of the signal to increase its robustness under impaired conditions. Radio transmitters go through the same steps, just in the opposite order.

Originally, all components were analog, but over time most were replaced by digital components, requiring the radio signal to be converted from analog to digital. The desire for flexibility in the number of radio bands led to software routines replacing some of these functions in programmable chips, such as digital signal processors. Because such processors are typically found in mobile devices, emphasis is placed on performance per joule to extend battery life, performance per square millimeter of silicon to reduce size and cost, and bytes per task to reduce memory size.

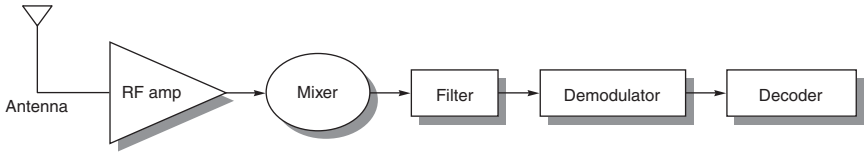


Figure E.15 A radio receiver consists of an antenna, radio frequency amplifier, mixer, filters, demodulator, and decoder. A mixer accepts two signal inputs and forms an output signal at the sum and difference frequencies. Filters select a narrower band of frequencies to pass on to the next stage. Modulation encodes information to make it more robust. Decoding turns signals into information. Depending on the application, all electrical components can be either analog or digital. For example, a car radio is all analog components, but a PC modem is all digital except for the amplifier. Today analog silicon chips are used for the RF amplifier and first mixer in cellular phones.

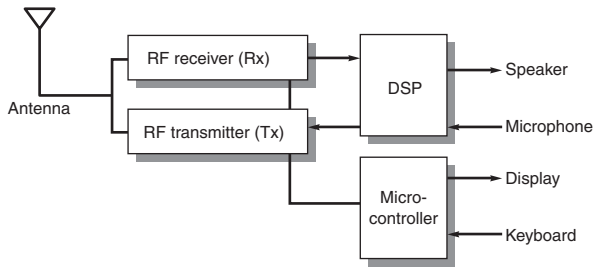


Figure E.16 Block diagram of a cell phone. The DSP performs the signal processing steps of Figure E.15, and the microcontroller controls the user interface, battery management, and call setup. (Based on Figure 1.3 of Groe and Larson [2000].)

Figure E.16 shows the generic block diagram of the electronics of a cell phone handset, with the DSP performing the signal processing and the microcontroller handling the rest of the tasks. Cell phone handsets are basically mobile computers acting as a radio. They include standard I/O devices—keyboard and LCD display—plus a microphone, speaker, and antenna for wireless networking. Battery efficiency affects sales, both for standby power when waiting for a call and for minutes of speaking.

When a cell phone is turned on, the first task is to find a cell. It scans the full bandwidth to find the strongest signal, which it keeps doing every seven seconds or if the signal strength drops, since it is designed to work from moving vehicles. It then picks an unused radio channel. The local switching office registers the cell phone and records its phone number and electronic serial number, and assigns it a voice channel for the phone conversation. To be sure the cell phone got the right channel, the base station sends a special tone on it, which the cell phone sends back to acknowledge it. The cell phone times out after 5 seconds if it doesn't hear the supervisory tone, and it starts the process all over again. The original base station makes a handoff request to the incoming base station as the signal strength drops offs.

To achieve a two-way conversation over radio, frequency bands are set aside for each direction, forming a frequency pair or *channel.* The original cellular base stations transmitted at 869.04 to 893.97 MHz (called the *forward path*), and cell phones transmitted at 824.04 to 848.97 MHz (called t*he reverse path*), with the frequency gap to keep them from interfering with each other. Cells might have had between 4 and 80 channels. Channels were divided into setup channels for call setup and voice channels to handle the data or voice traffic.

The communication is done digitally, just like a modem, at 9600 bits/sec. Since wireless is a lossy medium, especially from a moving vehicle, the handset sends each message five times. To preserve battery life, the original cell phones typically transmit at two signal strengths—0.6 W and 3.0 W—depending on the distance to the cell. This relatively low power not only allows smaller batteries and thus smaller cell phones, but it also aids frequency reuse, which is the key to cellular telephony.

Figure E.17 shows a circuit board from a Nokia digital phone, with the components identified. Note that the board contains two processors. A Z-80 microcontroller is responsible for controlling the functions of the board, I/O with the keyboard and display, and coordinating with the base station. The DSP handles all signal compression and decompression. In addition there are dedicated chips for analog-to-digital and digital-to-analog conversion, amplifiers, power management, and RF interfaces.

In 2001, a cell phone had about 10 integrated circuits, including parts made in exotic technologies like gallium arsinide and silicon germanium as well as standard CMOS. The economics and desire for flexibility have shrunk this to just a few chips. However, these SOCs still contain a separate microcontroller and DSP, with code implementing many of the functions just described.

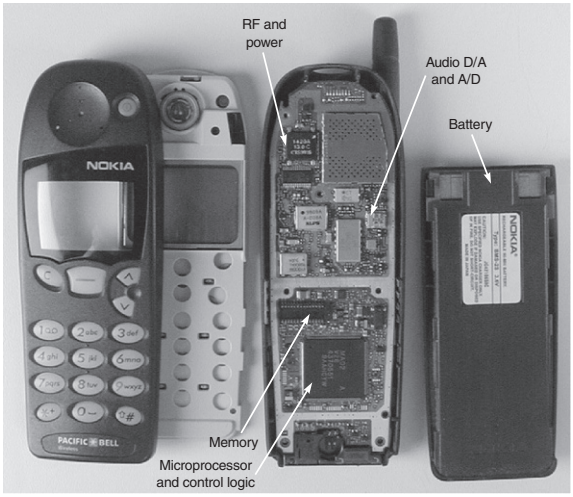


Figure E.17 Circuit board from a Nokia cell phone. (Courtesy HowStuffWorks, Inc.)

Cell Phone Standards and Evolution

Improved communication speeds for cell phones were developed with multiple standards. *Code division multiple access* (CDMA), as one popular example, uses a wider radio frequency band for a path than the original cell phones, called *advanced mobile phone service* (AMPS), a mostly analog system. The wider frequency makes it more difficult to block and is called *spread spectrum.* Other standards are *time division multiple access* (TDMA) and *global system for mobile communication* (GSM). These second-generation standards—CDMA, GSM, and TDMA—are mostly digital.

The big difference for CDMA is that all callers share the same channel, which operates at a much higher rate, and it then distinguishes the different calls by encoding each one uniquely. Each CDMA phone call starts at 9600 bits/sec; it is then encoded and transmitted as equal-sized messages at 1.25 Mbits/sec. Rather than send each signal five times as in AMPS, each bit is stretched so that it takes 11 times the minimum frequency, thereby accommodating interference and yet successful transmission. The base station receives the messages, and it separates them into the separate 9600 bit/sec streams for each call.

To enhance privacy, CDMA uses pseudorandom sequences from a set of 64 predefined codes. To synchronize the handset and base station so as to pick a common pseudorandom seed, CDMA relies on a clock from the Global Positioning System, which continuously transmits an accurate time signal. By carefully selecting the codes, the shared traffic sounds like random noise to the listener. Hence, as more users share a channel there is more noise, and the signal-to-noise ratio gradually degrades. Thus, the capacity of the CDMA system is a matter of taste, depending upon the sensitivity of the listener to background noise.

In addition, CDMA uses speech compression and varies the rate of data transferred depending upon how much activity is going on in the call. Both these techniques preserve bandwidth, which allows for more calls per cell. CDMA must regulate power carefully so that signals near the cell tower do not overwhelm those from far away, with the goal of all signals reaching the tower at about the same level. The side benefit is that CDMA handsets emit less power, which both helps battery life and increases capacity when users are close to the tower.

Thus, compared to AMPS, CDMA improves the capacity of a system by up to an order of magnitude, has better call quality, has better battery life, and enhances users' privacy. After considerable commercial turmoil, there is a new third-generation standard called *International Mobile Telephony 2000* (IMT-2000), based primarily on two competing versions of CDMA and one TDMA. This standard may lead to cell phones that work anywhere in the world.

E.8 Concluding Remarks

Embedded systems are a very broad category of computing devices. This appendix has shown just some aspects of this. For example, the TI 320C55 DSP is a relatively "RISC-like" processor designed for embedded applications, with very

fine-tuned capabilities. On the other end of the spectrum, the TI 320C64x is a very high-performance, eight-issue VLIW processor for very demanding tasks. Some processors must operate on battery power alone; others have the luxury of being plugged into line current. Unifying all of these is a need to perform some level of signal processing for embedded applications. Media extensions attempt to merge DSPs with some more general-purpose processing abilities to make these processors usable for signal processing applications. We examined several case studies, including the Sony PlayStation 2, digital cameras, and cell phones. The PS2 performs detailed three-dimensional graphics, whereas a cell phone encodes and decodes signals according to elaborate communication standards. But both have system architectures that are very different from general-purpose desktop or server platforms. In general, architectural decisions that seem practical for general-purpose applications, such as multiple levels of caching or out-of-order superscalar execution, are much less desirable in embedded applications. This is due to chip area, cost, power, and real-time constraints. The programming model that these systems present places more demands on both the programmer and the compiler for extracting parallelism.

F.1	Introduction	F-2
F.2	Interconnecting Two Devices	F-6
F.3	Connecting More Than Two Devices	F-20
F.4	Network Topology	F-30
F.5	Network Routing, Arbitration, and Switching	F-44
F.6	Switch Microarchitecture	F-56
F.7	Practical Issues for Commercial Interconnection Networks	F-66
F.8	Examples of Interconnection Networks	F-73
F.9	Internetworking	F-85
F.10	Crosscutting Issues for Interconnection Networks	F-89
F.11	Fallacies and Pitfalls	F-92
F.12	Concluding Remarks	F-100
F.13	Historical Perspective and References	F-101
	References	F-109
	Exercises	F-111

F

Interconnection Networks

Revised by Timothy M. Pinkston, University of Southern California; José Duato, Universitat Politècnica de València, and Simula

"The Medium is the Message" because it is the medium that shapes and controls the search and form of human associations and actions.

Marshall McLuhan
Understanding Media (1964)

The marvels—of film, radio, and television—are marvels of one-way communication, which is not communication at all.

Milton Mayer
On the Remote Possibility of Communication (1967)

The interconnection network is the heart of parallel architecture.

Chuan-Lin Wu and Tse-Yun Feng
Interconnection Networks for Parallel and Distributed Processing (1984)

Indeed, as system complexity and integration continues to increase, many designers are finding it more efficient to route packets, not wires.

Bill Dally
Principles and Practices of Interconnection Networks (2004)

F.1 Introduction

Previous chapters and appendices cover the components of a single computer but give little consideration to the interconnection of those components and how multiple computer systems are interconnected. These aspects of computer architecture have gained significant importance in recent years. In this appendix we see how to connect individual devices together into a community of communicating devices, where the term *device* is generically used to signify anything from a component or set of components within a computer to a single computer to a system of computers. Figure F.1 shows the various elements comprising this community: end nodes consisting of devices and their associated hardware and software interfaces, links from end nodes to the interconnection network, and the interconnection network. Interconnection networks are also called *networks, communication subnets*, or *communication subsystems*. The interconnection of multiple networks is called *internetworking*. This relies on communication standards to convert information from one kind of network to another, such as with the Internet.

There are several reasons why computer architects should devote attention to interconnection networks. In addition to providing external connectivity, networks are commonly used to interconnect the components within a single computer at many levels, including the processor microarchitecture. Networks have long been used in mainframes, but today such designs can be found in personal computers as well, given the high demand on communication bandwidth needed to enable increased computing power and storage capacity. Switched networks are replacing buses as the normal means of communication between computers, between I/O devices, between boards, between chips, and even between modules inside chips. Computer architects must understand interconnect problems and solutions in order to more effectively design and evaluate computer systems.

Interconnection networks cover a wide range of application domains, very much like memory hierarchy covers a wide range of speeds and sizes. Networks implemented within processor chips and systems tend to share characteristics much in common with processors and memory, relying more on high-speed hardware solutions and less on a flexible software stack. Networks implemented across systems tend to share much in common with storage and I/O, relying more on the operating system and software protocols than high-speed hardware—though we are seeing a convergence these days. Across the domains, performance includes latency and effective bandwidth, and queuing theory is a valuable analytical tool in evaluating performance, along with simulation techniques.

This topic is vast—portions of Figure F.1 are the subject of entire books and college courses. The goal of this appendix is to provide for the computer architect an overview of network problems and solutions. This appendix gives introductory explanations of key concepts and ideas, presents architectural implications of interconnection network technology and techniques, and provides useful references to more detailed descriptions. It also gives a common framework for evaluating all types of interconnection networks, using a single set of terms to describe the basic

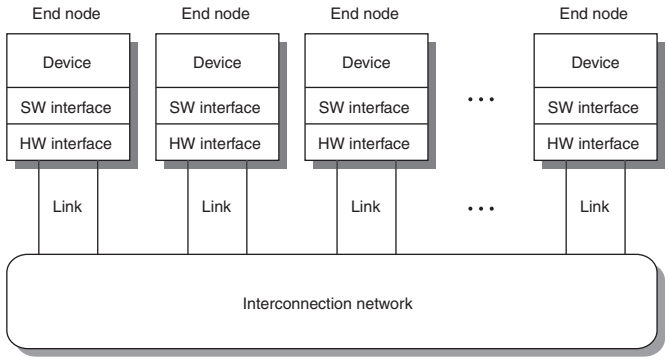


Figure F.1 A conceptual illustration of an interconnected community of devices.

alternatives. As we will see, many types of networks have common preferred alternatives, but for others the best solutions are quite different. These differences become very apparent when crossing between the networking domains.

Interconnection Network Domains

Interconnection networks are designed for use at different levels within and across computer systems to meet the operational demands of various application areas—high-performance computing, storage I/O, cluster/workgroup/enterprise systems, internetworking, and so on. Depending on the number of devices to be connected and their proximity, we can group interconnection networks into four major networking domains:

- *On-chip networks* (OCNs)—Also referred to as network-on-chip (NoC), this type of network is used for interconnecting microarchitecture functional units, register files, caches, compute tiles, and processor and IP cores within chips or multichip modules. Current and near future OCNs support the connection of a few tens to a few hundred of such devices with a maximum interconnection distance on the order of centimeters. Most OCNs used in high-performance chips are custom designed to mitigate chip-crossing wire delay problems caused by increased technology scaling and transistor integration, though some proprietary designs are gaining wider use (e.g., IBM's CoreConnect, ARM's AMBA, and Sonic's Smart Interconnect). Examples of current OCNs are those found in the Intel Teraflops processor chip [Hoskote07], connecting 80 simple cores; the Intel Single-Chip Cloud Computer (SCCC) [Howard10], connecting 48 IA-32 architecture cores; and Tilera's TILE-Gx line of processors [TILE-GX], connecting 100 processing cores in 4Q 2011 using TSMC's 40 nanometer process and 200 cores planned for 2013 (code named "Stratton") using

TSMC's 28 nanometer process. The networks peak at 256 GBps for both Intel prototypes and up to 200 Tbps for the TILE-Gx100 processor. More detailed information for OCNs is provided in Flich [2010].

- *System/storage area networks* (SANs)—This type of network is used for interprocessor and processor-memory interconnections within multiprocessor and multicomputer systems, and also for the connection of storage and I/O components within server and data center environments. Typically, several hundreds of such devices can be connected, although some supercomputer SANs support the interconnection of many thousands of devices, like the IBM Blue Gene/L supercomputer. The maximum interconnection distance covers a relatively small area—on the order of a few tens of meters usually—but some SANs have distances spanning a few hundred meters. For example, *InfiniBand*, a popular SAN standard introduced in late 2000, supports system and storage I/O interconnects at up to 120 Gbps over a distance of 300 m.
- *Local area networks* (LANs)—This type of network is used for interconnecting autonomous computer systems distributed across a machine room or throughout a building or campus environment. Interconnecting PCs in a cluster is a prime example. Originally, LANs connected only up to a hundred devices, but with bridging LANs can now connect up to a few thousand devices. The maximum interconnect distance covers an area of a few kilometers usually, but some have distance spans of a few tens of kilometers. For instance, the most popular and enduring LAN, *Ethernet*, has a 10 Gbps standard version that supports maximum performance over a distance of 40 km.
- *Wide area networks* (WANs)—Also called *long-haul networks*, WANs connect computer systems distributed across the globe, which requires internetworking support. WANs connect many millions of computers over distance scales of many thousands of kilometers. Asynchronous Transfer Mode (ATM) is an example of a WAN.

Figure F.2 roughly shows the relationship of these networking domains in terms of the number of devices interconnected and their distance scales. Overlap exists for some of these networks in one or both dimensions, which leads to product competition. Some network solutions have become commercial standards while others remain proprietary. Although the preferred solutions may significantly differ from one interconnection network domain to another depending on the design requirements, the problems and concepts used to address network problems remain remarkably similar across the domains. No matter the target domain, networks should be designed so as not to be the bottleneck to system performance and cost efficiency. Hence, the ultimate goal of computer architects is to design interconnection networks of the lowest possible cost that are capable of transferring the maximum amount of available information in the shortest possible time.

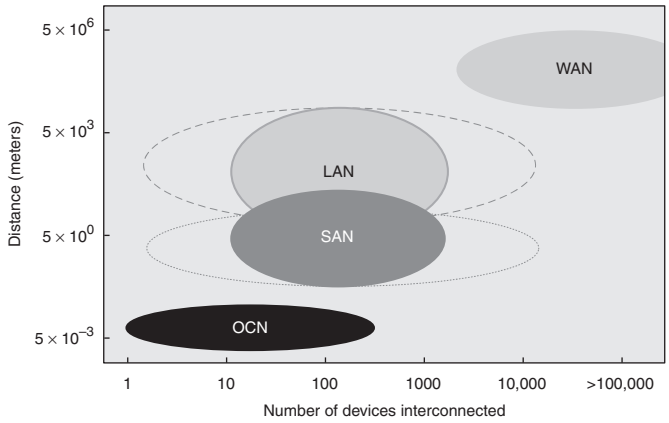


Figure F.2 Relationship of the four interconnection network domains in terms of number of devices connected and their distance scales: on-chip network (OCN), system/storage area network (SAN), local area network (LAN), and wide area network (WAN). Note that there are overlapping ranges where some of these networks compete. Some supercomputer systems use proprietary custom networks to interconnect several thousands of computers, while other systems, such as multicomputer clusters, use standard commercial networks.

Approach and Organization of This Appendix

Interconnection networks can be well understood by taking a top-down approach to unveiling the concepts and complexities involved in designing them. We do this by viewing the network initially as an opaque "black box" that simply and ideally performs certain necessary functions. Then we systematically open various layers of the black box, allowing more complex concepts and nonideal network behavior to be revealed. We begin this discussion by first considering the interconnection of just two devices in Section F.2, where the black box network can be viewed as a simple *dedicated link* network—that is, wires or collections of wires running bidirectionally between the devices. We then consider the interconnection of more than two devices in Section F.3, where the black box network can be viewed as a *shared link* network or as a *switched point-to-point* network connecting the devices. We continue to peel away various other layers of the black box by considering in more detail the network topology (Section F.4); routing, arbitration, and switching (Section F.5); and switch microarchitecture (Section F.6). Practical issues for commercial networks are considered in Section F.7, followed by examples illustrating the trade-offs for each type of network in Section F.8. Internetworking is briefly discussed in Section F.9, and additional crosscutting issues for interconnection networks are presented in Section F.10. Section F.11 gives some common fallacies

and pitfalls related to interconnection networks, and Section F.12 presents some concluding remarks. Finally, we provide a brief historical perspective and some suggested reading in Section F.13.

F.2 Interconnecting Two Devices

This section introduces the basic concepts required to understand how communication between just two networked devices takes place. This includes concepts that deal with situations in which the receiver may not be ready to process incoming data from the sender and situations in which transport errors may occur. To ease understanding, the black box network at this point can be conceptualized as an ideal network that behaves as simple dedicated links between the two devices. Figure F.3 illustrates this, where unidirectional wires run from device A to device B and *vice versa*, and each end node contains a buffer to hold the data. Regardless of the network complexity, whether dedicated link or not, a connection exists from each end node device to the network to inject and receive information to/from the network. We first describe the basic functions that must be performed at the end nodes to commence and complete communication, and then we discuss network media and the basic functions that must be performed by the network to carry out communication. Later, a simple performance model is given, along with several examples to highlight implications of key network parameters.

Network Interface Functions: Composing and Processing Messages

Suppose we want two networked devices to read a word from each other's memory. The unit of information sent or received is called a *message*. To acquire the desired data, the two devices must first compose and send a certain type of message in the form of a *request* containing the address of the data within the other device. The address (i.e., memory or operand location) allows the receiver to identify where to find the information being requested. After processing the request, each device then composes and sends another type of message, a *reply*, containing the data. The address and data information is typically referred to as the message *payload.*

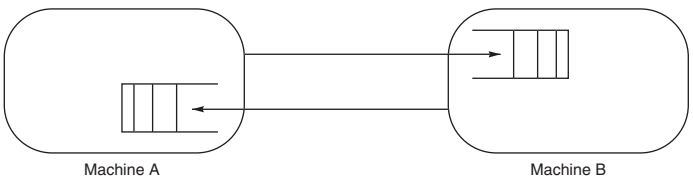


Figure F.3 A simple dedicated link network bidirectionally interconnecting two devices.

In addition to payload, every message contains some control bits needed by the network to deliver the message and process it at the receiver. The most typical are bits to distinguish between different types of messages (e.g., request, reply, request acknowledge, reply acknowledge) and bits that allow the network to transport the information properly to the destination. These additional control bits are encoded in the *header* and/or *trailer* portions of the message, depending on their location relative to the message payload. As an example, Figure F.4 shows the format of a message for the simple dedicated link network shown in Figure F.3. This example shows a single-word payload, but messages in some interconnection networks can include several thousands of words.

Before message transport over the network occurs, messages have to be composed. Likewise, upon receipt from the network, they must be processed. These and other functions described below are the role of the *network interface* (also referred to as the *channel adapter*) residing at the end nodes. Together with some direct memory access (DMA) engine and link drivers to transmit/receive messages to/from the network, some dedicated memory or register(s) may be used to buffer outgoing and incoming messages. Depending on the network domain and design specifications for the network, the network interface hardware may consist of nothing more than the communicating device itself (i.e., for OCNs and some SANs) or a separate card that integrates several embedded processors and DMA engines with thousands of megabytes of RAM (i.e., for many SANs and most LANs and WANs).

In addition to hardware, network interfaces can include software or firmware to perform the needed operations. Even the simple example shown in Figure F.3 may invoke messaging software to translate requests and replies into messages with the appropriate headers. This way, user applications need not worry about composing and processing messages as these tasks can be performed automatically at a lower level. An application program usually cooperates with the operating or runtime

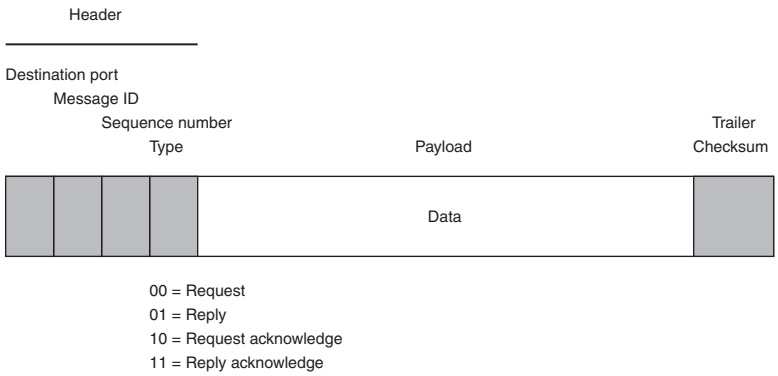


Figure F.4 An example packet format with header, payload, and checksum in the trailer.

system to send and receive messages. As the network is likely to be shared by many processes running on each device, the operating system cannot allow messages intended for one process to be received by another. Thus, the messaging software must include protection mechanisms that distinguish between processes. This distinction could be made by expanding the header with a *port* number that is known by both the sender and intended receiver processes.

In addition to composing and processing messages, additional functions need to be performed by the end nodes to establish communication among the communicating devices. Although hardware support can reduce the amount of work, some can be done by software. For example, most networks specify a maximum amount of information that can be transferred (i.e., *maximum transfer unit*) so that network buffers can be dimensioned appropriately. Messages longer than the maximum transfer unit are divided into smaller units, called *packets* (or *datagrams*), that are transported over the network. Packets are reassembled into messages at the destination end node before delivery to the application. Packets belonging to the same message can be distinguished from others by including a *message ID* field in the packet header. If packets arrive out of order at the destination, they are reordered when reassembled into a message. Another field in the packet header containing a *sequence number* is usually used for this purpose.

The sequence of steps the end node follows to commence and complete communication over the network is called a *communication protocol*. It generally has symmetric but reversed steps between sending and receiving information. Communication protocols are implemented by a combination of software and hardware to accelerate execution. For instance, many network interface cards implement hardware timers as well as hardware support to split messages into packets and reassemble them, compute the cyclic redundancy check (CRC) *checksum,* handle virtual memory addresses, and so on.

Some network interfaces include extra hardware to offload protocol processing from the host computer, such as TCP *offload engines* for LANs and WANs. But, for interconnection networks such as SANs that have low latency requirements, this may not be enough even when lighter-weight communication protocols are used such as message passing interface (MPI). Communication performance can be further improved by bypassing the operating system (OS). OS bypassing can be implemented by directly allocating message buffers in the network interface memory so that applications directly write into and read from those buffers. This avoids extra memory-to-memory copies. The corresponding protocols are referred to as *zero-copy* protocols or *user-level communication* protocols. Protection can still be maintained by calling the OS to allocate those buffers at initialization and preventing unauthorized memory accesses in hardware.

In general, some or all of the following are the steps needed to send a message at end node devices over a network:

1. The application executes a system call, which copies data to be sent into an operating system or network interface buffer, divides the message into packets (if needed), and composes the header and trailer for packets.

2. The checksum is calculated and included in the header or trailer of packets.
3. The timer is started, and the network interface hardware sends the packets.

Message reception is in the reverse order:

3. The network interface hardware receives the packets and puts them into its buffer or the operating system buffer.
2. The checksum is calculated for each packet. If the checksum matches the sender's checksum, the receiver sends an acknowledgment back to the packet sender. If not, it deletes the packet, assuming that the sender will resend the packet when the associated timer expires.
1. Once all packets pass the test, the system reassembles the message, copies the data to the user's address space, and signals the corresponding application.

The sender must still react to packet acknowledgments:

- When the sender gets an acknowledgment, it releases the copy of the corresponding packet from the buffer.
- If the sender reaches the time-out instead of receiving an acknowledgment, it resends the packet and restarts the timer.

Just as a protocol is implemented at network end nodes to support communication, protocols are also used across the network structure at the physical, data link, and network layers responsible primarily for packet transport, flow control, error handling, and other functions described next.

Basic Network Structure and Functions: Media and Form Factor, Packet Transport, Flow Control, and Error Handling

Once a packet is ready for transmission at its source, it is injected into the network using some dedicated hardware at the network interface. The hardware includes some transceiver circuits to drive the physical network media—either electrical or optical. The type of *media* and *form factor* depends largely on the interconnect distances over which certain signaling rates (e.g., transmission speed) should be sustainable. For centimeter or less distances on a chip or multichip module, typically the middle to upper copper metal layers can be used for interconnects at multi-Gbps signaling rates per line. A dozen or more layers of copper traces or tracks imprinted on circuit boards, midplanes, and backplanes can be used for Gbps differential-pair signaling rates at distances of about a meter or so. Category 5E unshielded twisted-pair copper wiring allows 0.25 Gbps transmission speed over distances of 100 meters. Coaxial copper cables can deliver 10 Mbps over kilometer distances. In these conductor lines, distance can usually be traded off for higher transmission speed, up to a certain point. Optical media enable faster transmission

speeds at distances of kilometers. Multimode fiber supports 100 Mbps transmission rates over a few kilometers, and more expensive single-mode fiber supports Gbps transmission speeds over distances of several kilometers. Wavelength division multiplexing allows several times more bandwidth to be achieved in fiber (i.e., by a factor of the number of wavelengths used).

The hardware used to drive network links may also include some encoders to encode the signal in a format other than binary that is suitable for the given transport distance. Encoding techniques can use multiple voltage levels, redundancy, data and control rotation (e.g., 4b5b encoding), and/or a guaranteed minimum number of signal transitions per unit time to allow for clock recovery at the receiver. The signal is decoded at the receiver end, and the packet is stored in the corresponding buffer. All of these operations are performed at the network physical layer, the details of which are beyond the scope of this appendix. Fortunately, we do not need to worry about them. From the perspective of the data link and higher layers, the physical layer can be viewed as a long linear pipeline without staging in which signals propagate as waves through the network transmission medium. All of the above functions are generally referred to as *packet transport*.

Besides packet transport, the network hardware and software are jointly responsible at the data link and network protocol layers for ensuring reliable delivery of packets. These responsibilities include: (1) preventing the sender from sending packets at a faster rate than they can be processed by the receiver, and (2) ensuring that the packet is neither garbled nor lost in transit. The first responsibility is met by either discarding packets at the receiver when its buffer is full and later notifying the sender to retransmit them, or by notifying the sender to stop sending packets when the buffer becomes full and to resume later once it has room for more packets. The latter strategy is generally known as *flow control*.

There are several interesting techniques commonly used to implement flow control beyond simple *handshaking* between the sender and receiver. The more popular techniques are *Xon/Xoff* (also referred to as *Stop & Go*) and *credit-based* flow control. Xon/Xoff consists of the receiver notifying the sender either to stop or to resume sending packets once high and low buffer occupancy levels are reached, respectively, with some hysteresis to reduce the number of notifications. Notifications are sent as "stop" and "go" signals using additional control wires or encoded in control packets. Credit-based flow control typically uses a credit counter at the sender that initially contains a number of credits equal to the number of buffers at the receiver. Every time a packet is transmitted, the sender decrements the credit counter. When the receiver consumes a packet from its buffer, it returns a credit to the sender in the form of a control packet that notifies the sender to increment its counter upon receipt of the credit. These techniques essentially control the flow of packets into the network by *throttling* packet injection at the sender when the receiver reaches a low watermark or when the sender runs out of credits.

Xon/Xoff usually generates much less control traffic than credit-based flow control because notifications are only sent when the high or low buffer occupancy levels are crossed. On the other hand, credit-based flow control requires less than half the buffer size required by Xon/Xoff. Buffers for Xon/Xoff must be large

enough to prevent overflow before the "stop" control signal reaches the sender. Overflow cannot happen when using credit-based flow control because the sender will run out of credits, thus stopping transmission. For both schemes, full link bandwidth utilization is possible only if buffers are large enough for the distance over which communication takes place.

Let's compare the buffering requirements of the two flow control techniques in a simple example covering the various interconnection network domains.

Example Suppose we have a dedicated-link network with a raw data bandwidth of 8 Gbps for each link in each direction interconnecting two devices. Packets of 100 bytes (including the header) are continuously transmitted from one device to the other to fully utilize network bandwidth. What is the minimum amount of credits and buffer space required by credit-based flow control assuming interconnect distances of 1 cm, 1 m, 100 m, and 10 km if only link propagation delay is taken into account? How does the minimum buffer space compare against Xon/Xoff?

Answer At the start, the receiver buffer is initially empty and the sender contains a number of credits equal to buffer capacity. The sender will consume a credit every time a packet is transmitted. For the sender to continue transmitting packets at network speed, the first returned credit must reach the sender before the sender runs out of credits. After receiving the first credit, the sender will keep receiving credits at the same rate it transmits packets. As we are considering only propagation delay over the link and no other sources of delay or overhead, null processing time at the sender and receiver are assumed. The time required for the first credit to reach the sender since it started transmission of the first packet is equal to the round-trip propagation delay for the packet transmitted to the receiver and the return credit transmitted back to the sender. This time must be less than or equal to the packet transmission time multiplied by the initial credit count:

$$\text{Packet propagation delay} + \text{Credit propagation delay} \leq \frac{\text{Packet size}}{\text{Bandwidth}} \times \text{Credit count}$$

The speed of light is about 300,000 km/sec. Assume we can achieve 66% of that in a conductor. Thus, the minimum number of credits for each distance is given by

$$\left(\frac{\text{Distance}}{2/3 \times 300{,}000\ \text{km/sec}}\right) \times 2 \leq \frac{100\ \text{bytes}}{8\ \text{Gbits/sec}} \times \text{Credit count}$$

As each credit represents one packet-sized buffer entry, the minimum amount of credits (and, likewise, buffer space) needed by each device is one for the 1 cm and 1 m distances, 10 for the 100 m distance, and 1000 packets for the 10 km distance. For Xon/Xoff, this minimum buffer size corresponds to the buffer fragment from the high occupancy level to the top of the buffer and from the low occupancy level to the bottom of the buffer. With the added hysteresis between both occupancy levels to reduce notifications, the minimum buffer space for Xon/Xoff turns out to be more than twice that for credit-based flow control.

Networks that implement flow control do not need to drop packets and are sometimes referred to as *lossless* networks; networks that drop packets are sometimes referred to as *lossy* networks. This single difference in the way packets are handled by the network drastically constrains the kinds of solutions that can be implemented to address other related network problems, including packet routing, congestion, deadlock, and reliability, as we will see later in this appendix. This difference also affects performance significantly as dropped packets need to be retransmitted, thus consuming more link bandwidth and suffering extra delay. These behavioral and performance differences ultimately restrict the interconnection network domains for which certain solutions are applicable. For instance, most networks delivering packets over relatively short distances (e.g., OCNs and SANs) tend to implement flow control; on the other hand, networks delivering packets over relatively long distances (e.g., LANs and WANs) tend to be designed to drop packets. For the shorter distances, the delay in propagating flow control information back to the sender can be negligible, but not so for longer distance scales. The kinds of applications that are usually run also influence the choice of lossless versus lossy networks. For instance, dropping packets sent by an Internet client like a Web browser affects only the delay observed by the corresponding user. However, dropping a packet sent by a process from a parallel application may lead to a significant increase in the overall execution time of the application if that packet's delay is on the critical path.

The second responsibility of ensuring that packets are neither garbled nor lost in transit can be met by implementing some mechanisms to detect and recover from transport errors. Adding a checksum or some other error detection field to the packet format, as shown in Figure F.4, allows the receiver to detect errors. This redundant information is calculated when the packet is sent and checked upon receipt. The receiver then sends an acknowledgment in the form of a control packet if the packet passes the test. Note that this acknowledgment control packet may simultaneously contain flow control information (e.g., a credit or stop signal), thus reducing control packet overhead. As described earlier, the most common way to recover from errors is to have a timer record the time each packet is sent and to presume the packet is lost or erroneously transported if the timer expires before an acknowledgment arrives. The packet is then resent.

The communication protocol across the network and network end nodes must handle many more issues other than packet transport, flow control, and reliability. For example, if two devices are from different manufacturers, they might order bytes differently within a word (Big Endian versus Little Endian byte ordering). The protocol must reverse the order of bytes in each word as part of the delivery system. It must also guard against the possibility of duplicate packets if a delayed packet were to become unstuck. Depending on the system requirements, the protocol may have to implement *pipelining* among operations to improve performance. Finally, the protocol may need to handle network congestion to prevent performance degradation when more than two devices are connected, as described later in Section F.7.

Characterizing Performance: Latency and Effective Bandwidth

Now that we have covered the basic steps for sending and receiving messages between two devices, we can discuss performance. We start by discussing the latency when transporting a single packet. Then we discuss the effective bandwidth (also known as throughput) that can be achieved when the transmission of multiple packets is pipelined over the network at the packet level.

Figure F.5 shows the basic components of latency for a single packet. Note that some latency components will be broken down further in later sections as the internals of the "black box" network are revealed. The timing parameters in Figure F.5 apply to many interconnection network domains: inside a chip, between chips on a board, between boards in a chassis, between chassis within a computer, between computers in a cluster, between clusters, and so on. The values may change, but the components of latency remain the same.

The following terms are often used loosely, leading to confusion, so we define them here more precisely:

- *Bandwidth*—Strictly speaking, the *bandwidth* of a transmission medium refers to the range of frequencies for which the attenuation per unit length introduced by that medium is below a certain threshold. It must be distinguished from the *transmission speed*, which is the amount of information transmitted over a medium per unit time. For example, modems successfully increased transmission speed in the late 1990s for a fixed bandwidth (i.e., the 3 KHz bandwidth provided by voice channels over telephone lines) by encoding more voltage levels and, hence, more bits per signal cycle. However, to be consistent with

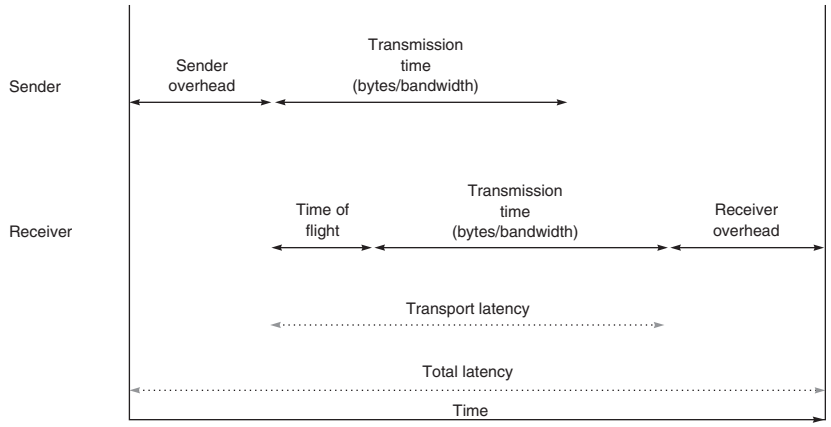


Figure F.5 Components of packet latency. Depending on whether it is an OCN, SAN, LAN, or WAN, the relative amounts of sending and receiving overhead, time of flight, and transmission time are usually quite different from those illustrated here.

its more widely understood meaning, we use the term *band-width* to refer to the maximum rate at which information can be transferred, where information includes packet header, payload, and trailer. The units are traditionally bits per second, although bytes per second is sometimes used. The term *bandwidth* is also used to mean the measured speed of the medium (i.e., network links). *Aggregate bandwidth* refers to the total data bandwidth supplied by the network, and *effective bandwidth* or *throughput* is the fraction of aggregate bandwidth delivered by the network to an application.

- *Time of flight*—This is the time for the first bit of the packet to arrive at the receiver, including the propagation delay over the links and delays due to other hardware in the network such as link repeaters and network switches. The unit of measure for time of flight can be in milliseconds for WANs, microseconds for LANs, nanoseconds for SANs, and picoseconds for OCNs.
- *Transmission time*—This is the time for the packet to pass through the network, not including time of flight. One way to measure it is the difference in time between when the first bit of the packet arrives at the receiver and when the last bit of that packet arrives at the receiver. By definition, transmission time is equal to the size of the packet divided by the data bandwidth of network links. This measure assumes there are no other packets contending for that bandwidth (i.e., a zero-load or no-load network).
- *Transport latency*—This is the sum of time of flight and transmission time. Transport latency is the time that the packet spends in the interconnection network. Stated alternatively, it is the time between when the first bit of the packet is injected into the network and when the last bit of that packet arrives at the receiver. It does not include the overhead of preparing the packet at the sender or processing it when it arrives at the receiver.
- *Sending overhead*—This is the time for the end node to prepare the packet (as opposed to the message) for injection into the network, including both hardware and software components. Note that the end node is busy for the entire time, hence the use of the term *overhead*. Once the end node is free, any subsequent delays are considered part of the transport latency. We assume that overhead consists of a constant term plus a variable term that depends on packet size. The constant term includes memory allocation, packet header preparation, setting up DMA devices, and so on. The variable term is mostly due to copies from buffer to buffer and is usually negligible for very short packets.
- *Receiving overhead*—This is the time for the end node to process an incoming packet, including both hardware and software components. We also assume here that overhead consists of a constant term plus a variable term that depends on packet size. In general, the receiving overhead is larger than the sending overhead. For example, the receiver may pay the cost of an interrupt or may have to reorder and reassemble packets into messages.

The total latency of a packet can be expressed algebraically by the following:

$$\text{Latency} = \text{Sending overhead} + \text{Time of flight} + \frac{\text{Packet size}}{\text{Bandwidth}} + \text{Receiving overhead}$$

Let's see how the various components of transport latency and the sending and receiving overheads change in importance as we go across the interconnection network domains: from OCNs to SANs to LANs to WANs.

Example Assume that we have a dedicated link network with a data bandwidth of 8 Gbps for each link in each direction interconnecting two devices within an OCN, SAN, LAN, or WAN, and we wish to transmit packets of 100 bytes (including the header) between the devices. The end nodes have a per-packet sending overhead of $x + 0.05$ ns/byte and receiving overhead of $4/3(x) + 0.05$ ns/byte, where x is 0 µs for the OCN, 0.3 µs for the SAN, 3 µs for the LAN, and 30 µs for the WAN, which are typical for these network types. Calculate the total latency to send packets from one device to the other for interconnection distances of 0.5 cm, 5 m, 5000 m, and 5000 km assuming that time of flight consists only of link propagation delay (i.e., no switching or other sources of delay).

Answer Using the above expression and the calculation for propagation delay through a conductor given in the previous example, we can plug in the parameters for each of the networks to find their total packet latency. For the OCN:

$$\text{Latency} = \text{Sending overhead} + \text{Time of flight} + \frac{\text{Packet size}}{\text{Bandwidth}} + \text{Receiving overhead}$$

$$= 5 \text{ ns} + \frac{0.5 \text{ cm}}{2/3 \times 300{,}000 \text{ km/sec}} + \frac{100 \text{ bytes}}{8 \text{ Gbits/sec}} + 5 \text{ ns}$$

Converting all terms into nanoseconds (ns) leads to the following for the OCN:

$$\text{Total latency (OCN)} = 5 \text{ ns} + \frac{0.5 \text{ cm}}{2/3 \times 300{,}000 \text{ km/sec}} + \frac{100 \times 8}{8} \text{ns} + 5 \text{ ns}$$

$$= 5 \text{ ns} + 0.025 \text{ ns} + 100 \text{ ns} + 5 \text{ ns}$$

$$= 110.025 \text{ ns}$$

Substituting in the appropriate values for the SAN gives the following latency:

$$\text{Total latency (SAN)} = 0.305 \text{ µs} + \frac{5 \text{ m}}{2/3 \times 300{,}000 \text{ km/sec}} + \frac{100 \text{ bytes}}{8 \text{ Gbits/sec}} + 0.405 \text{ µs}$$

$$= 0.305 \text{ µs} + 0.025 \text{ µs} + 0.1 \text{ µs} + 0.405 \text{ µs}$$

$$= 0.835 \text{ µs}$$

Substituting in the appropriate values for the LAN gives the following latency:

$$\begin{aligned}\text{Total latency (LAN)} &= 3.005\ \mu s + \frac{5\ \text{km}}{2/3 \times 300{,}000\ \text{km/sec}} + \frac{100\ \text{bytes}}{8\ \text{Gbits/sec}} + 4.005\ \mu s \\ &= 3.005\ \mu s + 25\ \mu s + 0.1\ \mu s + 4.005\ \mu s \\ &= 32.11\ \mu s\end{aligned}$$

Substituting in the appropriate values for the WAN gives the following latency:

$$\begin{aligned}\text{Total latency (WAN)} &= 30.005\ \mu s + \frac{5000\ \text{km}}{2/3 \times 300{,}000\ \text{km/sec}} + \frac{100\ \text{bytes}}{8\ \text{Gbits/sec}} + 40.005\ \mu s \\ &= 30.005\ \mu s + 25000\ \mu s + 0.1\ \mu s + 40.005\ \mu s \\ &= 25.07\ \text{ms}\end{aligned}$$

The increased fraction of the latency required by time of flight for the longer distances along with the greater likelihood of errors over the longer distances are among the reasons why WANs and LANs use more sophisticated and time-consuming communication protocols, which increase sending and receiving overheads. The need for standardization is another reason. Complexity also increases due to the requirements imposed on the protocol by the typical applications that run over the various interconnection network domains as we go from tens to hundreds to thousands to many thousands of devices. We will consider this in later sections when we discuss connecting more than two devices. The above example shows that the propagation delay component of time of flight for WANs and some LANs is so long that other latency components—including the sending and receiving overheads—can practically be ignored. This is not so for SANs and OCNs where the propagation delay pales in comparison to the overheads and transmission delay. Remember that time-of-flight latency due to switches and other hardware in the network besides sheer propagation delay through the links is neglected in the above example. For noncongested networks, switch latency generally is small compared to the overheads and propagation delay through the links in WANs and LANs, but this is not necessarily so for multiprocessor SANs and multicore OCNs, as we will see in later sections.

So far, we have considered the transport of a single packet and computed the associated end-to-end total packet latency. In order to compute the effective bandwidth for two networked devices, we have to consider a continuous stream of packets transported between them. We must keep in mind that, in addition to minimizing packet latency, the goal of any network optimized for a given cost and power consumption target is to transfer the maximum amount of available information in the shortest possible time, as measured by the effective bandwidth delivered by the network. For applications that do not require a response before sending the next packet, the sender can overlap the sending overhead of later packets with the transport latency and receiver overhead of prior packets. This essentially pipelines the transmission of packets over the network, also known as *link pipelining*. Fortunately, as discussed in prior chapters of this book, there are many application

areas where communication from either several applications or several threads from the same application can run concurrently (e.g., a Web server concurrently serving thousands of client requests or streaming media), thus allowing a device to send a stream of packets without having to wait for an acknowledgment or a reply. Also, as long messages are usually divided into packets of maximum size before transport, a number of packets are injected into the network in succession for such cases. If such overlap were not possible, packets would have to wait for prior packets to be acknowledged before being transmitted and thus suffer significant performance degradation.

Packets transported in a pipelined fashion can be acknowledged quite straightforwardly simply by keeping a copy at the source of all unacknowledged packets that have been sent and keeping track of the correspondence between returned acknowledgments and packets stored in the buffer. Packets will be removed from the buffer when the corresponding acknowledgment is received by the sender. This can be done by including the message ID and packet sequence number associated with the packet in the packet's acknowledgment. Furthermore, a separate timer must be associated with each buffered packet, allowing the packet to be resent if the associated time-out expires.

Pipelining packet transport over the network has many similarities with pipelining computation within a processor. However, among some differences are that it does not require any staging latches. Information is simply propagated through network links as a sequence of signal waves. Thus, the network can be considered as a logical pipeline consisting of as many stages as are required so that the time of flight does not affect the effective bandwidth that can be achieved. Transmission of a packet can start immediately after the transmission of the previous one, thus overlapping the sending overhead of a packet with the transport and receiver latency of previous packets. If the sending overhead is smaller than the transmission time, packets follow each other back-to-back, and the effective bandwidth approaches the raw link bandwidth when continuously transmitting packets. On the other hand, if the sending overhead is greater than the transmission time, the effective bandwidth at the injection point will remain well below the raw link bandwidth. The resulting *link injection bandwidth*, $BW_{LinkInjection}$, for each link injecting a continuous stream of packets into a network is calculated with the following expression:

$$BW_{LinkInjection} = \frac{\text{Packet size}}{\max(\text{Sending overhead}, \text{Transmission time})}$$

We must also consider what happens if the receiver is unable to consume packets at the same rate they arrive. This occurs if the receiving overhead is greater than the sending overhead and the receiver cannot process incoming packets fast enough. In this case, the *link reception bandwidth*, $BW_{LinkReception}$, for each reception link of the network is less than the link injection bandwidth and is obtained with this expression:

$$BW_{LinkReception} = \frac{\text{Packet size}}{\max(\text{Receiving overhead}, \text{Transmission time})}$$

When communication takes place between two devices interconnected by dedicated links, all the packets sent by one device will be received by the other. If the receiver cannot process packets fast enough, the receiver buffer will become full, and flow control will throttle transmission at the sender. As this situation is produced by causes external to the network, we will not consider it further here. Moreover, if the receiving overhead is greater than the sending overhead, the receiver buffer will fill up and flow control will, likewise, throttle transmission at the sender. In this case, the effect of flow control is, on average, the same as if we replace sending overhead with receiving overhead. Assuming an ideal network that behaves like two dedicated links running in opposite directions at the full link bandwidth between the two devices—which is consistent with our black box view of the network to this point—the resulting effective bandwidth is the smaller of twice the injection bandwidth (to account for the two injection links, one for each device) or twice the reception bandwidth. This results in the following expression for effective bandwidth:

$$\text{Effective bandwidth} = \min\left(2 \times \text{BW}_{\text{LinkInjection}}, 2 \times \text{BW}_{\text{LinkReception}}\right) = \frac{2 \times \text{Packet size}}{\max\left(\text{Overhead}, \text{Transmission time}\right)}$$

where Overhead = max(Sending overhead, Receiving overhead). Taking into account the expression for the transmission time, it is obvious that the effective bandwidth delivered by the network is identical to the aggregate network bandwidth when the transmission time is greater than the overhead. Therefore, full network utilization is achieved regardless of the value for the time of flight and, thus, regardless of the distance traveled by packets, assuming ideal network behavior (i.e., enough credits and buffers are provided for credit-based and Xon/Xoff flow control). This analysis assumes that the sender and receiver network interfaces can process only one packet at a time. If multiple packets can be processed in parallel (e.g., as is done in IBM's Federation network interfaces), the overheads for those packets can be overlapped, which increases effective bandwidth by that overlap factor up to the amount bounded by the transmission time.

Let's use the equation on page F-17 to explore the impact of packet size, transmission time, and overhead on $\text{BW}_{\text{Link Injection}}$, $\text{BW}_{\text{LinkReception}}$, and effective bandwidth for the various network domains: OCNs, SANs, LANs, and WANs.

Example As in the previous example, assume we have a dedicated link network with a data bandwidth of 8 Gbps for each link in each direction interconnecting the two devices within an OCN, SAN, LAN, or WAN. Plot effective bandwidth versus packet size for each type of network for packets ranging in size from 4 bytes (i.e., a single 32-bit word) to 1500 bytes (i.e., the maximum transfer unit for Ethernet), assuming that end nodes have the same per-packet sending and receiving overheads as before: $x + 0.05$ ns/byte and $4/3(x) + 0.05$ ns/byte, respectively, where x is 0 μs for the OCN, 0.3 μs for the SAN, 3 μs for the LAN, and 30 μs for the WAN. What limits the effective bandwidth, and for what packet sizes is the effective bandwidth within 10% of the aggregate network bandwidth?

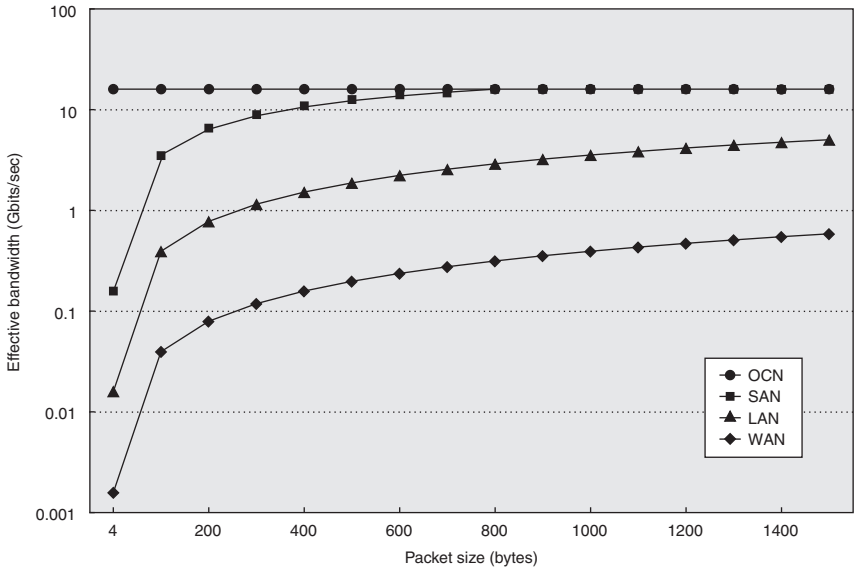


Figure F.6 Effective bandwidth versus packet size plotted in semi-log form for the four network domains. Overhead can be amortized by increasing the packet size, but for too large of an overhead (e.g., for WANs and some LANs) scaling the packet size is of little help. Other considerations come into play that limit the maximum packet size.

Answer Figure F.6 plots effective bandwidth versus packet size for the four network domains using the simple equation and parameters given above. For all packet sizes in the OCN, transmission time is greater than overhead (sending or receiving), allowing full utilization of the aggregate bandwidth, which is 16 Gbps—that is, injection link (alternatively, reception link) bandwidth times two to account for both devices. For the SAN, overhead—specifically, receiving overhead—is larger than transmission time for packets less than about 800 bytes; consequently, packets of 655 bytes and larger are needed to utilize 90% or more of the aggregate bandwidth. For LANs and WANs, most of the link bandwidth is not utilized since overhead in this example is many times larger than transmission time for all packet sizes.

This example highlights the importance of reducing the sending and receiving overheads relative to packet transmission time in order to maximize the effective bandwidth delivered by the network.

The analysis above suggests that it is possible to provide some upper bound for the effective bandwidth by analyzing the path followed by packets and determining where the bottleneck occurs. We can extend this idea beyond the network interfaces by defining a model that considers the entire network from end to

end as a pipe and identifying the narrowest section of that pipe. There are three areas of interest in that pipe: the aggregate of all network injection links and the corresponding *network injection bandwidth* ($BW_{NetworkInjection}$), the aggregate of all network reception links and the corresponding *network reception bandwidth* ($BW_{NetworkReception}$), and the aggregate of all network links and the corresponding *network bandwidth* ($BW_{Network}$). Expressions for these will be given in later sections as various layers of the black box view of the network are peeled away.

To this point, we have assumed that for just two interconnected devices the black box network behaves ideally and the network bandwidth is equal to the aggregate raw network bandwidth. In reality, it can be much less than the aggregate bandwidth as we will see in the following sections. In general, the effective bandwidth delivered end-to-end by the network to an application is upper bounded by the minimum across all three potential bottleneck areas:

$$\text{Effective bandwidth} = \min\left(BW_{NetworkInjection}, BW_{Network}, BW_{NetworkReception}\right)$$

We will expand upon this expression further in the following sections as we reveal more about interconnection networks and consider the more general case of interconnecting more than two devices.

In some sections of this appendix, we show how the concepts introduced in the section take shape in example high-end commercial products. Figure F.7 lists several commercial computers that, at one point in time in their existence, were among the highest-performing systems in the world within their class. Although these systems are capable of interconnecting more than two devices, they implement the basic functions needed for interconnecting only two devices. In addition to being applicable to the SANs used in those systems, the issues discussed in this section also apply to other interconnect domains: from OCNs to WANs.

F.3 Connecting More than Two Devices

To this point, we have considered the connection of only two devices communicating over a network viewed as a black box, but what makes interconnection networks interesting is the ability to connect hundreds or even many thousands of devices together. Consequently, what makes them interesting also makes them more challenging to build. In order to connect more than two devices, a suitable structure and more functionality must be supported by the network. This section continues with our black box approach by introducing, at a conceptual level, additional network structure and functions that must be supported when interconnecting more than two devices. More details on these individual subjects are given in Sections F.4 through F.7. Where applicable, we relate the additional structure and functions to network media, flow control, and other basics presented in the previous section. In this section, we also classify networks into two broad categories

Company	System [network] name	Intro year	Max. number of compute nodes [# CPUs]	System footprint for max. configuration	Packet [header] max size (bytes)	Injection [reception] node BW in MB/sec	Minimum send/ receive overhead	Maximum copper link length; flow control; error
Intel	ASCI Red Paragon	2001	4510 [×2]	2500 ft^2	1984 [4]	400 [400]	Few µs	Handshaking; CRC + parity
IBM	ASCI White SP Power3 [Colony]	2001	512 [×16]	10,000 ft^2	1024 [6]	500 [500]	~3 µs	25 m; credit-based; CRC
Intel	Thunder Itanium2 Tiger4 [$QsNet^{II}$]	2004	1024 [×4]	120 m^2	2048 [14]	928 [928]	0.240 µs	13 m; credit-based; CRC for link, dest.
Cray	XT3 [SeaStar]	2004	30,508 [×1]	263.8 m^2	80 [16]	3200 [3200]	Few µs	7 m; credit-based; CRC
Cray	X1E	2004	1024 [×1]	27 m^2	32 [16]	1600 [1600]	0 (direct LD ST accesses)	5 m; credit-based; CRC
IBM	ASC Purple pSeries 575 [Federation]	2005	>1280 [×8]	6720 ft^2	2048 [7]	2000 [2000]	~1 µs with up to 4 packets processed in ‖	25 m; credit-based; CRC
IBM	Blue Gene/L eServer Sol. [Torus Net.]	2005	65,536 [×2]	2500 ft^2 (.9 × .9 × 1.9 m^3/ 1 K node rack)	256 [8]	612.5 [1050]	~3 µs (2300 cycles)	8.6 m; credit-based; CRC (header/pkt)

Figure F.7 Basic characteristics of interconnection networks in commercial high-performance computer systems.

based on their connection structure—*shared-media* versus *switched-media* networks—and we compare them. Finally, expanded expressions for characterizing network performance are given, followed by an example.

Additional Network Structure and Functions: Topology, Routing, Arbitration, and Switching

Networks interconnecting more than two devices require mechanisms to physically connect the packet source to its destination in order to transport the packet and deliver it to the correct destination. These mechanisms can be implemented in different ways and significantly vary across interconnection network domains. However, the types of network structure and functions performed by those mechanisms are very much the same, regardless of the domain.

When multiple devices are interconnected by a network, the connections between them oftentimes cannot be permanently established with dedicated links.

This could either be too restrictive as all the packets from a given source would go to the same one destination (and not to others) or prohibitively expensive as a dedicated link would be needed from every source to every destination (we will evaluate this further in the next section). Therefore, networks usually share paths among different pairs of devices, but how those paths are shared is determined by the network connection structure, commonly referred to as the network *topology*. Topology addresses the important issue of "*What paths are possible for packets?*" so packets reach their intended destinations.

Every network that interconnects more than two devices also requires some mechanism to deliver each packet to the correct destination. The associated function is referred to as *routing*, which can be defined as the set of operations that need to be performed to compute a valid path from the packet source to its destinations. Routing addresses the important issue of "*Which of the possible paths are allowable (valid) for packets?*" so packets reach their intended destinations. Depending on the network, this function may be executed at the packet source to compute the entire path, at some intermediate devices to compute fragments of the path on the fly, or even at every possible destination device to verify whether that device is the intended destination for the packet. Usually, the packet header shown in Figure F.4 is extended to include the necessary routing information.

In general, as networks usually contain shared paths or parts thereof among different pairs of devices, packets may request some shared resources. When several packets request the same resources at the same time, an *arbitration* function is required to resolve the conflict. Arbitration, along with flow control, addresses the important issue of "*When are paths available for packets?*" Every time arbitration is performed, there is a winner and possibly several losers. The losers are not granted access to the requested resources and are typically buffered. As indicated in the previous section, flow control may be implemented to prevent buffer overflow. The winner proceeds toward its destination once the granted resources are switched in, providing a path for the packet to advance. This function is referred to as *switching*. Switching addresses the important issue of "*How are paths allocated to packets?*" To achieve better utilization of existing communication resources, most networks do not establish an entire end-to-end path at once. Instead, as explained in Section F.5, paths are usually established one fragment at a time.

These three network functions—routing, arbitration, and switching—must be implemented in every network connecting more than two devices, no matter what form the network topology takes. This is in addition to the basic functions mentioned in the previous section. However, the complexity of these functions and the order in which they are performed depends on the category of network topology, as discussed below. In general, routing, arbitration, and switching are required to establish a valid path from source to destination from among the possible paths provided by the network topology. Once the path has been established, the packet transport functions previously described are used to reliably transmit packets and receive them at the corresponding destination. Flow control, if implemented, prevents buffer overflow by throttling the sender. It can be implemented at the end-to-end level, the link level within the network, or both.

Shared-Media Networks

The simplest way to connect multiple devices is to have them share the network media, as shown for the bus in Figure F.8 (a). This has been the traditional way of interconnecting devices. The shared media can operate in *half-duplex* mode, where data can be carried in either direction over the media but simultaneous transmission and reception of data by the same device is not allowed, or in *full-duplex,* where the data can be carried in both directions and simultaneously transmitted and received by the same device. Until very recently, I/O devices in most systems typically shared a single I/O bus, and early system-on-chip (SoC) designs made use of a shared bus to interconnect on-chip components. The most popular LAN, Ethernet, was originally implemented as a half-duplex bus shared by up to a hundred computers, although now switched-media versions also exist.

Given that network media are shared, there must be a mechanism to coordinate and arbitrate the use of the shared media so that only one packet is sent at a time. If the physical distance between network devices is small, it may be possible to have a central arbiter to grant permission to send packets. In this case, the network nodes may use dedicated control lines to interface with the arbiter. Centralized arbitration is impractical, however, for networks with a large number of nodes spread over large distances, so distributed forms of arbitration are also used. This is the case for the original Ethernet shared-media LAN.

A first step toward distributed arbitration of shared media is "looking before you leap." A node first checks the network to avoid trying to send a packet while another packet is already in the network. Listening before transmission to avoid collisions is called *carrier sensing*. If the interconnection is idle, the node tries to send. Looking first is not a guarantee of success, of course, as some other node may also decide to send at the same instant. When two nodes send at the same time,

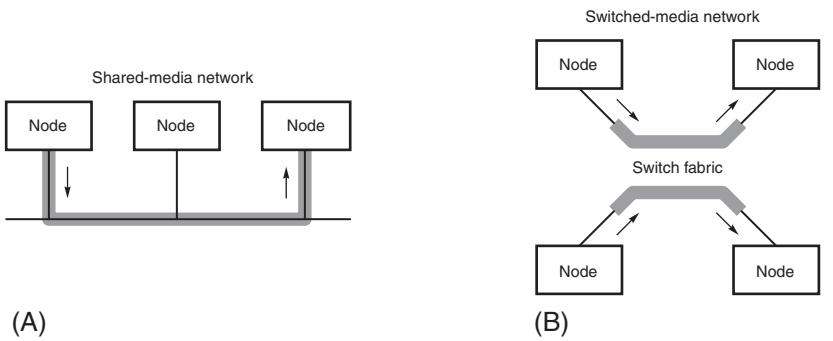


Figure F.8 (a) A shared-media network versus (b) a switched-media network. Ethernet was originally a shared media network, but switched Ethernet is now available. All nodes on the shared-media networks must dynamically share the raw bandwidth of one link, but switched-media networks can support multiple links, providing higher raw aggregate bandwidth.

a *collision* occurs. Let's assume that the network interface can detect any resulting collisions by listening to hear if the data become garbled by other data appearing on the line. Listening to detect collisions is called *collision detection*. This is the second step of distributed arbitration.

The problem is not solved yet. If, after detecting a collision, every node on the network waited exactly the same amount of time, listened to be sure there was no traffic, and then tried to send again, we could still have synchronized nodes that would repeatedly bump heads. To avoid repeated head-on collisions, each node whose packet gets garbled waits (or *backs off*) a random amount of time before resending. Randomization breaks the synchronization. Subsequent collisions result in exponentially increasing time between attempts to retransmit, so as not to tax the network.

Although this approach controls congestion on the shared media, it is not guaranteed to be fair—some subsequent node may transmit while those that collided are waiting. If the network does not have high demand from many nodes, this simple approach works well. Under high utilization, however, performance degrades since the media are shared and fairness is not ensured. Another distributed approach to arbitration of shared media that can support fairness is to pass a token between nodes. The function of the token is to grant the acquiring node the right to use the network. If the token circulates in a cyclic fashion between the nodes, a certain amount of fairness is ensured in the arbitration process.

Once arbitration has been performed and a device has been granted access to the shared media, the function of switching is straightforward. The granted device simply needs to connect itself to the shared media, thus establishing a path to every possible destination. Also, routing is very simple to implement. Given that the media are shared and attached to all the devices, every device will see every packet. Therefore, each device just needs to check whether or not a given packet is intended for that device. A beneficial side effect of this strategy is that a device can send a packet to all the devices attached to the shared media through a single transmission. This style of communication is called *broadcasting*, in contrast to *unicasting,* in which each packet is intended for only one device. The shared media make it easy to broadcast a packet to every device or, alternatively, to a subset of devices, called *multicasting*.

Switched-Media Networks

The alternative to sharing the entire network media at once across all attached nodes is to switch between disjoint portions of it shared by the nodes. Those portions consist of passive *point-to-point links* between active *switch* components that dynamically establish communication between sets of source-destination pairs. These passive and active components make up what is referred to as the network *switch fabric* or *network fabric*, to which end nodes are connected. This approach is shown conceptually in Figure F.8(b). The switch fabric is described in greater detail in Sections F.4 through F.7, where various black box layers for switched-media networks are further revealed. Nevertheless, the high-level view shown

in Figure F.8(b) illustrates the potential bandwidth improvement of switched-media networks over shared-media networks: aggregate bandwidth can be many times higher than that of shared-media networks, allowing the possibility of greater effective bandwidth to be achieved. At best, only one node at a time can transmit packets over the shared media, whereas it is possible for all attached nodes to do so over the switched-media network.

Like their shared-media counterparts, switched-media networks must implement the three additional functions previously mentioned: routing, arbitration, and switching. Every time a packet enters the network, it is routed in order to select a path toward its destination provided by the topology. The path requested by the packet must be granted by some centralized or distributed arbiter, which resolves conflicts among concurrent requests for resources along the same path. Once the requested resources are granted, the network "switches in" the required connections to establish the path and allows the packet to be forwarded toward its destination. If the requested resources are not granted, the packet is usually buffered, as mentioned previously. Routing, arbitration, and switching functions are usually performed within switched networks in this order, whereas in shared-media networks routing typically is the last function performed.

Comparison of Shared- and Switched-Media Networks

In general, the advantage of shared-media networks is their low cost, but, consequently, their aggregate network bandwidth does not scale at all with the number of interconnected devices. Also, a global arbitration scheme is required to resolve conflicting demands, possibly introducing another type of bottleneck and again limiting scalability. Moreover, every device attached to the shared media increases the parasitic capacitance of the electrical conductors, thus increasing the time of flight propagation delay accordingly and, possibly, clock cycle time. In addition, it is more difficult to pipeline packet transmission over the network as the shared media are continuously granted to different requesting devices.

The main advantage of switched-media networks is that the amount of network resources implemented scales with the number of connected devices, increasing the aggregate network bandwidth. These networks allow multiple pairs of nodes to communicate simultaneously, allowing much higher effective network bandwidth than that provided by shared-media networks. Also, switched-media networks allow the system to scale to very large numbers of nodes, which is not feasible when using shared media. Consequently, this scaling advantage can, at the same time, be a disadvantage if network resources grow superlinearly. Networks of superlinear cost that provide an effective network bandwidth that grows only sublinearly with the number of interconnected devices are inefficient designs for many applications and interconnection network domains.

Characterizing Performance: Latency and Effective Bandwidth

The routing, switching, and arbitration functionality described above introduces some additional components of packet transport latency that must be taken into

account in the expression for total packet latency. Assuming there is no contention for network resources—as would be the case in an unloaded network—total packet latency is given by the following:

$$\text{Latency} = \text{Sending overhead} + \left(T_{\text{TotalProp}} + T_{\text{R}} + T_{\text{A}} + T_{\text{S}}\right) + \frac{\text{Packet size}}{\text{Bandwidth}} + \text{Receiving overhead}$$

Here T_{R}, T_{A}, and T_{S} are the total routing time, arbitration time, and switching time experienced by the packet, respectively, and are either measured quantities or calculated quantities derived from more detailed analyses. These components are added to the total propagation delay through the network links, $T_{\text{TotalProp}}$, to give the overall time of flight of the packet.

The expression above gives only a lower bound for the total packet latency as it does not account for additional delays due to contention for resources that may occur. When the network is heavily loaded, several packets may request the same network resources concurrently, thus causing contention that degrades performance. Packets that lose arbitration have to be buffered, which increases packet latency by some *contention delay* amount of waiting time. This additional delay is not included in the above expression. When the network or part of it approaches saturation, contention delay may be several orders of magnitude greater than the total packet latency suffered by a packet under zero load or even under slightly loaded network conditions. Unfortunately, it is not easy to compute analytically the total packet latency when the network is more than moderately loaded. Measurement of these quantities using cycle-accurate simulation of a detailed network model is a better and more precise way of estimating packet latency under such circumstances. Nevertheless, the expression given above is useful in calculating *best-case lower bounds* for packet latency.

For similar reasons, effective bandwidth is not easy to compute exactly, but we can estimate *best-case upper bounds* for it by appropriately extending the model presented at the end of the previous section. What we need to do is to find the narrowest section of the end-to-end network pipe by finding the network injection bandwidth ($\text{BW}_{\text{NetworkInjection}}$), the network reception bandwidth ($\text{BW}_{\text{NetworkReception}}$), and the network bandwidth ($\text{BW}_{\text{Network}}$) across the entire network interconnecting the devices.

The $\text{BW}_{\text{NetworkInjection}}$ can be calculated simply by multiplying the expression for link injection bandwidth, $\text{BW}_{\text{LinkInjection}}$, by the total number of network injection links. The $\text{BW}_{\text{NetworkReception}}$ is calculated similarly using $\text{BW}_{\text{LinkReception}}$, but it must also be scaled by a factor that reflects application traffic and other characteristics. For more than two interconnected devices, it is no longer valid to assume a one-to-one relationship among sources and destinations when analyzing the effect of flow control on link reception bandwidth. It could happen, for example, that several packets from different injection links arrive concurrently at the same reception link for applications that have many-to-one traffic characteristics, which causes contention at the reception links. This effect can be taken into account by an *average reception factor* parameter, σ, which is either a measured quantity or a calculated quantity derived from detailed analysis. It is defined as the average

fraction or percentage of packets arriving at reception links that can be accepted. Only those packets can be immediately delivered, thus reducing network reception bandwidth by that factor. This reduction occurs as a result of application behavior regardless of internal network characteristics. Finally, $BW_{Network}$ takes into account the internal characteristics of the network, including contention. We will progressively derive expressions in the following sections that will enable us to calculate this as more details are revealed about the internals of our black box interconnection network.

Overall, the effective bandwidth delivered by the network end-to-end to an application is determined by the minimum across the three sections, as described by the following:

$$\begin{aligned}\text{Effective bandwidth} &= \min\left(BW_{NetworkInjection}, BW_{Network}, \sigma \times BW_{NetworkReception}\right)\\ &= \min\left(N \times BW_{LinkInjection}, BW_{Network}, \sigma \times N \times BW_{LinkReception}\right)\end{aligned}$$

Let's use the above expressions to compare the latency and effective bandwidth of shared-media networks against switched-media networks for the four interconnection network domains: OCNs, SANs, LANs, and WANs.

Example Plot the total packet latency and effective bandwidth as the number of interconnected nodes, N, scales from 4 to 1024 for shared-media and switched-media OCNs, SANs, LANs, and WANs. Assume that all network links, including the injection and reception links at the nodes, each have a data bandwidth of 8 Gbps, and unicast packets of 100 bytes are transmitted. Shared-media networks share one link, and switched-media networks have at least as many network links as there are nodes. For both, ignore latency and bandwidth effects due to contention within the network. End nodes have per-packet sending and receiving overheads of $x + 0.05$ ns/byte and $4/3(x) + 0.05$ ns/byte, respectively, where x is 0 µs for the OCN, 0.3 µs for the SAN, 3 µs for the LAN, and 30 µs for the WAN, and interconnection distances are 0.5 cm, 5 m, 5000 m, and 5000 km, respectively. Also assume that the total routing, arbitration, and switching times are constants or functions of the number of interconnected nodes: $T_R = 2.5$ ns, $T_A = 2.5(N)$ ns, and $T_S = 2.5$ ns for shared-media networks and $T_R = T_A = T_S = 2.5(\log_2 N)$ ns for switched-media networks. Finally, taking into account application traffic characteristics for the network structure, the average reception factor, σ, is assumed to be N^{-1} for shared media and polylogarithmic $(\log_2 N)^{-1/4}$ for switched media.

Answer All components of total packet latency are the same as in the example given in the previous section except for time of flight, which now has additional routing, arbitration, and switching delays. For shared-media networks, the additional delays total $5 + 2.5(N)$ ns; for switched-media networks, they total $7.5(\log_2 N)$ ns. Latency is plotted only for OCNs and SANs in Figure F.9 as these networks give the more interesting results. For OCNs, T_R, T_A, and T_S combine to dominate time of flight

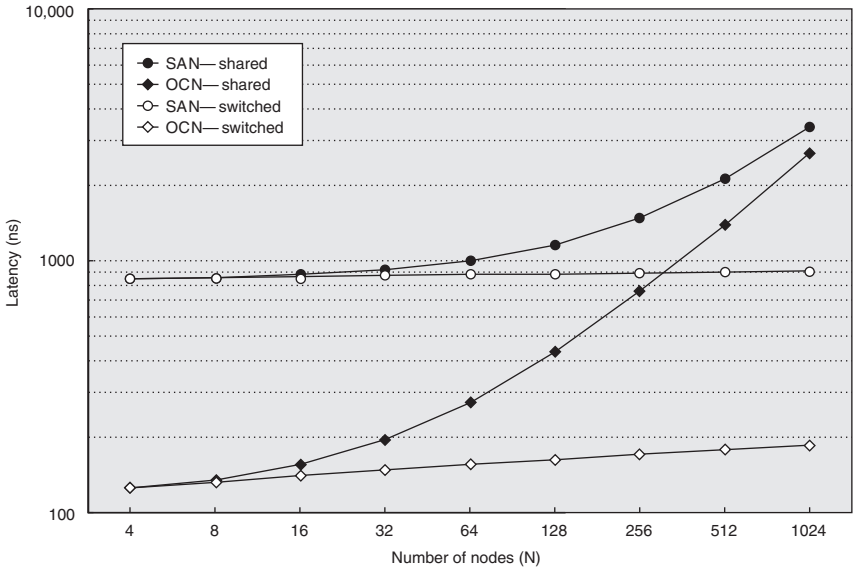


Figure F.9 Latency versus number of interconnected nodes plotted in semi-log form for OCNs and SANs. Routing, arbitration, and switching have more of an impact on latency for networks in these two domains, particularly for networks with a large number of nodes, given the low sending and receiving overheads and low propagation delay.

and are much greater than each of the other latency components for a moderate to large number of nodes. This is particularly so for the shared-media network. The latency increases much more dramatically with the number of nodes for shared media as compared to switched media given the difference in arbitration delay between the two. For SANs, T_R, T_A, and T_S dominate time of flight for most network sizes but are greater than each of the other latency components in shared-media networks only for large-sized networks; they are less than the other latency components for switched-media networks but are not negligible. For LANs and WANs, time of flight is dominated by propagation delay, which dominates other latency components as calculated in the previous section; thus, T_R, T_A, and T_S are negligible for both shared and switched media.

Figure F.10 plots effective bandwidth versus number of interconnected nodes for the four network domains. The effective bandwidth for all shared-media networks is constant through network scaling as only one unicast packet can be received at a time over all the network reception links, and that is further limited by the receiving overhead of each network for all but the OCN. The effective bandwidth for all switched-media networks increases with the number of interconnected nodes, but it is scaled down by the average reception factor. The receiving overhead further limits effective bandwidth for all but the OCN.

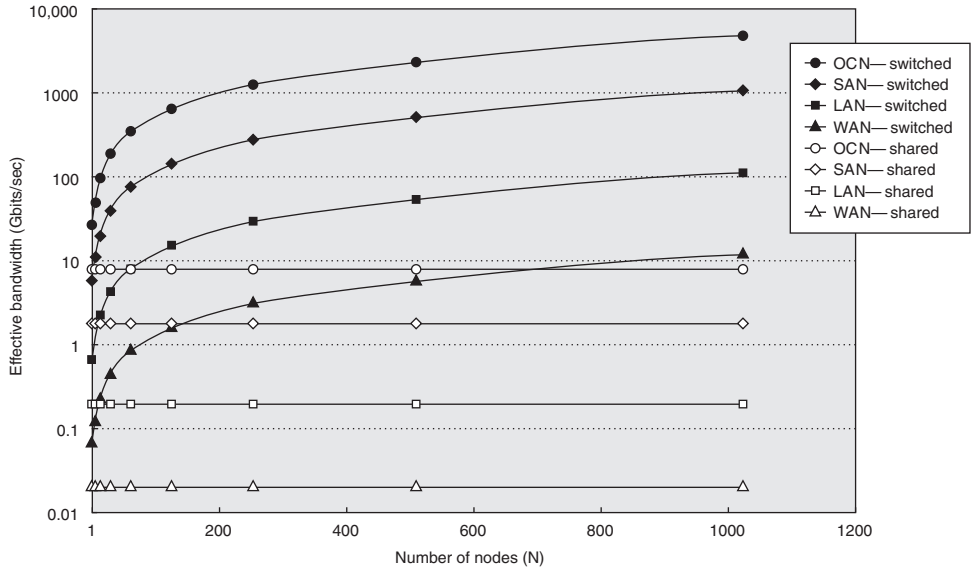


Figure F.10 Effective bandwidth versus number of interconnected nodes plotted in semi-log form for the four network domains. The disparity in effective bandwidth between shared- and switched-media networks for all interconnect domains widens significantly as the number of nodes in the network increases. Only the switched on-chip network is able to achieve an effective bandwidth equal to the aggregate bandwidth for the parameters given in this example.

Given the obvious advantages, why weren't switched networks always used? Earlier computers were much slower and could share the network media with little impact on performance. In addition, the switches for earlier LANs and WANs took up several large boards and were about as large as an entire computer. As a consequence of Moore's law, the size of switches has reduced considerably, and systems have a much greater need for high-performance communication. Switched networks allow communication to harvest the same rapid advancements from silicon as processors and main memory. Whereas switches from telecommunication companies were once the size of mainframe computers, today we see single-chip switches and even entire switched networks within a chip. Thus, technology and application trends favor switched networks today. Just as single-chip processors led to processors replacing logic circuits in a surprising number of places, single-chip switches and switched on-chip networks are increasingly replacing shared-media networks (i.e., buses) in several application domains. As an example, PCI-Express (PCIe)—a switched network—was introduced in 2005 to replace the traditional PCI-X bus on personal computer motherboards.

The previous example also highlights the importance of optimizing the routing, arbitration, and switching functions in OCNs and SANs. For these network domains in particular, the interconnect distances and overheads typically are small

enough to make latency and effective bandwidth much more sensitive to how well these functions are implemented, particularly for larger-sized networks. This leads mostly to implementations based mainly on the faster hardware solutions for these domains. In LANs and WANs, implementations based on the slower but more flexible software solutions suffice given that performance is largely determined by other factors. The design of the topology for switched-media networks also plays a major role in determining how close to the lower bound on latency and the upper bound on effective bandwidth the network can achieve for OCN and SAN domains.

The next three sections touch on these important issues in switched networks, with the next section focused on topology.

F.4 Network Topology

When the number of devices is small enough, a single switch is sufficient to interconnect them within a switched-media network. However, the number of *switch ports* is limited by existing very-large-scale integration (VLSI) technology, cost considerations, power consumption, and so on. When the number of required *network ports* exceeds the number of ports supported by a single switch, a fabric of interconnected switches is needed. To embody the necessary property of *full access* (i.e., *connectedness*), the network switch fabric must provide a path from every end node device to every other device. All the connections to the network fabric and between switches within the fabric use point-to-point links as opposed to shared links—that is, links with only one switch or end node device on either end. The interconnection structure across all the components—including switches, links, and end node devices—is referred to as the network *topology*.

The number of network topologies described in the literature would be difficult to count, but the number that have been used commercially is no more than about a dozen or so. During the 1970s and early 1980s, researchers struggled to propose new topologies that could reduce the number of switches through which packets must traverse, referred to as the *hop count*. In the 1990s, thanks to the introduction of pipelined transmission and switching techniques, the hop count became less critical. Nevertheless, today, topology is still important, particularly for OCNs and SANs, as subtle relationships exist between topology and other network design parameters that impact performance, especially when the number of end nodes is very large (e.g., 64 K in the Blue Gene/L supercomputer) or when the latency is critical (e.g., in multicore processor chips). Topology also greatly impacts the implementation cost of the network.

Topologies for parallel supercomputer SANs have been the most visible and imaginative, usually converging on regularly structured ones to simplify routing, packaging, and scalability. Those for LANs and WANs tend to be more haphazard or ad hoc, having more to do with the challenges of long distance or connecting across different communication subnets. Switch-based topologies for OCNs are only recently emerging but are quickly gaining in popularity. This section

describes the more popular topologies used in commercial products. Their advantages, disadvantages, and constraints are also briefly discussed.

Centralized Switched Networks

As mentioned above, a single switch suffices to interconnect a set of devices when the number of switch ports is equal to or larger than the number of devices. This simple network is usually referred to as a *crossbar* or *crossbar switch*. Within the crossbar, crosspoint switch complexity increases quadratically with the number of ports, as illustrated in Figure F.11(a). Thus, a cheaper solution is desirable when the number of devices to be interconnected scales beyond the point supportable by implementation technology.

A common way of addressing the crossbar scaling problem consists of splitting the large crossbar switch into several stages of smaller switches interconnected in such a way that a single pass through the switch fabric allows any destination to be reached from any source. Topologies arranged in this way are usually referred to as *multistage interconnection networks* or *multistage switch fabrics*, and these networks typically have complexity that increases in proportion to $N \log N$. Multistage interconnection networks (MINs) were initially proposed for telephone exchanges in the 1950s and have since been used to build the communication backbone for parallel supercomputers, symmetric multiprocessors, multicomputer clusters, and IP router switch fabrics.

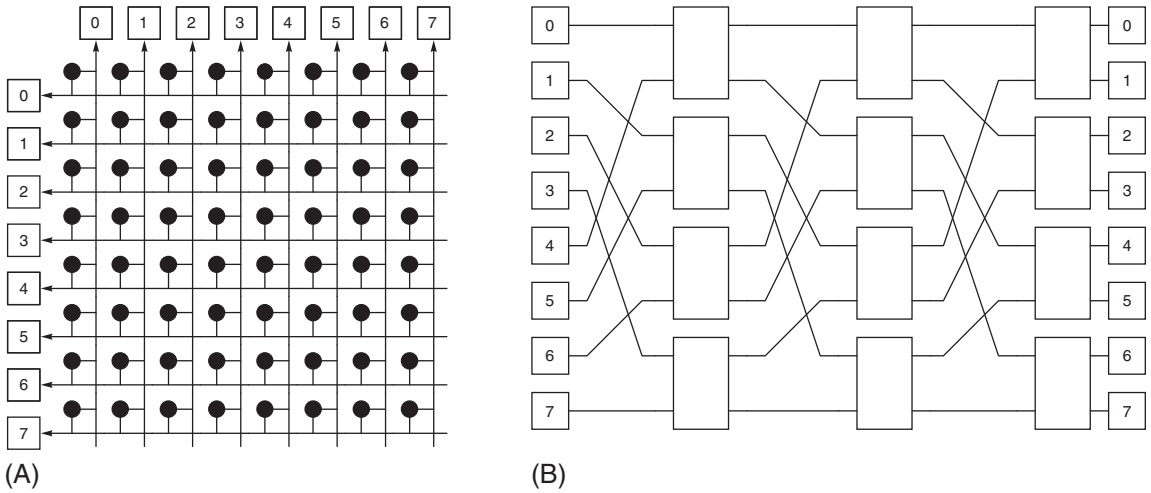


Figure F.11 Popular centralized switched networks: (a) the crossbar network requires N^2 crosspoint switches, shown as black dots; (b) the Omega, a MIN, requires $N/2 \log_2 N$ switches, shown as vertical rectangles. End node devices are shown as numbered squares (total of eight). Links are unidirectional—data enter at the left and exit out the top or right.

The interconnection pattern or patterns between MIN stages are permutations that can be represented mathematically by a set of functions, one for each stage. Figure F.11(b) shows a well-known MIN topology, the *Omega*, which uses the perfect-shuffle permutation as its interconnection pattern for each stage, followed by exchange switches, giving rise to a *perfect-shuffle exchange* for each stage. In this example, eight input-output ports are interconnected with three stages of 2×2 switches. It is easy to see that a single pass through the three stages allows any input port to reach any output port. In general, when using $k \times k$ switches, a MIN with N input-output ports requires at least $\log_k N$ stages, each of which contains N/k switches, for a total of N/k ($\log_k N$) switches.

Despite their internal structure, MINs can be seen as centralized switch fabrics that have end node devices connected at the network periphery, hence the name *centralized switched network*. From another perspective, MINs can be viewed as interconnecting nodes through a set of switches that may not have any nodes directly connected to them, which gives rise to another popular name for centralized switched networks—*indirect networks*.

Example Compute the cost of interconnecting 4096 nodes using a single crossbar switch relative to doing so using a MIN built from 2×2, 4×4, and 16×16 switches. Consider separately the relative cost of the unidirectional links and the relative cost of the switches. Switch cost is assumed to grow quadratically with the number of input (alternatively, output) ports, k, for $k \times k$ switches.

Answer The switch cost of the network when using a single crossbar is proportional to 4096^2. The unidirectional link cost is 8192, which accounts for the set of links from the end nodes to the crossbar and also from the crossbar back to the end nodes. When using a MIN with $k \times k$ switches, the cost of each switch is proportional to k^2 but there are $4096/k$ ($\log_k 4096$) total switches. Likewise, there are ($\log_k 4096$) stages of N unidirectional links per stage from the switches plus N links to the MIN from the end nodes. Therefore, the relative costs of the crossbar with respect to each MIN is given by the following:

$$\text{Relative cost } (2 \times 2)_{switches} = 4096^2 / \left(2^2 \times 4096/2 \times \log_2 4096\right) = 170$$

$$\text{Relative cost } (4 \times 4)_{switches} = 4096^2 / \left(4^2 \times 4096/4 \times \log_4 4096\right) = 170$$

$$\text{Relative cost } (16 \times 16)_{switches} = 4096^2 / \left(16^2 \times 4096/16 \times \log_{16} 4096\right) = 85$$

$$\text{Relative cost } (2 \times 2)_{links} = 8192/(4096 \times (\log_2 4096 + 1)) = 2/13 = 0.1538$$

$$\text{Relative cost } (4 \times 4)_{links} = 8192/(4096 \times (\log_4 4096 + 1)) = 2/7 = 0.2857$$

$$\text{Relative cost } (16 \times 16)_{links} = 8192/(4096 \times (\log_{16} 4096 + 1)) = 2/4 = 0.5$$

In all cases, the single crossbar has much higher switch cost than the MINs. The most dramatic reduction in cost comes from the MIN composed from the smallest sized but largest number of switches, but it is interesting to see that the MINs with 2×2 and 4×4 switches yield the same relative switch cost. The relative link cost

of the crossbar is lower than the MINs, but by less than an order of magnitude in all cases. We must keep in mind that end node links are different from switch links in their length and packaging requirements, so they usually have different associated costs. Despite the lower link cost, the crossbar has higher overall relative cost.

The reduction in switch cost of MINs comes at the price of performance: contention is more likely to occur on network links, thus degrading performance. Contention in the form of packets *blocking* in the network arises due to paths from different sources to different destinations simultaneously sharing one or more links. The amount of contention in the network depends on communication traffic behavior. In the Omega network shown in Figure F.11(b), for example, a packet from port 0 to port 1 blocks in the first stage of switches while waiting for a packet from port 4 to port 0. In the crossbar, no such blocking occurs as links are not shared among paths to unique destinations. The crossbar, therefore, is *nonblocking*. Of course, if two nodes try to send packets to the same destination, there will be blocking at the reception link even for crossbar networks. This is accounted for by the average reception factor parameter (σ) when analyzing performance, as discussed at the end of the previous section.

To reduce blocking in MINs, extra switches must be added or larger ones need to be used to provide alternative paths from every source to every destination. The first commonly used solution is to add a minimum of $\log_k N - 1$ extra switch stages to the MIN in such a way that they mirror the original topology. The resulting network is *rearrangeably nonblocking* as it allows nonconflicting paths among new source-destination pairs to be established, but it also doubles the hop count and could require the paths of some existing communicating pairs to be rearranged under some centralized control. The second solution takes a different approach. Instead of using more switch stages, larger switches—which can be implemented by multiple stages if desired—are used in the middle of two other switch stages in such a way that enough alternative paths through the middle-stage switches allow for nonconflicting paths to be established between the first and last stages. The best-known example of this is the Clos network, which is nonblocking. The multipath property of the three-stage Clos topology can be recursively applied to the middle-stage switches to reduce the size of all the switches down to 2×2, assuming that switches of this size are used in the first and last stages to begin with. What results is a Beneŝ topology consisting of $2(\log_2 N) - 1$ stages, which is rearrangeably nonblocking. Figure F.12(a) illustrates both topologies, where all switches not in the first and last stages comprise the middle-stage switches (recursively) of the Clos network.

The MINs described so far have unidirectional network links, but bidirectional forms are easily derived from symmetric networks such as the Clos and Beneŝ simply by folding them. The overlapping unidirectional links run in different directions, thus forming bidirectional links, and the overlapping switches merge into a single switch with twice the ports (i.e., 4×4 switch). Figure F.12(b) shows the resulting folded Beneŝ topology but in this case with the end nodes connected

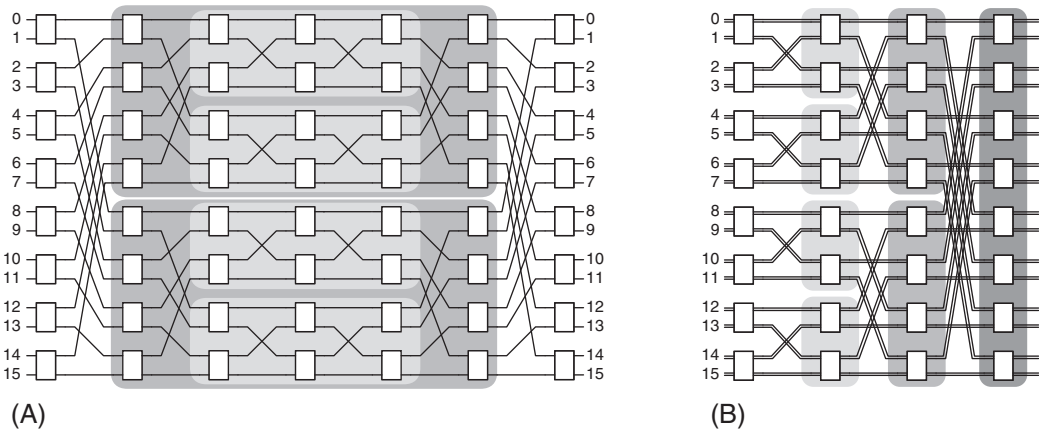


Figure F.12 Two Beneŝ networks. (a) A 16-port Clos topology, where the middle-stage switches shown in the darker shading are implemented with another Clos network whose middle-stage switches shown in the lighter shading are implemented with yet another Clos network, and so on, until a Beneŝ network is produced that uses only 2×2 switches everywhere. (b) A folded Beneŝ network (bidirectional) in which 4×4 switches are used; end nodes attach to the innermost set of the Beneŝ network (unidirectional) switches. This topology is equivalent to a fat tree, where tree vertices are shown in shades.

to the innermost switch stage of the original Beneŝ. Ports remain free at the other side of the network but can be used for later expansion of the network to larger sizes. These kind of networks are referred to as *bidirectional multistage interconnection networks.* Among many useful properties of these networks are their modularity and their ability to exploit communication locality, which saves packets from having to hop across all network stages. Their regularity also reduces routing complexity and their multipath property enables traffic to be routed more evenly across network resources and to tolerate faults.

Another way of deriving bidirectional MINs with nonblocking (rearrangeable) properties is to form a balanced tree, where end node devices occupy leaves of the tree and switches occupy vertices within the tree. Enough links in each tree level must be provided such that the total link bandwidth remains constant across all levels. Also, except for the root, switch ports for each vertex typically grow as $k^i \times k^i$, where i is the tree level. This can be accomplished by using k^{i-1} total switches at each vertex, where each switch has k input and k output ports, or k bidirectional ports (i.e., $k \times k$ input-output ports). Networks having such topologies are called *fat tree* networks. As only half of the k bidirectional ports are used in each direction, $2\ N/k$ switches are needed in each stage, totaling $2\ N/k\ (\log_{k/2} N)$ switches in the fat tree. The number of switches in the root stage can be halved as no forward links are needed, reducing switch count by N/k. Figure F.12(b) shows a fat tree for 4×4 switches. As can be seen, this is identical to the folded Beneŝ.

The fat tree is the topology of choice across a wide range of network sizes for most commercial systems that use multistage interconnection networks. Most SANs used in multicomputer clusters, and many used in the most powerful

supercomputers, are based on fat trees. Commercial communication subsystems offered by Myrinet, Mellanox, and Quadrics are also built from fat trees.

Distributed Switched Networks

Switched-media networks provide a very flexible framework to design communication subsystems external to the devices that need to communicate, as presented above. However, there are cases where it is convenient to more tightly integrate the end node devices with the network resources used to enable them to communicate. Instead of centralizing the switch fabric in an external subsystem, an alternative approach is to distribute the network switches among the end nodes, which then become *network nodes* or simply *nodes*, yielding a *distributed switched network*. As a consequence, each network switch has one or more end node devices directly connected to it, thus forming a network node. These nodes are directly connected to other nodes without indirectly going through some external switch, giving rise to another popular name for these networks—*direct networks*.

The topology for distributed switched networks takes on a form much different from centralized switched networks in that end nodes are connected across the area of the switch fabric, not just at one or two of the peripheral edges of the fabric. This causes the number of switches in the system to be equal to the total number of nodes. A quite obvious way of interconnecting nodes consists of connecting a dedicated link between each node and every other node in the network. This *fully connected* topology provides the best connectivity (full connectivity in fact), but it is more costly than a crossbar network, as the following example shows.

Example Compute the cost of interconnecting N nodes using a fully connected topology relative to doing so using a crossbar topology. Consider separately the relative cost of the unidirectional links and the relative cost of the switches. Switch cost is assumed to grow quadratically with the number of unidirectional ports for $k \times k$ switches but to grow only linearly with $1 \times k$ switches.

Answer The crossbar topology requires an $N \times N$ switch, so the switch cost is proportional to N^2. The link cost is $2N$, which accounts for the unidirectional links from the end nodes to the centralized crossbar, and *vice versa*. In the fully connected topology, two sets of $1 \times (N-1)$ switches (possibly merged into one set) are used in each of the N nodes to connect nodes directly to and from all other nodes. Thus, the total switch cost for all N nodes is proportional to $2N(N-1)$. Regarding link cost, each of the N nodes requires two unidirectional links in opposite directions between its end node device and its local switch. In addition, each of the N nodes has $N-1$ unidirectional links from its local switch to other switches distributed across all the other end nodes. Thus, the total number of unidirectional links is $2N+N(N-1)$, which is equal to $N(N+1)$ for all N nodes. The relative costs of the fully connected topology with respect to the crossbar is, therefore, the following:

$$\text{Relative cost}_{switches} = 2N(N-1)/N^2 = 2(N-1)/N = 2(1-1/N)$$

$$\text{Relative cost}_{links} = N(N+1)/2N = (N+1)/2$$

As the number of interconnected devices increases, the switch cost of the fully connected topology is nearly double the crossbar, with both being very high (i.e., quadratic growth). Moreover, the fully connected topology always has higher relative link cost, which grows linearly with the number of nodes. Again, keep in mind that end node links are different from switch links in their length and packaging, particularly for direct networks, so they usually have different associated costs. Despite its higher cost, the fully connected topology provides no extra performance benefits over the crossbar as both are nonblocking. Thus, crossbar networks are usually used in practice instead of fully connected networks.

A lower-cost alternative to fully connecting all nodes in the network is to directly connect nodes in sequence along a *ring* topology, as shown in Figure F.13. For bidirectional rings, each of the N nodes now uses only 3×3 switches and just two bidirectional network links (shared by neighboring nodes), for a total of N switches and N bidirectional network links. This linear cost excludes the N injection-reception bidirectional links required within nodes.

Unlike shared-media networks, rings can allow many simultaneous transfers: the first node can send to the second while the second sends to the third, and so on. However, as dedicated links do not exist between logically nonadjacent node pairs, packets must hop across intermediate nodes before arriving at their destination, increasing their transport latency. For bidirectional rings, packets can be transported in either direction, with the shortest path to the destination usually being the one selected. In this case, packets must travel $N/4$ network switch hops, on average, with total switch hop count being one more to account for the local switch at the packet source node. Along the way, packets may block on network resources due to other packets contending for the same resources simultaneously.

Fully connected and ring-connected networks delimit the two extremes of distributed switched topologies, but there are many points of interest in between for a given set of cost-performance requirements. Generally speaking, the ideal switched-media topology has cost approaching that of a ring but performance

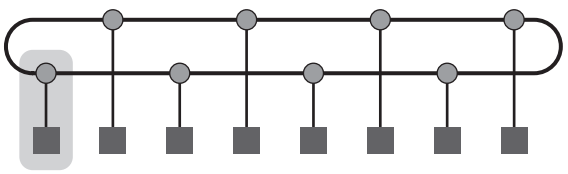


Figure F.13 A ring network topology, folded to reduce the length of the longest link. Shaded circles represent switches, and black squares represent end node devices. The gray rectangle signifies a network node consisting of a switch, a device, and its connecting link.

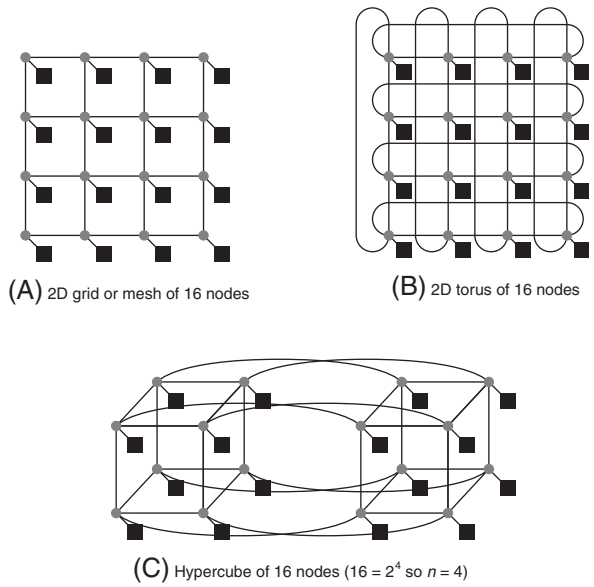


Figure F.14 Direct network topologies that have appeared in commercial systems, mostly supercomputers.
The shaded circles represent switches, and the black squares represent end node devices. Switches have many bidirectional network links, but at least one link goes to the end node device. These basic topologies can be supplemented with extra links to improve performance and reliability. For example, connecting the switches on the periphery of the 2D mesh, shown in (a), using the unused ports on each switch forms a 2D torus, shown in (b). The hypercube topology, shown in (c) is an n-dimensional interconnect for 2^n nodes, requiring $n+1$ ports per switch: one for the n nearest neighbor nodes and one for the end node device.

approaching that of a fully connected topology. Figure F.14 illustrates three popular direct network topologies commonly used in systems spanning the cost-performance spectrum. All of them consist of sets of nodes arranged along multiple dimensions with a regular interconnection pattern among nodes that can be expressed mathematically. In the *mesh* or *grid* topology, all the nodes in each dimension form a linear array. In the *torus* topology, all the nodes in each dimension form a ring. Both of these topologies provide direct communication to neighboring nodes with the aim of reducing the number of hops suffered by packets in the network with respect to the ring. This is achieved by providing greater connectivity through additional dimensions, typically no more than three in commercial systems. The *hypercube* or *n-cube* topology is a particular case of the mesh in which only two nodes are interconnected along each dimension, leading to a number of dimensions, n, that must be large enough to interconnect all N nodes in the system (i.e., $n = \log_2 N$). The hypercube provides better connectivity than meshes

and tori at the expense of higher link and switch costs, in terms of the number of links and number of ports per node.

Example Compute the cost of interconnecting N devices using a torus topology relative to doing so using a fat tree topology. Consider separately the relative cost of the bidirectional links and the relative cost of the switches—which is assumed to grow quadratically with the number of bidirectional ports. Provide an approximate expression for the case of switches being similar in size.

Answer Using $k \times k$ switches, the fat tree requires $2\,N/k\,(\log_{k/2} N)$ switches, assuming the last stage (the root) has the same number of switches as each of the other stages. Given that the number of bidirectional ports in each switch is k (i.e., there are k input ports and k output ports for a $k \times k$ switch) and that the switch cost grows quadratically with this, total network switch cost is proportional to $2kN \log_{k/2} N$. The link cost is $N \log_{k/2} N$ as each of the $\log_{k/2} N$ stages requires N bidirectional links, including those between the devices and the fat tree. The torus requires as many switches as nodes, each of them having $2n+1$ bidirectional ports, including the port to attach the communicating device, where n is the number of dimensions. Hence, total switch cost for the torus is $(2n+1)^2 N$. Each of the torus nodes requires $2n+1$ bidirectional links for the n different dimensions and the connection for its end node device, but as the dimensional links are shared by two nodes, the total number of links is $(2n/2+1)N = (n+1)N$ bidirectional links for all N nodes. Thus, the relative costs of the torus topology with respect to the fat tree are

$$\text{Relative cost}_{switches} = (2n+1)^2 N / 2kN \log_{k/2} N = (2n+1)^2 / 2k \log_{k/2} N$$

$$\text{Relative cost}_{links} = (n+1)N / N \log_{k/2} N = (n+1) / \log_{k/2} N$$

When switch sizes are similar, $2n+1 \cong k$. In this case, the relative cost is

$$\text{Relative cost}_{switches} = (2n+1)^2 / 2k \log_{k/2} N = (2n+1) / 2 \log_{k/2} N = k / 2 \log_{k/2} N$$

When the number of switch ports (also called *switch degree*) is small, tori have lower cost, particularly when the number of dimensions is low. This is an especially useful property when N is large. On the other hand, when larger switches and/or a high number of tori dimensions are used, fat trees are less costly and preferable. For example, when interconnecting 256 nodes, a fat tree is four times more expensive in terms of switch and link costs when 4×4 switches are used. This higher cost is compensated for by lower network contention, on average. The fat tree is comparable in cost to the torus when 8×8 switches are used (e.g., for interconnecting 256 nodes). For larger switch sizes beyond this, the torus costs more than the fat tree as each node includes a switch. This cost can be amortized by connecting multiple end node devices per switch, called *bristling*.

The topologies depicted in Figure F.14 all have in common the interesting characteristic of having their network links arranged in several orthogonal dimensions in a regular way. In fact, these topologies all happen to be particular

instances of a larger class of direct network topologies known as *k*-ary *n*-cubes, where *k* signifies the number of nodes interconnected in each of the *n* dimensions. The symmetry and regularity of these topologies simplify network implementation (i.e, packaging) and packet routing as the movement of a packet along a given network dimension does not modify the number of remaining hops in any other dimension toward its destination. As we will see in the next section, this topological property can be readily exploited by simple routing algorithms.

Like their indirect counterpart, direct networks can introduce blocking among packets that concurrently request the same path, or part of it. The only exception is fully connected networks. The same way that the number of stages and switch hops in indirect networks can be reduced by using larger switches, the hop count in direct networks can likewise be reduced by increasing the number of topological dimensions via increased switch degree.

It may seem to be a good idea always to maximize the number of dimensions for a system of a certain size and switch cost. However, this is not necessarily the case. Most electronic systems are built within our three-dimensional (3D) world using planar (2D) packaging technology such as integrated circuit chips, printed circuit boards, and backplanes. Direct networks with up to three dimensions can be implemented using relatively short links within this 3D space, independent of system size. Links in higher-dimensioned networks would require increasingly longer wires or fiber. This increase in link length with system size is also indicative of MINs, including fat trees, which require either long links within all the stages or increasingly longer links as more stages are added. As we saw in the first example given in Section F.2, flow-controlled buffers increase in size proportionally to link length, thus requiring greater silicon area. This is among the reasons why the supercomputer with the largest number of compute nodes existing in 2005, the IBM Blue Gene/L, implemented a 3D torus network for interprocessor communication. A fat tree would have required much longer links, rendering a 64K node system less feasible. This highlights the importance of correctly selecting the proper network topology that meets system requirements.

Besides link length, other constraints derived from implementing the topology may also limit the degree to which a topology can scale. These are available *pin-out* and achievable *bisection bandwidth*. Pin count is a local restriction on the bandwidth of a chip, printed circuit board, and backplane (or chassis) connector. In a direct network that integrates processor cores and switches on a single chip or multichip module, pin bandwidth is used both for interfacing with main memory and for implementing node links. In this case, limited pin count could reduce the number of switch ports or bit lines per link. In an indirect network, switches are implemented separately from processor cores, allowing most of the pins to be dedicated to communication bandwidth. However, as switches are grouped onto boards, the aggregate of all input-output links of the switch fabric on a board for a given topology must not exceed the board connector pin-outs.

The bisection bandwidth is a more global restriction that gives the interconnect density and bandwidth that can be achieved by a given implementation

(packaging) technology. Interconnect density and clock frequency are related to each other: When wires are packed closer together, crosstalk and parasitic capacitance increase, which usually impose a lower clock frequency. For example, the availability and spacing of metal layers limit wire density and frequency of on-chip networks, and copper track density limits wire density and frequency on a printed circuit board. To be implementable, the topology of a network must not exceed the available bisection bandwidth of the implementation technology. Most networks implemented to date are constrained more so by pin-out limitations rather than bisection bandwidth, particularly with the recent move to blade-based systems. Nevertheless, bisection bandwidth largely affects performance.

For a given topology, bisection bandwidth, $BW_{Bisection}$, is calculated by dividing the network into two roughly equal parts—each with half the nodes—and summing the bandwidth of the links crossing the imaginary dividing line. For nonsymmetric topologies, bisection bandwidth is the smallest of all pairs of equal-sized divisions of the network. For a fully connected network, the bisection bandwidth is proportional to $N^2/2$ unidirectional links (or $N^2/4$ bidirectional links), where N is the number of nodes. For a bus, bisection bandwidth is the bandwidth of just the one shared half-duplex link. For other topologies, values lie in between these two extremes. Network injection and reception bisection bandwidth is commonly used as a reference value, which is $N/2$ for a network with N injection and reception links, respectively. Any network topology that provides this bisection bandwidth is said to have *full bisection bandwidth.*

Figure F.15 summarizes the number of switches and links required, the corresponding switch size, the maximum and average switch hop distances between nodes, and the bisection bandwidth in terms of links for several topologies discussed in this section for interconnecting 64 nodes.

Evaluation category	Bus	Ring	2D mesh	2D torus	Hypercube	Fat tree	Fully connected
Performance							
$BW_{Bisection}$ in # links	1	2	8	16	32	32	1024
Max (ave.) hop count	1 (1)	32 (16)	14 (7)	8 (4)	6 (3)	11 (9)	1 (1)
Cost							
I/O ports per switch	NA	3	5	5	7	4	64
Number of switches	NA	64	64	64	64	192	64
Number of net. links	1	64	112	128	192	320	2016
Total number of links	1	128	176	192	256	384	2080

Figure F.15 Performance and cost of several network topologies for 64 nodes. The bus is the standard reference at unit network link cost and bisection bandwidth. Values are given in terms of bidirectional links and ports. Hop count includes a switch and its output link, but not the injection link at end nodes. Except for the bus, values are given for the number of network links and total number of links, including injection/reception links between end node devices and the network.

Effects of Topology on Network Performance

Switched network topologies require packets to take one or more hops to reach their destination, where each hop represents the transport of a packet through a switch and one of its corresponding links. Interestingly, each switch and its corresponding links can be modeled as a black box network connecting more than two devices, as was described in the previous section, where the term "devices" here refers to end nodes or other switches. The only differences are that the sending and receiving overheads are null through the switches, and the routing, switching, and arbitration delays are not cumulative but, instead, are delays associated with each switch.

As a consequence of the above, if the average packet has to traverse d hops to its destination, then $T_R + T_A + T_S = (T_r + T_a + T_s) \times d$, where T_r, T_a, and T_s are the routing, arbitration, and switching delays, respectively, of a switch. With the assumption that pipelining over the network is staged on each hop at the packet level (this assumption will be challenged in the next section), the transmission delay is also increased by a factor of the number of hops. Finally, with the simplifying assumption that all injection links to the first switch or stage of switches and all links (including reception links) from the switches have approximately the same length and delay, the total propagation delay through the network $T_{TotalProp}$ is the propagation delay through a single link, $T_{LinkProp}$, multiplied by $d + 1$, which is the hop count plus one to account for the injection link. Thus, the best-case lower-bound expression for average packet latency in the network (i.e., the latency in the absence of contention) is given by the following expression:

$$\text{Latency} = \text{Sending overhead} + T_{LinkProp} \times (d+1) + (T_r + T_a + T_s) \times d + \frac{\text{Packet size}}{\text{Bandwidth}} \times (d+1) + \text{Receiving overhead}$$

Again, the expression on page F-40 assumes that switches are able to pipeline packet transmission at the packet level.

Following the method presented previously, we can estimate the best-case upper bound for effective bandwidth by finding the narrowest section of the end-to-end network pipe. Focusing on the internal network portion of that pipe, network bandwidth is determined by the blocking properties of the topology. Non-blocking behavior can be achieved only by providing many alternative paths between every source-destination pair, leading to an aggregate network bandwidth that is many times higher than the aggregate network injection or reception bandwidth. This is quite costly. As this solution usually is prohibitively expensive, most networks have different degrees of blocking, which reduces the utilization of the aggregate bandwidth provided by the topology. This, too, is costly but not in terms of performance.

The amount of blocking in a network depends on its topology and the traffic distribution. Assuming the bisection bandwidth, $BW_{Bisection}$, of a topology is implementable (as typically is the case), it can be used as a constant measure of the maximum degree of blocking in a network. In the ideal case, the network always achieves full bisection bandwidth irrespective of the traffic behavior, thus

transferring the bottlenecking point to the injection or reception links. However, as packets destined to locations in the other half of the network necessarily must cross the bisection links, those links pose as potential bottleneck links—potentially reducing the network bandwidth to below full bisection bandwidth. Fortunately, not all of the traffic must cross the network bisection, allowing more of the aggregate network bandwidth provided by the topology to be utilized. Also, network topologies with a higher number of bisection links tend to have less blocking as more alternative paths are possible to reach destinations and, hence, a higher percentage of the aggregate network bandwidth can be utilized. If only a fraction of the traffic must cross the network bisection, as captured by a *bisection traffic fraction* parameter γ ($0 < \gamma \leq 1$), the network pipe at the bisection is, effectively, widened by the reciprocal of that fraction, assuming a traffic distribution that loads the bisection links at least as heavily, on average, as other network links. This defines the upper limit on achievable network bandwidth, $BW_{Network}$:

$$BW_{Network} = \frac{BW_{Bisection}}{\gamma}$$

Accordingly, the expression for effective bandwidth becomes the following when network topology is taken into consideration:

$$\text{Effective bandwidth} = \min\left(N \times BW_{LinkInjection}, \frac{BW_{Bisection}}{\gamma}, \sigma \times N \times BW_{LinkReception}\right)$$

It is important to note that γ depends heavily on the traffic patterns generated by applications. It is a measured quantity or calculated from detailed traffic analysis.

Example A common communication pattern in scientific programs is to have nearest neighbor elements of a two-dimensional array to communicate in a given direction. This pattern is sometimes called *NEWS communication*, standing for north, east, west, and south—the directions on a compass. Map an 8×8 array of elements one-to-one onto 64 end node devices interconnected in the following topologies: bus, ring, 2D mesh, 2D torus, hypercube, fully connected, and fat tree. How long does it take in the best case for each node to send one message to its northern neighbor and one to its eastern neighbor, assuming packets are allowed to use any minimal path provided by the topology? What is the corresponding effective bandwidth? Ignore elements that have no northern or eastern neighbors. To simplify the analysis, assume that all networks experience unit packet transport time for each network hop—that is, $T_{LinkProp}$, T_r, T_a, T_s, and packet transmission time for each hop sum to one. Also assume the delay through injection links is included in this unit time, and sending/receiving overhead is null.

Answer This communication pattern requires us to send $2 \times (64 - 8)$ or 112 total packets—that is, 56 packets in each of the two communication phases: northward and eastward. The number of hops suffered by packets depends on the topology. Communication between sources and destinations are one-to-one, so σ is 100%.

The injection and reception bandwidth cap the effective bandwidth to a maximum of 64 BW units (even though the communication pattern requires only 56 BW units). However, this maximum may get scaled down by the achievable network bandwidth, which is determined by the bisection bandwidth and the fraction of traffic crossing it, γ, both of which are topology dependent. Here are the various cases:

- *Bus*—The mapping of the 8×8 array elements to nodes makes no difference for the bus as all nodes are equally distant at one hop away. However, the 112 transfers are done sequentially, taking a total of 112 time units. The bisection bandwidth is 1, and γ is 100%. Thus, effective bandwidth is only 1 BW unit.
- *Ring*—Assume the first row of the array is mapped to nodes 0 to 7, the second row to nodes 8 to 15, and so on. It takes just one time unit for all nodes simultaneously to send to their eastern neighbor (i.e., a transfer from node i to node $i+1$). With this mapping, the northern neighbor for each node is exactly eight hops away so it takes eight time units, which also is done in parallel for all nodes. Total communication time is, therefore, 9 time units. The bisection bandwidth is 2 bidirectional links (assuming a bidirectional ring), which is less than the full bisection bandwidth of 32 bidirectional links. For eastward communication, because only 2 of the eastward 56 packets must cross the bisection in the worst case, the bisection links do not pose as bottlenecks. For northward communication, 8 of the 56 packets must cross the two bisection links, yielding a γ of $10/112 = 8.93\%$. Thus, the network bandwidth is $2/.0893 = 22.4$ BW units. This limits the effective bandwidth at 22.4 BW units as well, which is less than half the bandwidth required by the communication pattern.
- *2D mesh*—There are eight rows and eight columns in our grid of 64 nodes, which is a perfect match to the NEWS communication. It takes a total of just 2 time units for all nodes to send simultaneously to their northern neighbors followed by simultaneous communication to their eastern neighbors. The bisection bandwidth is 8 bidirectional links, which is less than full bisection bandwidth. However, the perfect matching of this nearest neighbor communication pattern on this topology allows the maximum effective bandwidth to be achieved regardless. For eastward communication, 8 of the 56 packets must cross the bisection in the worst case, which does not exceed the bisection bandwidth. None of the northward communications crosses the same network bisection, yielding a γ of $8/112 = 7.14\%$ and a network bandwidth of $8/0.0714 = 112$ BW units. The effective bandwidth is, therefore, limited by the communication pattern at 56 BW units as opposed to the mesh network.
- *2D torus*—Wrap-around links of the torus are not used for this communication pattern, so the torus has the same mapping and performance as the mesh.

- *Hypercube*—Assume elements in each row are mapped to the same location within the eight 3-cubes comprising the hypercube such that consecutive row elements are mapped to nodes only one hop away. Northern neighbors can be similarly mapped to nodes only one hop away in an orthogonal dimension. Thus, the communication pattern takes just 2 time units. The hypercube provides full bisection bandwidth of 32 links, but at most only 8 of the 112 packets must cross the bisection. Thus, effective bandwidth is limited only by the communication pattern to be 56 BW units, not by the hypercube network.
- *Fully connected*—Here, nodes are equally distant at one hop away, regardless of the mapping. Parallel transfer of packets in both the northern and eastern directions would take only 1 time unit if the injection and reception links could source and sink two packets at a time. As this is not the case, 2 time units are required. Effective bandwidth is limited by the communication pattern at 56 BW units, so the 1024 network bisection links largely go underutilized.
- *Fat tree*—Assume the same mapping of elements to nodes as is done for the ring and the use of switches with eight bidirectional ports. This allows simultaneous communication to eastern neighbors that takes at most three hops and, therefore, 3 time units through the three bidirectional stages interconnecting the eight nodes in each of the eight groups of nodes. The northern neighbor for each node resides in the adjacent group of eight nodes, which requires five hops, or 5 time units. Thus, the total time required on the fat tree is 8 time units. The fat tree provides full bisection bandwidth, so in the worst case of half the traffic needing to cross the bisection, an effective bandwidth of 56 BW units (as limited by the communication pattern and not by the fattree network) is achieved when packets are continually injected.

The above example should not lead one to the wrong conclusion that meshes are just as good as tori, hypercubes, fat trees, and other networks with higher bisection bandwidth. A number of simplifications that benefit low-bisection networks were assumed to ease the analysis. In practice, packets typically are larger than the link width and occupy links for many more than just one network cycle. Also, many communication patterns do not map so cleanly to the 2D mesh network topology; instead, usually they are more global and irregular in nature. These and other factors combine to increase the chances of packets blocking in low-bisection networks, increasing latency and reducing effective bandwidth.

To put this discussion on topologies into further perspective, Figure F.16 lists various attributes of topologies used in commercial high-performance computers.

F.5 Network Routing, Arbitration, and Switching

Routing, arbitration, and switching are performed at every switch along a packet's path in a switched media network, no matter what the network topology. Numerous interesting techniques for accomplishing these network functions have been

Company	System [network] name	Max. number of nodes [× # CPUs]	Basic network topology	Injection [reception] node BW in MB/sec	# of data bits per link per direction	Raw network link BW per direction in MB/sec	Raw network bisection BW (bidirectional) in GB/sec
Intel	ASCI Red Paragon	4816 [×2]	2D mesh 64×64	400 [400]	16 bits	400	51.2
IBM	ASCI White SP Power3 [Colony]	512 [×16]	Bidirectional MIN with 8-port bidirectional switches (typically a fat tree or Omega)	500 [500]	8 bits (+1 bit of control)	500	256
Intel	Thunder Itanium2 Tiger4 [QsNet^{II}]	1024 [×4]	Fat tree with 8-port bidirectional switches	928 [928]	8 bits (+2 of control for 4b/5b encoding)	1333	1365
Cray	XT3 [SeaStar]	30,508 [×1]	3D torus $40 \times 32 \times 24$	3200 [3200]	12 bits	3800	5836.8
Cray	X1E	1024 [×1]	4-way bristled 2D torus ($\sim 23 \times 11$) with express links	1600 [1600]	16 bits	1600	51.2
IBM	ASC Purple pSeries 575 [Federation]	>1280 [×8]	Bidirectional MIN with 8-port bidirectional switches (typically a fat tree or Omega)	2000 [2000]	8 bits (+2 bits of control for novel 5b/6b encoding scheme)	2000	2560
IBM	Blue Gene/L eServer Sol. [Torus Net.]	65,536 [×2]	3D torus $32 \times 32 \times 64$	612.5 [1050]	1 bit (bit serial)	175	358.4

Figure F.16 Topological characteristics of interconnection networks used in commercial high-performance machines.

proposed in the literature. In this section, we focus on describing a representative set of approaches used in commercial systems for the more commonly used network topologies. Their impact on performance is also highlighted.

Routing

The *routing algorithm* defines which network path, or paths, are allowed for each packet. Ideally, the routing algorithm supplies shortest paths to all packets such that

traffic load is evenly distributed across network links to minimize contention. However, some paths provided by the network topology may not be allowed in order to guarantee that all packets can be delivered, no matter what the traffic behavior. Paths that have an unbounded number of allowed nonminimal hops from packet sources, for instance, may result in packets never reaching their destinations. This situation is referred to as *livelock*. Likewise, paths that cause a set of packets to block in the network forever waiting only for network resources (i.e., links or associated buffers) held by other packets in the set also prevent packets from reaching their destinations. This situation is referred to as *deadlock*. As deadlock arises due to the finiteness of network resources, the probability of its occurrence increases with increased network traffic and decreased availability of network resources. For the network to function properly, the routing algorithm must guard against this anomaly, which can occur in various forms—for example, routing deadlock, request-reply (protocol) deadlock, and fault-induced (reconfiguration) deadlock, etc. At the same time, for the network to provide the highest possible performance, the routing algorithm must be efficient—allowing as many routing options to packets as there are paths provided by the topology, in the best case.

The simplest way of guarding against livelock is to restrict routing such that only minimal paths from sources to destinations are allowed or, less restrictively, only a limited number of nonminimal hops. The strictest form has the added benefit of consuming the minimal amount of network bandwidth, but it prevents packets from being able to use alternative nonminimal paths in case of contention or faults along the shortest (minimal) paths.

Deadlock is more difficult to guard against. Two common strategies are used in practice: avoidance and recovery. In *deadlock avoidance*, the routing algorithm restricts the paths allowed by packets to only those that keep the global network state deadlock-free. A common way of doing this consists of establishing an ordering between a set of resources—the minimal set necessary to support network full access—and granting those resources to packets in some total or partial order such that cyclic dependency cannot form on those resources. This allows an escape path always to be supplied to packets no matter where they are in the network to avoid entering a deadlock state. In *deadlock recovery*, resources are granted to packets without regard for avoiding deadlock. Instead, as deadlock is possible, some mechanism is used to detect the likely existence of deadlock. If detected, one or more packets are removed from resources in the deadlock set—possibly by regressively dropping the packets or by progressively redirecting the packets onto special deadlock recovery resources. The freed network resources are then granted to other packets needing them to resolve the deadlock.

Let us consider routing algorithms designed for distributed switched networks. Figure F.17(a) illustrates one of many possible deadlocked configurations for packets within a region of a 2D mesh network. The routing algorithm can avoid all such deadlocks (and livelocks) by allowing only the use of minimal paths that cross the network dimensions in some total order. That is, links of a given dimension are not supplied to a packet by the routing algorithm until no other links are needed by the packet in all of the preceding dimensions for it to reach its

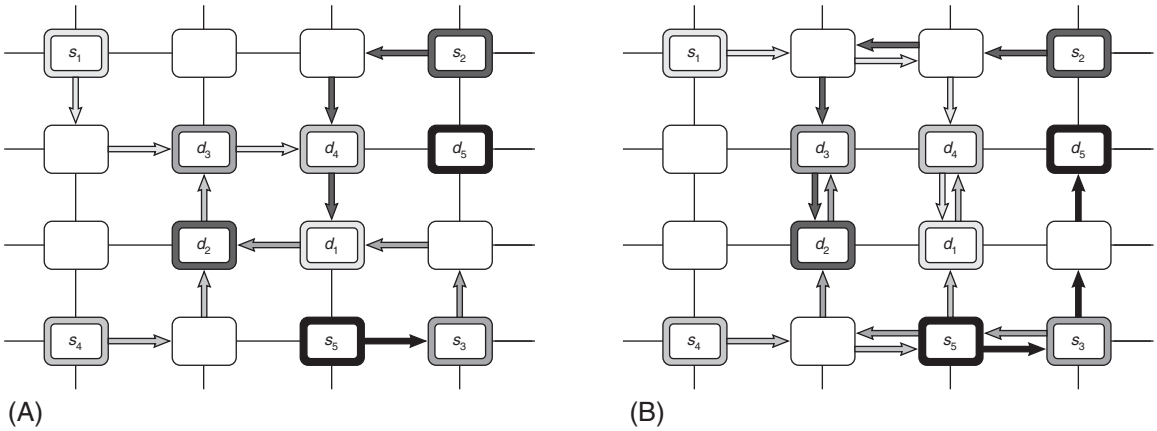


Figure F.17 A mesh network with packets routing from sources, s_i, to destinations, d_i. (a) Deadlock forms from packets destined to d_1 through d_4 blocking on others in the same set that fully occupy their requested buffer resources one hop away from their destinations. This deadlock cycle causes other packets needing those resources also to block, like packets from s_5 destined to d_5 that have reached node s_3. (b) Deadlock is avoided using dimension-order routing. In this case, packets exhaust their routes in the *X* dimension before turning into the *Y* dimension in order to complete their routing.

destination. This is illustrated in Figure F.17(b), where dimensions are crossed in *XY* dimension order. All the packets must follow the same order when traversing dimensions, exiting a dimension only when links are no longer required in that dimension. This well-known algorithm is referred to as *dimension-order routing* (DOR) or *e-cube routing* in hypercubes. It is used in many commercial systems built from distributed switched networks and on-chip networks. As this routing algorithm always supplies the same path for a given source-destination pair, it is a *deterministic routing* algorithm.

Crossing dimensions in order on some minimal set of resources required to support network full access avoids deadlock in meshes and hypercubes. However, for distributed switched topologies that have wrap-around links (e.g., rings and tori), a total ordering on a minimal set of resources within each dimension is also needed if resources are to be used to full capacity. Alternatively, some empty resources or *bubbles* along the dimensions would be required to remain below full capacity and avoid deadlock. To allow full access, either the physical links must be duplicated or the logical buffers associated with each link must be duplicated, resulting in *physical channels* or *virtual channels*, respectively, on which the ordering is done. Ordering is not necessary on all network resources to avoid deadlock—it is needed only on some minimal set required to support network full access (i.e., some *escape resource set*). Routing algorithms based on this technique (called Duato's protocol) can be defined that allow alternative paths provided by the topology to be used for a given source-destination pair in addition to the escape resource set. One of those allowed paths must be selected, preferably the most

efficient one. Adapting the path in response to prevailing network traffic conditions enables the aggregate network bandwidth to be better utilized and contention to be reduced. Such routing capability is referred to as *adaptive routing* and is used in many commercial systems.

Example How many of the possible dimensional turns are eliminated by dimension-order routing on an n-dimensional mesh network? What is the fewest number of turns that actually need to be eliminated while still maintaining connectedness and deadlock freedom? Explain using a 2D mesh network.

Answer The dimension-order routing algorithm eliminates exactly half of the possible dimensional turns as it is easily proven that all turns from any lower-ordered dimension into any higher-ordered dimension are allowed, but the converse is not true. For example, of the eight possible turns in the 2D mesh shown in Figure F.17, the four turns from $X+$ to $Y+$, $X+$ to $Y-$, $X-$ to $Y+$, and $X-$ to $Y-$ are allowed, where the signs ($+$ or $-$) refer to the direction of travel within a dimension. The four turns from $Y+$ to $X+$, $Y+$ to $X-$, $Y-$ to $X+$, and $Y-$ to $X-$ are disallowed turns. The elimination of these turns prevents cycles of any kind from forming—and, thus, avoids deadlock—while keeping the network connected. However, it does so at the expense of not allowing any routing adaptivity.

The *Turn Model* routing algorithm proves that the minimum number of eliminated turns to prevent cycles and maintain connectedness is a quarter of the possible turns, but the right set of turns must be chosen. Only some particular set of eliminated turns allow both requirements to be satisfied. With the elimination of the wrong set of a quarter of the turns, it is possible for combinations of allowed turns to emulate the eliminated ones (and, thus, form cycles and deadlock) or for the network not to be connected. For the 2D mesh, for example, it is possible to eliminate only the two turns ending in the westward direction (i.e., $Y+$ to $X-$ and $Y-$ to $X-$) by requiring packets to start their routes in the westward direction (if needed) to maintain connectedness. Alternatives to this west-first routing for 2D meshes are negative-first routing and north-last routing. For these, the extra quarter of turns beyond that supplied by DOR allows for partial adaptivity in routing, making these adaptive routing algorithms.

Routing algorithms for centralized switched networks can similarly be defined to avoid deadlocks by restricting the use of resources in some total or partial order. For fat trees, resources can be totally ordered along paths starting from the input leaf stage upward to the root and then back down to the output leaf stage. The routing algorithm can allow packets to use resources in increasing partial order, first traversing up the tree until they reach some *least common ancestor* (LCA) of the source and destination, and then back down the tree until they reach their destinations. As there are many least common ancestors for a given destination, multiple alternative paths are allowed while going up the tree, making the routing algorithm adaptive. However, only a single

deterministic path to the destination is provided by the fat tree topology from a least common ancestor. This *self-routing* property is common to many MINs and can be readily exploited: The switch output port at each stage is given simply by shifts of the destination node address.

More generally, a tree graph can be mapped onto any topology—whether direct or indirect—and links between nodes at the same tree level can be allowed by assigning directions to them, where "up" designates paths moving toward the tree root and "down" designates paths moving away from the root node. This allows for generic *up*/down* routing* to be defined on any topology such that packets follow paths (possibly adaptively) consisting of zero or more up links followed by zero or more down links to their destination. Up/down ordering prevents cycles from forming, avoiding deadlock. This routing technique was used in Autonet—a self-configuring switched LAN—and in early Myrinet SANs.

Routing algorithms are implemented in practice by a combination of the routing information placed in the packet header by the source node and the routing control mechanism incorporated in the switches. For *source routing*, the entire routing path is precomputed by the source—possibly by table lookup—and placed in the packet header. This usually consists of the output port or ports supplied for each switch along the predetermined path from the source to the destination, which can be stripped off by the routing control mechanism at each switch. An additional bit field can be included in the header to signify whether adaptive routing is allowed (i.e., that any one of the supplied output ports can be used). For *distributed routing*, the routing information usually consists of the destination address. This is used by the routing control mechanism in each switch along the path to determine the next output port, either by computing it using a finite-state machine or by looking it up in a local routing table (i.e., forwarding table). Compared to distributed routing, source routing simplifies the routing control mechanism within the network switches, but it requires more routing bits in the header of each packet, thus increasing the header overhead.

Arbitration

The *arbitration algorithm* determines when requested network paths are available for packets. Ideally, arbiters maximize the matching of free network resources and packets requesting those resources. At the switch level, arbiters maximize the matching of free output ports and packets located in switch input ports requesting those output ports. When all requests cannot be granted simultaneously, switch arbiters resolve conflicts by granting output ports to packets in a fair way such that *starvation* of requested resources by packets is prevented. This could happen to packets in shorter queues if a serve-longest-queue (SLQ) scheme is used. For packets having the same priority level, simple round-robin (RR) or age-based schemes are sufficiently fair and straightforward to implement.

Arbitration can be distributed to avoid centralized bottlenecks. A straightforward technique consists of two phases: a request phase and a grant phase. Let us assume that each switch input port has an associated queue to hold incoming

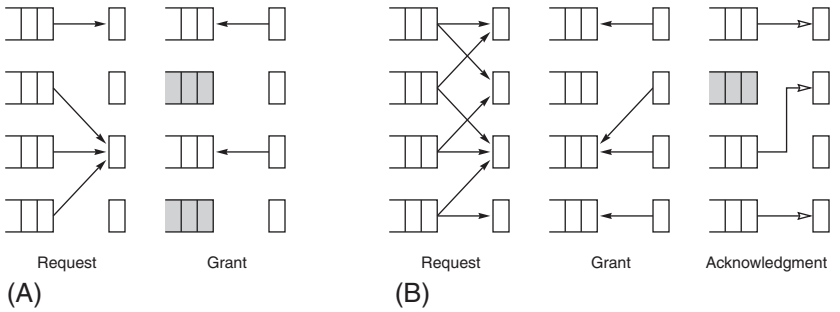


Figure F.18 Two arbitration techniques. (a) Two-phased arbitration in which two of the four input ports are granted requested output ports. (b) Three-phased arbitration in which three of the four input ports are successful in gaining the requested output ports, resulting in higher switch utilization.

packets and that each switch output port has an associated local arbiter implementing a round-robin strategy. Figure F.18(a) shows a possible set of requests for a four-port switch. In the *request phase*, packets at the head of each input port queue send a single request to the arbiters corresponding to the output ports requested by them. Then, each output port arbiter independently arbitrates among the requests it receives, selecting only one. In the *grant phase*, one of the requests to each arbiter is granted the requested output port. When two packets from different input ports request the same output port, only one receives a grant, as shown in the figure. As a consequence, some output port bandwidth remains unused even though all input queues have packets to transmit.

The simple two-phase technique can be improved by allowing several simultaneous requests to be made by each input port, possibly coming from different virtual channels or from multiple adaptive routing options. These requests are sent to different output port arbiters. By submitting more than one request per input port, the probability of matching increases. Now, arbitration requires three phases: request, grant, and acknowledgment. Figure F.18(b) shows the case in which up to two requests can be made by packets at each input port. In the request phase, requests are submitted to output port arbiters, and these arbiters select one of the received requests, as is done for the two-phase arbiter. Likewise, in the grant phase, the selected requests are granted to the corresponding requesters. Taking into account that an input port can submit more than one request, it may receive more than one grant. Thus, it selects among possibly multiple grants using some arbitration strategy such as round-robin. The selected grants are confirmed to the corresponding output port arbiters in the acknowledgment phase.

As can be seen in Figure F.18(b), it could happen that an input port that submits several requests does not receive any grants, while some of the requested ports remain free. Because of this, a second arbitration iteration can improve the probability of matching. In this iteration, only the requests corresponding to non-matched input and output ports are submitted. Iterative arbiters with multiple

requests per input port are able to increase the utilization of switch output ports and, thus, the network link bandwidth. However, this comes at the expense of additional arbiter complexity and increased arbitration delay, which could increase the router clock cycle time if it is on the critical path.

Switching

The *switching technique* defines how connections are established in the network. Ideally, connections between network resources are established or "switched in" only for as long as they are actually needed and exactly at the point that they are ready and needed to be used, considering both time and space. This allows efficient use of available network bandwidth by competing traffic flows and minimal latency. Connections at each hop along the topological path allowed by the routing algorithm and granted by the arbitration algorithm can be established in three basic ways: prior to packet arrival using *circuit switching*, upon receipt of the entire packet using *store-and-forward packet switching*, or upon receipt of only portions of the packet with unit size no smaller than that of the packet header using *cut-through packet switching*.

Circuit switching establishes a circuit *a priori* such that network bandwidth is allocated for packet transmissions along an entire source-destination path. It is possible to pipeline packet transmission across the circuit using staging at each hop along the path, a technique known as *pipelined circuit switching*. As routing, arbitration, and switching are performed only once for one or more packets, routing bits are not needed in the header of packets, thus reducing latency and overhead. This can be very efficient when information is continuously transmitted between devices for the same circuit setup. However, as network bandwidth is removed from the shared pool and preallocated regardless of whether sources are in need of consuming it or not, circuit switching can be very inefficient and highly wasteful of bandwidth.

Packet switching enables network bandwidth to be shared and used more efficiently when packets are transmitted intermittently, which is the more common case. Packet switching comes in two main varieties—store-and-forward and cutthrough switching, both of which allow network link bandwidth to be multiplexed on packet-sized or smaller units of information. This better enables bandwidth sharing by packets originating from different sources. The finer granularity of sharing, however, increases the overhead needed to perform switching: Routing, arbitration, and switching must be performed for every packet, and routing and flow control bits are required for every packet if flow control is used.

Store-and-forward packet switching establishes connections such that a packet is forwarded to the next hop in sequence along its source-destination path only after the entire packet is first stored (staged) at the receiving switch. As packets are completely stored at every switch before being transmitted, links are completely decoupled, allowing full link bandwidth utilization even if links have very different bandwidths. This property is very important in WANs, but the price to pay is packet latency; the total routing, arbitration, and switching delay is multiplicative with the number of hops, as we have seen in Section F.4 when analyzing performance under this assumption.

Cut-through packet switching establishes connections such that a packet can "cut through" switches in a pipelined manner once the header portion of the packet (or equivalent amount of payload trailing the header) is staged at receiving switches. That is, the rest of the packet need not arrive before switching in the granted resources. This allows routing, arbitration, and switching delay to be additive with the number of hops rather than multiplicative to reduce total packet latency. Cut-through comes in two varieties, the main differences being the size of the unit of information on which flow control is applied and, consequently, the buffer requirements at switches. *Virtual cut-through switching* implements flow control at the packet level, whereas *wormhole switching* implements it on flow units, or *flits*, which are smaller than the maximum packet size but usually at least as large as the packet header. Since wormhole switches need to be capable of storing only a small portion of a packet, packets that block in the network may span several switches. This can cause other packets to block on the links they occupy, leading to premature network saturation and reduced effective bandwidth unless some centralized buffer is used within the switch to store them—a technique called *buffered wormhole switching*. As chips can implement relatively large buffers in current technology, virtual cut-through is the more commonly used switching technique. However, wormhole switching may still be preferred in OCNs designed to minimize silicon resources.

Premature network saturation caused by wormhole switching can be mitigated by allowing several packets to share the physical bandwidth of a link simultaneously via time-multiplexed switching at the flit level. This requires physical links to have a set of virtual channels (i.e., the logical buffers mentioned previously) at each end, into which packets are switched. Before, we saw how virtual channels can be used to decouple physical link bandwidth from buffered packets in such a way as to avoid deadlock. Now, virtual channels are multiplexed in such a way that bandwidth is switched in and used by flits of a packet to advance even though the packet may share some links in common with a blocked packet ahead. This, again, allows network bandwidth to be used more efficiently, which, in turn, reduces the average packet latency.

Impact on Network Performance

Routing, arbitration, and switching can impact the packet latency of a loaded network by reducing the contention delay experienced by packets. For an unloaded network that has no contention, the algorithms used to perform routing and arbitration have no impact on latency other than to determine the amount of delay incurred in implementing those functions at switches—typically, the pin-to-pin latency of a switch chip is several tens of nanoseconds. The only change to the best-case packet latency expression given in the previous section comes from the switching technique. Store-and-forward packet switching was assumed before in which transmission delay for the entire packet is incurred on all d hops plus at the source node. For cut-through packet switching, transmission delay is pipelined across the network links comprising the packet's path at the granularity of the packet header instead of the entire packet. Thus, this delay component is reduced, as shown in the following lower-bound expression for packet latency: